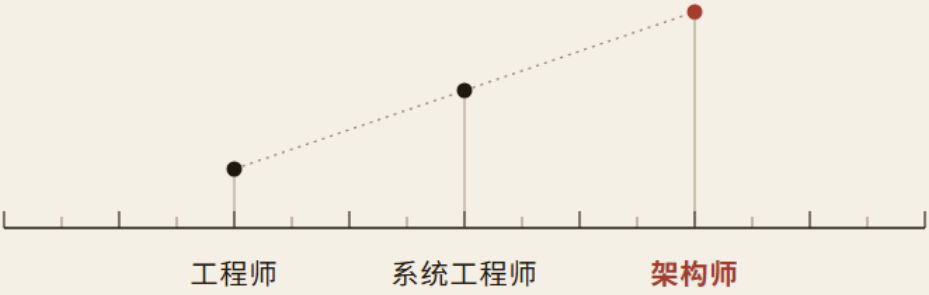
「大部分人在不准确的地基上盖房子」

# 架构师是个怎样的物种?

## —— 从工程师到架构师




学名 Architectus Systematis

特征：兼具架构活动能力和职责的工程师

把话说准

周拥军 著

系统工程出版社

# 架构师是个怎样的物种？

——从工程师到架构师

著：周拥军

2026 年 8 月

系统工程出版社 · EPUB / A5 打印稿

ISBN：—（待定）

# 序言

## 序一 · 概念的归零

去年给集团搭运营平台，我请各部门先把几个最常用的词对齐。说到”编码”——物料有物料编码，财务有科目编码，档案有文件编码，系统还有系统编码。会议室十个人，说出六种意思，谁都没觉得自己说错。

同一周，型号那头在评审技术状态文件。总体单位说”改接口”，配套单位也说”改接口”，吵到一半才明白：一个说的是接口定义，一个说的是接口实物。

一个是散得稀碎的企业运营，一个是管得死死的型号研制，病根是同一个——词没对齐。我在军工体系里待得越久越清楚：流程、标准、评审，没有几处比我们更严；可流程管得住手续，管不住脑子里的定义。评审会照开，会上”接口”“基线”“数据”还是各指各的。纪律能管住过程，管不住概念。

到了企业运营这一头，难处换了样子。最难的不是制度、不是流程、也不是 IT 系统，是让几百上千人用同一套词指同一件事。报表对不上账，多半不是数据的问题，是口径的问题。系统的墙，很多是词的墙。

我们管质量问题叫归零：回到源头，定位准确，举一反三。这本书做的，是给天天挂在嘴边的词做归零——不满足于告诉你”架构是什么”，还给你判据，给你”不是什么”的边界，给你四问：是什么、

不是什么、从哪来、往哪去。我给它的判定只有一句：概念先要说准，方法才有地方站。

回到开头那两场会。评审、基线、数据口径，恰恰是我们这两条线最常出事的三个抓手——评审会照开，是因为没人说得清”评审到底对什么”；基线一改再改，是因为没人分得清”基线”和”版本”；口径对不上，是因为没人把”数据”的定义钉死。它们看着是制度问题，根子都是概念问题。把词归零不是咬文嚼字，是把三个抓手同时校准。

我在军工集团这种既有产品、又有平台的地方待过，最清楚一本书通用不通用。书里一家造温控箱、一家做审批系统，一硬一软，是同一套概念的两种血肉——同一个概念，在硬件上这么长，在软件上那么长，两处都读得通。这份通用，经得起我这边两个世界同时检验。

如果要我给体系内的工程人员定一门必修课，我会先定概念归零这一门。工具可以后买，流程可以后补，词得先对齐——这一条，我在型号里和企业里都试过，成立。这本书写的，就是这门课的教材。

有心人可以按图索骥。管技术状态和基线的，翻第 13 章配置管理；跟数据口径较劲的，翻第 7 章产品数据；琢磨选人定岗的，把第 9 章工程师和第 18 章架构师连起来读——头衔不定义人，能力才定义人：能力和价值，都写在一个人交出来的产品数据里。这句话，在型号里和企业里一样成立。

我见过太多把名词当能力的人。这本书不承诺你读完就成为架构师，它只承诺把那些你天天用、却从没想清的词，放到你有把握判定的位置。我把它放进我那边的必修课，就是这个道理。

宋宁 2026 年 8 月 8 日

## 序二 · 概念，是留给人的那道门槛

我在这行做了三十多年。

三十多年里，我主持过的评审会，几百场。会议室里坐的人换了一茬又一茬，吵的内容却惊人地一致——多数时候不是技术路线，不是预算，是**词**。有人说”架构”是管理层级，有人说”架构”是模块边界，有人说”架构”是服务划分，三个人各说各的，吵了四十分钟，谁都觉得自己在讲同一件事。

这本书第一页，写的就是这样一场会。我读到这里停下来想：这不是虚构，这是我开过的会。我们这行人最熟的东西，它一页就摆出来了。

这些年我复盘过经手的失败项目。真正的技术难题，占不到三成；剩下的七成，死在同一个地方——**大家嘴上说的是同一套词，脑子里装的是不同的东西**。系统上线了，以为工程成功了；UAT 跑过了，以为确认过了；样机通过了，以为产品成功了；画完一张架构图，以为架构有了；名片印上”架构师”，以为能力到了。这些病，我给它们起一个名字：**词汇病**。

词汇病不分项目大小、不分系统新旧，我们做系统的，人人都得过。书里那些人物，我在现实里都见过原型：水忠那样手艺到顶、却把方法藏在心里说不出”为什么”的工程师；万辉那样顶着高级头衔、却答不上”架构师是什么”的人。他们的判词，我看过太多现实版本。能力与价值，最后都得落到产品数据上，才检验得出来。

这本书最不一样的地方，是它给每个概念都配了一把**尺子**。它不满足于告诉你”架构是什么”，它给你一个当场就能检验的判据，还给你它”不是什么”的边界。

第1章把这套方法收拢成四问——是什么、不是什么、从哪来、往哪去。四问答全了，概念才算**拥有**；答不全，只是**见过**。记住是见过的别名，拥有是会用。这个话题，在AI时代尤其紧要：记忆已经被机器外包，人剩下的、机器不替你做的，正是把概念说准、织成网、搬到新场景、划清边界的能力。这本书写的不是更多知识，是这门能力本身。

该怎么读？给你三条路。想入门的，从第1章读起，先握住那把四问的尺子，再往下走——每一章，都是把第1章的方法用在一个词上。想救火的，直接翻到附录——五个抽屉收着全书所有的尺子和判据，查得到，就不算没掌握。想带团队的，把第9章和第18章连起来读，选人、定岗、该不该给”架构师”这个头衔，都在这两张清单里。

它也没有一上来就讲”架构”，而是先教你”怎么学一个概念”，再带你走一条链——从第2章的”工程”出发，穿过需求、设计、评审，中段遇上的是工程师：第2章那几层工程，在他身上落成了一份能力清单（第9章）；再穿过架构、决策、配置、治理、整合，走到终点，是把它背到最后的架构师（第18章）。书名问”架构师是个怎样的物种？“，答案要读完整本书才落地：**架构师首先是一个工程师，是把整条链背在身上的人**。

书末的后记里，两间屋再聚一次，各守定义的一角，把这句话对到一处。

它不保证你读完就成为架构师——没有哪本书做得到。我只替它说一句同行的话：把那些你天天挂在嘴边、却从没真正想清楚的词，放到你有把握判定的位置，这门功夫，就是这行的门槛。合上书，回你的评审会，试着对外行讲清楚”架构”和”架构描述”、“验证”和”确认”的区别——讲得清、不用翻书，你就是真的拥有它了。

黄涛 2026 年 8 月 18 日

## 序三 · 另一个世界，同一把尺子

我第一次翻这本书，带着一条界线。书名问”架构师是个怎样的物种”——架构师？那不是我管的那一行。我这辈子在体系工程里，管装备、管系统之系统、管院校里的学员，跟”架构师”隔着不止一个行业。翻之前我打定主意：这是一本写给别人的书。

可读到第一场联试，我就放不下了。我参加过不止一次跨单位联合试验，最难忘的那一回：三个分系统单测全过，一联起来就瘫。总体的说”数据没问题”，配套的也说”数据没问题”，查到根上才明白：一个说数据格式，一个说数据内容，一个说传输协议——同一个”数据”，三家三个意思。单装都合格，体系接不上。这句话，我在体系工程的现场听了二十多年。

我以为这是装备行才有的病，书却告诉我不是。它讲的是两家民用公司——造温控箱的冰链，做审批系统的恒信——病名一模一样。一个造有形的设备，一个做无形的系统，中间还隔着我的体系工程，三个世界，用错词的样子却是同一张脸。概念不挑行业，我以前是半信半疑，现在信了。

而且这个病在我们这儿，代价是体系级放大的。单个系统里”数据”指错，影响的是一块屏、一条链路；放到体系里，各系统按各自的”数据”对接，错就沿着接口一路传下去，到联试那天才炸。不是谁家装备坏了，是概念差的那一点，被体系乘大了。

我慢慢明白这道界为什么跨得过来：因为接缝在哪儿都一样。体系的接缝，是词缝。单装的问题，还有单装的图纸去管；体系的问题，没有任何一张图纸管得住——因为接缝不在图里，在相关方脑子里的

同一个词里。书里第 17 章把”每一片都合格，接缝断了”讲得很透，我在联试现场见过它的现实版。

我在产品特征工程里也有一个老判断：特征不被验证，等于没有定义。装备的指标，本质是一批要验证的特征；特征工程干的事，是给每个特征一把尺子，再用评审、验证、确认三道关看住它。这本书对每个概念做的，和特征工程对每个特征做的，是同一种事——给词配尺子。

连教育这道界，它都跨过来了。在院校这些年，我最怕的不是学员学不会，是学员考得好、不会用。有一回学员答辩，把”架构”的定义背得一字不差，换一个没见过的系统再问，就答不上来了。这不是学员笨，是缺一把当场就能用的尺子。书里给每个概念都配了这样的尺子——这一条，我在课堂上尤其认。

该怎么读，我不给路数，只给一条跨界的话。读完，回你的世界，挑一个你天天用的词，用书里的尺子量一遍；再去隔壁世界，找一个同名的词，看看它是不是同一个意思。同一个词，两个世界量出同一把尺子，这本书就真的接住你了——不管你从哪个世界来。

宋海伟 2026 年 8 月 28 日

# 前言：先立枢纽

这本书写了很久。起因不是”缺一本架构书”，是一个越来越强的疑问：

**工具、流程、方法年年进步，项目的失败为什么没有变少？**

我给的答案是：多数失败不是技术失败，是概念失败。“需求”和”规定需求”，“验证”和”确认”，“架构”和”架构描述”，“配置”和”参数文件”——这些词天天挂在嘴边，能用、不会定义、各指各的。地基不准确，上面盖多少层都白搭。

这就是这本书的起点，也是它的终点：**把地基打准**。所以它不是教科书——教科书给你完整的体系，让你按图索骥；也不是参考手册——手册给你现成的答案，让你随时翻。它是一本有观点的书，观点只有一句：**概念先要说准，方法才有地方站**。

为了说清这个概念，我用了两个虚构的团队：造温控箱的冰链，做审批系统的恒信。它们不指涉任何真实公司，只是这个行业最常见的两种形态——一个造有形的设备，一个造无形的系统。

同一个概念，让它在两个世界里各长各的样子。**概念不挑行业**，这是这本书想证明的第一件事。

写作时，我把每一章都排成同一副骨架：先看这个词怎么被用错，再把它磨到边界清楚，最后看它往哪去。**磨边界比定义更见功力**——这是我反复对自己说的话，也写成了给读者的判据。

读的时候，我建议把这副骨架当三档注意力用：**开场的现场是引子，一遍读时可以快些过去**；**正文的论证要慢读**——那里每一个判断都挂着出处，是这本书的地基；**章末的判据是体检表**，值得停下来，对

着自己的项目打一遍勾。旁注是术语与出处的落脚处，读到拿不准的词，回看旁注比往前猜省事。若只有时间读一样，就读判据：**它是这本书真正想交到你手里的东西**。

判断一个概念讲没讲透，我只看三样：有没有可检验的判据、有没有失效的边界、有没有反例。三样缺一，就没讲透。这是这本书对自己的要求，也是读者可以拿来回敬它的尺子——**我写的每一把尺子，都欢迎你来检验**。

正文用两个角色收束：第 9 章的工程师，第 18 章的架构师。选它们当书挡，因为头衔最容易被误当成能力，“架构师”尤其如此。书里反复说一句话：**头衔不定义人，能力才定义人**。两个角色章，就是把这句话落到人身上——一个物种，先是工程师，再往系统级长；把整条链看在眼里、落成决定的，是架构师。

而能力与价值，最终都体现在他们交出的产品数据上。

正文之外还有一篇**后记**：两年后，两间屋再聚一次，把四个角色（系统架构师、系统工程师、模块架构师、模块工程师）和四份产品数据清单，交到你手里。

也说说这本书不做什么。它不承诺读完就能造出好系统——把词说准只是第一步，后面还有判断、取舍、时机，那些要在真实项目里练。它也不把概念当真理——概念是共识，共识可以修正。书里那把”四问”的尺子，本身就是来量概念的，包括量这本书自己的概念。

**如果哪一天你在实践里发现哪把尺子不好用，那不是尺子的失败，是它该被校准的时候**。这本书欢迎被反驳，就像它主张的：概念是活的。

周拥军 2026 年 8 月 19

## 致谢

这本书要感谢两个团队——造温控箱的冰链，做审批系统的恒信。书里的故事是虚构的，误解现场不是：它们没有一处不是从真实的工位上长出来的。

书里每一个人物——徐漾、玉杰、逸人、万辉、满海、小白、水忠、文正、永兵、董勐、李旭、黄程、张翔、泰格——都是我想说谢谢的人：我见过他们的词汇病，也用过他们的尺子。十四个名字，十四种把词说歪的姿势，十四种把尺子握稳的手势；他们教会我的第一课，是先承认自己也会说错。**这本书能还的，只有一句：把话说准。**

更要谢谢那些没能在书里留下名字的人。冰链和恒信的十四个名字是剪影，剪影背后是更多让我站在工位边上的人——他们让我看开会、看对账、看出错、看返工，允许我把这些写进书里。错话是真的，人换了名字，教训留了下来。

还要谢谢认真反驳这本书的人。初稿有人说不像话，有人说太像话；感谢那些真把尺子拿起来量的人——书里说尺子欢迎检验，检验它的人，就是这本书的合著者。它长成今天这个模样，是被量出来的，不是被捧出来的。

也谢谢读到这里的你。书里说概念是活的、尺子欢迎检验——这句话，要有人来量才立得住。接下来的正文，每一把尺子都是为你准备的：带回工位量一量，量得动的就留下，量不动的写信告诉我。我在这本书里能做的，到此为止；剩下的，要你去量。**尺子不是用来供奉的，是用来较量的。**

# 目录

序言 . . . . . 3

前言：先立枢纽 . . . . . 10

第 1 章 概念体系能力：学习能力的杠杆 . . . . . 39

1.1 引言 . . . . . 39

1.1.1 误解现场 . . . . . 39

1.1.2 这一章要回答什么 . . . . . 40

1.1.3 这一章的骨架 . . . . . 41

1.2 先说准哪个词：真正要紧的词很少 . . . . . 42

1.2.1 概念是什么 . . . . . 42

1.2.2 概念有主次：一般的、核心的、根上的 . . . . . 44

1.2.3 先抓哪个：核心概念与根概念 . . . . . 47

1.3 概念体系能力：哪几样本事 . . . . . 49

1.3.1 挑枢纽、连关系、搬得走、划得清 . . . . . 50

1.3.2 根概念能力：先抓源头，再展结构 . . . . . 52

1.3.3 记忆可以外包，把话说准不能 . . . . . 52

1.4 凭什么说回报最高：方向与表征 . . . . . 53

1.4.1 这条式子是乘号，不是加号 . . . . . 53

1.4.2 力臂、施力、支点：只负责放大 . . . . . 55

1.4.3 方向：挑错了词，全盘皆输 . . . . . 55

1.4.4 一个词能说到多深：能力四级 . . . . . 56
1.4.5 表征：第一级拿不到，上面全是零 . . . . . 61
1.5 有什么证据：一项压缩学习实验 . . . . . 64
1.5.1 差距在压缩，不在记性 . . . . . 64
1.5.2 实验：先学枢纽的人更快 . . . . . 65
1.5.3 把压缩用在自己身上 . . . . . 65
1.5.4 它证明了什么，没证明什么 . . . . . 66
1.6 怎么练：一条循环转起来 . . . . . 66
1.6.1 什么叫学会：会用才算掌握 . . . . . 67
1.6.2 循环：从立枢纽到用起来 . . . . . 67
1.6.3 每一步在练哪一样 . . . . . 69
1.6.4 校准：概念是共识，不是真理 . . . . . 70
1.7 怎么找：挑出枢纽与根概念 . . . . . 71
1.7.1 概念是点，原理是线 . . . . . 71
1.7.2 怎么挑出枢纽 . . . . . 71
1.7.3 判定矩阵与演绎链 . . . . . 73
1.7.4 根概念长什么样：阻抗 . . . . . 74
1.7.5 根概念长什么样：产品数据 . . . . . 75
1.8 怎么验：答不全，就只是见过 . . . . . 77
1.8.1 假懂长什么样 . . . . . 77

1.8.2 怎么问才算问全 . . . . . 78
1.8.3 词汇病：最常见的几种”没拿准” . . . . . 81
1.8.4 查全：概念档案 . . . . . 82
1.8.5 认知反转 . . . . . 83
第 2 章 工程：我们到底在做什么 . . . . . 93
2.1 引言 . . . . . 93
2.1.1 误解现场 . . . . . 93
2.1.2 这一章要回答什么 . . . . . 94
2.1.3 这一章的骨架 . . . . . 95
2.2 工程是什么 . . . . . 96
2.2.1 先给工程一个定义 . . . . . 96
2.2.2 定义里的每一样都不能少 . . . . . 97
2.2.3 权威定义：不同年代，同一个内核 . . . . . 99
2.2.4 交付能不能离开某个人 . . . . . 103
2.3 工程不是什么 . . . . . 105
2.3.1 工程没有单一对立面 . . . . . 105
2.3.2 不是手艺：靠人，还是靠方法 . . . . . 106
2.3.3 不是方法论：工程比它大 . . . . . 108
2.3.4 不是机械化：系统化不等于算法化 . . . . . 112
2.4 工程怎么用 . . . . . 118

2.4.1 系统工程：不造部件，只造整体 . . . . . 118
2.4.2 涌现属性：部件全对，系统未必对 . . . . . 119
2.4.3 工程系统包含技术系统 . . . . . 121
2.4.4 验收参照：上线不等于成功 . . . . . 125
2.4.5 回头给恒信量一遍 . . . . . 127
2.4.6 认知反转 . . . . . 129
第 3 章 需求、设计开发、质量：产品开发的图式 . . . . . 138
3.1 引言 . . . . . 138
3.1.1 误解现场 . . . . . 138
3.1.2 这一章要回答什么 . . . . . 139
3.1.3 这一章的骨架 . . . . . 140
3.2 需求、设计开发、质量是什么 . . . . . 141
3.2.1 需求：没说出口的也算 . . . . . 142
3.2.2 需要、期望与需求：内容与呈现方式 . . . . . 144
3.2.3 目的、目标与需求：价值、结果与规格 . . . . . 145
3.2.4 设计开发：需求怎么变得更详细 . . . . . 147
3.2.5 设计开发：每一级都在做同一件事 . . . . . 151
3.2.6 质量：固有特性满足需求的程度 . . . . . 153
3.2.7 概念卡片 . . . . . 155
3.3 需求、设计开发、质量不是什么 . . . . . 161

3.3.1 设计开发不始于点子 . . . . . 162
3.3.2 起点的判据：需求要可转化 . . . . . 162
3.3.3 需求开发的边界：特性有没有被规定 . . . . . 164
3.3.4 终点的判据：为什么不是可验证 . . . . . 165
3.3.5 终点的样子：方案配齐，指令可执行 . . . . . 167
3.3.6 参照的判据：等级不是质量 . . . . . 171
3.4 需求、设计开发、质量怎么用 . . . . . 173
3.4.1 程度怎么认定：从特性值到结论 . . . . . 174
3.4.2 分值为什么靠不住 . . . . . 175
3.4.3 质量与进度：改需求，不改质量 . . . . . 176
3.4.4 互相定义，所以拆不掉 . . . . . 181
3.4.5 认知反转 . . . . . 184
第 4 章 需求工程：从诉求到依据 . . . . . 193
4.1 引言 . . . . . 193
4.1.1 误解现场 . . . . . 193
4.1.2 这一章要回答什么 . . . . . 194
4.1.3 这一章的骨架 . . . . . 195
4.2 需求工程是什么 . . . . . 196
4.2.1 需求工程：开发与管理并列 . . . . . 196
4.2.2 为什么是并列，不是包含 . . . . . 198

4.2.3 需求开发：把诉求变成规定需求 . . . . . 200
4.2.4 需求的来路 . . . . . 203
4.2.5 规格与规定需求：一份 SRS 里有什么 . . . . . 205
4.2.6 需求管理：让它一直是那一份 . . . . . 209
4.2.7 开发与管理在哪里交接 . . . . . 213
4.3 需求工程不是什么 . . . . . 217
4.3.1 明示与可采：说出来了，就能当依据吗 . . . . . 217
4.3.2 开发与管理：需求乱，是谁的账 . . . . . 218
4.3.3 评审与验证确认：对文档能做什么 . . . . . 219
4.3.4 留档与受控：改名算不算版本管理 . . . . . 221
4.4 需求工程怎么做 . . . . . 222
4.4.1 ConOps：系统将被怎么用 . . . . . 223
4.4.2 ConOps 的位置：分析之后，规格之前 . . . . . 223
4.4.3 一份 ConOps 里有什么 . . . . . 224
4.4.4 一条好需求的判据 . . . . . 227
4.4.5 需求文档体系 . . . . . 228
4.4.6 认知反转 . . . . . 233
第 5 章 需求的完整性：谁在问，谁在验 . . . . . 242
5.1 引言 . . . . . 242
5.1.1 误解现场 . . . . . 242

5.1.2 这一章要回答什么 . . . . . 243
5.1.3 这一章的骨架 . . . . . 244
5.2 需求的完整性是什么 . . . . . 244
5.2.1 一份全的需求：覆盖什么 . . . . . 245
5.2.2 需求分类：看谁在问 . . . . . 246
5.2.3 功能需求：包括什么，从哪来 . . . . . 247
5.2.4 环境适应性需求：包括什么，从哪来 . . . . . 248
5.2.5 可实现性需求：包括什么，从哪来 . . . . . 250
5.2.6 每一类需求：都要有指标 . . . . . 253
5.3 需求的完整性不是什么 . . . . . 255
5.3.1 验证载体：各管一类需求 . . . . . 255
5.3.2 做全与验过：不是一回事 . . . . . 258
5.3.3 样机与产品：通过不等于成功 . . . . . 258
5.3.4 功能与非功能：这么分行吗 . . . . . 259
5.3.5 隐含需求：算不算单独一类 . . . . . 260
5.4 需求的完整性怎么用 . . . . . 262
5.4.1 谁主导，谁确认 . . . . . 262
5.4.2 主导不等于替客户拍板 . . . . . 265
5.4.3 全不全，是承制方的责任 . . . . . 265
5.4.4 可实现性需求从哪里补 . . . . . 266

5.4.5 从症状反推缺了哪条需求 ..... 267
5.4.6 成本是需求，不是结果 ..... 270
5.4.7 认知反转 ..... 271
第 6 章 设计开发落地：需求与方案结对 ..... 280
6.1 引言 ..... 280
6.1.1 误解现场 ..... 280
6.1.2 这一章要回答什么 ..... 281
6.1.3 这一章的骨架 ..... 282
6.2 设计开发的对象 ..... 284
6.2.1 需求与方案结对 ..... 284
6.2.2 设计文档要能定位、承接、追溯 ..... 285
6.2.3 产品数据的条目化、树形化与分配 ..... 288
6.2.4 框架定义与详细定义 ..... 291
6.2.5 框架、概要与定义 ..... 297
6.2.6 框架定义不是架构定义 ..... 298
6.2.7 树装不下的内容 ..... 301
6.3 设计开发的规则与动作 ..... 304
6.3.1 每一对先定规则 ..... 304
6.3.2 准备工作与设计循环 ..... 310
6.3.3 分配生成：从需求长出骨架 ..... 313

6.3.4 分析迭代：一条需求一个归宿 . . . . . 320
6.3.5 分解分析：切到可独立验证 . . . . . 325
6.3.6 接住的需求一定被实现 . . . . . 332
6.3.7 干枝与叶节点分开维护 . . . . . 337
6.4 一条链上的逐对接力 . . . . . 337
6.4.1 上一份的叶节点，下一份的需求 . . . . . 338
6.4.2 链上只有分配步在做承接 . . . . . 341
6.4.3 链尾交给构件设计方 . . . . . 343
6.4.4 一对从接需求走到冻结基线 . . . . . 344
6.4.5 设计开发的验收与追溯 . . . . . 348
6.4.6 认知反转 . . . . . 350
第 7 章 产品数据：开发产品就是开发产品数据 . . . . . 363
7.1 引言 . . . . . 363
7.1.1 误解现场 . . . . . 363
7.1.2 这一章要回答什么 . . . . . 365
7.1.3 这一章的骨架 . . . . . 366
7.2 产品数据是什么 . . . . . 367
7.2.1 产品：物理存在与信息存在 . . . . . 368
7.2.2 产品：需求、价值与过程 . . . . . 369
7.2.3 产品数据：形式化的信息表示 . . . . . 372

7.2.4 产品数据不等于什么 . . . . . 376
7.2.5 产品数据该有的样子 . . . . . 377
7.2.6 实现方案：最重的产品数据 . . . . . 378
7.3 开发产品造的是什么 . . . . . 383
7.3.1 产品开发：定义与复制 . . . . . 384
7.3.2 两域三态：产品的存在方式 . . . . . 386
7.3.3 开发产品就是开发数据 . . . . . 388
7.3.4 软件为什么特殊 . . . . . 389
7.3.5 产品质量的上限由数据质量决定 . . . . . 389
7.3.6 沿工位流动的是产品数据 . . . . . 395
7.4 产品数据怎么用 . . . . . 398
7.4.1 利益相关方围绕产品数据协作 . . . . . 398
7.4.2 数据能装下多少价值 . . . . . 401
7.4.3 职责为什么挂在工件上 . . . . . 403
7.4.4 活动值不值：产品数据增值判据 . . . . . 410
7.4.5 产品会死，数据体系活着 . . . . . 412
7.4.6 数字化不等于转型 . . . . . 415
7.4.7 认知反转 . . . . . 416
第 8 章 评审、验证与确认：目标、需求、用途 . . . . . 426
8.1 引言 . . . . . 426

8.1.1 误解现场 . . . . . 426
8.1.2 这一章要回答什么 . . . . . 428
8.1.3 这一章的骨架 . . . . . 428
8.2 评审、验证与确认是什么 . . . . . 429
8.2.1 病根：它们为什么被混用 . . . . . 429
8.2.2 各自的参照 . . . . . 431
8.2.3 分开它们的几处差别 . . . . . 436
8.2.4 评审：适宜、充分、有效 . . . . . 437
8.2.5 验证：对照、证据、证实 . . . . . 441
8.2.6 确认：换参照，不换判法 . . . . . 445
8.3 评审、验证与确认不是什么 . . . . . 451
8.3.1 评审不是核对需求 . . . . . 451
8.3.2 测试通过不等于验证通过 . . . . . 454
8.3.3 业务方点了不等于确认过了 . . . . . 456
8.3.4 验证够不着没写下来的需求 . . . . . 458
8.3.5 确认的参照在场景里 . . . . . 460
8.3.6 评审早拦，确认终审 . . . . . 463
8.4 评审、验证与确认怎么用 . . . . . 465
8.4.1 需求走完全程 . . . . . 465
8.4.2 评审贯穿整条流程 . . . . . 468

8.4.3 只做验证，会漏掉什么 ..... 472
8.4.4 评审怎么评 ..... 475
8.4.5 验证怎么验 ..... 480
8.4.6 确认怎么确认 ..... 482
8.4.7 认知反转 ..... 483
第 9 章 工程师：一份能力清单 ..... 494
9.1 引言 ..... 494
9.1.1 误解现场 ..... 494
9.1.2 这一章要回答什么 ..... 494
9.1.3 这一章的骨架 ..... 495
9.2 工程师是什么 ..... 496
9.2.1 工程师能力清单 ..... 496
9.2.2 ①科学素养：凭什么能行 ..... 499
9.2.3 ②方法论素养：把工作组织成方法 ..... 500
9.2.4 ③实践能力：亲手把方案变成现实 ..... 502
9.2.5 ⑤工程判断：在约束里取舍 ..... 503
9.2.6 ⑥可复现意识：让判断被接住 ..... 504
9.2.7 ④职业伦理：给能力定方向 ..... 505
9.2.8 工程师干的活：从需求到判定 ..... 506
9.2.9 清单不挑领域：变的只是权重 ..... 508

9.2.10 沟通与团队：执行手段 . . . . . 509
9.3 工程师不是什么 . . . . . 512
9.3.1 不是技术员 . . . . . 512
9.3.2 不是科学家 . . . . . 513
9.3.3 不是手艺人 . . . . . 513
9.3.4 不是”会写代码的人” . . . . . 514
9.3.5 不是头衔、证书、年限 . . . . . 514
9.3.6 不止一种角色：专责与终责 . . . . . 515
9.4 工程师怎么做成 . . . . . 519
9.4.1 亲手做出来 . . . . . 520
9.4.2 从我会做到别人会做 . . . . . 520
9.4.3 从照做到会取舍 . . . . . 521
9.4.4 知道什么不能牺牲 . . . . . 521
9.4.5 从模块工程师走到系统工程师 . . . . . 522
9.4.6 从失败看缺项 . . . . . 523
9.4.7 拿产品数据检验 . . . . . 524
9.4.8 认知反转 . . . . . 529
第 10 章 架构：定义与误解 . . . . . 540
10.1 引言 . . . . . 540
10.1.1 误解现场 . . . . . 540

10.1.2 这一章要回答什么 . . . . . 542
10.1.3 这一章的骨架 . . . . . 542
10.2 架构是什么 . . . . . 543
10.2.1 架构这个词的来路 . . . . . 543
10.2.2 架构是基本概念或属性与管控原则 . . . . . 546
10.2.3 架构无形，有形的是架构描述 . . . . . 550
10.2.4 基本是对目的必要且充分 . . . . . 553
10.2.5 元素与关系不在定义里 . . . . . 557
10.2.6 假判据冒充真判据 . . . . . 560
10.3 架构描述是什么 . . . . . 562
10.3.1 架构描述是一个工作产品 . . . . . 563
10.3.2 视角是一组惯例 . . . . . 564
10.3.3 视图是一个信息部件 . . . . . 567
10.3.4 关注点站在哪个位置 . . . . . 569
10.4 架构不是什么 . . . . . 572
10.4.1 不是一张图 . . . . . 573
10.4.2 不是一组决定 . . . . . 574
10.4.3 不是设计 . . . . . 575
10.4.4 不是一堆”XX 架构” . . . . . 576
10.4.5 不是框架与风格 . . . . . 577

10.4.6 不是”高层级” . . . . . 578
10.4.7 认知反转 . . . . . 579
第 11 章 架构设计：概念幸存、属性涌现、原则自加 . . . . . 593
11.1 误解现场 . . . . . 593
11.2 第一幕 架构设计是什么（定界） . . . . . 598
11.2.1 一个动作：把需求翻译成”架构” . . . . . 599
11.2.2 标准怎么说：设计是转化，过程以产出物命名 . . . . 600
11.2.3 “定义”与”设计”：划界 + 陈述，还是怎么做 . . . 602
11.2.4 要产出什么：只有两样，元素与关系去哪了 . . . . . 604
11.2.5 横切 vs 纵挖：架构设计 vs 详细设计 . . . . . 607
11.3 第二幕 架构设计不是什么：幸存、涌现、自加（磨边界弧） . . . . . 612
11.3.1 先看病：三样东西为什么都不是架构设计 . . . . . 612
11.3.2 吃进嘴的不是”全部需求原文”：输入六类 . . . . . 614
11.3.3 还得是”规定需求”：吃未分配的系统需求 . . . . . 617
11.3.4 质量属性不是”分”下去的：涌现属性只能译 . . . . 619
11.3.5 关注点不是设计出来的：是识别出来的 . . . . . 619
11.3.6 基本概念不是设计出来的：概念是幸存下来的 . . . . 624
11.3.7 基本属性不是设计出来的：属性是涌现出来的 . . . . 632
11.3.8 管控原则不是抄来的：原则是自加的 . . . . . 638

11.3.9 管控原则还要管住演化：怎么变而不破坏对 . . . . . 641
11.3.10 元素关系不是越多越好：关系是成本 . . . . . 644
11.4 第三幕 架构设计往哪去（运用） . . . . . 648
11.4.1 输入是一条链：基本概念直面世界 . . . . . 648
11.4.2 三个推论：输入链带来的体检指标 . . . . . 651
11.4.3 架构生成六步：动词链的工程化 . . . . . 653
11.4.4 系统基本概念和基本属性清单：目的判据的裁决产物 . . . . . 660
11.4.5 创新：链尾的评分席 . . . . . 661
第 12 章 架构决策：对不可逆的提前投资 . . . . . 683
12.1 误解现场 . . . . . 683
12.2 第一幕 架构决策是什么（定界） . . . . . 687
12.2.1 先看病：“改个字段”为什么炸了三个月 . . . . . 687
12.2.2 权威定义：选错了，要花大代价才能改对 . . . . . 688
12.2.3 三个让人”哦”的例子：书架、厨房、订单服务 . . 692
12.3 第二幕 架构决策不是什么（磨边界弧） . . . . . 698
12.3.1 错觉一：不可逆性是经济属性，不是技术属性 . . . . 698
12.3.2 错觉二：架构决策不可避免——不是可做可不做 . . 701
12.3.3 错觉三：被动做不是没得选 . . . . . 703
12.4 第三幕 架构决策往哪去（运用） . . . . . 707

12.4.1 怎么判断不可逆性：三层判断框架 ..... 707
12.4.2 第一层：修改的影响半径 ..... 708
12.4.3 第二层：修改的信息成本 ..... 709
12.4.4 第三层：修改的风险 ..... 710
12.4.5 综合判断：一张矩阵，一眼定级 ..... 711
12.4.6 完整演练：把三层框架套到”编号规则”上 ..... 715
12.4.7 抉择：什么时候做——定节奏与显式推迟 ..... 716
12.4.8 记录：决策记录六问，把决策变成资产 ..... 718
12.4.9 分级：把审查资源花在刀刃上 ..... 724
12.4.10 定位：设计链各层的架构决策 ..... 726
12.4.11 需求有架构吗？ ..... 730
12.4.12 链的两端，承受最高风险 ..... 733
第 13 章 配置管理：让一直在变的东西说得清 ..... 749
13.1 误解现场 ..... 749
13.2 第一幕 配置、版本、基线是什么（定界） ..... 753
13.2.1 配置：实体被记录下来的形态 ..... 754
13.2.2 三个隐形过滤器：不是所有特性都是配置 ..... 758
13.2.3 版本：时间轴上的演进刻度 ..... 762
13.2.4 基线：被批准冻结的参照 ..... 765
13.2.5 三种正式基线：功能、分配、产品 ..... 766

13.3 第二幕 配置、版本、基线不是什么（磨边界弧） ..... 770
13.3.1 配置不是参数文件：先看病 ..... 770
13.3.2 版本不是基线：一横一竖 ..... 771
13.3.3 变体不是版本：空间与时间正交 ..... 773
13.3.4 没有基线，变化就不可言说 ..... 774
13.4 第三幕 配置管理往哪去：守护与价值（运用） ..... 777
13.4.1 变更控制：守住基线的门 ..... 777
13.4.2 配置管理四大活动：标识、控制、记录、审计 ..... 778
13.4.3 配置管理的核心目的：三问，永远答得出 ..... 780
13.4.4 配置管理：架构决策的守护者 ..... 783
13.4.5 契约链：每一对相邻环节，都是局部甲乙方 ..... 787
13.4.6 两个易混场景：把尺子用起来 ..... 789
13.4.7 为什么必须做：三条铁律 ..... 790
13.4.8 带来什么：四个维度 ..... 791
13.4.9 经济价值：省钱、赚钱、保钱 ..... 792
第 14 章 治理：谁掌舵，谁划桨 ..... 807
14.1 误解现场 ..... 807
14.2 第一幕 治理是什么（定界） ..... 811
14.2.1 先看病：我们买的，到底是舵还是桨 ..... 812
14.2.2 舵手不是船主：govern 的本义是掌舵 ..... 812

14.2.3 从”治理危机”到”国家治理现代化”：一场有意的定译 . . . . . 813
14.2.4 “治”与”理”：治水疏导，治玉顺纹 . . . . . 814
14.2.5 为什么能跨领域：治理四问 . . . . . 816
14.2.6 治理落到架构域：架构治理 . . . . . 821
14.3 第二幕 治理不是什么（磨边界弧） . . . . . 827
14.3.1 先看病：我们做的”管理”，为什么不是”治理” . 827
14.3.2 掌舵与划桨：决策系统与执行系统 . . . . . 827
14.3.3 委托必有治理：责任可委托，担责不可委托 . . . . . 831
14.3.4 同一个要素，两个层次 . . . . . 834
14.3.5 不是越早越好：治理三条件 . . . . . 839
14.3.6 不是例行：治理是持续系统 . . . . . 842
14.3.7 两种失效：坏了要修，不适合了要演 . . . . . 843
14.4 第三幕 治理往哪去（运用） . . . . . 847
14.4.1 三条判据，一个门槛：别把”跨部门”当判据 . . . . 848
14.4.2 跨边界才是治理：权力 100%，精力 1% . . . . . 849
14.4.3 委员会是过滤器，不是处理器 . . . . . 851
14.4.4 三活动循环：感知、定规、裁例外 . . . . . 852
14.4.5 术语膨胀：用治理的名字，干管理的活 . . . . . 855
14.4.6 最终公式：XX 管理 = XX 治理 + XX 开发与运营 . . 858

14.4.7 一眼认出”假治理”：五问判据 . . . . . 860
第 15 章 架构定义落地：从目的到架构视图 . . . . . 883
15.1 误解现场 . . . . . 884
15.2 第一幕 架构定义落地是什么：一条九步的链（定界） . . 887
15.2.1 一条链：从目的到架构视图 . . . . . 887
15.2.2 第一步：目的是准绳，不是口号 . . . . . 889
15.2.3 第二步：把承重件一次写完 . . . . . 890
15.2.4 第三步：目标与关切——原料与产物 . . . . . 891
15.2.5 第四步到第六步：视角是规则，视图是产物 . . . . . 893
15.2.6 第七步到第九步：两把尺子，一张对账表 . . . . . 895
15.3 第二幕 架构定义落地不是什么：四处最像的地方（磨边界弧） . . . . . 896
15.3.1 不是”画图交付”：图是视图组件，不是架构 . . . . . 896
15.3.2 不是”关切清单”：关切不是判据，是判据的现场 . 898
15.3.3 不是”视角照抄”：三域的差别在行，共性在列 . . . 899
15.3.4 不是”文档审查”：评估的对象是架构，不是描述 . 900
15.3.5 不是”评估现场补图”：缺口回定义期，两个闸门都不许动 . . . . . 900
15.3.6 不是”标准说了算”：标准闭不上的缝，只能由责任来闭 . . . . . 901

15.4 第三幕 架构定义落地往哪去：把这条链走完（运用）. . 902
15.4.1 视角规格六项：写一次，用很多次 . . . . . 902
15.4.2 视角从哪来：四份蓝本，加一个自建的纪律 . . . . . 904
15.4.3 视图的三条硬要求 . . . . . 905
15.4.4 对账：正向查漏，反向查滥 . . . . . 906
15.4.5 关切分四关裁：举证前置，退件不是驳回 . . . . . 907
15.4.6 两个闸门与十二步最小清单 . . . . . 908
15.4.7 缺口长什么样：三种缺口，三种补法 . . . . . 910
第 16 章 架构定义到设计定义：架构与设计的交接 . . . . . 928
16.1 误解现场 . . . . . 929
16.2 第一幕 架构定义到设计定义是什么：两类工作，两条转化（定界） . . . . . 933
16.2.1 两类工作，不是粗细两版 . . . . . 933
16.2.2 交接的两端：一边交出去什么，一边接住什么 . . . . 936
16.2.3 交接的动作：入口、两条转化、一道出口 . . . . . 938
16.2.4 怎么算做完、做对：五条判据 . . . . . 941
16.2.5 一条必须处理的缝：设计会不会”长出新东西” . . 943
16.2.6 一条回路：不是接力，是递归里的一个换手点 . . . . 944
16.3 第二幕 架构定义到设计定义不是什么：四处最像的地方（磨边界弧） . . . . . 948

16.3.1 不是”细化”：细化只是形状，不是判据 . . . . . 948
16.3.2 不是”下游阶段”：并发、迭代、递归 . . . . . 948
16.3.3 不是”接口自由”：只细化不新增 . . . . . 949
16.3.4 不是”需求清单的副本”：转化与分配是两件事 . . . 949
16.4 第三幕 架构定义到设计定义往哪去：把交接走完（运用） . . . . . 950
16.4.1 过程四组活动：准备、开展、评估、管理 . . . . . 950
16.4.2 五类可指名的方法 . . . . . 951
16.4.3 描述符：一条最容易被当成细节的判据 . . . . . 953
16.4.4 判据怎么用：九查与三档结论 . . . . . 953
16.4.5 可实现性关切：八项不必配八个视角 . . . . . 956
16.4.6 一条纪律：先穷尽复用，再自建 . . . . . 958
第 17 章 整合：从一条需求到上线运维 . . . . . 979
17.1 误解现场 . . . . . 979
17.2 第一幕 整合是什么：十一把刀，切同一头大象（定界） . . . . . 983
17.2.1 先看病：每一片都合格，接缝断了 . . . . . 984
17.2.2 反直觉命题：所有概念是同一系统的不同切片 . . . . 985
17.2.3 十一把刀：切片有两种——顺次，与横切 . . . . . 987
17.2.4 一张时间线：双线合流 . . . . . 987

17.3 第二幕 整合不是什么：接缝与一象一刀（磨边界弧） . . 989
17.3.1 磨边一：接缝断了，不是部件坏了 . . . . . 990
17.3.2 磨边二：不属于任何一片，等于没人负责 . . . . . 990
17.3.3 磨边三：技术上线，不等于工程成功 . . . . . 991
17.3.4 四条接缝：切片靠什么缝合 . . . . . 991
17.3.5 跨域织线：概念是刀，系统是大象 . . . . . 993
17.4 第三幕 整合往哪去：一张时间线，从切片回到系统（运用） . . . . . 994
17.4.1 一张时间线，两种系统 . . . . . 994
17.4.2 站点① 需求：标尺从哪来 . . . . . 995
17.4.3 站点② 设计：标尺怎么被造 . . . . . 998
17.4.4 站点③ 把关：三个看门人 . . . . . 1000
17.4.5 站点④ 架构（含架构设计与架构决策）：不可逆的提前投资 . . . . . 1002
17.4.6 站点⑤ 配置：让变化说得清 . . . . . 1006
17.4.7 站点⑥ 治理：谁掌舵谁划桨 . . . . . 1008
17.4.8 站点⑦ 运维：从出生活到退役 . . . . . 1010
第 18 章 架构师：把整条链背在身上的人 . . . . . 1032
18.1 误解现场 . . . . . 1032
18.2 第一幕 架构师是什么：十项能力清单（定界） . . . . . 1036

18.2.1 一句开场白：把整个物种钉在底座上 ..... 1037
18.2.2 底座六项：三栏点名，每项给判据 ..... 1038
18.2.3 增量四项：十项能力清单，长在底座上 ..... 1039
18.2.4 ⑦ 识别后果严重的决策 ..... 1042
18.2.5 ⑧ 对系统的理论 ..... 1042
18.2.6 ⑨ 把架构说清楚 ..... 1043
18.2.7 ⑩ 判断不可逆性 ..... 1044
18.2.8 六个更锋利的差异 ..... 1044
18.2.9 整条链：architecting 七动词链 ..... 1047
18.3 第二幕 架构师不是什么：角色边界与分工（磨边界弧） ..... 1051
18.3.1 不是”更高级的工程师” ..... 1051
18.3.2 不是系统工程师：专责与终责的分野 ..... 1052
18.3.3 不是技术负责人 ..... 1054
18.3.4 不是项目经理 ..... 1055
18.3.5 不是领域专家 ..... 1055
18.3.6 不是画图匠 ..... 1055
18.3.7 失败四式：怎么死掉 ..... 1056
18.4 第三幕 架构师往哪去：成长、产品数据、判词落定（运用） ..... 1061

18.4.1 怎么长成：抽象跃迁，从系统到架构 . . . . . 1061
18.4.2 被任命 ≠ 成为 . . . . . 1063
18.4.3 三种来路，三种功课 . . . . . 1063
18.4.4 成长的标志：第一次”说不” . . . . . 1064
18.4.5 再往上：从系统到组织 . . . . . 1064
18.4.6 能力的最终体现：开发出高质量的产品数据 . . . . . 1069
18.4.7 接口契约 = 1:1 搬入点 . . . . . 1077
18.4.8 检验：换个人，能不能接住 . . . . . 1078
附录：把尺子收进五个抽屉 . . . . . 1096
附录 A 术语表：全书概念的英中对照 . . . . . 1096
附录 B 概念档案集：33 张完整九格 . . . . . 1098
附录 C 误解清单：全部反直觉命题速查 . . . . . 1129
附录 D 参考文献总表（单一权威） . . . . . 1131
D.1 标准 . . . . . 1132
D.2 经典文献（非标准） . . . . . 1139
D.3 内源（研习笔记与参考课） . . . . . 1142
附录 E 练习答案 . . . . . 1144
第 1 章 概念体系能力：学习能力的杠杆 . . . . . 1144
第 2 章 工程：我们到底在做什么 . . . . . 1146
第 3 章 需求、设计开发、质量：产品开发的图式 . . . . . 1147

第 4 章 需求工程：从诉求到依据 ..... 1148
第 5 章 需求的完整性：谁在问，谁在验 ..... 1150
第 6 章 设计开发落地：需求与方案结对 ..... 1151
第 7 章 产品数据：开发产品就是开发产品数据 ..... 1153
第 8 章 评审、验证与确认：目标、需求、用途 ..... 1155
第 9 章 工程师：一份能力清单 ..... 1156
第 10 章 架构：定义与误解 ..... 1157
第 12 章 架构决策：对不可逆的提前投资 ..... 1161
第 13 章 配置管理：让一直在变的东西说得清 ..... 1162
第 14 章 治理：谁掌舵，谁划桨 ..... 1163
第 15 章 架构定义落地：从目的到架构视图 ..... 1165
第 16 章 架构定义到设计定义：架构与设计的交接 ..... 1167
第 17 章 整合：从一条需求到上线运维 ..... 1168
第 18 章 架构师：把整条链背在身上的人 ..... 1169
附录收束 ..... 1171

# 第 1 章 概念体系能力：学习能力的杠杆

---

## 1.1 引言

这一章从一个现场讲起：一个词没对齐，两个项目停了工。接着回答一个问题：凭什么说”把话说准”是学习能力里回报最高的那一步？最后给出一副骨架，让你看清后面的路怎么走。一路读下来，落到手上的，是一套能用的本事。

### 1.1.1 误解现场

一个词，让两个项目停了工。

冰链的 M3 二代，模块边界划了两个月没划下来；恒信的合同系统，改造方案评了三个月没评完。一硬一软，卡住的原因一模一样：“架构”这两个字一在会上说出口，会就开不下去了。

冰链科技是造 M3 医疗温控箱的工程团队。这天他们开会，议题是 M3 二代怎么分块：

产品开发经理永兵把会拉起来：“M3 二代的边界划了两个月。今天先把’架构’这词说清楚，再谈分块。”

文正先把话钉死：“架构就是模块怎么分——产品架构。分块方案定下来，后面的活才好派。”

水忠摇头：“模块边界是设计的事。我说的是硬件架构——传感器、主控、通信怎么布，那是元件怎么划。”

董勐接话：“元件只是元素。系统架构在云端——服务边界、数据所有权，那是跨元素的约定。”

李旭皱眉：“那我按哪一层拆测试用例？验收判据挂在哪一层的定义上，我得先知道。”

逸人敲桌：“架构定不下来，物料清单和批次追溯也无从谈起。M3 二代每拖一个月，一代的订单就在外面压一批——架构对你们是张图，对我是一整条产线停不停。”

散会时，M3 二代的边界还是没划下来。

恒信集团是做合同审批系统的设计团队。这天他们在评改造方案：

产品开发经理满海把会议室占了：“改造方案评了三个月。今天先把’架构’这两个字对齐——对不齐，方案没法评。”

黄程先抱怨：“我们公司的架构不行，审批链跨三个部门扯皮。”

万辉纠正：“你说的那是组织架构。系统架构是服务边界和数据所有权——这些定不下来，图都画不出来。”

小白追问：“那我这边的接口清单，按哪张图对齐？前端等着开工。”

玉杰补一句：“还有部署架构呢？部署在哪、谁运维、上线怎么回滚——这一摊子总得有人说了算吧？”

徐漾放下笔：“停——‘架构’在我们嘴里已经是五件事了。先各自说清你指的是哪一件。”

这一问，没人接得住。同一个词，两个世界，都是五张脸——问题不在人，在词。

可把词当场对齐，就真懂”架构”了吗？就算十张脸都同意它指系统架构，让他们各答四问：它是什么、不是什么、从哪来、往哪去，多半还是答不齐。**词可以当场对齐；概念要花功夫，才能真正拥有。**

## 1.1.2 这一章要回答什么

这一章从头到尾只为回答一个问题，另有六个小问题替它铺路：

1. **为什么”把话说准”**，是学习能力里回报最高的那一步？——“把话说准”，一半是**说准哪个词**，先抓枢纽与根；另一半是把它说到底。
2. 一句话说不准，凭什么能让两个项目一停就是两三个月的工？（§1.1、§1.8）
3. 说准哪个词：概念为什么是一套三层包含的体系？枢纽与根各是什么？（§1.2）
4. 谁来说准：这副能力由哪几样组成，又怎么检验它强不强？（§1.3）
5. 凭什么说回报最高：这条乘法律的两个归零项是什么，有没有证据？（§1.4.1、§1.5）
6. 说到多深才算说到：能力分哪四级，“表征是 1，上面三级是 0”又是怎么来的？（§1.4.4、§1.4.5）
7. 怎么练、怎么找、怎么验：五环是什么？三步识别法怎么找枢纽？四问法怎么验”真拥有”？（§1.6~§1.8）

### 1.1.3 这一章的骨架

这一章要立的，是一个主张和一把量它的尺子。

> **把话说准，是学习能力里回报最高的那一步。**

这句话里有两件事：先**说准哪个词**（先抓枢纽与根），再把这个词说到底（从一个词说到一整片）。

凭什么说回报最高？因为这条乘法律的两个归零项，都落在”把话说准”上：**方向**（先抓错了词）与**表征**（话说得不准）。这两项里只

要有一项是零，整条式子就是零——冰链和恒信的两场会，失守的正是这两项。

说到什么程度？同一个词，你能说到好几级：先把词说准（表征）→ 看得出套路（图式）→ 能推演（心智模型）→ 能判断什么值得相信（解释框架）。量”说到多深”的那把刻度尺，本章 §1.4.5 交给你。

后面七节就是答这一问的七步：先说准哪个词（§1.2）→ 这副能力是什么（§1.3）→ 它凭什么决定成败（§1.4）→ 有什么证据（§1.5）→ 怎么练（§1.6）→ 怎么找（§1.7）→ 怎么验（§1.8）。

## 1.2 先说准哪个词：真正要紧的词很少

> 上一问：两个项目为什么停工？因为”架构”这个词没拿准。那该说准哪个词、先抓哪几个？——一个领域里真正要紧的词很少，它们连成一张有主次的网。

### 1.2.1 概念是什么

“工程”“需求”“质量”“架构”“评审”“治理”——这些词有什么共同点？它们都是各自领域的概念；而概念是什么，术语学标准里有一条严整的定义（ISO 1087:2019 §3.2.7，**经研习报告转引，正本不在库**）：**知识单元**。冰链和恒信那两场会里，“架构”就是这么个词——人人拿它当概念使，却没有一个人说得准它。

> **概念（concept）：由特性的独特组合而形成的知识单元**（ISO 1087:2019 §3.2.7）。判据：可指称（一个词或短语）＋可界定

> （内涵与外延）。概念与术语的区别（能指与所指），见 §1.8.3 病二。

一个概念拆开来看，有三个角。能指是词或符号，也就是概念的名字；所指是概念本身，你脑子里那个意思；指称（指称对象）是这个词指向的那个东西。三个角里，这本书说的”概念”是中间那个：它既不在嘴上，也不在物上。冰链和恒信那两场会吵的，正是中间这个角——嘴上说的是同一个词，脑子里装的是各家的意思；能指与所指、涵义与指称这两套分法的出处，见 §1.8.3 病二。

> 术语注：别把两个”指称”混成一个。判据里的”可指称”与 ISO 1087 的 designation 一样，是”给概念命名”义；上文的”指称（指称对象）“是”概念指向的客观事物”义——同名两个角，恰是病二（§1.8.3）要治的同词异义。

概念从哪里来?多数不是哪位天才发明的，它们的来路不外乎创造、发现与凝结：

| 方式 | 含义 | 例子 |
| --- | --- | --- |
| 创造（invented） | 人造的工具，无自然界对应物 | 变量（笛卡尔/韦达）、图灵机 |
| 发现（discovered） | 规律本来就在，被揭开 | 万有引力、能量守恒 |
| 凝结（distilled） | 共同体长期实践经验的结晶被固化 | baseline（源自军品工程实践，后由 ISO 10007 固化） |

baseline 不是哪位科学家发明的：它来自军品工程”版本失控、改坏了找不回来”的教训（库内无一手史料，见 00 号研究报告），工程共同体摸出”受控快照”的做法，最后才被标准写成条文（库内可核锚＝ISO 9000:2026 §3.10.12 配置基线、15288:2023 §3.8 基线；**ISO 10007:2017 属库外来源·不可核**）。多数要紧的概念都是这样”酿”出来的。这也预先交代了一条后文要用的性质：概念是共识，不是真理（§1.6.4）。

关键在于：概念不是孤立的词，而是一张网。一个词的意义，由它与别的概念的关系决定；冰链和恒信那两场会之所以开不下去，就是因为十张脸说的是同一个词，各自脑子里却是十张不同的网。这张网也不是平的：它有骨架，也有承重柱。

### 1.2.2 概念有主次：一般的、核心的、根上的

一个领域里的概念，不是一堆平级的名词，而是一套三层包含的体系：

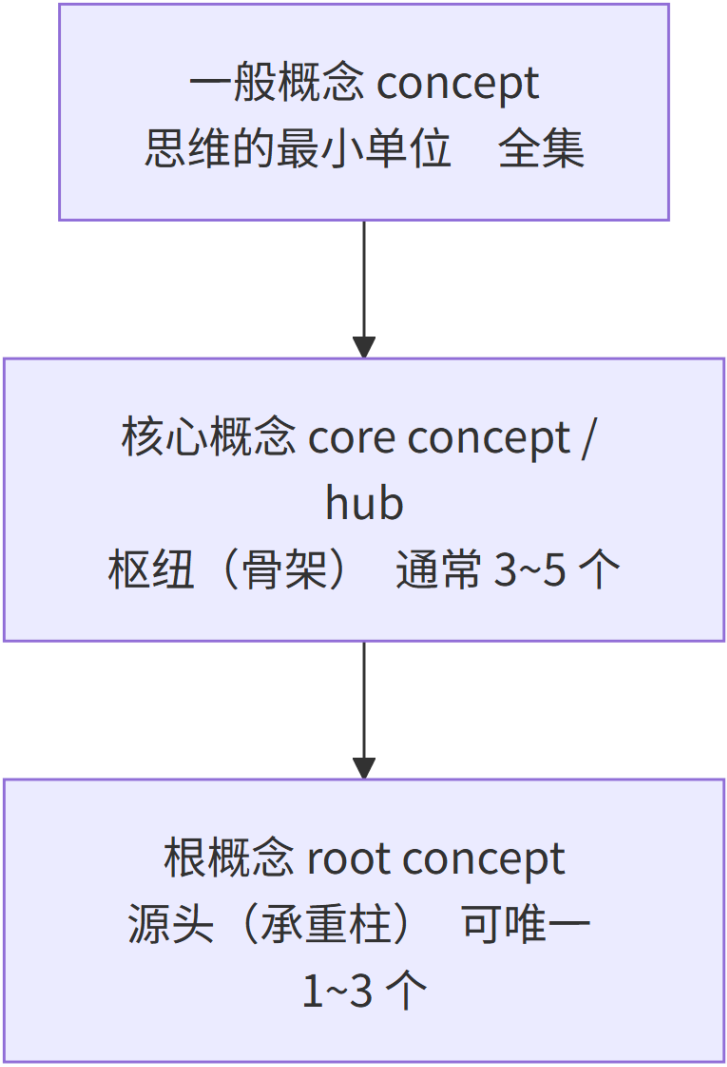

一般概念 concept
思维的最小单位　全集
核心概念 core concept / hub
枢纽（骨架）　通常 3~5 个
根概念 root concept
源头（承重柱）　可唯一
1~3 个

图 1　图 1-1

| 层级 | 名称 | 管什么 | 判据 | 数量 |
| --- | --- | --- | --- | --- |
| 全集 | 一般概念 | 可界定 | 可指称 + 可界定 | 全部 |
| 骨架 | 核心概念 | 连接多 | 反事实检验 + 词频／被引用度／前置数（后三条为代理判据） | 通常 3~5 个 |
| 承重柱 | 根概念 | 是源头 | 原子性 + 生成性 + 不可替代性 + 递归性 | 通常 1~3 个 |

这三层合起来，就是三**层体系**：一般概念 ⊃ 核心概念 ⊃ 根概念。层级越深、判据越严、数量越少。

它们的关系要连起来看。第一，一层套一层。根概念 ⊂ 核心概念 ⊂ 一般概念：每个根概念必然被全域依赖，每个核心概念必然可指称、可界定；反过来都不成立——大多数概念、甚至大多数核心概念，都不是根。

第二，越往里，门槛越高。一般概念只要求”能指称、能界定”，领域里绝大多数词都满足；核心概念还要过”枢纽性”：去掉它，领域不成立（反事实检验）；它被引用最多，前置最少。根概念再加一条”源头性”：不可再向下归约，其余概念由它导出，领域因它成立，多层

反复生效。一句话：能界定，是概念；被依赖得多，是核心概念；依赖它的一切、它却不依赖域内任何概念，才是根概念。

第三，视角不同，不冲突。这三层比的不是谁更重要，是看你说的那个概念在网里站得靠不靠中心：核心概念（hub，轮毂）管”连接多不多”，根概念（root，根）管”是不是源头”。一个概念可以连得多却非源头——“治理”很重要、被反复引用，但它由”对什么治理”定义，是派生的；质量也一样，在产品开发里被到处引用，却是需求与系统的派生。重要和源头，是两回事。

> 领域边界注记：一个概念算哪一层，要看在哪个领域里算。同一个词换个领域，位置就可能变——先看清是哪个领域，再判它站哪一层。

### 1.2.3 先抓哪个：核心概念与根概念

三层里最该先动手的是哪一层？先看中间那一层。

> **核心概念（core concept / hub）**：概念网络中处于枢纽地位的概念——解释领域内其他概念都要用到它，而理解它自己只需要很少前置知识。判据：反事实检验 + 词频／被引用度／前置数（后三条为代理判据；前置数，就是理解它所需的前置知识量，越少，越是枢纽）。

要解释”符合性”，得先解释”需求”；要解释”架构”，得先解释”系统”。“需求”“系统”被反复调用，自己却几乎不需要前置——它们就

是枢纽。章首冰链那场会上，三个人各说各的”架构”，谁也没有先问一句”系统”指什么；枢纽没立住，边界就划不下去。

枢纽的作用，每一处都关着整个领域。它决定这个领域用什么词说话（没有共同词汇，就没有精确沟通），也管着新知识往哪挂（新词要挂到已有的概念上才有意义），还管换个场景灵不灵（靠概念学到的本事能带走，靠死记硬背的带不走）。

再往深一层，是根概念——整张网的承重柱。

> **根概念（root concept）**：领域中不可再向下归约的源头概念，其余概念（含核心概念）都由它导出。四条判据：- **原子性**：不可由域内其他概念定义；- **生成性**：其余概念从它派生；- **不可替代性**：去掉它，领域崩塌；- **递归性**：多个抽象层级反复生效。

根概念 ⊂ 核心概念，一个领域里通常只有 1~3 个。概念是网上的节点，原理是连接它们的边；边也有层级：概念之间反复出现的规律（如”基线一经批准，变更须走受控流程”）是**核心原理**；再往上，领域中不可再被推导的法则，是**基本原理**——把根概念里本来就含着的道理，写成一句明白话。

> 术语注：一个概念是否含子概念，先讲父概念的内涵、把子概念作为其中一项，再讲子概念的内涵——本章 §1.2 正是这样做的。“根概念”四条判据、“基本原理”（root principle）为

> 本方法论命名，不是任何标准的条文；“概念体系能力”是本书的操作化命名，源出 Katz 1955 的概念技能研究（见参考文献 R-30）。

为什么先抓根？因为它是源头：先立它，新知识有地方挂，后面的学习越来越省力（§1.5 有实验证据）。因为它管方向：它辨对了，往哪使劲才不错（§1.4.3）。还因为它能生成一片：把它立住，领域知识由它导出，换个场景也能重新展开（§1.7.4、§1.7.5）。还有一条时间账：根概念变动的速度远慢于外围知识，立得越早越省事，立错了则后面每一层都跟着错。

怎么算立住了？拿判定矩阵与演绎链过一遍，证明它是根；答全四问（是什么、不是什么、从哪来、往哪去）；再拿它给一个陌生领域立骨架。工具见 §1.7.3 与 §1.8.2。

这一问答完就是一句话：先说准源头上的那几个词，再展开骨架，最后填细节。

## 1.3 概念体系能力：哪几样本事

> 接上一问：谁来把词说准？——识别枢纽、织关系、迁移、辨边界，这几样合起来才是它。学习里最容易白费力气的地方也就在这两处：记了一堆点，却织不成网；学会了套路，却搬不到新场景。

### 1.3.1 挑枢纽、连关系、搬得走、划得清

前面讲了概念这张网的结构；现在讲在这张网上干活的能力——概念体系能力（conceptual ability），简称”概念能力”。冰链的边界划了两个月、恒信的方案评了三个月——这些工不是卡在方案上，是卡在这张网没人搭起来。

> **概念体系能力（conceptual ability）**：以概念为单位的压缩与组织能力——识别枢纽 × 织关系 × 迁移 × 辨边界。判据（三级，一级比一级硬）：转述测试（§1.5.3 又叫”压缩测试”；做到”建构”这一级，§1.6.1）＋辨伪测试（要有人来回对质才算）＋迁移测试（最高档）。

> 术语注：标准里另有一条”知识—技能—能力”的链（ISO 9000:2026 §3.10.4 知识／§3.10.5 技能／§3.10.6 能力）：知识是使人能够做出有效的、依情境而定的决策与行动的领悟；技能是把一项任务做到规定期望的习得能力；能力是运用知识与技能达成预期结果的本领。那条链说的是人员能力由什么构成，本章说的是同一种能力练到多深——两把尺子，别并成一把。

它不是一样单独的本事，而是四样：

| 构件 | 做什么 | 反面对照 |
| --- | --- | --- |
| 识别枢纽 | 抓出决定其余部分的核心概念 | 被细节淹没，分不清主次 |
| 织关系 | 把概念连成有结构的网络 | 孤立记点，学一个丢一个 |
| 迁移 | 把网络搬到新场景 | 绑定场景，换个题就失灵 |
| 辨边界 | 用反例划清适用与失效范围 | 只举正例，不知何时失效 |

这四个构件合起来是什么成色，用转述、辨伪、迁移就能检验：

- **转述测试**（C 级，即”建构”那一级，§1.6.1）：陌生领域，三五句话讲清楚——讲不清，说明结构没成型（织关系没到位）；讲到卡壳处，说明边界没划清（辨边界没到位）。注意它的上限：讲给外行听只到”建构”这一级（§1.6.1），外行提不出有效质疑；
- **辨伪测试**（要到 I 级，即”互动”那一级，§1.6.1）：能举反例划清边界——举不出反例，说明只见过正例（辨边界没到位）；而且要**有人挑战**才算数，自己跟自己辩不算；
- **迁移测试**（最高档）：换一个情境，换行业、换粒度，你还认得出同一个结构吗？

> 概念体系能力强，不是”懂得多”，而是组织得好：同样的信息量，有人堆成一盘散沙，有人织成一张网。判断能力强弱，看的是这张网，不是信息量。

四构件之外，还有一个管方向的子能力，下一节专门说：**根概念能力**（root-concept ability）。

### 1.3.2 根概念能力：先抓源头，再展结构

四构件之外，还有一个管方向的子能力，它叫**根概念能力**（root-concept ability）。它是概念体系能力的一部分，做的是识别根概念、验证根概念、运用根概念。

说得再直白一点：它决定你先说哪个词。先从领域里挑出源头上的那几个词，再顺着它们把核心概念清单和彼此的关系铺开。四个构件负责”说出来之后怎么组织”，它负责”开口之前先站在哪儿”。

怎么算练到了？给你一个能当场试的判据：**20 分钟**，对一个陌生领域说出一组”根概念＋核心概念清单及其关系”，并且能为每一个候选说出理由。判定矩阵和演绎链就是用来辩护的，工具见 §1.7.3（候选清单的收集见 §1.7.2）。

不过这条判据有几处边界，先说清楚，免得你照着练却练歪：

- 20 分钟是练出来的，不是读出来的。第一次做，半小时能理出个头绪就算不错；
- 它是给陌生领域用的。对自己已经有底子的领域，一上来就跳枢纽，反而会让你把该看的细节漏掉；什么时候该换一种学法，§1.6.4 会讲；
- 它只管”先抓哪个词”，不管”拿主意”。把词说准和做决定，不是同一级能力；§1.4.4 会把这四级摊开给你看。

### 1.3.3 记忆可以外包，把话说准不能

这副能力在今天比过去更值钱，原因只有一句：**记忆与信息处理已经外包给机器**，剩下的识别枢纽、织关系、迁移、辨边界，才是你自己的活。

机器记得比你多、算得比你快，也写得出你没读过的句子；但它不替你判断”这个词在这里指什么”。恒信那场会就是现成的例子：“架构”在五个人嘴里是五件事，机器替你分不出谁在说哪一件。工具越强，方向与表征上的一次错被放得越大——这正是那条项项相乘、一处为零就全盘归零的说法（§1.4.1）。

一条边界同时立在这里：**AI 是好助手，不是判据**。你完全可以让它给候选清单，§1.7.2 就把”问 AI”列为收集候选的一条正路；但候选要自己用硬判据筛、用演绎链辩护。它能替你收集，不能替你拥有。

## 1.4 凭什么说回报最高：方向与表征

> 能力有了名字，接下来要问：它凭什么决定学习的成败？——因为学习产出是一个乘积，而它两头各握着一个能让结果归零的开关。

### 1.4.1 这条式子是乘号，不是加号

为什么要给这副能力单独立一个名字？因为别的能力都没有它这个位置——它是学习能力的杠杆。章题这么说，这一章就得论证到底：凭什么说把话说准回报最高？

先看这条式子：

> 学习产出 = 根概念能力（方向）× 概念体系能力（力臂）× 学习投入（施力）× 实践场景（支点）

它们的分工不同：方向决定往哪儿使劲，力臂决定同样的力气能撬动多少，施力是投入，支点是概念落脚的真实任务；后三样只负责放大，下一节逐个说。

关键在乘号。加号里的项可以互相补偿：这一项弱一点、那一项强一点，总分还过得去。乘号里不能：任何一项为零，整条式子就是零。同一门知识、同样多的时间，一个人先抓源头，一个人从细节啃起——过一阵子再看，差距不是努力拉开的，是方向拉开的。后者的方向那一项接近零，后面的力气全乘在零上。冰链和恒信那两场会就是这样：十张脸都在使劲，可各使各的方向。

这条式子因此叫**乘法律**。一错就让别的力气全白费的，却只有两个归零项：方向（先抓错了词，§1.4.3）和表征（话说得不准，§1.4.5）；其余都只负责放大——投入再多、场景再好，也只是把已经错了的东西放大。

这是”回报最高”的第一层理由：把话说准，是这条乘法律的两个归零项之一，归零项就是方向与表征这两根轴。它不是让学习好一点，而是决定学习有没有。而它同时是最便宜的一步：立枢纽只要 20 分钟，给一个词建一张概念档案只要一张纸（§1.8.4）。

“杠杆”这个说法，讲的是这条律的另一半——放大。所以章题没有说错，只是这个说法自带边界：杠杆放大的是力，却不分方向，枢纽选错，学得越快偏得越远；杠杆需要支点，概念不落真实任务，力臂再长也撬不动什么；杠杆自己不发力，没有投入，力臂无穷长也是零产出。

### 1.4.2 力臂、施力、支点：只负责放大

式子里的另外三项，性质不一样：它们只负责放大，不负责指方向。

- 力臂，就是概念体系能力。把枢纽连成关系、搬到新场景、划清边界；同样一份材料，会组织的人拿走的是结构，不会组织的人拿走的是一堆点；
- 施力，就是学习投入。时间、注意力、练习量，缺一样，力臂再长也不动；
- 支点，就是实践场景。概念要落在真实任务上才使得上劲；只记不用，前面两样也一起白费（§1.6.2 第 5 环、§1.6.3 迁移）。

它们有一个共同点：放大的都是已经确定的东西。方向对了，力臂越长撬得越多；方向错了，力臂越长偏得越远。冰链那边的力气和经验都不缺，施力、支点都在，卡住的只是最初的那一项。同一门知识，两种学法的差别就在这里：

| | |
|---|---|
| 先学核心概念（先立骨架） | 只背结论和招式（无框架） |
| 知识有地方挂：新概念迅速入网 | 碎片化记忆：学完就忘 |
| 举一反三：跨场景自由迁移 | 绑定具体场景：换个题就失灵 |
| 能辨伪：识别概念偷换 | 易被带偏：分不清概念边界 |

先立骨架再填细节，后面的学习越来越省力；先切入细节，越来越吃力。这不是经验之谈，§1.5 给出的实验会证明这一点。

### 1.4.3 方向：挑错了词，全盘皆输

四个部件里，最根本的不是力臂，是**方向**。为什么？

因为其他三个部件都在”放大”，只有方向在”指路”——放大是没有方向感的，方向一旦错了，后面每一份力气都加在错误上。压缩

学习实验里，枢纽优先组的全部优势，正来自”先识别枢纽”这一下。

其后 800 轮正式学习，都只是这个方向上的展开（§1.5）。

方向不是一次选定的，而是三层递进：

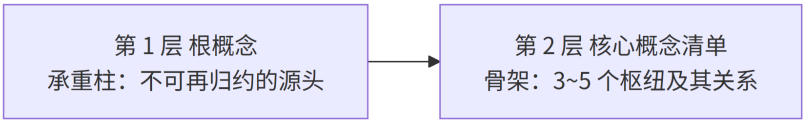



图 2　图 1-2

先问本体论问题：这个领域里，什么东西是存在的？**再抓根概念**：哪几个是不可再归约的源头？**最后展开**成核心概念清单及其关系。

第 0 层答错，上面两层再漂亮也是错的：把”产品”当成企业运行领域的根，与把”产品数据”当成根（§1.7.5），导出的是两张完全不同的网。根概念能力就是把这三层都定准的那一下。

这就是本章标题的一半意思：概念体系能力是学习能力的杠杆，而杠杆的第一性是方向。所以本书反复强调”先立枢纽”——它不是一条技巧，是把方向握在手里的那一下。

一个澄清，防止误读：概念体系能力管的是学习的头一步和放大那一步，不是学习的全部。那条训练循环（立枢纽、织关系、逼输出、勤校准、落实践）的每一环都在练它，其中立枢纽与织关系练得最直接；逼输出、勤校准、落实践这三环还要配另外三样：把学过的拿出来用、随时检查自己哪里想错了、以及坚持下去。它决定学得多快、看得多深；那三样决定学不学得动、学不学得远。

### 1.4.4 一个词能说到多深：能力四级

方向选定了，接下来要问的是说得有多准。同一个词，能说到好几个深度；说得浅，等于没说。

这一章后面有一场评审会，正好是这件事的例子。恒信的万辉把”架构”答了个滴水不漏：是什么、不是什么、从哪来、往哪去；可轮到”这个系统要不要拆”，他一个字也说不出来。不是他没准备，是他的能力只到第一级。

我们在大脑里操作的从来不是真实世界，而是它的代用品。你思考的不是经济活动本身，是那些概念和指标；你理解的不是历史，是某种历史叙事。能做多大的事，取决于手里的代用品长成了什么样。代用品本身有粗有细，粗到能判断，细到只是一个词。能力四级，就是这四种粗细：表征、图式、心智模型、解释框架。它不是四种知识，是同一种能力的四档深浅：

| 级 | 是什么 | 一句话判据 |
| --- | --- | --- |
| ① 表征 | 现实事物在头脑中的代用品；最小单位——概念、名词、对象、关系和边界 | 能提取关键表征，并把它画成关系图，才算入门 |
| ② 图式 | 表征聚起来、认得出的一类结构；一眼能认出的套路 | 看事情一块一块看；一听就知道这是哪一出，还能预判下一步 |
| ③ 心智模型 | 会动的图式——内部有变量、有因果、有反馈、有边界条件 | 能推演”如果 A 则 B”，能说出它的失效边界在哪 |
| ④ 解释框架 | 对一整个领域的整体看法：把一堆模型和套路攒成的一整套 | 能判断什么值得相信、什么只是叙事包装 |

第一级是表征。“苹果”这个词就是表征；真实的苹果是一堆原子，你脑子里的苹果是红的、圆的、甜的。零散地记住一批词，不叫入门；抓住关键表征、认出它们之间的关系、画出图来，才叫入门。

第二级是图式。表征聚成结构，就成了图式：一看就知道这是哪一出，还能预判下一步。没图式的人看什么剧情都新鲜；图式多的人看什么都是套路。反过来说，恒信那五张”架构”的脸，连表征这一级都还没对齐——更谈不上图式。

第三级是心智模型。图式回答”这是什么”，心智模型回答”它怎么运转”。万有引力定律是一个心智模型，费曼”在积分号下求导”的

绝招是一个心智模型；本书后面讲的每一条判据，说到底也是一个心智模型。

第四级是解释框架。它是当前某一派意见对一整个领域的系统性看法——回答的是”这一大类现象该怎么解释”。摸到这一级，你才开始像行家那样理解世界：不是追故事，而是问机制；不是摆立场，而是比解释力。

> 术语注：前三级的说法在认知科学里是通行的；**第四级”解释框架”是课程作者的自命名**（原话：“这个词是我发明的”），不是标准术语——本书借用它的分法，读者不必把它当成定论。

四级共用同一种能力，只是粗细不同：识别枢纽，第一级是挑出最该先认的那几个词，第四级是认出”这一派把什么当枢纽”；辨边界，第一级是说清这个词不指什么，第三级是说出模型的失效边界。同一个构件，换一级来用，做的事是同一件。

本章的工具停在第一级。四问是第一级（表征）用的工具，管的是”把词看准”：这个词指什么、不指什么、从哪来、往哪去。这是最底层的一级，也是最便宜、最容易被跳过的一级，但它不负责拿主意。万辉的四问答得再好，也只是把几个词说准了——他还没有能推演的机制，更谈不上判断该不该拆。本章后面给出的判据，也都还在这一级；它们长成能推演的机制，要到后面几章。

这里有一个可以自查的崩样：

> 能把一个概念的四问答全，却在一个真实的取舍面前说不出话——这不是表达能力问题，是能力层级问题。

那上面三级谁来建？本书后面每一章都在建，只是你此前不知道它们各在哪一级：

| 级 | 每一级处理什么、怎么算做到了 |
| --- | --- |
| ① 表征 | 本章的四问（是什么／不是什么／从哪来／往哪去）与术语工作；第 3～5 章的领域概念——判据：能提取关键表征，并画出关系 |
| ② 图式 | 本章的三层体系与”织关系”；第 2 章的工程四层与对立面；第 6、7 章的结构与产品数据——判据：能认出结构，预判下一步 |
| ③ 心智模型 | 第 10～12 章（架构、架构设计、架构决策）——判据：能推演，能说出失效边界 |
| ④ 解释框架 | 第 17 章整合、第 18 章架构师——判据：能判断什么值得相信 |

所以这本书不是一个一个知识点，是同一种能力逐级往上练：先在表征这一级把话说准，再在图式这一级把结构认出来，然后建能推演的心智模型，最后逼近判断。

这四级里，前三级可以讲、可以写、可以画出来；第四级有一块是讲不出来的。有一种知识只能在现场长在身上——老师傅调火候、老

工程师看一眼就知道哪不对，那不是天赋，是在现场待久了长出来的本事。本书教你把话说准、把结构理出骨架；但第四级里有一块，写不进书里。认下这条边界，比多背几条判据有用。

### 1.4.5 表征：第一级拿不到，上面全是零

“回报最高”还有第二层理由，就藏在这四级之间：

> **价值律**：**表征是 1，上面三级是 0**（本书命名。“1 和 0”说的是成不成立、以及量级，不是分数。）

先说”1 定成立”这一半：第一级决定的是**成不成立**，词指错了东西，上面三级再漂亮也是零。前言里那句话说的正是这一级——“地基不准确，上面盖多少层都白搭”。本书反复讲的”先把话说准”，不是风格要求，是这条式子里最靠前的那个数。

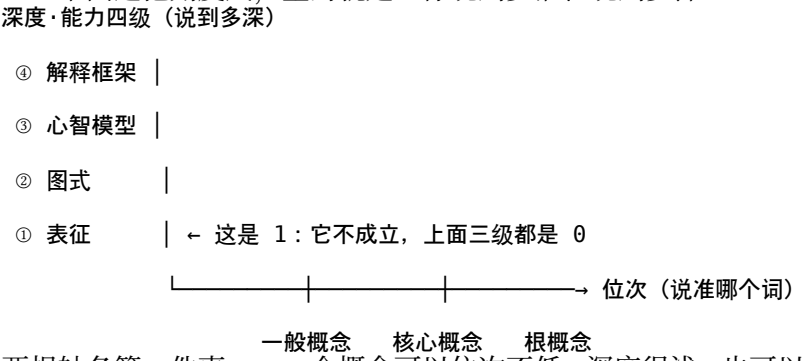
再说”0 定量级”这一半：上面三级不是不重要，它们决定的是**能做多大事**：从”看准一个词”，到”看出一套套路”（图式），到”能推演”（心智模型），再到”能判断什么值得相信”（解释框架），每加一级，同样一份信息能做的事就翻着倍往上涨。**1 定成立，0 定量级**——这就是为什么第一级最便宜，也最不能省。省掉 1，后面全是 0；有了 1，后面每一级都实打实地加量级。

于是这条乘法律的两个归零项到齐了：**方向**（§1.4.3：先抓哪个词）与**表征**（本节：话说得准不准）。方向管往哪使劲，表征管拿的是不是那个东西；两者都过，才轮到力臂与投入去放大。这就是章题的

完整意思——概念体系能力是学习能力的杠杆，因为它同时管住了这条律的归零项与放大项。

两个归零项，两根轴，合成一个坐标系：位次轴（一般 ⊃ 核心 ⊃ 根）回答”该抓哪个词”，深度轴回答”这个词你要说到多深”，它的刻度就是能力四级。它就是你量”说到多准、说到多深”的那把刻度尺。

下面这把刻度尺，量的就是”你说到多准、说到多深”：




两根轴各管一件事——一个概念可以位次不低、深度很浅，也可以位次不高、深度很深。万辉的”架构”就是前一种：位次上落在核心概念，深度上只到表征。枢纽抓到了、话也答全了，可能力只到第一级。所以谈到任何一个概念，都要同时报出它的两个坐标。

> 术语注：位次轴上的”三层”（一般／核心／根）与深度轴上的”四级”（表征／图式／心智模型／解释框架）是两个量词，别混：论位次说”第几层”，论能力说”第几级”。

最后交代一句”概念”这个词。四级处理的东西，本书一律叫概念。这是本书自己的用法，比 §1.2.1 的严谨定义宽：概念是头脑里对现实事物的代用品；它在四级里各是一个样子：表征这一级是**现实事物在**

**头脑中的代用品**，即概念、名词、对象、关系和边界；图式这一级是**认得出的一类结构**；心智模型这一级是**会动的图式**；解释框架这一级是**对一整个领域的系统性看法**。

> 术语注：严谨定义（ISO 1087:2019 §3.2.7 的知识单元、§1.2.1 的可指称＋可界定）管的是术语意义上最窄的那一层；四级通称”概念”，是本书的操作读法。

## 案例回场｜答得全，却说不出该不该拆

恒信的评审会上，方案要过一道关：合同系统要不要拆成两个。

万辉先把”架构”答了一遍——它是什么：一个实体在其环境中的基本概念或属性，以及用于该实体及其相关生存周期过程的实现和演化的管控原则（ISO/IEC/IEEE 42010:2022 §3.2）；它不是什么：不是一张架构图，不是一次技术选型，不是组织架构；它从哪来：从建筑学借入软件与系统领域，1970 年代结构化设计、1990 年代软件架构热，后由标准定稿；它往哪去：不可逆决策被做出之前用，变更影响评估时用。

四条一条不差。

徐漾问了一句：“那这次拆不拆?”

万辉停了几秒：“……我得再看看。”

他不是没准备。他手上的”架构”停在第一级：词说准了，关系也画得出；可拆与不拆要的是能推演的机制——服务边界怎么切、数据所有权怎么划、拆完之后哪条链会断。这些他还没有。

**四问答得全，只是拿到了那个 1**

## 1.5 有什么证据：一项压缩学习实验

> 道理讲完了，该给证据。凭什么说”先立枢纽”不是经验之谈——2026 年北京大学团队的一项实验，证明了枢纽优先的学习结构为什么占优。

### 1.5.1 差距在压缩，不在记性

2026 年，北京大学心理与认知科学学院等团队在《自然·通讯》发表了一项研究。它给出了一个反直觉的答案：高手和普通人的差距，不在记性好，而在擅长”压缩”。所谓压缩，就是把全量信息转化成关键信号。

> 术语注：这是本书对实验的读法。实验直接测量的是”枢纽优先的知识结构更容易掌握”（§1.5.2）；从”结构更易学”引申到”高手与普通人的差距在压缩”，是本书的一步引申——它经得起检验，但不是论文的原话。

经验医生把成千上万种疾病的知识，压缩成几个关键信号；资深调查记者在堆成山的文件里，只盯着”异常”的部分。他们都在做同一件事：压缩学习。章首那十张脸反着来——十套没压缩过的说法，谁也接不住谁。

压缩为什么有效？机制在带宽上：**工作记忆一次只能同时处理 4 到 7 个信息元素**（常被引用的经验值；机制依据见 R-106 第 38 讲引

的认知负荷理论）。学习失败的常见原因不是”存不下”，而是新信息过这道窄门时堵住了。压缩的单位叫**图式**（schema，认知心理学里的既有概念，即 §1.4.4 四级里的第二级）：把多个相关元素打包成一个整体，打包得越多，一次能过的信息就越多。北大团队这一步的贡献，是把”什么样的结构最容易被压缩”量了出来。

### 1.5.2 实验：先学枢纽的人更快

团队设计了几类结构相同、但连接方式不同的人工知识网络，找一批人来学。结果很清楚：少数枢纽占据大量连接的网络（无标度网）最好学；均匀对称、没有主次的结构反而难掌握。

另一个实验更有说服力。同一批知识，一组先学枢纽再学细节（枢纽优先），一组先学边缘再学枢纽（边缘优先），还有一组随机学。进入完全相同的正式学习后，枢纽优先组的优势随时间越来越大，而且扩散到整张网络——包括那些从没直接学过的纯细节节点。

脑成像也印证了这一点：枢纽优先的学习者在大脑里维持着更稳定的知识结构表征；先学边缘的人，早期也有表征，但迅速衰减。先搭框架的人，新信息进来只需判断”挂在框架哪个位置”；先切入细节的人，后期积压太多零散节点，整合不动。

### 1.5.3 把压缩用在自己身上

研究给了三步可以直接照做的动作。**找到枢纽**：学任何新东西前，先花 20 分钟找枢纽节点，翻书目录、读百科开头、问 AI、问行家都行；比如《思考，快与慢》的枢纽就是系统一、系统二、偏见。**建立联系**：先弄清枢纽之间的关系，再填细节，关系没通，细节学一个丢一个。**转**

**述**：讲给完全外行的人听，三五句话讲清楚，就是压缩成功；转述是”压缩测试”，费曼技巧的机制正是它。

它们不是孤立的技巧，而是后面那条循环的头三环。§1.6 把它们接成一条完整的练法。

### 1.5.4 它证明了什么，没证明什么

论文也有它的边界，每一条都要说清：

- **生态效度**：实验用的是抽象的知识网络，真实领域（教科书、标准、行业惯例）的结构复杂得多；怎么识别真实领域的枢纽仍是手艺活，论文没有替你解决——这正是 §1.7 要做的事：三步识别法＋判定矩阵；
- **测的是预测，不是应用**：范式测的是”预测下一张图片”这类隐含结构的学习，没有直接测应用与迁移。所以”转述能验证掌握”是本书补上的一步，不是论文的结论；
- **先有鸡还是先有蛋**：枢纽优先的前提是先知道枢纽在哪；面对完全陌生的领域，只能靠二手信号（目录、百科、专家、AI）先取候选，边学边修正（§1.7.2）。

另外，别把”框架形成后表征更稳定”过度引申成”框架不可改”——框架可改，见 §1.6.4 校准。

## 1.6 怎么练：一条循环转起来

> 道理和证据都有了，接下来是手上的活。这副能力不是天赋，怎么把它练出来——一条可以一圈一圈转的循环。

### 1.6.1 什么叫学会：会用才算掌握

先说”掌握”是什么意思。教育学界有一个公认的认知目标阶梯，叫布鲁姆分类学：知道 → 理解 → 应用 → 分析 → 评价 → 创造，六级。

大多数人的学习停在”知道”和”理解”：能复述，能听懂，考试能过；换个场景，就用不出来。分界线画在”应用”上——会用才算掌握。你背得出”质量＝客体的固有特性满足需求的程度”，不算掌握质量；你拿着它评一份验收标准、判断一次分歧算不算质量问题，才算掌握。恒信评审会上徐漾那句”那这次拆不拆”，问的正是”会不会用”。

还有一条更细的分界：同样一段时间，参与的方式决定收效。被动听讲（P）最弱，主动复述（A）强一些，**建构（C）**才算真学习，能生成材料里原本没有的新信息；能与人来回对质（互动，I）则更强。这条次序说明了：为什么”讲给外行听”只是入门线（外行提不出有效质疑），以及为什么第 3 环要配第 4 环（§1.6.2）。

### 1.6.2 循环：从立枢纽到用起来

掌握专业知识有一条循环，就叫**五环模型**：立枢纽 → 织关系 → 逼输出 → 勤校准 → 落实践。每转一圈，骨架更准、网络更密、输出更顺：

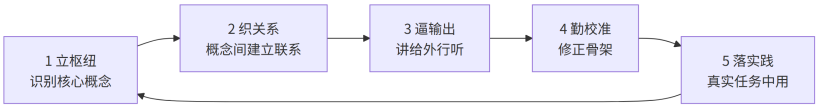



图 3　图 1-3

| 环节 | 做什么 | 为什么（依据） |
| --- | --- | --- |
| 1 立枢纽 | 学新领域前先花 20 分钟识别枢纽概念：翻目录、读百科开头、问 AI、问行家 | 压缩学习：先学枢纽，好处会扩散到整张网；一上来要记的东西最少（§1.5） |
| 2 织关系 | 弄清枢纽间关系（包含/依赖/对立）再填细节 | 知识是网络不是列表；记忆靠连接检索（§1.2.1） |
| 3 逼输出 | 三五句话讲给外行、写成文档、做题、教别人 | 自己回想一遍，比再看一遍记得牢（测试效应）；一讲就露馅，缺口自己冒出来 |
| 4 勤校准 | 随时准备推翻自己：枢纽选错、关系画反就改 | 想错的地方，得用更准的说法顶掉（概念转变） |
| 5 落实践 | 在真实任务中用：做项目、审文档、解真问题 | 只记不用的知识是死的（惰性知识）；真本事＝概念结构×在场景里练出来的经验 |

这张表里最容易被跳过的是第 3 环和第 4 环，而它们恰好是**刻意练习**的要件：目标明确到能判对错、反馈快到能马上改。没有反馈的重复不叫练习，叫重复——这也是”讲出去”和”勤校准”要挨着放的原因。恒信那场会上，徐漾那声”停”就是一次现成的反馈，可惜没人接住（§1.8.5）。

掌握 = 用结构消化信息，用输出验证结构，用实践把结构磨出来。学习不是把知识装进脑子，而是长出结构，让新知识有处安放。

### 1.6.3 每一步在练哪一样

五环的每一环都有名字：立枢纽 → 织关系 → 逼输出 → 勤校准 → 落实践。能力不是天生的，是练出来的。这不是说天赋不存在，而是说天赋与练习是乘法关系：天赋决定每次更新的利率，练习决定你有没有在更新。

专家与新手之间，差别也出在这两样上。一部分在正确性标准的分辨率，专家能分辨出新手看不出的差别；一部分在内化了的身体图式，手艺长在身上，说不出来但做得出来。都不是天赋，是路径。

怎么练？这条从立枢纽到落实践的循环，本身就是练法，四个构件在五环里各就各位：

- 识别枢纽 → 第 1 环”立枢纽”：四构件的第一个，就是五环的第一步；
- 织关系 → 第 2 环”织关系”：把概念连成网，正是织关系构件；第 3 环”逼输出”讲不清，说明结构没成型，也是织关系没到位；
- 辨边界 → 第 3 环”逼输出”：讲到卡壳处，就是边界没划清的地方；第 4 环”勤校准”：用反例推翻旧骨架，也是辨边界；
- 迁移 → 第 5 环”落实践”：换场景用，正是迁移构件。

所以五环不是五件事，是四构件在循环里转。每转一圈，四构件都更准一点——这是概念体系能力最实在的训练路径，也是”能力是练出来的，不是天生定死的”这句话怎么落到做上。

第 5 环”落实践”最常失效的地方在**迁移**。从学到用，中间有三道锁：**表征锁**是学的时候就没抓住结构，只记住了字面；**触发锁**是知道该用，但那个场景里想不起来用；**生态锁**是真到了现场，环境不给你用的条件。对应的两个策略很朴素：**拥抱**，主动把新知识往自己已有的场景里塞；**桥接**，拿旧场景里的经验去接新知识。迁移测试（§1.3.1 第三级）量的就是表征锁、触发锁、生态锁这三道锁开没开。（这三道锁是课程对迁移研究的提炼，本书沿用。）

### 1.6.4 校准：概念是共识，不是真理

五环的最后一根柱子，是第 4 环”勤校准”。它凭什么立得住？因为概念是共识，不是真理（本书底色：反绝对真理）。

化学的”燃素”、物理的”以太”，都曾是各自时代的核心概念，后来被推翻。库恩把它叫做”范式转换”——每个时代的枢纽清单，是那个时代认知的天花板，不是终点。

所以，学核心概念，就是花最小的力气，把这一行几百年攒下的共识接过来；但它是共识，不是真理——枢纽是起点，不是天花板。这也正是”勤校准”的意思：连领域公认的枢纽，都可以怀疑、修正。

对管方向的概念体系能力来说，这条尤其要紧：方向不是一次定死的，要一路校准。你今天立的枢纽，明天可能被证明是错的——那不是失败，是这条乘法律在正常纠偏。枢纽清单是活的，方向就永远是活的。

最后一条边界：先立枢纽不是一刀切。对完全陌生的领域，先花 20 分钟找枢纽最划算；对已经有基础的领域，一上来就跳枢纽反而会压住细节——同一种学法，在新手身上有效，在行家身上会失效。练

的时候还要挑**可取的困难**：在自己扛得住的范围内跟材料较劲（先自己回想再翻书、把话说给别人听），而不是跟疲劳和干扰较劲。

## 1.7 怎么找：挑出枢纽与根概念

> 说准哪个词、凭什么、怎么练，都答完了。落到手上还有一问：具体怎么找到那个词？——三步识别法、判定矩阵与演绎链，最后走三个真实领域。

### 1.7.1 概念是点，原理是线

找枢纽，先分清两个层级：

- 核心概念 = 节点（名词）：领域的”词汇”，如需求、配置、基线、版本；
- 核心原理 = 边/规则（一句判断）：概念之间反复出现的规律，通常用”当……必须……“”……取决于……”表达，如”基线一经批准，变更须走受控流程”“验证以规定需求为参照”。原理是更高级的枢纽——掌握原理，概念自动串联。

原理也有层级：核心原理之上，是基本原理，也就是原理侧的源头（见§1.2.3）。

### 1.7.2 怎么挑出枢纽

找核心概念（枢纽），可以顺着三步走：

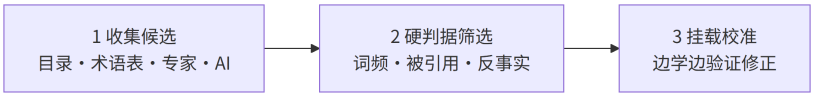



图 4　图 1-4

第一步，收集候选，借现成的材料，20 分钟就够。领域共同体已经帮你压缩过，直接”抄”就行：

| 来源 | 为什么有效 |
| --- | --- |
| 教材目录/章节标题 | 好教材的顺序本身是枢纽优先的（先基础后应用） |
| 专业术语表 Glossary | 共同词汇的浓缩，出现即被共同体认可 |
| 课程大纲/综述论文 | 专家设计并经多年检验；综述是高手压缩好的结构 |
| 问 AI/问专家 | 直接要”10 个核心概念及其关系”，再交叉验证 |

收集阶段宁多勿少：判据要求”20 分钟给出根概念与核心概念清单及其关系”，那是收敛之后的答案；收集时先要”10 个”，多出来的正好用来交叉验证、往上挂。

第二步，硬判据筛选，用枢纽的定义去验证：

- **词频**：领域文本里被反复提到的名词，多半是枢纽；
- **被引用度**：统计概念定义中被引用的其他概念，被引用最多者最基础——比如”需求”的定义，被整个质量体系引用；
- **前置数**：理解它需要的前置知识越少，越像枢纽——数一数”要讲清它，得先讲几个别的概念”；
- **反事实检验**：问一句”如果这个概念/原理不存在，这个领域还成立吗？“经济学少了”供需”，整个框架就塌了——它是枢纽。反事实检验是四条里最狠的一条。

> 术语注：前三条是**代理判据**——用可数的量代替真正要判的东西，方便，但会骗人。本书早期草稿就写错过一次：把”被引用度＝出度、前置数＝入度”当成枢纽的判据（出度是它连出去

> 的关系，入度是连进来、指向它的关系），那测的其实是**根**（被全部依赖），不是**枢纽**（连接最多）。真要判谁是根，回到**根概念的四条判据**与判定矩阵（§1.2.3、§1.7.3）。

第三步，往上挂。每学一个新概念，都问一句"它挂在哪个枢纽下？"挂不上去，说明枢纽清单有漏洞。清单是活的，随理解加深增删；专家与新手的区别，不是清单长度，而是清单准不准。

### 1.7.3 判定矩阵与演绎链

三步识别法给出的是核心概念清单（骨架）；要辨出其中哪几个是根概念（承重柱），还要过判定矩阵与演绎链。根概念 ⊂ 核心概念，正好对应 §1.2.2 的包含关系。

识别根概念不能靠感觉，要用两件能查、能对质的工具：**判定矩阵**（横向打分）与**演绎链**（纵向论证）。

**判定矩阵**：四条判据（原子性／生成性／不可替代性／递归性）× 候选概念，逐格打分并附证据：

| 候选 | 原子性 | 生成性 | 不可替代性 | 递归性 | 判定 |
|---|---|---|---|---|---|
| 候选 A | ✅/⚠/❌ + 一句证据 | …… | …… | …… | 根 / 派生 / 子类 |

**演绎链**：把结论写成能一条条查的论证链。每个前提（P）附出处，结论（C）由前提逐条推出：

```
P1：候选 = 三步识别法收集的候选清单（§1.7.2）

P2：根概念 = 不可再归约的源头概念，四条判据判定（§1.2.3）

P3：判定证据 = 标准族 / 工程现实（可核验）
```

**P4：候选 X 通过四条判据的判定矩阵（见上表）**

**⟶ C：X 是该领域的根概念**

> 演绎链的意义：把”我觉得是”变成”因为 P1、P2、P3、P4，所以是”——每一环都可以被反驳、被替换，结论因此可辩护。这正是根概念能力判据里”为每个候选辩护”的完整操作。

### 1.7.4 根概念长什么样：阻抗

这两件工具落在真实领域里长什么样？先看两个领域，各用几句话走完。

高速信号传输，根是阻抗。说起高速信号，你多半先想到速率、带宽——但它们都是结果，不是根。信号一上高频就失真：反射、串扰、衰减、损耗、时序，根子都在阻抗。

用四条判据核验：阻抗由传输线本身的特性（线宽、介质、间距）决定，不依赖域内其他概念（原子）；特性阻抗、反射、串扰、损耗、时序都由它派生（生成）；去掉它，信号完整性领域不成立（不可替代）；从芯片引脚到背板到系统，层层都生效（递归）。所以这个领域的根概念是阻抗（1 个）。

产品开发，根是需求、系统、设计开发。很多人以为根是”质量”——但质量是需求被满足的程度，是派生的；也有人以为是”产品”——那只是一个交付物的名称。三个源头各答一问：需求回答”要什么”，系统回答”承载在什么上”，设计开发回答”怎么把它一级一级说到可实现”。恒信要评的改造方案、冰链要分的 M3 二代，说到底都挂在这三个词上。

质量、功能、接口、组件、架构，都从这三者派生；四条判据逐条同样过：都不可再定义、都生成一片、去掉一个领域就失去立足点、从产品级到模块级层层生效。所以产品开发的根概念是需求、系统、设计开发，共 3 个。

### 1.7.5 根概念长什么样：产品数据

第三个领域，走一遍完整的判定——它是本书的主场。

企业运行体系的根概念是产品数据。为什么不是”产品”？设计与制造之间传递的不是实物而是模型与图纸，质量与合规证明的是数据记录，产品的”身份”就是它的数据档案——在企业运行层面，产品以数据形态存在。

更反直觉的是，根也不是流程。企业运行不就是跑流程吗？但流程由”数据的流转次序”定义——它不原子；去掉流程，数据仍在，只是无序。流程只是产品数据按时间排出来的样子；同理，组织是它按空间摆出来的样子、绩效是它按价值算出来的样子；配置管理管的是产品数据的受控状态、IT 系统是它的数字化载体——投影可再造、可重组，都不是根。

真正过得了这四条判据的是产品数据：自身即事实（原子）、一切由它派生（生成）、去掉它企业只剩空壳制度（不可替代）、概念级/流程级/实例级都生效（递归）。

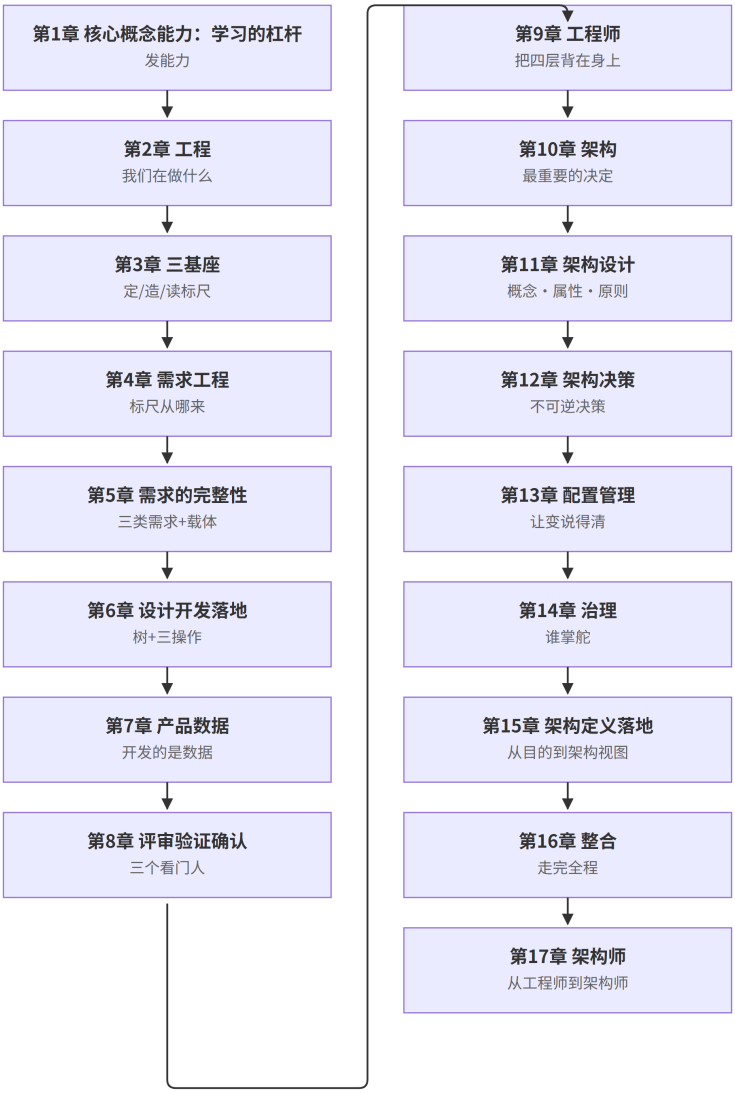

第1章 核心概念能力：学习的杠杆
发能力
第2章 工程
我们在做什么
第3章 三基座
定/造/读标尺
第4章 需求工程
标尺从哪来
第5章 需求的完整性
三类需求+载体
第6章 设计开发落地
树+三操作
第7章 产品数据
开发的是数据
第8章 评审验证确认
三个看门人
第9章 工程师
把四层背在身上
第10章 架构
最重要的决定
第11章 架构设计
概念・属性・原则
第12章 架构决策
不可逆决策
第13章 配置管理
让变说得清
第14章 治理
谁掌舵
第15章 架构定义落地
从目的到架构视图
第16章 整合
走完全程
第17章 架构师
从工程师到架构师

图 5　图 1-5

> 图注：流程、组织、绩效是产品数据在时间、空间、价值三个维度上的投影；配置管理、IT 系统是它的受控管理与数字化载体——投影可重组，产品数据这一个根不变。

所以企业运行体系的根概念，是产品数据（1 个）。这个域的核心概念则是流程、组织、绩效这三个投影，以及被反复引用的需求、质量、治理；它们都由产品数据派生，都不是根。

注意：同一个词换个领域，位置就可能变。需求在产品开发域里是根，在企业运行域里只是核心。产品数据这根承重柱，第 7 章会把它展开为”开发产品就是开发产品数据”。

（三个示例是本书按四条判据判定的结果，供对照，不是任何标准的条文。）

## 1.8 怎么验：答不全，就只是见过

> 最后回到手上最容易含糊的一步：一个词说到什么程度，才算说准了？——一把尺子（四问法）、几种最常见的”没拿准”（词汇病），以及要查全时去哪儿查。

### 1.8.1 假懂长什么样

怎么才算真正说准了一个枢纽概念？先看”假懂”长什么样：能背原文，却举不出反例；不知道它何时失效；孤立记住，不知道它挂在哪个枢纽下；不关心它从哪来；只在考试里答得出来。

真懂反过来：能用自己的话重述，能举正例也能举反例，知道它不是什么、何时不成立，知道它挂在哪个枢纽下、与谁冲突，知道它解决什么问题、为何被发明，讲给外行能懂、换个情境能用。

理论依据是维特根斯坦的一句话：“意义即用法”（meaning is use）——概念的意义不在字典里，在它的用法里。理解一个概念，就是会用它，而不是会背它。按这条看，章首那十张脸各说各的”架构”，说的确实是十个不同的概念：词是同一个，用法各是各的。**但十种用法不等于十个架构**——同一个实体只有一个架构；用法多出来的那些，是**关注面与描述**，不是多出来的架构（第 10 章 §10.4.4）。

### 1.8.2 怎么问才算问全

怎么判断自己真懂了？把上面那些差别压成四个问题：

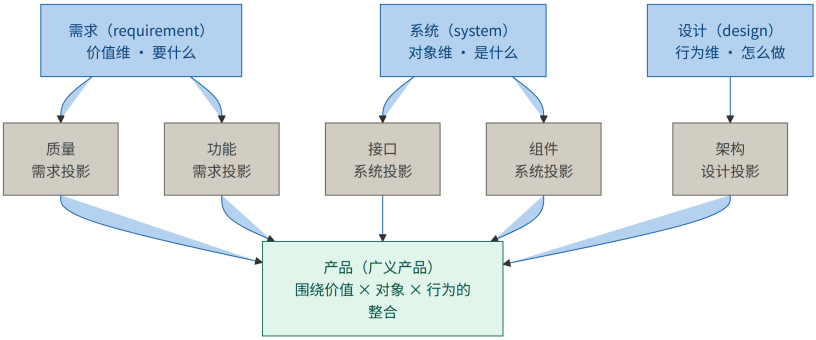



图 6　图 1-6

| 问 | 操作 | 理由 |
| --- | --- | --- |
| 它是什么? | 找三四个来源（标准、教材、百科、高手著作）对照定义，差异处即本质处 | 定义的差异暴露概念本质；同一个词，标准与教材表述不同，差的那一点往往就是本质 |
| 它不是什么? | 找正例和反例，用反例磨边界 | 边界靠反例定型；概念靠典型例子活，靠反例定型 |
| 它从哪来? | 回到概念被创造时的问题现场：解决什么问题？没有它之前人们怎么受苦 | 知道它当初解决什么问题，才记得住；词源能让它活起来 |
| 它往哪去? | 挂在什么流程、被谁使用、何时失效 | 用得越熟越知道适用条件；掌握失效边界＝深度理解 |

这四个问题都答全，才算拥有它；只能答第一个，只是见过它。这背后，是本书底色的反记忆崇拜——懂不等于记住。大多数人的”懂”，其实是”见过”：背得出定义，却答不出它”不是什么”的边界，也答不出它”往哪去”。

同一道门槛，本章量过三次：答全上面这四问，是给这个词做体检；把关键表征画成关系图，是把它摆进网里（§1.4.4）；三五句话讲给外行听，是压缩测试（§1.3.1）。三次都过，才算把它提到第一级。

问这四个问题之前还有一步，专门管词的身份，叫**术语锁定三问**：① 这个词出自哪部标准的哪个条款？② 那部标准的术语章里有没有定义它？③ 没定义的话，它是按通用语意用的吗？“概念”就是个现成

的例子：它在 ISO 1087:2019 §3.2.7 有正式定义，在 ISO/IEC/IEEE 42010 里却是按通用语意使用的（§1.2.1 的两个”指称”之争就是这么来的）。词的身份没锁住，那四个问题会问在错的对象上。

拿章首那个被吵翻天的词练一遍。量的是本书口径下的那个所指，也就是系统架构：

- 它是什么：一个实体在其环境中的基本概念或属性，以及用于该实体及其相关生存周期过程的实现和演化的管控原则（ISO/IEC/IEEE 42010:2022 §3.2）；
- 它不是什么：不是一张架构图（图会过时，架构的基本概念持续起作用）；不是一次技术选型；更不是组织架构、产品形态；
- 它从哪来：从建筑学借入软件与系统领域；1970 年代结构化设计、1990 年代软件架构热，后由 ISO/IEC/IEEE 42010（现行 2022 版）定稿为系统与软件工程标准；
- 它往哪去：在不可逆决策被做出之前用；在变更影响评估时用；在团队边界定义时用。失效边界——当组织架构、产品形态混进来时，它就说不清了。

答完四问，章首那两场吵架的解法已经出来了：他们吵的是各自不同的”它”。四问先帮他们分清在谈哪一个，再谈要不要改。

## 案例回场｜先分清在谈哪一个

回到章首那两场会。这一次，屋里有人带了四问：它是什么、不是什么、从哪来、往哪去。

冰链那边，文正先说自己指的是**产品架构**（他关心的是模块怎么分）；水忠说他指的是**硬件架构**（传感器、主控、通信怎么布）；董勐

说他指的是**系统架构**（服务边界与数据所有权）。三个人把各自的四问答完，才看清说的是三个不同的所指——**其中只有系统架构是本书的口径**（同一个实体只有一个架构，第 10 章 §10.4.4）；另两个是同一个系统的不同**关注面**。先分清在谈哪一个，再谈要不要改。

恒信那边同样：黄程说的是**组织这个实体的架构**；万辉、玉杰说的是**本系统架构的两个关注面**；小白要的是**接口那一张视图**。四问答下来，每个人都说清了自己指的那一件；先谈哪一个、后谈哪一个，也就排得出来了。

**没有人先把”在谈哪一个”问出来**

### 1.8.3 词汇病：最常见的几种”没拿准”

四问法为什么必要？因为”见过”的样子是有规律的，最常见的那几种，合起来叫”词汇病”：

| 病 | 症状 | 例子 |
| --- | --- | --- |
| 病一 说不出定义 | 用例子代替定义，问不到判据 | “评审就是开会过一遍” |
| 病二 同词异义 | 同一个词，各指各的概念 | 章首的”架构” |
| 病三 翻译伪朋友 | 英文原词不同，中文翻译形近 | 评审 / 验证 / 确认 |

- 病一，说不出定义，只能举例。例子是插图，不是定义：定义回答”它是什么”，例子只回答”它长这样”。要判断”一次不走流程的聊天算不算评审”，例子答不了，判据才答得了。治病的药是给定义，而且定义要带可检验的判据——这背后是本书的**判据主义**。“评审”的定义是”为达成既定目标，对客体的适宜性、充分性或有效性所作的确定”（ISO 9000:2026 §3.11.2），判据便是：有没有得出结论？有没有留下记录？

- 病二，同词异义，各指各的。同一个能指（词），挂在不同的人身上，就是不同的所指。这正是章首”架构”吵架的机制。两个来源说的是同一件事的不同侧面：索绪尔的能指与所指、弗雷格的符号／涵义／指称。词还会漂移：“架构”从建筑学漂到软件、组织、数据，每次借用叠一个新所指。漂移无害，不对齐才可怕。治病的药是把词和概念分开——不谈”架构是什么意思”，改谈”你说的架构，指称的是什么”。
- 病三，翻译伪朋友。几个英文原词完全不同，中文翻译后听起来像一类。本书第 8 章的主角评审（review）、验证（verification）、确认（validation），在 ISO 9000 里的定义动作词不同：评审用”确定”（determination，§3.11.1）定义，验证与确认用的是”证实”（confirmation）；“证实”没有自己的条款号，在这两条定义句里以句首词出现，另有 §3.11.7 的 metrological confirmation，那是计量领域的另一个词。中文都带”评/验/认”像近亲，其实不是。治病的药是回英文原词、查词根、看定义的动作词。

（冰链和恒信那两场会，病二占了大头；这本书每章章首的”误解现场”，大多也落在这几种词汇病里。比词更深的还有”乱用”，概念本身没认错，却用在不该用的场景，那是”往哪去”没答全，见 §1.8.4。）

### 1.8.4 查全：概念档案

这四问好记、也够用，但它记住的是四个方向，不是一张可以逐项打勾的清单。要查全，本书另备一样东西，叫**概念档案**：书末附录 B 给

每张核心概念配一张档案卡，把四问展开成固定的几栏，一栏一栏填过去，漏在哪一栏一目了然。全书约 45 张核心档案，其中 33 张完整版收在书末附录 B，其余散见各章。恒信那场会如果按这张档案过一遍，“架构”的几栏里一定有空着的——空着的那栏，正是他们卡了三个月的地方。

一条使用规则，必须遵守：档案不是每一栏都必须填，薄栏可省，但不可被跳过。跳过的栏，往往是误解潜伏的地方。本书各章给概念”钉死定义”，用的都是这张档案；你自己给手头的概念建档案，也照它来。

### 1.8.5 认知反转

回到章首那两场会。

其实十张脸都没有错。冰链的五张脸——产品、硬件、云端、测试、制造——各有各的立场；恒信的五张脸——**组织（另一个实体）**、系统、接口、部署、点醒——也各有各的道理（其中只有**系统架构**是本书的口径）。错的是这两场会本身：一场没有对齐概念的会议，注定是一场各说各话的表演。

如果当时有人带着四问，会议会是另一个样子：

> “等一下，我们说的’架构’**不是五件不同的事，是同一个架构的五张面孔**。先各自答四问：它是什么、不是什么、从哪来、往哪去？——**产品架构看模块边界，硬件架构看元件布局，系统**

> **架构才管服务边界和数据所有权**（组织架构则是另一个实体）。
>
> 分清在谈哪一面，再谈要不要改。”

这一章的认知反转，比”一场各说各话的表演”更深一层：你不是不懂那个词，你是还没拥有它。能把”架构”背出来、能画出架构图，都只是”见过”它；能回答它是什么、不是什么、从哪来、往哪去，才算拥有它。四问答不出后两问的人，和那两场会里的十张脸，站在同一条起跑线上。

顺手把一个更深的念头也放进你的口袋。如果连”架构”这么核心的枢纽词，都只是”见过”而非”拥有”，那你脑子里其他天天用的词呢？你真正拥有几个？位次上，你追到根了吗？深度上，你是刚把词说准（表征），看得出套路了（图式），能推演了（心智模型），还是能判断什么值得相信了（解释框架）？

两个坐标一起报，才是完整的答案。这一章的工具，把你送到第一级。

一个在造温控箱的工程世界，一个在做审批系统的设计世界——同一种能力量过来，同一个词都是五张脸。从第 2 章起，每章帮你把一个枢纽概念从”见过”变成”拥有”，这是第一级的活；上面那三级，从第 6 章起接着建。

---

## 小结

这一章不给你一个新概念，给你一套本事：概念体系能力，学习能力的杠杆。

说到底就两句话。一是挑对该先说准的那个词：先找根，再铺骨架；二是把这个词从”背得出”说到”用得上”：把话说准是那个 1，能推演、能判断是它后面添的 0。

这两步看着便宜，却最不能省：词挑错了、说歪了，后面花多大力气，都只是在把错的东西放大。

从第 2 章起，我们用这套坐标去量第一个枢纽概念——工程：工程是什么，不是什么，以及为什么”用敏捷 + CI/CD”不等于”在做软件工程”。

# 附录

## A 本章概念总图

主张：把话说准，是学习能力里回报最高的那一步

├─ 一个现场 ———————— 一个词，让两个项目停了工；词可以当场对齐，概念要花功夫拥有 §1.1

├─ 先说准哪个词 ———— 概念是一张网：一般概念 ── 核心概念（枢纽）── 根概念（源头） §1.2

│ └ 为什么先抓根：它是源头（新东西靠它挂）/ 方向（管往哪使劲）/ 能生成一片 / 立得越早越省事

├─ 概念体系能力 ———— 识别枢纽 × 织关系 × 迁移 × 辨边界；判据分三级 §1.3

│ └ 子能力：根概念能力（先抓源头，再展结构；20 分钟立骨架是练习目标）

├─ 凭什么说回报最高 ── 一条乘法律：方向 × 力臂 × 施力 × 支点；两个归零项：方向、表征 §1.4

│ ├─ 放大项：力臂、施力、支点—只放大，不指方向

§1.4.2

│ ├─ 归零项一·方向：先抓哪个词（第 0 层本体论立场 → 根概念 → 核心概念清单） §1.4.3

│ ├─ 深度轴：能力四级（表征 → 图式 → 心智模型 → 解释框架） §1.4.4

│ ├─ 归零项二·表征：表征是 1，上面三级是 0；位次轴 × 深度轴＝坐标系 §1.4.5

│ └─ 证据：压缩学习实验——枢纽优先的优势随时间扩散到整张网 §1.5

├─ 怎么练 ———— 一条循环：立枢纽→织关系→逼输出→勤校准→落实践；迁移有三道锁 §1.6

├─ 怎么找 ———— 三步识别法（收集候选→硬判据→往上挂）＋判定矩阵／演绎链 §1.7

└─ 怎么验 ———— 四问法（是什么／不是什么／从哪来／往哪去）＋术语锁定三问 §1.8

## B 本章的能力级

**第 ① 级为主**

- **表征**——把六个词说准：概念／核心概念（枢纽）／根概念／基本原理／概念体系能力／根概念能力（书末附录 B 六张卡）；用的工具是四问＋术语锁定三问
- **图式**——给出两个可识别的结构：三层包含（一般 ⊃ 核心 ⊃ 根）与能力四级（表征 ⊃ 图式 ⊃ 心智模型 ⊃ 解释框架），以及两轴合成的坐标系（§1.4.5）

- **心智模型**——给出三个能推演的机制：三步识别法＋判定矩阵／演绎链、乘法律与价值律、五环模型
- **解释框架**——本章不建此级；由第 17、18 章承担

> 能答全四问，才算拥有；只能答第一问，只是见过。记住这条次序：表征是 1，上面三级是 0——1 定成立，0 定量级。

## C 自查 · 这些说法对吗？

1. “概念是专业术语，背熟定义就算懂了。”——**错**。懂＝能答四问、能用、能迁移；背熟定义只是”见过”。
2. “概念能力＝知识渊博，懂得多能力就强。”——**错**。组织得好才关键；知识多不等于会抓枢纽。
3. “概念体系能力＝根概念能力，两个词是一回事。”——**错**。概念体系能力是全集（四构件）；根概念能力是它的子能力（方向盘：先抓承重柱，再展骨架）。
4. “杠杆的力臂越长越好，投入越多产出越大。”——**错**。乘号里有两个归零项：方向与表征。方向错了，力臂越长偏得越远；表征没拿到，上面盖多少层都白搭（§1.4.3、§1.4.5）。
5. “先学细节再搭框架，殊途同归，反正最后都学全了。”——**错**。压缩学习：先立枢纽，后面的学习越来越省力；先切细节，越来越吃力。

6. “把概念讲给外行听是浪费时间。”——**错**。转述是压缩测试，讲不清＝结构没成型，正好暴露缺口。
7. “四问法答出’它是什么’就够了。”——**错**。能答全四问才算拥有；答不出”不是什么/往哪去”＝只是见过它。
8. “核心概念一旦记住就不用再改。”——**错**。概念是共识不是真理，枢纽是起点，随时可修正（勤校准）。
9. “掌握一个领域＝记住更多信息。”——**错**。学习不是往脑子里堆信息，而是先搭骨架，再让输入各归其位。
10. “识别枢纽靠天赋，没有方法。”——**错**。有可操作的三步识别法：收集候选、硬判据筛选、往上挂；辨根概念还要过判定矩阵＋演绎链。
11. “核心概念都是科学家发明的。”——**错**。三种出生方式：创造/发现/凝结；多数是共同体长期实践”酿”出来的。
12. “枢纽就是最重要的那一个词，记住它就够。”——**错**。核心概念是骨架（通常 3~5 个）；之上还有根概念（承重柱通常 1~3 个）；还有原理一层。
13. “根概念就是最重要的概念。”——**错**。根概念是最”源头”的概念（原子/生成/不可替代/递归），未必最”重要”——质量很重要，但它不是根。

14. “概念只有节点，没有边。”——**错**。概念是节点、原理是边；核心原理之上还有基本原理（§1.2.3），掌握原理，概念自动串联。
15. “先把图式建起来更要紧，表征差一点没关系。”——**错**。表征是 1，上面三级是 0：1 定成立，0 定量级；1 没拿到，上面盖多少层都白搭（§1.4.5）。
16. “能答全四问，就说明我这一级已经修满了。”——**错**。四问是第一级（表征）用的工具；上面还有能推演的心智模型、能判断的解释框架（§1.4.4）。答全四问是**入场**，不是到顶。

## D 练习

1. 用四问法量你工作里天天用的一个词（架构/需求/质量/评审……）——哪一问最薄？为什么最薄的那一问，恰恰是你最容易出错的地方？量完再给它报个坐标：位次在哪一层、能力练到第几级？
2. 用三步识别法找出你领域的核心概念：收集候选（目录/术语表/专家/AI）、硬判据筛选（词频/被引/前置数/反事实）——“如果没有它，这个领域还成立吗？”再试着用判定矩阵辨出其中哪几个是根概念。
3. 把某个概念讲给一个完全外行的人，三五句话讲清楚。卡在哪一步？暴露了什么缺口？
4. 用五环模型给最近一次学习做个体检：你停在第几环？下一环是什么？为什么卡在那？

5. 用四问法重开章首那两场会：十张脸对”架构”各持所指，哪些是同一个所指、哪些不是？怎么让工程团队和设计团队先各自对齐，再谈别的？
6. 你领域里哪个概念是”凝结”出来的？它的共同体是谁？哪个是”发现”或”创造”的？
7. 你有没有”曾经坚信、后来修正”的概念？是五环哪一环让你改的？
8. 用乘法律给你的学习做个诊断：方向（根概念能力）、力臂（概念体系能力）、施力（学习投入）、支点（实践场景）各缺什么？两个归零项——方向与表征——哪一项你更没底？
9. 拿你最近学到的一个概念做一次**迁移测试**：换一个行业、换一个粒度，这条判据还成立吗？认出”同一个结构换了张脸”了吗？（§1.4.4 四级里，这是从图式走向心智模型的那一步。）

（答案要点见书末附录，与训练教材题卡同源）

---

## E 参考文献

**标准** - R-01 ISO 1087:2019 —— 概念定义依据（§3.2.7：“由特性的独特组合而形成的知识单元”） - R-103 ISO 9000:2026（第 5 版）—— 评审/验证/确认（§3.11.2 / §3.11.12 / §3.11.14，伪朋友辨析） - R-04 ISO/IEC/IEEE 42010:2022 —— 架构定义（§1.8.2 演示、概念档案·架构） - R-08 ISO 10007:2017 —— baseline 由军品工程实践固化（凝结三方式举例）

**经典文献** - R-24 Xiangjuan Ren 等《Compressive learning scaffolds higher-order network structure to enhance human knowledge acquisition》（Nature Communications, 2026, DOI: 10.1038/s41467-026-75843-7）—— 压缩学习实验 - R-25 Bloom 教育目标分类学（Anderson & Krathwohl 修订版, 2001）—— 五环掌握线”会用才算掌握” - R-30 Katz《Skills of an Effective Administrator》(1955) —— 概念技能（“概念体系能力”的管理学源头） - R-26 Roediger & Karpicke (2006) —— 测试效应 - R-27 Posner 等 (1982) —— 概念转变 - R-28 Whitehead《The Aims of Education》(1929) —— 惰性知识 - R-29 Lave & Wenger《Situated Learning》(1991) —— 情境学习 - R-61 Miller《The Magical Number Seven, Plus or Minus Two》(Psychological Review, 1956) —— 工作记忆容量的经典文献（“7±2”的来路）；**本章”4 到 7 个”这一口径与压缩单位”图式”的说法，依据均在 R-106（第 38 讲）**；原著**库外·不可核** - R-22 索绪尔《普通语言学教程》—— 符号=能指+所指；语言（langue）与言语（parole） - R-23 弗雷格《论涵义与指称》(1892) —— 符号→涵义→指称 - R-48 维特根斯坦《哲学研究》(1953) —— 意义即用法（假懂 vs 真懂） - R-49 库恩《科学革命的结构》(1962) —— 范式转换（共识 ≠ 真理） - R-50 Kahneman《思考，快与慢》(2011) —— 系统一/系统二（“找枢纽”的读者参照）

- R-106 万维钢《现代思维工具 100 讲》（得到 App 课程，第 38—47 讲）—— 工作记忆带宽与压缩单位、能力四级（表征→图式→心智模型→解释框架）、参与度分级、迁移三道锁、默会知识边界 - 费曼技巧 —— 转述=压缩测试的机制（无正式书目，沿用研习笔记 00 的标注）

# 第 2 章 工程：我们到底在做什么

## 2.1 引言

这一章先用一场季度会把问题摆出来，再把这一章要回答的问题说清楚，最后给出一副整章的骨架。看完这三段，这一章后面在争什么、凭什么说”有流程、有工具”还不算在做工程，也就清楚了。一路答下来，一条能当场用的判据就落到你手上：换个人、换团队，同样的需求与约束，还能不能同等交付。

### 2.1.1 误解现场

恒信集团 IT 部的季度会，上半年最后一天。

架构师万辉站在投影前，语气里有压抑不住的兴奋：

> “各位，我们敏捷转型成功了。双周迭代、CI/CD 全绿、自动化测试覆盖率 80%。按 IEEE 610.12 那三个词——systematic、disciplined、quantifiable——我们一条不缺。我们已经是工程化团队了。”

会议室里响起稀稀拉拉的掌声。这个团队过去一年上了 CI/CD、拆了微服务、引入了敏捷站会，每一步都踩在教科书的说法上。

产品开发经理满海接过话头：“系统上线了，指标也全绿——我们交付的东西跑起来了，工程就算成了。”

运维玉杰低头看了一眼手机。监控上，月末高峰的曲线又拱了起来：今天是上半年的最后一天，也是本月最后一个工作日。他想起上个月那场事故，想说什么，又忍住了。

只有需求分析师徐漾，在掌声落下去之后，低声问了一句：

“那……上个月上线那次事故，是为什么？”

万辉愣了一下：“那是人手不够，临时赶工。跟工程化没关系。”

徐漾张了张嘴，没再问。

如果你坐在这间会议室里，你大概也会点头。大多数人对”工程”这个词的直觉，和万辉一模一样：有流程、有工具、有测试、有敏捷，就是在做工程。而万辉那句”事故跟工程化没关系”，恰恰是这一章要推翻的东西。上个月的事故，几乎可以肯定是”不是工程”的证据，而不是”工程之外”的意外。

这一章要做的，是先回答一个最基础的问题：我们到底在做什么？是工程，还是包装成工程的手艺？这两个词的区别，决定你交付的东西能不能”换个人也交得出来”。

---

## 2.1.2 这一章要回答什么

读完本章，你应该能回答：

1. 一句定义能不能把”工程”说准？定义里的每一样各管什么？为什么约束不是麻烦，而是定义的一部分？
2. 不同年代、不同领域的权威定义，凭什么走到同一个内核上？
3. 换个人、换团队，还能不能同等交付？这条判据测的是能力高低，还是可复制性？

4. 为什么工程没有单一对立面？科学、手艺、艺术各自站在哪个维度上？
5. 工程不只是方法论。“敏捷 + CI/CD + 测试”占了其中哪几层？缺的会出什么事，为什么价值与伦理那一层最致命？
6. 系统化和算法化是一回事吗？自动化程度越高，是越接近工程，还是越远离工程？
7. 系统工程和软件工程是什么关系？工程系统和技术系统呢？“系统上线了”算不算工程成功？（“工程师”头衔为什么不等于在做工程，第 9 章专章回答。）

---

### 2.1.3 这一章的骨架

这一章要立一个主张。工程是把交付从人身上挪到方法上的一次转移，挪得走的是方法，挪不走的是判断。

> **工程 = 把”做成一次”变成”谁来做都成”。**

这件事分两半。挪得走的那一半是原理、流程、工具、标准，也就是工程科学、方法论、工具与实践。而工程之所以是工程，靠的是工程科学、方法论、工具与实践、价值与伦理这四层同时在场。挪不走的这一半是每一步上的判断，价值与伦理守的就是这条挪不走的线，所以工程不能被算法替掉。

这一章要说清的是这么几件。先给”工程”一个定义，再给一条能当场用的判据：换个人、换团队，还能不能同等交付。然后把工程和最容易混的那些东西分开：工程不等于方法论，缺了哪一层，就有一样东西重新落回某个人身上；系统化也不等于算法化，挪不走的那部分就是判断。最后交代验收看什么：工程系统包含技术系统，成不成，由”被用起来”说了算。

用第 1 章的话说，万辉只是”见过”工程：他背得出那三个形容词，却答不出”它不是什么”。有流程、有工具、有敏捷，唯独没问过：这套做法离了人，还成不成立。

§2.2 钉”工程”这个词本身，§2.3 把它和容易混的东西分开，§2.4 落到怎么用——从造部件到造整体，从上线到被用起来。

---

## 2.2 工程是什么

> 这一节把”工程”看清：先立一句定义，再把定义里的每一样拆开看，看不同年代的权威机构怎么走到同一个内核上，最后把全章反复使用的那条判据交给你。

### 2.2.1 先给工程一个定义

“工程”这个词用得太滥：信息工程、管理工程、社会工程，什么都能挂上”工程”二字。但在系统工程、软件工程、硬件工程里，它有一个很硬的共同内核，可以用一句话说完：

> **工程 = 科学原理 × 判断创造，在约束下，用系统方法，造出有用且安全的东西。**

> 术语注：这句定义是本书对 ECPD、ABET、INCOSE、IEEE 610.12 几份权威定义的**凝练**，不是任何标准的原文，原定义见 §2.2.3；本书的判据与命名同样是作者主张，欢迎检验、欢迎校准。
> **先说清”挪不走的”指什么**：指的是**判断**，每一步取舍上都要用的那个动作，从工程科学到价值与伦理层层都有。方法能把工程科学、方法论、工具与实践挪走，这个动作始终留在人身上。

这句定义里，每一样都各司其职。注意它的结构：科学原理 × 判断创造 是”料”，约束 是”框”，系统方法 是”路”，有用且安全 是”果”。缺了”料”是空谈，缺了”框”是失控，缺了”路”是碰运气，缺了”果”是自娱自乐。后面每一节，都是它们的展开。

### 2.2.2 定义里的每一样都不能少

① 科学原理 × 判断创造——地基。工程是应用原理，不是发现原理。物理定律、数学模型、算法理论，是工程的依托。没有力学定律，土木工程师只是个会搭积木的人；没有算法理论，软件工程师只是在瞎调参数。这一要素回答了”为什么这样能行”。

那个”×“不是装饰：判断创造是把原理用活的动作，包括选哪个模型、怎么取舍、何时偏离标准做法。它不单独占位，却贯穿每一步，下一项”约束”里会再见到它。

② 约束——精髓。成本、工期、资源、风险、法规、物理极限。注意一个反直觉的事实：约束不是工程的附加项，是定义的一部分。1941 年 ECPD 那份最早的工程定义里，就同时写着”运行经济性”和”生命财产安全”两条。一个没有约束的”创造”不叫工程，叫发明或艺术。

没有约束，就没有判断的空间。工程里的每一个决策，本质都是”在约束下做取舍”：成本有限，所以要权衡；工期有限，所以要排序；安全有红线，所以不能让。**约束越多，判断越值钱**。反过来也要防过头：无意义的限制、纯官僚式的流程不产生判断价值，那只是限制，不是约束。

③ 系统方法——过程。需求 → 设计 → 实现 → 验证 → 维护。它保证工作可计划、可度量、可复现。可计划，才知道每一步花多少资源；可度量，才知道每一步做没做对；可复现，才谈得上”换个人也能交付”。这几样缺一样，方法就只剩墙上一张流程图。

这也是本章主张里”挪得走”的那一半：方法可以写成文档、变成流程、交给工具，让别人接住。

④ 有用且安全的人造物——目的。一切为了交付能用的东西。工程不追求”有意思”，追求”有用且安全”。再看”人造物”这个词：工程交付的是人造的东西，不是自然的东西；光”有个想法”不算数。想法是发明或艺术的起点，只有被约束、被系统方法造出来、对人有用的，才是工程的产品。这也是为什么工程可以复现、可以检验，而灵感不能。

空气动力学之父冯·卡门（Theodore von Kármán）有一句话常被引来，说工程和科学的分工：

> **“科学家研究现存的世界，工程师创造从未存在的世界。”**
>
> Scientists study the world as it is; engineers create the world that never has been.

这里还有一条更深的推论要交代：判断不能写全成规则，因为判断就是在约束下做取舍，而约束一变，写好的规则就失效。把”哪种工况要降额”写进对照表，换了供应商、遇到新工况，表就罩不住了。所以”完全可复现”不是工程的极致：纯可复现是算法（见 §2.3.4），工程是”可复现的方法 × 不可复现的判断”。**方法是挪得走的，判断挪不走**，它只能长在人身上。

定义里的每一样都不是论文里的装饰，是可以逐条打勾的诊断清单。恒信怎么打这个勾，见案例进展①。

### 2.2.3 权威定义：不同年代，同一个内核

“工程”有多个国际权威机构的正式定义，年代不同、领域不同，却走到了同一个内核上。

ECPD（美国工程师职业发展委员会，ABET 前身，1941）：> 创造性地应用科学原理，设计或开发结构、机器、装置、制造过程……着眼于预期功能、运行经济性以及对生命财产的安全。

ABET（1979，现行认证标准）：> 将通过学习、经验和实践获得的数学与自然科学知识，运用判断力加以应用，以经济地利用自然的物质与力量，造福人类。

IEEE 610.12-1990（IEEE 标准术语，词条是软件工程）：> The application of a systematic, disciplined, quantifiable

approach to the development, operation, and maintenance of software; that is, the application of engineering to software.（把系统化的、规范的、可度量的方法应用于软件的开发、运行与维护——也就是把工程应用于软件。）

INCOSE（国际系统工程学会，2019）：> 系统工程是一种跨学科、整合性的方法，运用系统原理以及科学、技术和管理方法，使工程系统的成功实现、使用与退役成为可能。

把 ECPD、ABET、INCOSE 三家的说法并排看，共同的东西就浮出来了：科学原理是知识基础；判断创造是把原理用活的动作；约束权衡是取舍的范围；造福目的是”有用且安全的人造物”；系统方法保证这件事可复现。这不是巧合，是”工程”这个概念的稳定内核。IEEE 610.12 那条说的是软件工程，只从软件这一侧贡献了”系统方法”这一面。

> 出处：以上四家定义的文字转自报告库 05 号 v1.2 §5。其中 **INCOSE 那一条的英文逐字可在库内正本核**：`ISO/IEC/IEEE 15288:2023` §3.50 收了这一定义，正本逐字作 transdisciplinary and integrative approach to enable the successful realization, use, and retirement of engineered systems using systems principles and concepts and scientific, technological and management methods，来源条注明 `INCOSE-TP-2020-002-06`。**正本另有中**

**译**，作”一种跨学科、综合性的途径……使工程化系统得以成功实现、使用和退役”；本节所引的中译取 INCOSE 2019 的转述版。**ECPD 1941 与 ABET 1979 两条库内无正本**（IEEE 610.12-1990 同），故一律带年代引用，不写成”标准说”。

术语注：IEEE 610.12 定义的是**软件工程**，不是”工程”本身——它的原话是”把工程应用于软件”（the application of engineering to software），把”工程”当作已知概念引用。严谨的表述应该是：“据 IEEE 610.12 对软件工程的定义，工程化的过程表现为……”（它的三个形容词 systematic / disciplined / quantifiable，即”工程化的过程特征”，见 §2.3.4）。这个出处的分辨，正是第 1 章”翻译伪朋友”式的严谨：**同一个词，出处不同，所指不同**。

**概念卡片 · 工程**

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 科学原理 × 判断创造，在约束下，用系统方法，造出有用且安全的东西 |
| **它不是什么** | ⑥⑦ | 对立：手艺（方法论维度）· 科学（认识论维度）· 艺术（目标维度）——工程没有单一对立面；辨析：工程不等于方法论（方法论只是其中一层）、工程不等于科学（创造对发现） |
| **它从哪来** | ②③⑨ | 工业时代大规模建造中”凝结”出来的实践——铁路、桥梁、工厂撑不住单靠个人手艺，共同体把造东西的经验压成可教可查的专业；演化：1941 ECPD 首次成文定义 → 1979 ABET → 2019 INCOSE，三家几十年间走到同一张图纸 |
| **它到哪去** | ④⑤⑧ | 内容：如何把经验变成可被检验的通约体系；核心概念：原型、试验台、测试、迭代、风险、质量、可靠性、可维护性、可制造性、可运维性、可退役性；方法：V 模型、敏捷、精益、DevOps、数字孪生；实践：从概念到退役的全生命周期活动 |

顺着”判断创造”这半个要素再往下走一步。ABET 还给 engineering design（工程设计）下过定义：为系统、部件或流程设计出满足需要的形态，一个**想出办法**（devising）、**反复定夺**（iterative decision-making）、**创造性、开放式**的过程。

第 11 章会把这句话展开成一条三动词链：构思打开可能性，定义终结漂浮，设计转换到底。“判断创造”里的”创造”，只是第一棒。那里还有一处要留意：这里说的”设计”是广义的，构思、定义都算在里头；等构思、定义、设计并排站的时候，“设计”要退到下游的狭义上，而三者的先后是逻辑上的蕴含，不是日历上的三步。

至于”创造”算不算”创新”，那是另一把尺子：够新又成了，才追认创新（第 11 章）。

> 术语注：这句对 engineering design 的表述，是本书对 ABET 工程设计定义的凝练，不是 ABET 原文逐字（库内二手见报告库 12 号 L115／L616、16 号 L235）。

### 2.2.4 交付能不能离开某个人

定义里的每一样、权威机构的每一份定义，最后都指向同一条可检验的判据。这条判据，本书叫它**试金石**，全章反复使用：

> **换个人、换团队，用同样的需求和约束，能不能交付质量相当的成果？能，是工程；不能，那是手艺——哪怕它碰巧也用了代码、图纸或电路。**

试金石可以在这些场合直接使用：

- 团队交接时：“如果我走了，这套流程还能跑吗？”答不上来，就是手艺；
- 招聘时：判断一个人带来的是”方法”还是”经验”，方法可传播，经验只在本人；
- 复盘时：这次交付能复现吗？复现不了，就是一次性的手艺成功，不是工程能力。

试金石的妙处在于它测的不是能力高低，而是可复制性，而这正是工程与手艺的分野。它也是第 1 章反事实检验的落地：去掉那个人，交付还成立吗？

## 案例进展①｜会后打勾：恒信缺了哪一样

季度会散会后，几个人还留在会议室里。徐漾把定义里的那几样写在了白板上。

“我按 §2.2.2 那几样，挨个打一遍勾。”他说，“科学原理——他们的’为什么’有依据吗？容量评估是工程科学，砍掉它等于砍掉第一层。约束——他们没有识别’月末高峰’这个性能约束，等于没有约束可守。系统方法——CI/CD 在。有用且安全的人造物——系统上线了，可新流程没人用。”

运维玉杰把一摞监控曲线摊在桌上，指着每月末冲顶的那几段：“这条我最有发言权——曲线每个月最后几天都会冲顶，我报过好几轮，可没人把它当成约束来守。头一年我还较真，把报警写成整改建议，追着立项；万辉说’先记着，忙完这阵’，忙完就没下文了。后来

我就只发数据，不再追了。我那时候以为，把监控做好、把数据准时发出去，就算尽到了运维的责任。”

徐漾问他：“你报上去的是什么?”玉杰一愣：“报警记录。”

“报警只是数据。”徐漾说，“约束要有人识别、有人命名、有人立项，写成一条有人守的规则才算数——按这条判据，你递了三年数据，约束一次也没递上去。”

玉杰想了好一会儿，点头：“所以我报的是数据，不是约束。我今天才知道，一条没人管的数据，等于没提。”

业务方的黄程也接了茬：“没人用我最有体会——法务那边问电子签怎么走，财务那边问台账接不接，业务部嫌系统绕。上线是上线了，用起来是另一回事。你们说的技术系统，我承认它做完了；可我这边的账是：没人用，就等于没交付。”

---

## 2.3 工程不是什么

> 这一节把”工程”和它容易混的东西分开：科学、手艺、方法论、机械化，每个挨着的词都变成一句可检验的判据。

### 2.3.1 工程没有单一对立面

一个反直觉的答案：**“工程”没有单一对立面**，它的对立面取决于你站在哪个维度看。维度不同，对手就不同：

| 维度 | 对立面 | 问什么 |
| --- | --- | --- |
| 认识论 | 科学 | 发现世界 vs 创造世界 |
| 方法论 | 手艺 | 可复现 vs 依赖个人 |
| 目标 | 艺术 | 满足用户 vs 表达自我 |

认识论维度：工程 vs 科学。科学问”世界是什么”，工程问”应该造出什么”。科学家的失败可贵（推翻假说），工程师的失败是灾难（垮了就垮了）。这一对最容易被混淆，因为”科学”听起来比”工程”高级。但工程不是科学的应用附录，它们是两种目标：一个求真，一个求有用。

目标维度：工程 vs 艺术。艺术为表达自我，功能让位于审美；工程为满足用户，一切决策向可用、可靠、可维护汇报。两者的分界线不是”好不好看”，而是”为谁服务”。梵高的画不供人”可用”，但工程师造的桥，是要被人走的。**好不好看是加分项，能不能用是及格线**。

理解”工程没有单一对立面”，就不会再用”有没有用代码”来判断”是不是工程”。

### 2.3.2 不是手艺：靠人，还是靠方法

这一对最实用，因为它直接给出”你在不在做工程”的判断：§2.2.4 的试金石，在这里第一次用起来。

手艺靠个人经验与技巧，结果依赖人；工程靠流程、标准、文档、评审、测试，换个人也能交付。注意：手艺不是贬义词。一个经验丰富的老工程师本身就是最值钱的资产。但团队的能力和工程的能力是两回事。手艺依赖具体的人，工程依赖可复现的方法。你团队里最值钱的那个人走了，交付还能不能维持？能，你有工程；不能，你有手艺。

## 案例进展②｜水忠的手艺，恒信的流程

冰链科技的嵌入式硬件工程师水忠，做了几十年温控产品。他的车间里没有一份完整的测试方案，但他”心里有数”，哪个批次容易出问题、哪种工况下要降额、哪家供应商的电容不可靠，全在他的脑子里。

去年水忠请了两周假，温控箱的一个批次出了问题，产线愣是没人能定位——因为”知道该查哪里”的只有水忠一个人。水忠回来，十分钟找出问题。

这就是手艺：结果依赖具体的人。水忠在，质量在；水忠不在，质量跟着走。

制造侧的逸人看得更直接。他管产线，最怕的就是这种”人一走、线就停”：“试金石这一问，我们车间天天在答——这条线的工艺参数，是写在作业指导书里，还是只在老师傅手上？写在纸上的，换班照样跑；只在人手上的，他一休假我就得陪着熬夜。这跟谁手艺高没关系，是本事存在谁身上的问题。”

现在把镜头切回恒信。万辉的团队上了 CI/CD、有了测试，这算工程了吗？还不一定。如果这套 CI/CD 跑起来靠的是张三调试的三条隐藏命令、李四记得的两个环境变量、王五没写进文档的部署顺序，那它仍然是手艺，只是手艺的载体从”脑子”换成了”脚本”。工具不会自动把人从手艺变成工程，可复现才会。

工程化不是要把水忠的手艺废掉，而是要把他的判断显性化成可教的规则、可查的文档、可检验的标准——把个人能力变成组织能力。这才是试金石的意思：它测的不是”你牛不牛”，是”你不在的时候，团队还牛不牛”。

### 2.3.3 不是方法论：工程比它大

先分三个容易混的词：**方法（method）**是”一条具体做法”，比如”排序用快速排序”；**方法论（methodology）**是”关于怎么组织工作、怎么选方法的系统学说”——瀑布模型、敏捷、CMMI；**工程（engineering）**比前两个都大——方法论只是它的一部分。工程不等于方法论。

工程是一个完整的人类实践，从上往下立着这么几层：

**第四层 价值与伦理　为了什么？安全 / 可靠 / 负责　← 最容易被忽略的一层**

**第三层 工具与实践　用什么做？CAD / 编译器 / 测试链**

**第二层 方法论　　怎么组织？需求 → 设计 → 实现 → 验证 → 维护**

**第一层 工程科学　　依据什么？原理 / 模型 / 定律**

每一层都是一道把交付从人身上挪走的关。缺了哪一层，就有一样东西重新落回某个人身上：

- 缺工程科学：没人能回答”为什么这样能行”，只能靠某位老师傅的直觉，别人学不走；
- 缺方法论：做法只能师徒相传，人走了做法跟着走；
- 缺工具与实践：方法论悬在半空，落地全靠个人技巧；
- 缺价值与伦理：压力面前松不松手，全靠某个人的良心。

恒信那个”先上再说”的接口就是现成的样子：容量评估被砍，缺的是工程科学；变更没有控制、没有基线，“受控”这一项不及格（§2.3.4）；为进度让掉可靠性还不留记录，缺的是价值与伦理。缺的两层加上这一项，事故就来了。前三层缺了是”不专业”，第四层缺了是”会出事”。

前几层挪走的是”本事”，价值与伦理守的是”底线”。这里要跟§2.2.1 说过的”判断”分开放：判断是每一步取舍上的动作，四层里

层层都有；底线是判断不可越过的那条线（安全、可靠、负责）。一个是动作，一个是界限。正因为判断始终留在人身上，那条线才需要有人守着。

底线挪不到方法上，但这不等于只能靠良心，它可以用”让它看得见”来管住：把每一次妥协摆到台面上。下一节那几条判据，就是从这里来的。

回头看 §2.2.2 那句定义：科学原理 × 判断创造是”料”，其中科学原理对应工程科学那一层；系统方法是”路”，展开成方法论和工具与实践；有用且安全是”果”，正是价值与伦理守住的底线；约束是”框”。约束和判断这两样，都不单独占一层：约束是每一层做取舍时的范围，判断是每一层做取舍都要用的动作。定义说的是工程由什么构成，层次说的是它怎么立起来，两套说法说的是同一件事。

敏捷不等于软件工程。回到章首万辉那句话的意思：敏捷、CI/CD、测试齐了，就是在做软件工程。现在可以精准地指出它错在哪：敏捷、CI/CD、测试，只落在方法论和工具与实践这两层上。

而工程还有两层没兑现。先看工程科学：你的”为什么”有依据吗？性能压测的结论能复现吗？再看价值与伦理：赶进度时，安全需求会让位吗？万辉的团队，方法论和工具与实践这两层是新的；工程科学、价值与伦理那两层是旧的。只讲方法论不讲伦理，那不叫工程，叫”有组织的赌博”。

事故就出在这一层。为什么说价值与伦理最致命？因为每一次重大工程事故背后，几乎都不是方法论出错，而是这一层失守：

- 泰坦尼克号：号称”不沉之船”，救生艇数量按”不会沉”的假设配，为成本和进度牺牲了安全裕量；
- 挑战者号：O 形环在低温下会失效，发射前工程师就提出过反对，管理层也知情。但为了发射窗口和压力，管理者把风险”合理化”了。技术和检查都在场，缺的只有这一层。
- 波音 737 MAX：MCAS 系统（自动防失速）的设计缺陷，在进度与市场的压力下没有被认真对待，安全需求让位。（自动化怎样压缩了飞行员夺回控制权的余地，见 §2.3.4。）

> 出处：这三起事故的经过据公开报道与研究文献整理，库内没有一手调查报告。737 MAX 那两起坠毁的经过，可参 **JATR《Boeing 737 MAX Flight Control System》（2019 年 10 月提交 FAA）**、**印尼 KNKT 狮航 JT610 最终调查报告（2019）**与 **NTSB 安全建议（2019-09-26）**——均为**库外来源·不可核**。

这些事故的共同点：不是”不会做”，是”做的时候没守住”。怎么判断一次妥协是失守，而不是客观原因？问三句：利益相关方知不知情？有没有评审？有没有记录？客观原因不会专门留记录，失守恰恰发生在”没有记录”的地方。这是本章第一样能当场打勾的东西。

## 案例进展③｜有工具，为什么还不是工程

上个月那场事故，第一个被惊醒的是运维玉杰。

月末高峰，监控曲线冲顶，报警响了——数据库连接池耗尽，合同审批系统宕机。玉杰第一时间扩容、重启、回滚，把系统救回来，花了四十分钟。法务和财务几百个审批卡住。

他翻到那个新上线的接口，心里一沉：“这个接口，我从没见过它的容量基线——工程科学这一层，我们等于没做。”

徐漾事后把上线日志翻出来，找写接口的开发小白核对。小白盯着那行提交记录，没有回避：“评估在计划里排过，可月底要上，万辉说先上再说。我当时也想着，上线后再补来得及。”她顿了顿，又补了一句：“结果，没来得及。方法论那一层我们没缺，缺的是压住进度的那一层。”

“你们没有容量评估、没有压测，我连它正常水位是多少都不知道，”玉杰说，“出了事我只能重启、回滚、赌一把——这不是救火，是撞运气。”

徐漾把根因钉死：“按 §2.3.3 说的那几条对一遍：利益相关方知不知情？没有。有没有评审？没有。有没有记录？冲刺计划上写过一行，后来又划掉了。一条不占。这就是价值与伦理这一层失守，不是客观原因。”

他接着说：“敏捷、CI/CD、覆盖率 80%，这些都在。可一个团队工程化没有，不看它的流程有多完整，看它在压力面前会不会为质量让路。这个接口是万辉在站会上说了一句’先上再说’就上的——那句话的分量，比任何一层都重。”

万辉那句”跟工程化没关系”，正好说反了。事故恰恰证明：这个团队有工程的方法论和工具，还没有工程的价值与伦理这一层。

### 2.3.4 不是机械化：系统化不等于算法化

所谓工程化，说的是工程做起来是什么样。上一节讲的是工程由哪几层构成；这一节把它落到”怎么做”上。IEEE 610.12 的软件工程定义里用了三个形容词，给”工程化”画了像：

- **systematic（系统化）**——做事有章法；
- **disciplined（受控 · 有纪律）**——守规矩、受约束；
- **quantifiable（可度量）**——有证据、可检验。

这三样里，本节先说系统化。

> 出处提醒：这三个形容词出自 IEEE 610.12-1990 对**软件工程**的定义，所以本节讲的是”工程化的过程表现”，不是”工程的定义”。严谨的说法应该是：据 IEEE 610.12 对软件工程的定义，工程化的过程表现为……。**本书把这几样过程特征当尺子用，但它不是工程本身的定义**，工程本身的定义见 §2.2.3 的 ECPD、ABET、INCOSE 三家（IEEE 610.12 只答软件工程这一侧）。这条分辨是**库外来源**（IEEE 610.12-1990 库内无正本），文字转自报告库 05 号 v1.2 §5.1。

但”系统化”这个词，在中文里最容易翻乱：systematic（系统化）和 systemic（系统性）两个同源却不同的词，中文都译成”系统的”；而 algorithmic（算法化，也就是”机械化”）又常被当成”系统化”的本义，以为系统化就是把做法固化成算法、交给机器自动跑。它们各回答各的问题：

| 词 | 含义 | 回答的问题 |
| --- | --- | --- |
| **systematic（系统化）** | 做事有章法：步骤清晰、方法规范、可检查可复现 | 怎么做事 |
| systemic（系统性）／systems thinking（系统思维） | 看问题有全局：关注部分间关系、涌现行为、与环境交互 | 怎么看问题 |
| algorithmic（算法化） | 确定性执行：输入相同、输出必相同，无需判断 | 能否自动执行 |

出处：前两行转自报告库 05 号 v1.2 §6，第三行转自报告库 06 号 v5.8 §2.6（该行不在 05 号）。三处与原报告的出入须说明。一处是前两行的”含义”格系改写，原报告作”过程有条理——步骤清晰、方法规范、可检查可复现”与”视角看整体——关注部分间关系、涌现行为、与环境交互”，后者语出同报告 §6 末句。另一处是原报告两行所带的括注，第一行是”（IEEE 定义用词）“、第二行是”（INCOSE 定义的落脚点）“，此处没有引。**第三处是”可重复”的释义**。05 号 §6 释作”换人执行结果仍稳定”，本书按附录 C 的两条分工改。**成色·可重复＝过程本身能重复执行**（本节表、§2.3.3、§2.4.5 三处同口径）；而”换执行者仍稳定”归系统化自问那一问。

systematic 是”做事有章法”，systemic 是”看问题有全局”。好的工程两者都要：用系统思维发现问题，用系统化方法解决问题。顺带统一说法：本章说的”机械化”，指的就是 algorithmic（算法化）。

举个 systemic 的例子：恒信的合同审批系统，如果只用系统化的眼光盯住”审批流跑通了”，就会漏掉”法务、财务、业务三家的数据定义对不上”，而后者要系统性地看整体才看得见。系统化解决”这一步对不对”，系统性解决”整个系统对不对”。本书第 10、11、12 章（架构）就是 systemic 的主场。

那系统化（systematic）本身到底是什么？拆到底，它有这些可操作的特征：

- 有结构：分解为清晰步骤与阶段；
- 有章法（即”有方法”，含判断）：每步有标准与检查点，但不是机械执行，留判断的位置；
- 可重复：过程本身能重复执行；
- 全覆盖：从需求到退役，无遗漏环节。

反面是即兴发挥：无结构、无章法、不可复现、有盲区，想到哪做到哪，同一问题每次处理都不一样。

怎么判断一个流程是系统化还是装饰？问三句：每一步有明确的输入和输出吗？每一步有可检验的检查点吗？换个执行者，结果还稳定吗？都答”是”，才是系统化；答不上来，那只是一张流程图。

拿这几句去照同一个”给软件加新功能”的例子。非系统化的做法是：想起需求直接开写，写一半发现要改数据库，顺手改了表结构，没测旧功能，上线后报表挂了，连夜回滚，三个月后没人知道为什么

多了个字段。系统化的做法是：需求写清楚并评审，评估影响范围，设计评审，编码加单元测试，集成测试回归旧功能，预发布验证，上线，记录变更。

前者每一步都没有输入输出、没有检查点、换个执行者就抓瞎；后者每一步都有明确的输入、输出和检查点。同样是加一个功能，一个越加越乱，一个越加越稳，这就是系统化和非系统化的差别。

顺带把”系统化”的误解磨掉：系统化不是流程繁琐，是可检查。系统化的本质不是”多”，而是每一步都有输入、输出、检查点；每个决策都可追溯；整个活动可复制。它管的不只是”做没做”，而是”做没做对、能不能证明做对了”。流程是系统化的载体，可检查才是系统化的目的。

（恒信那个没做容量评估就上线的接口，见案例进展③，非系统化也能上线；但只有系统化能让你回答”改了什么、为什么、影响谁、怎么验证”。）

系统化不等于算法化。“系统化”就等于”有结构、有算法、可重复、全覆盖”吗？这几样里只有”有算法”站不住，该换成”有章法（有方法，含判断）“：算法是确定性指令，输入相同输出必相同，无需判断；工程方法有章法，但每一步都留着判断的位置，材料公差、需求变化、成本权衡，没有唯一答案。

更准确地说，系统化不等于按固定程序走：它有流程，但流程里留着判断的位置。

系统化的目的，从来不是消灭不确定性。需求会变、环境会变、人会犯错，工程面对的世界从来不是确定性的。它做的是把不确定性纳

入可控范围：用检查点、评审、测试、反馈回路把这些变化管起来。敏捷就是典型：流程完整，但每一步都依赖人的判断，高度系统化，却一点谈不上算法化。

工程化的完整图景如下。系统化只是其中之一；把 systematic、disciplined、quantifiable 三个词都展开，systematic 展开成有结构、有方法、可重复、全覆盖，disciplined 展开成”受控”，quantifiable 展开成”可验证”：

| # | 方面 | 出处 | 一句话 |
|---|---|---|---|
| 1 | 有结构 | systematic | 分解为清晰步骤与阶段 |
| 2 | 有方法（含判断） | systematic | 有章法，非算法执行 |
| 3 | 可重复 | systematic | 过程本身能重复执行 |
| 4 | 全覆盖 | systematic | 从需求到退役无遗漏 |
| 5 | 受控（有纪律） | disciplined | 有基线、有变更控制 |
| 6 | 可验证（可度量） | quantifiable | 有证据、可检查、可度量 |

这些方面，是把”工程做起来是什么样”钉住的判据。回看恒信：有结构、有方法、可重复，前几样沾了边；但”受控”和”可验证”呢？上个月那个”先上再说”的接口，既没有变更控制，也没有容量验证，连”评估环节”这个维度也一起缺位。这不是清单，是关卡：缺一样，就是”差一点”，而工程里”差一点”往往就是事故。

顺着有结构、有方法、可重复、全覆盖、受控、可验证这几样再往上，有一条链：系统化 → 工程化 → 流程化 → 信息化。系统化是方法底子，加纪律与可验证是工程化，再固化是流程化，系统承载是信息化。反直觉的是：流程化不等于工程化，有流程不等于有工程。恒

信工单录入有固定流程，可录完没人核对、出错查不到记录，那仍是即兴；把”录入→核对→抽检”写成规程、设标准与检查点、留台账，换个人也能稳定交付，才是工程化。

自动化把判断推到了哪里。这些方面里没有”算法化”，有的是”有方法（含判断）“。可算法化的诱惑不会消失：它在现实里以”自动化”的面目出现。既然算法化是”确定性执行、无需判断”，而工程是”有章法的判断”，那么自动化程度越高，是越接近工程，还是越远离工程？

- 自动化不是减少判断，是把判断提纯。执行交给机器，人手上剩下的是最难的判断：异常怎么处置、边界在哪里、目标有没有变。一个全自动发布系统，人剩的不是”不用判断”，而是”判断密度最高”。把”阈值设多少、异常怎么处置”也交给机器，机器不知道”为什么这个阈值”，换个场景就抓瞎，这缺的正是工程科学。
- 自动化拿走的是人阻止事故的能力。737 MAX 的 MCAS 本为防失速而自动接管控制；两次坠毁中，飞行员都没能在有限时间内夺回控制权（事故经过的出处见 §2.3.3 末注）。这里有两道关：权限上，机器优先于人，留给人的反应时间极短；能力上，长期不接管会失去情境意识，就算把控制权交回给你，你也不知道怎么接。

把这两件事合起来，就得到自动化与工程的分野：工程要的是”错误可见”（系统化），自动化追求的是”零错误”（把错误掩盖掉），方向相反。全自动的终点是消灭判断，而消灭判断，就是消灭工程。

---

## 2.4 工程怎么用

> 这一节把前面那些判据用起来：从”造部件”到”造整体”（系统工程），从”上线”到”被用起来”（工程系统），最后回头把恒信完整地量一遍。

### 2.4.1 系统工程：不造部件，只造整体

“系统工程 = 构建系统的工程，软件工程 = 构建软件的工程”，这个方向正确，但”系统”这个词是递归的：

- 软件工程 = 构建软件系统的工程。软件本身就是一个系统：模块、接口、数据流、运行时环境、版本演进……软件工程内部同样需要系统思维；
- 系统工程 = 构建复杂系统的工程。这里的”系统”特指由多个异构部件（硬件 + 软件 + 人 + 流程 + 数据）组成的整体。系统工程的重心不在”造部件”，而在整合。

系统工程不造任何部件，它负责”把部件整合成整体”：把总需求分解分配给各部件工程、定义接口、在系统层面完成集成，再做两道判定，即系统级验证与系统级确认。这两道判定怎么分、为什么不可互推，见§2.4.4。这就是为什么叫”元工程”（meta-engineering）：它是工程之上的工程。

递归还不止一层：当软件足够大（操作系统、微服务集群），软件工程内部也会长出系统工程的子集：系统架构师、接口契约、集成测试、端到端验证。这就是”软件系统工程”，软件世界里的元工程。

把”整合”说透一点：系统工程在”接口管理”上花的精力，比任何部件工程都多。总需求分解给各部件之后，部件之间的接口契约、数据定义、时序依赖，这些”缝”才是系统工程的战场。

恒信合同审批系统里，法务系统的”合同状态”和财务系统的”合同状态”是两张字典，集成时才发现同一个词两套定义，这就是接口层面的系统工程问题，也是第 1 章”词汇病”在系统级的重演。系统工程交付的是接口契约、集成方案、系统级验证与确认的方案，这些看不见摸不着的东西，恰恰决定系统能不能转起来。

> 术语注：这些方案都是工作产品。**对工作产品不是只能评审**：评审对目标、靠判断，不要求客观证据；而**验证的对象并不限于实物**——ISO/IEC/IEEE 15288:2023 §6.4.9 明写验证的是”系统、系统元素**或人工制品**”（如系统需求、架构描述、设计描述），它的客观证据可以是检验结果、替代计算或评审文件（ISO 9000:2026 §3.11.12 注 1）。评审的结论有三档，“条件性合规”那一档要带补项、责任方与期限。**别把评审通过当成验证通过**；三个词各自对什么、在哪儿用，第 8 章展开。

### 2.4.2 涌现属性：部件全对，系统未必对

**单个部件都合格，不等于整体合格**。这叫**涌现属性（emergent behavior / emergent property）**：每个发动机都正常、每块机翼都达标，飞机未必能飞。整体行为不是部分行为的简单相加。

> 出处：这不是本书的发明。ISO/IEC/IEEE 15288:2023 §3.46 给”系统”下的定义就是这件事——正本逐字作 arrangement of parts or elements that together exhibit a stated behaviour or meaning that the individual constituents do not（各部分或各元素的安排，它们合在一起呈现出单个构成部分所不具备的既定行为或意义）。**涌现这件事，正本写在 §5.2.1**：“系统的属性（作为整体）都源自或涌现于部件或元素及其各自的属性，以及各部件之间、系统与其环境之间的关系和相互作用。”标准对”系统”的定义本身就写着涌现属性，所以它在工程里无处不在。

涌现属性为什么在工程里无处不在？因为工程把一个大问题拆成许多小问题，分给许多人做。系统工程的职责，就是守住那些”部分之和的溢出”：接口、口径、时序，它们不属于任何一个部件，却决定整体成不成立。

这里要补一句：**这正是确认侧的题眼**。验证逐条对的是部件自己的规定需求，全过了也只说明”每一条都做对了”；系统整体在预期用途上成不成立，要由确认来答。恒信集团的三个模块单看都能验收：法务、财务、业务各自都做得不错；但三个部门的数据口径对不齐、审批流跨部门断点没人管，集成那天就是事故那天。

微服务世界每天上演同一出：每个服务单独健康，但调用链、数据一致性、超时熔断没人管，上线必出事。这些”灰色地带”就是系统工程的地盘。

### 2.4.3 工程系统包含技术系统

再往上，还要分清两个常被混为一谈的”系统”，其中一个包含另一个。

> **工程系统（engineered system）**：被设计或改造的、与预期运行环境交互、达成预期目的、遵守适用约束的系统（INCOSE 2019：a system designed or adapted to interact with an anticipated operational environment to achieve one or more intended purposes while complying with applicable constraints）。INCOSE 还说明，工程系统可以由人、产品、服务、信息、流程、甚至自然元素组成。

> 出处：这条定义转自报告库 05 号 v1.2 §8，属二**手来源**：库内有 INCOSE《系统工程手册》第五版中文译本，但所引英文**定义句**不在其中；`INCOSE-TP-2020-002-06` 号文件库内也没有。故带年份 `INCOSE 2019` 引用。库内可直核的相邻处有两处：`ISO/IEC/IEEE 15288:2023` §3.46 给”系统”下的定义，以及 §5.2.1 那句”系统的属性……源自或涌现于”各部分及其关系——后者正是 §2.4.2 那条涌现属性的正本依据。**与底本的一处分歧须**

> **说明**：05 号 §8 把软件系统直接列为工程系统，本书按这条定义里”人、流程、组织是否算进来”改判为技术系统（§2.4.3）。

工程系统的内核，是**技术系统**：仅由人工技术组件（硬件、软件、机械、电气）构成的系统，人只是使用者。

技术系统的特征是：纯人工制品，没有自然物、没有”活的”组件；功能靠技术实现，性能可量化、可测试（吞吐量、负载、响应时间）；人只是外部使用者，司机、操作员都在系统之外，不是组成部分。（这三条特征文字转自报告库 05 号 v1.2 §9；划分依据＝INCOSE 2019 的工程系统定义。）

一句话：技术系统是设备的集合；工程系统是整个社会技术系统，是技术子系统、人的子系统、组织子系统耦合在一起的整体，技术系统只是它的内核（§2.4.4）。

### 概念卡片 · 工程系统

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 被设计/改造、与环境交互、有目的、守约束的系统（INCOSE 2019）——人、产品、服务、信息、流程**甚至自然元素**都是它的一部分 |
| **它不是什么** | ⑥⑦ | 对立：自然系统（无设计者）· 社会系统（非被设计，如习俗、市场）· 概念系统（无运行环境）；社会系统不等于社会技术系统——后者就是工程系统（见§2.4.4）；辨析：工程系统包含技术系统，技术系统只是内核 |
| **它从哪来** | ②③⑨ | 20 世纪 70—80 年代系统工程界从大量失败项目发现”系统造出来、性能指标也达到了，但客户就是不用”，于是人、流程、组织、数据的重 |
| **它到哪去** | ④⑤⑧ | 被用起来<br>123 |

工程系统的生存周期：INCOSE 的系统工程定义里，工程系统要被”实现、使用、退役”，意味着工程系统有生存周期，要为”退役”做设计，让系统能安全关停、数据能迁移。恒信的合同审批系统，十年前的纸质台账怎么迁入？退役时的数据归档和权责移交？这两问，技术系统不问，工程系统必须答。

软件系统为什么不算概念系统：它虽然看不见摸不着，但它是被设计出来的，有明确目的，与操作系统、网络和用户交互，也守着性能、安全与法规这些约束。不过要留意：它**本身**仍是技术系统，人只是使用者、不在它之内；加上用户、业务流程和运维组织，才是工程系统（§2.4.4）。

> 术语注：软件系统不以”概念系统”归类。概念系统（数学理论那类纯信息形态）被判为非工程系统，理由是它**没有运行环境**；软件系统有运行环境、有生存周期，所以不归入概念系统；至于它算不算工程系统，要看人、流程、组织有没有算进来（§2.4.4），单看它自己，仍是技术系统。两处都”看不见摸不着”，是两件事，不能互推。

最典型的工程系统例子是机场：跑道和航站楼（物理）、雷达和行李分拣（技术）、塔台和地勤（人）、安检和调度（流程）、航班数据（信息），还有气象条件（环境）。合起来才产生”机场顺利运行”这个涌现行为。

单看任何一个子系统都合格，不等于机场能转起来。工程系统的边界，画在”整个运行环境”上，不是画在”设备的壳”上。

同样的道理，冰链的 M3 温控箱也不只是”设备集合”。硬件、固件、云端、App，那是技术系统；而它要交付的是”疫苗、药品冷链运输”——全程温度可追溯、断链可告警——背后连着仓储物流流程、法规义务、批次追溯和各环节的操作员。水忠把箱体做到极致，是把技术系统做到极致；箱体进了冷链运输流程、温度真被全程追溯、断链真能告警出来，M3 才算被”用起来”。按技术系统看，它达标了；按工程系统看，它才刚开始。

这个道理，制造侧的逸人最清楚：

“产线上我把这箱子造出来，可箱子什么时候算’成了’？不是下线那刻。是它装进冷链车、温度被全程追溯、断链真告警了——那才叫被’用起来’。我们车间考核的是下线合格率，客户认的却是全程没断链。按技术系统看，我们把箱子做完了；按工程系统看，我们才交了一半。”

### 2.4.4 验收参照：上线不等于成功

把技术系统当成工程系统来验收，是系统工程史上最贵的教训之一，也是恒信集团的写照。

按技术系统视角：恒信合同审批系统部署好了、服务器跑起来了、模块全部上线，就是”系统成功”。

按工程系统视角：采购流程没改、法务人员不习惯电子签、财务数据没清洗、审批权责没理清，就是”系统失败”。

砸了几千万，最后发现”系统”根本没被用起来：法务还是线下签、财务还是手工台账、业务还是走邮件。这不是技术问题，是把技

术系统误当成了工程系统来验收，忘了人、流程、组织也是这个系统的一部分。

这里还差最后一道分辨：就算系统真被用起来了，你怎么知道它”成”了？**验证（verification）对的是规定需求**：需求的每一条都实现了吗？测试过了吗？答的是”做对了吗”。**确认（validation）对的是预期用途**：这个系统在真实的使用场景里，解决了利益相关方的问题吗？答的是”做对了东西吗”。两者不可互推：**全部需求都验证通过，确认仍可能失败**。恒信的上线验收，只做了前者。

> 出处：`ISO/IEC/IEEE 15288:2023` §3.54／§3.55 的注 1 与 `ISO 9000:2026` §3.11.12／§3.11.14 两处定义说的都是这件事：确认对的是”针对特定预期用途或应用的需求”，验证对的是”规定需求”。**两句话只差一个词**：`§3.55` 注 1 末句作 The system was built right.（验证），`§3.54` 注 1 末句作 The right system was built.（确认），问的是两件事。

20 世纪 70—80 年代，系统工程界用大量失败项目发现：很多”系统”其实是社会技术系统，单独优化技术部分没有意义。这就是为什么 INCOSE 2019 特意把定义从 “technical system” 改写为 “engineered system”，把视野从设备扩到整个运行环境。

这个概念当年被系统理论家 Ropohl 等人点破：很多”系统”是技术子系统、人的子系统、组织子系统耦合在一起的整体。技术部分

是内核，但单独优化它没有意义，因为”用不起来”往往卡在人、流程、组织那一层。

软件系统本身是技术系统（代码、数据、服务器）；“软件 + 用户 + 业务流程 + 运维组织”才是工程系统。说”我们构建了一个技术系统”，说的是设备的集合；说”我们构建了一个工程系统”，承诺的是整个社会技术系统能运转起来：技术达标、人会用、流程顺、组织撑得住。恒信的敏捷转型要成为真正的工程，第一步是把验收的参照从”系统上线”换到”系统被用起来”。

这也给第 8 章埋下回声：把”结果当成功”的病，在确认环节的版本就是”UAT 跑了就是确认过了”，同一病的两种形态。

### 2.4.5 回头给恒信量一遍

把这一章的判据串起来，对恒信的合同审批系统完整地量一遍：

项目一：交付能不能离开某个人。换个团队来运维恒信的 CI/CD，能一样跑吗？徐漾把问题递给玉杰，他是运维，这话他最有发言权。

玉杰摇头：“换个人来，光那三条隐藏命令就够他找一天；还有两个环境变量，写在李四的备忘录里，不在任何文档里。这一项我认——按 §2.2.4 那条判据量，我们不合格。”

项目二：层次是否齐备。工程科学：容量评估被砍，这一层缺位；方法论：敏捷在，合格；工具与实践：CI/CD 在，合格；价值与伦理：为进度砍可靠性，这一层失守。结论：缺了两层，尤其缺了最致命的价值与伦理。

项目三：工程化的成色。有结构、有方法、可重复：前几样沾了边（可重复说的是过程本身能重复执行；“换个人还能不能交付”是项目一的事）；全覆盖：容量测试被砍，不及格；受控：“先上再说”，不及

格；可验证：没有容量验证，不及格。结论：缺了全覆盖、受控、可验证，不是”差一点”，是差一半。

项目四：系统化到什么程度。每一步有输入输出吗？有检查点吗？换执行者稳定吗？——最后一句答不上来。结论：系统化的”可检查”没立住，差的就是那另一半。

项目五：整合度。法务、财务、业务三个模块各自都做得好，但数据定义对不齐——涌现属性在系统的”缝”里等着。结论：部件工程合格，系统级整合缺位。

项目六：验收的参照。按技术系统看，系统上线了；按工程系统看，法务线下签、财务手工台账、业务走邮件。该做的两道判定只做了一道：验证勉强跑了一半，确认根本没做。结论：验收参照用错了，拿”上线”当”成功”。

量下来，项项不合格。恒信的方法论和工具是真的，缺的在别处。这里给一条建议：

> **先把”验收的参照”从”系统上线”换成”系统被用起来”，再谈敏捷转型成功。否则，这套流程与工具只会让”不会出事”的错觉维持得更久。**

这一章的判据不是彼此孤立的。试金石测可复制，层次测完整，工程化的成色测过程，系统化自问测可检查，系统工程（元工程）测整合，验证与确认测验收。合起来，才是一次完整的量。

### 2.4.6 认知反转

回到恒信那场季度会。万辉宣布”我们已经是工程化团队了”——会议室鼓掌，徐漾问了句扎心的。

第一处：他把”有方法论、有工具”当成了”在做工程”。工程不止那两层，敏捷和 CI/CD 只占其中一部分。上个月的事故不是”工程之外”的意外，恰恰是缺了价值与伦理那一层的证明——为进度悄悄牺牲可靠性，还不留记录，是这一层失守的标准动作。

第二处：他背得出 IEEE 610.12 的三个形容词，却没问过那三个词是在给谁下定义。那是一份软件工程的定义，说的还是”工程化的过程特征”——过程特征齐了，不等于工程齐了。

第三处：他默认了”系统上线了”就算工程成功。系统是技术系统还是工程系统，取决于它有没有被整个社会技术系统用起来——恒信的系统上线了，但法务还在线下签，这套系统就不是一个成功的工程系统。

而冰链的水忠，给了另一种对照：几十年手艺，个人能力顶天立地，但交付依赖具体的人。换个人来做，交付就维持不住。

这一章的反转是温和而彻底的：你可能没有在做工程，而你在做的那件事，可能比工程更难被复制。不要急着宣布”我们是工程化团队”。先问一句：换个人来做，交付还维持得住吗？再看那几层齐了没有。

用第 1 章的话说：对”工程”这个词，你原来只是”见过”：背得出定义、说得出”敏捷+CI/CD”，甚至背得出那几个英文形容词。这一章把它推到”拥有”：能答全四问（它是什么、不是什么、从哪来、

往哪去），能判一个系统是不是工程系统。能答全四问才算拥有——你当场就能判一个系统是不是工程系统。

> **工程 = 科学原理 × 判断创造，在约束下，用系统方法，造出有用且安全的东西。它不靠流程的多少定义，靠工程科学、方法论、工具与实践、价值与伦理是否同时在场定义。**

用第 1 章 §1.4.4 的尺子量一下：这一章把”工程”从**第 ① 级（表征）**推到了**第 ② 级（图式）**。你现在看得出那几层里缺的是哪一层了；至于能推演、能拿主意，那是**第 ③ 级（心智模型）**的事。

## 小结

这一章回答的是全书最基础的问题：我们到底在做什么。工程不是流程的堆砌，是工程科学、方法论、工具与实践、价值与伦理同时在场的人类实践；它不靠工具定义，靠那条判据和那几层齐不齐来检验。它把交付从人身上挪到方法上，却把判断留在人身上。这条边界，既解释了它为什么可复现，也解释了它为什么不能被自动化替掉。

下一章，我们看产品开发里最要紧的几个词：需求、设计开发、质量。工程那层底子立起来之后，就该看它们各自是什么、边界画在哪、又是怎么合起来用的。

散会时，玉杰在运维台账上添了一栏：这条曲线归谁。以前它只活在他的监控屏上，他嘴上念叨、从不上纸；这回他把它写下来了，写下来，别人才接得住。

# 附录

## A 本章概念总图

主张：工程是把交付从人身上挪到方法上的一次转移——挪得走的是方法，挪不走的是判断；

挪走了就成，没挪走就不成

工程 = 科学原理 × 判断创造，在约束下，用系统方法，造出有用且安全的东西

├─ 定义　科学原理 × 判断创造（料）· 约束（框）· 系统方法（路）· 有用且安全（果）§2.2

├─ 权威定义　ECPD 1941 → ABET 1979 → INCOSE 2019：三家同一个内核；

│　IEEE 610.12-1990 另给软件工程一侧的过程特征　§2.2.3

├─ 试金石　换个人、换团队，用同样的需求和约束，能交付质量相当的成果吗——能，是工程；不能，那是手艺　§2.2.4

├─ 对立面　认识论×科学　方法论×手艺　目标×艺术　§2.3.1

├─ 层次　工程科学 → 方法论 → 工具与实践 → 价值与伦理（四道挪走的关）　§2.3.3

├─ 近义词辨　systematic 系统化（有章法）≠ systemic 系统性（有全局）≠ algorithmic 算法化（确定执行）§2.3.4

├─ 自动化　提纯判断 + 拿走人阻止事故的能力——要错误可见，不是零错误

├─ 系统工程 = 元工程（整合部件，不造部件）→ 涌现属性：部件全对 ≠ 系

统对　　§2.4.1~2

├─ 验证与确认　验证对规定需求（做对了吗）· 确认对预期用途（做对了东西吗）—不可互推　§2.4.4

└─ 工程系统　包含　技术系统（人/产品/服务/信息/流程/自然元素都是它的一部分）　　§2.4.3

## B 本章的能力级

**第 ② 级为主**

- **表征**——把”工程”这个词说准（科学原理 × 判断创造、约束、系统方法、有用且安全的人造物），并把挨着它的那些词分开：手艺、科学、艺术、方法论、系统工程、技术系统、社会技术系统（附录 A T-13~T-21；“工程”完整九格见**书末**附录 B，“工程系统”见本章 §2.4.3）
- **图式**——几个可指认的结构：工程的层次（工程科学 → 方法论 → 工具与实践 → 价值与伦理，四道把交付从人身上挪走的关）、对立面（认识论对科学 · 方法论对手艺 · 目标对艺术）、包含关系（工程系统包含技术系统）
- **心智模型**——几条能推演的机制：试金石（换个人还能不能同等交付）、系统化到什么程度（有输入输出 / 有检查点 / 换执行者稳定）、价值失守问什么（利益相关方知不知情 / 有没有评审 / 有没有记录）、验证与确认不可互推、自动化把判断推到了哪里
- **解释框架**——本章不建此级；由第 17、18 章承担

> 能答全四问，才算拥有；只能背出那句定义、那几层名字，只是见过。真拥有的人，是在压力面前守得住价值与伦理那一层的人。

## C 自查 · 这些说法对吗?

1. “我们用了敏捷 + CI/CD + 测试，所以我们在做软件工程。”——**错**。敏捷和 CI/CD 只落在方法论和工具与实践上；工程还有工程科学、价值与伦理这两层。
2. “工程的对立面就是科学。”——**错**。工程没有单一对立面：认识论上是科学、方法论上是手艺、目标上是艺术。
3. “系统化就是把流程固化成算法，让机器自动跑。”——**错**。系统化是”有方法（含判断）“；算法化是”确定性执行”。工程每一步都留有人判断的空间。
4. “微服务每个都健康，系统就是健康的。”——**错**。涌现属性：调用链、数据一致性、超时熔断这些灰色地带没人管，上线必出事。
5. “软件部署上线了，系统就成功了。”——**错**。那只是技术系统成功；工程系统成功要看人会不会用、流程顺不顺、组织撑不撑得住。
6. “需求全部验证通过了，这个系统就成了。”——**错**。验证对的是规定需求，确认对的是预期用途，两者不可互推；需求全过而没人用，一样是失败。

7. “老工程师经验丰富，这就是工程。”——**错**。那是手艺；按那条判据一量：换个人还能同等交付吗？把经验写成文档只是第一步，还要可检验：换个人能按文档交付同等结果，才算工程化。
8. “自动化程度越高，系统越安全。”——**错**。执行交给机器，判断留给人；剩下的判断反而更难、更密。而自动化也削弱了人拦事故的权限和本事。工程要的是”错误可见”，自动化追求的是”零错误”。

## D 练习

1. 用试金石检验你们团队最近交付的一个系统：换三个人来做，能交付质量相当的成果吗？如果不能，缺的是**哪一层**（工程科学／方法论／工具与实践／价值与伦理）？
2. “敏捷 + CI/CD + 测试”落在工程的哪几层上？缺的那几层，用你们公司的实际情况各举一个”没守住”的例子。
3. systematic、systemic、algorithmic 三个词，各自回答什么问题？举一个”有章法但需要判断”的日常工作例子。
4. 系统工程和软件工程是什么关系？为什么说系统工程是”元工程”？你们系统最近一次”部件全对、整体出事”，是哪个灰色地带没人管？
5. 用”技术系统”和”工程系统”两个视角检查你们最近上线的一个系统：按技术系统看成功了吗？按工程系统看呢？如果两个答案不一致，缺的是哪一块（人／流程／组织／数据）？

6. 把你们最近一次验收的记录翻出来：哪几条做的是验证，哪几条做的是确认？有没有”验证全过、确认没做”的？
7. 那几起工程事故（泰坦尼克／挑战者／737 MAX）的共同点是什么？在你们公司找一个”价值与伦理先松手”的真实例子。
8. 冰链的水忠是”手艺”还是”工程”？如果让你给水忠的手艺做”工程化改造”，第一步做什么？
9. 照着有结构／有方法／可重复／全覆盖／受控／可验证给恒信的敏捷转型打分，指出缺的是哪几样。
10. 你们团队最值钱的”手艺”是什么？如果要把它工程化，第一步是把哪个”心法”写成可检验的规则？
11. 拿那句定义给你最近的一个项目打分：科学原理、约束、系统方法、有用且安全的人造物，各自缺了什么？哪一条缺了最致命？
12. 用三句话，把”工程 ≠ 手艺”讲给一个完全不懂工程的人（比如你的家人）——卡在哪个词上？那个词就是你概念最薄的地方。

（答案要点见书末附录，与训练教材题卡同源）

---

## E 参考文献

**标准** - R-102 ISO/IEC/IEEE 15288:2023 —— 本章几处一手锚：**§3.46** 系统定义；**§5.2.1** 系统属性”源自或涌现于”各部分及其关系（涌现属性的正本依据，§2.4.2）；**§3.50** 系统工程定义（来源条注明 INCOSE-TP-2020-002-06，§2.2.3 那三家之一在此可回正本逐字

核）；**§3.54 注 1 / §3.55 注 1**（The right system was built. / The system was built right.——确认与验证的分野，§2.4.4）- R-103 ISO 9000:2026（第 5 版）—— **§3.11.12 验证**对”规定的需求”／**§3.11.14 确认**对”针对特定预期用途或应用的需求”（§2.4.4 验证与确认不可互推的条款锚）- R-15 ECPD 1941 / ABET 1979 / INCOSE 2019 —— 工程权威定义（几家的定义走到同一个内核：科学原理／判断创造／约束／系统方法／造福目的）；INCOSE 2019 的工程系统定义与生存周期。**库内无正本的是三家的定义原文**：转自报告库 05 号 v1.2 §5、工程系统定义与生存周期转自同报告 §8，一律带年代引用；**INCOSE 那条可经 15288:2023 §3.50 回正本逐字核**（§2.2.3）- R-14 IEEE 610.12-1990 —— 软件工程定义（“把工程应用于软件”；systematic / disciplined / quantifiable → 工程化的那些方面；**它定义的是软件工程，不是工程**，§2.3.4 有引用规范）。**库内无正本** - R-18 CMMI —— 工程／开发方法论举例（敏捷、瀑布、CMMI）

**经典文献** - R-45 INCOSE Systems Engineering Handbook —— 工程系统（**译者注中文释义**，PDF p46／印刷页 31）；系统工程术语（PDF p397）- R-47 Ropohl —— 社会技术系统：技术子系统＋人的子系统＋组织子系统耦合，单独优化技术部分没有意义 - R-42 冯·卡门（von Kármán）——“科学家研究现存的世界，工程师创造从未存在的世界”（常被归于；库内另存一种英文措辞，见报告库 12 号）

> 工程师的能力清单见第 9 章。

# 第 3 章 需求、设计开发、质量：产品开发的图式

## 3.1 引言

这一章立产品开发里最要紧的三个词：需求、设计开发、质量。三节走一条线：先把它们各自说准，再把相邻的边界一一画开，最后合起来看它们为什么拆不掉。这条线立住之后，从第 4 章起，每一章都是它在某一个方向上的展开。

### 3.1.1 误解现场

周五下午四点，冰链科技的产品评审会开了整整两个小时，会议室里飘着咖啡和争论的气味。

产品经理文正把一页参数对比表拍到桌上，指着一个数字：

> “M3 的温度精度做到 ±0.3℃，竞品只有 ±0.5℃；断电保温 ≥ 72 小时，竞品 48 小时；重量还轻了 2 公斤。这几条都落到可测量的固有特性上了，设计规格全线优于竞品——**符合性没有问题**，质量这一关我看不出问题在哪。现在只剩一件事：对外怎么讲，是打’行业精度第一’，还是’保温时长远超竞品’。”

会议室里一片点头，有人甚至轻轻鼓了掌。坐在上首的产品开发经理永兵也点了头，在产品评审结论栏里签下”通过”：

> “三个参数全线领先，需求里写的每一条都超了。质量这一关，我签字。”

只有坐在角落的测试李旭，弱弱地举起手：

> “那……测试判据是什么？接收准则写在哪份测试方案里？”

文正摆摆手：“技术方案都定了，符合性判据后面自然会有。先定对外怎么讲。”

李旭张了张嘴，又闭上。会议继续。

如果你坐在这间会议室里，你大概也会点头。因为大多数人对”质量”和”设计开发”这两个词的直觉，和文正一模一样：

- **质量 = 东西做得比人强、指标压过竞品**；
- **设计开发 = 做出技术方案**，方案定了，后面的测试、采购、生产、交付，都是”自然会有”的事。

这两个直觉，是这一章要推翻的东西。错的不是文正，错的是这两个词在他脑子里指向的那个东西。李旭那句被淹没的问话，其实已经给出了结论：设计开发根本没有结束，到章末我们回来读它。

### 3.1.2 这一章要回答什么

读完本章，你应该能回答：

1. 需求、设计开发、质量各自是什么？它们为什么能构成产品开发的图式，彼此又不在同一个层级上？
2. 需要、期望、需求有什么区别？目的、目标、需求又有什么区别？
3. 需求里”明示的”“通常隐含的”“必须履行的”分别指什么？漏掉哪一种最要命？

4. 设计开发从什么时候开始、到什么时候结束？“技术方案做完了”够不够？
5. 需求开发和设计开发的边界画在哪？凭什么这么画？
6. 质量是”特性除以需求”吗？“等级高”等于”质量好”吗？特性超出需求是加分还是减分？
7. 质量能打分吗？为什么有些特性不能摊进平均数？
8. “质量与进度冲突”到底是一场什么冲突？该动的是什么，不该动的是什么？
9. 需求、设计开发、质量为什么互相定义、拆不掉？由它们合成的那条路径能推出什么，又推不出什么？

### 3.1.3 这一章的骨架

先把这一章要立的东西摆出来，后面三节，都是给它添内容。

这一章的主张是一句话。

> 需求、设计开发、质量不是三个并列的词，是一条能走的路径——定需求 → 出规格 → 判满足；这条路径立起来，“东西好不好”就从一句印象，变成一次可复核的判定。

需求、设计开发、质量在这条路径上各有位置：

> 需求 = 定需求：定义”要什么”。它是设计开发的输入，也是质量判定的依据。

> 设计开发 = 出规格：把需求逐级转化为更详细的需求，一级出一份规格，直到可实现。规格就是规定需求的集合。
> 质量 = 判满足：认定客体的固有特性满足需求的程度。单项看符合不符合，整体看满足到什么程度。

它们的分量并不一样，第 1 章把领域里的概念分成三层：一般概念、核心概念、根概念，**根概念 ⊂ 核心概念**，越往里门槛越高。产品开发这一行的核心概念不止三个，这一章只取这三个；第 1 章 §1.7.4 判出的根是需求、系统与设计开发，这一章取其中的需求与设计开发——它们是根，质量是派生的枢纽。

需求、设计开发、质量同属核心概念这一层，但三者并不平级。而它们聚起来形成的结构，就是这一章要说的产品开发的图式：**三个词是表征，聚起来的那条路径才是图式**（第 1 章 §1.4.4）。

这一章分三节走：第一节把需求、设计开发、质量各自是什么说准，包括它们的来处；第二节把相邻的边界一一画开：起点、终点、参照；第三节把定需求、出规格、判满足这条路径用起来：读出程度、解开冲突、看它们为什么拆不掉。

---

## 3.2 需求、设计开发、质量是什么

> 这一节先看定义。它们都在这里出场：需求给三种形态，设计开发给一条转化的定义，质量给一个程度。看完定义，后面的边界与运用才有话可说。

### 3.2.1 需求：没说出口的也算

打头的是需求，ISO 9000:2026 给它的定义只有一行：

> **需求（requirement）**：明示的、通常隐含的或必须履行的需求或期望（ISO 9000:2026 §3.5.1）。
> need or expectation that is stated, generally implied or obligatory

> 术语注：requirement 本书一律作”需求”；“要求”是另一个汉语词（**本书口径**，ISO 9000:2026 里没有 demand 这个术语条目），不用来指 requirement。ISO 9000:2026 的中文译本把 requirement 译作”要求”，本书引用时按本书术语写作”需求”，条款号不变；需要逐字对照的，以英文原文为准。

注意后半句是并列关系，没有主次：明示的（stated，说出来了）、通常隐含的（generally implied，惯例默认、没说出口）、必须履行的（obligatory，法规规定的那一类），三个都是需求。

拿 M3 温控箱展开，三种形态全占。

**明示的**——写在纸上的。客户在合同里写明”箱温波动不超过 ±0.5℃“”箱门可单手打开”。白纸黑字，谁都能看到，谁都赖不掉。

**通常隐含的**——没人写，但行业默认。冷链运输箱”应该扛得住装卸时的磕碰”“外壳不能有锐角划伤搬运工”“电池要能撑过全程并显示余量”。这几条没人写进合同，但你要是漏了，客户在使用的第一天就会认定”这产品不行”。隐含需求漏了是缺陷，不是”没写下来所

以不算”，它不会自己开口，需要组织主动去识别。这是起点判据里最贵的一条（见本章 §3.3.2）。

**必须履行的**——法规强制的那一类，无条件满足。法规依据是 ISO 9000:2026 §3.5.5 法定要求与 §3.5.8 法规要求。GSP（药品经营质量管理规范，**库外来源**）把疫苗冷链的温度、断链与异常报警列为硬性需求（冷藏疫苗常见 2~8℃，冻干制品 −20℃ 冷冻），M3 按 −20℃~8℃ 宽范围设计，断链即报废。不满足这条，M3 连上市资格都没有；必须履行的需求没有讨价还价的余地。

镜照一眼恒信集团（同一个概念，换到另一个世界照一照）：合同审批系统的明示需求是”合同在三个部门间流转审批”，隐含需求是”审批人能在手机上快速通过”“离职审批人自动移交待办”，必须履行的是”合同数据按公司法要求留档十年”。IT 世界里也是明示、隐含、必须履行，一个不缺。

三种形态的差异，到第 8 章会变得性命攸关：验证（verification）对照的是规定需求，也就是被明示的那一类（正本原注：例如写在成文信息中的）；确认（validation）对照的，是针对特定预期用途或应用的需求，也就是真实场景里的那些（ISO 9000:2026 §3.11.12、§3.11.14）。隐含需求没进文档，验证够不着它，只有确认拦得住。那是第 8 章的事，这里先埋一句。

“需求”这个词本身也有来处：requirement ← 拉丁 requirere（要求、寻求）。而这个词的所指经历过一次漂移，它原指”要的东西”，后来才分出明示、隐含、必须履行这三种形态。这正是第 1 章 §1.8.3

说的”同词异义”：一个词如何攒出多个概念。ISO 9000 把这场漂移固化成了标准定义。

### 3.2.2 需要、期望与需求：内容与呈现方式

英文在这里特别值钱，中文习惯把”需要”“期望”“需求”糊成一个词，英文是三个：

| 英文 | 中文 | 含义 | M3 的例子 |
| --- | --- | --- | --- |
| need | 需要 | 硬性诉求，缺了会痛 | 疫苗不能坏 |
| expectation | 期望 | 软性诉求，缺了会失望 | 精度越高越好、App 越顺手越好 |
| requirement | 需求 | 需要或期望 × 呈现方式 | 上面两者的全集 |

分清这一层，要害在于”需要”和”期望”是**内容**，而”明示”“隐含”“必须履行”是**呈现方式**。内容说的是”你要的那个东西本身”，呈现方式说的是”这个内容是怎么出现的”。两个方向交叉，才是需求的完整样子：一个 need 可以是明示的（“我们要 72 小时保温”），也可以是隐含的（“要能扛磕碰”）；一个 expectation 可以是隐含的（“手感要高级”），也可以是明示的（“外壳要金属拉丝”）。

> 术语注：“需要和期望是内容、明示和隐含和必须履行是呈现方式”这个两维说法是本书的梳理主张——ISO 9000:2026 §3.5.1 的定义句并列给出 need／expectation／

> requirement 三个词，其中只有 requirement 是术语条目，不这样分方向；两维画法是本书为方便使用而做的凝练。

为什么要较真这个区别？因为后面所有的设计开发、质量判定、评审、验证与确认，对上的都是”需求”这个完整样子，而不是其中某一个形态。文正心里想的”质量”，八成只有”期望”那一层，“越高级越好”；而法规、客户合同、行业惯例这些真正决定成败的部分，不在他的视野里。把需要、期望、需求糊成一个，等于把最重要的一半需求扔出视野。

镜照一眼恒信：黄程的”需要”是月底账能对上，缺了会痛；“期望”是查询响应再快一点，缺了会烦；而”查询响应有明确指标、数据留档十年”才是落到纸上的需求。IT 世界里同样是”内容 × 呈现方式”这个两维样子。

### 3.2.3 目的、目标与需求：价值、结果与规格

比上一条更常见的混用，是需求、目标、目的不分：ISO 9000:2026 给了目标（objective）和需求（requirement）的标准定义，没有单列”目的”（purpose）。三者怎么分，用这一份定义对齐（本书的梳理主张，作为对标准定义的解释）：

> **目的（purpose）** = 系统为什么被研制的理由。落在价值侧，可以不量化；它是目标与需求的来源，在最上游。它的确立过程对应 ISO/IEC/IEEE 15288:2023 §6.4.1 业务或任务分析，

> 任务与目的在那里定义；**ISO 9000 未给”目的”术语级定义**，15288 给的是”过程目的”（§3.29）。
> **目标（objective）** = 要实现的结果（ISO 9000:2026 §3.7.11）。落在系统语境，就是系统预期达成的可度量结果，是目的的可度量投影；主语是”要实现的结果”，可以是战略性的、战术性的或运行层面的，也可以是组织范围的、或针对某个项目、产品、服务或过程的（§3.7.11 注 1、注 2），不必是系统自身的属性。
> **需求（requirement）** = 系统自身的可验证规格。落在系统自身；三种形态见 §3.5.1。

对照着看：目的 = 存在理由，目标 = 要实现的可度量结果，需求 = 自身可验证规格，三者都是关于系统的陈述，区别在侧面，不在”内”和”外”。链条是：目的 →（展开）目标 →（转化）需求。

落到 M3（疫苗冷链）：目的 =“让疫苗在冷链运输中一路不失活、安全送达”（价值侧，可以不量化）；目标 =“到站疫苗活性 ⩾ 95%”（把”不失活”翻译成能盯着看的数）；需求 =“箱温 -20℃~8℃ 内偏差不超过 ±0.5℃、断链 30 秒内告警”（系统的可验证规格）。三者各占一个侧面：价值、结果、规格。

为什么不能混？目的定方向，需求定规格，目标是中间那层翻译。把目的当需求写进规格，设计开发会茫然：一张写着”疫苗不失活”的纸，没给任何可设计的数。把目标当需求，则跳过了”靠什么达成”这

一步：活性 ⩾ 95% 是个结果，活性靠箱温守住，得倒推出箱温条款，设计才落得下去。

| | 目的 | 目标 | 需求 |
|---|---|---|---|
| 回答 | 为什么做 | 达成什么可度量的结果 | 做成什么样 |
| 侧面 | 价值 | 结果 | 规格 |
| 能不能量化 | 可虚 | 必量化 | 必须可判定 |
| 锚点 | 15288:2023 §6.4.1 | ISO 9000:2026 §3.7.11 | ISO 9000:2026 §3.5.1 |

比混用更隐蔽的，是**根本不立目的**。目的由利益相关方定义、业务方签字确认，开发团队与架构师只承接、不拍板（第 18 章展开）。目的缺位的时候，团队不会停下来等，而是照自己猜的那个”目的”往前冲，做得越完美，离真实目的越远，这正是”正确地做错事”。先过目的这一关，再谈目标与需求。

恒信的合同审批系统，同一根链条也立得住：目的 =“让合同审批合规、月底账目清楚对得上”；目标 =“月底对账时间不超过 2 小时、逐笔一致率 100%”；需求 =“合同编号三方一致、25 号后签的合同归属下个统计月”。两个系统，同一套价值、结果、规格。

### 3.2.4 设计开发：需求怎么变得更详细

第二个词，也是这一章的主角。ISO 9000:2026 的定义，几乎每个词都是关键词：

> **设计和开发（design and development）**：将某一客体（§3.5.3）的需求（§3.5.1）转换为该客体更详细需求的一组过程（§3.3.1）。

拆开看，四样东西各有各的讲究。

客体——被设计的东西；定义用的是”客体”，不是”产品”。这不是措辞讲究：设计开发的对象绝不限于硬件，产品设计开发、服务设计开发、过程设计开发都成立（§3.3.11 注 3）。M3 温控箱是客体，恒信的合同审批系统是客体，连一条”疫苗出库流程”也可以是客体。

把”过程设计开发”落到最熟悉的例子：工艺编制。图纸定下”装配扭矩 15±2 N·m、密封无泄漏”，这是构件做成什么样；工艺编制就是过程（制造）的设计开发，把”做成什么样”转成”制造环节怎么做”的可实现需求，输出制造方案（工艺规程、工艺卡）。

产品开发不只是图纸或代码，还包括构件的加工、装配、检验技术的开发。缺了工艺，产线不知道”怎么装配、用什么工装、检验标准是什么”，设计就没到可实现。

需求——输入必须是需求，不是点子、灵感、创意；创意不表述为需求，就进不了设计开发的流程。顺着这条边界再往外划一格：目的和目标也不在设计开发之内，它们是设计开发的输入端与前提（15288:2023 §6.4.1 业务或任务分析、§6.4.2 利益相关方需要与需求定义）；设计开发从系统需求（§6.4.3）起算，系统架构定义（§6.4.4）、设计定义（§6.4.5）才是它的核心。

转换为——动词才是灵魂，输入与输出必须是不同的东西。流程走完了，输出和输入一个样，那不是设计开发，那是走流程。这个”不同”体现在”更详细”上。

**更详细的需求**——唯一的判定标准；注意，定义没说”更正确”“更优化”，只说”更详细”。设计开发的职责是把需求展开、分解，落到可实现和可验证的粗细；方案好不好，由评审和验证把关，不是设计开发自己说了算。

> 辨析栏 · 设计、开发、设计开发
> 英语里 design（设计）、development（开发）与 design and development（设计开发）三者，有时完全同义，有时指整个流程的不同阶段：design 偏概念与方案阶段，development 偏详细化与实现阶段。用哪个词不重要，重要的是整体是一个需求转化过程（§3.3.11 注 2）。落到中文，“设计开发”对应 §3.3.11 的完整术语，别把它拆成”设计”加”开发”两件事。

“更详细”这三个字，拆开来看是三个可查的成分：

| 成分 | 回答的问题 | 判据 | 锚点 |
| --- | --- | --- | --- |
| **更具体** | 详细体现在哪 | 需求通常按特性来定义，特性可以定量也可以定性 | §3.3.11 注 1 |
| **更可实现** | 详细到什么程度够 | 够实现为止，能直接被生产和服务提供执行 | 本章 §3.3.5 终点 |
| **更可验证** | 详细得对不对方向 | 落到特性上（定量或定性都可），让验证有对照的东西 | §3.3.11 注 1 |

注意”更可验证”这一格只证明方向对，不能当终点：验证以规定需求为对照，而规定需求正是设计开发的输出，所以验证能发生，前提是设计开发已经结束（本章 §3.3.4 有完整推理）。

还有一个被排除的成分：“更可理解”。它若指”文字浅显”，与”详细”无关甚至相悖；若指”能被下游理解执行”，已经被”更可实现”吸收了。

一句话记：具体到特性，详细到实现，可验证指方向。

这套判据在实战里很好用。既然”更详细”是设计开发有没有发生的唯一判据，判断”这个变更算不算设计开发”就有了硬标准：输入输出同样详细，就是没发生设计开发。

只改外观颜色、不产生任何新需求细节的变更，严格说不构成设计开发，可以走快速通道；而”把温度控制从单路改成双路冗余”产

生了新的可靠性需求、新的接口需求、新的测试需求，就是一次实实在在的设计开发，必须走完整流程。

这条标准在变更管理和审核时很管用：它把”改一下”和”重新设计”分开，后者不能图省事。

### 3.2.5 设计开发：每一级都在做同一件事

定义里还有一个注容易被忽略：**输入那份需求的表述，比输出”更宽泛、更概括”**，而一个项目中可以有若干个设计和开发阶段（ISO 9000:2026 §3.3.11 注 1）。后一句让设计开发在项目内部层层往下走：

**客户需求（宽泛）→ 概念设计 → 系统需求（细化中）**

**→ 详细设计 → 部件规格（更细）**

**→ 工艺设计 → 可生产可检验的需求（最细）**

概念设计把客户需求变成系统需求，详细设计把系统需求变成部件规格，工艺设计把部件规格变成可生产、可检验的需求，**每一级都在执行同一个 §3.3.11**。每个阶段的输出需求，就是下一阶段的输入需求，直到需求足以支撑实现和验证。

这对 M3 意味着什么？文正以为”设计开发 = 做一次技术方案”。不对，M3 的温度控制系统需求 → 制冷子系统规格 → 压缩机与温控板件规格 → 生产工艺需求，是四五个设计开发阶段叠在一起。任何一级的”更详细”没做透，下游就在半成品上开工。这也是为什么终点那么要紧：总得知道整条路径在哪里算走完。

设计开发内部怎么运转？有三个动词常被误当成三个并列的流程：构思（conceiving）、定义（defining）、设计（design）。它们不是并列，是包含，构思和定义都发生在设计开发这个范围里面。

> **构思（conceiving）**：从利益相关方关切出发，生成多个候选方案的发散性活动：产物是”可能是什么”，不是”决定是什么”。
> **定义（defining）**：在候选之间画边界，使选定方案从”多义”变为”单义”、从”漂浮”变为”确定”的活动：把可能性收敛为一个被承诺的方案。

| | 构思 | 定义 | 设计开发 |
|---|---|---|---|
| 一句话 | 打开可能性 | 终结漂浮 | 转换到底 |
| 干什么 | 发散：生成至少两个候选 | 收敛：画边界、挑定一个 | 把需求转化为更详细的需求 |
| 产物 | 候选清单 | 选定的方案 | 更详细的需求（规格） |
| 位置 | 内·发散段 | 内·收敛段 | 整体 |

设计开发”把需求转化为更详细的需求”，转化不是一步到位的：先想（构思：这个需求有哪些”怎么做”的候选），再选（定义：在候选之间画边界、挑定一个）。发散在前、收敛在后，合起来才是”转化”。

所以”设计开发包含构思与定义”，是包含关系，不是三个并列的流程。这落到第 6 章的结对六步，就是③出方案与⑤做裁决；落到第 11 章的架构活动动词链，就是”每一层都在执行同一个 §3.3.11”的内芯，每一层设计开发内部，都重复这一轮构思、定义、再细化。

> 术语注：构思与定义这两个动词取自 ISO/IEC/IEEE 42010:2022 架构工作（architecting）§3.1 的动词串（正本中译作”构想”），**上面两条定义句是本书的梳理**；“设计”一词有宽窄两义（宽义是装下构思与定义的容器，狭义是 15288:2023 §6.4.5 的向下细化），与 ISO 9000:2026 §3.3.11 的”设计开发”在这里按整体与部分对接：“设计开发”是整体，构思与定义是它内部的两段，详见第 11 章。

设计开发是递归的，落地时每一级都是一对”需求—方案”的结对，走完六步：接需求、验就位、出方案、对需求、做裁决、签定稿（见 §6.4.1）。

### 3.2.6 质量：固有特性满足需求的程度

第三个词，也是全章最容易被误解的一个。

> **质量（quality）**：客体（§3.5.3）的一组固有特性（§3.10.1）满足需求（§3.5.1）的程度（ISO 9000:2026 §3.5.2）。

三个关键词，一个都不能松。

**固有特性**——存在于客体本身的：功能、性能、材质。“固有”与”赋予”相对，是指存在于客体之中的特性（§3.5.2 注 2）；赋予特性不算——正本举的例是”某客体的价格”（§3.10.2 注 2），品牌、交付时间同属此类，后两例是本书的扩举。

这意味着设计开发的范围也被”固有”框住；价格、交付周期是其他过程的事，设计开发管不着，也不该管。但成本是另一回事：成本作为”经济可承受性”需求，仍会进入设计输入（见 §5.4.8）；管不着的是”价格”这个赋予特性，成本需求本身要接。

文正在评审会上比”精度、保温时长、重量”，这些是固有特性，比对了；但要是比”价格比竞品便宜”，那不是质量，是经营。镜照一眼恒信：合同审批系统的固有特性是查询响应时间、正确性；“用的是哪家平台”“上线时间”是赋予特性，设计开发管不着。文正式的”比参数”，在 IT 世界里换成”比平台名气、比上线速度”，一样比错了地方。

**满足**——符合性判定，不是数量比。特性达到需求规定的水平就是满足，达不到就是不满足。这是一次判定，不是一道除法。质量问的是”结果在标准上的位置”，不是”标准除以结果”，两边不同质，一个在客体里，一个在客体外，除不了。

**程度**——差、好、优秀，形容词，不是数字（§3.5.2 注 1）。质量是被认定出来的，不是被计算出来的，靠验证与确认拿客观证据判定。

这里藏着全书的一个方法时刻：**判据主义**——不给没有判据的定义（第 1 章 §1.8.3）。质量的定义，恰恰把”质量好”从一个主观印象，变成了一条可检验的判据：特性满足需求的程度。程度可以用客观证据认定（测量、比对、检验），因此”质量好不好”可以争论、可以复核、可以裁决。这不是给质量泼冷水，这是让质量第一次成为工程上可用的判据。整章后面每一次划线，用的都是它。

还有一处常被忽略：质量定义里的”需求”是广义的，明示、隐含、必须履行三种形态并列（§3.5.1），规定需求只是它的子集。定义这一层，质量已经把隐含需求罩进去了；但要衡量质量，就必须落到可衡量的规定需求上；隐含需求不显性化成文，就验不着、量不出。这个”定义宽、衡量窄”的落差，正是需求必须显性化的根源（第 5 章展开）。

“质量”这个词的来处，同样是一次漂移：quality ← 拉丁 qualitas（“如何”）。它原指”档次、用料”，好东西就是好质量；ISO 9000 把它纠正为”特性满足需求的程度”，与档次无关（呼应第 1 章 §1.8.3）。这一漂，是这一章要立的最大一个反转。

### 3.2.7 概念卡片

需求、设计开发、质量都亮过相了，档案收在这里，按四问查：

**概念卡片 · 需求**

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 明示的、通常隐含的或必须履行的需求或期望（ISO 9000:2026 §3.5.1）——定需求，定义”要什么” |
| **它不是什么** | ⑥⑦ | 对立：点子、创意——不表述为需求就进不了设计开发的流程；辨析：需要是内容里的硬性诉求、期望是内容里的软性诉求、需求是内容乘呈现方式；需求不等于目标，也不等于目的 |
| **它从哪来** | ②③⑨ | requirement ← 拉丁 requirere（要求、寻求）；“需求”原指”要的东西”，后漂移出明示、隐含、必须履行三种形态（第 1 章 §1.8.3）；ISO 9000 把它固化为标准定义 |
| **它到哪去** | ④⑤⑧ | 起点与指引：需求变化是设计开发的输入；原型、测试、评审都围绕已识别 |

## 概念卡片 · 设计开发

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 将某一客体的需求转换为该客体更详细需求的一组过程（ISO 9000:2026 §3.3.11）——逐级出规格，直到可实现 |
| **它不是什么** | ⑥⑦ | 对立：走流程——输入输出同样详细，就是没发生设计开发；辨析：设计、开发、设计开发三个词的关系；需求开发（ISO 9001:2015 §8.2）与设计开发（§8.3）的边界看”特性有没有被规定”（见本章 §3.3.3）；构思与定义是设计开发内部的发散段与收敛段，不是并列的流程 |
| **它从哪来** | ②③⑨ | design ← 拉丁 designare（标注、指明）；“设计开发”由”设计”“开发”两个词组成，但不是两件事 |
| **它到哪去** | ④⑤⑧ | 终点判据：它能要什么样的结果就行，不必规定“怎么做” |

## 概念卡片 · 质量

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 客体的一组固有特性满足需求的程度（ISO 9000:2026 §3.5.2）——判满足，判出满足到什么程度 |
| **它不是什么** | ⑥⑦ | 对立：等级——等级高不等于质量好；辨析：符合性回答”满足了吗”，质量回答”满足到什么程度”，等级回答”按哪套需求分类”（见本章 §3.3.6） |
| **它从哪来** | ②③⑨ | quality ← 拉丁 qualitas（“如何”）；“质量”从日常的”档次、用料”漂移到”符合需求的程度”——ISO 9000 纠正了”好东西就是好质量”的直觉（第 1 章 §1.8.2） |
| **它到哪去** | 等级不是质量 | 质量等于被满足要求的程度；在检验、验证和确认 |

## 案例进展①｜没做调研的文正

冰链科技立项 M3 的第七周，产品、研发、销售开联合评审。

文正拿着方案书进场，标题是”M3：行业精度第一的冷链温控箱”。他开口第一句：“客户要的是精度——这条需求我们直接做到行业第一。”

销售小冯打断他：“谁告诉你的？上周我拜访了三个省疾控中心，他们问的第一句话全是——能不能塞进我们现有的疫苗运输车？你的 M3 尺寸比现在那款大了一圈，塞不进去。这条需求合同里没写，你也没问过。”

会议室安静了。

文正愣住：“……合同里没写这条。”

“是没写，”小冯说，“是没写。但这是他们所有人都在问的——按 §3.5.1，通常隐含的也是需求。你立项前做过市场调研吗？”

文正没做过。他的起点是一个点子，“把精度做上去，就能赢”，跳过了研究，直接进了设计开发。七周后，方案书被推倒重来，重新做尺寸、重新评估保温层厚度、重新谈供应链。

这一轮返工，损失的七周，就是起点没收好的全部代价。设计开发转化的是需求，起点给错了需求，转化得越彻底，浪费得越彻底。

---

## 3.3 需求、设计开发、质量不是什么

> 这一节把相邻两个词之间最容易混的地方分开。概念靠正例活，靠反例定型：起点那处边界在需求与设计开发之间，终点那处在设计开发与质量之间，参照那处在需求与质量之间。

### 3.3.1 设计开发不始于点子

边界最容易打架的地方是交接处。第一处在需求与设计开发的交接处：设计开发从什么时候开始?

先说清”开始之前”。设计开发要拿到被研究确认过的需求，才谈得上开始：

> 点子、灵感、想法 →（市场调研、可行性分析、技术预研、客户需求识别）→ 确定的需求 → 设计开发开始

为什么必须插这一步?因为设计开发的输入是需求，而输入的需求（按§3.3.11 注 1）往往是研究的成果。一个没经过研究确认的点子，设计开发会把它老老实实做成一个”错误的需求”的完美实现，它不会自己发现方向错了，它只管把给它的东西转化到底。

很多项目”设计得很辛苦、产品没人要”，根源不在终点没管好，而在起点没收好：错的不是施工，是拿到的需求本身就错了。

### 3.3.2 起点的判据：需求要可转化

那”开始”这一步怎么算过?设计开发的起点，是一份需求规格被正式确立为设计输入的时刻。而这份需求规格，必须先满足”可转化”判据，四条前提，一条都不能松：

1. 完整——功能、性能，以及由产品和服务性质导致的潜在失效后果，一个不缺。M3 不只是”能制冷”，还包括”制冷失效时自动告警”“断电后保温时长”“结霜影响”这些潜在失效的处置。缺一项，设计开发就可能在错误的方向上越走越远。

2. 无歧义——每条需求只能有一种理解。“精度要高”不是需求，“箱温波动不超过 ±0.5℃”才是。歧义就是留给未来的争吵。
3. 不冲突——需求之间自洽。“重量轻 2 公斤”和”保温 72 小时”可能打架（保温靠保温层，保温层厚则重），这要在需求阶段协调掉，而不是等设计开发做完再发现。
4. 隐含需求已识别——最容易被忽略、却最决定规格对不对的一条。它没写进规格，却决定规格对不对；它不会自己出现，需要主动去挖。

这四条前提不是拍脑袋凑出来的标准，而是”设计开发就是需求转化”这一定义想成立所必须具备的条件。逐条反推就知道它们躲不开：

- 需求有缺口，转化在残缺的输入上开工，输出必然残缺——转化不可靠；
- 需求有歧义，同一份输入生出多种理解——转化没有确定结果；
- 需求冲突，满足 A 就违反 B——转化无解；
- 隐含需求没识别出来，方向就错了，验证全绿、确认全红——转化白做。

所以可转化判据的正当性不来自任何更高层的东西，就来自定义本身。标准（ISO 9001:2015 §8.3.3）做的，只是把这四条前提写成了条款。

> 出处提醒：ISO 9001 在库内没有正本，属**库外来源**。本章引它的条款号，**首次出现处带标准名与年份（ISO 9001:2015），其**

> **后同一节内可简写；**表格与对白各自独立起算，**文字转自报告库** 01 号 v1.8（ISO 9001:2015 各条）；27 号 v1.2 附 A 是 ISO 9000 的术语换号表（138 条），不涉 ISO 9001**。库内可直核的相邻处是 ISO 9000:2026 §3.3.11 注 1 与 §3.5.1 注 2。

> 起点没收好，转化的是错误需求——终点越完美，偏离越远。

### 3.3.3 需求开发的边界：特性有没有被规定

管理评审时一定会遇到一个问题：市场调研、可行性研究、URS（用户需求说明书）这些文档的开发，算不算设计开发？

概念上：需求开发也算设计开发，而且同构。§3.3.11 的定义不挑阶段：客户说”我要一台能低温运行的小型设备”，需求开发把它变成”工作温度 -40℃、体积不超过 0.5m³、……“，这本身就是一次”需求变详细”。需求开发也是需求转化，同样具备 §3.3.11 的定义特征。它和设计开发是同一种认知活动的不同阶段，只是转化发生在更靠前、更粗的层级。

标准组织上：两者可合可分。ISO 9001:2015 把需求确定（§8.2：顾客沟通、需求确定、需求评审）和设计开发（§8.3）分成两条，不是因为”需求开发不是需求转化”，而是职责的组织方式不同：§8.2 管”要什么”（面向顾客与市场），§8.3 管”怎么实现”（面向技术与实现）。复杂组织常把需求开发作为设计开发的第一阶段纳入 §8.3；简单组织放在 §8.2 即可。两种策划都合规。

实用判据：看特性有没有被规定。这是把质量定义（§3.5.2”客体的一组固有特性”）用起来的硬判据：

> 需求开发阶段，客体还没确立，特性压根不存在，质量没有承载体；设计开发阶段才开始”把需求落到客体的固有特性上”。特性还没开始规定，是需求开发（ISO 9001:2015 §8.2）；特性开始被规定，才是设计开发（§8.3）。需求规格就是那道分界。

必须补的一句提醒：不管这些文档（市场调研、可行性研究、URS）归§8.2 还是 §8.3 的第一阶段，它们都是设计输入的来源，直接影响产品质量特性，审核时”这些文档不在受控范围”是过不去的。它们可以不属于”设计开发”，但不能不受控、不可追溯。文正的”M3 立项调研”如果能受控存档，案例进展①那七周返工至少不会连个”当时为什么这么做”的记录都没有。

### 3.3.4 终点的判据：为什么不是可验证

第一处边界在起点，第二处在设计开发与质量的交接处：设计开发什么时候算结束？

绝大多数团队（包括冰链科技）说”技术方案做完就结束了”。但结论不能拍脑袋，得把候选终点一个一个删掉：

| 候选终点 | 为什么被否定 |
| --- | --- |
| 确认通过 | 错位：确认的判定对象是已可使用的客体、发生在设计开发形式结束之后很久；而设计开发这一段锚定的是”需求” |
| 责任移交点 | 只是制度动作（设计基线发布），不是定义内部的条件 |
| 可验证 | 逻辑倒置——详见下面 |
| 可实现 | 定义内部唯一自然的终止条件 |

“可验证”为什么不对，这是全书最要紧的一处推理，值得单独看。

验证（verification）的定义是：通过提供客观证据，证实规定的需求已得到满足（ISO 9000:2026 §3.11.12）。注意：验证的对照物是规定需求，而规定需求正是设计开发的输出。也就是说：

> 验证能发生，前提是设计开发已经结束（已经输出了规定需求）。拿”可验证”当设计开发的终点，等于让设计开发结束在验证的起跑线上——循环倒置。

更致命的是第二层：验证的注 1 说，客观证据可以是检验的结果，也可以是其他确定形式的结果，如进行替代计算或评审文件（§3.11.12 注 1）。那么几乎所有写清楚的需求都”可验证”，这条判据区分不了完成与未完成。你说”需求写完了所以可验证了所以设计开发完了”，那到底是”写完了”还是”可验证”？绕回去了。

所以终点只能落在”可实现”：

> 设计开发止于”可实现”：需求被细化到足以支撑客体实现、生产和服务提供可直接执行的程度（输出成为完整的规定需求）。验证与确认都在终点之后，各对一把尺子。验证对照这份规定需求；确认对照”针对特定预期用途或应用的需求”（ISO 9000:2026 §3.11.14 正本中文用字）。“可实现”是入场券（必要条件），验证通过是及格线（符合），确认通过才算在真实场景里站住——但终点在验证之前。

顺带把确认的位置摆正：它的判定对象是已可使用的客体，对照物是”针对特定预期用途或应用的需求”（§3.11.14），使用条件可以真实也可以模拟（§3.11.14 注 3）。它不是设计开发内部的终点，而是终点之后的终审。

### 3.3.5 终点的样子：方案配齐，指令可执行

“可实现”在实物产品上是什么样子？下游各个环节要用的方案，全部完成：

**技术方案 · 物料采购方案 · 制造方案（工艺）· 测试方案**

**交付方案 · 部署方案 · 操作方案 · 运维方案**

这些方案不是彼此独立的几件事，是一样东西的不同侧面，“规定需求”按下游环节分配下去：采购照着买、生产照着做、检验照着测、实施照着装、用户照着用、维护照着修。每个方案，都是给一个下游环节的”可实现需求”。这正是”满足后续产品和服务提供过程的需要”（ISO 9001:2015 §8.3.5 b）的完整展开。

> 术语注：“止于可实现”与这份方案清单是本书的主张——本书对 ISO 9001:2015 §8.3.5 b（设计输出应满足后续产品和服务提供过程的需要）的操作化，不是 ISO 原文的列举。标准给的是判据方向；拆成哪几个方案、怎么命名，由组织按自己的下游环节裁剪。

每个方案对应的条款，都对着下游过程的需求：

| 方案 | 对应条款 | 下游环节 |
| --- | --- | --- |
| 技术方案 | ISO 9001:2015 §8.3.5 d：规定对预期目的至关重要的特性 | 设计本身 |
| 物料采购方案 | ISO 9001:2015 §8.4：外部提供的需求 | 采购 |
| 制造方案（工艺） | ISO 9001:2015 §8.5.1：生产和服务提供的受控条件 | 生产 |
| 测试方案 | ISO 9001:2015 §8.3.5 c：监视和测量需求 + 接收准则 | 检验与验证 |
| 交付、部署、操作、运维 | ISO 9001:2015 §8.5.1 及生存周期各环节 | 交付、实施、使用、维护 |

退役与回收不在默认清单内：本章未在库内核到 ISO 9001:2015 的退役环节对应条款；退役处置属 ISO/IEC/IEEE 15288:2023 的处置过

程（§6.4.14）。它的需求侧由第 5 章的可回收性需求承接（§5.2.5），方案侧可以按下游环节增删。

还有一处用词要精确：“方案”不是”结果”。测试方案属于设计开发：它规定测什么、判据是什么；测试的执行和测试结论属于验证。停在”方案完成”这一步，正好守住”设计开发止于可实现”这条线。

> 终点由”最后一个配齐的方案”决定，不由”第一个完成的技术方案”决定。

为什么技术方案不够?因为技术方案只回答”客体本身长什么样、怎么实现”，它是这些方案里最先完成、最显眼的一个，但只是一个侧面。只做到技术方案就宣布终点，后果是下游连环停摆：

- 采购没有物料规格和接收准则——采购停在原地；
- 生产没有工艺依据——产线开不了工；
- 测试没有判据——验证无法启动，李旭那句”测试判据是什么”没人能答；
- 交付、部署、操作、运维没有指令——产品出门即失联，客户拿到手不知道怎么装、怎么用、怎么修。

镜照一眼软件世界。恒信集团的合同审批系统，这些方案长这样：技术方案、数据方案、接口方案（与法务、财务系统对接）、测试方案（验收标准）、部署方案、操作方案、运维方案、安全方案。

一个不少，只是名字从”物料、制造”换成了”数据、接口、部署”。凡是只做了技术方案就宣布”设计完成”的软件项目，和只有技术方案的 M3 一样，都会在下游连环停摆。

“可实现”这个词容易引起误会，好像设计开发完了，东西就能跑起来。所以要分清三层：

| 层次 | 谁给的 | 性质 |
| --- | --- | --- |
| 原理可实现 | 技术方案 | 纸上走得通 |
| 方案可实现 | 方案全部配齐 | 设计开发的终点——从”原理可行”跨到”可执行指令齐备” |
| 事实可实现 | 验证与确认 | 终点之后——“能不能真正实现并运行”由它们裁决 |

落到 M3：双压缩机冗余在纸面上完全成立，这是原理可实现；方案配齐、可执行指令齐全，这是方案可实现，也就是设计开发的终点；真造出样机、在真实冷链运输里跑通验证与确认，才是事实可实现，而这一层，冰链连边都没摸到。从”纸面走得通”到”真能跑”，中间隔着整个验证与确认。

设计开发交付的不是事实，而是一份声明，一份可信的可实现声明：方案齐备、逻辑自洽，靠评审、样机、仿真来提高把握，但最终裁决权在终点之后的验证与确认。守住这条界线，设计开发就不会把”我们觉得能做”当成”我们做完了”。

这就是第二处边界的全部内容：设计开发把裁决权交在终点之外，它只负责把规格做到”可实现”，把”到底行不行”交给质量判定。各管一段，边界在这里。

### 3.3.6 参照的判据：等级不是质量

第三处在需求与质量的交接处：中文里最容易和”质量”混的，是”等级”和”符合性”。符合性、质量、等级不是同一个意思的不同说法，而是回答三个不同的问题：

| 词 | 回答的问题 | 形态 | M3 的例子 |
|---|---|---|---|
| 符合性 conformity | 满足了吗? | 二者取一（符合或不符合） | 箱温波动是否在 ±0.5℃ 以内? |
| 质量 quality | 满足到什么程度? | 程度（差、好、优秀） | 一组特性综合下来，好不好? |
| 等级 grade | 按哪套需求分类? | 分级（高档、低档） | M3 是高配冷链箱，还是经济型? |

三者都以”需求”为对照，而需求是设计开发定出来的。等级的定义在这里给全：对功能用途相同但需求不同的客体所做的类别或级别划分（§3.5.4）。

关键的一处反直觉：等级高不等于质量好。高档车可以是差质量（功能设计有缺陷），低档车可以是好质量（按它自己的需求做到了极致）。因为等级是”按不同需求分类”，质量是”满足各自需求的程度”。M3 把精度做到行业第一，它只是等级定得高；如果需求只要 ±0.5℃，那么超出的部分对”质量”没有贡献。

这条边界走到头，就得到过度设计的一条硬判据：特性超出需求范围的，不是质量加分，是过度设计，只加成本，不加质量。这给”砍需求”提供了一条干净的标准：砍过度设计，是唯一”既省时间又不伤质量”的操作，不是牺牲质量，是给质量做减法。

回到冰链那场评审会：文正比的”±0.3℃ 对 ±0.5℃“是等级之争；要问质量，得先问需求写的是 ±0.5℃ 还是 ±0.3℃。需求范围之外的那 0.2℃，就是过度设计的起点。

镜照一眼恒信：合同审批系统的需求是”合同在三个部门间流转审批”；设计开发把这条需求做进审批流设计；质量读的是查询响应时间、审批时效、正确性，同一套”定需求、出规格、判满足”，只是从”温度精度”换成了”审批流转”。

一个硬件、一个软件，起点、终点、参照这三处边界画开的方式一模一样。这正是本书用两条线讲故事的原因：这套概念不挑行业。

起点、终点、参照画完，“判据主义”的手法也亮了相，概念靠正例活、靠反例定型：等级靠”高档车可以是差质量”定型，过度设计靠”超出需求范围只加成本”定型，终点靠”可验证是循环倒置”定型。每一处边界，都有一条可检验的判据收口。

## 案例进展②｜只有技术方案的 M3

时间回到章首那场评审会之后。

M3 的技术方案确实漂亮：双压缩机冗余、±0.3℃ 恒温控制、全数字监控。文正拿着它宣布”设计完成”，产品进入”等待量产”。

量产准备会上，采购先问：“压缩机我们按什么型号采？物料采购方案的接收准则是什么？”——采购方案：没有。

制造与供应链的逸人顺着技术方案往下追。他翻到装配那页：“装配工艺规程呢？产线上按哪份文件装、用什么工装？设计开发止于可实现，这两条不落纸，规格就没走完。”文正没接上。他又翻到测试那节：“低温密封测试的接收准则写在哪？不合格判到哪一档？”还是没接

上。他敲着纸面说了最后一句：“本章 §3.3.5 说终点是方案配齐——制造方案和测试方案都不在，这条线没走完。”

李旭接着问：“验证的对照物是规定需求。规定需求在哪份文件里？接收准则写在哪份测试方案里？”——测试方案：没有。他章首那句被淹没的问话，在这里第二次冒出来，这次没人能再忽略。

交付团队问：“客户拿到箱子，怎么装、怎么部署、怎么培训？这几条下游需求，谁写？”——交付方案、部署方案、操作方案：都没有。

云端软件的董勐问：“追溯平台的数据谁备份、App 断连了谁发现？运维方案里这两条需求写在哪儿？”——运维方案：没有。

五轮追问下来，八个方案里七个”没有”。会议室从”技术方案真漂亮”的气氛，跌进”我们离能交付还差得远”的沉默。

水忠，那个一直不怎么说话的嵌入式硬件工程师，说了句：“我们以为设计开发已经结束了。其实按 §3.3.11，它连一半都没走完。”

设计开发止于”可实现”，方案配齐的那一刻。八个方案里冰链只完成了一个（技术方案），就宣布了胜利。

> 还有哪个环节会在执行时反问”这到底怎么做”？——有，设计开发就没结束。

---

## 3.4 需求、设计开发、质量怎么用

> 这一节把定需求、出规格、判满足这条路径用起来：先读出结果，再解开冲突，最后看它们为什么互相定义、拆不掉。

### 3.4.1 程度怎么认定：从特性值到结论

上一节把”程度”这个词说清了：它是认定出来的，不是计算出来的。

认定分三步，一步不缺：

**第一步　确定特性值（§3.11.1）　　查明一项或多项特性及其特性值**

**手段：测量（定量）、检验（符合性）、试验、评审、计算**

**产出：客观证据（§3.8.6）**

**第二步　与需求比对　　判符合性（§3.5.9）**

**用接收准则——设计输出 ISO 9001:2015 §8.3.5 c 里提前写好的判定标准**

**第三步　认定满足程度　　得出质量（§3.5.2）**

**综合一组特性的符合情况，认定程度：差、好、优秀**

**动作上就是验证（§3.11.12）与确认（§3.11.14）**

每一处锚点都缺不得：工具在 §3.11.1 确定、§3.8.6 客观证据，判定在 §3.5.9 符合性、结论在 §3.5.2 质量；动作落在 §3.11.12 验证与 §3.11.14 确认；对照物是 §8.3.5 的接收准则，写在测试方案里，它是终点那批方案中的一个。缺了测试方案里的接收准则，程度就出不来。

这里有一处重要的不对称：

· 单项是二者取一——每一条特性：符合，或者不符合；

· 整体是程度——综合一组特性：差、好、优秀。

整体程度按特性的重要程度综合评价，关键特性优先（§8.3.5 d 说的”对预期目的至关重要的特性”）。这是第 8 章评审与质量判定的底子，这里先立起来。

“接收准则”落到纸面是什么样子？M3 的测试方案里必须写清楚：箱温波动在 ±0.5℃ 以内判”符合”还是”不符合”，用什么设备测、测多久、抽多少样本，判定结论写在哪份记录上。没有这几样，程度就出不来：你说 M3”质量好”，拿什么证据？哪条特性符合了、哪条没符合、符合到什么程度？

接收准则就是那份提前写好的判定标准，它让”质量好”从一句感觉，变成一次可复核的认定。这也正是测试方案必须占终点一席的原因：没有接收准则，质量永远只能”感觉”，不能”认定”。

镜照一眼恒信：合同审批的”查询响应”怎么读？测试方案里写清楚：在多大数据量下测、多少并发、响应超过几秒判”不符合”、结论记在哪张记录上。同一套三步，只是把”温度”换成了”响应”。

### 3.4.2 分值为什么靠不住

既然”程度”是形容词，那能不能给它打分，比如 M3”质量 85 分”？

标准这一层：没有。符合性、质量、等级三个近亲都不是分值，而且标准通篇没有给质量一个单点分值：特性是多维的、彼此不同质，加不成一个数；隐含需求无法完整数值化；§3.5.2 注 1 也明说，质量用形容词修饰。

组织这一层：分值可以造，但它是组织的度量，不是标准概念。打分等于定权重（特性重要程度）、单项评分（把接收准则映射成分数）、加权汇总，得到”85 分”。三个前提，缺一个分就是假的：需求完整（起点）、特性规定完整（终点）、证据可靠（验证）。

最要命的一条：关键特性不能被摊平；ISO 9001:2015 §8.3.5 d 说”对预期目的至关重要的特性”要优先。加权公式若把安全性、可

靠性这种一票否决的特性摊进平均数，安全项失了分，被”精度超强”补回来，结果还是 85 分。这个 85 分，就是用假象盖住了致命问题。分值的最大陷阱：它给了你一个精确的错误。

## 案例进展③｜平均分遮住的致命项

质量评审会上，李旭拿着刚按设计参数估算出来的评分表：“我给 M3 的质量打了 85 分——温度控制 92 分、保温 88 分、外壳做工 90 分、重量 70 分……”

水忠打断他：“权重怎么算的?”

李旭：“平均。”

水忠指着评分表一行小字：“你把’断电告警’放进去了吗?”

李旭：“放了。我把它当普通特性，权重和其他项一样，20 分里得 18 分。”

水忠：“断电告警是保命的。ISO 9001:2015 §8.3.5 d 说它是’对预期目的至关重要的特性’，不是平均数里的一项。它不合格，产品就不该上市。你把它摊进平均数，安全项失 2 分，被’温度控制 92 分’补回来，最后 85 分——这 85 分，是在替一个致命缺陷打掩护。”

会议室安静了。水忠继续说：“要么给关键特性设一票否决，要么用加权而不是平均。否则这个分数，就是给了你一个精确的错误。”

李旭默默改评分规则。他在心里记下：打分看起来比”程度”精确，但精确的前提是把权重量对，量错了，精确就是假象。

### 3.4.3 质量与进度：改需求，不改质量

读出结果之后，一定会撞上一个绕不开的冲突：质量与进度。这场冲突是一场被误判的取舍。

先拆掉两个站不住的前提。

第一个：质量不是时间的函数。质量等于特性满足需求的程度，和花多少时间没有必然关系，你不能说”多给我三个月，质量就自动变好”。

第二个，也是更深的那个：进度本身就是一种需求。据 01 号 v1.8 转引的 ISO 9001:2015 §8.2.2：交付时间和交付条件也是顾客需求的一部分。所以”质量与进度”本质是同一份需求集合内部打架：性能与功能需求，对上交付时间需求。不是两个系统在拔河，是一份需求清单自己起了内讧。

解法不是二选一，是重新平衡这份需求集合。

第一步：判真伪。一半以上的”质量进度冲突”是估算失真、计划虚报、范围蔓延造成的假冲突，进度算错了，却怪质量太慢。假冲突，改计划，不动质量。M3 的”八周做不完”里，有多少是因为七周返工浪费的？那一部分不是质量的问题，是起点欠下的账。

第二步：用需求谈判，不用质量谈判。调整的是需求集合（哪条可降、可删、可延），不是”质量原则”。必须走正式变更（ISO 9001:2015 §8.2.4 需求的更改、§8.3.6 设计更改）：书面、评审、通知利益相关方。口头”差不多就行”是最危险的变更形式：需求被悄悄换掉，验证与确认全在跟一个没被承认的靶子打。把”实时推送”降成”两小时同步”这类降级，必须落在变更记录里，而不是散会后的记忆里。

第三步：分清可妥协与不可妥协。

- 不可动：必须履行的需求（安全、法规，§3.5.1 里”必须履行**的**”那一类；法规来源见 §3.5.5／§3.5.8）、关键特性（§8.3.5 d 对预期目的至关重要的那些）。“断电告警”就在这一类；

· 可动：非关键特性、性能上的富余、过度设计（可砍，见本章 §3.3.6）。

第四步：度量环节可缩不可砍；最蠢的赶进度方式是”先交付，验证确认后补”。验证与确认是质量的度量，砍掉等于不度量就宣布合格，风险全部后移成现场返工（失败成本是质量成本里最贵的）。正确的做法是：把验证范围缩到关键特性，而不是把验证整个砍掉，只验关键的，别全验；但不验，不行。

三条禁忌，一条比一条贵：

1. 砍验证与确认赶进度——最贵，风险后移，返工爆炸；
2. 口头改需求——最危险，需求被偷换，整条定义链失效；
3. 把”降需求”说成”降质量”——概念错。等级（§3.5.4）不等于质量（§3.5.2）：降低需求水平（砍非关键）是合法变更；交付不满足已承诺需求的东西，才是真降质量。

正道：与利益相关方一起剪裁需求。

> 单方面砍需求叫偷换，和客户一起剪裁才叫变更，中间差着整个需求的所有权。

为什么”一起”是硬要求？四条理由，层层递进：

1. 需求的所有权在利益相关方手里。需求（§3.5.1）是 need 或 expectation 的表述，是利益相关方（尤其客户）的资产，组织只是受托转化。单方面剪裁等于拿别人的东西替别人做主：剪掉的那块可能恰好是客户最在乎的隐含需求——他没说，但你剪了，事后必炸；

2. 剪裁改变的是验证的靶子。验证（§3.11.12）对照规定需求——需求一剪，靶子就变了。不和利益相关方一起剪，验证就是在打一个自己偷偷移动的靶子；
3. 剪裁的正确动作是排序，不是删减。让利益相关方排优先级：命根子（不可动）、想要的（可降）、贪多的（可剪）；
4. 剪裁要留痕，剪下来的部分要存档。走 §8.2.4 与 §8.3.6 的正式变更：剪了什么、为什么剪、谁决定的、哪天恢复，全部记录。剪掉的需求不是消失了，是挂账：这一轮进度解了，下一轮迭代或改型时就是现成的输入。剪裁是暂存，不是销毁。

把需求、设计开发、质量放回原位看：质量等于特性满足需求的程度，需求集合变了，质量判定跟着变。所以”解进度冲突”从来不是牺牲质量，而是和利益相关方一起，重新把参照摆正。

## 案例进展④｜进度不够，客户被迫说真话

M3 的量产进度亮起红灯，逼出了第一次真正的剪裁：距离向省疾控中心首批交付还剩八周，可制造方案、测试方案都还没影。

逸人把产线的日历摊开：“装配工艺、低温密封测试、工装夹具，排进产线就要六周；压缩机供应商谈了两家，都卡在’接收准则是什么’。这两份方案一天不定，产线要的可实现需求就一天没落纸。”

文正这才明白，“八周做不完”不是缺时间，是缺制造和测试两个方案，量产准备会上那七个”没有”里的两个。

会议室里，对面坐着疾控中心的刘主任，这位”客户”其实是采购方代表。

文正硬着头皮开口：“刘主任，进度不够了。我们可能得砍掉一部分需求才能按时交付。”

刘主任：“砍哪部分？”

文正：“比如把’箱门单手打开’这条需求……”

刘主任打断他：“不行。护士经常一手抱孩子一手开箱——这条是合同里明示的，也是保命的一条，不能砍。”

文正：“那’App 实时温度推送’这条需求呢？”

刘主任想了想：“这个可以降级——不用实时推送，两小时同步一次就行。但断电告警必须保留，那是保命的——按 §3.5.1 的分类，它属于必须履行的需求，不在可谈的范围里。”

文正：“还有’重量再轻 2 公斤’这条需求？”

刘主任笑了：“那条本来就是你们销售自己吹出来的，我从没要过——超出需求范围的，就是过度设计，砍。”

三十分钟，需求清单重新排了序：命根子（断电告警、箱门单手开）、想要的（实时推送改成定时同步）、贪多的（重量减 2 公斤、宣传用的”行业精度第一”）。李旭在旁边记录，每一条都写下谁决定、为什么、何时恢复。

散会后，李旭对文正说：“你知道吗，我们开产品会开了七周，问客户’你还有什么需求’，刘主任每次都说’没有了’。今天让他砍需求，他十分钟就把真正在意的全说出来了。”

进度冲突，会把隐含需求逼出来。平时问”你还有什么需求”他答不出；你说”进度不够，必须砍一样，砍哪个”，他立刻答得出。

这也是这一章最实在的一课：文正一直以为”质量与进度”是两件事在拔河，真坐下来和客户一起排需求，才发现那是同一份需求集合的内部排序，参照重新摆正。

### 3.4.4 互相定义，所以拆不掉

需求、设计开发、质量各自是什么，起点、终点、参照的边界各画在哪，一场冲突怎么解，最后一步，是把它们放回一起看：

- 需求（§3.5.1）是输入，定义”要什么”；
- 设计开发（§3.3.11）是转化，把需求变成特性化的规定需求；
- 质量（§3.5.2）是判定，认定固有特性满足需求的程度。

> 一个产品开发流程，无非是：定需求 → 出规格 → 判满足。

两条案例线在这条路径上完全同构：冰链定需求（-20℃~8℃ 宽范围、断链即报废的需求）、出规格（方案配齐）、判满足（箱温精度的符合性）；恒信定需求（合同三部门审批流转）、出规格（接口、数据、部署方案）、判满足（查询响应、审批时效）。一个是硬件，一个是软件，这条路径纹丝不动，这正是这套概念不挑行业的证据，也是全书用两条线讲故事的原因。

说它们撑得起”核心概念”这个位置，得看证据。

证据一：定义互相引用，拆不掉。设计开发（§3.3.11）引用需求（§3.5.1）；质量（§3.5.2）引用需求和特性（§3.10.1）；§3.3.11 注 1 又说需求”通常按特性来定义”。抽掉任何一个，另外两个都悬空，自洽，且不可拆。

证据二：角色不可替代。需求管输入、设计开发管转化、质量管判定，三个角色各司其职，谁也替不了谁。

证据三：能推出领域里现有的全部方法。

- 需求管理、需求评审、变更控制 ← 从”需求”推出；

· 设计输入与设计输出、评审、验证与确认 ← 从”设计开发 + 质量判定”推出；

· 质量控制、质量策划、质量改进 ← 从”质量”推出。

再往外看，主流方法也各有各的位置：

| 类别 | 方法 | 与需求、设计开发、质量的关系 |
| --- | --- | --- |
| 直接展开 | 系统工程（V 模型）、软件工程、硬件工程 | 这条路径的工程化形态 |
| 流程框架 | IPD | 这条路径的流程化、组织化形态 |
| 运行容器 | 过程方法 | 这条路径的运行方式——设计开发本身就是一组过程 |
| 扩展学科 | 可靠性工程、安全性工程 | 质量里”随时间、随失效”的专门化 |

逐个检验。V 模型为什么是 V？因为需求从哪一级分解下去，验证就必须从哪一级收回来：左半边是需求逐级转化，右半边是质量判定逐级回收。

IPD 为什么必须设技术评审点与决策检查点？因为质量判定不能等到最后才看，要在每个关口用”特性满足需求的程度”检查一遍。需求为什么要有追溯矩阵？因为需求是质量判定的参照，参照断了，判定就无据。变更为什么要控制？因为改需求就是改参照，不改参照，验证就是打移动靶。

这些”为什么”都能从需求、设计开发、质量推出来，这就是”核心概念”和”几个相关概念”的区别。

但这里必须补一句刹车，否则前面那些”为什么”就被说满了：需求、设计开发、质量长不出技术本体。

> 工程 = 概念这一侧的方法（需求、设计开发、质量，管”要什么、怎么证明做对”）× 技术本体（物理、材料、算法、工艺，管”怎么做”）。

需求、设计开发、质量能推出的是”如何组织做什么”和”如何证明做对了”的全部方法；它长不出牛顿定律，也长不出快速排序。判别标准只有一条，看问题的主语是谁：

- 主语是”做不做得出、怎么做” → 技术本体（跟着客体走）；
- 主语是”做没做对、值不值得、怎么保证” → 概念这一侧（跟着需求与质量走）；
- 两者都在 → 两者杂交的产物。

真正的产品开发方法，几乎都是两侧杂交出来的，越靠中间，越值钱。

路径上还嵌着两个承重构件：客体（§3.5.3）和特性（§3.10.1）。没有客体，质量的定义（“客体的一组固有特性”）和设计开发的定义（“某一客体的需求”）无处安放；没有特性，需求落不到客体上，质量也无从度量。

需求、设计开发、质量的层级也不完全平级：三者同属核心概念这一层，其中需求与设计开发是根概念，质量是派生的枢纽——它被到处引用，却由需求与系统派生出来。最底下站着的是需求与设计开

发这两个根；质量架在其上，而把质量接回需求的那段转化，正是设计开发。

> 术语注：“需求与设计开发是根概念、质量是派生的枢纽”是本书对三者层级关系的凝练主张，不是任何标准的术语——ISO 9000 只分别定义它们，不划定层级；层级判定见第 1 章 §1.2.2 与 §1.7.4。

后面第 4 章到第 16 章，除了第 9 章回到”工程师”这个人的角色章，其余都是需求、设计开发、质量在不同方向上的展开；到第 17 章合起来落成一件真实的事，第 18 章回到那个把这条路径背在身上的人。

第 2 章把”工程”说清了：工程不是手艺，是可复现的方法。这一章的需求、设计开发、质量，就落在那层底子上；它们不是终点，是后面所有章节的出发点。

### 3.4.5 认知反转

回到周五下午那场评审会。文正拍着参数对比表说”符合性没有问题”。

现在我们知道，他说错了一个字：他想说的不是”质量高”，是”等级高”。M3 按更高的需求设计制造、精度行业第一，它是高档产品，等级高。但等级高不等于质量好：高档车可以是差质量，低档车可以是好质量。M3 的精度远超需求（±0.3℃ 对 ±0.5℃），对”质量”没有贡献，只贡献了成本和复杂度，那部分不是质量，是过度设计。

而真正决定 M3 成不成的，是另外两件事，一件在终点，一件在起点。

终点没守住。冰链团队只做了技术方案就宣布”设计完成”，采购、制造、测试、交付、部署、操作、运维的方案一个都没有。李旭那句被淹没的”测试判据是什么”，不是一句无关紧要的问话，它宣告设计开发没有结束。到案例进展②里它第二次冒出来，这次谁都无法再忽略。

起点没收好。文正跳过研究、从”把精度做上去”的点子直接开工，把七周浪费在一个没有对过需求的方向上。“箱体能塞进标准疫苗车”这种客户开口第一句就在问、合同里却没写的需求，反而在立项七周后才被销售带回来。

这一章其实给了三个反转：一个属于术语（等级不等于质量），一个属于流程（设计开发止于可实现），一个属于起点（研究到需求再到设计开发这条链不能跳）。它们都从同一个动作出发：

> 把一个你以为懂、其实各指各的词，摆到定义面前；把一条你以为是流程的路径，从起点到终点重新走一遍。

而画起点、终点、参照这三处边界、解这一场冲突用的工具，正是第 1 章那套方法里的**判据主义**，每个概念都给一条可检验的判据：质量等于特性满足需求的程度，设计开发止于可实现，需求要过”可转化”。需求、设计开发、质量从此合成一条能走的路径，“东西好不好”这一问也就有了可复核的依据。

用第 1 章的话说一句：对”需求”“设计开发”“质量”这三个词，你原来只是”见过”：背得出定义，说得出”质量好”“设计完成了”。

这一章把它们推到”拥有”：能答全四问（各是什么、不是什么、从哪来、往哪去），能用”定需求、出规格、判满足”把任何一个项目从头走到尾，能说出每一处边界的判据。能答全四问，才算拥有。

用第 1 章 §1.4.4 的尺子量一下：这一章把”需求”“设计开发”“质量”三个词从**第①级（表征）**推到了**第②级（图式）**。你现在看得出”定需求、出规格、判满足”这条路径怎么接起来，而不只是背得出三个定义。

## 小结

这一章是需求、设计开发、质量的入场，也是全书方法论的底子：定需求、出规格、判满足。第 2 章把”工程”说清了，它们就落在那一层上。

第一节把需求、设计开发、质量各自是什么说准：需求（明示、隐含、必须履行三种形态，还要和需要、期望、目的、目标分开）、设计开发（“更详细”是唯一判据，而且层层往下走）、质量（固有特性、满足、程度）。

第二节画开三处边界：起点（研究先于需求、“可转化”的四条前提、特性有没有被规定）、终点（止于可实现、方案配齐、声明不等于事实）、参照（等级不是质量、过度设计），判据主义在这里成了亲身体验。

第三节把路径用起来：读结果（从特性值到结论）、解冲突（质量与进度是同一份需求集合内部打架，与利益相关方一起剪裁）、合起来看（定义互相引用、角色不可替代、能推出领域里的全部方法，但长不出技术本体）。

它们互相咬合、推不翻，还能推出后面所有章节的方法，V 模型为什么是 V、IPD 为什么有技术评审点、为什么要求要有追溯矩阵、为什么变更要控制，答案都在需求、设计开发、质量里。

下一章，我们看需求是怎么被开发出来的：需求工程。

产线计划本上，添了两行字：制造方案、测试方案。以前产线排不动，大家只说”八周做不完”，两份方案从不上纸；这一章之后逸人明白了：方案不落纸，设计就没到可实现，产线排得再满，也是空转。

---

# 附录

## A 本章概念总图

需求、设计开发、质量的关系，一张图收住：

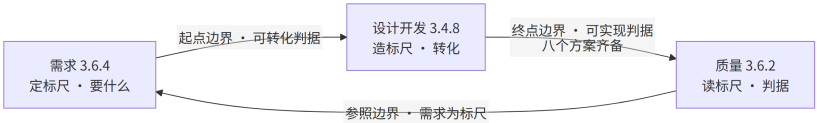



图 7　图 3-1

- 第一节 · 需求、设计开发、质量是什么：需求（明示、通常隐含、必须履行，以及需要、期望、需求的分别）、设计开发（“更详细”判据、层层递归）、质量（固有特性、满足、程度）；以及三者各自的来处；
- 第二节 · 三处边界：起点（研究先于需求、可转化的四条前提、特性有没有被规定）；终点（止于可实现、方案配齐、声明不等于事实）；参照（等级不等于质量、过度设计）；

- 第三节 · 运用：读结果（从特性值到结论、关键特性不能摊平）→ 冲突（质量与进度是同一份需求集合内部打架；与利益相关方一起剪裁：排序、留痕、挂账）→ 合起来（定需求、出规格、判满足；客体与特性是承重构件；需求与设计开发是根概念，质量是派生的枢纽；能推出领域里的方法，但长不出技术本体）。

M3 走完这张图的正确姿势：定需求（-20℃~8℃ 宽范围、断链即报废的硬需求）→ 出规格（方案配齐）→ 判满足（箱温精度的符合性）→ 全程让利益相关方参与剪裁。恒信集团走的是同一张图，只是从”温度”换成了”审批流转”。

## B 本章的能力级

**第 ② 级为主**

- **表征**——把”需求”“设计开发”“质量”三个词说准：明示、通常隐含、必须履行三种形态；设计开发的输入、客体、转换、更详细的需求；质量的固有特性、满足、程度。三者同属核心概念一层，其中需求与设计开发在第 1 章 §1.7.4 的判定里是根概念（那里判出的第三个根是系统），质量是派生的枢纽；
- **图式**——需求、设计开发、质量聚起来形成的结构：“定需求 → 出规格 → 判满足”这条路径，以及路径上的起点、终点、参照三处边界；章题说的”产品开发的图式”，就是它；
- **心智模型**——起点判据”可转化”（完整、无歧义、不冲突、隐含需求已识别）；终点判据”可实现”（方案配齐、执行时没人反问你”

这到底怎么做”）；参照判据”等级不是质量”（特性超出需求范围就是过度设计）；

- **解释框架**——本章不建，由第 17、18 章承担。

> 能答全四问，才算拥有；这把尺子现在装进你手里了。

## C 自查 · 这些说法对吗？

1. “我们把参数做得比竞品高一倍，质量肯定没问题。”——**错**。参数超出需求范围是等级高、是过度设计，不是质量好；质量等于特性满足需求的程度。
2. “技术方案完成了，设计开发就结束了。”——**错**。止于”可实现”，方案配齐才算；只到技术方案，下游环节连环停摆。
3. “质量进度冲突，砍掉测试先上线，后面再补。”——**错**。验证与确认是质量的度量，可缩不可砍；砍了等于不度量就宣布合格，风险后移成现场返工。
4. “客户没说要，我们就不用做。”——**错**。隐含需求也是需求，漏了就是缺陷；它没进文档，验证够不着，只有确认拦得住。
5. “这个变更只是改个颜色，不用走设计流程。”——对。输入输出同样详细，就是没发生设计开发，可以走快速通道；但”把温控改成双路冗余”这种产生了新需求细节的，就是设计开发，不能图省事。
6. “我们和客户一起砍需求，是降低质量。”——**错**。和客户一起剪裁需求是合法变更（排序、留痕、挂账）；单方面砍需求才叫偷换。

7. “把’成为行业第一’写进需求规格，设计开发就知道怎么做了。”——**错**。那是目的，回答”为什么做”；目标把目的翻译成可度量的结果（比如”到站疫苗活性 ⩾ 95% “），需求回答”做成什么样”，必须可判定。目的定方向，需求必须可判定。
8. “质量能打分，M3 质量 85 分，就是客观结论。”——**错**。程度是认定不是计算；关键特性（安全、可靠）不能摊进平均数——那个 85 分是”一个精确的错误”。
9. “需求开发是需求的事，设计开发是画图的事，两不相干。”——**错**。需求开发也是需求转化，和设计开发同构（同一个 §3.3.11）；边界只看一条——特性有没有被规定。

## D 练习

1. 设计开发的输入必须是”需求”。一个”点子”算不算输入？用你手上的一个产品举例，指出”点子”和”需求”的分界在哪。
2. 设计开发什么时候算结束？“技术方案做完了”够吗？列出至少三个还缺的方案，并说明为什么终点由”最后一个配齐的方案”决定。
3. 质量等于客体固有特性满足需求的程度。特性超出需求范围（过度设计），质量是升了还是降了？为什么？
4. “质量与进度”的冲突本质是什么？砍掉验证与确认赶进度为什么最贵？口头改需求为什么最危险？

5. “和客户一起剪裁需求”和”单方面砍需求”的本质区别是什么？为什么剪下来的需求算挂账，不算消失？
6. 符合性、质量、等级三者各自回答什么问题？“高档车可以是差质量”这句话为什么成立？
7. 用”特性有没有被规定”这条判据，判断你们公司一个正在进行的工作，是需求开发还是设计开发。
8. 检查你当前一个项目：“要什么”（需求规格）满足完整、无歧义、不冲突、隐含需求已识别吗？哪一条最弱？会造成什么后果？
9. 用”它从哪来”量一遍需求、设计开发、质量：各自为什么被造出来、如何演化成今天的定义？哪个词曾从”档次、用料”漂移成”符合需求的程度”？
10. 用”定需求、出规格、判满足”把你最近的一个项目走一遍：需求定全了吗？规格出到”可实现”了吗？读出来的是”程度”还是”感觉”？起点、终点、参照这三处边界，各在哪里容易混？

（答案要点见书末附录，与训练教材题卡同源）

## E 参考文献

**标准** - R-102 ISO/IEC/IEEE 15288:2023 —— §6.4.1 业务或任务分析（任务与目的在那里定义，§3.2.3）、§6.4.2 利益相关方需要与需求定义（§3.2.4）、§6.4.3 系统需求定义（设计开发的起算点，§3.2.4）、§6.4.4 系统架构定义、§6.4.5 设计定义（§3.2.5）；§6.4.14 处置过程（退役，§3.3.5）- R-103 ISO 9000:2026（第 5 版）

—— 需求（§3.5.1: 明示的、通常隐含的或必须履行的需求或期望）、设计开发（§3.3.11：把需求转换为更详细的需求）、质量（§3.5.2：固有特性满足需求的程度）、等级（§3.5.4）、符合性（§3.5.9）、特性（§3.10.1）、客观证据（§3.8.6）、确定（§3.11.1）、验证与确认（§3.11.12、§3.11.14）、目标（§3.7.11）、过程（§3.3.1）、项目（§3.2.11）- R-03 ISO 9001:2015（**库内无正本，属库外来源**；条款号照录、版本随引）—— 需求确定（§8.2；§8.2.2 交付时间和交付条件也是顾客需求）、设计开发（§8.3：§8.3.3 设计开发输入的可转化判据、§8.3.5 b 满足后续产品和服务提供过程的需要、§8.3.5 c 监视和测量需求与接收准则、§8.3.5 d 对预期目的至关重要的特性、§8.3.6 设计更改）、§8.2.4 需求的更改、§8.4 外部提供、§8.5.1 生产和服务提供的受控条件

**经典文献** - R-63 V 模型（V-Model）—— 开发过程模型：验证与确认的 V 形对应（从这条路径推出的主流方法）- R-64 IPD（集成产品开发）—— 技术评审点 TR、决策检查点 DCP（从这条路径推出的主流方法）

# 第 4 章 需求工程：从诉求到依据

## 4.1 引言

第 3 章把需求、设计开发、质量三个词立在了产品开发的路径上，需求是那条路径的起点。可需求不会自己站到纸上；它得有人去问、去分析、去写成条文，写成之后还得有人看着它不被悄悄换掉。这一章看的就是这件事：一份需求是怎么被做出来的，做出来之后又靠什么一直是那一份。先把这件事分成两段看，再把最容易混的几处边界画开，最后看它在项目里怎么用。

### 4.1.1 误解现场

恒信集团合同审批系统项目启动后的第三周，管理层开了个会。

IT 总监拍板：“需求最近总是变来变去，合同评审、财务对账、法务条款，改一版崩一版。我们要加强需求管理。谁负责？徐漾，你来。”

徐漾是需求分析师，系统工程师团队里的需求侧专职。他没接话，先问了三个问题：

> “需求为什么会变？变之前，我们的需求开发做到位了吗——调研做了吗？分析做了吗？每条需求写得能当依据用吗？”

会议室安静了几秒。

IT 总监皱眉：“这些不都是需求管理的一部分吗？管好需求，自然就都做了。”

如果你坐在这间会议室里，大概也会点头。因为大多数人对需求的直觉，跟这位总监一模一样：需求乱，是管理没跟上；把需求管理做好，需求就稳了。

这一章要推翻的正是这个直觉。它错在一个词上：**需求管理不包含需求开发**。需求是开发出来的，不是管出来的。

### 4.1.2 这一章要回答什么

读完本章，你应该能回答：

1. 需求工程、需求开发、需求管理是什么关系？“需求管理包含需求开发”错在哪？
2. 需求开发有哪几个活动？最后一个活动为什么叫校核（评审），不叫确认？
3. 一条需求”说出来了”算什么？为什么口头说的不能当验证的依据？
4. 需求从哪来？除了从目标转化，还有哪些来路？
5. 需求”总在变”的时候，怎么先分清是需求变了，还是当初没说清？
6. 运行概念（ConOps）在整条链的什么位置？为什么”先写条款、后补故事”顺序反了？
7. 需求管理管的是什么？变更控制要走过哪几步才算完整？基线凭什么当分界线？
8. “另存为 v2”为什么不是版本管理？
9. 一条好需求的判据是什么？哪些文档任何规模的项目都不能省？

### 4.1.3 这一章的骨架

> 需求工程，是把模糊的诉求做成一份能当依据用的规定需求
> ——开发负责做出来，管理负责让它一直是那一份；两段并列，
> 只在”批准 → 基线”这一处交接。

反过来说更刺人：**管理的严格补不了开发的欠账**。需求乱，多半不是没人管，是没人做。

两段各有各的活动、各有各的判据，中间只隔着一道分界：批准之前需求还在流动，批准之后一切改动都要走变更。这一章把这两段分别看清，再把最容易混的几处画开。说出来了不等于能当依据（明示与可采）、需求乱得先分清是谁的账（开发与管理）、对着需求文档能评审也能验证、做不了确认（评审与验证确认）、给文件改名不等于版本管理（留档与受控）。

最后一节把这条链用起来：先用运行概念把”系统怎么被用”讲清楚，再写规格里的条款。这是第 1 章说的先立枢纽、再填细节在本章的现身：先立问题域，再写条款；顺序反了，条款写得再整齐也是悬着的。

---

## 4.2 需求工程是什么

> 这一节先看定义。需求工程由哪两段构成、每一段各做什么、需求从哪来、做出来的是什么、两段在哪里交接。这些立住了，后面的边界与运用才有话可说。

### 4.2.1 需求工程：开发与管理并列

先看它由哪两段构成，再看它凭什么配得上”工程”二字。

> **需求工程（requirements engineering）= 需求开发 + 需求管理**（并列，不是包含）

- **需求开发（requirements development）**：把利益相关方的需要与期望转化为规定需求，经过获取、分析、规格说明、需求校核。它是创造性的：从没有到有。
- **需求管理（requirements management）**：对规定需求进行基线化治理，经过基线、变更控制、追溯、状态与度量。它是控制性的：从有到一直是那一份。

一句话：**开发负责做出来，管理负责让它一直是那一份**。

这不是本书的分法，标准就是这么分的。CMMI 把需求管理（REQM）与需求开发（RD）列为两个并列的过程域；ISO/IEC/IEEE 29148（需求工程）按”开发 + 管理”两条线组织全篇（该书库内无正本，此处只锚标准与名称，不引条款号）。

> 出处提醒：本章引到的库外来源有八家：CMMI（REQM／RD）、ISO/IEC/IEEE 29148、IEEE 1362、IEEE 830-1998、IEEE 610.12-1990（经第 2 章转引）、MoSCoW、Kano、Boehm 一系的研究，库内都没有正本，属**库外来源**；其中 IEEE 830-1998 与 IEEE 610.12-1990 带版本年，其余按名称锚定。本章对它们只锚标准与名称，不引条款号；凡本章**正文**给出条款号的，都是库内可核的正本：ISO 9000:2026 与 ISO/IEC/IEEE 15288:2023（**附录 E 的 ISO 9001:2015 一处属库外来源，已就地标注**）。

在 ISO/IEC/IEEE 15288:2023 的过程清单里，这条链有三步：业务或任务分析（§6.4.1）定义任务与目的，利益相关方需要与需求定义（§6.4.2）把需要与期望做成利益相关方需求，系统需求定义（§6.4.3）再把它做成系统需求。需求工程做的是后两步，第一步是它的输入。第 3 章 §3.2.4 已经把这个位置划开过。

那为什么配得上”工程”二字？因为这件事有方法（访谈、原型、排序、评审）、有过程（每一步都有输入、输出、角色和准则）、有纪律（基线、变更控制、追溯），而且可复现，换个人来做，结果不会差太远。第 2 章讲”工程化的成色”时列的那几条（有结构、有方法、可重复、全覆盖、受控、可验证），需求工程一条不缺：获取讲方法，规格讲结构，规定需求讲可验证，管理讲受控。这套说法的出处是 IEEE

610.12-1990 对软件工程的定义，出处层级第 2 章 §2.2.3 交代过。它和方法论、手艺的分野也在这里：可复现、可检验，换个人也能做。

### 4.2.2 为什么是并列，不是包含

流行一种说法：“广义的需求管理包含需求开发。”这个说法错在把两类性质相反的活动揉成了一团。

| | 需求开发 | 需求管理 |
|---|---|---|
| 性质 | 创造性：收集、分析、建模、写规格 | 控制性：冻结、变更、追溯、跟踪 |
| 回答 | 需求是什么、合理吗 | 需求定了吗、变了吗、谁改的 |
| 失败模式 | 来源没摸清、分析没做透 | 基线乱、变更失控 |

把开发包进管理，会盖住一个关键事实：**需求是开发出来的，不是管出来的**。管理再严，开发没做好，一样出烂需求。章首恒信那场会，就是”只谈管理、不谈开发”的典型。

两种欠账的后果也不一样。开发欠账出在源头：来源没摸清、话没说清，之后再怎么严管，管的也还是那些没说清的东西。管理欠账出在过程：需求原本写清楚了，却被无序变更一点点磨掉。

恒信”改一版崩一版”，如果只去加强管理，那些没说清的需求依然没说清，只是从此被更严格地看住而已。**先把开发的欠账补上，再谈管理的严**：顺序不能反。

> **概念卡片 · 需求工程**

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 需求开发与需求管理两段并列的总称（CMMI 的两个并列过程域、29148 的两条线）：开发负责做出来，管理负责让它一直是那一份 |
| **它不是什么** | ⑥⑦ | 对立：广义的”需求管理”，把创造性活动包进控制性活动里，会盖住”需求是开发出来的”；辨析：需求工程是两段的总称，需求开发与需求管理在它下面；需求工程不等于需求管理 |
| **它从哪来** | ②③⑨ | 从系统工程的需求侧过程演化来：ISO/IEC/IEEE 15288:2023 §6.4.2 利益相关方需要与需求定义、§6.4.3 系统需求定义是它的过程锚点；CMMI 的 REQM 则 |
| **它到哪去** | 两段并列 | 能说清”做出来”从哪来直接后续，”让它一直是”谁管。 |

### 4.2.3 需求开发：把诉求变成规定需求

需求开发走四个活动：

**需求获取 → 需求分析 → 需求规格说明 → 需求校核（评审）**

**收集需要与期望　　澄清/分解/排序/建模/协商　　写成可验证条文　　对工作产品做评审**

| 活动 | 做什么 | 典型输出 |
| --- | --- | --- |
| 获取 | 访谈、调研、竞品分析，收集需要与期望 | 利益相关方登记册、访谈调研记录 |
| 分析 | 澄清、分解、排序、建模、协商冲突 | 需求池、优先级排序表、业务规则清单 |
| 规格说明 | 把分析结果写成可验证条文 | SRS、术语表、追溯矩阵初版 |
| 需求校核（评审） | 对工作产品判断适宜、充分、有效 | 需求评审报告、需求评审·确认记录、批准记录 |

> 术语注：表内”确认”是**利益相关方确认**，指签字批准前那句”是不是用户要的”；它与 ISO 9000:2026 §3.11.14 的确认（validation）不是同一件事——后者要在使用中兑现：对一份需求文档，能做的是评审与验证，**确认要等它被实现、被用起来**（§4.3.3）。

四个活动不是跑一遍就完，它们是一个循环：分析时发现缺信息就回获取，规格时发现问题就回分析。为什么是循环？因为”模糊的诉求”不会一次被看清。访谈时对方说”重要合同要审批”，分析时发现”重

要”没定义；回去再问”多重要算重要”，得到”金额 ⩾ 500 万或三年期以上”，才进得了规格。需求开发的每一步都可能推翻前一步的假设。循环不是低效，是正视”需求天然是渐进的”这个事实。

最后一个活动为什么叫校核（评审），不叫确认？因为需求文档是一份工作产品，还不是能用的东西。一份还没实现的文档，能对它做的只有评审。三把尺子的并排对照在 §4.3.3，这里先把名字钉住。

> 出处注：「工作产品」这个范畴取自 ISO/IEC/IEEE 15288:2023。它用”工作产品”统称过程产出，人工制品是其中一类：§3.6 人工制品条写的是”在项目期间产生并用于捕获和传递信息的工作产品”（原文其后还有一串示例，此处只引定义语），表 B.1 列的则是”一组可能的工作产品，包括人工制品与信息部件”；ISO 9000:2026 的定义里只有评审一条点了”客体”（§3.11.2）；验证说”规定需求”、确认说”针对特定预期用途或应用的需求”，三条都不说工作产品。

所以需求开发的末位活动，准确的名字是**需求校核（评审）**：评审人对着目标，判断这份需求适不适宜、够不够、有没有用。真正的验证与确认，要等到有东西可用了才做。那是第 8 章的活，本章只把它挂在链上。

> 术语注：需求工程标准（ISO/IEC/IEEE 29148:2018）里有一个说法叫 requirements validation，指的是”需求本身对不对”，与 ISO 9000:2026 §3.11.14 的确认（validation）同名不同物：后者判的是已实现客体在特定用途下满不满足。该书库内无正本，故此处只锚标准与版本，不引条款号。

获取与分析：来源与澄清。获取是需求的第一手证据。恒信的合同审批系统，获取的对象是三个部门的人：法务（条款、合规）、财务（金额、账期、对账）、业务（审批流、时效）。获取做没做到位，判据只有一条：**每条需求有没有来源**，无来源需求不得入需求池。

分析是第一道加工，五件具体的事：

1. 澄清：把”重要合同要审批”问成”金额 ⩾ 500 万或涉及三年期以上的合同须审批”；
2. 分解：把父需求切出子需求。这与第 6 章的分解是同一条规则、两处现身：切点都由消费方的语义范围定，不存在”需求侧”“设计侧”两套分解；
3. 排序：MoSCoW、Kano 决定先做什么、后做什么；第 3 章”与利益相关方一起剪裁需求”在这里落地；
4. 建模：把澄清过的关系画成模型（原型、状态图、业务规则图），让分歧先在图上暴露；
5. 协商冲突：三个部门对”合同状态”的理解不一致，在这里拍平，而不是等集成时炸。

排序值得单独说一句：MoSCoW（必须／应该／可以／不要）和 Kano（基本型／期望型／兴奋型）不是两张玩具表，它们直接决定”需求不够时砍哪条、保哪条”。恒信的”合同数据留档十年”（公司法强制）在 MoSCoW 里是 Must，在 Kano 里是基本型。两套框架指向同一个结论：这条需求不在被砍的清单里。

需求缺陷的成本。常被引用的一组数据：同一个缺陷在需求阶段修复成本是 1，发布后修复是 30 到 70 倍。需求阶段写错一个字，可能意味着设计、开发、测试全部返工。需求工程的每一分投入，都在给下游省十倍百倍的返工钱。这组数据出自 Boehm 一系的研究（附录 E）。

### 4.2.4 需求的来路

第 3 章 §3.2.1 把需求分成三种形态：明示的、通常隐含的、必须履行的。三种形态不是分类游戏，它们各自对应一条来路。问”需求从哪来”，答案就是这三条。

**目的（为什么做）—展开→ 目标 —转化→ 需求　　来路一：目标转化（主流）**

**环境与法规 ——————————→ 需求　　来路二：法规强制（属必须履行；环境约束另计）**

**利益相关方的隐含需要 ——————————→ 需求　　来路三：通常隐含（未必被目标覆盖）**

来路一：从目标转化。目的展开成目标、目标转化成需求，这是主流来路。第 3 章 §3.2.3 已经把目的、目标、需求三个词摆开，这里只钉一处用词：**目标到需求是”转化”，不是”分解”**。

分解是同类拆分，语义守恒，整体等于部分之和；目标→子目标、需求→子需求才是分解。目标回答”要实现的结果”，需求回答”做成什么样”，两者不同类。这条链是跨类导出：一条目标往往导出一组需求，导出的每一条都要判断；阈值是权衡出来的，不是从目标里分出来的。ISO 9000:2026 §3.3.11 说设计开发是”将某一客体的需求转换为该客体更详细需求的一组过程”，那是同类细化；目标到需求比这更远一步，本书按跨类导出处理，理由是语义不守恒。

这不是本书的私分法：ISO/IEC/IEEE 15288:2023 把任务与目的、利益相关方需要与需求、系统需求分放在三个过程里，业务或任务分析（§6.4.1）、利益相关方需要与需求定义（§6.4.2）、系统需求定义（§6.4.3）。三段各是一个过程，不是同一条链上的粗细三档；它们之间是跨类导出，不是同类拆分。往下走，需求到子需求是分解、需求到方案是求值（第 6 章），分解与求值都以语义可守恒为前提。目标到需求这两样都不占，所以那一步要判断，不能照着分。

来路二：从环境与法规。这一支要分成两条走：法规强制那一支属”必须履行”，环境那一支另立一类。恒信”合同数据留档十年”是公司法强制：组织就算没立过”合规目标”，这条需求照样成立，它不经过组织目标这道转化。气候、地域、文化等环境适应性要求，归第 5 章承接。

来路三：从利益相关方的隐含需要。案例进展②里财务那句”25 号之后签的合同能不能算下个月”，就是来路三：行业惯例默认，没人写进文档，但漏了就要出事。

三条来路合起来说明一件事：**需求比目标转化的结果更宽**。目标的完备性靠评审的充分性把关（目标有没有被需求覆盖全，第 8 章）；另外两条来路不经过目标，只能靠需求工程自己补齐。利益相关方的需要要收集全，法规要列成清单（第 5 章展开）。

> 术语注：“需求三形态对应三条来路”是本书为方便使用做的梳理，不是标准原文；标准只并列给出需求的三形态，不给来路。

### 4.2.5 规格与规定需求：一份 SRS 里有什么

规格说明把分析结果写成可验证的条文，核心交付物是需求规格说明书（SRS）。一份 SRS 通常有六章：

**引言（目的·范围·读者）→ 总体描述（产品视角·环境·约束）**

**→ 功能需求（逐条编号 REQ-xxx）→ 非功能需求（性能·安全·可用性）**

**→ 外部接口（用户·硬件·软件·通信）→ 验证方法（每条需求的接收准则）**

各章各有职责：引言把读者领进来；总体描述给出产品视角；功能需求逐条编号，无歧义、可验证；非功能需求写性能、安全、可用性；外部接口定义与外部世界的边界；验证方法给每条需求配接收准则。这一章最容易被砍，但它正是”规定需求可验证”的兑现处：没有接收准则，需求就退化成愿望。

> 承接第 3 章 §3.3.3：那一节说过”特性还没开始规定，是需求开发；特性开始被规定，才是设计开发”。那份分界文档就是 SRS。写完之后，才谈得上把需求落到客体的固有特性上。

这里的枢纽概念是规定需求。

> **规定需求（specified requirement）**：被明示的需求（ISO 9000:2026 §3.5.1 注 2）。

正本的原话是 A specified requirement is one that is stated, e.g. in documented information（库内中译作”规定的要求是明示的要求，例如写在成文信息中的要求”；本书按术语口径把 requirement 一律作”需求”，下同）。要紧的是那个 e.g.：写在成文信息里是最常见的一种明示方式，不是”规定”的条件。说出来了就是规定需求，口头说的也算，它已经进了这个集合。

那口头说的和成文的差在哪？差在能不能拿来当依据。规定需求要当验证或合规的参照，还得过两道关：先**可判定**，有编号、有对象与适用范围、有断言、有判定方法；再**可采**，四条都要过：**相关性**，能回溯到目的；**可检验性**，自带判定方法与准则；**时效性**，属于已冻结的那一版；**同一性**，有唯一标识。口头那句”页面要快一点”是规定需求，但不可采：四条里它只对得上相关性，没有判定方法、不可复现、也没有版本可查。

> 出处注：“可判定”四件套与”可采”四条件取自报告库 **24 号 v1.7** §7.6／§11.4（术语表 T-184 同此登记）。

所以”说了不等于规定”这句话，准确的说法是：说出来了就是规定需求，但**说出来了不等于能当依据**。本书对这句话的改法只换理由，不换结论。口头需求照样不能当验证的靶子，不是因为它”不算规定

需求”，而是因为它不可采。这也顺带解释了需求工作的两道动作：需求评审要把口头内容写成条目，配置管理要把条目冻进基线。成文与基线不是”升格为规定”的仪式，是取得可采性的两步：写下来才有判定方法与唯一标识，冻进基线才谈得上时效。

### 概念卡片 · 规定需求

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 被明示的需求（ISO 9000:2026 §3.5.1 注 2）：验证的参照；成文与基线是它的可采条件，不是它的构成条件 |
| **它不是什么** | ⑥⑦ | 对立：不可采的明示，口头说了但没判定方法、不可复现、无版本，够不上”依据”；辨析：明示 = 规定（一个等号），成文 ⊂ 明示（最常用的一种方式） |
| **它从哪来** | ②③⑨ | ISO 9000:2026 §3.5.1 把需求按”明示的、通常隐含的、必须履行的”三形态并列，其中”明示”这一支被单列，因为验证需要一个能对账的稳定参照；specified（拉丁 specificare，明确指明）；“可验 |
| **它到哪去** | ④⑤⑧ | 它能需要能使调整 |

### 4.2.6 需求管理：让它一直是那一份

需求开发把需求做出来了，之后归需求管理看着。它保证的只有一件事：谁动了、动在哪、为什么动，都说得清。

标准侧给过一条过程锚：利益相关方需求要记录下来，记成适合整个生存周期做需求管理的形式。这些记录建立需求基线，保留需要的变化及其来源，并成为向业务或任务分析的决策、利益相关方需要、系统需求和后续系统元素追溯的基础（ISO/IEC/IEEE 15288:2023 §6.4.2.3 注 24）。记录、基线、变更、追溯四件事，在那一条注里是一次到位的。

四个活动：

**基线管理 → 变更控制 → 可追溯性 → 状态与度量**

| 活动 | 解决什么问题 | 典型产出 |
| --- | --- | --- |
| 基线管理 | 需求”定了吗” | 需求基线、基线登记表 |
| 变更控制 | 需求”变了吗、谁改的” | 变更请求单 CR、CCB 评审记录 |
| 可追溯性 | 需求”从哪来、到哪去” | 追溯矩阵 RTM |
| 状态与度量 | 需求”整体什么状态” | 需求状态报告、度量报告 |

四者不是均分的：**变更控制是核心战场**（大多数团队的需求管理问题都出在这里）；基线是它的前提，追溯让它查得清影响面，状态与度量让它看得见整体情况。合起来回答的是一串问题：定了吗、变了吗、影响谁、整体什么状态。

> 术语注：§4.2.1 那句”对规定需求进行基线化治理”里的”治理”，取”受控地管住”这一层意思，与第 14 章作为专名的治

> 理（谁掌舵、谁划桨）不是同一个概念。那一个讲的是跨层级的校正机制，这一个讲的是把已批准的需求看住。

变更怎么走完一圈。需求管理最容易失控的地方是变更。一条标准的路径：

**变更请求单 CR → 影响分析（经追溯矩阵反查）→ CCB 评审（批准/拒绝/暂缓）→ 实施变更（同步 SRS·RTM）→ 再验证（符合新的规定需求）→ 关闭（更新基线·登记日志）**

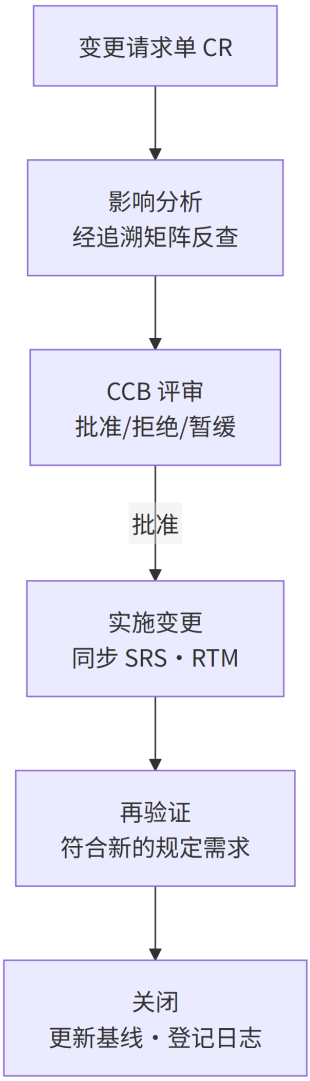



图 8　图 4-1

每个节点都是一个必须可判定的关卡：CR 有没有立案、影响分析有没有做、CCB 有没有结论、再验证有没有通过。任何一个答不上来，

变更就在流程外流窜。一圈走完，回到下一个变更请求单立案：变更控制是闭环，不是一条直线。

三条铁律：

> **无 CR 不得变更；不分析不得上会；验证通过方可关闭。**
> 术语注：这三条是本书对变更控制纪律的操作化凝练（源出 CMMI 配置管理与需求工程标准的变更管理需求，非标准原文照录），可当场检验。

变更还要分级：微小（需求负责人批）、一般（项目经理批）、重大（CCB 全体评审）。时限上有组织级约定（如 SRS 完成后 5 个工作日内评审、影响分析 3 个工作日、CCB 5 个工作日内决策、紧急变更 24 小时内追认）；度量上有预警阈值（变更率 ≥ 15%、稳定性 ≤ 80%、追溯覆盖率 < 100%、变更平均周期 ≥ 10 天；触及其中任一条线，就启动根因分析）。这些数字是组织级示例，要害在于：变更控制的每一步都要可判定，“以什么标准”必须写死，否则流程就是摆设。

一个走完整条路径的例子：恒信法务提出”电子签章的合同也走审批流”。这是一条 CR。影响分析经 RTM 反查，发现影响审批状态机、签章模块、归档模块共 3 处；CCB 评审通过；实施后同步 SRS 与 RTM；再验证”电子签章合同审批通过后进入归档”；关闭，更新基线。整个过程留下 6 份记录，每个节点一份。任何一环断了，这条变更就成了查无实据的暗改。

基线是分界线。线左是需求开发的产物（还在流动），线右归需求管理（一切改动走变更控制）。

> **基线**：经批准、确立了产品或服务某一时点特性、作为生存周期参照的配置信息（ISO 9000:2026 §3.10.12；本书行文简作”被批准冻结的参照”）。
> 术语注：此定义取自配置管理的通用概念，第 13 章展开全貌。

第 13 章会把这个概念讲透。这里只需要记住它在需求链上的作用：基线是变更的出发原点。从基线出发，一切变更都可描述；离开基线，一切变化都不可言说。

基线和追溯是一体两面：基线给”变”一个参照原点，追溯给”变”一份影响清单。没有基线，追溯无从谈起；没有追溯，基线一建立就失效。恒信先建基线、再建 RTM，顺序对了。

判据问句：这一版需求被批准了吗？被冻结了吗？生存周期拿它当参照吗？ 三问全过，才是基线。

### 4.2.7 开发与管理在哪里交接

接口只有一处：批准 → 基线。需求开发的最后一步是需求批准记录（评审通过、利益相关方确认），批准之后建立需求基线，进入需求管理的领地。

> 这道分界是整条需求工程唯一的接口——批准前，需求还可以流动，归开发负责；批准后，一切改动走变更请求，归管理负责。这道分界不咬合，两段就各管一段，需求会在中间断掉。

需求管理计划（RMP）。程序文件（组织级”怎么管需求”的通用规定）与 RMP（项目级执行计划）是两个层级：每个项目启动时，按程序文件编制 RMP。RMP 通常有六块，每块回答一个问题：

| 模块 | 回答的问题 |
|---|---|
| 目的范围 | 这份计划管什么 |
| 角色职责 | 谁是 CCB、谁批微小变更 |
| 需求组织规则 | 需求怎么编号、怎么入池 |
| 基线策略 | 什么时候建基线、什么算重大变更 |
| 变更控制流程 | CR 怎么走、时限多久 |
| 追溯与度量安排 | RTM 怎么建、什么指标预警 |

RMP 是程序文件的项目版：把”组织怎么管需求”翻译成”这个项目怎么管需求”。

需求管理的正式产出，以台账和凭证为准。审计、扯皮、追责全靠它们：

| 活动 | 文档 |
|---|---|
| 计划 | 需求管理计划 RMP |
| 基线 | 需求基线 · 基线登记表 |
| 变更 | 变更请求单 CR · 影响分析报告 · CCB 评审记录 · 变更验证记录 |
| 追溯 | 覆盖分析记录 · 追溯矩阵 RTM |
| 状态与度量 | 需求状态报告 · 变更日志 · 需求度量报告 |

其中 CR、CCB 记录、变更日志最要紧，它们是”改了没有、谁批的、为什么改”的唯一凭证。**变更日志只追加不修改**，审计时调取。需求管理最忌口头的变更：落到文档层面，就是没有凭证的变更。

台账与凭证之外的东西见 §4.4.5：不可省的骨架文档、整套编号；这里只说交接处要交出去的那几件。

## 案例进展①｜徐漾的”需求管理”升级会议

会后，徐漾把”加强需求管理”这条指令翻成了两件事。

第一件，他调了恒信需求库里近三个月的变更记录。四十七条变更里，二十九条不是需求变了，而是当初就没说清：合同评审什么时候启动，写的是”重要合同要审批”，可”重要”从没定义过；财务要的”对账报表”从没说清统计范围。

第二件，他列了两张单子：一张是开发欠账（没调研、没分析、没写可验证的规格），一张是管理欠账（没基线、没变更流程、没追溯）。

> “加强需求管理这件事，一半是补需求开发的课。管理能让一份需求一直是那一份，可它没办法让没说清的需求变清楚——说不清的，只有开发能补。”

万辉盯着那张开发欠账的单子看了半天。

> “这四个星期我们加了三道流程、两张模板，变更数一条没降。现在看，是把两件事当成一件事了：管理管的是’定了的需求不变形’，开发管的是’需求本身立得住’。”

> “流程没错，是顺序反了。按 ISO 9000:2026 §3.3.11，设计开发是把需求转换成更详细的需求——那些连’重要’都没定义的条目，压根过不了第 3 章 §3.3.2 那条可转化的线。拿没过线的条目去开发，开发得越彻底，返工越彻底。”

两周后，徐漾补了需求调研和需求池，建了第一版基线。万辉说”需求稳了一点”。不是需求变少了，是欠账开始还了。

这两张单子，正是这一章要分的两笔账：§4.3 磨它们的边界，§4.4 看怎么用起来。

## 案例进展②｜合同的”隐含需求”

做合同审批系统需求调研时，徐漾访谈法务、财务、业务各三场。每次最后他都问一句”还有别的吗？“，得到的都是”没有了”。

直到财务随口说了一句：“对了，我们月底要对账，合同如果 25 号之后才签，能不能算到下个月？”

徐漾记下了。这句话暴露了一条隐含需求：合同审批的统计归属月份规则。没人把它写进任何需求文档。它”通常隐含”，是行业惯例默认。但如果漏了，月底对账会错，财务会觉得”这系统根本不能用”。

需求开发里，**隐含需求不会自己开口**。它需要主动去挖：访谈里的随口一句、业务流程里的惯例、前代产品的教训。挖出来的隐含需求要显性化、写进规格、变成规定需求。否则验证够不着它（验证对照规定需求），只有确认拦得住它（确认对照特定预期用途或应用的需求）。那是第 8 章的核心，这里先埋种子。

需求开发到底开发什么？不是把用户的话抄下来，是把不开口的、没成文的、说不清的，都挖出来、写下来、定下来。

---

## 4.3 需求工程不是什么

> 这一节把最容易混的几处分开。概念靠正例活，靠反例定型：一处混在”说了”与”能当依据”之间，一处混在开发与管理之间，一处混在评审、验证与确认之间，一处混在留档与受控之间。

### 4.3.1 明示与可采：说出来了，就能当依据吗

“审批要严谨”“报表要一眼看懂”“导出要方便”，这些话客户说了，领导也说了。它们算需求吗？

算。按 ISO 9000:2026 §3.5.1 注 2，说出来了就是明示的需求，明示即规定。它们已经是规定需求了。

但它们还当不了依据。

验证要的是一个能对账的靶子，而一句口头的话对不了账：“快一点”是几秒？“好用”怎么证明？所以同一句需求，要从”说了”走到”能当依据”，中间还得满足可采的条件：

| | 口头明示 | 成文且已定版 |
|---|---|---|
| 是规定需求吗 | 是（明示即规定） | 是 |
| 能当验证的参照吗 | 不能：四条件里只对得上相关性，没有判定方法、不可复现、没有版本 | 能：四条都占，有编号、有判定方法、在已冻结的那一版上 |
| 传得下去吗 | 转述容易走样 | 白纸黑字不漂移 |
| 追得回来吗 | 没有版本可查 | 有版本可查 |

这条边界的判据问句：一条需求能提供客观证据吗（测试只是其一）？能，就可采、能当依据；只能”感觉”，就还得写下来、定下来。

恒信那句”重要合同要审批”就是标本。它是明示的需求，也是规定需求，可它不可采：“重要”有两种以上的理解，谁也没法拿它去验证。把它规定成”金额 ≥ 500 万或三年期以上”，它才第一次能当依据用。

> 承接第 3 章 §3.3.2：那一章从设计开发那一侧看同一件事，给的是”可转化”那条线（完整、无歧义、不冲突、隐含需求已识别），管的是能不能进厂加工。这一节从验证那一侧看，管的是能不能当对照的靶子。两条线不是一回事：可转化管进不进得去，可采管作不作数。

### 4.3.2 开发与管理：需求乱，是谁的账

案例进展①里徐漾翻了三个月的变更记录，发现一多半”变更”根本不是需求变了，而是当初就没说清楚。

这条边界钉死的是一条诊断判据：当需求”总在变”，先别急着上管理手段，先分清它是哪个病。

| 病 | 症状 | 药 |
| --- | --- | --- |
| 开发欠账 | 没调研、没分析、没写可验证的规格 | 补开发：调研、分析、写规格 |
| 管理欠账 | 没基线、没变更流程、没追溯，于是变更失控、查无实据 | 补管理：基线、变更流程、追溯 |

两种欠账的症状不一样，问法也不一样：听到”这条需求当初是谁提的、为什么提”答不上来，多半是开发欠账，来源没留下；听到”这

一版是谁批的、上一版长什么样”答不上来，多半是管理欠账，痕迹没留下。

判据问句：最近这次需求混乱，是”当初没说清”（开发欠账）还是”定了之后没人管”（管理欠账）？答错了病，药就开错了。章首那位总监开的就是这个药。

### 4.3.3 评审与验证确认：对文档能做什么

需求评审开完，主持人常问一句：“验证了吗？确认了吗？”这句话问错了。

按 ISO 9000:2026，这三件事各有各的对照，对象也不一样：

| | 评审 review | 验证 verification | 确认 validation |
|---|---|---|---|
| 定义 | 为达成既定目标，对客体的适宜性、充分性或有效性所作的确定（§3.11.2） | 通过提供客观证据，证实规定需求已得到满足（§3.11.12） | 通过提供客观证据，证实针对特定预期用途或应用的需求已得到满足（§3.11.14） |
| 对照 | 为客体确立的目标 | 规定需求 | 特定预期用途或应用的需求 |
| 对象 | 工作产品（文档、模型）——**能评审，也能验证** | 实现出来的客体，**或人工制品**（15288:2023 §6.4.9） | 已可使用的客体——**确认的现场在使用态** |

> 术语注：确认一条的正本中文用字是”针对特定预期用途或应用的要求”（§3.11.14），本书一律照正本引作”针对”。

需求文档是工作产品。**对它做的是评审与验证**：评审判断这份需求适不适宜、够不够、有没有用；验证认定它满足”这份文档自身应满足的规定需求”（如条目齐备、可判定），证据形态就是评审文件（15288:2023 §6.4.9 的验证对象含人工制品）。**确认做不了**——确认要在使用中兑现，那是第 8 章的三个看门人。

这一处为什么非分清不可？因为它决定了一句话能不能被承诺。把需求评审叫成”需求确认”，等于给一份还没实现的东西签了确认的字：评审会上说”确认通过”，验收时才发现用户要的不是这个。那句”确认通过”就成了将来扯皮的证据。第 8 章会把这条线完整展开，这里先钉住一句：**需求文档做不了确认**——确认要在使用中兑现；能对文档做的是评审与验证。

> 术语注：需求工程标准里的 requirements validation（“需求本身对不对”）与这里的确认同名不同物，见 §4.2.3 的术语注。

判据问句：你现在要判的这份东西，是做出来的客体，还是纸面上的工作产品？是工作产品，**能评审、也能验证**（验证它满足文档自身应满足的规定需求）；要说”确认通过”，得先把东西做出来、并且用起来。

### 4.3.4 留档与受控：改名算不算版本管理

需求管理里最像版本管理的动作，是给文件改名，也最容易被当小事放过去。

**改名、留档，不等于版本管理**。版本管理要有三样：**基线**（批准的参照）、变更控制（怎么变、谁批）、追溯（从哪变起、波及谁）。

> 术语注：这里说的”版本管理”是**配置管理管到文件层面**的日常说法（第 13 章展开全貌）；**上面的三样，正是”留档”与”受控”的分界**。

“另存为 v2”只做到了留档，留下了每一版的快照，却没回答三个问题：哪一版是权威基线？这次改为什么、谁批的？这条需求从哪来、影响谁？

这是全书”另存为 v2”的首次出场。第 13 章会用合同版本 final_2.zip 再演一遍，同一个病，换了文件后缀。

判据问句：这次”版本管理”，能回答”哪一版是基线、这次为什么改、从什么变到什么”吗？答不上来，就只是改名。

## 案例进展③｜“另存为 v2”的合同需求

恒信的合同审批需求改了六版。v1 是没后缀的原稿，v2 以后靠文件名管版本：“合同需求 最终版 v3.doc”“合同需求 真最终版 v4.doc”。

后果是：需求池里同一份合同的需求散在六个文件里，追溯矩阵没法建，验证没有参照。小白是开发，她接口写到一半，对着六个文件名发愁：

> “我照着哪个版本写？接口这一半按 v4 的字段名写的，可 v6 里’合同状态’多出一个’已签章待归档’——六个文件三种说法。哪一版是批过的需求规格说明书，我看不出来，数据库的表结构就定不下来。”

玉杰是运维，他把发布记录也摊到桌上：

> “文件名记不下谁批的、为什么改。我们发布认的是可复现——哪个包配哪套环境参数，能复现出当时的运行形态，才算数。你们这六个文件，连哪一版是基线都对不上；没有变更请求单，也没有追溯矩阵，等于每次改动都没发生过。”

万辉终于承认：“我们不是需求变太多，是这几份需求文件从来没立过基线。”

徐漾把这几份需求文件收进了基线管理，之后每个变更都走 CR 流程。六周后，变更失控率降了七成。不是需求变少了，是每次变更都留下了痕迹。

---

## 4.4 需求工程怎么做

> 这一节把这条链用起来：先用一份运行概念把”系统怎么被用”讲清楚（问题域），再写规格里的条款（解空间），最后看一条好需求的判据和不可省的文档。

### 4.4.1 ConOps：系统将被怎么用

SRS 回答”系统必须做什么”，可在这之前还有一个更根本的问题：系统怎么被用？法务怎么审合同、财务怎么对账、业务怎么发起审批；这些”怎么被用”的叙事，是 SRS 的上下文。

没有它，SRS 的每一条都像悬空的条款：你知道要做什么，却不知道为什么、为谁、在什么场景下。

运行概念（Concept of Operations，ConOps）就是回答”系统怎么被用”的那份文档（IEEE 1362 定义其结构）。它是需求工程里唯一一份叙事性的正式输出，别的文档都是条目和条款，只有它讲一段过程。标准侧的位置也是明的：ISO/IEC/IEEE 15288:2023 把”开发运行概念及其他生存周期概念”列为利益相关方需要与需求定义（§6.4.2）的一项活动；§6.4.2.3 的注 5 又给出展开的去处——运行概念的更多信息见 ISO/IEC/IEEE 29148:2018 第 5、6 章，系统运行概念的带注释大纲见该标准附录 A。

### 4.4.2 ConOps 的位置：分析之后，规格之前

ConOps 不在流程最前，也不在最后：

**业务目标 → 需求获取/分析 → 运行概念 ConOps（系统怎么被用）→ SRS（系统必须做什么）→ 设计·测试**

链头那段”业务目标／系统目的”，由业务方定义并签字确认，对应后记里系统架构师那份《系统目的或任务理解确认单》（第 18 章）。目的、目标、需求三个词怎么分，第 3 章 §3.2.3 已经立过，这里只补分工：目的由业务方定；目标由需求侧帮它立成可衡量的结果；架构师在架构语境里把它当判据用，但立得对不对仍要业务方点头。

需求从三条来路上来（§4.2.4），三件东西各有一种落纸形态：目的落成确认单，目标落成操作目标清单；评审的通过标准，正是会前从这份清单里选出的可判定目标（第 8 章）；需求落成规格里的条目。

为什么 ConOps 必须在”分析之后、规格之前”？因为它的完整定稿（运行场景、异常、应急、角色）必须建立在已经摸清需要的基础上：先获取分析，再综合成运行概念，最后翻译成 SRS 条文。

跳过 ConOps 直接写 SRS，等于从问题域一步跨进解空间，中间那段没了。

ConOps 还有第二重身份，比”上下文”更要紧：**它是确认判据的载体**。确认对照的是”针对特定预期用途或应用的需求”，而这份需求的正文，恰恰写在运行场景里，并且主体是隐含需求那一支。隐含需求不在派生链上（派生链承载的是明示的那些），验证够不着它，只有确认拦得住它；没有 ConOps，确认就没有靶子。第 8 章会把这条线接上。

> ConOps 讲一段过程，SRS 讲一条条条款——先讲清楚系统怎么被用，才谈得上系统必须做什么。

这正是本章的方法底座时刻：**先立问题域，再写条款**。第 1 章叫它先立枢纽、再填细节。第一节立的”开发与管理并列”是第一个枢纽，ConOps 是第二个。先扎进条款、不立骨架，条款越多越悬空。

### 4.4.3 一份 ConOps 里有什么

IEEE 1362 给 ConOps 的结构列了六部分：

| 部分 | 内容 |
| --- | --- |
| 引言 | 目的、范围、读者 |
| 系统概述 | 系统的使命与地位 |
| 运行环境 | 部署环境、外部接口 |
| 运行场景 | 正常·异常·应急（核心） |
| 利益相关方与角色 | 谁用什么、谁管什么 |
| 运行支持 | 部署、培训、维护、升级 |

其中**运行场景是核心**，而且要写三种：正常（一份合同走完审批全流程）、异常（法务驳回、财务校验不过、审批人离职）、应急（系统宕机时的线下替代流程）。

写 ConOps 有个实用判据：让一个不熟悉这套系统的人读完，能不能画出审批流程图？画得出来，ConOps 合格；画不出来，故事没讲清楚。

它的失效边界也要说清：ConOps 服务的是”怎么被用”的复杂度。运行方式一眼看透的小系统（单机、单场景）可以省掉；场景越复杂、角色越多，越不能省。

### 概念卡片 · 运行概念 ConOps

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 从用户与运营视角描述”系统将如何被使用”的文档（IEEE 1362 定义其结构）：需求工程里唯一一份叙事性的正式输出 |
| **它不是什么** | ⑥⑦ | 对立：SRS, ConOps 讲一段过程，SRS 讲一条条条款；辨析：不是运营手册（不含运维操作细节），不是需求清单（是运行叙事），不是系统设计（不落进解空间） |
| **它从哪来** | ②③⑨ | 系统工程早期发现：只写”系统必须做什么”，利益相关方对”系统将被怎么用”各想各的，于是把运行方式单独立成一份文档；Concept of Operations，让回答”BRS 想要系统 |
| **它到哪去** | ④⑤ | 画得出流程 |



IEEE 1362 定其结构，能画出流程，可追溯到后续的 SRS 与测试

### 4.4.4 一条好需求的判据

一条需求工程意义上合格的需求，须同时满足六条：

> 完整（该有的都有，含隐含需求）· 一致（互不打架）· 无歧义（只能有一种理解）· 可验证（能提供客观证据）· 可追溯（知道从哪来、到哪去）· 有优先级（知道先做哪个）

> 术语注：这六条不是任何标准的固定清单，是**本书对”SRS 写得好不好”的判据**。库内可核的邻近表述在 ISO/IEC/IEEE 15288:2023 §6.4.2.3 注 26：那一条对利益相关方需求既分析单条需求的特性（必要的、实现独立的、无歧义的、完整的、单一的、可实现的、可验证的、符合的），也分析需求集的特性：完整的、一致的、可行的（或可负担的）、有界的；本节这六条，是本书按 SRS 编写与评审的用法从这些特性里取的。

六条不是并列的形容词，它们互相支撑：无歧义是可验证的前提，可验证是规定需求当依据的条件，可追溯是变更影响分析的依据，有优先级是”与利益相关方一起剪裁需求”（第 3 章）的操作基础。

六条里最常被牺牲的是可追溯，因为它不立刻显形，直到需求变更时才发现”这条需求当初为什么加、影响谁”全答不上来。

落地表现：SRS 里禁止”快速”“大约”“友好”这类不可验证的表述，这正是无歧义与可验证的检查项。恒信那句”重要合同要审批”

为什么烂？它同时违反两条：无歧义（“重要”有两种以上理解）和可验证（提供不出客观证据）。

六条之上还有一套全集，分两层立。整体层管”这份规格能不能当设计输入”，第 3 章 §3.3.2 的可转化四条前提；单条层管”这一条写得好不好”，本节这六条。需求工程标准另给”单条好需求”列了一份更长的特性清单（必要性、恰当、正确、符合、单一、可实现等），本书按二手来源处理：只锚标准与版本，不逐条照抄条款号，也不写成”标准说……“。

这些判据是怎么来的？不是标准组织凭空发明的，是失败模式的反面。工程界用几十年数据统计需求缺陷：约半数的软件错误要到后期才被发现，而且多数发生在写代码之前（Boehm 一系的研究，附录 E）。于是把”歧义、缺失、冲突、不可验证、不可追溯”这些缺陷类型逐条写成它们的对偶：“不许歧义”就成了无歧义。标准制定走的是同一条路：收集实践 → 提炼失败模式 → 写成正向条款 → 投票评审 → 修订。判据不是被”想”出来的，是被”反推”出来的。

### 4.4.5 需求文档体系

需求工程的文档可以有十几份，按”素材 → 分析 → 正式交付 → 校核”四类组织：

| 类 | 文档 | 用途 |
| --- | --- | --- |
| 素材类 | 利益相关方登记册 · 访谈调研记录 · 竞品分析 · 用户故事 | 需要与期望的第一手证据 |
| 分析类 | 需求池 · 优先级排序表 · 业务规则清单 · 原型 · 可行性分析 | 需求怎么定 |
| 正式交付 | SRS · 术语表 · 追溯矩阵 RTM | 需求是什么 |
| 校核交接 | 需求评审报告 · 需求评审·确认记录 · 需求批准记录 | 对不对、交出去 |

素材类文档即使不写成正式文档，也要留原始记录，它们是追溯链的最底层。

骨架文档不可省。上面这些文档里，有五份不能省，任何规模的项目都一样：

**SRS（需求规格说明书）· 需求池 · RTM（追溯矩阵）· 评审报告 · 基线**

- SRS：规定需求的载体；
- 需求池：需求统一入口，无来源需求不得入池；
- RTM：追溯矩阵，变更影响分析的依据、测试覆盖的对照、合规审计的证据；
- 评审报告：校核的凭证；
- **基线**：变更的参照。

RTM 的终极检验：任取一条需求，能不能双向走通，往上游知道它从哪来，往下游知道它影响谁？走不通，RTM 就是一张漂亮的表格，不是追溯。

这套文档可以统一编号：开发域 QR-01~12（利益相关方登记册、访谈记录、需求池、优先级、业务规则、ConOps、SRS、术语表、RTM、评审报告、需求评审·确认记录、批准记录）、支撑文档 QRS-01~04（竞品分析、用户故事、原型模型、可行性分析）、管理域 QR-13~23（RMP、基线、基线登记表、CR、影响分析、CCB 记录、变更验证记录、覆盖分析、状态报告、变更日志、度量报告）。

编号让”哪份文档属于哪个活动”一目了然。**编号不是目的，可引用才是**：“这条需求在哪个文档、哪一条”要能指到具体位置，否则追溯就是空谈。

文档体系是不是太重了？判据很简单：没有文档的环节，就是不可追溯的环节。裁掉文档不是省事，是把追溯链砍断；追溯链断了，需求变更时你就只能靠记忆和运气。

用恒信走一遍 SRS 六章：引言（这份 SRS 覆盖合同审批系统 1.0 的功能与非功能需求）；总体描述（系统替代线下纸质审批，部署于内网，用户为法务、财务、业务三方）；功能需求 REQ-001~REQ-187（发起、初审、校验、终审、签章、归档）；非功能需求（月末高峰 500 并发、合同数据留档十年、操作日志不可篡改）；外部接口（与 OA、电子签章平台、财务系统的接口契约）；验证方法（每条需求的接收准则，如”审批通过后 5 分钟内通知利益相关方”）。连同非功能与接口，全

份约三百条。六章写完，一个不熟悉项目的人能独立判断这份规格全不全。

恒信补了 RTM 之后，一次”合同金额校验规则”变更，五分钟内定位了受影响的 17 条需求、3 个模块；没有 RTM，这要翻遍六个版本文件找两天。

## 案例进展④｜恒信没人写的 ConOps

恒信的第一版 SRS 写得飞快，三个月，三百条需求。评审会上，法务负责人翻了两页就问：

> “你们写’合同审批通过后自动通知利益相关方’——哪个利益相关方？审批链上有几个人？通知不到会怎样？这三问答不上来，这一条就不是能验收的需求，是一句表态。”

没人答得上。因为没人写过 ConOps：“法务、财务、业务三方怎么用这套系统审批一份合同”这个叙事，从没被写进过任何文档。三百条条款悬在半空，每一条都缺上下文。

产品开发经理满海翻了翻那三百条：

> “条款一条条都对，可凑在一起，说不清一份合同到底是怎么被用起来的。评审到这儿只能一条条过——我们要的不是三百条各自成立的条目，是一份看完能明白这系统怎么运转的运行概念。”

徐漾花三周补 ConOps：把一份 200 万的采购合同从发起、初审、校验、终审、签章到归档的完整过程写成故事，逐环节标注角色、异常与应急。

业务方的黄程陪着徐漾一段段对审批链。对到一半他提醒：

> “审批人要是离职了，谁接得上？流程图上这一格是空的。运行场景里’异常’那一栏不写，上线第一个月就会有人拿着纸质单子来找我们——这一栏不是补充材料，它就是需求。”

写到”运行支持”，徐漾去问运维玉杰。玉杰说：

> “系统宕机了，审批走什么替代流程？数据怎么备份、监控什么、谁负责升级？这些都得写进运行概念——运行支持那一栏不落纸，上线那天我们就是临时摸索。”

写完，三百条 SRS 里有一半需要改写，因为先写条款、后补故事，顺序反了。

规格的问题，往往要到运行概念里才看得出来。

镜照一眼冰链：M3 的 ConOps 是”疫苗从出库到接种全程温度不断链”。运行场景里写着”运输车在高速上抛锚 2 小时”这种应急，直接决定了”断电保温时长”这条硬需求。ConOps 写不出来的场景，就是将来要出事故的场景。

冰链的另一面在需求管理：文正评审会上随口一句”精度差不多就行”，没有 CR、没有记录，一句口头变更把验证的靶子当场换掉。一个缺的是把系统怎么被用写下来，一个缺的是把改动管起来。

### 4.4.6 认知反转

回到章首那场会。IT 总监说”我们要加强需求管理”。现在我们知道，他说错了一半。

需求工程是需求开发与需求管理两段并列，不是包含。恒信的需求乱，一大半是需求开发的欠账：没调研（来源不清）、没分析（“重要合同”没澄清）、没写可验证的规格（三百条悬空条款）。这些欠账，管理补不了：管理能让一份需求一直是那一份，不能让一份没说清的需求变清楚。

徐漾补的三样东西，各自补在了正确的位置上：ConOps 补上问题域（先讲清怎么被用，再写条款），需求基线补上变更的参照原点（改名留档不等于版本管理），CR 流程补上变更的痕迹（无 CR 不得变更）。

这一章的反转是：“需求管理”这四个字被用得太顺口，以至于人们忘了它不包含”需求开发”。需求是开发出来的，不是管出来的。

再深一层：恒信以为”加强需求管理”是往流程上加东西，结果一半是补开发的课。这提示一条普遍的次序，当一个团队说”要管起来”的时候，先问一句：源头清了吗？管理的价值建立在开发的质量之上，这是需求工程里最容易被倒置的一对关系。

这一章还改了两句流行话的说法，结论没变、理由变了：

- “说了不等于规定”。说出来了就是规定需求（ISO 9000:2026 §3.5.1 注 2 是一个等号）；要紧的是**说出来了不等于能当依据**，缺的那一样叫可采。
- “需求评审就是需求确认”：需求文档是工作产品，**能评审、也能验证，做不了确认**——确认要在使用中兑现。

用第 1 章的话收一次束：这一章从头到尾演示的是同一只手。先立枢纽、再填细节。第一节立”开发与管理并列”这个枢纽，ConOps 立”问题域”这个枢纽，四处边界一条条磨开，都在对”陷细节而不立枢纽”说不。

对”需求工程”这个词，你原来只是”见过”，背得出”需求工程 = 开发 + 管理”；这一章把它推到”拥有”，能答全四问：它是什么，开发与管理两段并列；不是什么，明示与可采、开发与管理、评审与验证确认、留档与受控这四处边界；从哪来，15288 的需求侧过程、CMMI 与 29148 的两分；往哪去，先问题域再条款。能答全四问，才算拥有。

用第 1 章 §1.4.4 的尺子量一下：这一章把”需求工程”推到了**第 ② 级（图式）**。你现在看得出”开发做出来、管理看住、批准与基线交接”这条结构，而不只是背得住那个并列式。

## 小结

承接第 3 章，这一章讲需求是怎么被开发出来的：需求工程把模糊的诉求变成能当依据用的规定需求。需求开发负责做出来（获取、分析、规格说明、需求校核；以 ConOps 为问题域、SRS 为载体、规定需求为枢纽），需求管理负责让它一直是那一份（基线管理、变更控制、可追溯性、状态与度量），两段并列，交接只在”批准 → 基线”这一处。

它磨开了四处边界：明示与可采（说出来了不等于能当依据）、开发与管理（需求乱先分清是谁的账）、评审与验证确认（需求文档做不了确认）、留档与受控（改名不等于版本管理）。全章反复演示的是同

一个方法底座时刻：先立枢纽、再填细节，也就是先立问题域、再写条款。

需求域的那份产品数据，也在这几周里补齐了：ConOps、第一版需求基线、CR 流程，都是徐漾一件件补起来的，第 7 章会看到，这些东西本身就是产品。

下一章看另一件事：需求要全到什么程度。功能、环境适应性、可实现性三类需求及其指标齐备，规格才算立全；而需求工程产出的规定需求怎么进设计开发的树形结构、怎么被分解和分配，是再下一章（第 6 章 设计开发落地）的事。

---

# 附录

## A 本章概念总图

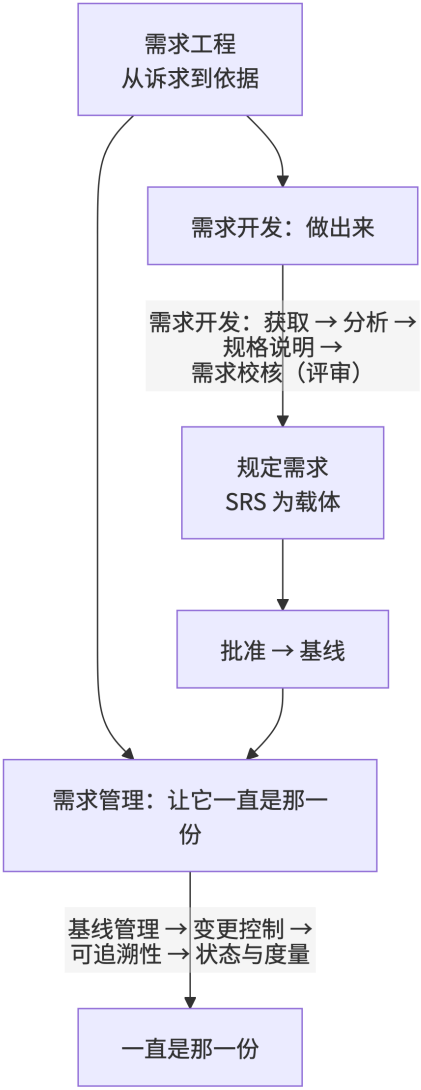



图 9　图 4-2

- 第一节 · 开发与管理并列：需求工程 = 需求开发 + 需求管理（不是包含）；四个活动与循环；需求的来路；规定需求与可采条件；管理的四个活动、变更控制、基线；两段交接在”批准 → 基线”；
- 第二节 · 四处边界：明示与可采（说出来了不等于能当依据）／开发与管理（先分清是谁的账）／评审与验证确认（需求文档做不了确认）／留档与受控（改名不等于版本管理）；
- 第三节 · 怎么做：ConOps 在分析之后、规格之前（先立问题域，它同时是确认判据的载体）；一条好需求的判据；需求文档体系与不可省的骨架五份（SRS · 需求池 · RTM · 评审报告 · 基线）。

恒信走完这张图：徐漾补 ConOps（问题域）→ 写 SRS（条款）→ 补 RTM 与基线 → 变更走 CR。冰链走的是同一张图，只是把”审批流转”换成了”温度不断链”。

## B 本章的能力级

**第 ② 级为主**

- **表征**——把”需求工程”“需求开发”“需求管理”“规定需求”“运行概念 ConOps”几个词说准：两段并列而非包含；规定需求是被明示的需求（明示即规定，成文属可采条件）；ConOps 是需求工程里唯一一份叙事性的正式输出；
- **图式**——两段路径合成的结构：“开发做出来 → 批准 → 基线 → 管理让它一直是那一份”，以及这条路径上的四处边界（明示与可采、开发与管理、评审与验证确认、留档与受控）；

- **心智模型**——欠账二分（需求乱先分清是开发的账还是管理的账）；可采条件四项——相关性、可检验性、时效性、同一性，缺判定方法、不可复现、没有版本，就当不了依据；变更控制怎么走完一圈、三条铁律（无 CR 不得变更、不分析不得上会、验证通过方可关闭）；
- **解释框架**——本章不建，由第 17、18 章承担。

> 能答全四问，才算拥有——分清”做出来”和”一直是那一份”：先补开发的课，再谈管理的严。

## C 自查 · 这些说法对吗？

1. “需求管理包含需求开发，管好需求自然就开发好了。”——**错**。两段并列不是包含；开发是创造、管理是控制；需求是开发出来的，不是管出来的。
2. “客户说了’页面要快一点’，这算不上需求吧。”——**错**。说出来了就是明示的需求，明示即规定；它缺的是可采——四条里只对得上相关性，没有判定方法、不可复现、没有版本，所以当不了验证的靶子。
3. “需求写完了开个评审会，就是确认通过了。”——**错**。需求文档是工作产品，**对它做的是评审与验证**（评审判适宜、充分、有效；验证认定它满足文档自身应满足的规定需求）；**确认**要等有东西可用、并且用起来才做。

4. “需求变更多，说明需求管理没做好。”——**错**。先分清是需求变了，还是当初没说清——后者是开发的欠账，加流程治不了。
5. “版本文件名改一改，就是需求管理。”——**错**。那是留档，不是受控；没有基线，变更就说不清”从什么变到什么”。
6. “ConOps 是给运营部门看的故事，跟开发没关系。”——**错**。它是问题域那一侧的正式输出，也是确认判据的载体；跳过它直接写 SRS，条款就悬空。
7. “需求分析就是开会讨论一下需求。”——**错**。分析有五件具体的事（澄清、分解、排序、建模、协商冲突），每件都有可检验的产出。
8. “写需求文档就是把用户的话抄下来。”——**错**。抄下来的是获取的原始记录；规格说明要把分析结果写成可验证的条文。
9. “隐含需求会自己冒出来，客户没提就是没有。”——**错**。隐含需求不会自己开口，要主动去挖；挖出来还得显性化、写进规格，否则验证够不着。
10. “需求开发四个活动跑一遍就完。”——**错**。它们是一个循环：分析发现缺信息回获取，规格发现问题回分析；需求天然是渐进的。

## D 练习

1. 用”开发做出来、管理让它一直是那一份”给恒信的困境做诊断：徐漾列的两张单子分别是什么？你们团队最近一次需求混乱，属于哪一张？

2. 找出你们公司一条”明示但不可采”的需求（说了，但拿不出客观证据），把它改写成可采的规定需求。
3. 你最近一次写需求（或接收需求），有没有跳过分析直接进规格？跳过了，代价是什么？
4. 你们团队的”需求评审”有没有被叫成”需求确认”？如果改口叫评审，签字那一刻的责任归属会有什么变化？
5. ConOps 和 SRS 的区别是什么？“先写条款、后补故事”为什么顺序反了？
6. 用六条判据给一条你正在做的需求打分：完整、一致、无歧义、可验证、可追溯、有优先级——哪一条最弱？后果是什么？
7. 你们当前项目的需求基线在哪？没有基线的话，最近一次变更”从什么变到什么”说得清吗？
8. 需求变更走完一圈（CR → 影响分析 → CCB → 实施 → 再验证 → 关闭），你们哪一环最常被跳过？跳过之后那条变更还查得到吗？
9. “隐含需求不会自己开口”——用恒信财务那句”25 号之后签的合同算下个月”举例，说明隐含需求怎么被挖出来、怎么显性化。
10. 用 MoSCoW 给恒信合同审批系统的需求排一次序（必须/应该/可以/不要），“合同数据留档十年”属于哪一档？为什么？
11. 需求”从哪来”？用三条来路（目标转化、环境与法规、隐含需要）各举一例，说明哪一条最容易被漏掉。

（答案要点见书末附录，与训练教材题卡同源）

## E 参考文献

**标准** - R-103 ISO 9000:2026（第 5 版）—— §3.5.1 需求（注 2：规定的需求是明示的需求，例如写在成文信息中的需求）、§3.11.2 评审、§3.11.12 验证、§3.11.14 确认、§3.3.11 设计和开发 - R-102 ISO/IEC/IEEE 15288:2023 —— §6.4.1 业务或任务分析、§6.4.2 利益相关方需要与需求定义、§6.4.3 系统需求定义（需求工程的过程锚点）- R-03 ISO 9001:2015 —— §8.3.3 设计开发输入（可转化判据在标准里的位置；库内无正本，条款号照录、版本随引）- R-12 ISO/IEC/IEEE 29148:2018 —— 需求工程（需求开发与需求管理并列；单条需求质量特性与需求集特性；shall 语句形式规则）（库内无正本，本书按二手来源处理） - R-13 IEEE 1362 —— 运行概念 ConOps（结构六部分，运行场景为核心）（库内无正本） - R-18 CMMI（REQM / RD）—— 需求管理与需求开发是并列过程域（库内无正本） - R-86 IEEE 830-1998 —— 软件需求规格说明书推荐实践（好 SRS 八特性，29148 的前身经典）（库内无正本）

**经典文献** - R-51 MoSCoW 优先级方法（源自 DSDM）—— 需求优先级排序（Must / Should / Could / Won’t） - R-52 Kano 模型（1984）—— 基本型/期望型/兴奋型需求分类——“与利益相关方一起剪裁需求” - R-87 Boehm & Basili，“Software Defect Reduction Top 10 List”（IEEE Computer, 2001）—— 需求缺陷的实证溯源（约半数软件错误到后期才被发现，判据从失败反推的经验锚点）

# 第 5 章 需求的完整性：谁在问，谁在验

## 5.1 引言

第 4 章把规定需求做出来了，也让它一直是那一份；那一章没有回答的是：这一份需求本身，全不全。这一章看的就是这件事。先把”全”拆成几类内容看清，再把最容易混的几处边界画开，最后看它在项目里怎么用。

### 5.1.1 误解现场

冰链科技的 M3 医疗冷链温控箱，原理样机做出来了。

水忠把一台样机摆在桌上：它能稳定把箱内温度控制在 2°C、波动不超过半度。金属箱体、半导体制冷片、一块单片机，走线是手焊的，电源是实验室的。它跑了一周，温度曲线漂亮极了。

文正站在白板前宣布：“原理样机验证通过，我们的产品成功了。接下来就是放大生产。”

产品开发经理永兵点了头：“样机跑通了，验证也过了——放大生产，我签。”

会议室一片掌声。只有角落里制造代表逸人皱着眉。他翻着样机的零件清单，摆了三件事，最后问了一句：

> “制冷片我们买得到，可这颗控温芯片是定制料——供应商说最小起订量五万片，交期二十周。外壳是 CNC 一件件铣出来

> 的，量产要改注塑，这个结构和壁厚注不出来。还有，这个精度是靠手工校准的，产线上没人会做。你们管这些叫’成功’？”

会议室安静了。

如果你坐在冰链的会议室里，大概也会觉得文正没错：样机都跑通了，验证也过了，怎么会不是产品成功？

这一章要推翻的正是这个直觉。它错在一个词上：“原理样机验证通过”只验证了一类需求。

### 5.1.2 这一章要回答什么

读完本章，你应该能回答：

1. 一个产品的需求要”全”，要覆盖哪几类内容？为什么每一类都必须有指标？
2. 给需求分类，看什么？为什么同一个特性会同时落在两类需求里？
3. 为什么”原理样机验证通过”不等于”产品成功”？哪一类需求只有真正的产品才能验？
4. 三类内容各自由谁提出？为什么可实现性需求客户根本不关心？
5. 为什么说承制方是需求开发的主导方？“引导”与”确认”的边界在哪？
6. 一个组织怎么才能不反复踩”可实现性需求缺失”的坑？成本为什么是需求、不是结果？

### 5.1.3 这一章的骨架

> 需求要全，全在三类内容上：功能需求、环境适应性需求、可实现性需求各有各的代言人，各有各的专属验证载体——缺哪一类，交付物就降一格。而最容易被漏掉的那一类没有人替你要，所以”全不全”是承制方自己的责任。

反过来说更刺人：功能需求客户会要，环境需求法规会要；可实现性需求，只有你自己会要。

这一章分三块走。第一块把”全”拆开：一份需求要覆盖哪些内容、每类由谁提出、为什么每类都要有指标。第二块把最容易混的几处画开：样机通过为什么不等于产品成功、“非功能需求”这个分法为什么不能用、隐含需求为什么不是单独一类。第三块回答最扎心的那个问题：可实现性需求客户不关心，那它靠谁提、靠什么补、缺了怎么诊。

---

## 5.2 需求的完整性是什么

> 这一块先立”全”的答案：需求要覆盖哪些内容、按什么判据分类、每一类各自包括什么、从哪里来。

### 5.2.1 一份全的需求：覆盖什么

第 3 章立过一句话：需求必须可判定。第 4 章立过另一句：需求是开发出来的，要能当依据用。这一章往前走一步，问一个更基本的问题：一份需求要”全”，到底要覆盖哪些内容？

答案藏在”产品”这个词里。一份讲正向设计的培训材料里，“产品”与样机的分别写得很干净：

> **产品**：满足功能需求、环境需求和可实现性需求的成品。

也就是说，“产品”这个词的成立，本身就要求这三类内容都在场。反过来推：需求缺了哪一类，交付物就降一格。缺可实现性需求，交付物停在环境样机；环境适应性需求也缺，连环境样机都算不上。冰链那台 M3 缺的正是可实现性需求。

三类内容，每一样都不是一条笼统的要求，而是一组能展开、能设指标、能指派给利益相关方的条目：

| 问的问题 | 需求类别 | 一句话定义 |
| --- | --- | --- |
| 它能不能干我们要它干的事？ | **功能需求** | 系统所具备的行为特征，以输入、处理、输出方式定义 |
| 它在真实环境里能不能活？ | **环境适应性需求** | 对气候、国家、文化、风俗、法律等各类环境的适应 |
| 它能不能被经济、可重复地造出来？ | **可实现性需求** | 生存周期各环节”方便实现”，加经济可承受 |

这套分法不只对系统成立。构件的需求同样由这三块构成，与系统完全同构。同一个模板，在系统级和构件级各复现一次。

> 术语注：这一章把需求内容分成”功能／环境适应性／可实现性”三块，并让每一块各配一个专属验证载体，是本书对军品与工程界”完整需求”实践的**凝练与命名**，不是任何标准的原文。这个分法的一手来源是这份培训材料（§7.2 给出这三类内容的正式清单），本书在这份材料之上做了两件事：把分类判据统一到”谁在问”，把内容与载体的对应讲成一条因果。标准这边给的是另外两样东西：需求本身的定义与三种呈现方式（ISO 9000:2026 §3.5.1），以及需求过程的框架（ISO/IEC/IEEE 15288:2023 §6.4.2、§6.4.3，第 4 章已引）。欢迎检验、欢迎校准。

### 5.2.2 需求分类：看谁在问

这三类最容易搞混的地方，是拿”特性的名字”来分。但分类的判据不是特性本身，是**谁在问**：

- **功能需求**，是客户与使用者在问：我要的产品，做不做得到我要的事？
- 环境适应性需求，是使用环境在问（监管方、气候、法律）：你到了我的地盘，守不守规矩、活不活得下去？

· 可实现性需求，是承制方在问：我造得出来吗？造得起吗？交付得了吗？维护得了吗？

这条判据解释了为什么同一个特性会同时出现在两类需求里。拿”测试性”说：它在使用环境下被问，问的是”产品在役期间测得出状态、定得了位”，归环境适应性需求。它在承制方的生产与交付里被问，问的是”产线上测得方便、测得快”，归可实现性需求。同一个特性，两个主人，两种归类。

叫法也跟着主人走：在使用环境下被问，写”测试性”；在承制方的生产交付里被问，写”可测试性”。

### 5.2.3 功能需求：包括什么，从哪来

功能需求是系统所具备的**行为特征**，以输入、处理、输出方式定义。它不是”要有个温控功能”，而是”给定箱内目标温度 X，处理器采样、决策、输出制冷驱动，使实测温度稳定在 X±0.5℃“。有输入可构造，有输出可检查，这样才可验证。

它还包括一条从业务走到系统的展开链：

**业务用例 → 业务场景 → 系统用例 → 用例场景**

**（用产品完成什么业务）（典型过程）（系统为支撑业务的行为）（可运行的具体过程）**

这是三类内容里唯一自带展开通道的一块：环境适应性需求靠环境分析展开，可实现性需求靠实现方案展开，只有功能需求能从”利益相关方要办什么事”一路走到”系统在什么场景下做什么动作”。这条链也是后面把需求往下分解时最主要的素材来源（第 6 章）。

**来源**主要有五个：

1. 客户与使用者的需要与期望——三类内容里最强的一个外部代言人，客户最愿意、也最能主动表达的就是这一块；

2. 业务用例与场景分析——不是听客户罗列功能，而是把业务建模成用例与场景，再从场景里推出系统该有的行为；
3. 需求分析师的专业捕获——客户以为自己在提需求，往往提的是”想要的东西”甚至”解决方案”，分析师要穿透到”真正的需要”。这一类需要常常连客户自己都没意识到，因而往往不成文；
4. 组织资产——同类业务的用例骨架、场景模板可以跨产品复用，但功能需求的主体仍是产品特异的；
5. 利益相关方确认（签认）——需求是对需要或期望的规范性陈述，并经利益相关方确认，才算成立。

功能需求的陷阱不是”没人提”，而是”提了但没提对”：客户自信满满说出的，常常是方案，不是需要。

### 5.2.4 环境适应性需求：包括什么，从哪来

环境适应性需求是对**各类环境**的适应。这里的”环境”是广义的：不只是自然环境，还包括法规环境与社会文化环境。

它通常展开成这么几项，每一项都答”在什么环境下，产品必须做到什么”：

1. **可靠性**——在规定条件与时间内完成规定功能、不失效，跑冷链车不出故障；
2. 安全性——运行与维护中不产生不可接受的伤害或风险，不漏电、不伤人；
3. 测试性——产品在役期间便于测试与故障定位，能快速确认自身状态；

4. 保障性——使用与维护所需的保障资源（备件、工具、人力）易于配备；
5. 适航性——符合航空器适航法规，可安全进入空中运行；
6. 气候适应性——在温度、湿度、盐雾、海拔等气候条件下保持正常功能；
7. 电磁兼容性——在电磁环境中既不受外界干扰，也不对外界产生超标干扰；
8. 法律合规性——符合投放地域的法律法规与强制性标准，例如环保回收的新规。

负责 App 与追溯平台的董勐跟着补了一句：“冷链车进了山区连不上网，App 得先把温度数据暂存在本地，出了山区再补传——在弱网环境下，产品必须不丢数据。”

它与可实现性需求有一处重叠：测试性、保障性这一类特性，在那边也会出现（写作可测试性、可运维性）。这不是重复，是同一个特性同时服务两个主人：面向使用环境，产品在役期间要测得出、保得住；面向承制方，生产、交付、运维过程中要测得方便、维得方便。归到哪一边，还是看谁在问（§5.2.2）。

来源主要有五个：

1. 产品投放地域的自然与电磁环境本身——卖到哪、用在什么环境，需求就从哪来；
2. 强制性的标准、法律、法规——这些不是可选项，捕获的动作是”摘录”：从法规文本里摘进需求条目；

3. 文化、风俗、国家——使用地的社会文化环境，决定人机交互、内容、外观、操作习惯方面的要求；
4. 环境利益相关方，并经其确认——监管方确认合规，最终用户确认真实环境下的使用预期；
5. 组织的环境适应性需求资产库——这一类需求不应从零发明，应基于组织积累下来的资产开发。

捕获环境适应性需求的关键不在技术，在先把产品打算投放的完整环境集定下来，再逐项确认。

> 术语注：法规强制的那几条，属于需求三种呈现方式里的”必须履行的”。正本给过两个条目：法定要求是立法机构规定的必须履行的需求（ISO 9000:2026 §3.5.5），法规要求是立法机构授权的机构规定的必须履行的需求（§3.5.8）。三种呈现方式与本章的内容分类是两条轴，§5.3.5 会把它们厘清。

### 5.2.5 可实现性需求：包括什么，从哪来

可实现性需求是承制方对自己提的要求，一共八个方面：前七个各对生存周期的一个过程，第八个是横切这些过程的总账：

| 生存周期过程 | 可实现性需求 | 对应的实现方案 | 提出者 |
| --- | --- | --- | --- |
| 物料采购 | 物料可采购性 | 物料采购方案 | 采购业务代表 |
| 制造 | 可制造性 | 制造（集成）方案 | 制造业务代表 |
| 测试 | 可测试性 | 测试方案 | 测试业务代表 |
| 交付 | 可交付性 | 交付方案 | 交付业务代表 |
| 部署 | 可部署性 | 部署方案 | 部署业务代表 |
| 运维 | 可运维性 | 运维方案 | 运维业务代表 |
| 回收 | 可回收性 | 回收方案 | 回收业务代表 |
| 生存周期全程 | **经济可承受性** | 成本方案 | 市场代表 |

这张表有三处要点。

其一，三条线是锁死的：过程、需求、方案。实现方案是对可实现性需求对应内容的**符合性解答**：物料采购方案回答物料可采购性需求，制造方案回答可制造性需求，如此类推。一条可实现性需求缺了，与它对应的那条方案也就没有依据。

其二，最后那一条与前面七条不是并列关系，它是横切前面七项的总账。经济可承受性差，几乎总是”物料不方便采购、不方便制造、不方便测试”等等共同把成本堆上去的结果；它算的是总账，所以它是**需求项**，不是过程。七个过程加上这条总账，才是完整的八个方面（§5.4.6 单独说它）。

其三，构件级与系统级同构。可实现性需求也还是那些方面，只是构件级的”可运维性”在有些材料里写成”可服务性”。那是同一个方面在系统级与构件级的两种叫法，本章一律按系统级的叫法写。

> 术语注：第 3 章讲过设计开发的终点是”可实现”，那时说的是设计输出的完成度（下游各方案配齐、输出完整的规定需求）；本章讲的”可实现性需求”是需求集合里的一类内容（承制方在需求阶段提出的诉求）。一个是终点判据，一个是需求类别，两处不是一回事，别混。两张表合看才完整：第 3 章从设计输出面向下游，本章从承制方在需求侧提出诉求。另有一张表要分清：本书讲设计开发的终点”可实现”时，列的是下游各方案（技术、物料采购、制造、测试、交付、部署、操作、运维）；本章这八个方面多”回收”与”经济可承受性”、少”技术”与”操作”——**两者不是同一张清单的两种写法**。

来源主要有五个：

1. 承制方各业务代表——采购、制造、测试、交付、部署、运维、回收的业务代表，加市场代表。可实现性需求不是客户提的，是组织对自己提的；
2. 承制方现有的能力与资源约束——人、机、料、法、环，需求的指标边界据此设门槛；
3. 开发过程中的现实检验——产品开发本身就包括开发加工与装配技术、开发供应商、研制新设备与工装；一旦某条实现不了（物料买不到、技术不成熟、设备拿不到），必须回头改技术方案，而不是悄悄绕过这条需求；

4. 组织资产——工艺、供应商、设备方面的长期积累；

5. 市场与竞争研究——经济可承受性来自对竞品与替代产品的价格成本研究。

客户不关心这一块，所以得由承制方自己提。客户只关心结果：价格、交期、质量、售后；他不关心你的成本结构、物料好不好采购、好不好制造。更麻烦的是，客户还可能反向挤压它：定制化要求违背货架产品策略，不停加功能抬高制造难度，压缩报价挤压成本空间。可实现性需求常常是那个需要组织顶住客户压力才守得住的底线。

### 5.2.6 每一类需求：都要有指标

三类内容各自包括什么、从哪里来，到这里都看清了。还剩一条三类共用的规矩：需求等于描述加指标，缺了指标就不成立。

拿 M3 说：

- **功能需求的指标**：温控精度、响应时间、续航时长、容量；
- 环境适应性需求的指标：工作温度范围、防护等级、电磁兼容的限值、法规条款的符合性；
- 可实现性需求的指标：目标成本、关键尺寸公差、交期、服务周期。

一句”要控温准”是愿望；“箱温波动不超过 ±0.5℃”才是需求。没有指标的需求，三块都会退化成愿望。

> **概念卡片 · 三类内容**

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 产品需求要全，全在三类内容上：功能需求（以输入、处理、输出定义的行为特征）、环境适应性需求（对气候、国家、文化、风俗、法律等各类环境的适应）、可实现性需求（生存周期各环节”方便实现”加经济可承受）——每块都”及其指标” |
| **它不是什么** | ⑥⑦ | 对立面：只写功能需求，把”产品需求”当成”功能清单”，这是最常见的伪全。辨析：功能与非功能的二分是这三块的一个子集，不是它的替代（§5.3.4 展开）；隐含需求也不是单独一类，它是”怎么 |
| **它来自哪** | ④⑤⑧ | 各 |

## 5.3 需求的完整性不是什么

> 这一块把”全”和它容易混的样子分开：先立每类内容各配一个验证交付物，再逐条磨开四处边界。

### 5.3.1 验证载体：各管一类需求

三类内容各自对应一个**专属的验证交付物**，本书叫**验证载体**。这不是分法的巧合，是一条因果链：

**需求类别 → 决定验证条件 → 决定交付物形态**

| 需求类别 | 要在什么条件下验证 | 验证载体 | 确认方 |
| --- | --- | --- | --- |
| 功能需求 | 把功能从环境与量产成本约束里**剥离**出来，单独验证原理 | **原理样机** | 客户与使用者 |
| 环境适应性需求 | 把产品放进**真实环境**去跑 | **环境样机** | 环境相关方 |
| 可实现性需求 | 在**量产经济性**约束下实现出来 | **真正的产品** | 承制方 |

为什么必须各有一个?因为三类内容需要的验证条件互不相容:验证”原理对不对”，环境干扰越少越好；验证”真实环境下活不活得下来”，就必须上真实环境；验证”能不能经济、可重复地造出来”，只能在量

产的约束下见真章。一个载体替不了另一个。原理样机跑通了，也证明不了你量产能做出来。

负责测试的李旭点头应道：“那我手里的活也跟着载体走——原理样机上我只能测功能，环境样机上测真实环境，能不能上产线，得等真正的产品出来。按这一章的说法，每一类需求各有一个专属载体；样机不换，测来测去还是那一块。”

工程界有一个同构的刻度可以借来看这件事：美国航空航天局的技术就绪水平（TRL），从”基本原理被观察到”到”在真实环境中运行”，一共九级。原理样机的演示通常落在中间几级，离”真实环境运行”还隔着距离；而”能不能造出来”归另一个刻度（制造就绪水平 MRL）管。这两种就绪水平都不在本书的库内，属二手来源，这里只借它们说明一件事：样机通过，不等于产品就绪。

概念卡片 · 验证载体

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 每一类需求的专属验证交付物：功能需求对原理样机、环境适应性需求对环境样机、可实现性需求对真正的产品 |
| **它不是什么** | ⑥⑦ | 对立面：用一个载体验证全部需求——拿原理样机的通过宣布产品成功。辨析：验证载体与”评审、验证、确认”（第 8 章）不是一回事——载体回答”每一类需求用什么证明”，**评审、验证、确认各自回答”对照什么”**；另外，确认的参照物是使用情境、运行概念、用例这一族东西，它与三类验证载体并列，不能互相替 |
| **它来自哪** | ②③⑧类 | 每一类需求用什么验证，由在验证阶段的物理样机的通理 |

载体、验证、确认，这三样不是一回事，这里先划一条边界。三类载体承载的是验证，判据是明示的规定需求（ISO 9000:2026 §3.5.1 注 2）。

而”确认”对准的不是条目，是为特定预期用途或应用的需求。确认要在真实或模拟的使用条件下做一次（§3.11.14 注 3：使用条件可真实可模拟）。所以环境样机上”逐条环境指标是否达标”属验证，“放进真实使用环境看合不合用”属确认；确认的参照物是使用情境这一族东西，与三类验证载体并列，不能互相替代。展开留给第 8 章。

### 5.3.2 做全与验过：不是一回事

细看会发现，交付物是一级级**累加**起来的：环境样机当然得先把功能做全（不工作就谈不上环境适应），真正的产品当然得先满足功能与环境。所以很多人顺口就把”原理样机、环境样机、真正的产品”当成需求逐级累加的三段，以为做到最后一段，前面几块内容就都验过了。

累加的是交付物，**不是验证**。每一块内容有且只有一个专属载体：功能需求靠原理样机验证，环境适应性需求靠环境样机验证，可实现性需求靠真正的产品验证。原理样机不验证可实现性需求，环境样机也不验证它。

可实现性需求，只有真正的产品（量产、交付、运维跑过一遍）才能验证。这就是”原理样机通过”与”产品成功”之间那段距离的来源。

### 5.3.3 样机与产品：通过不等于成功

回到章首那场会。文正说”原理样机验证通过，我们的产品成功了”，他做对了什么？他确实验证了功能需求：温控原理成立，这一关过得漂亮。

他漏掉的是什么？环境适应性需求还没验：真上了冷链车，温差、振动、零下二十度的库房，曲线还平不平？可实现性需求根本没写进需求清单：定制料断供、注塑结构做不出来、手工校准上不了产线。

精确地说，他把**一个载体**的验证结果，当成了**全部载体**的验证结果。而那些”不方便”，压根不是”问题”，是可实现性需求缺失的症状。

一句话：样机通过只证明了它是一台好的原理样机。把它当产品，是需求的完整性欠账在交付环节结的账。

### 5.3.4 功能与非功能：这么分行吗

工程界最常见的需求分类不是三分，是两分：功能需求与非功能需求。这个分法听着顺手，却是需求分类里最误导人的一个。

“非功能”是一条排除法划出来的：凡是不能当场演示”它能干什么”的，全被划到这个名目下。性能、可靠性、环境适应性、可维护性、可部署性、成本，它们的主人、来源、验证载体全不一样，却在同一个名字下被当成同类需求处理。

拿”谁在问”这条判据一量就清楚了：这个名目底下装着的，是**环境适应性需求**（环境在问，用环境样机验）和**可实现性需求**（承制方在问，用真正的产品验）两块。

它的害处有三条。一是判据丢了：排除法答不出”谁在问”，你问”这条非功能需求该归谁提”，没人答得上来。二是验证载体糊了：环境样机与真正的产品是两回事，装进同一个名目就分不清该用哪个去验。三是”全”的标准消失了：“非功能”没有上限，只能越列越多；而”内容分几块、每块及其指标”有判据、有终点。

所以本章不用”非功能需求”来给需求分类：功能与非功能的二分，是三类内容的一个子集，不是它的替代。判据主义在这里磨出的一条边是：别用排除法给需求分类，要用”谁在问”。

### 5.3.5 隐含需求：算不算单独一类

前面磨掉了一条边：别用”功能与非功能”给需求分类。这里再磨一条，专治”三”字撞车：“怎么出现”与”归哪一类”，是两条不同的轴。

第 3 章讲过需求的三种呈现方式：**明示的**（说出来了）、**通常隐含的**（惯例默认、没说出口）、必须履行的（法规强制的那一类），三个都是需求。本章讲的是按内容分类：功能、环境适应性、可实现性，判据是”谁在问”。

两条轴互不相干。一条功能需求可以是明示的（合同写明”箱门可单手打开”），也可以是隐含的（谁都没写，但你漏了，客户第一天就会说这产品不行）。一条环境适应性需求多半是必须履行的（法规摘录），也可以是惯例默认的。

三种呈现方式落到哪一支，大致有偏向：环境适应性需求多为必须履行的那一支，可实现性需求多为明示的那一支（组织自设），功能需求则是明示与通常隐含两支的混合。

所以隐含需求不是单独一类。它不是”谁在问”的一种答案，而是”怎么出现”的一种形态，任何一块内容都可能以隐含形态出现。

隐含需求的来路是一条旁路。需求的上游大致有三条通道：从目标转化而来、从环境与法规而来、从利益相关方没说出口的需要而来。隐含需求正是第三条通道的产物。它不经目标转化，未必被目标覆盖，而且常常不成文。客户自己都没意识到的需要，写不进目标，也就无

从”转化”，只能由分析师从利益相关方的需要里挖出来（§5.2.3 来源三）。

隐含需求不成文，靠文档做的验证够不着；拦得住它的，是用真实使用场景做的确认（第 8 章）。这也给本章的判据补了一条边界：“谁在问”只回答”归哪一块”，不回答”怎么出现的”。归好类之后还得追问一句：这一条是明示的，还是隐含的？隐含的那一条，才最容易被漏掉。

---

## 案例进展①｜恒信的”样机”是什么

恒信的合同审批系统没有物理样机，但同一副框架照样成立。徐漾复盘时发现，公司里反复出现的”系统上线后一堆问题”，很多不是功能问题，而是三类内容在 IT 世界里以另一种形态缺位：

- **功能需求**（合同能发起、能审批、能归档）——恒信做得最早，原型阶段就把流程跑通了；
- **环境适应性需求**（月末五百人并发、审计留痕、内网安全合规）——只在试点（放进真实业务环境跑）时才验；
- 可实现性需求（可部署、可运维、可升级、成本可承受）——只有真正的产品（投产、运维跑过一年）才验。

恒信的”样机”是原型、试点、真正的产品；冰链的”样机”是原理样机、环境样机、真正的产品。同一副框架，两个世界，长一个样。

徐漾在复盘会上写下一句：“恒信大部分’上线翻车’，翻的不是功能，是可实现性需求——部署文档缺失、运维没人接、升级要停机、

一年运维成本是预算的两倍。原型跑通那天，这三样一条都验不了；要等真正的产品跑过一年，它们才会浮出来。”

---

## 5.4 需求的完整性怎么用

> 这一块回答最扎心的问题：可实现性需求客户不关心，那它靠谁提、靠什么补、缺了怎么诊。

### 5.4.1 谁主导，谁确认

前面把三类内容的来源一条条列过了。需求开发其实是一场三方戏：

| 角色 | 是谁 | 干什么 |
| --- | --- | --- |
| **来源方** | 客户与使用者、监管与环境、组织资产 | 提供原始素材：需要与期望、强制性标准、经验与问题 |
| **主导方** | **承制方** | 驱动需求开发全过程：捕获、分析、建模、引导范围、形成条目 |
| **确认方** | **利益相关方** | 终审：需求条目成立，以利益相关方**确认（签认）**为准 |

来源方提供原料，承制方主导加工，利益相关方负责确认。客户说了算的不是”需求是什么”，而是”这条需求成不成立”：内容由承制方主导，成立由利益相关方确认。

> 术语注：这里的”确认”是**签认受理**——需求条目成不成立；不是第 8 章的 validation（对预期用途的确认，ISO 9000:2026 §3.11.14）。同一个词两个义，本章分开写。

承制方凭什么主导？因为客户知道**自己要什么**（他的业务、他的痛点），却不知道另外四件事，而这四件恰好是承制方的强项：

1. **什么技术可能**——有没有更优的实现路线；
2. 行业横向视野——服务过很多客户与产品，知道同类业务普遍缺什么、普遍踩什么坑，单个客户永远没有这个广度；
3. 标准与环境——法规、环境、行业规范，是承制方在摘录；
4. 资产积累——组织长期攒下来的经验、建议、问题，这是专业性的物质基础。

正因为这四条，承制方不只是被动接收客户需求，而是主动引导需求：把客户”想要的”穿透成”需要的”；提出客户没想到的需求（行业惯例、标准、未来趋势）；建议技术路线与范围；把可实现性约束主动注入需求。

引导能力本身也是一条判据：只能逆向模仿的承制方，是被动接收客户规格的执行者，谈不上引导；只有具备理论推导内核的承制方，才可能提出客户自己都想不到的需求。正向设计的能力，从需求捕获那一刻就开始用了。

> 概念卡片 · 承制方主导

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 需求开发由承制方主导（捕获、分析、建模、引导范围、形成条目），利益相关方保留确认终审权 |
| **它不是什么** | ⑥⑦ | 对立面：客户主导，把需求开发当成”客户说、承制方记”；承制方对产品更专业，是主导方而非转述方。辨析：承制方（主导，负责加工）与来源方（客户、环境、资产，提供原料）、确认方（利益相关方，终审）三分工；引导可以主动，确认不能缺席 |
| **它从哪来** | ②③⑨ | 背景：客户知道”自己要什么”，却不知道技术可能、行业视野、标准环境、资产积累这四件事。需求开发只能由承制方主导加工，历史、源头承制方说了算 |
| **它到哪去** | ④⑤⑧ | 需求开发的产出不能直接用，必须经过分析、建模、引导范围、形成条目，才可以交付给设计与实现 |

### 5.4.2 主导不等于替客户拍板

承制方主导的是**过程与专业内容**，确认权始终在利益相关方手里。引导的**力度**可以拉满：该追问的追问到底，该提的主动提，该顶住压力挡回去的就挡回去。被限制住的从来不是力度，而是最后那一个动作：替对方做决定。

引导越界有几种常见的样子：为了可制造性，悄悄删掉一条功能；为了用货架件，把标准件硬塞进需求；用一句”客户其实不需要”把需求压下去。这些动作看着都是在替客户着想，实质是把自己的约束伪装成了客户的需求。一旦越界，需求就不再是”客户要办的事”，退化成承制方自己方便的设想。

所以这一层边界是一条铁律：

> 承制方引导可以主动到拉满，利益相关方的确认一次都不能省。

这条与第 14 章要讲的”目的的所有权在业务方”是同一件事：内容可以主导，目的与成立的终审在外部。后记里那份《系统目的或任务理解确认单》由业务方签字生效，走的正是同一条规矩。

### 5.4.3 全不全，是承制方的责任

既然是主导方，“需求全不全、明不明确、可不可度量”这三件事，就是承制方的责任，不是客户的。

客户可以说”我不知道自己要什么”，那是他的本分；承制方不能说”客户没提，所以需求不全”，那是失职。

这条责任在本章格外扎人，因为最容易漏的那一块内容，恰好是**没有外部代言人**的那一块。功能需求客户会主动提，环境适应性需求法规会强制提，可实现性需求，客户根本不关心，只有组织自己会要。没有人催的那一块，只能靠主导方自己记住。

### 5.4.4 可实现性需求从哪里补

可实现性需求还有一个性质，让它比另外两块好补：**它的复用度最高**。

功能需求随产品而变，环境适应性需求随投放市场而变；可实现性需求不一样：组织的供应链、工艺、设备、人员约束跨产品持续存在。过去的产品在制造上踩过的坑，下一个产品十有八九还会踩。采购难、装配难、运维难，这些痛点在同一个组织里几乎是常量。

| 类别 | 重新捕获的比重 | 复用组织资产的比重 |
| --- | --- | --- |
| 功能需求 | 高（随产品而变） | 低 |
| 环境适应性需求 | 中（随投放环境） | 中（法规与标准类可积累） |
| **可实现性需求** | **低** | **高（组织约束长期恒定）** |

所以这一块**最适合、也最应该**基于组织资产开发，而不是每个项目从零发明。它的来路是一个回路：

**问题 / 经验 / 建议 →（转化）→ 可实现性需求资产 →（复用）→ 新产品需求 →（产生新问题）→ 回到起点**

关键词是”转化”：问题不是照抄进需求，而是被提炼成一条规范的需求条目，并配上指标：

- 问题：“这批零件公差太严，装配车间返工率三成” → 转化 → 可制造性需求：“零件关键尺寸公差 ⩾ X，装配间隙 ⩾ Y”；
- 建议：“接口统一成标准件” → 转化 → 物料可采购性需求：“优先选用标准外购件（货架产品，COTS）”；

· 教训：“上一代低温试验反复失败” → 转化 → 环境适应性需求资产条目。

“自觉”两个字是这套积累成立的前提，也定义了它的反面：问题靠口口相传，只在现场救火，从不落成条目。这样的组织看着也在积累，其实那些经验是散的、隐性的、没法复用的。判断一个组织自不自觉，只看一条：它有没有一条制度化的回路，问题、分析、转成需求条目、入库、新产品复用、复盘更新。

于是有一个更硬的推论：没有资产积累机制的组织，天然捕获不到全面的可实现性需求。不是承制方的代表不肯说，是组织根本没有可复用的东西可以拿来提。“大部分团队都不考虑可实现性需求”，不是一次需求访谈的疏漏，是组织知识管理缺位的必然结果。

### 5.4.5 从症状反推缺了哪条需求

既然这一块内容这么重要、又这么容易被漏，那漏了之后怎么查出来？

先看它为什么会被漏——四个错位：

1. **利益相关方错位**：可实现性需求的提出者是承制方，不是客户。团队习惯只访谈客户，自然捕获不到；
2. **时间错位**：功能需求缺失，原理样机阶段就爆；环境适应性需求缺失，环境样机阶段爆；可实现性需求缺失，要到量产、交付、运维才爆——最晚，也最贵，于是在短期评估里显得”不重要”；
3. 话语权错位：研发阶段由研发主导，采购、制造、运维的代表没有在需求阶段发声的机制，等产品定了稿才说”做不出来”；

4. 认知错位：团队把可实现性当成”后续工艺的事”，而可实现性其实是产品的固有特性，必须写进需求。

四个错位导致一个最常见的现场：团队在量产时吐槽”物料难买、零件难造、测试难做、交付难搞、部署难办、运维难弄、退役麻烦、成本失控”，然后互相指责采购、制造、运维”没做好”。

但按本章的框架，这些症状不是执行问题，是需求缺失问题：

| 事后症状 | 缺失的需求条目 |
| --- | --- |
| 物料不方便采购 | 物料可采购性 |
| 零部件不方便制造 | 可制造性 |
| 不方便测试 | 可测试性 |
| 不方便交付 | 可交付性 |
| 不方便部署 | 可部署性 |
| 不方便运维 | 可运维性 |
| 不方便退役 | 可回收性 |
| 成本很大 | 经济可承受性 |

团队每抱怨一个”不方便”，就等于供认了一条当初没写进需求的可实现性需求。诊断的方向整个反过来：不是”执行要不要改进”，而是”需求起点全不全”。

这一招正是第 1 章那套方法里的反事实检验在需求侧的动作：第 1 章问”去掉这个枢纽，这个领域还成立吗”，本章问”没有这条可实现性需求，量产会怎样”。它把”缺了会怎样”从一句抽象假设，变成一次从现场症状反推的查账。

---

## 案例进展②｜冰链的量产翻车清单

逸人在会后列了一张单子，给文正看。这张单子后来被冰链叫作”量产翻车清单”：

> 定制控温芯片——物料不方便采购，最小起订量五万片、交期二十周；箱体外壳 CNC 铣出来——不方便制造，注塑的模具与壁厚做不出来；精度靠手工校准——不方便测试，产线上没人会、也来不及；电源适配器没有认证——不方便交付，进不了冷链资质清单；固件没有远程升级通道——不方便部署，客户现场只能返厂；备件库里没有温控模块——不方便运维，坏一块换整机；电池按环保新规要回收——不方便退役，回收成本比电池还贵；全部加起来——成本超预算四成，经济可承受性崩了。

文正越看脸越白：“这八条，没有一条是’制造没做好’，全是’需求当初没写’——定制料对应物料可采购性，注塑做不出来对应可制造性，手工校准对应可测试性。我们一直在查执行，该查的是需求起点。”

每一条都是一个”不方便”，每一条都对应一条可实现性需求：物料可采购性、可制造性、可测试性、可交付性、可部署性、可运维性、可回收性、经济可承受性。它们不是”量产时冒出来的问题”，是从一开始就缺位的需求。

永兵站在旁边，脸色也没比文正好到哪去：“放大生产是我签的字——我签的是’样机跑通了’，不是’产品造得出来’。第 3 章说设计

开发止于可实现，第 4 章说需求是开发出来的；这两句今天落到我头上：这份需求从起点就缺着一块，而那一块，从来没有外部代言人。”

### 5.4.6 成本是需求，不是结果

可实现性需求的八个方面里，最后一条要单独说，因为它的地位和其他几条不一样：**它是总账**。

经济可承受性差，几乎总是”物料不方便采购，加上不方便制造，加上不方便测试”等等共同把成本堆上去的结果。它有四处特殊。

第一，成本是需求，不是结果。产品的成本是相关需求经过分解、分配、分析之后的结果，不是单纯”设计出来”的。没有成本相关的需求，设计就没有依据和目标：每次选型、定工艺、定测试深度，都没有预算约束可以对着优化，成本只能失控。等量产报价发现太贵，再开”降本大会”换供应商、压工艺——每一次降本动作，都等于承认经济可承受性需求当初没写过。

第二，提它的不是客户，是市场代表。客户要的是报价越低越好，财务管的是账不是需求；提出经济可承受性这一条的是市场代表，依据是对竞品与替代产品的价格成本研究，给出价格与成本的建议，其他人负责承接和实现。团队没让市场代表在需求阶段发声，成本需求就缺位；就算设了一个成本目标，没确认利益相关方、没落到承接者，它仍然不是一条需求。

第三，它验证得最晚，几乎不可逆。成本要到真正的产品阶段才能用完整的成本记录来验证；而压成本的杠杆（优先选用标准外购件、避免定制化外包、把核心技术模块做成可复用资产）全是设计期的决定。等发现贵了再改，架构基线与物料清单已经冻结。

第四，它同样需要指标。目标成本、生存周期成本上限、降本目标——不可衡量，同样不成为需求。

### 5.4.7 认知反转

回到章首那场会。文正说”原理样机验证通过，我们的产品成功了”。现在我们知道，他把一**块内容**的验证通过，当成了**全部内容**的验证通过。

精确地说：水忠的样机验证了功能需求，温控原理成立，这一关他过得漂亮；环境适应性需求还没验；可实现性需求根本不在需求清单里。那八条”不方便”，不是量产时冒出来的执行问题，是需求起点就缺位的证据。

冰链交付的不是产品，充其量是一台经过验证的原理样机：原理对、能控温，但造不出、交付不了、用不起。

这一章的反转是：“产品需求”这几个字，大多数人以为等于”功能清单”。真相是功能只占一份，另外两份分别是环境适应性与可实现性；而最容易被漏掉的，恰恰是没有外部代言人的那一块——客户不关心你造不造得出来，只有组织自己会要，而组织自己又常常忘了要。

判断一份需求全不全，不要问”功能写全了吗”，要问”每一块内容，都能答出用什么交付物验证它吗”——答不上来的那一块，就是欠账。

还有更深的一层：需求全不全，是承制方的责任，不是客户的。客户可以不知道自己要什么；承制方不能说”客户没提，所以不全”。

而”承制方为什么不提可实现性需求”的答案，多半不是他们不肯，是组织没有积累下可提的东西。这给出一条普遍的提醒。当一个团队说”下次吸取教训”的时候，先问一句：这次的问题，有没有变成一条可复用的需求？问题不转成资产，教训就永远是教训。

用第 1 章的话收一次束：对”需求的完整性”，你原来只是”见过”，背得出”样机不是产品”，也喊得出”需求要全”。这一章把它推到”拥有”：能用”谁在问”给任何一条需求分类，能用验证载体查出哪一块欠账，能从一声”不方便”反推出当初没写的那条需求。能答全四问，才算拥有。

用第 1 章 §1.4.4 的尺子量一下：这一章把”需求的完整性”推到了第 ② 级（图式）。内容与载体那张对应表，你现在一眼看得出缺口在哪。

## 小结

承接第 3、4 章，这一章回答的是”一份需求要全到什么程度”。答案是把需求拆成三类内容，功能（客户在问，以行为特征定义）、环境适应性（环境在问）、可实现性（承制方在问），每块及其指标；判据是”谁在问”。

然后立起内容与载体的对应：原理样机验证功能需求，环境样机验证环境适应性需求，真正的产品验证可实现性需求。“样机通过不等于产品成功”不是一句谨慎话，是需求的完整性在交付环节结的账。

最后回答最扎心的问题：可实现性需求为什么总被漏掉。答案不是能力，是四个错位、没有外部代言人，以及组织没有积累下可提的东西。而这一切的责任都在承制方——它是需求开发的主导方，引导可以主动到拉满，利益相关方的确认一次都不能省。

把这条链反过来用，就是本章给的现场工具。从症状反推缺了哪条需求：每一声”不方便”，都是一条缺失需求的自白。

散会那天，文正把那句”我们的产品成功了”又说了一遍，说完自己先摇了头——他要的那份需求，从一开始就缺着一块，而那一块没有人替他提。他记住了自己欠的是哪一块账。

到这里，“要什么”这一问答完了：工程是什么，需求、设计开发、质量三个词，需求从哪里来、要全到什么程度。这些都立起来了，接下来看它怎么被用来造东西。

下一章是第 6 章 设计开发落地：需求与方案结对。它看的是这份完整的需求集合怎么被用来造东西：一份需求与一份方案结成一对，靠分配生成、分析迭代、分解分析，一层层变成可实现的方案。

# 附录

## A 本章概念总图

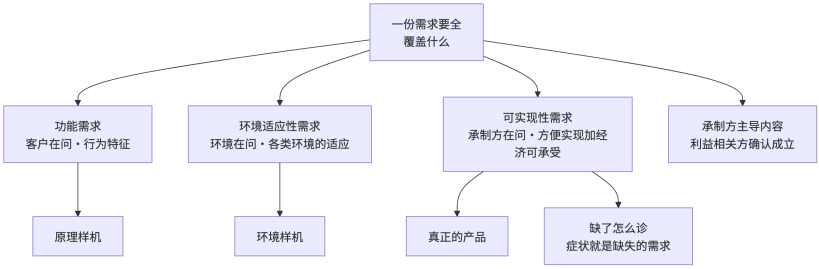



图 10　图 5-1

· 第一节 · 内容：功能需求（输入、处理、输出定义的行为特征，自带从业务到系统的展开链）、环境适应性需求（对气候、国家、文化、风俗、法律等各类环境的适应）、可实现性需求（生存周期各环节”方便实现”加经济可承受）；每块都”及其指标”；分类判据是”谁在问”；

- 第二节 · 边界：验证载体各管一类需求（原理样机、环境样机、真正的产品）；交付物累加而验证不累加；样机通过不等于产品成功；拿”非功能”给需求分类，分不出谁在问；隐含需求不是单独一类，它与内容分类是两条轴；
- 第三节 · 怎么用：承制方主导、利益相关方确认；责任在承制方；可实现性需求靠组织资产补，从问题转化而来；缺了就从症状反推；成本是需求，不是结果。

恒信走完这张图：原型验功能、试点验环境适应性、投产运维才验可实现性。冰链走的是同一张图，只是把”并发、审计、合规”换成了”温差、振动、断链”。

## B 本章的能力级

**第 ② 级为主**

- **表征**——把”功能需求”“环境适应性需求”“可实现性需求”“验证载体”“承制方主导”几个词说准：三类内容各有各的代言人；验证载体是每一类需求的专属验证交付物；内容由承制方主导、成立由利益相关方确认；
- 图式——内容与载体的对应结构：需求类别决定验证条件、验证条件决定交付物形态，缺哪一块内容，交付物就降一格；以及”谁在问”这条分类判据；

- 心智模型——从症状反推缺了哪条需求（每一声”不方便”对应一条缺失的可实现性需求）；四个错位（利益相关方、时间、话语权、认知）；问题、资产、需求这条回路；成本是需求、不是结果；
- 解释框架——本章不建，由第 17、18 章承担。

> 能答全四问，才算拥有——一份需求缺的是哪一块、该拿什么去验，你现在看得出来；样机通过，也不再等于产品成功。

## C 自查 · 这些说法对吗?

1. “原理样机验证通过了，产品就成功了。”——**错**。只验证了功能需求；环境适应性需求还没验，可实现性需求压根没进需求清单。
2. “需求写全了，就是功能写全了。”——**错**。功能只是三类内容里的一块；缺了可实现性需求，交付物是环境样机，不是产品。
3. “可实现性需求是后续工艺的事，不归需求管。”——错。它是产品的固有特性，必须写进经利益相关方确认的需求；没有它，设计就没有依据和目标。
4. “成本是算出来的，设计完再核算。”——错。成本是需求分解、分配、分析的结果；没设成本相关的需求，设计就没有优化的目标。
5. “客户不关心的需求，就是不重要的需求。”——错。可实现性需求客户根本不关心，可它决定组织能不能活下去。

6. “承制方就是听客户要什么就做什么。”——错。承制方是需求开发的主导方，更专业、引导范围；但引导不等于替客户拍板，利益相关方的确认不能缺席。
7. “需求不够全，是客户没提清楚。”——错。需求全不全是承制方的责任；客户可以不知道自己要什么，承制方不能说”没提所以不全”。
8. “量产翻车是采购、制造、运维没做好。”——错。每一个”不方便”，都是一条当初没写进需求的可实现性需求。
9. “一个特性归哪一类，看特性的名字。”——错。分类的判据是”谁在问”；同一个测试性，在使用环境下是环境适应性需求，在承制方的生产交付里是可实现性需求。
10. “原理样机、环境样机、产品是需求逐级累加的三段，做到最后一段就都验过了。”——错。累加的是交付物，不是验证；可实现性需求只有真正的产品能验。
11. “八条可实现性需求配不配实现方案，是后面设计的事。”——错。过程、需求、方案三条线是锁死的；实现方案是可实现性需求的符合性解答，一条缺了，对应的方案就没有依据。
12. “我们经验很丰富，问题口口相传，这就是在自觉积累。”——错。散的、隐性的、没法复用的经验不是资产；自觉积累指的是那条制度化的回路。

13. “‘非功能需求’就是功能之外的需求，管性能、可靠性这些。”——错。功能与非功能的二分只是三类内容的子集；这个名目底下装着环境适应性需求与可实现性需求两块，它们的主人、来源、验证载体都不同。
14. “隐含需求是三类之外的又一类需求。”——错。隐含是”怎么出现”的一种形态，不是”谁在问”的一种答案；任何一块内容都可能是隐含的。

## D 练习

1. 拿冰链的八条”不方便”复盘：它们分别对应哪八条可实现性需求？如果重做需求，哪几条最该在需求阶段就写出来？
2. 找你们的一个产品，把它的功能需求、环境适应性需求、可实现性需求各列三条。哪一块最少？为什么？
3. 对你们的产品问一遍验证载体那一条判据：每一块内容，用什么交付物验证？答不上来的那一块，就是欠账。
4. 你们最近一次量产翻车或上线翻车，症状是哪一个”不方便”？反推出来缺的是哪条需求？当时的需求评审里，为什么没人提？
5. 用”测试性”与”可测试性”举例：同一个特性为什么有两个名字、两种归类？
6. 你们做需求捕获时访谈的对象都有谁？采购、制造、运维、市场的代表，有没有在需求阶段发过声？

7. 画一条你们组织的”问题、资产、需求”回路：最近一次事故，有没有变成一条可复用的需求条目？没有的话，卡在哪一步？
8. 经济可承受性这条需求，你们由谁提出、设没设指标、有没有经市场代表确认？如果没有，你们的”降本大会”是不是每隔一段时间就开一次？
9. 用恒信的镜照走一遍：你们的原型、试点、真正的产品分别对应什么？“上线成功”与”产品成功”之间差了什么？
10. 用四问法量一量”验证载体”或”承制方主导”：它是什么、不是什么、从哪来、往哪去？哪一问最薄，为什么最薄的那一问恰恰是你最容易出错的地方？

（答案要点见书末附录，与训练教材题卡同源）

## E 参考文献

**标准** - R-103 ISO 9000:2026（第 5 版）—— §3.5.1 需求（注 2：规定的要求是明示的要求）、§3.5.5 法定要求、§3.5.8 法规要求、§3.5.9 合格、§3.3.11 设计和开发、§3.11.12 验证、§3.11.14 确认（注 3：使用条件可真实可模拟） - R-102 ISO/IEC/IEEE 15288:2023 —— §6.4.2 利益相关方需要与需求定义、§6.4.3 系统需求定义（需求过程锚点，第 4 章已引）- R-10 NASA TRL（技术就绪水平）—— 技术成熟度九级刻度：原理样机演示落在中间几级、真实环境运行落在最高两级，“样机通过不等于产品就绪”的工程刻度（库内无正本，二手来源）

**经典文献** - R-65 MRL（制造就绪水平）—— 生产就绪是另一把尺子（与 TRL 对应：TRL 管技术成熟度、MRL 管制造就绪）（库内无正本）- R-66 COTS（货架商用产品）—— 组织资产复用的一条通路（§5.4.4 那句”优先选用标准外购件”；库内无正本）

> 对照阅读：本章”内容与载体”呼应第 3 章”需求、设计开发、质量”与第 4 章”需求工程”，“从症状反推”是第 1 章反事实检验在需求侧的动作，验证与确认的界分留给第 8 章。

# 第 6 章 设计开发落地：需求与方案结对

## 6.1 引言

第 5 章把一份需求立全了：功能需求、环境适应性需求、可实现性需求，各有各的代言人，各有各的专属验证载体。立全之后要回答的是下一个问题：这份需求，怎么被用来造东西。

本章看的就是这一段，设计开发。一句话说清它的样子：一份需求配一份方案，结成一对；一对之内先定规则，再走分配生成、分析迭代、分解分析三个动作，把每一条需求落到一个能负责它的节点上；上一对交出来的东西，就是下一对的输入，一对一对往下接。

### 6.1.1 误解现场

恒信集团合同审批系统项目评审会，下午两点，会议室坐满了人。

万辉在投影上打开了《合同审批系统总体设计方案》，六十页，他花两个月写成，评审会前夜还在改格式。他清了清嗓子：“这份方案覆盖了需求调研的全部三百条需求，下面我按章节过一遍——”

徐漾问了一句：“它跟需求文档是什么关系？”万辉：“设计文档嘛，就是把需求写详细一点。”

法务部的人翻开打印稿，翻了十几页，打断他：“审批链上，谁能驳回、谁不能？”

万辉翻到那一页：“在这里，第 23 页的流程图。”徐漾追问：“那这条需求，落在方案哪个节点上？”

万辉：“……图上没有它。”

财务的人翻到第 31 页：“月底对账的统计口径，合同 25 号之后签的算哪个月?”

万辉：“这个……在需求文档里写过。”

业务部的小周问：“我们上线之后，发起一份合同，先点哪里?”

万辉：“……界面上。”

运维的玉杰补了一句：“那它部署在哪、监控什么、怎么回滚?”

万辉翻了翻稿子，没翻到：“……这些后面会写。”

玉杰追问：“那我上线要部署几样、监控什么，稿子里数得出来吗?”

产品开发经理满海坐在后排，把六十页掂了掂：“两个月、六十页、三百条需求，都在了。可我们要的是一份’评审完就能开工’的方案，不是一本要通读的书。”

会议室安静下来。徐漾坐在角落，把六十页从头翻到尾，又翻回第一页，轻声说了一句：

“万辉，这不是一篇方案。这是一本小说。”

“方案不该是一篇文章。”徐漾把打印稿合上，“一篇文章是线性的，只能从头读到尾。可你的方案有五类读者——法务要看审批规则、财务要看校验和台账、业务要看怎么发起、开发要知道模块边界、运维要知道怎么部署。五个人的问题分散在六十页里，谁也找不到自己那一枝。”

万辉不服：“那这三百条需求，该往哪儿放?”

徐漾说：“写成结构。三百条需求，一条一条落到该负责它的地方去——写成一篇文章，就没有地方可落。”

万辉没接话。他忽然意识到，自己两个月里做的每一件事都对：访谈做了、需求收了、方案写了、格式改了。可这份方案没有一个位置是给”这条需求由谁负责”准备的，它只有页码。

### 6.1.2 这一章要回答什么

读完本章，你应该能回答：

1. 设计开发的对象为什么是”一对”文档，而不是一份？什么叫”由需求开发出方案、并由方案迭代需求”？
2. 记成一次 Y=F(X) 的求值，X、Y、F 各是什么？为什么说”单承接”就是函数的单值性？
3. 设计文档为什么必须能定位、能承接、能追溯？产品数据凭什么可以条目化、可以成树、可以按 N:1 分配？
4. 框架定义与详细定义各管什么？同一棵树为什么有框架、概要、定义三个形态，“最深形态”又指什么？
5. 为什么框架定义不是架构定义？
6. 每一对开工之前为什么先要定规则？三套规则各管什么，分解规则为什么必须带着消费方的语义范围来定？
7. 分配生成、分析迭代、分解分析这三个动作各在做什么，各自靠什么判据收敛？
8. 一条需求为什么只能归一个节点？违规了分几种情形、各回哪一步修？
9. 链上相邻两份之间，哪些步在承接、哪些步不做映射？链尾那一刀交给谁？

### 6.1.3 这一章的骨架

这一章要立的是”需求-方案”这一对。先把骨架立起来，后面三块，都是给它添东西。

> 设计开发的单元是一对”需求-方案”：上一份的叶节点，就是下一份的需求。一份需求配一份方案，一次 Y=F(X) 的求值（X 是需求，Y 是方案，F 是三套规则）。每份文档都是一棵树：干枝是框架定义，叶节点是详细定义；树按定义到多深，分框架、概要、定义三个形态。一对之内，先定规则，再走分配生成、分析迭代、分解分析三个动作。一条链上逐对接力，链尾把最后一刀交给构件设计方。

**一对**——设计开发的单元是一份需求文档配一份方案文档，有需求必有方案。结对是双向的：正向由需求开发出方案，反向由方案迭代需求。记成函数，就是一次 Y=F(X) 的求值（§6.2.1 展开）。

**一棵树**——每份文档都是一棵树，不是一篇文章：干枝管结构、叶节点管内容；同一棵树按定义到多深，分框架、概要、定义三个形态，最深的那个形态是这一对实际交出去的东西（§6.2.4、§6.2.5 展开）。

先定规则，再走三个动作，每一对开工先把 F 立起来：分解、构建、承接三套规则由人类主导制定。然后走三个动作：**分配生成**（把需求叶节点落到下游能负责它的节点上，长出方案骨架）、分析迭代（守住”一条需求只归一个节点”，违规就回炉重切需求）、分解分析（节点长出下层的叶节点，并检查”接住的需求一定被实现”）（§6.3 展开）。

一条链——链上相邻两份之间，上一份的叶节点就是下一份的需求；一份一份往下接，追溯边自然生成。链尾那一份只到框架，把最后一刀交给构件设计方，执行换手，概念不分（§6.4 展开）。

这一章的方法时刻，是第 1 章概念能力里的织关系：方案是结构，不是堆，把条目织成树、让每个部件各就其位彼此承接，而不是排成一列摞起来。

---

## 6.2 设计开发的对象

> 这一块把设计开发的对象看清：先立”一对”——一次 Y=F(X) 的求值；再看设计文档为什么必须能定位、承接、追溯；再推产品数据凭什么能条目化、能成树、能按 N:1 分配；然后把干枝与叶节点、树的三个形态、框架定义与架构的分界、树装不下的内容一一分开。

### 6.2.1 需求与方案结对

先回答一个问题：设计开发的对象是什么？不是一份文档，是一对——一份需求文档，配一份方案文档。

有需求必有方案，有方案必有需求：需求说”要什么”，方案说”怎么做”；上一层的需求，靠下一层的方案承接；而这份方案接住了需求，转过身又成为更下一层的需求，中间每一层，既是方案，又是需求（§6.4.1 展开）。

结对是双向的。正向，由需求开发出方案，按规则把需求一步步转换成方案；反向，由方案迭代需求，方案做下去发现需求切得不合用，就回头重切需求、调整承接（§6.3.4）。正反交替，直到这一对收敛：需求是合规的需求，方案是合规的方案。

这个”由 X 按规则得到 Y”的过程，记成一个函数最清楚：**Y=F(X)**。X 是需求，Y 是方案，F 是三套规则，需求侧的树形结构与节点规则、方案侧的树形结构与节点规则、方案节点承接需求叶节点的承接规则。F 由每一对开工时先定（§6.3.1）。

函数有一条铁律：**必须单值**——多个 X 可以映射同一个 Y，多条需求叶节点落到同一个节点上，没问题；一个 X 映射多个 Y，一条需求叶节点归两个节点，F 就不成其为函数。单承接，就是函数的单值性（§6.3.4 磨开）。

> Y=F(X)：由符合规则的 X，按函数规则生成符合规则的 Y。
> 术语注：这一记法是本书对设计开发落地方法的凝练命名，非标准条文——欢迎检验、欢迎校准。

所以这一章说”一棵树”时，先记住：树是对一份文档而言的。需求文档是一棵树，方案文档也是一棵树，一对”需求-方案”，就是两棵树。这一块先看清”每份文档都是一棵树”；下一块的三个动作，正是这两棵树之间的事。

### 6.2.2 设计文档要能定位、承接、追溯

现在回到万辉那份六十页。它为什么失败？不是他写得不好，他写得太像文章了。一篇文章有三样东西，恰好是设计文档不能缺的。

第一是不能定位。一篇文章是线性流动的，从第 2 页读到第 60 页。但”审批驳回的规则”在第几页？“对账的统计口径”在第几页？读者只能从头找起。

而方案从来不是给一个人从头读到尾的。方案是给五类人分别”查询”的：法务只找审批规则，财务只看校验与台账，开发只管模块边界。一篇文章逼着五类人都通读全文，白费功夫；更糟的是通读之后还是找不到，于是开始猜。评审会开到第四十分钟还在争”这条需求到底有没有被方案覆盖”，不是大家不认真，是文章没有给”覆盖”一个可定位的位置。

第二是不能承接。下游开发拿到六十页文章，怎么开工？他需要三件事：负责哪一块、要满足哪些需求、做到什么程度算完成。文章没有模块边界，开发只能”全文通读，然后自己揣摩”，各揣出各的边界，集成必然撞车。文章没有叶节点，需求没有地方落。

第三是不能追溯。需求是会变的（第 4 章刚讲完）。一条需求变了，影响方案的哪几页？文章时代只能重读全文、靠人肉找相关性，找不全就漏改。

三样合起来，其实是一件事：文章里没有”位置”。没有位置，读者定不了位，开发接不了手，变更找不到落点。

树的回答恰好是三件事：

| 部分 | 是什么 | 干什么 |
| --- | --- | --- |
| **根与层级** | 从根节点到枝梢节点的完整分级 | 层层细化，同层保持同一抽象级别 |
| **分类节点** | 非枝梢节点（如”审批业务域”“平台服务”） | 组织层级——像文件夹，本身不含实体内容 |
| **枝梢节点** | 框架骨架的终点 | 承接需求——需求落在这个节点上（一个节点可落多条） |

一篇文章与一棵树，差别不在长短，在有没有叶节点：

- 文章里，“审批驳回”是正文里的一个段落；树里，它是”审批流程”这个枝梢节点上挂着的一条需求。
- 文章的段落不能被”承接”，你不能说”这一段归张三做”；树的枝梢节点可以，枝梢节点是需求落脚的钩子，也是开发认领的任务单。
- 文章的段落不能被”追溯”，需求变了你不知道哪段要改；树的枝梢节点可以，沿着”需求→枝梢节点”这条边，一步命中。

所以树的关键，是枝梢节点与叶节点怎么咬合。它有两个判据，一个管”落”，一个管”搬”：

> 枝梢节点 = 需求落脚的主要落脚点（下游还能负责这条语义的节点，见 §6.3.3）——一个枝梢节点可以落多条需求（N:1）；**落点以语义定级**。叶节点（详细条目）= 能一对一搬走的原子条目——一个叶节点只能属于下一棵树的一个节点。
> 术语注：“干枝管结构、叶节点管内容”是本书（通用设计规范）的落地操作，非标准条文；**本章各术语的英译均为本书自拟，非标准术语**。后记四角色清单里，框架定义／详细定义这对文档为模块级（模块架构师／模块工程师），系统级对应物为系统架构师的架构描述。

“一对一”是这一章最要紧的一个约束，先记住它，第二块会展开：一个叶节点，只能被搬到下一棵树的唯一一个节点上。

最后是最值得问的一句：既然树这么好，为什么大家天然写文章？因为写作是线性的，一支笔顺着思路走就行；而一棵树要有层级、要

让每条需求落到枝梢节点、要让叶节点长出来，这需要方法。大多数团队没有这个方法，方案就退化成文章。翻一翻公司里任何一份”总体设计”，十有八九是按章节排列的文章，不是大家不愿意写树，是没人教过这套方法。

### 6.2.3 产品数据的条目化、树形化与分配

上一节回答”文章为什么不行”，是从读者角度看的。还有个更根本的问题：“树”为什么就行？树能成，是拍脑袋定的吗？ 不是，它底下有三条依据，每条都有出处。

依据一：产品数据可以被条目化。设计的每一步产物都要能被独立判定”是否被满足”，而通行的需求书写标准正是这样要求单条需求的，单数、可验证、无歧义（ISO/IEC/IEEE 29148 对良构需求的要求；库内无该标准正本，可核的一手位是 ISO/IEC/IEEE 15288:2023 §6.4.2.3 注 26 与 §6.4.3.3 注 15，其中”单数”一名就是”一条只讲一件事”；“唯一解释”是本书的读法）。

于是最小的判定单元是条目：一条条目能独立定义、独立验证、独立分配、独立追溯。文章里没有这个最小单元，段落不能被单独验证，也不能被单独交出去。

依据二：产品数据可以被树形化。因为分层是复杂系统的常态结构（复杂系统研究里的层级与近可分解性，见 H. A. Simon《The Sciences of the Artificial》1996 第 7 章〔库外来源·不可核〕），工程上也早有分解纪律（工程分解纪律里的”百分之百规则”，即互斥且

完备，本书转述）。但要注意：“能组树”不是自动的，同一层的分组必须同时满足三条，否则得到的不是树：

| # | 条件 | 不满足则 |
|---|---|---|
| 一 | 判据单维——同一层只用一个判据分组（如”系统职责”），不得混用两个（如职责与业务域并用） | 同层出现交叉分类，两个判据交叉切出的是一张格，不是树 |
| 二 | 条目单归属——每条只归一个父节点 | 得到的是有向无环图，不是树 |
| 三 | 分组完备互斥——子层完整覆盖父层，且互不重叠 | 出现缺口或重叠，不是合格的树 |

依据三：产品数据可以由需求设计方案，按 N:1 分配。因为设计被建模为从需求域到方案域的映射（公理化设计讲的 FR 与 DP 映射，见 N. P. Suh《Axiomatic Design》2001〔库外来源·不可核〕）。

这条映射的形状，由两端与一条约束定下来：定义域取需求侧的原子（叶节点）。粗粒度节点内部含着多条性质不同的需求，交出去后没法判定谁负责哪一部分；值域取方案侧当前能负责的实体；映射必须是函数，允许多对一（一个实体承担一族需求，这是内聚），不允许一对多（一条需求散在多个实体，那是变更传播与重复实现）。

两侧的理由还不同源：定义域取原子，是为了可判定；值域取当前实体，是因为这一刻方案侧的叶节点还没产出。

> 产品数据能条目化、能成树、能按 N:1 分配——三条依据合起来，才撑得起一套可执行的生成方法。
> 术语注：三条依据引自通用设计规范与 EOS 产品数据分解结构（见章末参考文献与库内路径），本书只做转述与举例。

三条依据还有一层用处：它们把”相邻两份产品数据之间的步”分成三类：分配步、整段搬入步、形态轴步（本章叫它**链上三类步**），是第三块要用的分界（§6.4.2）。

条目化之后，还有一件事必须回答：条目一多，怎么组织？真实系统几百上千条，平铺着三条都不成立：人脑装不下，需要”先看结构、再进细节”；一条需求变了、影响哪些条目，要聚类才能定位；要按领域把条目分给不同团队。所以条目必须被组织。

而组织成什么形态?这里藏着树最深的理由,为什么偏偏是树,不是列表、不是表格、不是网状?

> 因为树是唯一能让”导航路径”与”承接路径”合一的形态。

列表没有层次，一多就应付不了；表格能表达”谁承接谁”，但那是映射表，不是内容组织；网状能表达复杂关系，但需求必须”从根落到叶节点”，网状没有唯一根，答不了”这条需求归谁”。只有树，从根到叶节点有唯一路径：既是读者找路的导航，也是需求落地的承接。顺着树走一遍，就是一次完整的承接验证。

这个”合一”,就是第 1 章概念能力里的织关系:把散点织成结构,让导航与承接走同一条路。设计做的是把三百条需求织成一棵树，每

条需求有唯一的位置。方案是结构，不是堆：堆是线性排列，结构是有承接关系的网。

### 6.2.4 框架定义与详细定义

树组织好了，还差最后一步：细到什么程度？细到每条需求都被接住、每个叶节点都可验证，可整棵树都细到叶节点，你就看不见树了。管理与理解需要先看骨架（结构、分类、承接关系）；落实与验证需要逐个看叶节点（定义、指标、验收标准）。

两种抽象级别、两种目的、两种变更频率，于是树天然分成两大部分：**干枝**与**叶节点**——这不是谁规定的，是”既要看懂、又要做实”这两件事逼出来的。它们各有各的名字：

| 部分 | 叫什么 | 管什么 | 回答什么问题 |
| --- | --- | --- | --- |
| 干枝（结构） | 框架定义 | 组织与承接 | “结构是什么、承接了谁、往下分解出什么” |
| 叶节点（内容） | 详细定义 | 定义与验证 | “每个模块具体做什么、做到什么程度” |

框架定义就是树的干枝：从根节点一路分叉到枝梢节点，每个枝梢节点回答”结构是什么、承接了谁”，系统被组织成哪几大块，每个枝梢节点上要落哪些需求。

详细定义就是树的叶节点：长在枝梢节点上的每一条，回答”每个模块具体做什么”，一条可独立定义、可独立验证的条目，有编号、需求描述、性能指标、验收标准、映射目标。

最关键的一点：叶节点不是漂着的，它长在枝梢节点上。每个叶节点都明确属于某个枝梢节点。“顺序审批”这个叶节点长在”审批流

程”这个枝梢节点上。树正是用”叶节点属于哪根枝”这件事，把结构与内容连成一个整体。

这也回答了”为什么缺一不可”：干枝上没有叶节点是枯树，叶节点没有干枝是落叶。如果只有干枝，树就是空骨架，枝梢节点上没有内容；如果只有叶节点，叶节点就散落一地。一棵好树：干枝挂得住，叶节点长得出。

画出来，就是一棵实实在在的树，干枝部分，是框架定义：

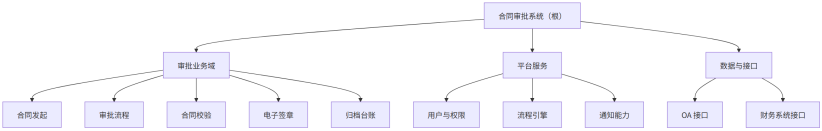



图 11　图 6-1

中间层的”审批业务域”“平台服务”是**分类节点**（干枝的分叉、负责组织的文件夹）；“合同发起”“审批流程”这些枝梢节点是框架树的终端。每个枝梢节点上长着叶节点，叶节点就是详细定义：

**「审批流程」这个枝梢节点上，长着四条叶节点：**

- **SYS-002.01　顺序审批　　支持按法务→财务→业务顺序审批，审批后 5 分钟内进入下一节点**
- **SYS-002.02　驳回回退　　支持驳回，驳回后回到发起人并可修改重提，响应 < 1s**
- **SYS-002.03　会签　　支持多方同时审批，会签发起后同步通知各方**
- **SYS-002.04　超时提醒　　审批超过 48 小时未处理自动提醒，提醒后 24 小时内升级**

注意这个例子里已经藏着三个动作的影子：“审批流程”这四条叶节点，是一把分解的刀切出来的；每条叶节点都要落到下一棵树的一个

节点上，落靠在分配生成时对账（§6.3.3）。干枝与叶节点这两个部分，正是为这些动作而生的。

> 概念卡片 · 框架定义

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 设计树的干枝：从根到枝梢节点的树形结构 + 映射表 |
| **它不是什么** | ⑥⑦ | 对立＝详细定义（叶节点）——干枝组织与承接，叶节点定义与验证，合起来才是一棵完整的树；辨析＝框架定义 ≠ 架构（见 §6.2.6） |
| **它从哪来** | ②③⑨ | 从结构骨架与内容条目分层来：设计文档先立骨架、再填条目；为拆掉”架构＝文档”的词汇病定名（见 §6.2.6）；英文 preliminary definition，preliminary ← 拉丁 praeliminaris（门槛之前） |
| **它往哪去** | ④⑤⑧ | 没有它，方案不可整体看懂，需求没有地方落；在设计树结构搭建，需求落靠定位 |
| **判据** | 落枝问 | 任取一条需求，能说出它落在哪个节点上吗？说不出的，这 |

## 概念卡片 · 详细定义

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 设计树的叶节点：长在枝梢节点下的条目列表，每条可独立定义、独立验证 |
| **它不是什么** | ⑥⑦ | 对立＝框架定义（干枝）——叶节点给干枝填内容，干枝给叶节点定位；辨析＝详细定义 ≠ 需求文档——它既写”是什么”也写”做到什么程度”（指标＋验收标准），是设计内容的最终落点 |
| **它从哪来** | ②③⑨ | 与框架定义同源孪生：把”可验证、可落实”的最小单元独立成层（见 §6.2.3 条目化）；“详细”之名与”框架”对称而成；英文 detailed definition，detail 与 talis（如此）同源，“如此这般” |
| **它到哪去** | ④⑤⑧ | 它跟着”指标”验收；条目化使得设计可被分配、可被测试、可被追踪 |

### 6.2.5 框架、概要与定义

同一棵树，可以停在不同的深度上。按定义到多深，它有三个形态：

| 形态 | 包含什么 |
| --- | --- |
| **框架**（xx 框架） | 根 → 枝梢节点；叶节点只以枝梢节点的展开项露名字 |
| **概要**（xx 概要） | 根 → 叶节点，各级节点只到说明 |
| **定义**（xx 定义） | 概要 + 各节点的详细定义 |

同一棵树上，层名分三级：**干枝**（叶以外的各级）、**枝梢节点**（这一次框架的最深一级）、**叶节点**（树的末级）。本章下文单说”节点”时，指下游能负责这条语义的那个节点（可能是干枝，也可能是枝梢，§6.3.3 展开）。

三个形态由三级设计动作产出：框架设计出框架，概要设计出概要，详细定义出定义。相邻两级可以合做，一个设计动作一次交出两级；也可以按需省略或裁剪，某份产品数据不必每一级都停。

这一对实际交出去的东西，是各自最深的那个形态。本书因此不用”框架与完整态”这种两分说法：说”完整态”会把”概要”这一档挤没，它既不等于框架，也不等于定义，而是”根到叶节点、只到说明”的独立一档。“最深形态”才是这一对两端的正式说法。

有两处细节，越往后越要紧，先在这里立住：

- 框架有阶：枝梢取哪一级，由这一次框架设计定；取得越浅，框架的阶越高。“枝梢”始终指这一次框架的最深一级，不是一个固定的树位，同一份业务流程产品数据，可以给出构件级的高阶框架（枝梢＝产品构件），也可以给出文档级的高阶框架（枝梢＝文档级端

到端任务），每一阶都是一份合法的框架。因为枝梢是承接的落点（§6.3.3），框架的阶越高，这一对承接时下游能提供的落点越浅。

- 叶节点有三种身份：在框架里，它只是枝梢节点上的一条展开项（名称级）；到概要，它才成为树节点（说明级）；到定义，它才带着详细定义。

还有一条随之而来的口径：分解源判据附着于这一次形态的最深一级。框架停在哪一阶，判据就挂哪一级；到了概要，叶节点已是树节点，判据的内容就是叶节点本身，不再另设展开清单。

最后一点，叶不只是”最深的那一级”，它有判据：叶节点必须切到可被独立解释、可独立验证、可被唯一交出去的粒度。所以”某棵树到哪一级为止”是一个判断——不是一个位置约定。

### 6.2.6 框架定义不是架构定义

细心的读者会问：在很多方法论文档里，这份树形文档常被叫作**架构定义**（architecture definition），一棵树的形状，不就是架构吗？（按 ISO/IEC/IEEE 42010:2022，这项工作的规范名字是 **§3.3 架构描述**，即”用于表达架构的工作产品”；“架构定义”是习惯叫法。）

正是这个”听着顺耳”，是个词汇病。这一节要竖起的又一个反直觉命题，就在这里：

> 框架定义 ≠ 架构定义。

三个理由，每一个都值得想清楚。

理由一：与”架构”撞车。“架构”（architecture）在全书极其重量级。第 10 章会依 ISO/IEC/IEEE 42010:2022 **§3.2** 给它一个

精确的定义（“实体在其环境中的基本概念或属性，以及为实现和演化该实体及其相关生存周期过程的管控原则”）；本书把这个定义读作系统不可还原的涵义——**关于系统根本性质的那组基本概念或属性**；**管控原则是与它并列的第二半**，不在”涵义”之内。“决定”只是认出并记下它的抓手（第 10 章 §10.2.2 术语注）。

按 ISO/IEC/IEEE 15288:2023 §3.13，设计是”系统元素及其关系的规定，其完整程度足以支撑架构的合规实现”。架构是设计的支配层，设计是让它落成的东西。框架定义是一份文档，架构是系统本身的那组**基本概念或属性与管控原则**；**“决定”是记录（ADR），不是架构本身**。

镜照一眼冰链，同一个病，换了世界。冰链把 M3 的方案结构文档直接叫”架构定义”，这名字一听就重：架构都”定义”了，还动什么？

于是人人都以为架构已定、画完即可，文档再没人维护。可那棵树天天在长：加一点需求、改一条逻辑，它就得跟着长。等第 10 章把”架构”说清楚，水忠才明白：被他们叫”架构定义”的只是一棵还没长成的树；真正的架构（**那组”去掉就不是 M3”的基本概念或属性，以及与之并列的管控原则**）一直活着，只是从没人写下来。

用已经承担重量级含义的词，去命名轻量级文档，是给自己埋词汇病的坑。

理由二：不对称。它的孪生兄弟叫”详细定义”。既然另一半叫”详细”，按对称，这一半就该叫”框架”。叫”架构定义”，反而跟这对名字自己的对称关系打架。

理由三：名实相符。它回答”结构是什么、承接了谁”，是设计的结构骨架——比架构轻：初步、框架，用于组织与承接。真正”不可还原的涵义”确实藏在树的枝条走向里，但那是从树里读出来的判断，不是树本身。第 10、11、12 章会专门去挖。

> 框架定义是目录，详细定义是正文，架构是这本书真正要表达的思想。目录不是思想，但好的目录帮你定位思想。

不过，把”不等于”说死之前，得分清两层。文档层面，框架定义不等于架构，框架定义是一份设计文档（结构骨架＋映射表），架构是系统不可还原的涵义（第 10 章），两者不在一个层面。但在”把系统结构写下来”的描述层面，框架定义可以作为架构的一种描述——**它描述的是架构”可以关乎”的结构面（42010:2022 §5.2.2），同时它本身是一份设计定义文档**（第 7 章 §7.2.4：AD 与 DD 平级）。两层不打架：不等于，是在文档与涵义之间划线；可作描述，是在描述与涵义之间搭桥。

所以这一章里，我们统一用框架定义这个名字。它比”架构定义”少一分误导，多一分老实。

万辉后来问徐漾：“为什么不叫架构定义？”徐漾反问：“你那六十页’总体设计方案’，叫不叫架构？”万辉哑然。徐漾：“所以名字上就要分开，文档是文档，架构是架构。”

### 6.2.7 树装不下的内容

本方法有一个基础假设：需求或方案类产品数据都能组织成树形结构。这个假设成立，但依据是组织性的，不是内容性的：说”组织成树”, 只保证每条内容有一个唯一的归属位置; 不保证内容之间只有父子关系。

需求与方案的语义本质是图，有四类内容，纯树放不下：

| 内容类型 | 现象 | 树形处理 |
| --- | --- | --- |
| **横切** | 一条约束影响多个分支（“可用性 ⩾ 99.9%”“所有操作需审计留痕”） | 给它独立的树位置（如非功能分支），用影响面标注勾连被影响的节点 |
| **共享** | 一个定义被多处引用（“客户”贯穿多个流程、接口契约被多个组件消费） | 归属到一处，其余引用——跨文档只记引用、不复制正文 |
| **多维** | 同一批需求可按功能／场景／性能／利益相关方多个维度切分 | 选一个稳定的主组织维度作树的轴，其余维度降为节点属性 |
| **循环／递归** | 业务流程自调用、嵌套，调用关系成环 | 树按归属组织层级，调用关系记在节点内容里（递归：每一级元素的设计开发都是一次完整的设计开发） |

由此读出本方法对”树”的读法：树是存储骨架，引用与追溯是关系载体。承接因此分两种并存，**归属式承接**（功能性分配，走单承接约

束）与**横切式承接**（非功能、约束、安全策略：留在独立分支作叶节点，用影响面标注表达对多个方案节点的承接，不走归属分配）。

假设在两种情况下不成立：一是要求树同时表达全部关系（把横切、共享也塞进分支结构），解法已给出：横切走独立分支加标注；二是选不出稳定的主组织维度，树组织变成任意的，内容一增长就会反复重构。后者的解法是：每一对开工前先定死主组织维度（对应下一块的”每一对先定规则”）。

还有一层边界要提前说清：上面讨论的是”哪些内容树装不下”，另有一层是”哪些东西根本上不了链”，**确认的判据**（预期用途、使用情境，主体是隐含需求）与方针不在链上，因为本链按派生关系生成、只承载明示的条目。这决定了这条链能支撑评审与验证，支撑不了确认（§6.4.3 收口）。

## 案例进展①｜六十页文章变成一棵树

评审会散了之后，万辉回去熬了一夜。

他把六十页文章揉碎了，提炼成一页树：合同审批系统＝三个枝（审批业务域／平台服务／数据与接口），十个枝梢节点（发起、流程、校验、签章、归档／权限、流程引擎、通知能力／OA、财务接口）。

第二天他把这一页树拿给徐漾看。徐漾点头：“好多了。现在你告诉我，法务关心的审批规则，在哪个枝上?”

万辉指向”审批流程”那个枝梢节点。

“那这个枝梢节点上，落了哪几条需求?”

万辉愣住了。树还只是一张图，枝梢节点上还没有叶节点。

“这就对了。”徐漾说，“树还只是骨架。要变成方案，得把三百条需求一条一条落到枝头上——落得准、落得全、落得能验证。”

“怎么落，先立骨架还是先切叶节点?”

“先定规则，再一个动作一个动作来。”徐漾竖起一根手指，“第一个动作是分配生成——把三百条需求落到这十个节点上。先学落，再学查，最后学长叶节点。”

---

## 6.3 设计开发的规则与动作

> 这一块先立规矩、再磨动作：每一对开工先定规则——三套规则由人类主导制定，分解规则必须带着消费方的语义范围来定。然后三个动作各靠反例定型：分配生成不是”简单分派”、分析迭代不是”返工丢人”、分解分析不是”多列几条”更不是”空话”；最后说清干枝与叶节点为什么不焊死在一起。

### 6.3.1 每一对先定规则

上一块的树，回答了”设计文档长什么样”。现在的问题更根本：树是怎么长出来的？答案是先定规则，再走下面三个动作。

三个动作不是三个先后的大阶段，而是每一对”需求-方案”都要走的功课，各有各的发生地：

| 动作 | 在哪儿做 | 做什么 | 自检 |
| --- | --- | --- | --- |
| **分配生成** | 需求叶节点 → 方案框架（两棵树之间，加树上） | 叶节点落到节点上（复用 → 改进 → 新增），新节点自下而上调树 | 三自检 + 收敛判据 |
| **分析迭代** | 两棵树之间（查），加回需求侧（迭代） | 检查分配与生成的正确性：单承接成立吗、承载量合适吗、改进波及谁——违规就迭代需求分解 | 违规两分类处理 |
| **分解分析** | 方案树上（节点→叶节点） | 节点按分解规则分解为方案叶节点，再做承接覆盖检查 | 语义四查三场景 + 覆盖公式 |

但在这三个动作之前，还有一件每对必做的事：**定规则**（分解、构建、承接三套）。规则不是从天上掉下来的通用教条，是这一对自己的本钱：没有它，三个动作没有可执行的依据。

规则由谁制定？人类主导，AI 辅助。定义各级节点的物理意义与父子关系、选定主组织维度、制定本对的具体规则，这些是判断性工作，不存在机械算法：语义归纳、边界界定的制定权在人。AI 承担三类辅助，候选生成、一致性检查、反例检验，并按规则执行设计动作

与机械核对。父节点的生成是最典型的一例：只有人类制定的语义与质量判据下的检查，没有机械的生成算法。

三套规则按动作对象分三块：

| 规则块 | 内容 |
| --- | --- |
| **需求侧规则** | 需求侧的分解规则与构建规则（首对在这一步制定；链上各对沿用上一对——本对的需求树就是上一对的方案树） |
| **方案侧规则** | 方案侧的构建规则与分解规则（每对新立） |
| **承接规则** | 需求叶节点 → 方案节点（每对新立） |

三套执行规则各有一条机械判据：

| 执行规则 | 管什么 | 机械判据／收敛判据 |
| --- | --- | --- |
| **分解规则** | 节点 → 叶节点（两处现身：本对方案侧在分解分析中产出；需求侧叶节点是上一对同规则的产出） | 单承接 + 语义原子性（防止切得过碎） |
| **构建规则** | 子节点 → 父节点（自下而上调树的依据） | 覆盖完整 + 兄弟互斥（收敛）；父语义三查：独立可定义、非兜底、最小覆盖 |
| **承接规则** | 需求叶节点 → 方案节点（分配依据） | 单承接（可 N:1、不可 1:N）+ 承载量约束（超限触发节点拆分） |

分解规则怎么定，是最要紧的一条：制定时必须把消费方的语义范围带进来。叶节点长出来是给下游用的，消费方就是承接、使用这些叶节点的一方。四条考量：

- 消费方的语义范围：把消费方承接结构的节点物理意义及其特征作为制定输入，切点落在其语义范围的边界上，叶节点天然可被单点承接。单承接由分解规则直接保证，而不是迭代试错的目标。本

对内，需求的消费方是方案框架，方案的消费方是下一对的承接结构（§6.4.1）。

- 语义原子性：叶节点保有独立语义、可独立验证。它的作用是防止切得过碎：不为迁就消费方的切法把语义整体切碎。
- 切不动不硬切：消费方语义范围切不动的语义整体（一致性约束、接口语义等关系性内容）走两个出口，节点拆分、横切式承接（§6.2.7）。
- 随消费方结构演进：分配生成中消费方的节点被新增或改进时，受影响的分解规则随影响面复查同步修订，“先定”指对当前消费方结构的首次制定，不是一劳永逸。

构建规则再补两句。父节点的语义按三条质量判据自查：**独立可定义**——父语义能独立于子节点清单表述，回答”这一组共同是什么”，不是清单的拼接；**非兜底**，“其他／杂项”式语义筐不合格，没有独立语义范围的父节点会在后续承接中变成语义黑洞；最小覆盖——选能覆盖子节点语义之并的最小类，只留生长余量、不留模糊空间。

收敛判据两条机械可查：覆盖完整（每个非叶节点语义包含其子节点语义之并）＋兄弟互斥（同层兄弟两两不重叠），两者满足即收敛。

定规则还有一层容易被忽略的分工：

| | 切法（内容规则） | 判据（形式判据） |
|---|---|---|
| 回答的问题 | 按什么维度切 | 切得好不好 |
| 典型例子 | 按利益相关方／按流程／按能力／按职责拆 | 语义四查、三场景、单承接、覆盖公式 |
| 性质 | 领域经验 | 逻辑要求 |
| 适用范围 | 结对特有（每对结对不同） | 全链通用（所有结对相同） |
| 该不该统一 | 不该——必须业务化 | 必须——不统一则对账失败 |

判据必须统一：分配要做对账，需要一套所有结对都认的账本语言，A 结对送来的叶节点，在 B 结对的下层能对上账。切法必须业务化：业

务专家直接参与分解规则制定与评审，每次退回重切的裁决都沉淀成新的切法经验，切法指南是项目自己的记忆，越用越准。

还有两个把”切”与”落”接上的装置：候选声明（每个叶节点在切出时声明一个候选节点，声明是给分配的输入，不是切法的验收）、骨架先行（对着已约好的下层节点清单切，候选声明才有节点可指）。

> 定规则 = 人类主导（AI 辅助）× 切法业务化（结对独有）× 判据统一（全链共用）× 契约共享（候选声明 + 骨架先行）。
> 术语注：这套”切法／判据／契约”三分，是本书对通用设计规范”分解分配解耦”的凝练命名，非标准条文——欢迎检验、欢迎校准。

除了解耦，还有组织级约定：术语表、命名规则、验收模板、反模式案例库。判据全链通用、切法结对独有，但都要落到组织的本地语言，统一叫法、统一格式，否则各对写出的叶节点长得不一样，全链照样对不齐。

### 6.3.2 准备工作与设计循环

规则定完，进入干活。干活分两段：正式设计之前有一段准备，然后是**正式设计的循环**。

准备这一段做三件事：定规则（每一对必做，见 §6.3.1）；把存量材料结构化成树（如果需要，入口材料已经是最深形态就直通）；处理可以直接承接的需求叶节点，准备阶段只做直接承接，不动已有的方案结构：需要改进或新增才能接住的，留给正式设计。

准备的第三件事也要过同一道关口：承接落定后，对被承接的节点执行承接覆盖检查（§6.3.6），没通过的叶节点不算直接承接，退回未处理。处理完能直接承接的，其余全部作为未处理的需求叶节点进入循环。

正式设计是一个叶节点驱动的循环，直到全部需求叶节点处理完。一轮走五步（走完一对是六步，见 §6.4.4）：

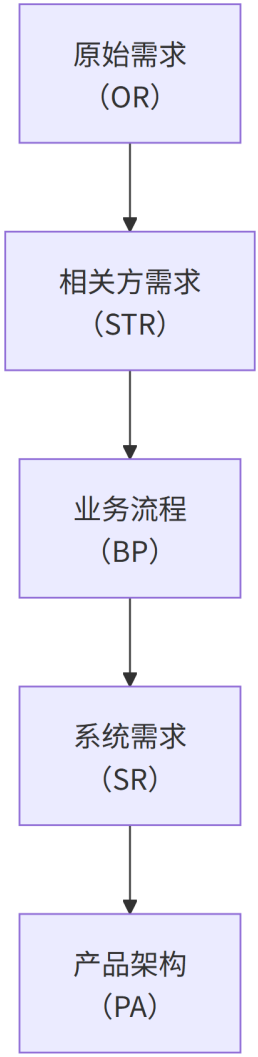



图 12　图 6-2

1. 选择待设计的一组：预计由同一个（或同一小簇）节点承载的叶节点归为一组，判据方向是落点相近，可查的信号是同一业务场景、

同一功能主题、内容互相牵连；拆开设计会导致重复承接或互相等待。

2. 分配生成（阶段一）：这一组叶节点落到方案侧能负责它的节点上，走复用 → 改进 → 新增，改进伴随影响面复查；有新增或改进就自下而上调树至收敛。
3. 分解分析（阶段二）：把本轮涉及的节点按分解规则分解，产出方案叶节点；上一轮覆盖检查要求补叶节点的节点一并处理。
4. 承接覆盖检查：对本轮涉及的节点执行检查，该节点承接的所有需求的语义，包含于其叶节点语义之并；未覆盖就回第 2 步调整承接，或回第 3 步补叶节点。
5. 更新追溯——进入下一轮；没有未处理的需求叶节点时循环终止。

细心的读者会发现一个问题：需求侧的”分解”去哪了？答案在整条链上看，本对的需求叶节点，正是上一对分解分析切出来的；本对开工时验收移交，只在分析迭代检出不合规时才重切。分解只有一条规则、两处现身（§6.3.5 收口）。

### 6.3.3 分配生成：从需求长出骨架

分配生成：把需求叶节点落到方案框架能负责它的节点上，并为新增或改进的节点自下而上调树。

它回答的问题：这条叶节点，归下一棵树的哪个节点负责？这棵树的骨架该怎么长？

先纠正一个容易想错的地方：落点不一定是枝梢节点。落点是下游能负责这条语义的那个节点，干枝或枝梢，由语义定，“上游叶节点

落到下游枝梢节点”只是”上游已到最深形态、下游只到框架”这一种完成度差下的样子。往下游若已交出概要，落点也可以落在干枝上。所以落点是一次判断：判据是语义，不是位置。

恒信走到这里，万辉手上有”系统需求”这棵树的叶节点（SYS-002.01 顺序审批、SYS-003.01 金额校验……），下一棵树是”产品构成方案框架”，前端、审批引擎、合同台账、签章服务、通知服务。

分配就是给每一条叶节点找一个归宿：

```
需求叶节点（系统需求详细条目）        →    产品构成方案节点

SYS-002.01 顺序审批                  →    审批引擎

SYS-002.04a 超时检测                 →    审批引擎

SYS-002.04b 提醒发送                 →    通知服务

SYS-003.01 金额校验                  →    审批引擎

SYS-005.02 电子签章                  →    签章服务

SYS-006.01 合同归档                  →    合同台账
```

（SYS-002.04a／b 系由 SYS-002.04”超时提醒”重切而来，故事见§6.3.4。）

分配完成不是结束，要过三关：

| 自检 | 问的问题 | 违反的后果 |
|---|---|---|
| **全覆盖** | 所有叶节点都落上了吗? | 没落的叶节点在下层方案中无对应设计，需求直接遗漏 |
| **单承接（一对一)** | 每条叶节点都只落一个节点吗? | 重复实现，浪费资源、无法验证 |
| **N:1 合理性** | 每个节点落的叶节点数合理吗? | 太少，节点太碎；太多，节点承载过宽 |

- 全覆盖最容易被”觉得没问题”带过。万辉第二次分配，数一遍叶节点总数、再数一遍落上去的，数字对不上，有一条”审批历史查询”没地方放。方案树里压根没有”检索”这个节点，补也没处补，这条叶节点还是从需求里消失了，直到上线后用户问”审批历史哪去了”才发现。
- **N:1 合理性**的经验值：一个节点落 2 到 5 条叶节点较优，超过 8 条就该考虑这个节点是否太宽、要不要触发节点拆分。但这是经验不是铁律。判断标准永远是语义相关：落在同一个节点上的叶节点，应该确实是一类事。

分配不是把叶节点贴到一张现成的树上，方案框架本身就是分配生成长出来的。三步，优先级从高到低：

| 动作 | 资产优先原则 | 说明 |
| --- | --- | --- |
| **直接承接** | Reuse（复用） | 现有节点的语义范围直接覆盖需求，落上即完成 |
| **改进后承接** | Improve（扩展／拆分／合并） | 现有节点扩展、拆分、合并后能承接，就落到拟改进节点，登记改进意图 |
| **新增承接** | Add（新增） | 基于需求语义与节点的物理意义，生成新的节点 |

优先级始终不变：能复用就不改进——改进有代价（影响面复查，§6.3.4）；但既有结构仍应先用尽，能改进就不新增，新增只留给改进也接不住的叶节点。

新增或改进之后，**自下而上调树**——归组子节点、归纳或修订各级父节点的物理意义，直至收敛：覆盖完整 + 兄弟互斥。调树由人类主导、AI 辅助；每长一个新父节点，先过三条质量判据：独立可定义、非兜底、最小覆盖。新节点归不进任何现有父节点的语义范围时，才在更高层新增父节点。

还有一层名字要分清：**分割、对齐、分配**这三个动词（ISO/IEC/IEEE 15288:2023 **§6.4.5.1 注 2** 作 partitioning／alignment／allocation——正本说的是**设计定义过程把”架构实体→系统元素”的这三件事反馈回架构定义过程，供其合并或确认**；**§5.9 讲概念与系统定义的整体关系，不含这三个动词**，原注有误），那是架构级把边界落到元素与契约上的动作（第 11 章六步的第④步”分配与接口”）；本章的”分配”是需求叶节点与方案节点之间的落靠，两级同名，说的是两件事。

概念卡片 · 分配生成

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 把需求叶节点落到方案侧能负责它的节点上（复用→改进→新增），并为新节点自下而上调树——阶段一：由需求长出方案骨架 |
| **它不是什么** | ⑥⑦ | 对立＝分析迭代（查与迭代）——分配生成负责"长"，分析迭代负责"查"，查出的违规才回炉；辨析＝分配生成 ≠ 简单分派——单承接是硬约束（一条叶节点一个归宿），落点由语义定级而非位置约定，且必须先复用再改进后新增 |
| **它从哪来** | ②③⑨ | 源自系统工程的需求分配与资产复用实践（复用优于改进，改进优于新增）；"分配生成"命名是本书对阶段 |
| **它到哪去** | ④⑤⑧ | 在两棵树之间接受检查并驱动迭代；"分析迭代"命名是本书对阶段 |

## 案例进展②｜万辉的第一次结对

万辉撸起袖子，把系统需求的叶节点往产品构成方案框架的节点上搬。

徐漾拦住他：“搬之前，叶节点验过了吗？父需求里’三年期以上要重点审’这条，你的叶节点齐吗？”

万辉一愣，回去一查：“合同校验”节点的叶节点只切了金额校验、条款校验、资格校验三个，期限校验漏了。“合同期 ≥ 3 年进入重点校验”这条要求，叶节点里没有对应。

“这批叶节点是上一对切出来交到你手上的，开工前就要验收。”徐漾说，“漏一条叶节点，下层方案就不会做期限校验。将来五年期合同不走重点审，出了事追溯回来，就是这个漏洞。判据在 §6.3.5 的语义四查里。”

万辉补切了期限校验，又看了一遍，越看越不对劲：“我好像还多切了一个’合同打印’……这不是校验该干的。”

徐漾：“那叫影子需求，切叶节点是精确切分，不是顺手多切。删掉。”

万辉删掉，又复查约束继承，把”数据留档十年”这条系统约束，落到归档那个节点上，确保有叶节点显式承接了它。

小白凑过来看了一遍，她是开发，问：“这批叶节点齐了，我是不是就能照着开工了？”

徐漾：“还早。叶节点齐了、落上了，才算走完分配生成；接着要分解分析，让节点长出自己的叶节点，再查一遍’接住的需求一定被实现’。这两步没过，开工就是白干（§6.3.3、§6.3.5）。”

“好，我先把这批叶节点过一遍。”万辉说。

徐漾：“现在，先把落上去的每一条都查一遍——下一个动作，分析迭代。”

### 6.3.4 分析迭代：一条需求一个归宿

分析迭代：检查分配与生成的正确性，并根据检查结果迭代，迭代的主要对象，是需求分解。

它回答的问题：叶节点落对了吗？落得下吗？错了，回哪一步修？

很多团队怕这两个字，“迭代”听着像返工、像前面的活白干了。恰恰相反：分析迭代是有方向的修正，不是推倒重来。方向规定只有一条：

> 节点是基准，需求是适配方。违规时重切需求，而不是迁就需求改框架。

为什么？基准不只被动接收分配。分解规则预先定好了需求侧怎么切（切点带进消费方的语义范围，§6.3.1）；叶节点天然可单点落靠，迭代试错不是常态机制，而是兜底。改进与新增属正向承接设计，框架为承接切好的叶节点而生长，不是修正；修正只在检查出违规时发生，方向只指向需求侧。

**单承接：函数的单值性**

分析迭代守的核心约束只有一条，也是这一章最重要的约束：

> 一条需求叶节点，只能落到下一棵树的唯一一个节点上（单承接：可 N:1、不可 1:N）。

这就是 Y=F(X) 的单值性：多条叶节点落同一个节点（N:1）是聚合，一条叶节点落两个节点（1:N）让函数不成其为函数。为什么这么严？三个理由：

1. 避免重复实现。一条叶节点（比如”金额校验”）同时落到”审批引擎”和”前端”，两个模块各实现一遍，同一逻辑两处实现，改一处漏一处，这是重复代码的根源。
2. 保证验证有唯一对象。“金额校验”对不对，该测哪个模块？单承接让验证有唯一目标；一对多让验证不知道该查谁。
3. 让变更可定位。需求改了金额校验，只改一个模块；分到两处就要找齐两处，漏一处就是缺陷。

注意一个对称：节点承接叶节点是 N:1（一个节点落多条叶节点），叶节点归宿节点是单承接（一条叶节点只有一个归宿）。前者是聚合，后者是定位。很多团队把方向搞反，让叶节点落多个节点（违反单承接），却让一个节点只落一条叶节点（浪费聚合能力）。

还有一条随之而来的口径：承接落定的标志是一次明示化。被承接的需求叶节点以**规定需求**的形态落到目标节点上：有编号、有对象与适用范围、有断言、有判定方法。停留在”大致知道归它管”，不算落定。

**违规两分类：回炉，还是走出口**

仍出现一对多时，先判类型，别急着返工：

- **程序违规**——分解规则没按四条考量制定（切点没带进消费方的语义范围）。处理：回炉重定分解规则，再重切需求。把消费方节点的物理意义带进来重新定切点。
- **切不动的语义整体**：关系性内容怎么切都要跨节点（判据见§6.3.1 第三条考量）。走两个出口：承载量超限触发节点拆分（比

如”超时提醒”拆成超时检测与提醒发送）；约束与策略类内容走横切式承接（留在独立分支作叶节点，用影响面标注表达，§6.2.7）。顺带把”防切过碎”说全：重切不是越切越细。语义原子性判据守着下界：叶节点保有独立语义、可独立验证，防止为迁就节点的切法把语义整体切碎。

**改进有涟漪：影响面复查**

“改进后承接”不是局部操作。对节点的扩展、拆分、合并会影响已承接的其他需求：拆分后，原已承接的需求要重新分配到子节点；合并后，节点的语义范围变窄，某些既有需求可能失配。

任何改进操作后必须执行”影响面复查”：重新核对既有承接，失配则批量重分配或回退改进。否则会在循环内留下隐性失配，到符合性审查才暴露。改进同时改变分解规则的制定输入。新增或改动的节点物理意义即时更新消费方的语义范围，受影响的分解规则须随影响面复查同步修订。

> 概念卡片 · 分析迭代

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 检查分配与生成的正确性（单承接、承载量、影响面），并根据检查结果迭代——迭代的主要对象是需求分解 |
| **它不是什么** | ⑥⑦ | 对立＝分配生成（长骨架）——迭代不是否定生成，是修正输入（重切叶节点）或补全结构（节点拆分、新增）；辨析＝分析迭代 ≠ 需求分析（第 4 章·加工）——需求分析往前看（需求清不清楚），分析迭代往回修（叶节点合不合用） |
| **它从哪来** | ②③⑨ | 源自单承接约束（Y=F(X) 的单值性）与资产变更管理实践；“分析迭代”是本 |
| **它到哪去** | 两问⑤⑧查 | 每轮 |

## 案例进展③｜一个”悬空”的叶节点

万辉把系统需求的叶节点往产品构成方案框架上搬，搬到最后，剩下一条没人要：

SYS-004.01 合同全文检索， 支持对已归档合同按全文检索，响应 < 3s。

“全文检索……”万辉先看系统需求树：三个干枝——审批业务域、平台服务、数据与接口；再看产品构成方案框架的节点——哪个都不像。硬塞给”合同台账”？台账只管存储，不做检索。（SYS 编号按需求流水编排、不对应节点序——004 是”数据与接口”域下的需求，不表示缺了第 4 个节点。）

顺带分清两层名字：系统需求树里叫”流程引擎”“通知能力”（能力），产品构成方案框架里叫”审批引擎”“通知服务”（构件），名字别跨层混用。

分析迭代检出了这条悬空叶节点。徐漾让他先别硬塞：“一条叶节点没有归宿，只有两种可能：要么**叶节点切粗了**（欠切分，该重切成’建索引’和’查索引’两条），要么**方案树没长够**（缺一个’检索服务’的节点，生成新节点）。你先判断是哪种，两条路的判据在 §6.3.4。”

万辉想了半天：“这条叶节点拆不开，‘对已归档合同全文检索’就是一件事，再拆就把’建索引’和’查索引’的接口写进去了，那是设计不是需求。所以是方案树缺节点。”

于是分配生成走了一步新增：产品构成方案框架新增一个节点”检索服务”，叶节点有了归宿。至此产品构成方案框架六个节点齐了。

玉杰一直在旁边听着，他是运维，接过话茬：“这根’检索服务’立住了，我运维就有得部署、有得监控了，一个节点，就是一样要上线

伺候的东西？可反过来，树没长全，我连该部署几样都数不清，少一个节点就少一样要伺候的东西，到上线那天才露馅。”

徐漾点头：“对。方案里的节点，就是你运维要伺候的对象。你的部署清单，就是这棵树的全覆盖检查，你数得清几样，树就长了几个节点。”

他顿了顿，补了一句：“分析迭代检出来的问题，一半是分配的错，缺枝要生成新节点；一半是上游的错，叶节点切粗了要重切。三个动作是互相把门的。”

### 6.3.5 分解分析：切到可独立验证

分解分析：方案节点按分解规则分解为方案叶节点（阶段二），并做承接覆盖检查，接住的需求一定被实现。

它回答两个问题：这个节点要长出哪些叶节点，才能把它承接的叶节点全部实现？长出来的叶节点，真的把承接的需求都实现了吗？

先回头看一眼叶节点的来历。需求叶节点是上一对用同一条分解规则切出来的（首对由链首从存量材料结构化产出）；本对的方案叶节点，由本对在分解分析中产出，同一条分解规则，全链两处现身。链尾对例外：节点的分解移交构件设计方执行（§6.4.3）。

**切到哪为止**

万辉面对的”合同校验”这个节点（上一对的产物），落着几条需求。它被切成了四条叶节点。“拆成几条”谁都会，难的是拆得对。分解规则的执行，先记住切法：

> 切点沿消费方节点的语义范围边界切分——本对方案叶节点的消费方是下一对的承接结构，链尾对的消费方是构件设计方。叶节点天然可被消费方单点承接（单承接由分解规则直接保证，§6.3.1）；“再分就失去独立语义”只是下界——防止切得过碎；切不动的语义整体不硬切（走节点拆分或横切式承接两个出口）。

切出来对不对，过语义四查：

| 检查 | 问的问题 | 违反的样子 |
| --- | --- | --- |
| **覆盖完整性** | 父节点里的每条要求，叶节点都有对应吗? | 漏——父里有的，叶节点缺了 |
| **不超出性** | 叶节点引入父范围外的新内容了吗? | 超出——凭空冒出来的要求 |
| **逻辑一致性** | 叶节点之间互相矛盾吗? | 冲突——两条叶节点打架 |
| **约束继承** | 父的非功能约束，至少一条叶节点承接了吗? | 丢失——约束在分解中消失 |

- 覆盖完整性：“合同校验”要审”金额、期限、条款”三件事，只长出”金额校验”“条款校验”，期限校验就漏了，这是最常见的失败（案例进展②刚抓过一次），漏一条，下层就不知道要支持它。
- 不超出性：如果顺手长出第五条叶节点”合同打印”，这是”校验”节点上不该有的，分解不是多列几条，是精确切分。没经上游确认，就是范围蔓延（也叫影子需求）。

- 逻辑一致性：“金额 ⩾ 500 万进入重点校验”和”所有合同一视同仁”不能同时存在，两条叶节点打架，下层方案无所适从。
- 约束继承：“合同数据留档十年”是系统级约束，分解时必须至少一条叶节点显式承接（比如归档的叶节点写”数据保存 10 年”）。

约束在分解中丢了，下层根本不知道，直到审计才发现没留档。四条检查里，“覆盖完整”与”不超出”是姊妹对：一条防漏，一条防多。漏了，需求在方案里消失；多了，长出没被授权的需求，方向相反，都让树偏离它该承接的需求。

**指标怎么跟着分**

分解还有第二个工作面：性能指标怎么分。父需求带着一个指标（比如”整笔审批在 48 小时内完成”），分解后指标怎么落到各条叶节点上？三种情况，按顺序判断。

情形一：可拆分，子指标之间有函数关系。响应时间、可靠度、功耗这类指标，子项可按运算关系拆。恒信的例子：“整笔审批 48 小时完成”拆成”发起 ＋ 各级审批 ＋ 签章 ＋ 归档”各段加起来不超过 48 小时（串联可加），每个子项分到预算，总和不超过父值。

镜照一眼冰链，这是最漂亮的一类：M3 温控箱的”全程可靠度 ⩾ 0.99”，拆成硬件可靠度 × 固件可靠度 × 云端可靠度 ⩾ 0.99（**串联乘积**——任何一个环节断链，整条链就断）。于是硬件分到 0.995、固件 0.995、云端 0.999，三者乘积约 0.989，不够 0.99。

数字一旦落纸，立刻发现预算不够，就得回炉，这正是”可拆分”的价值：把账算清楚，不靠感觉。

情形二：下放，所有子需求各自独立满足同一指标。工作温度范围这类指标没法拆分：M3 的”工作温度 -20℃~60℃“（设备工作环境温度；箱内冷链货温是另一档，见第 3 章），是指每个部件，传感器、电池、压缩机、主控板，都必须在 -20℃~60℃ 正常工作，不是把温度区间分给不同部件。所以所有叶节点**同值继承**——恒信的例子：”操作日志不可篡改”是系统级约束，全部下放。

情形三：暂不分解，系统级耦合，需求阶段算不清。M3 的”整机噪声 ⩽ 45dB”：噪声是”声源—传播路径—接收者”的系统级问题，需求分解阶段还没有结构数据，没法预先分给某个部件。这时候**不做假分解**——在分解记录里标注”向下一层传递”，指标不消失，留到方案设计阶段再算。

最怕的是第三种情况当成第二种，硬塞给一条叶节点，结果那条叶节点怎么都满足不了。

> 指标分解三情形：能拆就拆（按函数关系）、拆不动就下放（同值继承）、算不清就传递（标注”向下一层传递”）。最忌讳的是假装能拆。

**切法上的七种错**

分解的错误不止”漏”和”多”，一共七种，值得一次记全：

| 类 | 反模式 | 表现 | 后果 |
| --- | --- | --- | --- |
| 语义 | 过分解 | 把实现步骤当功能切（“先加热→再保温→最后冷却”） | 需求里混入实现细节，压缩下层设计空间 |
| 语义 | 欠分解 | 一条需求含多个独立功能（“上料、加工、检测、分拣一起做”） | 无法分配——一条要多个节点承接 |
| 语义 | 影子需求 | 凭空长出父需求没有的内容 | 范围蔓延，未经上游确认 |
| 语义 | 约束丢失 | 父的非功能约束在叶节点中消失 | 下层不知有约束，交付后才发现不满足 |
| 指标 | 指标错配 | 子指标口径变了（父”响应 < 2s（95 分位）“→子”< 2s（平均）“） | 名义继承、实质偏移，验证时对不上 |
| 指标 | 指标落空 | 父有量化指标，子只保留功能描述（“加热响应 < 5s”→“支持加热”） | 指标约束彻底丢失 |
| 指标 | 指标超预算 | 子指标之和超过父指标（成本 5000 → 2000＋ | 预算被突破，整体无法收敛达标 |

过分解最值得多说一句：它把”怎么做”混进”做什么”。第 3 章讲过需求答”要什么”、设计开发答”怎么做”，分解是精确切分，只切”要什么”，不写”怎么做”。“支持温度控制”是需求，“先加热再保温”是设计，后者一旦写进需求，下游就失去选”怎么实现”的自由（万一有更好的方案呢），这是需求被过度约束最常见的形式。

> 概念卡片 · 分解分析

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 节点按分解规则分解为方案叶节点（阶段二），并做承接覆盖检查——接住的需求一定被实现 |
| **它不是什么** | ⑥⑦ | 对立＝分配生成（树间落靠）——分解分析在树内（节点→叶节点）；辨析＝分解分析 ≠ 多列几条——切点由消费方语义范围定，语义四查＋指标三情形防切过碎、七种反模式防切错 |
| **它从哪来** | ②③⑨ | 源自工程分解实践：把整体拆回组成部分，让复杂设计可逐个验证、逐个分配；同一条分解规则全链两处现身；“分解分析”是本书凝练命名，英文decomposition & analysis，do（反） |
| **它到哪去** | 四查⑤公式 | 切点带消费配落定范围；唯一义四查与指标三情形防过 |

### 6.3.6 接住的需求一定被实现

叶节点切完、落好，是不是就结束了？没有。分配只保证”每条叶节点都有归宿”，不保证”每个节点都接住了叶节点”。这是分解分析的第二半，分析：

每个节点分解完成后，检查：

> 该节点承接的所有需求的语义，包含于其叶节点语义之并（满足度）

这是需求符合性审查第三层（满足度）的设计期预演——未覆盖就回分配生成调整承接，或回分解分析补叶节点。这防止”接住了但没实现”的空洞，这道关口，分配生成里落了承接的每个节点都要过。

覆盖检查是两阶段之间的关口：分配生成决定”谁承接谁”，分解分析决定”承接的有没有被实现”，两者在这一对会合。

覆盖检查落成文档动作，就是符合性分析：检查每个方案节点，是否充分满足了它落着的全部需求，分语义、指标两个维度。

先清一个词。“分析”这个词，到本章已经一分为三：第 4 章的”需求分析”是**加工**（往前看，需求清不清楚）；§6.3.4 的”分析迭代”是**查分配与生成**（往回修，叶节点合不合用）；这里的”符合性分析”是**查承接的实现**（往后看，方案接没接住）。同词三义，各指各的，评审会上听到”分析”，先问一句查的是什么。

语义符合性：方案的定义覆盖了需求吗？ 三问：

| 问 | 充分的样子 | 不充分的样子 |
| --- | --- | --- |
| **覆盖度** | 方案节点的定义里，提到了每条承接需求的对应处理 | 定义只覆盖了部分需求 |
| **映射正确性** | 需求的术语和方案的术语能对应 | 需求是”安全审计”，方案是”操作日志”——范围不同 |
| **分析具体性** | “通过实现 X 机制满足 Y 需求，具体体现在 Z” | “本节点满足上述需求” |

“本节点满足上述需求”是最典型的空话——它什么都没说。合格的符合性分析长这样：

> 本节点（审批引擎）通过以下方式满足所承接的需求：- 针对 SYS-002.01（顺序审批）：实现状态机驱动的审批流，支持法务→财务→业务顺序流转，覆盖了”按序审批”的全部语义；- 针对 SYS-003.01（金额校验）：实现金额规则引擎，校验金额 ≥ 500 万进入重点审，响应 < 1s。

每一条叶节点，都有一个”怎么满足它”的具体回答。写不出来的，就是没接住。

指标符合性：方案指标不低于需求指标吗？语义接住了，指标也要接住：

- 需求”审批响应 < 2s”，方案节点”审批引擎响应 1.8s” → 满足；
- 方案节点”审批引擎响应 2.5s” → 降级（必须说明理由、经评审确认，不能默默降）；
- 方案节点根本没提响应时间 → 丢失（指标在方案里消失了）。

和指标分解一样，比较前要先确认运算关系：需求 2s 是整体（可拆分），方案要算的是组合值；需求是下放（同值继承），方案直接比。指标分析最强的形式是把账算出来："本节点各环节 0.6＋0.7＋0.5＝1.8s < 2s，结论：满足。"

> 符合性分析的基本句式："针对需求 A，本节点通过 X 机制满足 Y，指标 Z 满足／降级／丢失。" 写不出这句的，等于没分析。

时机：每一层分配完立刻做，不是末次全量验证。符合性分析最容易犯的错是"留到最后做"，等全部设计完成才做一次总检查，这是倒置的：分析越晚，缺陷离源头越远，返工越贵。正确做法是每一对"需求-方案"分配完成，立刻对每个节点做分析（本章把一对"需求-方案"走完的功课叫结对六步，它是其中第⑥步"签定稿"内的检查关，见§6.4.4）。逐层清账，而不是算总账。

这呼应着第 8 章要讲的评审、验证与确认，那时我们会看到，验证把"符合性分析"纳入了更完整的框架：设计期这一遍是预演——链尾构件规格评审上是正式审查（§6.4.3）。这里先记住：分析是每一层都要做的动作，不是最后的仪式。

> 概念卡片 · 符合性分析

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 检查承接的实现：每个方案节点是否充分满足所承接的全部需求（语义＋指标）——承接覆盖检查的落纸形态 |
| **它不是什么** | ⑥⑦ | 对立＝需求分析（第 4 章·加工）与分析迭代（查分配与生成）——符合性分析查的是”承接的实现”；辨析＝符合性分析 ≠ 评审——它是设计动作（每层分配后立刻做），评审是关口（第 8 章）；分析是评审的输入 |
| **它从哪来** | ②③⑨ | 源自验证实践：检查方案是否满足所承接的需求，从”评审验证确认”家族分出，成为每层分配后立即执行的动作；英文：compliance analysis / conformance analysis |
| **它到哪去** | ⑤⑧ | 每层分配完成后立刻执行；为评审（第 8 章）提供”满足性”证据；为迭代（第 9 章）提供”缺口”输入 |

## 案例进展④｜覆盖检查抓住重要合同

恒信走到这一步，徐漾组织了一次覆盖检查（符合性分析），对产品构成方案框架的每个节点，逐条核对”承接了的需求，它的定义接得住吗”，再报”满足／降级／丢失”。

报到”审批引擎”这个节点，万辉卡住了。节点上落着顺序审批、金额校验那几条。

但第 4 章徐漾好不容易澄清的那条”重要合同要审批”。触发条件”金额 ≥ 500 万或三年期以上”，在审批引擎的定义里，没有对应的处理。

“那条需求哪去了?”万辉翻了半天：“我把它归到’合同校验’去了，校验过了就触发重点审……”

徐漾摇头：“那你看看’审批引擎’的承接判据，有没有写’承接重点审触发’?”没有。校验触发只是触发，“进入重点审批流程”这个动作本身，没有节点认领。覆盖检查失败：需求写着”重要合同要审批”，方案里”审批”这一环没人认领。

这不是小事。没有这个承接，重点合同会和其他合同一样走普通流程，三年期合同漏掉重点审，是合规事故。

黄程坐直了，他是业务方：“‘金额 ≥ 500 万或三年期以上要重点审’，这是我们业务方定的红线。方案里没接住，业务上就是漏审。”

万辉在”审批引擎”的承接判据里补上：承接”重要合同触发重点审流程”，实现”校验结果 → 重点审路由”。

徐漾合上本子：“这就是为什么覆盖检查每一层都要做。分配让每条叶节点有归宿，但归宿了不等于接住了，树是不是真的长起来，要靠分解分析来验。”

### 6.3.7 干枝与叶节点分开维护

动作的边磨完了，还有一个容易焊死的地方，干枝与叶节点。

这是设计文档里最容易被低估的一件事。干枝和叶节点必须分开维护、各自演进，因为它们的变更频率和变更原因完全不同：

- **修剪枝条**（调整分类层级，比如”归档台账”从”审批业务域”挪到”数据与接口”）→ **只动框架定义**——叶节点一条都不用动。
- **长新叶节点**（新增需求，落在已有节点下）→ 动的是内容（详细定义加叶节点、框架定义映射表补一行承接），干枝结构不用动。
- 只有当节点本身变了（新增或删除节点）时，干枝和叶节点才同时动。

第 4 章刚讲过需求变更控制。这里你看到了”结构”与”内容”分离带来的变更弹性：干枝管结构，结构不轻易变；叶节点管内容，内容随时长。把结构和内容焊死在同一份文档里，每次内容变更都要拖着结构一起动，变更成本成倍上升。这呼应着第 13 章的配置管理：干枝和叶节点是两个配置项，分别纳入基线。

---

## 6.4 一条链上的逐对接力

> 这一块把一对放大成一条链：链上相邻两份是什么关系、哪些步在承接哪些步不做映射、链尾那一刀怎么交出去、一对之内要走完哪些功课、设计开发怎么验收。

### 6.4.1 上一份的叶节点，下一份的需求

前面两块在讲”一对需求-方案之间”怎么干活：叶节点落到节点上、骨架长出来（分配生成），违规就迭代需求（分析迭代），节点长出自己的叶节点、检查承接的实现（分解分析）。但真实设计不止一对”需求-方案”。

从一个真实的链条看起，恒信的项目从头到尾是这样：

- 老总一句话”合同审批要数字化”，**原始需求**——还没被组织；
- 徐漾访谈法务、财务、业务，把”数字化”展开成各方需要的集合，**利益相关方需求**；
- 运行概念写清”一份合同怎么被三方审批”，**业务流程**；
- 业务流程翻译成系统必须提供的能力，系统需求（上一块的 SYS 叶节点）；
- 系统需求变成可交给构件设计方的构件骨架，产品构成方案框架。

从”一句话”到”可设计的构件”，一份一份往下接，每相邻两份之间都是一个”需求-方案”结对。而链首那一句还没被组织的话，要先进一段入口加工：把外部材料文本化、切分、澄清，加工成第一份需求产品数据。入口加工出来的，才是链上的第一份；“原始需求”本身不是链上的一份产品数据。

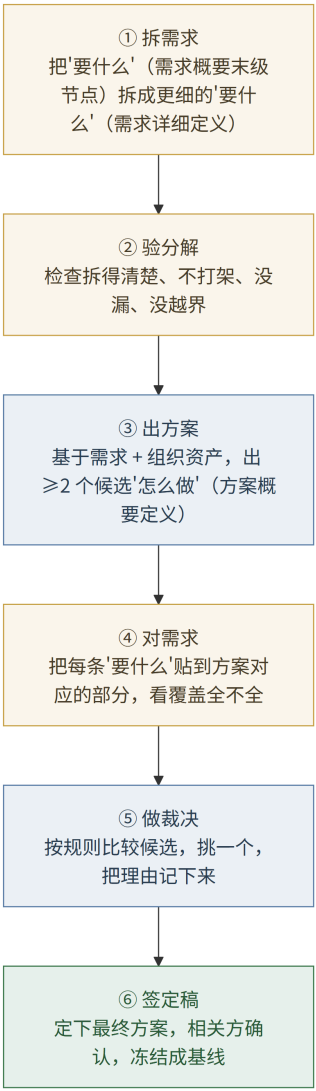

① 拆需求
把'要什么'（需求概要末级节点）拆成更细的'要什么'（需求详细定义）
② 验分解
检查拆得清楚、不打架、没漏、没越界
③ 出方案
基于需求＋组织资产，出≥2 个候选'怎么做'（方案概要定义）
④ 对需求
把每条'要什么'贴到方案对应的部分，看覆盖全不全
⑤ 做裁决
按规则比较候选，挑一个，把理由记下来
⑥ 签定稿
定下最终方案，相关方确认，冻结成基线

图 13　图 6-3

关键在中间层。“利益相关方需求”“系统需求”这两层，同时扮演两个角色：向上，它是方案；向下，它是需求。这就是”需求与方案相对”，需求不是文档的固有标签，而是相对相邻文档扮演的角色：

| 层 | 对上层 | 对下层 |
| --- | --- | --- |
| 原始需求（存量材料） | — | 入口加工的输入，不是链上产品数据 |
| 利益相关方需求 | 方案（承接原始需求） | 需求（驱动业务流程） |
| 业务流程 | 方案（承接利益相关方需求） | 需求（驱动系统需求） |
| 系统需求 | 方案（承接业务流程） | 需求（驱动产品构成方案框架） |
| 产品构成方案框架 | 方案（承接系统需求） | —（链尾：节点＝构件设计方可消费的单元，§6.4.3） |

这个相对关系不是纯概念。它是三个动作能接力工作的前提：上一对的分解分析刚切完叶节点，下一对的分配生成紧接着就从这批叶节点开工。利益相关方需求刚当完”方案”，转过身就当”需求”。

这也是为什么树结构在每一层都长得一样：树不挑角色，需求是树，方案也是树。

每一对”需求-方案”的正式名字叫文档结对：上层叶节点（详细定义条目）是需求方，下层节点（方案侧能负责这条语义的那个节点）是方案方。有需求必有方案，有方案必有需求，这是结对的第一条律。

每一对都遵循同样的两阶段工作法：

1. 骨架阶段：先约定下层方案的框架树，长哪几根枝、每个节点叫什么，并建立初步映射；

2. 填充阶段：上下层并行填充，上层完善叶节点（详细定义），下层完善节点（节点定义、承接、分析）。

为什么骨架先行？回到案例进展③那条悬空叶节点。它如果放在骨架阶段，第一天就会暴露”方案树缺一个检索节点”，因为骨架阶段就要约定”方案有哪些节点、哪些叶节点落哪些枝”。万辉先长完叶节点再统一搬，悬空的叶节点到最后才浮出。先约骨架、再填内容，缺枝问题最早暴露，而不是最晚。

### 6.4.2 链上只有分配步在做承接

上一节说”上一份的叶节点就是下一份的需求”。这句话很顺手，但它不是对链上每一步都成立。把相邻两份之间的步分一分，得到的就是**链上三类步**，边界也就清楚了——这也是 §6.2.3 三条依据的第三层用处。

| 步型 | 判据 | 本链实例 | 做与不做 |
| --- | --- | --- | --- |
| **分配步** | 两侧的叶节点不是同一批，须按映射定落点 | 业务流程定义 → 系统需求框架；系统需求概要 → 产品构成方案框架 | 做承接：Y=F(X) 在这里求值；单承接与承接覆盖检查在这里适用 |
| **整段搬入步** | 两侧的叶节点就是同一批，叶节点由上游直接给出 | 利益相关方需求 → 业务流程概要；业务流程概要的两个视角之间 | 不做映射：整段搬入；这一步的活是长干枝（把上游节点的属性长成下游的各级节点），并逐级判定其落位 |
| **形态轴步** | 同一份产品数据在两个形态之间 | 业务流程概要 → 业务流程定义；系统需求框架 → 系统需求概要 | 不是承接：形态轴上的深浅，不换产品数据、不换叶节点 |

本方法只在分配步上做 Y=F(X) 的求值。这条区分，是与”逐级分解”式说法最要紧的界线：如果把三类步一律当成承接，就会得出”上游第 N 级对应下游第 N−1 级”这类数量关系，而落点由语义定，不由位置约定（§6.3.3）。

顺带说清一个词的两种用法：承接有时指承接规则（定义域只取上游叶节点、基数可 N:1 不可 1:N），有时指承接记录（“某份上游文档由哪份下游文档收下”的逐对对应）。后者是一份备档，不含映射语义，不受本条规则约束。本章其余各处，一律用前一个意思。

“上一份的叶节点就是下一份的需求”因此要加一句限定语：它在分配步上是机制，在另两类步上只是形态与视角的搬入。写进”下一份的需求”，不等于写进”下一次承接”。

### 6.4.3 链尾交给构件设计方

先把一句常被说错的话立正。理论上，设计的终点是生产的起点，设计画到”可以照着造”，就该交给生产了。但它不能说成”设计的终点是开发的起点”：设计与开发是一个概念（第 3 章），没有”设计完了才开始开发”这回事。

真正发生的，是设计层级之间的换手。本章这条链，走的是系统设计，系统设计的终点，是构件设计的起点。

整条链走到最后一份，产品构成方案框架，它只到框架这个形态落纸。它的节点（审批引擎、前端、合同台账这些构件）就是构件设计方可直接消费的最小单元。系统设计的终点，是一份带分解源判据的节点清单。

每个节点都带着两样东西：**承接判据**（它承接了哪些需求）和**分解源判据**（它该展开成哪类规格）。

分解源判据就是移交出去的规则接口，构件设计方接手后的第一件功课，就是照着它把构件展开成规格，这仍然是一次设计开发（第 3 章的递归：每一级都是一次完整的设计开发），只是执行者从系统设计换成了构件设计。规格落定，照着规格把构件造出来，生产的起点，就在这条链的终点上。

所以链尾的换手，不是概念的断裂，是执行的接力：整条链每一对都是一次完整的设计开发；链尾这一次设计开发（构件 → 规格），由

构件设计方执行。因为最后一刀的切法由消费方（构件设计方的实现结构）锚定，自然由构件设计方来切（§6.3.1 分解规则的制定输入）。设计与开发是一个概念，贯穿全链；换的只是谁来执行这一级。

覆盖也在这一步核验：§6.3.6 检查的”接住的需求一定被实现”，在链尾不再是设计期预演，而是构件规格评审中的正式审查。追溯链相应停在”需求 → 方案框架 → 节点”。

还有一层边界要说清：这条链按派生关系生成，只承载明示的条目。所以”链路自洽”只保证验证这一侧。

确认的判据是”**针对**特定预期用途或应用的需求”（ISO 9000:2026 §3.11.14），主体常常是隐含需求；方针是承诺加框架的关系，与派生关系不同构。确认的判据与方针都不在这条链上，它们要由需求基线之上另一个与派生链不同构的参照层承载。这也解释了 §6.2.7 留下的那句话：树装不下的内容可以另开分支，链装不下的东西只能另设一层。

### 6.4.4 一对从接需求走到冻结基线

两阶段说的是时间怎么组织；真正把一对”需求-方案”从头走完，要走六步（本章叫它**结对六步**）：**接 · 验 · 想 · 对 · 裁 · 签**

| # | 步骤 | 白话 | 概念归属 | 三个动作的展开 |
|---|---|---|---|---|
| ① | 接需求 | 需求就位：链首把存量材料结构化成需求树；链上各对验收上一对移交的最深形态——不新拆 | 需求就位（做） | 上一对分解分析的产出，本对的输入 |
| ② | 验就位 | 检查移交来的叶节点能不能用：可转化（完整、无歧义、不冲突、隐含已识别）、可分配（原子性、全覆盖、单承 | 需求就位的检查关 | 验 · 做 |
| ⑤ | 做需求 | 接、候选 | 345（先造树） | 分解 · 做 · 封存 |

六步不是六个并列的动作，而是三个动作各自的展开，每件都缺一样它特定的东西，于是各长成一对：

- 需求就位长成接与验（①②）：叶节点是上一对的产出，接手必须验收（可转化、可分配），不合格退回上一对或走分析迭代重切；
- 分配生成长成想、对与裁（③④⑤）：下一棵树不会自己出现，分配前必须先想出骨架，骨架不是第四样，而是分配的目标树：没有目标树，分配无处可落；候选多个，必须先裁一个才算落定；
- 分解分析长成签（⑥）：骨架落定后切叶节点、查覆盖、确认冻结，阶段二的活全在⑥里收口。

> 六步 = 三个动作 + 各自缺的那件东西（验 · 想 · 裁）。

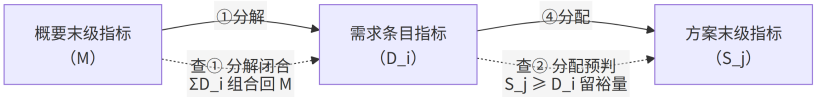



图 14　图 6-4

第②步的验收，用三把尺子加两把数值尺子：

| 尺子 | 检查什么 | 不过怎么办 |
| --- | --- | --- |
| 可转化（入口） | 移交来的框架定义本身够不够格用：完整、无歧义、不冲突、隐含已识别 | 退回上一对补识别——不是自己重切 |
| 可分配（出口） | 拆出来的叶节点能不能用：原子性、全覆盖、单承接、候选声明 | 不合格的叶节点登记待迭代——分析迭代重切（§6.3.4） |
| 边界不越位（深度） | 切到哪为止：止于”可被分配”，太细就侵入方案层 | 调粒度：太粗继续切，太细把细节剥回方案侧 |

三把尺子管”切得规不规范”，看形状、不动数值。另有两把数值尺子管”指标断没断”：**指标闭合**（切出来的指标组合，能回到父指标吗？整机 ≤10W → 各模块之和 ≤10W，误差与可靠性同理）与分配预判（方案节点的指标，给需求留了满足空间吗?）。为什么这两查更要紧：切得再规整，指标闭合不了，系统照样不达标，指标闭合不过，全链返工；分配预判不过，方案白想、分配必然撞墙。

三个精化点，处处咬着本章既有概念：

1. ④ 先挂接、⑤ 后权衡：④ 是轻量覆盖映射（候选声明：只落不细），只给⑤ 的裁决提供覆盖率、均衡性、冲突性证据；正式细化留到⑤ 选定之后，被否掉的方案不白做工。
2. 派生需求回流：⑥ 定稿会派生对需求侧的新需求（接口约束、性能指标、可行性限制），必须回流需求基线、双向对账，方案侧不能单方面定义。这正是”结对双向”的一个落点：方案也在迭代需求。

3. ⑥”利益相关方确认”的双重身份：内部评审对质（决策理由、备选死因）＋ 利益相关方确认承诺，确认通过才冻结基线。这里的”确认”是利益相关方认可承诺，属理解确认与签认；第 8 章三把尺子里的确认要等真实使用场景取证。

> 与第 11 章划清：第 11 章的”架构生成六步”是**定驱动 → 生成备选 → 权衡选定 → 分配接口 → 视角规格＋表达基线 → 评审背书**（第 11 章 §11.4.3），那是架构设计这一级的生成动作；“构思—定义—表达—记录—沟通—认证”是 42010:2022 **§3.1 architecting 的动词串**，不是那六步。本章的六步是每一对”需求-方案”的通用功课。两套六步层级不同、对象不同，别混为一谈。

### 6.4.5 设计开发的验收与追溯

做了这么多功课，设计开发怎么验收？答案是回到树本身，五个维度，每个都呼应着本章的一个概念。这五维是喂给第 8 章三道验收签字的检查内容：评审、验证、确认依次签下去，签的是这五维过了没有。

| 维度 | 检查什么 | 合格线 |
| --- | --- | --- |
| 干枝（框架定义） | 树完整（根到节点无中断、同层抽象一致）、每个节点四要素齐（节点定义／承接需求／符合性分析／映射关系） | 100% |
| 叶节点（详细定义） | 分解完成、叶节点原子（守住防切过碎的下界）、指标完整（数值＋单位＋测量条件） | 100% |
| 分配生成 | 全覆盖、单承接、N:1 合理 | 100% |
| 分解分析 | 每个节点语义与指标都过（覆盖检查＋符合性分析），无”满足”空话 | 100% |
| 追溯 | 从原始需求到构件正向走通，从构件到原始需求反向走通 | 100% |

“干枝与叶节点”验收的是”一棵树”本身；“分配生成”是三自检；“分解分析”是覆盖检查与符合性分析；而追溯是把全链的树串起来的总验证——任取一条原始需求，看能否一路走到对应构件（正向）；任取一个构件，看能否说清它由哪些需求而来（反向）。走不通处，就是树与树之间断了。

最后回到第 3 章的一句话：设计开发止于可实现。现在你看到了”可实现”在树上的样子，叶节点切到可被单点落靠，节点具体到能被开发承接（带着分解源判据），每个节点都做过覆盖检查与符合性分析。设计开发的可靠性，不靠写得长，靠每一对”需求-方案”咬合得好。

### 6.4.6 认知反转

一个月后，恒信又开了一次方案评审会。还是那间会议室，还是那些人。

万辉在投影上打开的，不再是六十页文档，而是三页：

- 第一页：合同审批系统的框架树，一页纸，三个枝、十个节点；
- 第二页：需求叶节点与映射表，每条叶节点各有编号、描述、指标、验收标准、映射目标；
- 第三页：三张自检表，分配三自检、分解分析四查、每个节点的覆盖检查与符合性分析。

法务一秒钟定位到”审批流程”，找到驳回规则；财务翻到”合同校验”，看到对账口径；业务的小周指着”合同发起”说”这就是我要的”。

会议开了一小时，散会。万辉对徐漾说：“我第一次觉得，这份方案是’活’的，每一条叶节点都能被找到，每个决定都有来路。”

回到章首那场会。你以为设计开发是”写方案”，质量取决于文笔和长度。这一章的反转是：设计开发的对象是一对文档、不是一份，走的是三个动作、不是一支笔，方案不是写出来的，是种出来的。

四件事背后，是同一次方法时刻，第 1 章说的**织关系**——万辉把三百条需求排成一列（堆），徐漾教他织成树（结构）：堆起来的方案找不到路，织起来的每条需求都有位置。第 10 章说架构是压缩，也是同一个原理：用结构消化复杂度。

用第 1 章的话收一次束：对”设计方案”“框架定义”“分配生成”“分析迭代”“分解分析”这些词，你原来只是”见过”，知道设计开发对着”需求-方案”一对、每份文档是棵树、有三个动作。

这一章把它们推到”拥有”：能答全四问，

- 它是什么：一对”需求-方案”、一次 Y=F(X) 求值、每份文档是棵树（框架／概要／定义三个形态）、三个动作（分配生成、分析迭代、分解分析）；
- 不是什么：不是单独一份文档、不是文章、不是简单分派、不是返工丢人、不是多列几条、不是空话、不是架构；
- 从哪来：从”可验证、可落实”的内涵推出来，从 Y=F(X) 的规则里长出来；
- 往哪去：链上每一对逐对接力，只在分配步求值；链尾交给构件设计方。

能答全四问才算拥有，你手里有了一对”需求-方案”，一个函数、一棵树一棵树往下走。

> 方案不是写出来的，是种出来的——一对”需求-方案”、一次 Y=F(X) 求值、三个动作，在整条链上逐对接力。

用第 1 章 §1.4.4 的尺子量一下：“需求-方案结对”是**第 ② 级（图式）**的功夫，你学会了把结构一块一块搭出来；至于”这么拆对不对”的判断，要到**第 ③ 级（心智模型）**。

## 案例进展⑤｜一条线从老总画到开发

恒信第一次把整条链走通，徐漾和万辉坐在白板前，从下往上推了一遍。

最顶层，老总的一句话：“合同审批要数字化。”，存量材料，还没被组织。第二层，三方需求：法务要条款校验、财务要对账口径、业

务要流程时效。第三层，业务流程：一份合同从发起、法务初审、财务校验、业务终审、签章、归档，运行概念里的故事，现在长成了树。

第四层，系统需求：SYS 叶节点，上一对分解分析切出来的那批（顺序审批、金额校验、期限校验……）。第五层，产品构成方案框架：审批引擎、前端、合同台账、签章服务、通知服务、检索服务。

徐漾在白板上画了一条线，从”金额 ⩾ 500 万要重点审”这条要求，一路画到”审批引擎”那个节点：

“一条需求，从老总嘴里到开发手里，过了四对结对——每一对都是一次分配生成、分析迭代、分解分析。中间任何一道断了，这条需求就丢了。”

万辉盯着那条线看了很久，说：“所以方案不是写出来的，是每一对都走一遍三个动作、接力接出来的。”

徐漾笑了：“你现在才懂。那六十页文章，就是四层功课都没做，只写了一篇感想。”

---

## 小结

这一章回答的是”这份需求怎么被用来造东西”：一份需求与一份方案结成一对，一次 Y=F(X) 的求值，由需求开发出方案，并由方案迭代需求。

一对——设计开发的单元是一份需求文档配一份方案文档；一棵树——每份文档都是一棵树，干枝（框架定义）管结构、叶节点（详细定义）管内容，按定义到多深分框架、概要、定义三个形态，结构不轻易变、内容随时长。

**先定规则**——分解、构建、承接三套规则由人类主导制定，分解规则带着消费方的语义范围来定，单承接由分解规则直接保证；**三个动作**——分配生成（复用 → 改进 → 新增，长出骨架）、分析迭代（守住函数的单值性，违规就迭代需求分解）、分解分析（节点长出叶节点，接住的需求一定被实现）。

一条链——从存量材料经入口加工到第一份需求，再一份一份往下接：上一份的叶节点就是下一份的需求，追溯边自然生成；只有分配步在做承接，整段搬入步与形态轴步不做映射；链尾那一份只到框架，节点带着分解源判据移交构件设计方，执行换手，概念不分。

方案是结构，不是堆，第 1 章说的织关系，这一章在”需求-方案”这一对上长成了两棵树。

下一章，我们先看这棵树长成什么，产品数据：开发产品就是开发产品数据。一棵树的干枝、叶节点、根，全是产品数据。再下一章，评审、验证与确认，三个看门人上场，这一章的”覆盖检查与符合性分析”在更完整的验证框架里归位。

小白的工作夹里，多了一个”金额校验”的叶节点——她把这批叶节点的编号、指标、验收标准，一条条抄了进去。以前她得追着万辉问”做到什么程度算完”；这一章之后，编号、指标、验收标准都写在叶节点上，不用问人了。

---

# 附录

## A 本章概念总图

一对（需求-方案结对 = 一次 Y=F(X) 求值）：由需求开发出方案、并由方案迭代需求——正反交替、逐对收敛

Y=F(X)：X=需求、Y=方案、F=三套规则（分解／构建／承接）——每一对先定规则，规则由人类主导制定（AI 辅助）

分解规则以消费方的语义范围为制定输入——单承接由分解规则直接保证，迭代试错只是兜底

单承接约束 = 函数的单值性：可 N:1、不可 1:N

落点以语义定级：下游能负责这条语义的节点——干枝或枝梢，不预设落点在枝梢

一棵树（对一份文档而言）：设计文档是树，不是文章

树要能定位、能承接、能追溯（文章三样都缺）

干枝 = 框架定义（管结构：根到节点 + 映射表；节点双向可用：承接判据 + 分解源判据）

叶节点 = 详细定义（管内容：五件套，单承接搬走）

三个形态：框架（根→枝梢）／概要（根→叶，只到说明）／定义（概要＋详细定义）；框架有阶、叶有三种身份

三条依据：可条目化／可树形化（成树三条件：判据单维、条目单归属、分组完备互斥）／可按 N:1 分配

树形假设有边界：横切／共享／多维／循环四类纯树放不下——树是存储骨架，引用与追溯是关系载体

框架定义 ≠ 架构定义：文档层面不等于；描述层面可作架构的一种描述

三个动作（叶节点驱动的循环：每轮处理相关联的一组，直至处理完）：

准备：定规则 · 结构化存量材料 · 只做直接承接（复用）

分配生成（阶段一）—— 叶节点落到能负责它的节点上：复用 → 改进 → 新增

自下而上调树：收敛 = 覆盖完整 + 兄弟互斥；父语义三查：独立可定义、非兜底、最小覆盖

三自检：全覆盖／单承接／N:1 合理

分析迭代 —— 检查分配与生成：单承接成立吗？承载量合适吗？改进波及谁？

违规两分类：程序违规回炉重定分解规则再重切需求；切不动的整体走节点拆分、横切式承接

改进必做影响面复查；修正方向单向：节点是基准，需求是适配方

分解分析（阶段二）—— 节点 → 方案叶节点：切点沿消费方语义范围；语义四查 + 指标三情形

七种反模式：过分解／欠分解／影子需求／约束丢失／指标错配／指标落空／指标超预算

承接覆盖检查：该节点承接的所有需求的语义，包含于其叶节点语义之并（语义＋指标；满足／降级／丢失）

一条链：入口加工 → 利益相关方需求 → 业务流程 → 系统需求 → 产品构成方案框架 → 构件设计方

需求与方案相对：中间层是双面人（对上是方案、对下是需求）；上一份的叶节点就是下一份的需求

链上三类步：分配步（做承接，Y=F(X) 在此求值）／整段搬入步（不做映射，长干枝）／形态轴步（不是承接）

文档结对：有需求必有方案；骨架先行、填充并行；六步：接 · 验 · 想 · 对

· 裁 · 签

追溯边自然生成；链尾节点带分解源判据移交构件设计方—执行换手、概念不分链的覆盖边界：按派生关系生成、只承载明示条目；确认的判据与方针不在链上，另设参照层

验收五维：干枝 / 叶节点 / 分配生成 / 分解分析 / 追溯；设计开发止于可实现

## B 本章的能力级

**第 ② 级为主**

- **表征**——把”需求-方案结对”“框架定义”“详细定义”“分配生成”“分析迭代”“分解分析”“符合性分析”“单承接”几个词说准：结对是一对文档、树有两个部分三个形态、落点以语义定级、单承接是硬约束；
- **图式**——一次 Y=F(X) 的求值这条结构；每份文档一棵树（根 → 枝梢 → 叶节点）；框架、概要、定义三个形态与”最深形态”；两阶段（阶段一长骨架、阶段二填叶节点）；一条链上的逐对接力；
- 心智模型——为什么必须是树（可条目化、可树形化、可按 N:1 分配，以及成树三条件）；单承接为什么是函数的单值性、违规两分类各回哪里修；指标三情形怎么判；改进的涟漪为什么要做影响面复查；
- 解释框架——本章不建，由第 17、18 章承担。

> 能答全四问才算拥有——你手里有了一对”需求-方案”，一个函数、一棵树一棵树往下走。

## C 自查 · 这些说法对吗?

1. “设计方案写得越详细越好，六十页比一页树强。”——错。文章不可定位、不可承接、不可追溯；没有结构的详细只是堆砌。
2. “结对就是由需求开发出方案，需求侧等着就行。”——错。结对是双向的：由需求开发出方案，并由方案迭代需求（重切需求、调整承接、修订规则），正反交替、逐对收敛。
3. “一条需求分给两个模块做，双保险。”——错。违反单承接（函数的单值性），重复实现、验证无对象、变更难定位。
4. “叶节点切得越细越好，切到不可再分才负责。”——错。切点由消费方的语义范围定；语义原子性只是防止切得过碎的下界，为迁就节点把语义整体切碎，下游消费不了。
5. “分解规则应该全项目统一，一套切法拆所有结对。”——错。切法必须业务化，每个结对按自己的领域经验定；全链统一的是判据。
6. “符合性分析写’本节点满足上述需求’就够了。”——错。那是空话；要对每条承接的叶节点说清”怎么满足、指标多少”。
7. “指标拆不动的就先不管。”——错。要标注”向下一层传递”，指标不消失，留到方案阶段再算。
8. “先把需求全部写完、切完，再开始设计。”——错。结对设计是骨架先行、并行填充；叶节点驱动的循环逐轮推进，否则悬空叶节点、方案缺枝要到最后才暴露。

9. “第 4 章的需求分析和本章的分析迭代是一回事。”——错。需求分析是加工（往前看，需求清不清楚），分析迭代是检查分配与生成并迭代需求分解（往回修，叶节点合不合用）。
10. “分配生成、分析迭代、分解分析是三个先后大阶段，走一遍就完。”——错。它们是叶节点驱动的循环：每轮处理相关联的一组叶节点，循环推进直到全部处理完。
11. “框架定义和详细定义焊在一起维护，省得两处对不上。”——错。变更频率与原因不同：结构不轻易变、内容随时长；焊死则每次内容变更都拖着结构动。
12. “设计文档交付一次就完了，画完即定型。”——错。需求会变；不可追溯就漏改；干枝和叶节点分开演进，文档是活的。
13. “规则定一次，终生不动。”——错。分解规则随消费方结构演进：改进或新增节点时，受影响的分解规则随影响面复查同步修订。
14. “链尾系统设计方把构件规格也写完，才算交付到位。”——错。执行换手：设计方交付到节点（带分解源判据），构件规格是构件设计方接手后的第一件功课，它仍是一次设计开发，只是换了执行者。
15. “上一份的叶节点就是下一份的需求，所以链上每一步都在做承接。”——错。只有分配步在做承接；整段搬入步不做映射，形态轴步根本不是承接。落点是判断，判据是语义。

## D 练习

1. 找一份你们团队最近的设计文档，判断它是树还是文章。如果是文章，找出它”不能定位”“不能承接”“不能追溯”各一个具体例子。
2. 把你们系统的一个功能域画成一棵框架树（根→枝梢节点），标出分类节点和枝梢节点，再给一个节点写出它的承接判据和分解源判据（双向可用）。
3. 用语义四查检验一批需求叶节点：覆盖完整吗？不超出吗？一致吗？约束继承了吗？每查都给出你检查到的证据。
4. 取一条带指标的需求，用指标分解三情形走一遍：先判断是”可拆分／下放／暂不分解”，写出你的判断过程和结论。
5. 给你们的方案节点做一次分配三自检：全覆盖吗？单承接吗？N:1 合理吗？有没有悬空的叶节点？
6. 找一条”不知道该归谁”的叶节点，判断它是”叶节点切粗了（迭代重切）“还是”方案树缺枝（生成新节点）“，并说明你的判断依据。
7. 用承接覆盖检查核一个方案节点：它承接的所有需求的语义，包含于它的叶节点语义之并吗？试按”针对需求 A，本节点通过 X 机制满足 Y，指标 Z 满足／降级／丢失”写一句。

8. 解释”框架定义”和”架构”为什么不能混，并举一个你们团队（或你听过的项目）“架构＝一份文档”的词汇病例子。
9. 一棵树的干枝和叶节点为什么能独立演进？举一个”修剪枝条不动叶节点、长新叶节点不动干枝”的例子，说明这对变更控制的好处。
10. 把一个文档结对走一遍：取上层三条叶节点，分配到下层两个节点（复用、改进、新增各至少试一次），再对每个节点做覆盖检查，记录每一步的自检结果。
11. 用第 1 章的四问法量”分配生成”（或”分解分析”）：它是什么、不是什么、从哪来、往哪去？哪一问最薄？最薄处往往正是最易混用处。
12. 取一个文档结对，走一遍六步：①接需求 → ②验就位（三把尺子加两把数值尺子）→ ③出方案（至少两个候选）→ ④对需求（复用→改进→新增）→ ⑤做裁决（写下选择的理由）→ ⑥签定稿（分解分析＋利益相关方确认＋冻结基线），并说明每一步对应本章的哪个动作或概念。
13. 给一段相邻产品数据之间的步判类型：它是分配步、整段搬入步，还是形态轴步？判完再说这一步该做什么、不该做什么。

（答案要点见书末附录，与训练教材题卡同源）

## E 参考文献

**标准** - R-09 ISO/IEC/IEEE 15288:2023 —— §6.4.5 设计定义过程；§3.13 设计（设计是系统元素及其关系的规定，**其**完整程度足以支撑架构的合规实现）〔附录 D 该条锚 2015 版，本章引 2023 版条款号，随该条版本双轨一并校准〕 - R-04 ISO/IEC/IEEE 42010:2022 —— §3.2 架构；§3.3 架构描述（“框架定义 vs 架构”的界分参照） - R-103 ISO 9000:2026 —— §3.3.11 设计和开发；§3.5.1 需求（注 2：“规定的**要求**是明示的要求”——正本中译作”要求”，本书术语口径 requirement→需求，见附录 A T-26）- R-03 ISO 9001:2015 —— §8.3.3 设计开发输入〔库内无正本，属库外来源；条款号照录、版本随引〕- R-12 ISO/IEC/IEEE 29148:2018 —— 单条需求的质量特性〔条款号待核：仓库未见正式文本〕 - R-02 ISO 9000:2015 —— §3.4.8 设计和开发〔已被 R-103 取代；本章只作版本对照〕

经典文献 - R-70 SEBoK〔库外来源·不可核〕—— 分割／对齐／分配三动词，与本章”分配生成”两级同名不同物；三动词的库内一手位＝ISO/IEC/IEEE 15288:2023 **§6.4.5.1 注 2（§5.9 不含此三动词，旧注有误**—— §5.9 只给架构与设计的分工句：架构管适宜性·存续性·合意性，设计管兼容性与可行性）

库内方法件（内部引用，不给编号） - `00-generalspec/01-generalspec-sysdev.md` §2.2.2 —— 分解终止标准 - `20-pl4eos/10-pl4eos-subpl-sysdev/10-wfsysdev-4-eos/94-`

`eos-sys-dev-dbs.md`（v1.44）—— 产品数据清单、三条依据、三个形态与树间对应

> 对照阅读：本章”需求与方案结对”呼应第 3 章”需求、设计开发、质量”、第 4 章”需求工程”、第 5 章”需求的完整性”、第 8 章”评审、验证与确认”。

# 第 7 章 产品数据：开发产品就是开发产品数据

## 7.1 引言

第 6 章把一份需求做成了方案：一份需求配一份方案，结成一对；每份文档都是一棵树，干枝管结构、叶节点管内容，一对一对往下接。本章接着那棵树往下问一句：**这棵树到底是什么？**

它是产品数据。开发阶段你亲手造的不是实物，而是产品数据；实物只是这份数据复制出来的产物。这一章要立的就是这句话：**开发产品就是开发产品数据**。

先把这一章的位置说清。它在中篇的第二站，接第 6 章的设计开发落地，补上”造的是什么”这一问；它也是全书暗线的立论章——一个人的能力，最终体现在他交出的产品数据上，而这句话的根扎在本章。

先看产品数据的定义与它该有的样子，再磨开”开发产品就是开发产品数据”这句话的成立范围，最后看它在组织里怎么被用起来、归到哪里去。

### 7.1.1 误解现场

冰链科技 M3 二代的开发会，会议室墙上贴满了图纸。

水忠站在最前面，指着投影上的固件架构图，语气笃定：

“二代加了边缘断链预判——传感器异常时，箱体自己在本地判一次，不依赖云端。这一块是我最得意的地方。硬件已经改了两版，固件这周就能跑。”

会议室里有人点头，文正却没动；他翻着手里的本子，问了一句：

“二代的功能基线呢？‘边缘断链预判’这条需求，验收标准定了吗？拆到哪些模块？谁负责验证？”

水忠愣了一下：“需求……不是写完了吗？在共享盘那个文件夹里。”

“哪个文件夹？”文正追问，“我搜’二代’，搜出三个版本的需求文档，还有一个叫’M3 二代最终版’——哪个是准的？你刚才说硬件改了两版，改了什么、为什么改，有没有记录？”

会议室安静了。水忠这才意识到，他一直在说”把产品做出来”，可产品到底长什么样、为什么长这样、怎么证明做对了——这些他一件都说不全。

文正合上本子，轻声说：

“水忠，你把’开发产品’想成’把实物做出来’了。可我们开发一个产品，真正在开发的，是一整套**数据**——需求数据、设计数据、质量数据。实物，只是这套数据被’复制’出来的结果。”

水忠还是没转过弯：“数据不就是文档吗？开发完补一补不就行了？”

“数据不是文档。”文正说，“文档是容器，数据是内容。你刚才说’需求在共享盘’，那是文档的位置，不是需求本身。”

水忠又问了一句：“那实现方案呢？画图、BOM、工艺这些，算产品数据吗？”

“算，而且是最重的一类。”文正说，“八个方案，从技术方案一路排到运维方案。第 3 章说设计开发止于可实现，止的就是这里；方

案齐备，才算可实现的终点。制造全照着它造箱子，错一处，一万台箱子错一万处。”

水忠盯着那面墙，第一次开始想这个问题。**产品是实物，数据是文档**——他干了几十年硬件，这个直觉从来没被问过。

一直没开口的产品开发经理永兵接了一句：“第 3 章那张参数表，是我签的’通过’。当时我们比的是谁做得高，没问一句——它是不是 M3 真正要的东西。”

这一章要推翻的，正是这个直觉：你造的是数据，不是箱子。接下来的三块，把”产品数据”这棵树的根扎下去，再看它怎么被造出来、为什么决定产品质量、又怎么变成企业资产。

### 7.1.2 这一章要回答什么

读完本章，你应该能回答：

1. 产品有几个”存在”？开发阶段你亲手造的是哪一个？
2. 什么是产品数据？它和文档、和配置项、和产品本身，分别差在哪？
3. 一份设计方案为什么必须能定位、能承接、能追溯？
4. 为什么说”开发产品就是开发产品数据”？这句话在什么范围内成立、什么范围不成立？
5. 产品的三种存在状态是什么？设计态为什么特殊？
6. 数据质量能决定产品质量吗？能决定到什么程度？
7. 用户和客户在产品数据里扮演什么角色？谁是”出题人”，谁是”判卷人”？
8. 职责该锚定在流程上，还是锚定在产品数据上？为什么？
9. 实物产品的实现方案包括哪些？为什么它最重？

10. 产品开发的流水线上，流动的是什么？缺陷为什么会沿流动放大？

11. 怎么判断一个活动值不值？

12. “数字化”和”转型”是一回事吗？差别在哪？

### 7.1.3 这一章的骨架

这一章要立的是”产品数据”这棵树。先把骨架立起来，后面三块都是给它添血肉。

> **开发产品就是开发产品数据**：产品有一个信息存在（产品数据）；设计态里开发产品 ≡ 开发产品数据；数据质量决定产品质量的上限；产品数据是组织协作的对象，也是企业最重要的资产。

这一章的骨架是**一个判断、一横一纵、一个收口**。

**一个判断**——产品有两个存在，物理存在和信息存在。开发阶段你亲手造的，是信息存在（产品数据）；实物，是信息存在被”复制”出来的结果。第 6 章你学会把方案种成树，这一章告诉你：那棵树，就是产品数据。

换个说法，这也是外化地图：每个人脑子里都有一张地图（第 1 章 §1.4.4 的四层：表征、图式、心智模型、解释框架），而产品数据就是那张图从个人脑子里搬出来、变成组织能共用的那一份。开发产品，说到底是把心里的图，绘成大家都读得懂的图。

一横，说的是产品的三种状态：设计态（as-designed，纯数据）、制造态（as-built，数据＋实物）、使用态（as-maintained，实物＋

实况）。设计态里没有实物，它的全部存在就是数据，所以开发产品，开发的就是产品数据。

一纵——产品数据往纵深走，最终落到企业的账本上。它是协作的对象、评价的锚点、问责的落点，也是最重要的资产（产品会死，数据体系活着）。

这一章的方法底座时刻，是第 1 章概念能力里的织关系：把”产品”“产品开发”“产品数据”“两域三态”“质量”“资产”织成一张网，让每个概念各就其位，而不是把它们当成一堆词堆在一起。

这一章还有四样值得先立起来：实现方案（八个方案，最重的一类）、流水线（流动的是产品数据，缺陷沿流动放大）、增值判据（活动值不值看数据是否增值）、数字化与转型（存起来 vs 用起来）。它们会依次立起来，最后收进企业的账本。

第一块把”产品数据”看清（它是什么、该是什么样子、含实现方案），第二块把”开发产品＝开发产品数据”的边磨开（它成立在哪、质量机理如何、流动的是什么），第三块看它怎么用（协作、价值、职责、增值、资产、数字化）。

---

## 7.2 产品数据是什么

> 这一块把”产品数据”看清：先看产品有两个存在，再拆产品数据的定义与六类，然后把四个最容易混的边界磨开——它不等于文档、不等于配置项、不等于产品、也不等于架构本身。

### 7.2.1 产品：物理存在与信息存在

水忠盯着墙上的图纸，其实他盯的正是这个问题：M3 温控箱，到底”是”什么？

在制造车间里，M3 是一台台能插电、能制冷、能报警的实物箱子。可在开发部的电脑里，M3 是另一副样子：一份需求规格、一张张图纸、一版版固件源码、一堆测试报告、一张 BOM 表。这副样子没有重量、不能制冷，但它**决定了**那台能制冷的箱子长什么样。

这就是产品的两个存在。

**物理存在**——你摸得到、用得了的那台实物；它在制造、使用中发生，会老化、会报废。

**信息存在**——描述这产品”是什么、为什么、怎么做、怎么证明对”的全部数据。它在开发、验证中发生，可以被复制、被传递、被留存。

水忠几十年只跟物理存在打交道：图纸画完给车间，固件编完烧进芯片，剩下的事与他无关。所以他天然以为”产品 = 实物，数据 = 文档”。

可问题在于：物理存在的每一件，都是从信息存在里”复制”出来的。车间照着图纸造箱子，生产线照着 BOM 采物料，固件照着源码编译烧录。同一份图纸，可以造一万台箱子。

所以从开发的角度看，你真正”造”的东西只有一个：信息存在。实物只是信息存在的复制品。

> 产品 = 需求的物化 + 价值的载体 + 过程的输出；而这三样，全部以数据为形态存在。

> 术语注：这个”三视角一句话”是本书对工程界产品视角的凝练（三视角的出处见 §7.2.2：ISO 9000:2026 §3.7.9 产品；又，ISO/IEC/IEEE 15288:2023 §3.32 的产品定义与 §3.7.9 同文，其源注记为据 9000:2015 改），不是任何标准的原文；“以数据为形态存在”是本章的主张，它把三视角收束到”开发造的是信息存在”。

水忠的直觉错在哪？他以为”开发”是对着物理存在工作。可事实上，开发全程没有一件实物可摸——你写需求、画图纸、编源码、跑测试、攒证据，全部是在数据上工作。直到移交制造，数据才第一次被”复制”成实物。

信息存在还有一个实物没有的特性：它不磨损、可复制、可传递。实物用一次旧一次、报废一件少一件；产品数据复制一万份不损耗、传递一百年不衰减。这也是为什么企业真正的资产是数据不是实物：实物会折旧，数据会增值。

### 7.2.2 产品：需求、价值与过程

在进入”产品数据”之前，先把”产品”这个词也钉一钉，否则”产品数据”会失去底座。

工程界看”产品”，有三个视角，合起来是一句话：

| 视角 | 产品是什么 | 一句话 |
| --- | --- | --- |
| 对需求 | 规定需求的实现物 | 需求是”应然”，产品是”实然” |
| 对顾客 | 价值交换的介质 | 价值从组织流向顾客的载体 |
| 对过程 | 开发过程的输出 | 产品是过程的结果物，不是过程本身 |

水忠的 M3，三个视角都对得上：它是需求规格的实现物（需求说制冷到 -20℃，它就要能制冷）、是给冷链客户的载体（价值靠它兑现）、是设计开发过程的输出（图纸、固件、验证证据的结果）。

这里还有一条容易混的边界：**产品与服务的分界**。ISO 9000:2026 §3.7.9 给产品的定义是”组织的一种输出，这种输出可在组织与顾客之间**不发生交易**的情况下产生”（英文：output of an organization that can be produced without any transaction taking place between the organization and the customer）；§3.7.10 给服务的定义是”**组织的**、至少有一项活动必须在组织与顾客之间进行的输出”。分界的关键词是 transaction（交易）。而按 §3.7.8 注 1，一项输出算产品还是算服务，“取决于所涉特性的主导性”；管理口语里两者才并列。

把”产品”钉住之后，“产品数据”就清楚了：它是产品的信息形态，三个视角的每一份记录。

再顺带把”产品”和它最容易混的几个词分开：

| 概念 | 判据要点 | 与产品的关系 |
| --- | --- | --- |
| 服务 | 至少一项活动必须在组织与顾客之间进行（ISO 9000:2026 §3.7.10） | 与产品由”所涉特性的主导性”分开（§3.7.8 注 1） |
| 系统 | 由部分组成或元素的安排，它们合起来呈现出各构成部分所不具有的某种既定行为或含义（ISO/IEC/IEEE 15288:2023 §3.46） | 产品通常是系统，但系统不必然可交付 |
| 项目 | 创造独特产品、服务或成果的临时性工作（PMBOK® Guide〔库外来源·不可核〕） | 创造产品的过程载体 |
| 结果 | 某事的后果；可取输出（output）的形式，也可取成果（outcome）的形式（ISO 9000:2026 §3.7.1） | 产品的”目的端” |

水忠嘴里的”M3”，同时是产品（可交付）、系统（元素与既定行为）、项目（开发它的临时组织）、结果（冷链运输能力），四个词四个层面，别混成一团。

还有一层经营视角值得点一句：在集成产品开发（IPD）里，“产品”常指产品包（offering），即硬件、软件、服务与解决方案的组合交付，多了一个商业维度。技术视角回答”产品是什么物”，经营视角回答”顾客愿意为什么付钱”，两个视角不冲突。

### 7.2.3 产品数据：形式化的信息表示

“信息存在”这个词太文绉绉。工程界有更精确的名字：**产品数据（product data）**。

ISO 10303-1（STEP，产品数据表达与交换标准）给的定义是：

> **product data**: representation of information about a product or a family of products in a formal manner suitable for communication, interpretation, or processing by human beings or by computers——关于产品或产品族的信息的、以形式化方式所作的表示，适合由人或计算机进行通信、解释或处理。
> 版本与出处说明：定义句引自 ISO 10303-1:2021 §3.1.29；**库内无该标准任何版本的正本**，此处据《产品数据研究报告》（14 号）§3.1 与《设计与开发落地方法研究报告》（20 号）§1.1 转引，条款号待正本复核。早期版本的定义还强调一个特征——“与产品整个生存周期相关”：不只开发期，从概念到退役，凡是描述产品的信息都是产品数据。

拆开看，四个要点：

第一，“关于产品的信息”。产品数据是产品的信息形态，是描述被描述者的那一层，不是产品本身。

第二，“一种表示（representation）”。数据是内容，不是载体，载体可换。

第三，“形式化”。表示是形式化的，有确定的结构与语义，才能被一致地解释：同一棵设计树，你读、我读、机器读，读出的是同一个意思。这是第 6 章那套机械判据（单承接、覆盖公式）能成立的前提：比对对象必须先是形式化表示。

第四，“由人或计算机处理”。产品数据既给人读，也给机器处理，这一项是第 6 章人机分工的根据：人类据表示制定规则，算法据规则执行设计动作与机械核对。

产品数据到底包括哪些？工程实践里大致分六类：

| 类别 | 典型内容 |
| --- | --- |
| 需求数据 | 需求规格、利益相关方需要、验收准则（含操作目标清单——系统操作目标的可判定集合，需求侧编制、业务方确认，见第 4 章） |
| 设计数据 | 图纸、CAD 模型、架构描述、设计规范 |
| 制造数据 | 工艺文件、BOM、NC 程序 |
| 质量数据 | 测试报告、验证证据、缺陷记录 |
| 服务数据 | 维护手册、备件清单、故障数据 |
| 管理数据 | 变更记录、评审记录、基线标识 |

六类数据，覆盖产品从”该做什么”到”做对了吗”到”用起来怎么样”的全程。水忠的”M3”，在这张表里占了一整行，可他从没把它们当成同一个东西。

> 术语注：六类数据里的”架构描述”是设计数据的一类，其定义承 ISO/IEC/IEEE 42010:2022——本书把**架构**凝练为”系统不可还原的涵义”（第 10 章展开）；**架构描述（AD）是它落纸的工作产品**（§3.3：An AD is an expression of an architecture）。

六类数据不是平起平坐的。其中设计数据 + 制造数据合起来，就是常说的实现方案——实物产品按”止于可实现”的口径具体化为八个方案，§7.2.6 专门展开。这里先辨析一句：八个方案里含测试、操作、运维方案，其产出归质量数据与服务数据；八方案是”实现过程要交的方案”，六类是”沉淀下来的数据”。两把尺子按”方案 vs 数据”分开，不重复。

顺着 ISO 10303 的定义，把底层的词也拆一层：

- 对象（object）：任何可感知或可想象的事物（ISO 9000:2026 §3.5.3）——那台 M3，或它的图纸；
- 数据（data）：关于对象的事实（§3.8.1）——“制冷 −20℃”“固件 v2.4”；
- 信息（information）：有意义的数据（§3.8.4）——把”固件 v2.4 在 3 月 15 日烧录”组织起来，才有意义。

三层一叠，“产品数据”就精确了：它是有意义地组织起来的、关于产品的事实，也就是产品信息的形态。

再往下一层，术语学还分了”属性”与”特性”两支，别混：**property（属性）**是对象的特征（ISO 1087:2019 §3.1.3），**characteristic（特性）**是属性的抽象（§3.2.1）；而 ISO 9000:2026 §3.10.1 的 characteristic 译作”区别性特征”，是质量管理语境下的另一套用法（distinguishing feature，可固有、可赋予）。attribute（属性）是属性的命名化形式（ISO/IEC 2382-17:1999 §17.02.12，实体的命名属性）。两套语境不可互推：一个走术语学，一个走质量管理。本书凡说”固有特性满足需求”，用的是后者；而”特性的抽象／属性的命名”这一支，本章只在辨析时点到。

六类数据再展开一层，每一类都有”从哪来、到哪去”：

- 需求数据：从利益相关方需要转译而来，流向设计——它的质量决定”做的东西对不对”；
- 设计数据：从需求数据物化而来，流向制造——它的质量决定”方案好不好”；
- 制造数据：从设计数据复制而来，流向实物——它的质量决定”实物准不准”；
- 质量数据：从验证活动取证而来，流回需求与设计——它的质量决定”怎么证明对”；
- 服务数据：从使用现场回流，流回下一代设计——它的质量决定”产品能不能变好”；

· 管理数据：从变更与评审记录而来，贯穿全程——它的质量决定”怎么说得清”。

### 7.2.4 产品数据不等于什么

把六类数据放上桌，第一件要磨的边，是”产品数据不等于文档”。

**数据是内容，文档是容器**。同一份需求数据，今天在 Word 里、明天在数据库里、后天在墙上——容器换来换去，数据还是那份。反过来，一个 Word 文件里可能装着需求数据、设计说明、评审记录三样东西：一个容器，装着多份数据。

水忠说”数据不就是文档吗”，错就错在把内容当成了容器。文档只是产品数据最常用的载体，不是产品数据本身。

**产品数据不等于配置项**。配置项是”被纳入配置管理范围的实体”（第 13 章会细讲）。产品数据是全部信息，配置项只是其中受控的那部分——被基线化、被变更控制的才是，草稿不是。六类数据里，只有被”管起来”的才算配置项。

产品数据不等于产品。描述与被描述者，本来就是两个东西。但验证要求两者一致——配置审计查的就是”数据说的”和”实物做的”对不对得上。水忠说”图是图、箱是箱”，对；但他没意识到，箱子的好坏，先由数据的好坏决定，这一点第二块专门讲。

产品数据也不等于架构本身。**架构没有形态、交不出去**，它是系统”少了它就不是它”的那组基本概念或属性与管控原则（第 10 章专讲）；能进清单、能冻结基线的，是架构描述（AD）。它是设计数据的一类，与设计描述（DD）平级。把架构当成交付物，是把没有形态的东西当成了可交付的**工作产品**（ISO/IEC/IEEE 15288:2023 §3.6：人

工制品是”在项目期间产生并用于捕获和传递信息的工作产品”）。第 6 章”框架定义不是架构定义”，讲的是同一条线的另一半。

先把这句话钉在这：产品数据，是产品的信息形态，也是开发工作的全部对象。水忠说的”把产品做出来”，正确翻译是”把产品数据造全、造准、造到可制造可验证”。

把几个词排一排，关系就清楚了：

· 产品是”物”；

· 产品数据是”物的信息”（全部）；

· 配置项是”被纳入配置管理范围的实体”（受控的那部分）；

· 文档是”装信息的盒子”（一个容器）；

· 架构描述是”架构的落纸件”（设计数据的一类，可冻结基线）。

几个词几层，水忠以前全混成”产品 + 文档”两个词；第 13 章配置管理会用上这套区分。

### 7.2.5 产品数据该有的样子

第 6 章你刚学会一件事：设计方案不是一篇散文，是一棵树，干枝（框架定义）管结构，叶节点（详细定义）管内容。

现在把两章接起来：**那棵树里的每一样，都是产品数据**。

一棵树里有什么？干枝上有”结构是什么、承接了谁”（框架定义）；叶节点上有”每个模块具体做什么、指标多少、验收标准是什么”（详细定义）；分配表把叶节点搬去下一棵树；符合性分析记录每片叶子接没接住上游。这些东西，全部落在 §7.2.3 的六类数据里。设计数据、需求数据、质量数据，没有一样是”实物”。

所以第 6 章的树，不是”文档结构”，是**产品数据的结构**。万辉把方案写成六十页散文，问题不止在”不好查”，更在”那不是产品数据

该有的样子”。产品数据必须可定位、可承接、可追溯，树形正是为这三样而生。

**可定位**——每条内容有编号、有位置，说得清”它在哪”；可承接——上一份的叶节点有明确的落点，说得清”谁接住了它”；可追溯——从任意一片叶节点往回走，走得到它是从哪条需求来的。散文式的写法，一条也做不到：你没法在六十页里指认”这条需求落在哪一段”。

这一层接上之后，后面的话就好说了。既然设计开发造的就是产品数据这棵树，那么评审、验证、确认（下一章）查的就是这棵树；模块工程师（第 9 章）种叶节点，模块架构师（通常即模块工程师自己）种干枝，系统架构师（第 18 章）种架构描述与决定；配置管理（第 13 章）管的，是让这棵树”一直在变还说得清”。

这棵树往下长，就是下一节的实现方案：树是产品数据的骨架，实现方案是树落到车间里的样子。

### 7.2.6 实现方案：最重的产品数据

§7.2.3 埋下的**设计数据 + 制造数据 = 实现方案**，这一节展开。它是产品数据里分量最重的一类——其他数据定义”该做什么、怎么证明对”，实现方案定义”怎么做”。

实物产品的实现方案，按设计开发”止于可实现”的口径，是**八个方案**，每个方案 = 给一个下游环节的”可实现需求”：

| 方案 | 给谁 | 内容 |
| --- | --- | --- |
| 技术方案 | 客体本身 | 结构、模块划分、接口关系、关键技术指标 |
| 物料采购方案 | 采购 | 物料清单（BOM）、规格、合格准则 |
| 制造方案（工艺） | 生产 | 工艺路线、工序卡、工装、NC 程序 |
| 测试方案 | 检验 | 测试计划、接收准则、验证用例 |
| 交付方案 | 交付 | 包装、运输、交接要求 |
| 部署方案 | 实施 | 安装、调试、上线要求 |
| 操作方案 | 用户 | 操作手册、使用场景 |
| 运维方案 | 维护 | 维护规程、备件清单、故障处理 |

八个方案的本质，是一个东西的八个面向，不是八件独立的事。“规定需求”按下游环节分配下去，采购照着买、生产照着做、检验照着测、实施照着装、用户照着用、维护照着修。第 6 章那棵树的叶节点，落到制造环节就是这一整套方案；**方案齐备 = 可实现 = 设计开发的终点**（第 3 章讲过）。

把”分配”落到物料这一层，物料的需求就现形了：它是产品需求经设计开发转化到物料层的”规定需求”，落纸形态就是物料采购方案里的规格 + 合格准则。这份需求不是采购自己定的，是设计开发分配给采购的。

设计开发止于”可实现”（第 3 章）。物料需求落在信息域（产品数据），物料实物落在物理域。来料检验（IQC）正是两域之间的对照动作：拿信息域的规定需求，量物理域的实物特性。

定制件还多一层。物料需求往下游走，是采购与验证的输入（它是产品设计开发的输出）；往上游走，又是供应商那一级设计开发的输

入。供应商拿它再转化一轮，输出更细的物料规格与工艺。每一级元素都走一遍完整的设计开发过程。这就是设计开发的递归。

> 术语注：六类数据与”八方案”是本章按第 3 章”止于可实现”口径对”实现方案”的归纳与操作化，不是任何标准的清单。八方案的名目与”方案齐备即终点”的判据，依设计开发概念研究报告（01 号）§4.3；其条款落点在 ISO 9001:2015 §8.3.5、§8.4、§8.5.1，库内无 ISO 9001 正本，属库外来源、不可核。标准层面只有 ISO 10303-1 的”产品数据”与 ISO 9000:2026 §3.3.11 的”设计开发”定义。八个方案可按你们产品的下游环节增删（退役与回收属 ISO/IEC/IEEE 15288:2023 生存周期末段，增列即九个方案，其需求侧见 §5.2.5）。

拿水忠的 M3 二代落进这张表：技术方案里是结构布局和接口契约，物料采购方案里是 BOM 和压缩机选型，制造方案里是注塑工艺和固件烧录，测试方案里是制冷性能试验大纲。实现方案不是一张图纸，是八份方案合起来的整套产品数据。

为什么它最重？因为制造环节完全照着它复制——错一处，一万台箱子错一万处。第 6 章说”方案不是凭空想出来的，是骨架分解出来、候选构思出来的”，这一章补一句：实现方案，是产品数据里最重的一类。

软件系统（比如恒信）的实现方案呢？一样是产品数据，只是它没有独立的制造环节，实现方案几乎就是产品本身（§7.3.4 展开）。

## 概念卡片 · 产品数据

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ①⑥ | 关于产品或产品族的信息的、形式化的表示，适合由人或计算机处理（ISO 10303-1:2021 §3.1.29〔库内无正本〕）——产品开发每一条输出都是产品数据；对立面：产品本身（物理存在），描述与被描述者 |
| **它不是什么** | ⑦⑧ | 辨析：≠文档（数据是内容、文档是容器）、≠配置项（受控的那部分才是）、≠产品（实物）、≠架构本身（可交付与可冻结的是架构描述 AD）；误区：“数据是开发完再补的文档”——数据是开发工作的全部对象 |
| **它在哪类** | ②③④⑤⑨ | ISO 10303（STEP）为数据交换而定义；product data 泛指设计/生产/开发产品数据，强调作为对象、评价的锚 |

## 案例进展①｜“BOM 和图纸，对不上”

M3 二代的实现方案出来两周，制造的逸人来找水忠。

“水忠，你这份 BOM 和装配图对不上。”逸人把两张表摊开，“BOM 里压缩机写的是 A 型号，装配图上画的是 B 型号——底座孔位都不一样。我按图纸备了料，车间一装就装不上。”

水忠翻了半天：“图纸是设计那边出的，BOM 是工艺那边配的……两边各干各的。”

“所以实现方案不是一份，是好几份拼起来的。”文正在旁边说，“技术方案、图纸、BOM、工艺、固件、测试，它们是一个整体——同一台 M3 的实现方案。任何一层对不上，制造就照着错。”

水忠第一次意识到：他以为”图纸”就是实现方案，其实实现方案是一整套产品数据（技术、物料采购、制造、测试、交付、部署、操作、运维八个方案），图纸只是其中一层。BOM 错、工艺错、固件错，跟图纸错一样致命——因为制造全照着它来。

---

# 7.3 开发产品造的是什么

> 这一块把”开发产品 = 开发产品数据”磨开：先看产品开发这件事本身（定义与复制），再看产品在域与态里的存在方式，划出那句话的成立范围，看软件为什么特殊，讲数据质量与产品质量的关系，最后把流水线放进来——流动的正是产品数据。

### 7.3.1 产品开发：定义与复制

进这一块之前，先看”开发”这个动词本身，不然”开发产品就是开发产品数据”没有落点。

ISO 9000:2026 §3.3.11 给设计开发（design and development）的定义是：

> **design and development**: set of processes that transform requirements for an object into more detailed requirements for that object——把某一客体的要求转换为该客体更详细的要求的一组过程。

注意落点：输出的仍是**要求**（更详细的要求），不是”实物”。产品开发的直接产物是产品的定义与基线，生产环节才把它复制成实物。

顺着这个定义，产品开发有三个特征，正好解释水忠的 M3：

**第一，一次性定义，多次复制**。开发定义基线，生产复制基线；开发高风险、高成本、低重复（定义错了，后面全错）；生产高重复、低成本（复制对了，可以无限造）。水忠花两年定义 M3 二代，车间用两周造出第一批：定义和复制的代价，差着两个数量级。

第二，跨职能协同：市场、研发、制造、采购、财务同时参与。不是研发画完图丢给车间，而是需求、设计、制造、验证的数据在多个职能之间接力。

第三，以评审为门：每个阶段出口用评审把关，防止错误向后传递放大；第 8 章的三个看门人，就是站在这些门上的。

一句话收住：产品开发是把”应然（规定需求）“变成”实然（产品）“的系统化过程；这个过程的每一步，都落在数据上。

开发的输入是市场需求与已有基线，输出是产品工件与产品规范、基线（baseline）。开发在”不断逼近并冻结一组基线”，而基线就是一组受控的产品数据（ISO 9000:2026 §3.10.12：经批准的配置信息，确立**产品或服务**在某一时点的特征、作为产品或服务整个生存周期各项活动参照；第 13 章配置管理会接上）。

这条链上有一处容易漏的接驳：从”需求定义”到”设计定义”之间，还夹着一个架构定义过程（ISO/IEC/IEEE 15288:2023 §6.4.4 system architecture definition process 与 §6.4.5 design definition process）。架构定义的输出，正是设计定义的输入——它的身份从”成果”转为”规定需求”。省略这一环，链条就不完整。（第 10、11、16 章分别展开。）

开发产品的全过程，需要的技能正好对应数据六类：

- 需求工程 → 需求数据；
- 架构与设计 → 设计数据；
- 工艺与制造工程 → 制造数据；
- 评审与验证 → 质量数据；
- 跨职能协同与市场洞察 → 服务与需求数据；
- 项目管理与配置管理 → 管理数据。

一个团队能不能开发好产品，就看成这六项能力对应的数据，造得全不全、准不准、可不可追溯。水忠的团队，硬件和固件是强项，需求工程和配置管理是弱项，所以 M3 二代在”功能基线”和”数据断层”上翻车。

### 7.3.2 两域三态：产品的存在方式

产品数据不是抽象概念，工程实践给它记了两笔账：它在哪个域，又在哪个态。

**两个域**是信息域和物理域。

信息域里发生的是开发与验证：需求被写下来、设计被画出来、验证被记录下来。物理域里发生的是制造与使用：实物被造出来、被用起来、被修起来。两个域之间靠**映射**连着，工程上把它叫数字孪生。

> 术语注：本章”数字孪生”只指”信息域的模型跟上物理域的实况”这一层意思，不作展开。该词在报告库中**仅三份报告涉及**（概念学习方法论、设计开发概念、产品数据研究报告）且**无英文对应写法**，属**边界澄清**、不引作论据。

三种状态（PLM 视角）是产品在生存周期三个阶段的不同形态：

| 态 | 数据内容 | 产品 = |
| --- | --- | --- |
| 设计态 as-designed | 需求、模型、规范、**EBOM**（工程 BOM） | 数据（还没有实物） |
| 制造态 as-built | **MBOM**（制造 BOM）、序列号、批次、制造记录 | 数据 + 实物 |
| 使用态 as-maintained | 维护记录、配置实况、现场变更 | 实物 + 实况 |

注意三种状态里的产品越来越重：设计态里产品 = 纯数据（一台 M3 二代还没诞生任何实物，全部存在就是那套需求、图纸、源码、验证计划）；制造态里产品 = 数据 + 实物（车间照图纸造出第一台，箱子还

连着 MBOM、序列号、制造记录）；使用态里产品 = 实物 + 实况（维护记录、配置实况、现场变更反过来成为新数据）。

这三态的依据主要来自一份报告（产品数据研究报告，14 号）；其中 as-maintained 一态在报告库中为孤例，此处如实标注，供读者自行校准。

水忠的直觉错在只认制造态和使用态：他眼里产品就是实物，但开发工作全部发生在设计态，那里只有数据。

再把”开发产品 = 开发产品数据”用三个视角检验一遍：产品数据 = 产品的信息本质，成立；开发的直接产物 = 产品规范与定义，成立；产品 ≠ 数据，这一条成立（还有实物、体验、价值实现）。合起来就是这一章反复说的那句话：开发产品开发的是产品的”定义”，制造产品制造的是产品的”实物”；“开发 = 定义生成”成立，“产品就是数据”不成立。

水忠在冰链看到的”云端追溯平台”，本质就是两个域之间映射的一个雏形。但只有使用数据回流、设计数据实时更新，它才真正跟得上现实，否则只是一个存数据的仓库。

管这套追溯平台的董勐在旁边补了一句：“追溯数据就是产品用起来之后往回吐的那一环——箱子把温度实况吐回云端，平台收进来、存成可查的记录。它要作数，先得采集可靠：现场断一秒，追溯就缺一块，缺了块，整条链都不完整。”水忠听到这里愣了一下：连使用期的实况，也是产品数据。

### 7.3.3 开发产品就是开发数据

现在可以给本章标题那句话划线了——那句话的全称是**开发产品就是开发产品数据**；本节就给它划成立的边界。

> 术语注：这里的”同一性”（**设计态同一性**，开发 ≡ 定义生成）与”同一性”（**同一性目的论**，系统是谁由目的构成、演化中仍是它自己；见附录 A T-144）是同一个词的两个概念。前者讲”开发这个动作和生成定义是同一件事”，后者讲”系统是谁跟着目的走”，别混。

**在设计态，开发产品 ≡ 开发产品数据，没有歧义。** 因为设计态里产品没有物理实体，它的全部存在就是那组受控的产品数据。需求数据定义”该做什么”，设计数据定义”怎么做”，质量数据定义”怎么证明对”。数据完备到”可制造、可验证”，开发才算完成。

这一句为什么重要？因为它把”开发”的实质钉死了：开发不是”做东西”，是”生成定义”。但这句话有两个边界，必须划清：

第一，只在设计态、对纯信息产品严格成立。制造态里有实物环节（制造偏差、材料、工艺、体验），不成立；物理域的一切，数据管不着。严格的说法是”开发 = 定义生成”。

第二，“开发产品 = 开发产品数据”不等于”产品就是数据”。产品还有实物、还有体验、还有价值实现。数据是产品的信息形态，不是产品本身——§7.2.4 刚划过的边界，这里不收回。

一句话：水忠以为自己是在”做东西”，其实他是在”把定义造全”。

这条同一性不是一句口号，它有实操意义：它决定你把力气花在哪。既然开发 = 定义生成，开发的管理重心就该放在数据上——评审

数据、质量数据、管数据基线，而不是盯着”样品有没有做出来”。水忠团队以前考核看”样机出来没”，现在该看”数据齐不齐、准不准、可不可追溯”。样机只是数据完备性的一个旁证。

### 7.3.4 软件为什么特殊

有一类产品，开发与制造之间的那道界几乎不存在：**软件**。

对硬件，开发和制造是两件事。开发产出的是产品的定义（需求、图纸、BOM、工艺），制造照这份定义把实物做出来；定义写得全不全、对不对，直接决定做出来的箱子对不对。这就是”数据质量决定产品质量”的机理。

软件没有独立的制造环节：你写的源码、编译出的程序，本身就是最终交付物。开发就是生产，“定义”与”实物”合成一步。这就是软件迭代能快到按天算、硬件要按版本爬坡的原因：硬件每改一次都要重新打样、试产、验证，软件直接改定义本身。

软件和硬件的数据特征还能再对比一列：

| | 软件 | 硬件 |
|---|---|---|
| 开发与制造 | 合成一步（编译即交付） | 两件事（定义与复制） |
| 制造环节 | 无（编译即交付） | 有（打样、试产、量产） |
| 修改成本 | 改定义（低成本） | 改定义 + 重做实物（高成本） |
| 版本节奏 | 按天迭代 | 按版本爬坡 |

水忠做硬件，最深的体会是”改一次固件，要重烧、重测、重验证”。软件没这个负担，但反过来也更容易”改着改着忘了改了什么”。所以软件的配置管理，比硬件更依赖产品数据的严谨。

### 7.3.5 产品质量的上限由数据质量决定

水忠还有个朴素信念：数据是文档，质量在车间里，这句话对了一半。

对的一半：**实物环节确实不在数据里**；制造偏差、材料批次、工艺水平，数据管不着。

错的一半：**开发阶段可评价的质量，就是数据质量**。设计态里没有实物，你能拿什么评判”产品好不好”？只能拿产品数据——需求全不全、设计对不对、验证有没有证据。水忠说”质量在车间里”，可车间开动之前，产品的”质”已经由数据定下一大半了。

把两半拼起来，是一个**多环相乘**的模型：

> 最终质量 = 需求正确性 × 数据质量 × 制造质量 × 使用质量
> 术语注：这个”多环相乘”模型是本书对质量成因的凝练。它把第 3 章的质量定义（ISO 9000:2026 §3.5.2：对象的一组固有特性满足要求的程度）落到开发环节，把”质量是设计出来的”变成可相乘的式子，不是任何标准的公式，欢迎检验、欢迎校准。

每一环都是乘法，任何一环为零，全盘皆输；这个模型回答了水忠的困惑：

数据质量决定产品质量的上限，不决定达成值。数据完美，最多把”能到的高度”抬到极限；但需求方向错了（验证全绿、确认全红），数据再好也没用；制造环节不达标，数据再准也复制不出好实物。

所以水忠说”质量在车间里”，对制造环节成立，对开发环节不成立：开发环节的质量，就是数据质量。这就是”质量是设计出来的，不是检验出来的”的真正含义——检验只能发现数据里的缺陷，缺陷早在数据里了。

把乘法链落到最具体的一环，一个常见误会就现形了：“合格物料是合格产品的前提”。严格逻辑上，它既不是充分条件，也不是对每一条要求的必要条件。

好料未必出好产品：装配错、工艺偏、设备磨，任何一环出错，好料照样出次品。反过来，个别物料超差，经让步接收（评审无害后放行）、规格过严（偏差无害）、或产品验收恰好测不到，产品仍可能合格。

必要的不是”每颗料都合格”，是”物料合格性被受控管理”：按接收准则验收、不合格不放行、让步接收留痕。这才是外部提供控制要管物料的理由（其条款落点在 ISO 9001:2015 §8.4，库内无该标准正本，属库外来源、不可核）。合格物料，是合格产品的基础，不是保证。

数据质量本身怎么评？ISO/IEC 25012 的数据质量模型分十五项特性（据 42030:2019 表 A.3 转述）。本章取最要紧的五把尺子：准确性、完整性、一致性、现时性、可信度。五把尺子全是拿”数据”当对象，再一次印证：开发在数据上工作。

> 出处说明：ISO/IEC 25012 在库内无正本；其数据质量模型的库内可达载体是 ISO/IEC/IEEE 42030:2019 表 A.3（Table A.3 — ISO/IEC 25012 Data quality model），另见 ISO/IEC/IEEE 42010:2022 正文引用（data qualities as described in ISO/IEC 25012:2008, Clause 4 …）。第五把尺的中译按 42030 中英文对照版作”**现时性**”（Currentness）；产品数据

> 研究报告（14 号）旧译作”时效性”，本章从定名。此处不写成”标准说……“。

五把尺子各举一个冰链的例子，就知道怎么用了：

| 特性 | 含义 | 冰链的例子 |
| --- | --- | --- |
| 准确性 | 数据反映真实对象 | 固件 v2.4 的记录，跟实际烧录的版本一致吗 |
| 完整性 | 数据覆盖无遗漏 | “边缘断链预判”的需求、设计、验证，都齐吗 |
| 一致性 | 数据内部不矛盾 | BOM 里的压缩机和装配图上的，是同一个型号吗 |
| 现时性 | 数据跟上现实 | 需求改了，共享盘里的文档更新了吗 |
| 可信度 | 数据经得起验证 | 验证报告是实测跑出来的，还是纸面写出来的 |

五把尺子一量，M3 二代的病就现形了：准确性和一致性，正是案例进展①里 BOM 与图纸对不上那件事。数据质量是能当场量的：这五把尺子一量一个准。

数据质量这五把尺子，正好呼应第 3 章的质量定义。数据质量，就是拿产品数据这组”固有特性”满足要求的程度；五把尺子是这个程度的具体化。第 3 章把质量钉在”程度”，这一章补一句：开发阶段，这个程度全靠产品数据来量。

还有一个机理必须钉死，缺陷传播链：需求错 → 设计错 → 制造错 → 使用错，越往下游修复成本越高（早期修是 1，使用期修是 30~70 倍；此为业界经验数字，转引自《需求工程概念研究报告》（02 号）§六，一手来源未核，读者可按自家数据校准）。这是乘法模型的必然，与惩罚无关。

## 概念卡片 · 两域三态

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ①⑥ | 产品在信息域、物理域与设计态、制造态、使用态中存在的模型；对立面：物理存在（实物）——数据管不着的那一半 |
| **它不是什么** | ⑦⑧ | 辨析：三态是同一产品的三种存在方式，不是三个产品；设计态同一性 ≠ 产品就是数据；误区：“质量在车间里”——开发环节的质量就是数据质量 |
| **它从哪来** | ②③⑨ | PLM 与 STEP 的数据管理实践；as-designed ← 设计、as-built ← 制造、as-maintained ← 维护；数字孪生即信息域对物理域的实时映射 |
| **它往哪去** | ④⑤ | 划定”开发产品 = 开发产品数据”的成立范围；多环相乘模型 |

### 7.3.6 沿工位流动的是产品数据

把产品开发看成一整条流水线，这一章的话就全串起来了。

流水线上有三样东西：**工件**（被加工的物）、**工位**（加工动作）、**增值**（加工效果）。制造流水线上，工件是钢铁零件，工位是机床，增值是零件越来越接近成品。产品开发的流水线上呢？

| 流水线要素 | 产品开发中的对应 |
| --- | --- |
| 工件 | 产品数据（需求规格、设计模型、验证证据……） |
| 工位 | 需求定义（转译）、设计（物化）、实现（实例化）、验证（取证）、移交（冻结） |
| 增值 | 每个工位往数据上添加价值，也可能添加缺陷 |

水忠在制造流水线上干了一辈子，一眼就懂这个模型。但他从没意识到：他的开发流水线上，流动的从来不是实物，是产品数据。需求数据流进设计，设计数据流进验证，验证证据流进基线，一路变成受控配置。

流水线视角立刻显出一个关键差异：制造流水线和开发流水线有三处本质区别。

第一，工件性质：制造是实物（复制），开发是信息（原创）；信息可以被复制一万次而不磨损，实物不行。

把第一点里的”工件”再往细里看：产品开发流水线的工件是产品数据，而设计开发这一站加工的具体对象，是规定需求。需求从利益相关方那里”进厂”，经分解分配加工成更详细的需求，一路传到验证当尺子。

它和制造流水线的物料最像，但有一个关键区别：物料进了成品就被消耗，需求在链条里始终有效。它既是每级加工的输入，又是逐级核对的标尺，更像图纸、不像原料。

第二，加工方式：制造是确定性复制（照图纸做），开发是创造性转换（把需求译成设计）。所以开发每个工位都有判断、有取舍、有人的专业在里面。

第三，缺陷行为，这是最要命的一条：制造缺陷在单件上，开发缺陷沿流动传播并放大。需求里的一个错误，流进设计、流进实现、流进验证，越往下游修起来越贵（上一节讲过的缺陷传播链，在这里沿流水线放大）。

制造流水线的次品可以挑出来扔掉；开发流水线的缺陷已经长进产品数据里，跟着数据一路传到每个下游。

这一节把整块串成一条线：开发产品 = 开发产品数据（定义生成），数据沿流水线流动增值，缺陷沿流动放大。第 8 章的三关在关键工位检查数据，第 13 章的基线在关键节点冻结数据——两条防线，都是站在流水线上、对产品数据设的关卡。

流水线视角还给第 13 章埋了个位置：配置管理，就是这条流水线上的”标识 + 基线 + 记实”，给每个工件发身份（标识）、在关键节点冻结（基线）、记录流动轨迹（记实）。没有它，数据沿流水线流到哪、哪一版是准的，全都说不清。这一章把”流动的是什么”讲清了，第 13 章把”流动的怎么管”接上。

这里的”流水线”和常说的”瀑布流程”不是一回事：瀑布是流程组织方式（阶段串行），流水线是数据流动视角（工件沿工位增值）。不

管跑瀑布还是敏捷，产品数据都沿工位流动。流水线视角看的是”数据在不在流动、有没有增值”，不是”流程怎么排”。

## 案例进展②｜“二代功能基线，还是对不上”

开完会一周，冰链开了第一次 M3 二代设计评审。

文正把六类数据铺在投影上，问水忠：“我们一件件过。需求数据——‘边缘断链预判’的验收标准，定了吗?”

水忠翻出需求文档，指着一行：“定了。传感器异常时，箱体本地判定并报警，30 秒内完成。”

“这条需求拆到哪几个模块了？每个模块的接口契约呢?”

“拆到固件和传感器两级。”水忠愣了一下，“接口契约……我们以前都是直接干，没写过。按第 6 章那套判据，需求数据的叶节点得在设计数据里有落点，可设计数据里没有这一条。”

李旭接过话：“那验证就做不了。验收标准在需求数据里，接口契约在设计数据里，这两层对不上，我的测试用例根本没法定——我测’本地判定 30 秒’，可我不知道是固件负责还是传感器负责，也不知道哪个接口给我触发信号。”

董勐也接了一句，他的云端追溯平台正好要存这条告警：“我这边一样断——箱子本地判完，告警还得回传云端落记录，追溯平台才知道那一刻发生了什么。接口契约没定，我不知道存哪个字段、由谁触发，这条追溯记录就写不全。”

会议室里安静了三秒。水忠第一次看到，他做了一辈子的事，原来是**一层一层的数据接力**：需求数据接住客户需要，设计数据接住需求数据，质量数据接住设计数据。任何一层断，产品就断。

“所以你说开发产品就是开发产品数据……”水忠慢慢说，“我明白了，不是补文档，是**把每一层的定义造全、造准，让下一层接得住**。”

“对，这就像一条流水线。”文正点头，“需求数据流进设计，变成设计数据；设计数据流进验证，变成验证证据。每一站都往数据上加价值，也可能加缺陷。第 13 章的基线就冻结在这条线上。你以前只盯着最后的实物，其实你该盯的是这条线上流动的数据。它哪一站断了，产品就在哪一站断。”

水忠看了看投影上那六类数据，忽然觉得，自己干了几十年的东西，第一次有了一张完整的图。

---

## 7.4 产品数据怎么用

> 这一块看产品数据往哪去：先看它是组织协作的对象，再看它能装下多少价值，然后把职责挂到工件上，最后看它变成企业资产——产品会死，数据体系活着。

### 7.4.1 利益相关方围绕产品数据协作

产品数据不是开发部自己的事：协作落到工程里，就是一张名单，谁读什么、谁写什么。

说”协作”之前，先把参与者钉一钉。ISO/IEC/IEEE 15288:2023 §3.44 给利益相关方（stakeholder）的定义是：

> **stakeholder**: individual or organization having a right, share, claim, or interest in a system or in its possession of characteristics that meet their needs and

> expectations——对系统、或对其具备的满足自身需要和期望的特性享有权利、份额、主张或利益的个人或组织。

用户和客户都在其中，他们是 needs and expectations 的所有者。

内部利益相关方（市场、研发、制造、质量、采购、项目），是围绕产品数据**读写协作**的人。大家不围绕”目标”协作，也不围绕”计划”协作，而是围绕**同一份产品数据**协作。

把内部利益相关方的读写列细一点，就是一张协作分工表：

| 利益相关方 | 读 | 写 |
| --- | --- | --- |
| 市场 | 需求数据、使用数据 | 洞察、需求 |
| 研发 | 需求数据、接口规范 | 设计数据、实现产物 |
| 制造 | 设计数据、工艺 | 制造数据、BOM |
| 质量 | 设计数据、验证计划 | 验证证据、缺陷 |
| 采购 | BOM | 供应商、物料数据 |
| 项目 | 全部 | 进度、管理数据 |

每一格都是一份产品数据的读写。协作的真相就藏在这张表的每个格子里：水忠以前觉得”协作 = 开会”，其实协作是数据的读写，会议是数据对齐的形式，数据才是协作的实质。

外部利益相关方（用户、客户），是产品数据的源与宿：数据从这里进来，价值在这里兑现。

用户是使用产品的人（使用价值、验收裁判）；客户是接收产品或服务的**个人或组织**（交易价值、需求源头，ISO 9000:2026 §3.9.1；该条 EXAMPLE 里的 purchaser 即”购买者”一例）。买奶粉的是客户，喝奶粉的是用户——一句口诀，两个角色。

> 术语注：“客户”是本书行文用词；ISO 9000:2026 §3.9.1 的中译定名为”**顾客**”，两词所指同一（该定义句也含”个人**或组织**”，不止自然人）。本书各章统一用”客户”，读者读标准时按”顾客”对接。

用户与客户通过四条路径贡献并提升产品数据：

| 路径 | 机制 | 产物 |
| --- | --- | --- |
| 需要（源头） | 痛点与期望经代理转译 | 需求数据（规定需求） |
| 使用（新数据） | 遥测、日志、行为 | 使用数据（使用态的一部分） |
| 反馈（修正） | 缺陷报告、变更请求 | 变更与缺陷数据 |
| 验收（把关） | 确认”是否满足需要” | 校正需求数据 |

有一条精确的表述：产品数据的内容由用户与客户给出，形式由内部利益相关方写出。除了”使用数据”是用户直接产生的，其余三条都是间接但根本的贡献。

水忠现在能看懂冰链的业务了：疾控中心（客户）的审计需求，是需求数据的源头；冷链车上的箱子（使用），每小时吐回温度实况，是使用数据；现场反馈的故障，是缺陷数据。用户与客户不写答卷，但出题和判卷都是他们。

不过这条闭环有两处必须标清的边界。

第一，需求数据链只承载”明示”的部分。从需要到规定需求，是一条按派生关系生成的链，它只承载 stated 的部分（ISO 9000:2026 §3.5.1 注 2，规定的需求是明示的需求）；而确认的判据与方针不在链上：它们不是从需要派生出来的，须另设一层与派生链异构的参照。（第 5 章、第 8 章分别展开。）

第二，转译有损耗。期望的集合严格大于需求的集合：利益相关方说的、写下来的、下游读到的，逐级都会掉东西。所以”需要进来了”不等于”需求就对了”。

把三个角色放进去收个尾：产品数据是大家对齐的那一份东西，评审是当场对表，配置管理是版本把关。评审的人不自己动手改数据，但所有人对着同一份数据对齐；配置管理不参与内容判定，但确保没人看错版本、改错条目。人可以换，那份数据不变，这就是产品数据作为协作对象的意义：它让组织不依赖任何一个人的记忆。

把四条路径和内部利益相关方的读写合起来，就是一个完整的产品数据闭环：需要进来（需求数据）→ 开发转译（设计数据）→ 验证取证（质量数据）→ 交付使用（实物 + 实况）→ 反馈回流（使用与缺陷数据）→ 反哺下一代。闭环每转一圈，产品数据就厚一层。“数据体系活着”，正因为它不是静态仓库，而是一直在转的闭环。

### 7.4.2 数据能装下多少价值

上一节说产品数据是协作的对象，有人会接一句：“那产品数据的价值，就是全部价值?”不是。这里要磨一道更细的边。

利益相关方对产品的贡献，沉淀下来分三层：

| 层 | 内容 | 是否在产品数据中 | 如何评价 |
| --- | --- | --- | --- |
| 物化层 | 需求、架构描述、实现工件 | 完全在 | 可证伪、可度量 |
| 决策层 | 取舍、转译、选型判断 | 部分沉淀（ADR） | 评审（事中）+ 结果回溯（事后） |
| 过程层 | 沟通、反馈、评审、影响力 | 结果记入 | 下游指标 + 复盘 |

前两层好懂。**决策层**是重点：架构师拍板”用微内核”，这个判断本身不在产品数据里，它的结果（架构描述、ADR）沉淀下来了，但判断的”为什么”只留了一半。**过程层**更飘：一次评审会上谁说服了谁、谁的影响力推着方向走，这些根本没有落成数据，只留下”评审记录”这一行。

所以三层模型给出一个重要的修正：**产品数据是价值的”投影与锚”，不是价值本体**。

工件是判断的投影：一份完美的设计文档，可能是错误决策的产物（验证全绿、确认全红，第 8 章会细讲）。

这个修正不是否定产品数据，而是给它定位：数据是锚，让人能评价、能追溯、能问责；但价值真正的源头，是人的判断与协作。水忠的价值不在他画的那张图里，而在画图时的判断里；可没有那张图，他的判断就无处锚定。

顺着这个定位，评价一个角色就有了完整式子：角色价值 = 工件质量 × 判断质量 × 下游影响（乘数）。前两项好理解；第三项最容易

被忽略——架构师的一个好决策，让下游一百个工程师少走弯路，他的价值体现在别人的工件质量上。

> 术语注：这个式子（工件质量 × 判断质量 × 下游影响）是本书给出的评价口径，不是任何绩效标准的公式。它是”价值三层”的操作化，作用是把”角色价值”从印象变成可核对的量纲。

顺着这三层，有一句话值得记住：隐性知识写不进数据（Polanyi《个人知识》〔库外来源·不可核〕：“我们知道的多于我们能说出的”）。决策层与过程层的价值，能沉淀进数据的只有一小半。

所以评价一个角色，要三维一起量：只量物化层（工件），会漏掉决策与过程的价值；只夸判断，又没有锚点可考；而三维的落点，全在产品数据上（§7.4.3 那张表）。

### 7.4.3 职责为什么挂在工件上

职责可以有不止一个落点，但能被问责的只有一个。

两种候选：挂在**流程**上（角色 × 活动），还是挂在**产品数据**上（角色 × 工件）；答案是后者，三条理由：

**数据职责可证伪**——工件可以对照规定需求检查，对不对一目了然；活动无法证实也无法证伪。

数据职责可映射——权限、签字、审批链、考核指标，四样硬机制都能挂到工件上；流程没有硬机制。

数据职责可度量：缺陷率、覆盖率、追溯完整率，都拿数据说话；流程只能评执行率（形式化）。

反证更锋利：缺了流程，数据职责还在（降级可运行）；缺了数据，流程职责空转（数据无人负责）。工程佐证随手可得：GitHub 的 CODEOWNERS，就是把”每份代码谁负责”写进数据。

所以职责的完整定义是：角色 ×（活动 × 工件），而锚点在工件，不在活动。落地三步法：

1. 识别工件——列出配置项清单：需求规格、架构描述、设计模型、验证证据、基线；
2. 指定 owner——每个配置项唯一 owner（一件只一人，一人可多件）；
3. 挂机制——工具权限 + 评审签字 + 变更审批链 + 考核指标，全部挂到 owner 上。

职责还有三种类型，别都压成”负责”：

| 职责类型 | 依据 | RACI 对应 |
| --- | --- | --- |
| 所有权职责（主） | 产品数据 / 配置项 | A（Accountable） |
| 执行职责（辅） | 流程 / 活动 | R（Responsible） |
| 决策职责（辅） | 评审门 / 基线 | Approve |

三类职责里，所有权是主——挂在产品数据上，可问责、可考核；执行和决策是辅，挂在流程和门上。

一句口诀分清主次：以产品数据为主（操作），它是唯一可问责、可考核、可改进的通道；以判断与协作为本（价值），那是价值真正的来源。主是管道，本是源头。

把”以产品数据为主”落到冰链的三个角色上，职责就活了：系统架构师、系统工程师、模块工程师，各按工件所有权分层（其责任矩阵的原始出处是工程角色谱系研究报告，13 号 §7）：

| 角色 | 拥有（工件） | 决策 |
| --- | --- | --- |
| 系统架构师 | 系统目的或任务理解确认单、系统基本概念和基本属性清单（入场券，§18.4.6）+ 架构描述、决定清单（ADR）、管控原则、系统接口契约、系统级验证方案 | 架构基线、技术路线 |
| 系统工程师 | ConOps、规定需求与追溯矩阵、八方案、系统集成与缝合清单、验证与确认报告、变更记录 | 需求基线、验证放行 |
| 模块工程师 | 模块详细定义、实现物、单元测试记录、版本与基线记录 | 组件变更 |

模块工程师在其模块内还兼模块架构师（后记四角色）。**架构不止系统有——模块也有**（42010:2022 把被架对象由 system 扩为 **entity**：模块既是一个系统元素，也可以是一个被架实体），对应的角色就是模块架构师，通常即模块工程师自己或他的团队：模块内部那组定义（模块框架定义、内部结构、内接口、模块级准则）归它，模块级实现归模块工程师。

三条协作规则跟着工件走。约束向下：架构师定**本质与管控原则**、系统工程师定义”语义”，模块工程师在约束里实现；反馈向上：模块工程师发现偏差反馈系统工程师、再反馈架构师；追溯贯穿：追溯矩阵是三方共同维护的唯一工件。

一句话：架构师管”**立得住**”（本质与管控原则），系统工程师管”要得对”（语义），模块工程师管”做得对”（细节）；各管一段产品数据，各负其责。第 9 章（模块工程师）和第 18 章（系统架构师）会把这套分工各自展开。

再各钉一句正确履职：架构师定义边界与接口、用 ADR 记录决策理由；系统工程师转译不丢原意、维护追溯矩阵、需求可测试才放行；模块工程师遵循架构约束、实现与设计一致（偏差必走变更）、单元验证自证。

划责规则一句话，一个缺陷只定位一个责任工件——需求缺陷找系统工程师，**本质、管控原则或接口契约的缺陷找架构师**，实现缺陷找模块工程师。数据在哪，责任就在哪；数据有 owner，缺陷就有归处。

顺着三维的尺子，把三个角色的价值各量一遍，就是下表：

| 角色 | 工件质量（直接） | 判断质量 | 下游影响（乘数） |
| --- | --- | --- | --- |
| 架构师 | 架构评审缺陷率 | 评审质疑解决率、路线复盘 | 下游架构偏差率 |
| 系统工程师 | 需求缺陷密度、追溯完整率 | 转译正确性复盘 | 下游返工率 |
| 工程师 | 实现缺陷率、单元验证通过率 | 设计评审复盘 | 集成缺陷率 |

缺陷率、追溯完整率、返工率，一查数据就知道；数据是评价的锚点，不靠印象、不靠人情。

### 概念卡片 · 价值三层

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ①⑥ | 利益相关方对产品贡献价值的沉淀层（物化、决策、过程三层）；数据是价值的”投影与锚”；对立面：价值本体（人的判断与协作）——数据管不到的源头 |
| **它不是什么** | ⑦⑧ | 辨析：三层不等于全部价值（决策层与过程层只部分沉淀）；误区：“数据全 = 价值全”——完美工件可能是错误决策的产物 |
| **它从哪来** | ②③⑨ | 源于”工件是判断的投影”；Polanyi《个人知识》〔库外来源·不可核〕“我们知道的多于我们能说出的”；ADR 即决策层的部分沉淀 |
| **它往哪去** | ④⑤ | 让产品数据可评价、可追溯、可问责；把职责挂到工件上；增 |

## 案例进展③｜恒信的”上线了，但数据没人管”

冰链在磨数据，恒信那边也没闲着，只是病不同。

恒信的合同审批系统上线半年，万辉志得意满：系统跑起来了，UAT 也过了；可这天，徐漾拉他开了个会。

“万辉，你回答我三个问题。”徐漾摊开一张纸，“三个问题都不问系统跑没跑起来，只问数据在不在、归谁管。第一，‘25 号之后签的合同算哪个月’——这条规则，现在在系统里，还是在谁的脑子里？第二，上线后改过四次审批流，每一次谁改的、为什么改，有没有记录？第三，下个月玉杰调走了，这套系统的设计逻辑，他从哪交接给新人？”

万辉答不上来。系统”上线了”，可系统的**数据**——规则、变更、设计逻辑——散在开发部每个人的脑子里和各自的文件里。

“这就是产品数据。”徐漾说，“你以为开发完了，其实数据没归位。软件的产品数据就是产品本身——你的需求、设计、规则、变更，都在这套数据里。它没人管，等于系统没人管，只是’跑起来’骗过了所有人。”

恒信的问题，和冰链隔着一整个行业，却撞在同一棵树上：**产品数据的完整性**，才是产品站不站得住的根本。

万辉沉默了一会儿：“我一直以为’上线’是终点，代码跑起来就成功了。现在才知道，上线只是把数据从开发部交出去，数据有没有归位、有没有人接，才是系统活不活得下去的根。”

徐漾点头：“这就是这一章说的——开发产品就是开发产品数据；产品会死，数据体系活着。恒信的合同审批系统会迭代、会退役，但那套规则、决策、变更记录的数据体系，才是恒信真正留下的东西。”

### 7.4.4 活动值不值：产品数据增值判据

组织里每天一大堆活动，有些忙完一圈，产品数据一动没动。

有一条可操作的口径：**看这个活动有没有让产品数据增值**。评审让需求数据更准，增值；改设计让图纸更对，增值。可有些活动忙了一圈，产品数据没动，就要追问一句”值不值”。

但要给”增值判据”划三条边，否则会误伤：

**第一，增值的内容必须顾客需要**。过度加工也是”往数据上加东西”：加了一堆顾客不要的额外特性，数据是多了，价值没涨。贡献了数据 ≠ 有价值，还得顾客需要（呼应第 5 章”可实现性需求客户不关心”）。

**第二，不增值不等于浪费，状态改变也是增值**。增值有两种：内容增值，是数据本身更准更对（评审让需求更准、改设计让图纸更对）；状态增值，是内容没变、状态变了（审批让草稿变受控、合规打上标注使数据满足规则且证据可追溯）。只走流程、数据既没变好也没变状态的活动，才是必要非增值，要压缩保留，不能一刀砍。

第三，判据要双维：既要看工件增量（数据有没有变好），也要看流动效率（数据有没有卡住）。等待、交接这些不直接动数据的环节，恰恰是要打击的核心。

把这个判据画成一张判定树，一个活动落下来就能落进一格：

| 活动 | 让数据增值? | 顾客需要? | 结论 |
| --- | --- | --- | --- |
| 完善需求 | 是 · 内容 | 是 | 增值 · 优化 |
| 加顾客不要的特性 | 是 · 内容 | 否 | 过度加工 · 浪费 |
| 审批草稿转受控 | 是 · 状态 | — | 状态增值 · 必要 |
| 合规打标注满足规则 | 是 · 状态 | — | 状态增值 · 必要 |
| 重复造轮子 | 否 | — | 纯浪费 · 消除 |

用这张表看冰链：水忠花三个月优化一个没人用的参数（过度加工）、李旭重复测已测过的用例（重新学习）、审批卡了两周（等待），全是”产品数据没增值”的活动。产品数据增值判据，是把精益落到研发里最可操作的检视点。

精益研发把这套口径落成一张表，研发语境下的七大浪费：

| 浪费 | 研发语境含义 |
| --- | --- |
| 部分完成的工作 | 半成品工件堆积（没冻结的需求、写了一半的设计） |
| 额外特性 | 过度加工——贡献了数据但顾客不要 |
| 重新学习 | 同样的知识重复获取（没有沉淀为产品数据） |
| 交接 | 信息在角色间传递的损耗 |
| 任务切换 | 上下文切换的成本 |
| 等待 | 信息阻塞（等评审、等决策、等数据） |
| 缺陷 | 错误数据流入下游并放大 |

注意看，这张表里的”额外特性”和”重新学习”两行，正是本节增值判据在精益里的两个反面教材。

增值还有一个要点：数据增值不只是变好，更是流动起来。一条需求写完躺在文档里，不算增值；被设计接住、被验证测过、被基线

冻结，才算活了；等审批、等决策、等数据，都是流动的停滞，也是价值流失。

### 7.4.5 产品会死，数据体系活着

产品数据的最后一站，是企业的**账本**。

什么是资产？至少满足四条，**可积累、可复用、可转移、可评价**；拿这四条检一遍候选：

| 候选资产 | 可积累 | 可复用 | 可转移 | 可评价 |
| --- | --- | --- | --- | --- |
| 产品数据 | √ | √ | √ | √ |
| 人才（判断、协作） | × | × | × | △ |
| 客户关系 | △ | △ | × | △ |
| 流程文档 | △ | √ | √ | △ |

**能同时满足这四条的，只有产品数据**。人才是价值的源泉，但不是资产的形态：人是本，数据是产；资产形态只能是数据。

> 术语注：这里的四条（可积累、可复用、可转移、可评价）是本书判断”价值载体能否沉淀为资产”的操作化口径，不是会计或标准对”资产”的定义（会计口径强调”拥有或控制 + 预期经济利益”）。本书用它比的是产品数据与人才、客户关系的资产性差异。

更扎心的一句话：产品会死，数据体系活着。单个产品会过时、会退市，它是资产的输出，不是资产；真正的资产，是能跨产品、跨代际复用的产品数据体系：需求知识、架构模式、验证基线、配置基线。

水忠做了几十年冷链温控，真正留下来的不是那些报废的箱子，而是几十年攒下的那套数据：哪一版固件采的、那个坑怎么趟过去的、那条需求为什么这么定。

四类数据资产，是研发企业真正攒下来的东西：需求资产（知道该做什么）、设计架构资产（知道怎么做得好）、验证资产（知道怎么证明对）、管理资产（知道为什么这么做）。这才是配置管理、PLM 存在的终极理由。

为什么抄不走？因为追溯链与决策史是一点点攒出来的，不是一次能补的。竞争对手可以买你的图纸，买不到你十年里”为什么这么定”的决策史；可以挖你的工程师，挖不走”这批数据哪一版是对的”的追溯链。产品数据是一年年攒出来的：你今天攒下的每一份决策记录，别人明天都补不上。

这也是”产品会死，数据体系活着”的另一半：这四类数据资产，是跨产品、跨代际复用的。水忠做 M3 二代，用的正是 M3 一代攒下的需求知识、架构模式、验证基线；没有那套数据体系，二代等于从零开始。

从架构的视角再看一眼，这一半可以更尖：产品数据凭什么跨代际复用？因为产品数据，是产品架构的表达。架构本身没有别的持久化载体，只能活在产品数据里。“全程可追溯、断链必告警”这条不变量，靠追溯数据结构与告警接口这些产品数据把它写死。

所以单个产品会死，产品架构却靠产品数据跨代际活着。

这正是 §7.2.5 说的那棵树：干枝叶节点是数据，“为什么这么定”的根，也是数据；也正是架构师这份角色的日常——把架构写下来、落成产品数据（第 18 章 §18.4.6 的文档族）。同一件事的三种说法。

## 案例进展④｜十年后，谁还记得怎么做箱子

水忠的师傅退休那年，把一本手写的本子交给水忠，里面是他干了一辈子温控箱积累的”为什么”：这条工艺为什么这么定、那个故障当年怎么解的、哪个供应商的料出过事。

水忠接过来，翻了几页，问：“师傅，这本子传给谁?”

“传给你，你以后再传给李旭。”

“本子会被弄丢的。”水忠说，“上个月逸人的工艺卡就找不回来。那上面的校准参数本来就是产品数据，只是它没进系统，也就没进资产。”

逸人在产线那头接话：“那是我最疼的一张卡——断链检测的校准参数、老化试验的判定线，全在上面。丢的不是一张纸，是几条工艺线怎么校准的’为什么’。老师傅走了，新来的按剩下的卡试，差半度都试不出来。”

旁边的文正听到，补了一句：“所以产品数据要进系统、进资产，不是靠师傅的本子，是靠组织的数据库。需求、设计、验证、配置，每一件都归档、都追溯、都可查。十年后师傅不在了，做 M3 的方法还在；产品退市了，做下一代的底子还在。”

“这就是你说的’产品会死，数据体系活着’?”水忠问。

“对。人会把记忆带走，数据不会。只要你把它当成资产，年年往里存、年年往外用。”

---

### 7.4.6 数字化不等于转型

最后一站，看产品数据在组织里怎么被”用起来”。这常常被叫成数字化转型。

先拆词。**数字化（digitization）**是把纸变成电：模拟变数字，结果是被**存档**（档案化）。**转型（transformation）**是让数据变成价值：从”被管理”变成”被使用”，改变工作方式。

一句口诀：数字化是把数据存起来，转型是让数据用起来。

很多企业把第一步当成全部：数据存进系统就宣布”转完了”。其实那只是存起来，数据没被用起来。真正的转型，是让产品数据从”被管理的档案”变成”驱动决策的依据”：需求数据驱动下一个产品，缺陷数据驱动质量改进，配置数据驱动每一次变更决策。

数字化转型不止”产品数据数字化”一条，它通常有五条战线，但主战场是产品数据：

| 维度 | 内容 | 与产品数据的关系 |
| --- | --- | --- |
| 产品数据数字化 | 最重要资产的数字化 | 主战场 |
| 流程数字化 | 评审、变更线上化 | 让产品数据流动起来 |
| 决策数字化 | 数据驱动的决策 | 让产品数据被用起来 |
| 产品智能化 | 智能硬件、SaaS | 产生新的使用数据 |
| 商业模式数字化 | 订阅、在线服务 | 让产品数据变成服务 |

五条线都往产品数据汇聚，所以它是主战场：数字化要让它活起来。

还有一个常见的坑：把”买平台”当”做转型”。上 ERP、上 PLM、上数据中台，买的是工具，不是转型。工具存数据，转型用数据。水忠如果听到”我们上了 PLM 就转完型了”，可以直接反问一句：“那数据里跑出来什么决策了吗?”答不上来，就是只存起来了、没被用起来。

### 7.4.7 认知反转

一个月后，冰链 M3 二代的第一轮验证会。

水忠打开的，不再是墙上那堆图纸，而是那套数据：需求基线、接口契约、验证证据、追溯矩阵。每一片叶节点都有编号、有来路、有验证记录。李旭指着”边缘断链预判”的验证报告：“本地判定 30 秒，实测 28 秒，两条边界用例覆盖。”

水忠对文正说：“我第一次觉得，产品是我’造’出来的——每一层数据，我都接得住。”

回到章首那场会。你以为开发产品是把实物做出来、数据是开发完再补的文档；其实开发全程没有一件实物可摸——你造的是产品数据，实物只是它的复制品。这一章的反转，是数据质量决定产品质量的上限、不决定达成；产品数据是协作的对象、职责的落点，也是企业最重要的资产——产品会死，数据体系活着。

这些事背后，是同一次方法时刻，第 1 章说的织关系。水忠原来把需求、图纸、报告当一堆散件（堆）；现在把它们织成树、织成链（结构）：数据不是文档堆，是产品唯一的信息形态；第 6 章把方案织成树，这一章告诉你那棵树就是产品数据。

用第 1 章的话收一次束：对”产品数据”“两域三态”“数据质量”“价值三层”这些词，你原来只是”见过”，知道它们重要；这一章把它们推到”拥有”：能答全四问，也就是它是什么、不是什么、从哪来、往哪去。能答全四问才算拥有——你会说出产品有两个存在，开发的就是数据。

> 开发产品就是开发产品数据：产品是价值的载体，产品数据是价值沉淀的唯一形态。

用第 1 章 §1.4.4 的尺子量一下：这一章给的不只是第 ② 级（图式），还落了”外化地图”这一步，产品数据就是那张从个人脑子里搬出来、变成组织共用的图。

## 小结

这一章讲的是”开发的产品到底是什么”：不是实物，是**产品数据**，也就是产品的信息存在。

我们从三个角度把它看透。**它是什么**：产品有两个存在，开发造的是信息存在；六类数据覆盖全程；实现方案是最重的一类；数据不是文档、不是配置项、不是产品、也不是架构本身。**开发产品 = 开发产品数据**：一次定义多次复制、两域三态划出设计态；数据质量决定上限不决定达成；流水线上流动的是产品数据，缺陷沿流动放大。它怎么用：协作的对象、价值的投影与锚、职责的落点（按工件所有权分工）、增值判据、企业的资产。产品会死，数据体系活着；数字化是存起来，转型是用起来。

第 6 章你种下了一棵树（方案是树），这一章告诉你那棵树就是产品数据。现在这棵树站起来了，可它还要被人检查、被人守护、被人一路送到上线运维。

下一章，三个看门人上场——评审、验证、确认，查的正是这棵树。评审对目标、验证对需求、确认对用途；再往后，工程师（第 9 章）种下半个棵树，架构师（第 18 章）种上半个；配置管理（第 13

章）让它一直在变还说得清；整合（第 17 章）让它沿变更闭环流到上线运维。

散会时，文正想起自己宣布”产品成功”那天，手里连一半这样的数据都没有。他今天点破了别人，也认了自己那一笔欠账。

# 附录

## A 本章概念总图

产品数据：产品的信息存在（开发工作的全部对象）

两个存在：物理存在（实物）/ 信息存在（产品数据）——实物是数据的复制品

产品 = 需求的物化 + 价值的载体 + 过程的输出；≠ 服务 / 系统 / 项目 / 结果

六类数据：需求 / 设计 / 制造 / 质量 / 服务 / 管理

边界：≠ 文档（数据是内容，文档是容器）；≠ 配置项（受控的才是）；≠ 产品；≠ 架构本身（可冻结的是架构描述 AD）

实现方案 = 八方案（技术/物料采购/制造/测试/交付/部署/操作/运维）——最重的一类

该有的样子：可定位、可承接、可追溯

开发产品 = 开发产品数据（设计态同一性）

产品开发：定义与复制；跨职能协同；以评审为门；基线 = 一组受控的产品数据

两域三态：信息域 / 物理域；设计态（纯数据）/ 制造态（数据+实物）/ 使用态（实物+实况）

严格说：开发 = 定义生成；软件：开发与制造合成一步

数据质量决定上限，不决定达成：最终质量 = 需求正确性 × 数据质量 × 制造

质量 × 使用质量

流水线：流动的是产品数据；缺陷沿流动传播并放大（越下游修复越贵）

产品数据怎么用

协作: 内部利益相关方读写协作; 用户/客户是源与宿（出题人判卷人）；需求链只承载 stated，转译有损耗

价值三层：物化 / 决策 / 过程—数据是投影与锚，不是本体

职责: 挂在工件上（工件所有权 + 系统架构师 / 系统工程师 / 模块工程师），以判断与协作为本

增值判据：活动是否让数据增值—内容增值（顾客需要 × 流动效率）+ 状态增值（草稿转受控、合规标注）；不增值 ≠ 浪费

企业资产：可积累 / 可复用 / 可转移 / 可评价四条兼备；产品会死，数据体系活着（需求 / 设计架构 / 验证 / 管理四类资产）

数字化 vs 转型: 存起来 vs 用起来

前向：配置管理（第13章）= 操作；整合（第17章）= 流动；工程师/架构师（第9/18章）= 角色侧

## B 本章的能力级

**第 ② 级为主**

- **表征**:产品数据、两域三态、价值三层、设计态同一性、实现方案、数据质量的准确性 / 完整性 / 一致性 / 现时性 / 可信度、流水线——这些词你现在能答全四问；并分得开产品 / 产品数据 / 配置项 / 文档 / 架构描述。

- **图式**：产品的两个存在与”实物是信息存在的复制品”；两域三态（设计态 = 纯数据）；六类数据覆盖全程；“开发 = 定义生成”的成立范围；多环相乘（需求正确性 × 数据质量 × 制造质量 × 使用质量）。
- 心智模型：数据质量决定上限、不决定达成；缺陷沿流水线传播放大；价值三层与”数据是投影与锚、不是本体”；职责为什么挂在工件上；资产检验（可积累、可复用、可转移、可评价）与”产品会死、数据体系活着”。
- 解释框架：本章不建此级，见第 17、18 章。

> 能答全四问，才算拥有；你会说出产品有两个存在，开发的就是数据。

## C 自查 · 这些说法对吗？

1. “产品就是实物，数据是文档，开发完再补。”——**错**。产品有两个存在；开发全程在信息存在上工作，数据不是事后补的文档。
2. “开发产品，开发的是产品数据。”——**对**。设计态同一性；严格说是”开发 = 定义生成”。
3. “配置项就是全部产品数据。”——**错**。配置项是受控的那部分（被基线化、被变更控制），草稿不是。
4. “产品数据就是文档换个说法。”——错。数据是内容，文档是容器；一个容器可以装多份数据。

5. “产品数据就是架构。”——错。**架构没有形态、交不出去**；可交付、可冻结基线的是架构描述（AD）。
6. “数据质量决定最终产品质量。”——错。决定上限，不决定达成；质量 = 需求 × 数据 × 制造 × 使用，多环相乘。
7. “需求方向错了，数据再好产品也错。”——对。需求正确性决定方向；验证全绿、确认全红。
8. “用户与客户只负责提需求，不碰产品数据。”——错。他们是源与宿：需要进来、使用产出、反馈修正、验收把关。
9. “职责挂在流程上，比挂在产品数据上更清楚。”——错。数据职责可证伪、可映射、可度量；缺数据流程空转。
10. “产品数据是企业最重要的资产。”——对。能同时满足这四条的只有产品数据；人是源泉（本），数据是资产（产）。
11. “软件没有独立的制造环节，开发就是生产。”——对。定义与实物合成一步；软件迭代快，因为直接改定义本身。
12. “质量是检验出来的，不是设计出来的。”——错。开发环节的质量就是数据质量；检验只能发现数据里的缺陷。
13. “实现方案只是产品数据里普通的一份。”——错。它是产品数据里最重的一类——制造完全照着它复制，错一处，一万台箱子错一万处。

14. “开发流水线上流动的是实物零件。”——错。流动的是产品数据（需求数据 → 设计数据 → 验证证据 → 基线）；实物只是设计态数据被复制的产物。
15. “往产品数据上加东西就是增值。”——错。增值还要顾客需要；加顾客不要的特性是过度加工（浪费）。
16. “数字化就是把产品数据存进系统，就算转完型了。”——错。数字化是存起来，转型是用起来、驱动决策。

## D 练习

1. 列出你们产品的六类数据（需求 / 设计 / 制造 / 质量 / 服务 / 管理），每类举两个具体工件。
2. 找出你们团队一个”把产品数据当文档、开发完再补”的例子，说说它造成了什么后果。
3. 用两域三态画你们产品的信息域与物理域：三种状态各有什么数据，两域靠什么映射？
4. 用多环相乘模型分析一次你们的质量事故：哪一环断了？数据质量在中间起了什么作用？
5. 用 ISO/IEC 25012 的五个特性（准确性 / 完整性 / 一致性 / 现时性 / 可信度）检查你们的一份产品数据，逐项给证据。
6. 你们产品的”出题人”和”判卷人”是谁？他们通过哪条路径（需要 / 使用 / 反馈 / 验收）影响产品数据？

7. 用价值三层分析一个你熟悉的产品决策：哪些沉淀进了产品数据，哪些没有？漏掉的那层，后来怎么补？
8. 用职责落地三步法（识别工件 → 指定 owner → 挂机制）给你们的核心配置项各指定一个唯一 owner。
9. 回答”产品会死，数据体系活着”：你们企业最值钱的数据资产是哪几类？能不能跨产品复用？
10. 列出你们实物产品的实现方案八个方案（技术 / 物料采购 / 制造 / 测试 / 交付 / 部署 / 操作 / 运维），标出哪一个最容易漏、为什么。
11. 画你们开发流水线的工位与工件：数据在哪些工位流动？找一条”缺陷沿流动放大”的真实例子，说清它从哪一环开始、最后贵了多少倍。
12. 用产品数据增值判据评价你上周做的三件事：哪件增值、哪件过度加工、哪件必要非增值、哪件纯浪费？

（答案要点见书末附录，与训练教材题卡同源）

## E 参考文献

**标准** - R-103 ISO 9000:2026（第 5 版） —— §3.3.11 设计开发、§3.5.1 需求、§3.5.2 质量、§3.5.3 对象、§3.7.1 结果、§3.7.9 产品、§3.7.10 服务、§3.8.1 数据、§3.8.4 信息、§3.9.1 顾客、§3.10.1 特性、§3.10.10 配置信息、§3.10.12 配置基线（产品三视角 / 数据—信息—对象链 / 用户与客户 / 开发＝定义与复制） - R-03 ISO 9001:2015 —— §8.4 外部提供（物料合格性受控管理的条款落

点：按接收准则验收、不合格不放行、让步接收留痕）〔库外来源·不可核〕- R-04 ISO/IEC/IEEE 42010:2022 —— 架构描述（设计数据的一类）- R-102 ISO/IEC/IEEE 15288:2023 —— §3.6 人工制品、§3.32 产品、§3.44 利益相关方、§6.4.4 系统架构定义过程、§6.4.5 设计定义过程（系统生存周期过程）- R-102 已含 §3.32 产品与 §3.6 人工制品两条（前者与 ISO 9000:2026 §3.7.9 同文）；本条原引 ISO/IEC/IEEE 24765”产品 = 由过程产出的工件”——库内无正本，且该主张已可由 15288:2023 §3.6 一手承住，故不以 24765 为据 - R-71 ISO 10303-1:2021（STEP）—— §3.1.29 产品数据的定义〔库内无正本〕- R-72 ISO/IEC 25012 —— 数据质量模型（本章取五把尺子：准确性 / 完整性 / 一致性 / 现时性 / 可信度）〔库内无正本；经 ISO/IEC/IEEE 42030:2019 表 A.3 可达〕 - R-73 PMBOK® Guide —— 项目 = 创造独特产品、服务或成果的临时性工作（产品与项目辨析）〔库外来源·不可核〕

**经典文献** - R-74 Polanyi《个人知识》/ 隐性知识 —— “我们知道的多于我们能说出的”（价值三层·决策与过程层的不完整沉淀）〔库外来源·不可核〕

> 对照阅读：本章”产品数据”呼应第 6 章”需求与方案结对”（树 = 产品数据的结构）、第 8 章”评审、验证与确认”（查这棵树）、

> 第 13 章”配置管理”（操作这棵树）、第 17 章”整合”（流动的工件）、第 9 章与第 18 章（工程师、架构师 = 角色侧）。

# 第 8 章 评审、验证与确认：目标、需求、用途

## 8.1 引言

第 7 章把那棵树立了起来：开发产品就是开发产品数据，你亲手造的是那棵树，不是实物。树立起来了，接下来是：谁来查它，拿什么当尺子，查完放不放行。

这一章回答这一问。它做三件事：把评审、验证、确认这三个天天被混着用的词分开；把它们的边界一条条画开；再看它们在一份需求从生到用的一生里，各站在哪儿。

### 8.1.1 误解现场

恒信集团合同审批系统上线前的评审会，周三下午两点，会议室坐满了人。

**系统架构师**万辉在投影上打开 UAT（用户验收测试）报告，语气笃定：

“UAT 全部通过。用例通过率 96%，P0/P1 缺陷清零，三条业务主流程全部走通。我们验收合格，可以上线。”

产品开发经理满海已经把报告翻到最后一页的签字栏，笔尖悬在上面：“UAT 全绿，业务方也点了测试环境——验收合格，这个字我签了。”

会议室里有人开始收东西了。财务对接的**黄程**却没动，他盯着报告翻了两页，问了一句：

“你们让我们点的那个测试环境，我们月底对账的场景——25 号之后签的合同算哪个月——测了吗?”

万辉愣了一下:“测试用例是照着需求清单写的，需求清单里……没有这条。这是上一轮口头补的需求，还没排进用例。”

“没排进用例?”**黄程**皱起眉，“那我们月底对账靠什么?”

法务的人也放下手机:“等一下。上线前那条’电子签章合同也走审批流’的变更，你们说做了影响分析。可我们法务从头到尾没被拉去评审过这条变更——影响谁、谁审批、出了问题谁兜底，你们自己定的?”

会议室安静下来。徐漾坐在角落，把 UAT 报告合上，轻声说:

“万辉，你把三件事当成一件事办了。”

“哪三件事?”

“评审、验证、确认。”徐漾竖起三根手指，“你说业务方点了测试环境，所以确认过了——那不是确认，那是让业务方当测试员，替你跑用例。你说测试全绿，所以验证过了——是过了，但你验证的是’按需求做对了’，不是’真实场景能用’。你说上线前评审过了——评审对的是目标，你们只对了需求清单。”

万辉不服:“这三样……不都是把关吗？方向对、做得对、能用，验收不就是看这个?”

“正因为三样都叫把关，才必须分清。”徐漾看着他，“你这一句话里混了三个词，三个词各对着一处东西——平时混，没事；混到验收签字那一刻，就是合同纠纷。”

如果你坐在这间会议室里，大概会站在万辉那边。因为大多数人对这三个词的直觉，和他一模一样：都是”检查一下”，都是把关，差不多——中文里都带一个”评/验/认”字，听起来像近亲。

这一章要推翻的，正是这个直觉。

### 8.1.2 这一章要回答什么

读完本章，你应该能回答：

1. 评审、验证、确认这三个词，差在哪？为什么中文看起来像近亲？
2. 它们各自对着什么——评审对什么、验证对什么、确认对什么？
3. 一件事该走哪一道，凭什么判？各自的判据是什么？
4. 三句话——“评审就是过一遍”“测试全绿就是验证过了”“业务方点了就是确认过了”——各错在哪？
5. 这三道关在一条链上怎么排？缺了哪一道，会漏掉什么？
6. 它们只对做出来的实物做吗？一份需求文档、一份设计描述，算不算它们的对象？
7. 为什么”只做验证不做确认”，最后会得到”测试全过、没人用”？

### 8.1.3 这一章的骨架

先把三个词各自是什么立起来，再把它们最常被认错的地方画开，最后看它们怎么用。

这一章的骨架，是一句话加一条因果。

> **评审对目标、验证对规定需求、确认对预期用途。这三样判定不挑节点，也不挑对象——实现可以查，产品数据也可以查。缺哪一样，就漏一类东西；而最贵的一课是：只做验证不做确认，隐含需求系统性漏失，测试全过、没人用。**

三块：

- **是什么**——三个词为什么被混用、各自参照什么、判据各是什么；
- **不是什么**——三个最贵的误解、两处射程、方向由谁看；

- **怎么用**——一条需求怎么走完这几道关、这些关怎么排满全程、三套手法各怎么做。

---

## 8.2 评审、验证与确认是什么

> 这一块把三个词分开看清：先讲病根——它们为什么被混用；再立三个参照对象；用一张表把它们摆开；最后逐个钉死各自的判据。

### 8.2.1 病根：它们为什么被混用

万辉的问题不是他笨，是中文给了他一个陷阱。

把三个词摆到英文面前，陷阱当场现形：**review、verification、validation**。三个词源不同、构词不同、归属也完全不同，译成中文是”评审 / 验证 / 确认”，都带一个”评/验/认”字。

这是第 1 章说的第三种词汇病”翻译伪朋友”的教科书案例：英文一眼能看出的不同，中文因为都是”一个动词加一个形近字”，看起来像三个近亲。三个词不是”检查一下”的几种说法，而是三件不同的事。

更狠的是，标准在定义里就把它们分了家——靠的不是分组，而是”确定”与”证实”的分野。

- **评审（review，§3.11.2）**的动作是”**确定**”：它的定义直接建立在 **determination（§3.11.1）**之上——查明特性、得出结论，是一次主动调查，**不要求客观证据**；

- **验证（verification，§3.11.12）**和**确认（validation，§3.11.14）**的动作是”**证实**”：定义用 **confirmation** 配 **objective evidence**——结果已经在那儿了，去证实它，**必须有客观证据**。一个是判，一个是证。判，靠判断力；证，靠客观证据。中文的形近字把这层区别糊上了，糊了四十年。

这里要把一个同形的词分开。第 3 章 §3.4.1 的节题叫”程度怎么认定”，那里判的是质量满足程度（差、好、优秀）；**本章的动作词统一用”证实”**。同一个动作，两章各叫各的名：第 3 章要”把程度定出来”，本章要”拿出客观证据证明它”。往后见到”认定”，先看它判的是程度，还是规定需求。

一个版本细节值得留意：ISO 9000 的历版编排里，这三个词曾分属两个不同的术语组——评审在”确定”组，验证与确认在”数据、信息与文件”组。到 2026 版，三者并入同一个”**有关确定的术语**”组，组内次第仍留痕：评审紧跟确定（3.11.1 → 3.11.2），验证与确认在组尾（3.11.12 / 3.11.14）。

分组是编排，动作词才是本质。标准把三个词收进同一组，却仍用”确定”与”证实”分别定义它们。这恰恰说明判与证的区别不是编排出来的，是概念本身的。

第 1 章点过的那场翻译伪朋友病，最重的代价是一场合同纠纷。一家医疗器械公司和一家医院在合同里写了”系统通过验收后付清尾款”，但谁也没定义”验收”是什么：公司说验收是功能测试通过，医院说验收是临床科室用得顺。双方扯皮四个月。

那四个月，就是把验证和确认糊在一个词”验收”里，等纠纷发生时一次性结算的代价。这一章把三个词分家，就是那场纠纷的预防针。

### 8.2.2 各自的参照

为什么各参照一处，而不是共用一个参照？因为它们看向三处不同的东西。把开发的世界画成四层：预期用途挂在旁边的场景层，另有两条旁路通道从侧边汇入需求。一图看清。

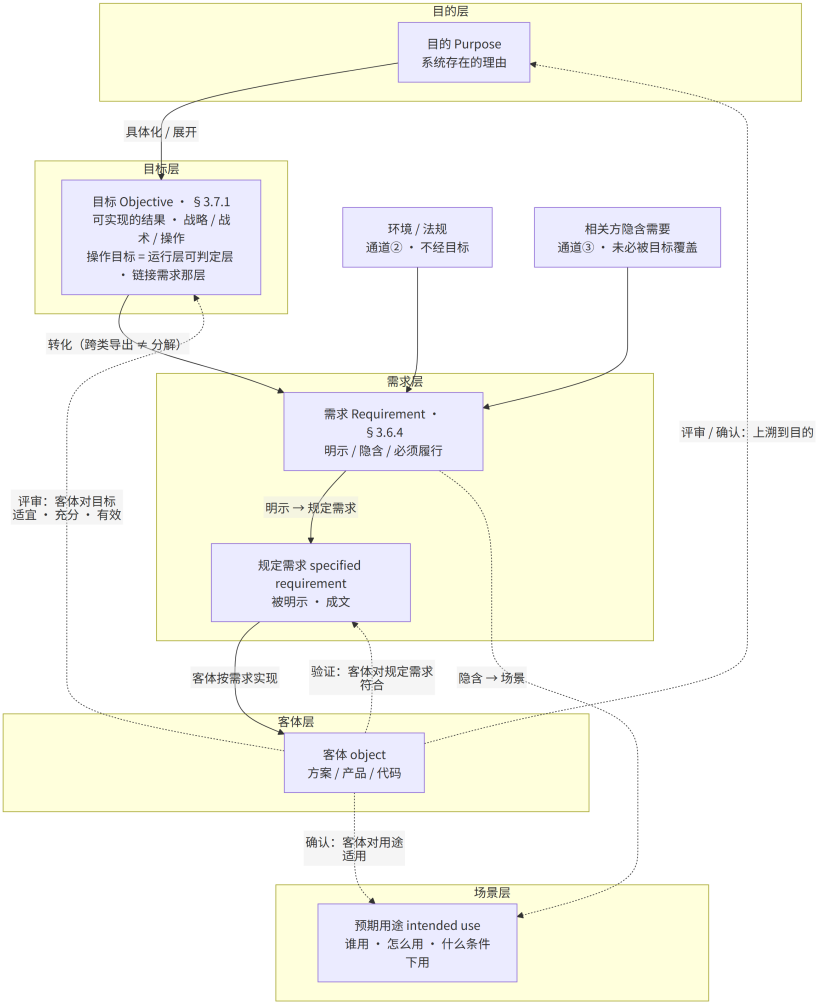



图 15　图 8-1

- **目的（purpose）**：系统存在的理由，回答为什么存在；目标是它在操作层的展开；

- **目标（objective，§3.7.11）**：要实现的结果，可分战略、战术或操作层面。其中**操作目标**是运行层可判定的那一层，也是链接需求的那层。“我们要让月底对账不再靠手工”是目标；
- **需求（requirement，§3.5.1）**：要满足的需要或期望，分明示、隐含、必须履行三种呈现方式，不都是规定需求；
- **规定需求（specified requirement，§3.5.1 注 2）**：被明示的需求，是”明示”这一形态的子集。“合同必须经财务、法务、业务三方审批”是规定需求（第 3 章展开过）；
- **预期用途（intended use）**：这个客体被设计出来，到底要在什么场景、给什么人、干什么用。

**先把两个系统的目的钉在这里**。冰链 M3 的目的，是”让疫苗在冷链运输中一路不失活、安全送达”；恒信合同系统的目的，是”让合同审批合规、月底账目清楚对得上”。两个系统的目标、需求与案例，都从各自的目的上长出来：恒信的”月底对账不再靠手工”、M3 的”断链即报废时还能撑到接入冷库”，分别是这两个目的在操作层的展开。

这张图里最容易被忽略的有几点。目的在最顶层，目标次之，规定需求居中，客体在最下面；预期用途不在这一列，它在旁边的场景层。需求还挂着两条不经目标的旁路通道。向下的建造是这条链：目的具体化成目标，目标转化出需求，需求明示成规定需求，再约束客体。三个判定则全部从客体出发，分别向三处不同的参照看去。

- 评审看客体对**目标**——方向对不对；
- 验证看客体对**规定需求**——做对了吗；

· 确认看客体对**预期用途**——做对的东西能用吗。

这里有一处最容易被漏掉的前提。图里画的”客体”是最笼统的说法：它既指实现出来的实物与系统，也指一纸规定需求、一份架构描述、一份设计描述——那些都是产品数据。第 7 章说过，开发全程你亲手造的是数据，实物是数据复制出来的结果。所以下面这几道判定，量的从来不只是车间里的东西；**数据本身也在射程内**（§8.3.2、§8.4.1 说透）。

把三个参照放回闭环，接缝才看清：目标是系统目的在操作层的展开，需求是目标转化（跨类导出，不是分解）的结果——但需求不只有目标一个来源。环境法规与利益相关方隐含需要这两条通道不经目标，直接进需求。目标的完备性靠评审的充分性把关，旁路通道靠需求工程兜底。

需求再往下，三种呈现方式在这里分叉。明示的那一种**写下来、定下来**（成文与基线是可采条件），就是规定需求；验证的参照在文档里。隐含的那一种写不出来，只能靠场景兜；确认把它放进真实用途，让它现形。三段的接缝，恰好是需求三种形态的分叉口。

评审与确认，都沿链上溯到目的：评审以目标为参照，根子上看的是系统目的；确认在真实场景里终审，方向照样对到目的。方向上的把关因此不是一处，是前后两处（§8.3.6）。

三种参照落在三个不同的世界里：目标属于”想要的”，需求属于”写下来的”，预期用途属于”真实的”。三者由此各守一种错。评审守方向错，验证守转写错，确认守匹配错。

这也解释了一处结构差：目标可以推导出需求，向下建造；唯独预期用途不能推导。它不是被想出来的，是真实场景里本来就有的事实，只能放进场景里发现。

一句话定案，各就各位：

> **评审对目标，验证对规定需求，确认对预期用途。**

换成三个可当场问的问句，按”哪个错最贵”排：

> **产品的方向对不对？（评审）→ 在真实场景里立不立得住？（确认）→ 是不是满足规定需求？（验证）**

方向错，整个产品白做；用途错，做对了也没人用；实现错，可以返工。若按站岗先后排（开工前 → 做完了 → 交给用户），就是评审 → 验证 → 确认，本章定案句与接力顺序都照此排。但要点在这里：这几处站位是典型站位，不是排他的位置——一份数据在它的每一个交付口上，都可以过这几道判定（§8.4.1）。

> 术语注：这句”评审对目标，验证对规定需求，确认对预期用途”是本书对 ISO 9000 三个定义（§3.11.2 / §3.11.12 / §3.11.14）的**凝练**，不是任何标准的原文——三个定义的原文见 §8.2.4~§8.2.6；本书的命名与判据拆法同样是作者主张，欢迎检验、欢迎校准。

### 8.2.3 分开它们的几处差别

光说参照不同还不够，因为参照不同可以只是修辞。把三个词摆成一张对比表，你会看到几处差别一起把它们分开：验证与确认的动作相同、证据要求也相同，只在参照上不同；评审则与前两者分了家。

| 维度 | 评审 review §3.11.2 | 验证 verification §3.11.12 | 确认 validation §3.11.14 |
|---|---|---|---|
| **定义**（汇总） | 为达成既定目标，对客体的适宜性、充分性或有效性所作的确定 | 通过提供客观证据，证实规定需求已得到满足 | 通过提供客观证据，证实针对特定预期用途或应用的需求已得到满足 |
| **参照** | 目标 | 规定需求 | 预期用途 / 应用 |
| **动作** | 确定（判断） | 证实（证明） | 证实（证明） |
| **证据** | 不要求客观证据 | 必须有客观证据 | 必须有客观证据 |
| **对象** | 工作产品（纸面即可） | **实现物，或人工制品**（产品数据） | **不限**（须在使用中兑现） |
| **时机** | 事前 / 事中做判断 | 对结果做证实 | 在真实或模拟场景中做证实 |
| **软件落点** | 需求评审、设计评审、代码评审 | 单元 / 集成 / 系统测试、压测 | UAT、Beta、灰度、试运行 |

> 注：表中”定义”一行是三个词各自权威定义的总览，算**汇总行**；真正的对比维度是下面几行。**“对象”一行是本章 2026-10-08**

> **新增的**：评审桌上摆的是工作产品；验证与确认的对象并不限于实现出来的东西，人工制品也就是产品数据，同样在内。依据见 §8.2.5 的术语注。

逐处看，三处差别最要紧：

动作不同，是最根本的分家。评审的动作是确定，验证与确认的动作是证实。确定是一次主动调查：从”没有结论”到”有结论”，结论是造出来的；证实的前提是结果已经在那儿，动作只是去证实它。这就是为什么评审不要求客观证据，而验证与确认必须有。

证据要求不同，是最容易踩的坑。“测试通过了”是验证的证据；这条证据不是评审本身。评审的结论是”这方案对目标合不合适”，不是证据，是判断。很多团队把评审开成读文档会，把证据念一遍，没有判断，那不算评审。

参照不同，是决定性的差异。评审对目标，验证对规定需求，确认对预期用途。这一条是后面所有部分的骨架。

评审、验证与确认并排立在这里，判据主义也在这里登场：第 1 章说过，不给没有判据的工具。评审三问（适宜、充分、有效）、验证与确认的三要素（对照、证据、证实），就是各自的判据。下面逐个钉死。

### 8.2.4 评审：适宜、充分、有效

先看第一道。评审的权威定义（ISO 9000:2026 §3.11.2）：

> **评审 review：为达成既定目标，对客体的适宜性、充分性或有效性所作的确定。** determination of the

> suitability, adequacy or effectiveness of an object to achieve established objectives.
> 示例：管理评审、设计和开发评审、顾客要求评审、纠正措施评审和同行评审。（注：本书按统一口径写作”设计开发评审”——设计开发不拆开。）

拆开定义，四个关键词：客体（被评的东西）、既定目标（尺子）、确定（动作）、**适宜性 / 充分性 / 有效性**（三个问句）。前三个都好懂，关键是最后这个三连问——评审不是问”对不对”，是问三件事。

| 问句 | 英文 | 通俗 | 恒信的例子（目标：月底对账不再靠手工） |
|---|---|---|---|
| **适宜性** | suitability | 对症吗? | 想解决月底对账，方案却只做了审批提速，不适宜 |
| **充分性** | adequacy | 够全吗? | 对账口径只覆盖了当月，25 号之后的归属没算，不充分 |
| **有效性** | effectiveness | 管用吗? | 功能全上线，月底对账还是要手工算三个小时，无效 |

三个问句层层递进，一个都不能跳过。

> **方向不对，做得再全也是白费（适宜性）；覆盖不全，方向对了也会翻车（充分性）；前两个都过了，还要看结果说话（有效性）。**

ISO 9000 还补了一句注：评审也可以包括确定”效率”（efficiency）——不仅看管不管用，还要看投入产出比。管理评审（**据** ISO 9001:2015 §9.3，**转述**，库外来源·不可核）同样落在这三问上——适宜性、充分性、有效性，三问是同一套，从产品评审一路用到公司管理。

拿冰链的 M3 走一遍三问，面目会更清楚。目标：断链即报废时，还能撑到接入冷库。

第一问是适宜性。方案选了双压缩机冗余，断一台还有一台，与”撑到接入冷库”相称。第二问是充分性。方案覆盖了压缩机故障，却没覆盖”运输车抛锚两个小时”：只做了设备级冗余，没做旅程级冗余。

第三问有效性——就算设备级、旅程级都齐了，真断链时能不能撑到冷库，要等真实运输的结果说话（§8.3.4 那辆会颠的车，会给出答案）。

三问不是同一个会里一次问完：第一问挡掉方向错的方案，第二问在方案细化时不断追问还漏了什么场景，第三问要等真实结果。很多人觉得评审是开一次会。错了：评审是一个动作，这个动作从方案出生那天起，就一直没停过。

> **概念卡片 · 评审**

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 为达成既定目标，对客体的适宜性、充分性或有效性所作的确定（ISO 9000:2026 §3.11.2）；动作是确定（判断），不要求客观证据 |
| **它不是什么** | ⑥⑦ | 与验证构成最接近的辨析——评审对目标（方向对不对，判断），验证对规定需求（做对了吗，证明）；评审不是核对需求——核对需求是验证的姿势，评审是唯一在纸面阶段抬头看目标的环节（确认是建成后的实证终审，见 §8.3.6） |
| **它从哪来** | ②③⑨ | 源自工程里的把关实践——设计评审、代码评审让多[illegible] |
| **它到哪去** | ④⑤⑧ | [illegible] |

### 8.2.5 验证：对照、证据、证实

第二道，也是大家最熟悉的——验证。它的权威定义（ISO 9000:2026 §3.11.12）：

> **验证 verification：通过提供客观证据，证实规定需求已得到满足。** confirmation, through the provision of objective evidence, that specified requirements have been fulfilled.

定义里有三个要素，**缺一个都不算验证**。

| 要素 | 问什么 | 反例 |
| --- | --- | --- |
| **① 对照：规定需求** | 有白纸黑字的尺子吗？ | 口头”大概要快一点”，无法验证 |
| **② 证据：客观证据** | 有可复核的证据吗？ | “我认为满足”，不算验证 |
| **③ 证实：明确结论** | 有明确结论吗？ | “差不多吧”，不是验证 |

第一个要素呼应第 4 章的规定需求：验证的参照必须是**可采**的规定需求——成文与基线是可采的条件，不是构成条件。“口头补的”需求（章首那条”25 号之后归属”）没进文档，就永远不在验证的射程内。

第二个要素最容易被低估。“客观证据”是验证和评审的分水岭：评审可以不依赖证据，验证必须拿证据说话。第三个要素决定验证的输出是结论：“已验证 / 未验证”，不是”跑完了”。

三要素背后还藏着一条推论，值得单独看一眼，它是第 4 章”需求质量里那条可验证”的真正出处。ISO 9000 定义规定需求（§3.5.1 注 2）时，只说它是”被明示的需求”，**并没有说”可验证”**。

“可验证”是从哪来的？从验证的定义（§3.11.12）推出来的。验证要求提供客观证据，而验证的参照是规定需求；那么能被当参照的规定需求，就必须能提供客观证据，也就是应当可验证。这是一条逻辑推导，不是定义原话。

> **“规定需求可验证”不是标准直接写的，是从”验证要求客观证据，验证以规定需求为参照”推出来的——可验证，是规定需求当验证参照的资格证**。

这条推论解释了第 4 章为什么把”可验证”列进需求质量：它不是一个可选的加分项，而是”这条需求能被验证”的入场券。口头那句”大概要快一点”，连被验证的资格都没有。没有资格，就没有证据；没有证据，就永远停在感觉上，到不了证实。

> 术语注（**对象不限，是本轮新立的一条口径**）：标准说验证的对象时用的是 a system, system element, or artefact——系统、系统元素**或人工制品**（ISO/IEC/IEEE 15288:2023 §6.4.9），并举例说人工制品包括系统需求、架构描述、设计描述。所以**验证并不是只对做出来的实物做**：一份规定需

求、一份设计描述，都可以被验证——验证它满足”这份文档自身应满足的规定需求”。第 6 章的”符合性分析”就是这种验证（§8.3.2）。至于**确认，它的现场在使用态，必须在使用中兑现**；工作产品层拿不到确认结论，能给的最重也只是**条件性合规**（§8.3.6）。

同族的两个动作也要分清。**检验（inspection，§3.11.11）**是对规定要求符合性的确定；**表明合格时**，它的结果可以作验证的证据。**试验（test，§3.11.13）**是按针对特定预期用途或应用的要求所作的确定；**表明合格时**，它的结果可以作确认的证据。这里说的是”哪一族的证据”，不是”哪个动作更高级”。

另有一处正本内部的张力：两册互为属概念、方向相反——42010 把工作产品说成人工制品，15288 把人工制品说成工作产品（15288 §3.6 与附录 B.1 的 NOTE）。本章取 15288 的父子序。

**概念卡片 · 验证**

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 通过提供客观证据，证实规定需求已得到满足（ISO 9000:2026 §3.11.12）；三要素＝对照规定需求＋客观证据＋做出证实 |
| **它不是什么** | ⑥⑦ | 与评审构成最接近的辨析——验证对规定需求（做对了吗，证明），评审对目标（方向对不对，判断）；验证不是测试——测试是提供证据的手段之一，验证要对全部规定需求做证实（§8.3.2） |
| **它从哪来** | ②③⑨ | 随”按规格建造”的工程实践而生——产品对规格的符合性必须能被客观证明，做对了才算定案；ISO 9000 用证据（objective evidence）做实（confirmation） |
| **它到哪去** | ④要素 | 三要素对照，证据与证实缺一不可 |

> 引：第 6 章的”符合性分析”在这里归位——它和测试一样满足验证三要素，只是证据形态是文档评审（§8.3.2）。

### 8.2.6 确认：换参照，不换判法

第三道，也是全书最被低估的一个。它的权威定义（ISO 9000:2026 §3.11.14）：

> **确认 validation：通过提供客观证据，证实针对特定预期用途或应用的需求已得到满足**。confirmation, through the provision of objective evidence, that the requirements for a specific intended use or application have been fulfilled.

注意：确认和验证**共享同一套三要素**：对照、证据、证实。它们唯一的区别在第一个要素——验证对照”规定需求”（文档），确认对照”特定的预期用途”（场景）。就这一处参照之别，决定了二者完全不同的射程和命运。

“预期用途”不是一句空话，它由三样东西构成。

| 构成 | 问什么 | 恒信的例子 |
| --- | --- | --- |
| **目标用户** | 给谁用? | 法务、财务、业务三方审批人, 不是 IT 测试员 |
| **使用场景 / 任务** | 用来干什么? | 月底对账、电子签章合同审批、审批人出差时手机处理 |
| **环境与条件** | 什么情况下用? | 月末高峰并发、弱网、审批人休假交接 |

> **这套三构成，和 UAT 那句”真实用户加真实任务加真实场景”一一对应：目标用户对真实用户，使用场景对真实任务，环境与条件对真实场景**。后文 §8.3.3、§8.4.6 会反复用它。

确认的尺子上, 还有一处更精确的写法值得钉死: ISO 9000 对照的是「特定的预期用途或应用的要求」——注意是”**要求**”(requirement, §3.5.1)。按标准，要求＝明示的、通常隐含的、必须履行的需要或期望，**隐含形态天然就在里面**。这把尺子拆开是两半:

- **明示的那半**: 操作目标——运行层确认的显性判分线，§8.4.4 那句”月底对账时间不超过两个小时”就挂在这里;
- **隐含的那半**: 未明示的使用需要与期望——“审批人休假交接也能顺”“月末高峰不卡”，靠场景确认兜住（§8.3.5 展开)。

操作目标不是预期用途要求的全部，只是它的显性子集。它当”所确立目标”进评审，是全身进入；进确认，只是参照的一部分。这也预告一句要紧话：操作目标全绿，确认未必通过（§8.3.5）。

确认的证据，ISO 9000 注 1 给了：试验结果，或替代计算、文件评审。注 3 更重要：使用条件可以是真实的，也可以是模拟的——这一句给了灰度发布、仿真测试的合法性。

M3 温控箱的”真实运输车跑全程”是真实条件；恒信”用测试环境模拟出月底高峰的并发”是模拟条件。都算确认，都满足”放进场景里看结果”。

镜照一眼冰链，把 M3 的预期用途填满三构成。目标用户是疾控中心和运输司机，不是实验室的测试员。使用场景是疫苗出库到接种的全程运输，不是恒温台架模拟。环境与条件是低温车厢、路面颠簸、信号弱的山区，不是平稳的测试环境。

M3 的预期用途，浓缩在”全程温度不断链”这几个字里。后面案例进展里那辆会颠的车，就是在用真环境填这三个格子。

还有一层要说清：确认的对象也可以是一份产品数据。数据的预期用途，就是接收方拿它干什么：制造数据给产线，场景记录给运维，需求数据给设计。这一层是本书的演绎。标准的确认定义不限定对象，但另一册把确认锚在”系统在使用时”（15288:2023 §6.4.11）；本书的桥是第 7 章那句话，开发产品就是开发产品数据，实物是数据复制出来的结果。演绎归演绎，底线的两样不动：确认的证据必须是客观证据，确认必须在使用中兑现。

还要当场划清一类”确认”：本书也会在一般汉语意义上用这个词——《系统目的或任务理解确认单》、“业务方确认”“同行确认”。这类确认是理解确认、是签认：对目的或任务的理解达成一致、靠签字生效，不要求客观证据，也不对照预期用途——它不是这一节说的那个确认。

往后见到”确认单”，别把它当成第三道关。这一节的确认只认一件事：真实或模拟的场景里，那份为特定预期用途的要求，满足没有。

### 概念卡片 · 确认

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 通过提供客观证据，证实针对特定预期用途或应用的需求已得到满足（ISO 9000:2026 §3.11.14）；三构成＝目标用户＋使用场景／任务＋环境与条件 |
| **它不是什么** | ⑥⑦ | 不是验证的加强版——参照不同（场景对文档）、射程不同（确认兜得住隐含，验证够不着）；也不是签认——《系统目的或任务理解确认单》那一类是理解确认，不要求客观证据、也不对照预期用途（本节末） |
| **它从哪来** | ②③⑨ | 随”产品要在真实用途里兑现”的工程实践而生。validation 源自 validus（强有效），一问的是”能用起来、老板满意”；verification 源自 verus（真实），一问的是”做出来的东西是不是图纸上要的那个” |
| **它到哪去** | ④要素 | 它要素齐备时的照见 |

> 引：确认的对象也可以是一份产品数据——数据的预期用途，就是接收方拿它干什么（本节末，**本书演绎**）；但确认必须在使用中兑现，工作产品层只出条件性合规（§8.3.6）。

## 案例进展①｜恒信重开上线评审会

章首那场会散会后，徐漾提议第二天重开。这次他先做的，不是讲方案，是定目标。

“这次评审的目标是什么?”徐漾问。

万辉：“上线前评审，目标就是……系统能上线。”

“‘能上线’评不了。目标要有对象、有时限、有可衡量结果。”徐漾在白板上写下三行：

- **月底对账**：上线后第一个月底，对账单与台账逐笔核对一致率 100%，对账时间不超过两个小时；
- **法务合规**：电子签章合同的审批链完整、每一步可追溯，法务可随时抽查；
- **业务可用**：业务人员三个工作日内完成一次合同发起到归档，不用求助外援。

“今天这场会，就按这三条评审。每条都有通过标准，会前定的，会后逐条打勾。”

万辉对着这三条，忽然卡住了：“……法务合规这条，我们的方案里，电子签章合同的审批链，到底谁审、谁签？这条变更当初是口头定的，需求清单里没有，方案里也没有。”

会议室安静了。

徐漾没有意外：“这就是评审对目标的意义。如果今天只核对需求清单，你会说全部覆盖、通过。可你的尺子不是需求清单，是法务合

规这个目标——目标一摆出来，你立刻发现：这条变更连需求都不是，是口头的一句承诺。”

万辉后来对徐漾说：“我以为评审是核对需求，原来评审是先把目标摆出来，再拿目标照需求。”

## 8.3 评审、验证与确认不是什么

> 这一块把这三个词最常被认错的地方分开：评审不是核对需求，测试通过不等于验证通过，业务方点了不等于确认过了；再磨清两处射程；最后看清方向上的把关为什么是前后两处。

### 8.3.1 评审不是核对需求

这是评审三问背后那个最容易被跳过的问题。如果评审以需求为参照，不是更省事吗？需求是现成的文档，一条条核对就是——很多团队就是这么做的（冰链团队的病，案例进展②会看到）。

四条理由，每一条都足以让”以需求为参照”破产。

理由一：需求本身可能是错的。需求是人写的——理解错、互相矛盾、不完整，都是常态。如果评审拿需求当尺子，就等于默认需求一定对。那谁来审需求本身？

需求评审的本质，恰恰是审需求对业务目标是否适宜、充分、有效。恒信那条电子签章变更没被评审：不是没核对需求，是需求背后那个目标（法务审批合规）没人去看。

这里有个对称值得记下：规定需求是**评审的对象**（需求评审审的就是它），也是**验证的参照**（验证对照的就是它），还是**验收与责任的依据**（争议时以它为准）。第 4 章把它立为全章枢纽、定成验证的参照；

到这一章，前两样正好各归一道：评审审它、验证对它；确认不参照它——确认的参照在场景里（预期用途），兜住的是验证够不着的东西。

同一个词，评审和验证都绕不开它；它没成文，这两道就都闭门。这正是章首那条口头补的需求的下场。

理由二：否则评审和验证就重复了。验证（§3.11.12）本来就是”以规定需求为参照、确认符合性”。如果评审也以需求为参照，两个术语就变成同一件事——标准用”确定”定义评审（§3.11.2）、用”证实”定义验证（§3.11.12），这套区分就塌了。两个词必须是两件事，否则标准不会用”确定”与”证实”分别定义。

理由三：满足所有需求不等于达成目标。需求全做完，做出来的可能还是没人要的东西：目标错了，需求再对也白做。评审是整条链上唯一在纸面阶段抬头看目标的环节：验证低头对规格、不沾方向；确认走进场景当实证终审。所以说”早拦方向，只有评审拿着目标这把尺子”；确认也看方向，但它要到使用态才登场（§8.3.6）。

理由三值得再往前推一步：它是最容易被”需求管理”掩盖的一种失败。恒信的目标是”月底对账不再靠手工”，需求清单里却全是审批提速、流程优化。需求一条条满足了，目标一个都没碰到。需求满足得越彻底，目标的偏离走得越远。

评审不是要否定需求，是要每隔一段就问一句：我们做的这些需求，还朝那个目标吗？

理由四：目标本身会漂移，漂的是系统目的。前三条理由默认目标是对的、静止的，这一条戳破它：目标不是钉死的靶子。

先把链点破。目标是系统目的在操作层的展开：“月底对账不再靠手工”是目标，“让合同审批合规、月底账目清楚对得上”才是系统目

的。评审抬头看目标，根子上看的是系统目的：《系统目的或任务理解确认单》锚定的就是它。

真正会漂移、漂了最危险的，正是系统目的。产品定位从专业工具滑向大众应用，业务方在评审里一点一点改口径，没有任何一个环节宣告过目的变了，需求却还钉在旧目的上。这就是第 14 章说的目的漂移：无症状、事后才发现，只能靠定期评审、以系统目的为参照检出。评审因此是捕捉缓慢漂移的传感装置。反过来，系统目的显式变更（战略调整、产品定位改变）就是第 14 章说的目的变更：显式发生的强制扳机，必须靠变更流程挂钩、触发架构重审。

评审以目标而非需求为参照的深层理由就在这里：需求是静止的文档，目标与目的是活的判据——只有评审，能在目的悄悄漂走时把船拉回来。

把四条收成一句话：

> **要求是路，目标是目的地。评审的价值，就是在低头检查路况之外，抬头看看目的地对不对。**

最失败的评审不是没查出问题，而是把评审做成了验证：只核对满足需求没有，从不问这需求本身对吗。章首万辉那条”UAT 全绿”，就是这个病的晚期。

## 案例进展②｜冰链镜照：评审做成了核对

冰链科技的需求评审会，开了整整一下午。

流程很标准：文正拿着需求清单，逐条念，逐条打勾——“有编号 √、有来源 √、可测吗 √、验收标准有 √”。三十条念完，清单上划满了对勾，文正宣布：“评审通过。”

测试李旭没有跟着打勾。他翻到第十一条，问了一句：

“这条是’断电保温不少于七十二小时’。我们的目标是什么？疫苗断链即报废，断电后要撑到接入冷库。那这套需求里，有没有写’运输车抛锚两个小时’这种场景？断电是从哪儿断的、断多久、什么时候算撑不住？”

会议室安静了。文正翻了翻：“需求清单里……没写。但这些是应急场景，要不要写进需求，可以后面再说。”

李旭：“后面再说，就是评审没过。你刚才的评审，只核对清单有没有写——那是验证的姿势，对的是需求清单。可评审要对目标：这套需求，能不能撑起’断链即报废时还能撑到接入冷库’？这才是评审要回答的问题。”

文正的病，和章首万辉是同一个病的镜像。万辉把确认做成验证（让业务方当测试员）；文正把评审做成验证（逐条核对需求清单）。两边的病根一模一样：这三个词里，只认识中间那道。评审对目标、验证对需求、确认对用途——把中间那道当成了全部。

文正后来在需求清单顶部加了一行字：“评审第一问：这套需求，对目标够不够？”逐条核对，是评审的最后一小时，不是评审本身。

### 8.3.2 测试通过不等于验证通过

这是验证这一节最要紧的一个辨析。测试只是提供客观证据的手段之一。验证的证据形态不少，本书统一列举为五种：测试、检验、审查、计算、文档评审。这是本书的归纳，不是标准的清单。标准在验证的

注里举的是检验的结果、替代计算、评审文件（§3.11.12 注 1）；在确认的注里举的是试验结果、替代计算、文件评审（§3.11.14 注 1）。

把”测试过了”等同于”验证过了”，是把手段当成了目的。

拿恒信的例子说透：UAT 报告写”用例通过率 96%“——这是一份**测试结果**，是证据的一种。但验证要回答的是”规定需求是否得到满足”，证据要覆盖全部规定需求——96% 意味着 4% 的用例没过或没写，这 4% 对应的需求是否满足，测试没有给出证据。

测试报告是验证的输入，不是验证本身；验证是拿着全部证据、对全部规定需求做出证实。

顺带把”检验”也请出来。它有双重身份：在 ISO 9000 里归 §3.11 这一组、和评审同组（§3.11.11）；但在证据链上又服务于验证——和测试一样，是提供客观证据的手段之一。检验是”对符合规定要求之合格状况的确定”，听起来和验证几乎一样，区别在粒度。检验盯着特性（温度波动不超过半个摄氏度这种单条）；验证盯着规定需求（一条需求可能含多条特性）。

第 6 章的符合性分析展示了另一种证据形态：文档评审。它没动一行代码，把”方案枝梢对需求的满足”当证据，也是验证，分两档。设计期的覆盖检查是满足度的**预演**；链尾构件规格评审上的那一遍，才是**正式审查**。

第 6 章说”符合性分析不等于评审，分析是评审的输入”，现在补全另一句：符合性分析是评审的输入，也是验证的一种。同一个动作，在”看它对不对”时是评审的输入，在”证明它满足需求”时是验证的证据。关键在于你在哪道门里用它。

> **符合性分析是设计层的验证（文档评审式）；测试是实现层的验证（试验式）。两者都满足验证三要素，只是证据不同。**

这里正好接上 §8.2.5 那条口径。验证一份规定需求、一份设计描述、一份架构描述，都是名副其实的验证——标准把人工制品明明白白写进了验证的对象里（15288:2023 §6.4.9）。所以”这只是一份文档，没法验证”这句话是错的：能验证，只是它的证据形态是文档评审、替代计算这类，不是跑一遍程序。

验证的证据有很多种：测试、检验、审查、计算、文档评审。别把验证窄化成跑测试，也别把跑测试升级成验证过了。

### 8.3.3 业务方点了不等于确认过了

确认最容易被误解的形态，是 UAT。拿它说透：

> **UAT 不是让业务方点一遍测试环境，而是真实用户、真实任务、真实场景三样齐了的确认。**

章首万辉的 UAT 错在哪？他让法务、财务、业务在测试环境点了一圈——那是让业务方当测试员，替测试团队跑用例，对照的还是需求清单。

真正的 UAT 是：财务用真实的对账数据、在真实的月底时间点、跑真实的月底对账任务——对照的是”月底对账能用”这个预期用途。前一种 UAT，测的还是需求；后一种 UAT，测的是场景。

让业务方点测试环境，是把确认做成了验证。业务方替你打勾的，还是需求清单上的格子；真实场景里会不会对不上账、审批人休了假怎么办，测试环境替不了真环境回答。

## 案例进展③｜恒信 UAT 的教训

上线前两周，徐漾把那次 UAT 的整个过程调出来，把万辉、玉杰、小白叫到会议室复盘。

“你说业务方点了测试环境，通过了。我问你三个问题。”

“第一，月底对账的数据，用的是真实数据还是造的数据?”——造的数据，二十条，没有一条是 25 号之后签的。

“第二，月底对账，是在真实月底的时间点、真实并发下做的吗?”——不是，周二下午，三个人轮流点。

玉杰说：“测试环境那三台机器是我部署的。周二下午三个人轮流点，是人在点；月底是几十个人同时冲进去对账，测试环境替不了真环境——那种并发，它扛不住，也测不出来。”

“第三，法务的电子签章审批链，业务方验证过谁审谁签吗?”——没有，那条变更压根没排进 UAT。

小白说：“UAT 的数据和用例都是我准备的，全照着需求清单写——清单里没有 25 号后归属，用例里就没有；清单里没有电子签章，用例里也没有。我以为跑到 96% 就是确认过了，现在才明白：我验证的只是需求，不是它好不好用。”

徐漾合上文件夹：“所以那次不是确认，是让业务方当测试员。你把确认做成了验证，让业务方对着需求清单替你打勾。真正的确认要回答的问题只有一个：真实业务场景里，这套系统转得起来吗？你没问。”

万辉：“那现在怎么办?”

徐漾：“补。确认不能后补到上线后——上线后补确认，就叫上线事故。把月底对账、电子签章、审批人休假交接三个真实场景，排进下一轮 UAT，用真实数据、真实任务、真实时间点。验证证明做对了，确认证明能用——两个都要，缺一个，另一个就是自我安慰。”

### 8.3.4 验证够不着没写下来的需求

验证有一处致命的边界，这一节把它看清。验证的参照是**规定需求**——规定，意味着**明示**；成文与基线是它的**可采条件**。那么：

> **没写进文档的需求，验证够不着。因为验证的尺子在文档里，没文档就没有参照。**

第 3、4 章讲过隐含需求——“箱体要能扛住装卸磕碰”“25 号之后签的合同归属下个统计月”，这些没人写，但漏了用户会骂。它们不是规定需求，于是不在验证的射程内。

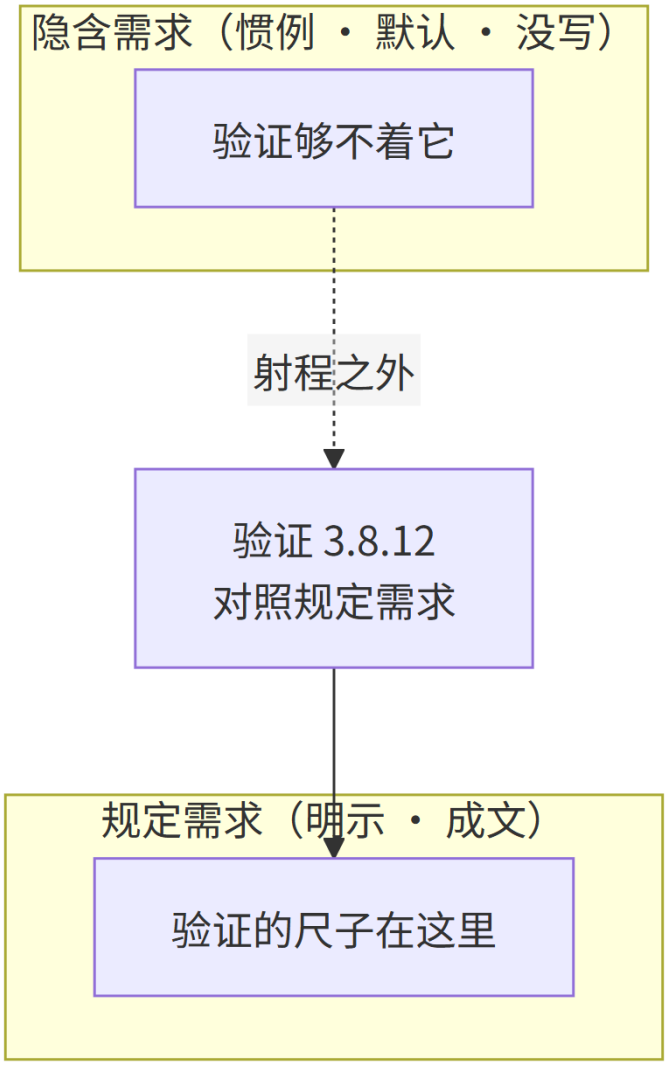



图 16　图 8-2

这就是”只做验证不做确认，测试全过没人用”的前半截：如果团队只做验证，那么所有没写进文档的隐含需求，全部在体系外漏着。测试

全绿，恰恰是漏光的那批需求绿不了，因为根本没被测。后半截，由确认来补。

### 8.3.5 确认的参照在场景里

把验证和确认摆在一起，一句话划界。

> **验证的参照在文档里，确认的参照在场景里。**

规定需求**通常以文档形态出现**：明示，成文与基线是可采条件。预期用途不是一份文档，它是一种**真实使用情境**：把产品放进情境里，好用不好用是自然显现的事实。这个区别带来一处决定性的推论——**确认能兜住验证够不着的东西**。

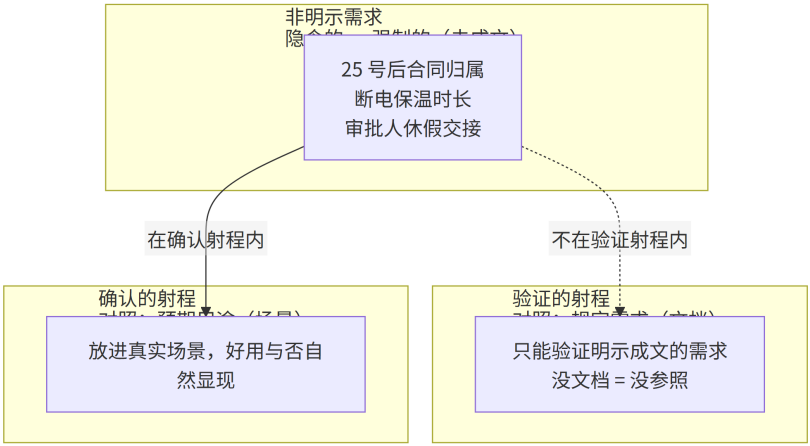



图 17　图 8-3

为什么场景兜得住、文档够不着？因为真实场景自带答案。月底对账、运输车颠簸、审批人休假——这些场景不会因为你没写文档就不

存在。把产品放进场景，隐含需求漏没漏，当场现形：对不上账、断档半分钟、待办卡在离职审批人手里，全是看得见的缺陷。

顺着这把尺子再往前一步：操作目标全绿，不等于确认通过。操作目标只是预期用途要求的显性子集（§8.2.6）——它达标是确认的必要条件，不是充分条件：指标全绿、真实场景却用着别扭，确认照样失败。

确认的独特价值，恰恰在兜住操作目标没有覆盖的隐含要求：验证够不着的是文档外的隐含（§8.3.4），确认还要兜住操作目标外的使用期望——两层”外”，都只在场景里现形。

这里有个比喻，值得放进记忆——隐含需求像冰山，水面下的部分占了大半。传递到下游的只能是浮出水面的部分；但没有”水下还有冰山”这个认知，船就直接撞上去了。

验证是量吃水线的尺子，它只管浮在水面上的部分；确认是把船开进那片水域的试航，水下撞没撞到，试航时才知道。第 4 章说过隐含需求不会自己开口；它不开口，就靠把船开进真实水域，让水下部分撞出声来。

冰山下面不只有隐含，还有强制。法规与安全的强制要求（第 3 章 M3 的断链即报废、恒信的合同数据留档十年），梳理成清单之前同样是未成文的。它们同样不在验证的射程内：要么靠确认看真实场景里法规是否满足，要么先梳理成合规清单再走验证。

## 案例进展④｜冰链镜照：测试全过，路上不行

M3 温控箱交付前，李旭组织了一次真实场景测试。不是测试用例，是一场真的运输：疫苗装箱，上真实的疫苗运输车，跑出库、运输、到达全程。

第一天晚上，问题就出来了。车厢过减速带时，温度记录仪因振动断电重启，断档三十秒。温度数据不是连续的。

李旭回看数据，发现断档发生在每次颠簸之后。他试着在测试台架复现——台架没有颠簸，复现不了。翻用例：里面只有”温度记录连续”，没有”振动下温度记录连续”。

“这不是测试用例没写好。”水忠看完数据，说了句，“是测试环境里没有路。测试台架稳得像桌面，真实车厢会颠。用例全过，说明不了上路能过。”

他们把 M3 装到真实车上连跑三天，又抓出三个问题：电池在低温下掉电快于预期、箱门在振动下轻微漏温、云端断网重连要四十秒。

三天，四个问题，没有一个能被测试用例提前发现——因为用例的参照是需求文档，而真实场景的参照是”疫苗能不能安全送到”。

把这四个问题翻回需求清单，一条都对不上：颠簸断电、低温掉电、振动漏温、断网重连，没有一条被写进过需求文档——它们是典型的隐含需求，真实存在、没人写过、验证够不着。确认的价值正在这里：它把没写下来的需求显影成看得见的缺陷——那次颠簸断档三十秒，就是需求文档里缺”振动下温度记录连续”这一条的证据。

这就是确认和验证的差别：验证在文档的尺子上跑，确认在真实世界的尺子上跑。冰链的测试台架做得再好，也替代不了那辆会颠的车。

### 8.3.6 评审早拦，确认终审

这三个词里，验证是唯一不看方向的——需求本身错了，它照样放行。

抬头看方向的是评审和确认两双眼睛，不是并列的两处，是一前一后。

| | 评审（前一处，早拦） | 确认（后一处，终审） |
|---|---|---|
| **时机** | 事前 / 事中，系统未建成 | 事后，系统投入使用 |
| **方式** | 预测性把关：判断与论证推演”方向应该对” | 实证性把关：真实或仿真使用证实”方向确实对” |
| **对象** | 纸面**工作产品**（目标、需求、**架构描述**、方案） | 系统本身，或载体为产品数据的人工制品 |
| **覆盖** | 显性部分（写在目标里、可逐条论证的） | 连隐含部分一起兜（真实使用暴露一切） |

> **术语注**：评审桌面上摆的是**工作产品**，不是”架构”本身——**架构没有形态、交不出去**，表达它的是**架构描述（AD）**。两个英文词互为属概念、方向相反：`artefact`（**人工制品**，ISO/IEC/IEEE 15288:2023 §3.6 即 work product (3.32) that is produced and used during a project to capture and convey information）；`work product`（**工作产品**，ISO/IEC/IEEE 42010:2022 §3.3 注 1 即 A work product is an artifact produced by a process）。15288 侧 artefact 是

> work product 的下位，42010 侧 work product 是 artifact 的下位。本项目按所属标准不同、属概念层级不同分流译名：指 15288 侧写”人工制品”，指 42010 侧写”工作产品”。

评审能前置，这是它最便宜的地方。一份需求文档没法被使用，也就做不了确认；但它可以评审，也可以验证。评审不要求客观证据，只要产品一定型就能做；验证则要等到有证据可举。这也是为什么评审能洒在整个流程的每一个节点上（§ 8.4.2）。

评审通过，只代表”基于当时信息的最佳判断”——它可能被确认推翻：纸面上再顺的方案，放进真实场景站不住，终审照样打回。所以两处关缺一不可：只靠评审，隐含需要无法被论证覆盖，评审对隐性盲区无能为力；只靠确认，方向把关全推到建成之后，纠偏成本爆炸。

但这里有一条不能越的界：评审替代不了确认。评审判的是”按既定目标够不够”，而既定目标本身可能从一开始就定错了。一屋子专家对着错误的目标评审，只会一致同意那个错目标。这正是确认必须在链外、且必须用起来的原因。

同理，也不能靠工作产品层拿确认结论。工作产品不能被使用，该层能给的最重结论是**条件性合规**：若实现如承诺则合规。**本书口径**：工作产品不能被使用，只能出条件性结论。真正的合规要由实现举证——ISO/IEC/IEEE 15288:2023 § 3.13 `design` 的定义句要求的，正是”足够完整以支撑**架构的合规实现**”（a compliant

implementation of the architecture)。这一点第 2 章已经点过一次，本章把它落到判定动作上。

> **一句话：方向对不对，评审先拦、确认终审；验证只管按写下的做对了吗，从头到尾不抬头。**

## 8.4 评审、验证与确认怎么用

> 这一块把这三个词用起来：先看一份需求怎么走完全程，再看这些关怎么排满整条流程，然后看清那条最硬的因果，最后是三套手法各怎么做。

### 8.4.1 需求走完全程

这三个词不是三座孤立的门。它们串起来，就是第 3 章”程度三步得出法”的第三步落地。回顾第 3 章：质量是特性满足需求的程度，程度怎么得来？三步——确定特性值（测量、检验、试验、评审、计算），与需求比对（**接收准则**——到评审那一关就叫**通过标准**），认定满足程度。

第三步的”认定”，动作上正是验证和确认：验证证实特性满足规定需求，确认证实客体满足预期用途。

所以这几道判定不是质量体系的外挂，是质量判定那个”程度”从感觉变成认定的必经动作。第 3 章说”没有接收准则，程度永远只能感觉”，这一章补全后半句：“没有验证与确认，连感觉都算不上，因为没人证实。”

先说清两处口径，后面才不会读歪。

第一处，这几道判定不挑节点。下面按开工前、交付时、交给用户排，是因为这三个站位最好记，也是它们最常出现的地方；但它们不是排他的位置。评审从方案出生那天起就能做，一直到发布之后还能做；验证在数据交到哪一级就在哪一级做；确认在任何一个已经用起来的场景里都能做。

第二处，它们也不挑对象。查的可以是实现物，也可以是一份规定需求、一份设计描述、一份架构描述——都是产品数据。下面这张全景图画的虽然是一份需求从生到用，它同样适用于树上的每一份数据。数据交到哪一级，验证就在哪一级回收；数据连同它的实现放进真实场景，确认才做得了终审。

把三部分合起来，让一份需求从生到用走一遍。这是本章最重要的一张全景图：

场景：恒信口头补了一条变更——“电子签章合同也走审批流”，它连需求都不是，只是口头承诺。

**第一关 · 评审（对目标）**：这条变更的目标是什么？法务审批合规。评审问：电子签章合同走审批流，谁发起、谁审批、谁签章、谁归档？法务认可这个审批链吗？这条变更对月底对账、业务可用两个目标有没有副作用？

评审在开工前，把目标摆出来照方案。案例进展①里恒信就是在这一关发现这条变更连需求都不是。

第二关 · 验证（对规定需求）：开发完成，这条需求被规定成文：“电子签章合同提交后进入审批流，审批通过后自动进入归档。”

验证问：这条需求满足了吗？单元测试过（签章触发逻辑）、集成测试过（审批引擎与签章服务）、系统测试过（全流程走通）、符合性

分析过（审批引擎这个枝梢的语义与指标）。这一关的对象不只是跑起来的系统：那条需求本身、承载它的设计描述，同样在射程内。

验证在交付时，拿规定需求当尺子，逐条证实。

第三关 · 确认（对预期用途）。上线前，法务、财务、业务用真实场景跑一份电子签章的采购合同。从发起、法务初审、财务校验、业务终审、签章到归档，全程能不能走通？月底对账时，电子签章合同的统计口径对不对？

确认在交给用户时，把产品放进真实场景，问能用吗。

缺哪一关，故事都不一样：每缺一关，缺的是一道具体判定，后果完全不同：

| 缺哪一关 | 后果 | 恒信的例子 |
| --- | --- | --- |
| **缺评审** | 需求错了没人发现，白做 | 电子签章变更没有目标可对照，它连需求都不是，是口头承诺 |
| **缺验证** | 做错了没人知道，返工 | UAT 通过率 96%，剩下 4% 没有证据，却照样宣布验证通过 |
| **缺确认** | 做对了也没人用，白做 | 真实对账场景从没测过，测试全过，月底对账对不上 |

这三个词各看一处，谁也替不了谁。

反过来，这张表还能做反向诊断：系统上线运行了，业务问题却还在——需求做对了、真实场景也能跑，说明转写（需求译成设计那一步）与匹配（放进真实场景那一步）都没错。根子只可能在最上游。评审没把”这系统对达成目标有效吗”这一关。目标是可度量结果，上

线只是系统在跑，不等于目标达成。建外场排故知识库是为解排故慢，需求却写成支持库存查询，上线了排故依旧慢：缺的正是评审这一关。

这三个词的产出——评审记录、验证证实、确认结论——合起来，就是后记四角色清单里系统工程师团队的《验证与确认报告》：怎么过的，都有证据落纸。这份报告本身就是产品数据，也要被管起来、被引用、被追溯。

同一套判定，在冰链长得一模一样，这是本书最喜欢的双域对照。评审：M3 的目标是”断链即报废时还能撑到接入冷库”，评审拿它照需求清单（案例进展②的教训）。验证：零部件试验、系统联调、整机低温试验，对照的是各项规定需求。确认：真实运输车跑全程（案例进展④的教训），对照的是”疫苗能不能安全送到”。一个软件，一个硬件；一条审批流，一辆运输车。同一个概念，两扇世界的门。

## 8.4.2 评审贯穿整条流程

把关要卡在每一次交接之后——从需求到发布，每个关键节点站一班。

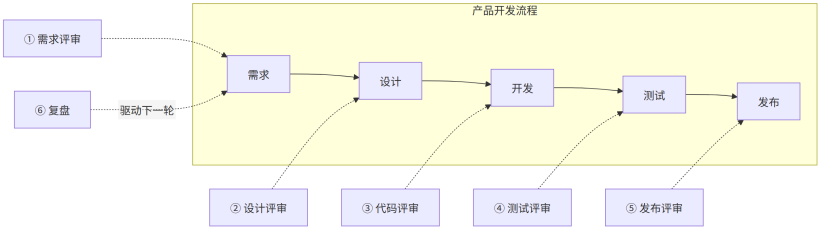



图 18　图 8-4

| 把关节点 / 复盘节点 | 目标陈述 | 判定内容 |
| --- | --- | --- |
| **① 需求评审** | 需求达到可转化状态（可作设计开发输入） | 符合业务目标（适宜）/ 无遗漏（充分）/ 可落地（有效） |
| **② 设计评审** | 方案可行且满足全部需求 | 对症（适宜）/ 周全（充分）/ 能落地（有效） |
| **③ 代码评审** | 代码正确、可维护 | 正确性 / 可维护性 / 规范性 |
| **④ 测试评审** | 质量达到准出 | 覆盖充分 / 缺陷收敛 / 结果可信 |
| **⑤ 发布评审** | 可安全上线、风险可控 | 就绪度 / 风险 / 兜底 |
| **⑥ 复盘** | 经验沉淀、流程改进 | 归因准确 / 行动闭环 / 记录归档（复盘是改进而非把关，见下） |

评审类的关口，都是评审三问（适宜、充分、有效）这一判据精神在具体关口的一次落地。上表各关口的判定词不同（如代码评审看正确性、可维护性，测试评审看覆盖、收敛、可信），追问的仍是同一件事：方向对吗、够全吗、管用吗。复盘除外。

而且注意时机：问题越早发现，修复成本越低——需求阶段的缺陷修复成本是一，发布后是三十到七十倍（第 4 章讲过）。这些节点排下来的本质，是把把关从终点挪到全程：让返工成本最高的那一段堵在源头。

为什么偏是这些节点？因为它们每个都是一次**转写**：需求译成设计、设计译成代码、代码译成测试、测试译成发布。每译一次，想要的

就可能被带偏一层：方向带歪、内容漏掉、意思歧义。这些关口恰好卡在每一次转写之后，对照上一层的源检查转没转偏：设计评审对兑现需求且站得住、代码评审对兑现设计且站得住、发布评审对可上线。评的都是本层产物自己够不够格，不是验证那种逐条对照需求。评审点的密度，就是转写层的密度：不是评审点选得巧，是转写层本来就有这么多。

评审类关口的通过标准，都该是一组**可勾选的硬条件**。拿需求评审和发布评审举例，两组清单的样子：

| 需求评审通过标准 | 发布评审通过标准 |
| --- | --- |
| □ 每条需求有唯一 ID 加验收标准（可测） | □ 发布清单逐条通过 |
| □ 业务场景覆盖 100% | □ 回滚预案就绪（已演练） |
| □ 无未决 TBD 项、无需求冲突 | □ 无未决 P0/P1 缺陷 |
| □ 每条需求可测 | □ 已知缺陷有缓解方案 |
| □ 资源、排期认可 | □ 运营、合规、安全准备到位 |

两组清单的共同点：全部由数字加动作构成，没有”质量要好”这种形容词。会前发出去，会后逐条打勾——打勾齐了，才叫通过。

这里有个连锁推论：通过标准会前定，意味着评审的尺子不是评审会当天造出来的，是方案出生时就该有的。第 3 章李旭那句”测试判据是什么”问的就是这个：测试方案里没有接收准则，评审就没有打勾的资格。

镜照一眼冰链，这些节点在硬件侧一个不少：需求评审、设计评审、试产评审、测试评审（出厂检验标准）、上市评审、复盘——名字换成试产、出厂，把关的骨架不变。硬件团队尤其容易犯”评审只在

试产时做一次”的错。这组节点不分行业，分的是你有没有把它铺到全程。

再说复盘。它是这排节点里最容易和评审划到一起的一个：两者都有回顾审视的直觉，但它和评审是两件事。

| | 评审（review） | 复盘（retrospective） |
|---|---|---|
| **时机** | 干活之前或之中把关 | 干完了，事后总结 |
| **归属** | IEEE 1028-2008 五类 / ISO 9000 术语 | 敏捷实践（Scrum Guide，库外来源·不可核）/ 组织过程改进 |
| **目的** | 把关：判断适宜、充分、有效 | 改进：沉淀经验、驱动下一轮 |
| **位置** | 在流程链条上 | 绕回起点 |

一句话：评审管把关，复盘管改进。复盘不在 IEEE 1028-2008 的五类里，也不是 ISO 9000 的评审。它绕回起点，把上一轮的教训变成下一轮评审的目标。

复盘还有一层身份，容易和治理混。**复盘是检测器**——它像传感装置一样例行运行：每个迭代、每轮上线后定期复盘，把偏差显影出来（对不上账、漏了场景、口头变更没记录）。

但复盘发现偏差，不等于裁决：把偏差显影出来的是检测，对偏差做裁决（要不要改口径、谁有资格批准、边界要不要挪）才是治理——那是第 14 章的事，本章只把检测器立稳。

复盘的通过标准同样要会前定：每个行动项有负责人和截止日期、上轮行动项关闭率不低于八成、问题按根因归类而不是甩锅、记录归档可追溯。复盘不产出”我们很努力”，产出”下一轮改什么、谁

改、何时改完”。案例进展⑤里恒信那次对账事故复盘，产出的就是两条行动项——变更必须先过评审对目标这一关、UAT 改真实场景跑，各配负责人和期限。复盘的价值，一半在诊断，一半在闭环；只诊断不闭环的复盘，是把同样的错再复一遍。

### 8.4.3 只做验证，会漏掉什么

现在把第 4 章的种子、第 6 章的动作、本章的两道判定串起来——这是全章最硬的一条推论。

> **只做验证不做确认，隐含需求系统性漏失，测试全过、上线没人用。**

拆成三段：

1. **隐含需求没进文档**（第 3、4 章）——它真实存在，但验证够不着；
2. **只做验证的团队**——所有没写进文档的需求，全部漏在体系外，连被检验的机会都没有；
3. **上线后**——功能全对、测试全绿，但用户用起来处处别扭：对账对不上、要的操作找不到、离职审批人把流程卡死。用户嘴上不说，行动上不用——**这就是”测试全过、没人用”的完整因果**。

反过来推一句更狠的，这是确认的第一层意义：确认的存在价值，恰恰建立在非明示需求上。如果所有需求都能写下来被验证，确认这个活动就不需要存在了——验证就够了。正因为永远有写不下来的需求（顺手、直觉、动效、节奏、惯例），确认才不可替代。

而确认兜住的，不止漏掉的隐含需求，还有一类**文字说不清的需求**。这是确认的第二层意义：冰链 M3 的手感要高级、恒信审批系统的用起来要顺，这类体验类需求显性化写不出来，只有真实使用能裁决。确认是这类需求的唯一裁判。

还有第三层意义，最容易被忽略：确认把漏掉的隐含需求变成可发现的缺陷。隐含需求没被写下来，所以它不是文档里的空洞：它是用户用不出来、卡住、放弃、抱怨。确认的价值，是把隐性缺失显影成缺陷证据：用户卡住了是证据，对不上了也是证据。

隐性缺失看不见，显影出来的缺陷看得见。章首万辉的 UAT 报告里没有一条缺陷——那不是因为他没有缺陷，是因为他把产品放进了一个造不出缺陷的场景（测试环境），缺陷无处显影。

## 案例进展⑤｜月底对账，对不上了

上线两周后，恒信第一个月底对账日。

黄程打开系统，导出对账单，按合同编号逐笔对着台账核对。两个小时，越对脸色越差。下午三点，他把徐漾和万辉叫到了财务办公室：

“对不上。25 号之后签的合同，编号全落在当月，系统把它们统计进了当月。我们月底对账的口径是——25 号之后算下个月。”

万辉：“这条……当初财务提过，后来口头说的，一直没进需求清单。合同编号的归属规则，谁也没定过。”

黄程：“我知道没进清单。但现在是真金白银的对不上，不是清单对不上。”

玉杰在边上查完一圈回来：“系统本身没坏——月底并发我盯着，没崩、没卡、日志干净。坏的从来不是机器，是’25 号后算哪个月’这条规则，从没被写进系统。”

徐漾让万辉把这次事故从头捋一遍，捋到最后，他在白板上写了八个字：验证做了，其余没做。

- 验证：UAT 全绿，测试证明按需求做对了——做了；
- 评审：这条变更从没评审过，目标（月底对账口径）没人对照过——没做；
- 确认：真实对账场景从没测过——当初排进下一轮 UAT 的三个真实场景，上线前被赶进度挤回了测试环境，25 号后的合同编号是真实对账时第一次出现——没做。

徐漾放下笔：“测试全过、没人用的下一句，是测试全过、月底对账对不上。你们以为它是运气问题，其实它是因果问题——该来的判定只来了一道，剩下两道，在等一场事故。”

万辉这次没有反驳。他后来在变更流程里加了一条：任何需求变更，必须先过评审对目标这一关，再谈开发；UAT 改成真实场景跑。第二个月底，对账在一个小时内对完：不是系统变聪明，是这三个词都站到了该站的地方。

这场复盘，徐漾让万辉把两条行动项记下来——变更必须先过评审对目标一关、UAT 改真实场景跑。复盘显影了偏差，但偏差的裁决（合同编号归属规则要不要定、归谁定）复盘给不了，那是第 14 章治理的事。

合同编号这条线，从这次对不上开始，后面还会在第 12 章被当成字段想改就改、在第 14 章三方编号对不上、在第 17 章上缝合清单。

### 8.4.4 评审怎么评

评审对目标，有一处操作性难题：目标是抽象的，怎么评？答案是先把目标具体化，再评审。抽象的目标评不了，“做出高质量的产品”无法评审；具体的目标评得了，“月底对账不再靠手工、对账时间不超过两个小时”可以评审。

目标具体化的配方，三个成分缺一不可：

> **目标＝对象（谁 / 什么）＋时限（何时）＋可衡量结果（多少）**

| 抽象（评不了） | 具体（可评审） |
| --- | --- |
| “做出高质量的产品” | “上线后三个月内，月底对账时间从三个小时降到两个小时以内” |
| “提升用户体验” | “审批人在手机端完成一次审批平均不超过一分钟” |
| “做好电子签章” | “本季度完成电子签章与审批流的打通，签章合同可全程追溯” |

评审会开场前，先做一遍**会前三检**——任何一个答不上来，就是目标还没定好，先别开会：

1. 评审结束后，**通过的标准**是什么（怎么算达标）？
2. 这个达标有没有**可观察、可测量的判定**（数字、行为、状态）？
3. 如果两个人理解不一致，**看什么能裁决**（哪份文档、哪个数字）？

三个都答得上，这个评审有尺子；答不上来，会开了也是白开——因为大家连过没过都没共识。而通过标准的正确用法，是**随评审邀请一**

**起发出**：“本评审通过标准：……全部打勾等于通过；任一不满足等于打回修改。”不是开会时现想，是会前定好、会后逐条打勾。

还有一个容易踩的点：目标本身还**分层级**。ISO 9000 说目标可以是战略的、战术的或操作层面的——注意这里的操作是运行层面，不是动手操作：它指管理层级里最贴近执行的那一层，不是让人去操作机器。同一份方案挂不同层级的目标，结论可能完全不同。

“我们要成为行业内合同数字化的标杆”是战略目标，激励人用；“月底对账时间不超过两个小时”是操作目标，评审用。战略目标让人相信值得做，操作目标让评审开得下去。评审要挂可判定那一层的目标：拿战略目标当尺子，等于没有尺子。

还要看清目标的来历：它不是供方自定的，是需方与供方就成功判据达成的约定。业务方提方向（要达成什么可度量结果），供方译成可评审的操作目标，再请业务方确认。确认过，它才成为评审的尺子。第 4 章链头说”立得对不对要业务方点头”，说的就是这一笔。

操作目标还有第二身份：它是确认参照里明示的那一半——进评审全身进入，进确认只是显性子集，所以操作目标全绿不等于确认通过（§8.2.6、§8.3.5）。

还记得第 3 章文正那句”符合性没有问题”吗？那是评审不了的——因为没问题不是标准。把它翻译成”温度波动不超过半个摄氏度、断电保温不少于七十二小时、交付前百分之百通过出厂检验”，才评得起来。评审对目标，但目标必须是能打勾的目标。

讲到这里，有人会问：评审这么好，凭什么信它是评审，不是聊天？IEEE 1028-2008（软件评审与审计标准，库外来源·不可核）给系统性评审钉了三个必备属性：

> **团队参与、评审结果文档化、评审过程文档化**。三个属性缺一个，按标准的话说，就是非系统性评审——说人话：**没记录的评审不算评审，最多算聊天**。

这一条值得停下来多看一眼，因为它是评审重形式的正当理由。为什么非要记录？因为评审的职责是主动暴露问题，不是汇报成果——而问题要被人看见、被追踪、被验证已解决，就必须落纸。第 6 章讲过覆盖完整性：漏一片叶子，下层方案就不知道要做；评审记录同理，不记录的问题，下次评审没人记得它存在过。

IEEE 1028-2008 把评审做法分成五类，各管一段：

| 类型 | 英文 | 干什么 | 落点 |
| --- | --- | --- | --- |
| **管理评审** | management review | 评估项目状态，支持管理层决策 | 项目级定期评审（进度、风险、资源） |
| **技术评审** | technical review | 对照规格，识别技术偏差 | 需求评审、设计评审、发布评审 |
| **审查** | inspection | 最正式，逐行检测，缺陷检出率最高 | 代码评审（最正式时） |
| **走查** | walk-through | 作者引导讲解，同行熟悉产品 | 代码评审（非正式时） |
| **审计** | audit | 独立于开发方，核实符合合同与标准 | 上线前独立核查 |

这里有一处术语分工要留意：IEEE 1028-2008 里的 inspection 本书译审查，让给 ISO 9000 §3.11.11 的 inspection（检验）——同一个英文词在两册里指的不是一件事：一册看规定需求，一册看软件工作产品。

五类从轻到重：走查最轻（作者带看一遍），审查最重（逐行挑刺）。但无论轻重，三个属性不变——有团队、有结论、有记录。

ISO/IEC/IEEE 12207:2017（软件生存周期过程，库外来源·不可核）把评审定为软件生存周期里的支持过程，并强调评审方与被评方

角色分离：评的人不能只评自己写的东西。这正是”有团队”的深层含义。

那评审到底图什么？把 IEEE 1028-2008 和工程实践里的评审目的收拢一下，是五件事，每一件都独立成立。

| 目的 | 说明 | 恒信的例子 |
| --- | --- | --- |
| **质量把关** | 多双眼睛盯同一份产出，把缺陷拦在上线之前而不是出事之后 | 设计评审挡掉”电子签章变更没有审批链” |
| **风险识别** | 不只看对不对，更看会不会出事，给项目做体检 | 发布评审发现”月底对账场景没人验证” |
| **对齐共识** | 把每个人脑中的想象差异摆上桌面当场敲定 | 三方审批人对谁审谁签当场对齐 |
| **决策推进** | 给出明确结论：通过、有条件通过、不通过 | 上线评审给出”暂缓上线，补确认” |
| **经验沉淀** | 新人学团队标准和坑，老手梳理思路 | 复盘沉淀”口头变更必炸”的教训 |

五条里，最容易被轻视的是决策推进——因为大多数评审会开完就散，没人敢说不过。开完会没下文的评审，前三件事全白做。决策推进的落地是三个动作：明确结论（通过、有条件通过、不通过）加有条件通过必须写明条件、复核人、期限，加记录留档。

这三条与 ISO 9001:2015 §8.3.4 的方向一致（转述，库外来源·不可核）：评审发现的问题要采取措施，并保留成文信息。评审的终点不是散会，是有一个字面结论、有一份记录、有下一步谁负责。

### 8.4.5 验证怎么验

验证不是最后做一次就完——它和设计开发的分解一样，是**逐级**的。第 3 章讲过 V 模型：需求的左半边（需求、设计、详细设计）逐级分解下去，验证的右半边（部件验证、系统验证、确认）逐级回收上来。左右是镜像。

需求从哪一级分解下去，验证就从哪一级收回来。这就是 V 模型的形状。

一个具体的样子（恒信合同审批系统）：

| 验证层级 | 验证对象 | 证据 |
| --- | --- | --- |
| **单元验证** | 单个函数、模块（金额校验逻辑） | 单元测试、代码走查 |
| **集成验证** | 模块之间（审批引擎与签章服务） | 集成测试、接口契约检查 |
| **系统验证** | 整个系统（合同审批全流程） | 系统测试、压测、合规检查 |

关键在左右镜像：“金额不低于五百万触发重点审”这条需求从系统需求层一路分解下去，它对应的验证就必须在每一层收回来。不是在最后一次性验，是分解到哪一级，验证跟到哪一级。

这里的镜像还有另一半，正好接上 §8.4.1 那条口径。分解下去的是需求数据，回收上来的是验证数据，而这两样都是产品数据。所以”分解到哪一级、验证就在哪一级回收”换句话说：数据交到哪一

级，验证就在哪一级做。数据停在纸面，验证就停在纸面（文档评审、替代计算）；数据变成实物，验证就跟到实物。

这个镜像可以推到底，推演一遍你就明白它为什么不是教条：需求分解到哪一级，说明需求在哪一级被规定；被规定的需求在哪里，验证就在哪里回收。

恒信把”金额不低于五百万触发重点审”分解到系统需求层，系统验证就在这一层回收（系统测试覆盖重点审触发）。它再分配到审批引擎，集成验证就在该引擎与签章服务的接口处回收。它落进金额校验这个函数，单元验证就在这一层回收。

每一级分解，都对应一级回收——V 模型的形状不是画出来的，是”需求在哪一级被规定，验证就在哪一级证明”这条逻辑逼出来的。

只做最后一次性验证，等于让所有分解白做了：需求分解到每一层，验证却只在最后一层收，中间断掉的层级，恰恰是缺陷藏身的地方。

镜照一眼冰链，硬件侧的验证长得不一样，但同一个骨架。单元验证是零部件试验（传感器精度校准、压缩机耐久台架）；集成验证是系统联调（温控单元、云端、应用软件协同）；系统验证是整机试验（低温环境箱里跑全温域）。

软件叫测试，硬件叫试验，V 模型的左右镜像纹丝不动。

你下次听到”我们要加强测试”，先问一句：是加强哪一级的验证？只加强系统级，等于只验最后一层，前面几层的分解没人接。

### 8.4.6 确认怎么确认

确认怎么落地？关键就在 §8.2.6 提到的那句 ISO 9000 注 3——使用条件可以是真实的，也可以是模拟的。这一句给了确认一整个手段谱系，按成本从轻到重：

| 手段 | 场景 | 谁当裁判 |
| --- | --- | --- |
| **仿真测试** | 模拟场景（测试环境造一个月底高峰让系统跑） | 造出来的场景 |
| **灰度发布** | 一小部分真实用户加真实流量先跑 | 真实场景的小样 |
| **试运行** | 全部真实场景 | 真实场景全量 |

灰度发布的合法性就在这里——它是确认，不是缩小版的验证。放一成真实流量进去，就是用真实场景当裁判。别把”灰度没出问题”读成”验证过了”，要读成”确认初步通过了，剩下九成的真实场景还没当裁判”。

确认还有一类照不到的强制需求，值得提醒：法规与安全的强制要求（第 3 章 M3 的断链即报废、恒信的合同数据留档十年），梳理成清单之前同样是未成文的。它们同样不在验证的射程内：要么靠确认看真实场景里法规是否满足，要么先梳理成合规清单再走验证。冰山下面，不只有隐含，还有强制。

数据侧的确认，前面说过一句，这里补完。数据的预期用途，是它的接收方拿它干什么：制造数据交给产线，产线能不能照着造；场景记录交给运维，运维能不能照着判。这一层是本书的演绎，不是标

准的原话；做法与实物侧一致——把数据连同它的实现放进真实使用里，看它立不立得住。演绎归演绎，标准那条底线不动：必须有客观证据，必须在使用中兑现。

时机上，确认常在设计开发形式结束之后（第 3 章说过，确认锚定产品，不锚定需求）。它站在工程系统的门口（第 2 章）：技术系统上线，不等于工程系统成功。确认问的正是：整个社会技术系统能转起来吗？确认不能后补到上线后。上线后补确认，就叫上线事故。

### 8.4.7 认知反转

回到章首那场会。万辉说”UAT 全绿，验收合格，可以上线”：他说错的不止一处。

第一处，他把评审做成了核对需求。评审对的是目标，不是需求清单。恒信那条电子签章变更没被评审，不是没核对，是没有人拿”法务合规”这个目标去照它。核对需求是验证的姿势；评审是唯一在纸面阶段抬头看目标的环节，方向这事，确认还会在真实场景里终审一次（§8.3.6）。

第二处，他把测试当成了验证。96% 的通过率是一份证据，验证是对全部规定需求做出证实。测试报告是验证的输入，不是验证本身。

第三处，也是最深的一处，他把确认做成了验证。让业务方点测试环境，是让业务方当测试员；真正的确认是真实用户、真实任务、真实场景。只做验证不做确认，隐含需求就系统性漏失——测试全过、没人用，不是运气问题，是因果问题。

这一章的反转，是那层被翻译糊了四十年的动作词区别被重新撕开。

> **评审对目标，验证对规定需求，确认对预期用途——方向对不对、做对了吗、真实场景好用吗，谁也替不了谁。**

而它最后指向的，是第 2 章那个没算完的账：恒信的系统上线了，但工程系统没成功——因为确认那道门没开。这一章让你看见的是：工程系统的成功，不靠上线这个词，靠这三个词各自站住。

回到章首徐漾那句警告——混到验收签字那一刻，就是合同纠纷。这一章看完，你应该说得出那句警告的全部含义：验收不是一句话，是这三个词依次签字。评审签方向对，验证签做对了，确认签能用——三个字都签齐，才轮到验收通过四个字。

少了任何一个，验收就是把几个词糊成一个的代价——§8.2.1 那场四个月扯皮的预演。

用第 1 章的话收一次束：对评审、验证、确认这三个词，你原来只是见过——背得出定义、叫得出名字，却分不清它们各自参照什么。这一章把它们推到拥有。

能答全评审、验证与确认各自的四问（各是什么、不是什么、从哪来、往哪去），能用三问和三要素量一张 UAT 报告、一场评审会、一次月底对账。

能答全四问才算拥有——评审、验证、确认，现在装进你手里了。

用第 1 章 §1.4.4 的尺子量一下：这几道判定站在**第 ② 级与第 ③ 级之间**——它们是把看得出结构校准成判断得住的那道质检关。

## 小结

这一章讲的是评审、验证与确认。**评审对目标**——站在开工前，拿目标这把尺，问方向对不对（适宜、充分、有效三问，目标具体化才评得起来，没记录的评审最多算聊天）。

**验证对规定需求**——站在交付时，拿规定需求这把尺，提供客观证据做证实（测试只是证据之一，第 6 章的符合性分析在这里归位，V 模型逐级回收）。

确认对预期用途——站在交给用户时，拿预期用途这把尺，把产品放进真实场景（真实用户、真实任务、真实场景，UAT 是确认不是验证）。三者接力，构成一份需求从生到用的几道关。方向上的把关是前后两处：评审在纸面早拦，确认在真实或模拟的场景里终审——验证不沾方向（§8.3.6）。

这一章还立了一条常被忽略的口径：这几道判定不挑节点，也不挑对象。开工前、交付时、交给用户只是三处最好记的站位；而它们查的从来不只有实现出来的实物——产品数据同样在射程内。数据交到哪一级，验证就在哪一级回收。

最要紧的那句因果，值得贴在墙上：只做验证不做确认，隐含需求系统性漏失——测试全过、没人用，不是运气问题，是因果问题。月底对账那天合同编号对不上的教训，就是这句因果的活例证：该来的判定只来了一道，剩下两道在等一场事故。

复盘站在另一头：它是检测器，例行地显影偏差，但复盘本身不裁决——复盘触发的裁决，才是第 14 章治理的事。

下一章，我们先去看把关背后那个人——工程师：头衔不定义他，六项能力才定义他，判断换个人接得住才算数。再下一章（第 10 章），

才进入全书概念的重心——架构；到第 11 章，架构活动的收尾一步是评审与背书，那里会看到架构评审的看家本领。

你会看到，第 6 章那棵树的枝条走向里藏着的，正是评审最该抬头看的那两样：系统的不可还原涵义（基本概念或属性），以及管住它实现与演化的管控原则。

运维的场景表里，添上了月底对账一格。这些规则以前只在业务方脑子里、不在纸上；这一章之后玉杰知道：场景不写下来，确认就够不着。

---

# 附录

## A 本章概念总图

评审、验证、确认（判据主义：每道判定各带可检验判据）：

评审 review 3.11.2 —— 对目标 · 确定（判断）· 不要求客观证据

· 三问：适宜性（对症）→ 充分性（够全）→ 有效性（管用）

· 系统性三属性：团队参与 · 结果文档化 · 过程文档化（没记录 = 聊天）

验证 verification 3.11.12 —— 对规定需求 · 证实（证明）· 必须客观证据

· 三要素：对照规定需求 + 客观证据 + 做出证实

· 测试只是证据之一；逐级回收（V 模型右半边）；射程够不着隐含

确认 validation 3.11.14 —— 对预期用途 · 证实 · 必须客观证据

· 参照在场景里：真实用户 + 真实任务 + 真实场景（UAT/Beta/灰度/试运行）

· 兜隐含需求（冰山水下）：只做验证不做确认 → 测试全过没人用

对象不限（本章 2026-10-08 立）：评审桌上摆的是工作产品；验证与确认的对象，实现物与人工制品（产品数据）都在内

—— 依据：15288:2023 §6.4.9「a system, system element, or artefact」；确认侧为本书演绎（桥＝第7章实物是数据的复制品）

节点不限：开工前 / 交付时 / 交给用户，是三处典型站位，不是排他位置

ISO 9000 的动作分野：评审＝确定（判），验证与确认＝证实（证）——同组而动作词不同，中文糊了四十年

接力链：评审（开工前·方向）→ 验证（交付时·需求）→ 确认（交给用户·用途）

把关铺在流程全程：需求 → 设计 → 代码 → 测试 → 发布 → 复盘

辨析：评审 ≠ 核对需求 | 验证 ≠ 测试（证据 vs 手段） | 确认 ≠ 让业务方点环境 | 评审 ≠ 复盘

方向的前后两处：评审（早拦·纸面·预测）→ 确认（终审·真实·实证）——验证不沾方向

评审替代不了确认：一屋子专家对着错目标评审，只会一致同意那个错目标

工作产品层只出条件性合规（合规的承担者是实现）

复盘 = 检测器（例行显影偏差），复盘触发的裁决才是治理（第14章）

合同编号对不上（月底对账）：该来的判定只来了一道，剩下两道在等一场事故

## B 本章的能力级

> **第 ② 级为主**，并落在 ② 与 ③ 之间。

- **表征**——把评审、验证、确认三个词说准，并把挨着它们的词分开：检验、试验、审核、核对需求、复盘、UAT、签认（确认单那一类）。
- **图式**——三处参照（目标、规定需求、预期用途）加一条接力链（开工前、交付时、交给用户，是典型站位不是排他位置）加一条因果链（只做验证不做确认，隐含需求系统性漏失，测试全过没人用）。
- **心智模型**——从症状反推缺了哪一道；方向由谁看（评审早拦、确认终审，验证不沾方向）；评审能前置；工作产品层只出条件性合规。
- **解释框架**——本章不建（归第 17、18 章）。

> 能答全四问，才算拥有——评审、验证、确认，现在装进你手里了。

## C 自查 · 这些说法对吗？

1. “评审就是开会过一遍需求。”——**错**。评审对目标，不是核对需求；没结论、没记录，最多算聊天。
2. “UAT 跑了就是确认过了。”——**错**。让业务方点测试环境是当测试员，不是确认；确认要真实用户、真实任务、真实场景。
3. “测试全绿就是验证过了。”——**错**。测试是提供客观证据的手段之一；验证要对照全部规定需求做出证实。

4. “验证和确认差不多，都是证明没问题。”——**错**。翻译伪朋友：验证对规定需求（做对了吗），确认对预期用途（做对的东西能用吗）。
5. “满足所有需求就是达成目标。”——**错**。需求本身可能是错的；评审是唯一在纸面阶段抬头看目标的环节——方向还有一处实证终审（确认，§8.3.6）。
6. “确认是验证的加强版，范围更大而已。”——**错**。参照不同（文档对场景），射程不同（验证够不着隐含，确认兜得住隐含）。
7. “这一份需求文档、这一份设计描述，谈不上验证，只能评审。”——**错**。标准把人工制品明写进了验证的对象（15288:2023 §6.4.9）；对文档能做的是评审与验证，做不了的是确认——确认要在使用中兑现。
8. “把评审通过标准写出来太形式化。”——**错**。目标不具体就评不了；对象、时限、可衡量结果，会前三检都答得上才开会。
9. “复盘也是评审，因为都有回顾审视的直觉。”——**错**。评审管把关，复盘管改进；复盘是检测器，复盘发现的偏差要等一道裁决才成为治理。
10. “测试全过、没人用是运气问题，多用心就能避免。”——**错**。只做验证不做确认，隐含需求系统性漏失，是因果问题不是运气问题。

11. “验证最后做一次、系统级全量跑一遍就够了。”——**错**。V 模型左右镜像：需求从哪一级分解，验证就从哪一级回收；中间断掉的层级恰是缺陷藏身的地方。
12. “评审目标太抽象，评不了，先看方案再定标准。”——**错**。目标具体化配方（对象、时限、可衡量结果）加会前三检；通过标准会前定、会后逐条打勾。
13. “评审过了就等于确认过了，两道门做一道就行。”——**错**。评审只代表基于当时信息的最佳判断，可能被确认推翻；而一屋子专家对着错误的目标评审，只会一致同意那个错误的目标。

## D 练习

1. 用一张表列出评审、验证、确认的参照、动作、证据、对象，再用你自己最近一个项目的具体例子各填一行。
2. 找出你们团队最近一次把评审做成逐条核对需求的例子——把这次评审重做一遍：先写出目标（对象、时限、可衡量结果），再问三问。
3. 你上一次说测试全绿的时候，验证三要素（对照、证据、证实）齐了吗？缺的是哪一条？
4. 拿你们一份需求文档或设计描述，试着做一次验证：它自身应满足的规定需求是什么？证据拿出来了吗？再说一遍：它做不了的是确认，为什么？

5. 用评审对目标检查你们一条正在做的需求：这条需求对应的目标是什么？拿目标照需求，需求够吗？
6. 测试全过、没人用的病根是什么？用验证射程与确认射程，解释隐含需求为什么只有确认兜得住。
7. 找出你们一条口头改的需求（没进文档、没评审、没验证的）——它现在在哪一步漏着？
8. 设计一次真正的确认：选一个你们系统的核心场景，列出真实用户、真实任务、真实场景、真实数据，写一份确认方案。
9. 用 V 模型解释：为什么”金额不低于五百万触发重点审”这条需求要逐级验证，而不是最后一次性验？这条链上流动的是什么，是实物还是产品数据？
10. 评审和复盘的区别是什么？你们团队最近一次复盘，是改进还是把关？复盘发现的偏差，后来由谁做了裁决？
11. 你们一次上线前评审，通过标准是会后现想的吗？如果是，把它改成会前定好、会后逐条打勾。
12. 用三关走一遍你们最近一次上线：评审对目标过了吗？验证对需求过了吗？确认对用途过了吗？缺的那一关，代价是什么？

（答案要点见书末附录，与训练教材题卡同源）

## E 参考文献

### 标准

- R-103 ISO 9000:2026（第 5 版）—— 评审（§3.11.2）/ 验证（§3.11.12）/ 确认（§3.11.14）/ 确定（§3.11.1）/ 检验（§3.11.11）/ 试验（§3.11.13）/ 目标（§3.7.11）/ 需求与规定需求（§3.5.1 及注 2）/ 客观证据（§3.8.6）；三者同属 §3.11”有关确定的术语”组，靠”确定”与”证实”分野——中文糊了四十年
- R-03 ISO 9001:2015 —— §8.3.4（评审、验证、确认三分：问题须采取措施加保留成文信息）、§9.3 管理评审（适宜性、充分性、有效性）（**库外来源·不可核**）
- R-11 IEEE 1028-2008 —— 五种评审类型（管理、技术、审查、走查、审计）加系统性评审三属性（团队、结果文档化、过程文档化）加评审产出（异常清单加行动项）（**库外来源·不可核**）
- R-16 ISO/IEC/IEEE 12207:2017 —— 联合评审是支持过程（评审方与被评方角色分离）（**库外来源·不可核**）
- R-102 ISO/IEC/IEEE 15288:2023 —— §6.4.9 验证过程（对象含 a system, system element, or artefact）、§6.4.11 确认过程（系统在使用时）、§3.6 `artefact`、§3.13 `design`
- R-53 Scrum Guide（Schwaber & Sutherland）—— 敏捷实践：复盘归属对照（**库外来源·不可核**）

> 对照阅读：本章的目标、需求与用途呼应第 3 章的”需求、设计开发、质量”、第 4 章的规定需求、第 6 章的符合性分析、第

> 7 章的产品数据就是那棵树。架构评审（评审四看、对质三问、背书分工）不在本章展开——它是架构活动的收尾一步，见第 11 章生成六步的第⑥步。

# 第 9 章 工程师：一份能力清单

## 9.1 引言

### 9.1.1 误解现场

恒信 IT 部要招一名”高级工程师”，徐漾主持面试。

候选人甲是算法高手，笔试几乎满分，口答流畅得像背书：“会写代码的，不就是工程师吗?”

候选人乙带着软考高级证书，职称材料叠了一沓：“有职称、有证书，还不算工程师?”

候选人丙干了二十年，项目数不清。徐漾问他：“你的判断，换个人接得住吗?”他答不上来：“干了这么多年，经验还不够吗?”

一个把载体当判据，一个把名义当判据，一个把年资当判据。徐漾没有反驳他们，他只问了一句：

> **换个人，用同样的需求，交付的东西能和你质量相当吗?**

三个人都沉默了。旁听席上的万辉，名片上印着”架构师”。这个问题，他这些年也没有答上来过。

第 2 章问过一件事：工程师头衔为什么不等于在做工程。第 8 章把三道门讲完，末了留了一句：下一章去看把关背后那个人——头衔不定义他，能力才定义他，判断换个人接得住才算数。这一章就是那句话的兑现，换个问法就是——**如果不看头衔，凭什么说一个人是工程师**。

### 9.1.2 这一章要回答什么

读完本章，你应该能回答：

1. 为什么头衔、证书、年限都不定义工程师？定义他的是什么?

2. 工程师的能力是一份什么样的清单？它从哪来，为什么是这几项？
3. 这份清单里，哪些管”能做事”、哪些管”定方向”、哪些管”传下去”？
4. 判断一个人是不是工程师，最少要问几句话？
5. 工程师和技术员、科学家、手艺人、会写代码的人，分界各在哪一条？
6. 同样是工程师，为什么背的账不一样？专责和终责差在哪？
7. 从一个新手到一个工程师，路上有哪几道关口？先过哪一道？
8. 工程师最常失败的方式有哪几种？每一种缺的是哪一项？
9. 为什么能力最后要到产品数据上去看？
10. 冰链的水忠和恒信的万辉，谁是工程师？

### 9.1.3 这一章的骨架

这一章只做一件事：把”工程师”这个人，放到第 2 章那把量工程的尺子下面量一遍，交给你一份可以逐项打勾的清单，和一条能走的路线。

先说工程师是什么：立清单，一项一项立。这份清单不是本章凭空立的，它长在前面的章上。能力来自第 2 章那几层；活来自第 3 章到第 8 章，按第 3 章立的那条路径依次走一遍：需求、设计开发、产品数据、评审验证确认。每一项按同一套问题走：它是什么、它不是什么、凭什么说你有、怎么练、缺了会怎样。

再说工程师不是什么。先和外面分开：不是技术员，不是科学家，不是手艺人，不是会写代码的人，不是头衔、证书、年限；再往里划一道：工程师这个名字底下，不止一种人。

最后说工程师怎么做成：从一个新手到一个工程师，路上有几道关口，每道关口先过什么、怎么算过了，最后落到一件看得见的东西上。

全章只问一句话：换个人、换团队，用同样的需求和约束，能不能交付质量相当的成果。清单里的每一项都是这句话下的一个检查点：每一项的物证是产品数据，侧面是粒度、熟练度、几种死法。

这条判据不是本章新立的——第 2 章 §2.2.4 立过它，本章只是把它从”量一件事”用到”量一个人”身上。这一章也不再发明新概念：工程科学、方法论、工具与实践、价值与伦理这几层，第 2 章已经立好了，这里只用它们一次。

往前追，能力与活都不是本章的发明：那几层是第 2 章立的，那条路径是第 3 章立的，需求、设计开发、产品数据、评审验证确认都各有出处。往后落，第 13 章会把⑥可复现意识长成配置与基线，第 14 章会把④职业伦理长成治理；到第 18 章，再加上系统级的那几项增量，合成架构师的一份清单。

---

## 9.2 工程师是什么

先说工程师是什么：把第 2 章那套分层，落到一个人身上。

### 9.2.1 工程师能力清单

先把清单立起来。

第 2 章立过：工程科学、方法论、工具与实践、价值与伦理，这几层同时在场，工程才成立。那里讲的是学科的样子；这里要看的是人的样子。同样的几层，长在一个人身上，会变成什么。

答案不复杂：每一层，都要在一个人身上长成一项能力。工程科学长出科学素养，方法论长出方法论素养，工具与实践长出实践能力，价值与伦理长出职业伦理。学科缺一层，工程不完整；人缺一项，工程师不完整。

还有不占任何一层的能力。

**工程判断**横切在这几层之上。原理用到哪里为止、方法要留多少判断空间、工具选哪一套、底线划在哪——每一层都有取舍，取舍的本事不归任何一层独有。

**可复现意识**管着全部。它问的不是”你会不会”，而是”你会的这些，换个人能不能接住”。

清单如下。从层里长出来的那几项排在前面，工程判断与可复现意识另算；④职业伦理放在最后讲，它管方向与底线，收束整份清单。

| 能力 | 长在哪一层／与这几层的关系 | 缺了会怎样 |
| --- | --- | --- |
| ① 科学素养 | 工程科学 | 换个场景就抓瞎 |
| ② 方法论素养 | 方法论 | 靠人传人，人走方法走 |
| ③ 实践能力 | 工具与实践 | 有想法，落不了地 |
| ④ 职业伦理 | 价值与伦理 | 会出事 |
| ⑤ 工程判断 | 横切在这几层之上 | 有流程也选不对，约束一变就懵 |
| ⑥ 可复现意识 | 不占哪一层（元） | 手艺——你在质量在，你走质量走 |

这份清单不是拍脑袋列的。

> **术语注**：本清单以第 2 章那套分层为骨架，对照 ABET 工程师成果七条凝练而成（能力清单及其分层映射取自报告库 11 号 §6；ABET 七条的文字转引自同报告 §7.2；库内无 ABET 正本，属库外来源）。⑤横切、⑥元、“职业伦理定方向”这些构造与判据，是本书主张，不是任何标准的条文；欢迎检验，欢迎校准。

清单立到这儿，只立了一半。上面那几样是能力，长在人身上；另一半，是他干出来的活。

工程科学、方法论、工具与实践、价值与伦理这几层，第 2 章讲的是学科的样子；落到一个工程师手上，它们除了长成能力，还要落到需求、设计开发、产品数据、评审验证确认这几件事上。

这几件事不是本章另立的，它们来自第 3 章到第 8 章：第 3 章给了它们的次序，定需求 → 出规格 → 判满足；第 4、5 章把”需求”这件事做透；第 6 章给”设计开发”，也给出产品数据那棵树；第 7 章给”产品数据”；第 8 章给”评审验证确认”。§9.2.8 一件一件说。

能力问”我有没有”，活问”我会不会干”；两样都答得上来，才叫是个工程师。

这份清单两组都立起来了；它的用法在 §9.4 讲，先记一句：**它不能背**。这不是知识点清单，是自问清单。

按 ISO 9000:2026 §3.10.6，competence 是”应用知识和技能以取得预期结果的能力”，它是干成事的本领，不是背下来的知识。这

份清单属于这一类，所以它只能拿来逐项自问。可复现意识之所以也要写进清单，正是为了让清单自己受检验：一份只能背下来的清单，本身就过不了它。

还有一件事要在这里说清：这份清单的每一项，最后都指向同一句话——就是章首徐漾问的那句。往后每一节的”凭什么说你有”，问的都是它的一个侧面。

### 9.2.2 ①科学素养：凭什么能行

“科学素养”在科普里指”懂科学、不被忽悠”。那句话的对象是公众，判据是不被忽悠。在工程里它是另一回事：对象是工程师，判据是一句更硬的话——我做的东西，凭什么能行？

拆开来，是这几件事。**理解原理**：知道它成立的逻辑，不是背下公式。**判断边界**：知道它在什么条件下成立、什么条件下失效。**应用建模**：把现场简化成模型，先估出量级，再谈精度。

第 2 章说过，没有力学定律，土木工程师只是个会搭积木的人。这句话反过来读也成立：一个只会套模板的人，换一个场景就会露馅，他的依据不在原理上，在”上次这么做成了”上。

所以科学素养不是懂很多公式。公式可以查；“依据成不成立”这件事，不能外包。

怎么练？第一句动作是写下预测，事后对账。

改动之前先把预测写下来：改了之后会怎样、哪个数会变、变多少。事后拿实际结果对账，找出偏差。偏差回来，先问一句”是哪条原理越了界”——是原理用错了范围，还是模型简化掉了什么。问清楚了，再决定是修原理还是修模型。

这一条和第 2 章那句”错误可见”是同一件事：能看见偏差，才谈得上修正。没有预测的实践不产生科学素养，只有”调到对为止”，那是手艺。

日常还有几个短练习可以用：反向追问（拿一个顺手的工具，问一句”它凭什么能行”）、数量级估算（不查资料先推个量级）、讲给外行（讲不通就是没懂）。

> **判据**：我做的东西，凭什么能行？换个上下文，还成立吗？

缺了这一项会怎样？换个场景就抓瞎，只会套模板。恒信的万辉砍掉容量评估那天，没有人问他”凭什么能行”，他自己也没有问（案例进展②）。

### 9.2.3 ②方法论素养：把工作组织成方法

方法论素养不是会背方法论名词：敏捷、瀑布、CMMI 张口就来，那是方法论词典，不是素养。

方法论是”关于怎么组织工作、怎么选方法”的学问；素养是把工作组织成方法的能力。它的做法也落在几件事上。

**过程设计**：每一步有输入、有输出、有检查点。**显性化**：把个人做法写成文档、规则、标准，可教可学。**过程治理**：检查过程有没有在跑、偏差出在哪、下次怎么改。

本章把设计开发当作工程里最要紧的那套方法，也就是由需求到方案的逐对生成。第 6 章给了它全套：三套规则、三个动作、结对六步。

它不是一句”最佳实践”，是一组有判据的过程：每一步有输入、有输出；承接有单值性，一条需求只落到唯一一个节点上；覆盖有公

式，一个节点承接的所有需求，语义包含于它叶节点语义之并。一个过程有判据，才算方法；没有判据的，只是习惯。

ISO 9000:2026 §3.3.11 给设计和开发的定义是”将某一客体的需求转换为该客体更详细需求的一组过程”。本书读法：一组有判据的过程，就是一套方法。所以②要问的不是”你会不会跑那套设计开发”，那是第 6 章的功课；它要问的是：第 6 章那套章法，你能不能照你这摊活组织出来、写下来、交给别人跑。

第 3 章说过这句话的另一半：工程＝概念这一侧的方法（需求、设计开发、质量，管”要什么、怎么证明做对”）×技术本体。

从”我会做”到”这个方法谁都能用”，中间是一条转化链：做法在心里，是隐性判断；写成文档与规则，是显性化；定下步骤、输入输出与检查点，是标准化；换个人也能交付，才成了组织能力。

这条链上每一段都有风险。写不出来的，死于”人走方法走”；写得太细的，死于”算法化”：判断空间被写没了，方法退化成一张照单执行的清单。第 2 章那条界线在这里最要紧：方法要有章法，但每一步都得给判断留位置。

怎么练？第一句动作是把你这摊活的规则写出来，交给同事照做，看他卡在哪。

写你最近一次做对的事，步骤、依据、坑都写上，交给一个没参与过的人照着做一遍。他卡住的地方，就是你还没显性化到位的地方。拿着现有流程再过三问：每一步有输入输出吗？有可检验的检查点吗？换个人做，结果稳不稳？答不上来的环节，通常就是事故多发处。

> **判据**：我的方法能被别人学会吗？还是只有我会？

缺了这一项会怎样？工程靠人传人，人走了方法也走了。冰链的水忠就卡在这里：他的手艺是团队最值钱的资产，但那份判断说不成体系（§9.2 案例进展①）。

### 9.2.4 ③实践能力：亲手把方案变成现实

实践能力不是会写代码、会画图，那是载体。实践能力是亲手把方案变成现实、东西坏了能修好。

拆开也是这几件事。**动手实现**：想法变成代码、图纸、样机。工具驾驭：编辑器、集成流水线、测试环境；方法论必须落成工具和动作，才产生价值。排错集成：不工作时能定位，把部件接起来跑通——缝里的问题，常常只有上手才发现。

这一项只能从”做”里长出来。读和听几乎不产生它。它有一条强度光谱：

阅读（看不会）→ 看演示（眼睛会了）→ 跟做（手还没会）→ 仿做（照猫画虎）→ **独立做**（判断进场）→ **救火**（排错定力）。

到”独立做”那一级，判断才开始进场；到”救火”，才练出”坏了能修好”的定力。

怎么练？第一句动作是把强度光谱往外推一格：这个月在”跟做”，下个月就找一件能”独立做”的事；已经在”独立做”的，主动接一次救火。

它的边界也要划清。不是埋头苦干——做完要显性化，配合方法论素养，否则只是手艺。不是工具集——换个工具就抓瞎，说明缺的是科学素养。自动化时代它反而更值钱——机器接走执行，人接住的是判断密度最高的那部分执行。

> **判据**：我最近一次亲手落地，是什么时候？

缺了这一项会怎样？有想法落不了地。这一项缺了，其他几项都悬空——水忠之所以是本团队里最值钱的人，首先因为他这一项顶配。

### 9.2.5 ⑤工程判断：在约束里取舍

工程判断，是在约束下做对的取舍，而且担得住。它不是”会做决定”。万辉那句”先上再说”也是个决定，问题在于那是个坏的取舍：他做了取舍，却没让任何利益相关方知情，也没走评审。

第 2 章那句话可以搬到这里用：约束越多，判断越值钱。

好的工程判断是一个闭环。

**识别约束**：先看清冲突在哪——成本、工期、安全、法规，哪几个在打架。**权衡取舍**：排序、找平衡点，trade-off 要有依据，不能凭手感。**担责闭环**：让人知情、走评审、留记录。

选得对只完成了一半，另一半是能不能负责。这也正是工程判断和职业伦理的交界：判断负责选，伦理负责划不可越的线。

判断力是清单里唯一”必须亲自做错”的能力。它不能教，只能长；生长点不是”做决定”，是复盘。做错的决定加上认真复盘，胜过做对的决定加上不复盘。

怎么练？第一句动作是把每一个重要取舍记下来：当时哪些约束在打架、选了哪个、后果如何、重来会怎么选。低风险的补充练法是案例裁决——拿恒信”先上再说”、挑战者号发射决策这类真实案例，先自己裁，再对照书里的判断；判错了不会出事，但会留下痕迹。

> **判据**：规则没有覆盖的地方，我能做对的取舍吗？

缺了这一项会怎样？有流程也选不对，约束一变就懵。这也解释了这一章开头那场面试的第三个人：二十年经验如果没有变成判断，那只是把同一年重复了二十次。

### 9.2.6 ⑥可复现意识：让判断被接住

可复现意识，管的是你不在场的那一刻。手艺人的全部本事长在本人身上；工程师的要求是”换个人也能交付”——第 2 章的试金石，落到个人身上就是这一项（报告库 11 号 §4.2 把这一项称作试金石的个人版）。

它和方法论素养最容易混。方法论素养是”写成什么样”的本事；可复现意识是”必须写成那样”的自觉。前者会写，后者主动写。很多人前者有、后者缺：文档写得出来，就是不写。那不是能力问题，是意识问题。

它从近到远要做几件事：**判断显性化**（把心法写成规则、文档、标准）、**方法可传授**（换个人照着做，结果质量相当）、**组织化**（变成组织能力，人走判断不走）。

怎么练？第一句动作是做一次交接演练。

假设明天你就把手上这摊活交出去，写一份交接包：步骤、依据、坑。写不出来的地方，就是你还没想清楚的地方。

它也有边界。不是复制粘贴模板——可复现的是方法，不是结果。不是消灭个人价值——水忠的经验仍然最值钱，这一项是把它写出来，不是把它废掉。不是把判断写成规则——§9.2.3 说过，判断不能写全成规则；这里写下来的是方法和依据，最后的判断仍然留给人。

> **判据**：我的判断换个人能接住吗？我把它写成规则、文档、标准了吗？

缺了这一项会怎样？手艺：你在，质量在；你走，质量走。恒信那套流水线就栽在这里：脚本在万辉那台机器上跑得顺，换台机器就缺那三条隐藏命令和两个环境变量（案例进展②）。

### 9.2.7 ④职业伦理：给能力定方向

先正名：职业伦理不是道德说教，也不是规章制度。

它是”工程师”这个身份的定义性特征：工程是一种职业（ABET 用 the profession，文字转引自报告库 12 号 §1.1；库内无 ABET 正本）。NSPE 伦理准则的序言把这件事说得更直白：工程师的服务必须致力于保护公众的健康、安全与福祉（文字转引自报告库 12 号 §1.1；库内无 NSPE 正本）。

这一项之所以要单独说，是因为能力不是中性的。

清单里前几项都在回答”能不能做成”；这一项回答”往哪个方向做”。一个科学素养、实践能力、工程判断都顶配、而职业伦理为零的人，破坏比”不会做”的人大得多：他会算、能做、判断准，只差守住底线。737 MAX 里不缺会算的工程师，缺的是守住底线的工程师。

恒信那次月底急上、没做容量评估的接口也是这个形状：赶进度时，“写一半顺手改审批状态机”，能顺手改，说明能力不差；没评估影响就改，说明这一项缺席。

一次妥协算不算”客观原因”？第 2 章 §2.3.3 那三问，到这里就是这几句话：

利益相关方知不知情？有没有评审？有没有记录？

客观原因不留记录；价值与伦理失守，恰恰发生在”没有记录”的地方。这三问也正是第 8 章那三道门禁在压力下的落点：评审要利益相关方知情、结论要记录在案，验证确认要客观证据。

它在这份清单里管的是方向与底线：**定义方向**——能力是选择的规模，伦理是选择的方向；**守底线**——压力面前不越线。所以它和前几项不是一类：前几项是增量型，做多做少都算数；它不能靠”多做练习”增值，只能靠在压力下不越线来证明。

但”守”这件事可以练：写下自己的不可越线清单（安全、合规、隐私、数据真实性），先划线才谈得上守线；每次取舍主动留痕；再靠机制兜底——把评审和记录制度化，不靠个人自觉。

它的边界也要划清：**不是规章清单**（规则背不完，压力时刻判断要在场）；不是风险管理（风险管理算概率，伦理定立场——概率再低，红线也不能越）；不是服从组织（对公众负责，不是对老板负责）。

> **判据**：压力面前，我会不会为进度悄悄牺牲可靠性？利益相关方知情吗？有记录吗？

缺了这一项会怎样？会出事。而且它缺位时，能力越强、破坏越大。

### 9.2.8 工程师干的活：从需求到判定

前面几章各交出一件事。它们不是并列的几件，是第 3 章那条路径按次序走一遍：定需求 → 出规格 → 判满足。这条路的起点不是一份现成的规定需求，是利益相关方的需要与期望（ISO/IEC/IEEE 15288:2023 §6.4.2）。本章把”评审、验证与确认”简称**判定**。

| 他干的活 | 它是什么 | 来自哪几章 | 用得上哪几项能力 |
| --- | --- | --- | --- |
| 需求 | 从利益相关方的需要与期望出发，把它们开发成能当依据用的规定需求，再管住它一直是那一份 | 第 3、4、5 章 | ②方法论素养、①科学素养、⑤工程判断 |
| 设计开发 | 由需求到方案的逐对生成，把依据做成能实现的结构 | 第 3、6 章 | ②方法论素养、①科学素养、⑤工程判断、③实践能力 |
| 产品数据 | 这条路径上造出来、又被查着的东西，你亲手造的就是它 | 第 6、7 章 | ③实践能力、⑥可复现意识 |
| 评审、验证与确认 | 拿证据判”对没对”，三次判定各对自己的对象 | 第 8 章 | ⑤工程判断、④职业伦理 |

（评审、验证、确认三条同属 ISO 9000:2026 §3.11 一组，组内根术语是 §3.11.1”确定”；本章要用的就是这三条。）

这几件事之间的交接，各章都立好了，这里只点名：

- 需求这一段内部，是批准 → 基线：开发负责做出来，管理负责让它一直是那一份（第 4 章）；
- 需求交到设计开发手上：那份规定需求就是设计开发的输入（第 3 章）；
- 设计开发交到产品数据手上，是”上一份的叶节点，就是下一份的需求”（第 6 章）；
- 产品数据交到判定手上，是三次判定各对自己的对象：目标、规定需求、针对特定预期用途或应用的需求（第 8 章）。

> **术语注**：“评审对目标、验证对规定需求、确认对预期用途”这一句，是本书对 ISO 9000:2026 §3.11.2、§3.11.12、§3.11.14 三个定义的凝练，不是任何标准的原文。

到这儿，第 2 章给了能力，第 3 到第 8 章给了活，都点到人身上了。它们最后都落到同一件东西上：产品数据。§9.4.7 会用它把这份清单量一遍。

### 9.2.9 清单不挑领域：变的只是权重

同样一份清单，换一个领域还成立吗？

系统工程、软件工程、硬件工程，领域的工程特征不一样。系统工程是元工程：整合部件、管接口、做系统级验证，自己不造部件。软件工程把工程化落在过程上：版本控制、持续集成、评审、测试链，每

一步要求可重复、受控、可验证。硬件工程的物理约束最硬：力学、材料、制造、安全裕量，任何一处算错都要重新开模。

而清单是同一份，哪个领域都用它，一项不少。活也一样：设计开发在软件工程里落在过程上，在硬件工程里落在图纸与工艺上；需求、设计开发、产品数据、判定，哪一样都不缺。

区别只在权重。系统工程最吃科学素养与工程判断——接口管理、涌现意识，“部件全对不等于系统对”，靠的是这两项。软件工程的主战场是方法论素养与实践能力——过程不健全，代码写得再好也难以为继。硬件工程的红线是科学素养与职业伦理——安全裕量不是参数，是红线。

一句话：系统工程管”缝”，软件工程管”过程”，硬件工程管”物理”；清单一样，只是配比不同。本书的两个案例团队也正落在这条线上：冰链造有形的箱子，恒信做无形的系统。

> **判据**：换个领域，这份清单还成立吗？

### 9.2.10 沟通与团队：执行手段

清单有一个看起来的缺口：它没有单列”沟通能力”和”团队协作”。

这个缺口是真的。ABET 把沟通与团队列为独立的工程师成果（文字转引自报告库 11 号 §7.2；库内无 ABET 正本）。工程教育界几十年补不上的，也正是这一块：雇主抱怨最多的，常常不是技术，是”说不清、合不来”。

但本书不把它们另立一项。原因是：它们不是另一项能力，是已有几项能力的执行手段。

- 职业伦理要让利益相关方知情——靠沟通；

· 工程判断要把取舍讲清楚、让判断被记录——靠沟通；

· 可复现意识要让方法被接住——还是靠沟通。

一个判断做得再对，说不清楚，等于没做；一套方法写得再好，不主动沟通，没人会用。团队协作同理：工程师几乎从不独自交付，他的能力要通过接口、评审、交接与他人接上。

怎么练？第一句动作是把你的一次取舍讲给别人听，看他能不能复述出你的依据。他复述得出的部分，是已经传出去的能力；复述不出的部分，还留在你身上。

> **判据**：我的判断讲出去，别人接得住吗？

对模块工程师和系统工程师都一样：沟通是让能力落到别人手上的方式。把它当可选项，前几项就永远只长在一个人身上。

## 案例进展①｜水忠：手艺的极致

冰链的水忠，手艺是第 2 章量过的那一份：判断全在他脑子里，不在纸上。这一章看另一面：这份手艺在团队里留下了什么缺口。

这几个人，都绕不开水忠那句”心里有数”。李旭要做验证，翻遍台账找不到成文的放行判定口径，只能拉着水忠现场口述”哪几项测、容差多少”；逸人排产，放行单卡了三天。

规程写着”依放行判定口径判定”，可口径在他脑子里，不在台账里；他问遍测试、生产，产线照样转，却没人敢替他签这个字。

他只好拨通水忠电话。

“这批 M3 周二出货，放行的判定口径还没进文档，你不在，谁敢签？”

电话那头，水忠一句句报：哪个批次降额、哪家供应商得多留三天老化。报完，他自己接了一段：

“我报得出，可我说不清换个人怎么接。哪批降额、哪家多留三天老化，我能给你数，凭哪条原理、边界划在哪，我也说得上来；可要问换个人照这套判断做得出做不出——我答不上来。这些年我是试出来的，没记过。按判据说，这不是可复现的判断，是手感。”

逸人一边记一边想：这些判断只有水忠给得出，换个人连依据都问不出来；他要是走了，这条产线也跟着停。

文正主持评审，最后总要追问一句”水忠，你确认？“；董勐盯实时数据，读数一不对，第一个电话就打给水忠。

按这份清单量：水忠的科学素养顶配（他懂原理）、实践能力顶配、工程判断顶配（每个取舍都准）。但他缺方法论素养——他的方法说不成体系；缺可复现意识——他的判断换个人做不出来。

水忠是①③⑤顶配、缺②⑥的人——手艺的极致，不是工程师的画像。他不是不会做，是做的能力不可复制。这里先给水忠挂一句判词：清单第一次落到人身上，量出的是”有里子、缺面子”——里子是做对与守住的那几项（①③④⑤），面子是别人接不接得住的那两项（②⑥）。

把他放进粒度谱系看（§9.3.6），水忠的岗位是**模块工程师**——嵌入式硬件工程师，负责 M3 温控箱的**温控硬件与固件**这一个元素。而他在做的很多事已经是系统级的事：管 M3 整机的追溯、拦下会破坏数据所有权的直连、把老规矩写下来。**模块工程师的岗位，系统工程师的活**——这句先记下，§9.4 再展开。

---

## 9.3 工程师不是什么

上一节把清单立起来了。这一节办另一件事：把”工程师”和挨着它的东西分开。

先往外分：技术员、科学家、手艺人、会写代码的人。再看名义：头衔、证书、年限。最后往里划一道：工程师这个名字底下，不止一种人。清单里的能力，要划的是对外的边界；那几件活，要划的是对内的边界——谁在哪一级干、账归谁。

先说边界的分法。磨边界靠反例：正着说”工程师是什么”，容易各说各的；拿一个挨着的东西来比，界线就出来了。下面的每一条，都是同一个问法：它和工程师差在哪一条上。

### 9.3.1 不是技术员

技术员照单执行：操作规程怎么写，就怎么做；判据是什么，就按什么判。

工程师也执行。但工程师在规则没有覆盖的地方做判断。

两者的分界不是学历，是规则留白时谁拍板。技术员遇到表里没有的工况，会停下来问；工程师遇到表里没有的工况，要自己判断、自己负责。

这不是高下之分。一条成熟的产线上，多数工况本来就该被规则覆盖：那正是第 2 章说的”把交付从人身上挪到方法上”。要留神的是另一件事：规则覆盖不到的地方，如果也没有人判断，那里就是空的。

> **判据**：我做的事里，哪些是照单执行，哪些是规则没覆盖、要我拍板的？

### 9.3.2 不是科学家

科学家研究现存的世界，工程师创造从未存在的世界。第 2 章引过冯·卡门那句话，这里把它放到人身上。

科学家问”世界是什么”，工程师问”应该造出什么”。同一个问题，科学家要的是”说清”，工程师要的是”能用”。

失败的样子也不同。科学家的失败常常是可贵的——推翻一个假说，认识也往前走了；工程师的失败是灾难——桥垮了就垮了。

求真和求有用，是两种目标，也是两种物种。

但这不等于工程师可以不求真。上一节的①科学素养，要的正是求真的那一面；它只是服务于”能用”，不服务于”说清”。这两件事在一个人身上可以同时长着，判据却不一样：说得清不等于造得出。

> **判据**：我手上这件事，是在把世界的道理说清楚，还是在造一个还没有的东西？

### 9.3.3 不是手艺人

这一对最容易混，因为两边都”很厉害”。

区别只有一条：**交付依不依赖具体的人**。

手艺人靠个人经验与技巧，结果绑在人身上——水忠在，质量在；水忠不在，质量跟着走。工程师靠可复现的方法，结果能脱离具体的人——换个人，用同样的需求和约束，能交付质量相当的成果。

第 2 章那条试金石，到这里落成一条角色判据：

> **如果你不在，交付还能不能维持？能，你是工程师；不能，你是手艺人——哪怕你也会写代码、画图纸。**

手艺不是贬义词。水忠几十年的经验是团队最值钱的资产。但”个人厉害”和”工程厉害”是两回事：前者依赖人，后者可复制。

> **判据**：我这一摊活，我不在的时候，还转不转？

### 9.3.4 不是”会写代码的人”

这是最流行的一对混淆，区别只有一条：**写代码是工程师的载体，不是工程师的判据**。

一个会写代码的人，可能只是把需求翻译成程序的手艺人：判断留在规则里，代码写成什么样全凭个人手感。工程师也写代码，但他在写之前问”为什么这么设计”，写完问”换个人能维护吗”，上线前问”这一改会不会破坏别的”。

载体人人可以有，判据不是人人有。把”会写代码”当成”工程师”，就像把”会用刀”当成”厨师”——刀人人会用，火候不是人人有。

> **判据**：我做的事里，有多少是载体，有多少是判断？

### 9.3.5 不是头衔、证书、年限

回到章首那场面试。头衔是组织发给你的位置：可以被印上，也可以被收走。证书是社会给你的认证：它证明你通过了一次考试，不证明你天天在做工程。年资是时间的累积：干了二十年，也可能二十年都停在同一套手艺里。

**名义不定义能力**。头衔、证书、年限合起来，都回答不了章首那个问题。

判断一个人是不是工程师，可检验的不在名片上，在这份清单上。而且不是看他能不能背出清单，而是看他的判断换个人接不接得住、压力面前守不守得住底线。

> **判据**：我认一个人是工程师，凭的是他的名义，还是他的清单？

### 9.3.6 不止一种角色：专责与终责

最后一道边界在内部。“工程师”这个名字底下，不止一种人。工程实践横跨不同的尺度：有人负责系统的一个元素，有人负责整个系统，有人负责系统的**架构**（基本概念或属性与管控原则）与它的守持。它们的大小差着量级，责任也不一样。这条尺度，本书叫**粒度**。

| 角色 | 负责什么 | 责任性质 |
| --- | --- | --- |
| **模块工程师** | 系统元素（system element，15288:2023 §3.47）或模块的实现 | 专责——划得清的账 |
| **系统工程师** | 系统全生存周期的技术事务 | **终责——甩不掉的账，不可委托** |
| **系统架构师** | 架构（基本概念或属性与管控原则）的成立与守持 | 专责——可委托、可追责 |

先说责任性质，这是最硬的一条分界。

系统工程师对”系统成败”负的是**终责**（accountability）：出了事他兜底，这本账不可委托给任何人。模块工程师和系统架构师对”自己那一块”负的是**专责**（responsibility）：这本账可以委托出去，也

可以被追责。一句话：专责≠终责——终责是甩不掉的账，专责是划得清的账。

架构师把架构搞砸了，追责追到架构师；但对外向客户、向监管扛雷的，是系统工程师。

再说分形。模块工程师不是”小一号的系统工程师”那么粗：他是系统工程师的分形：在模块这一级，他跑的是与系统级同构的完整闭环。系统级走的是”需要与期望→规定需求→架构→设计→实现与集成→验证与确认”，模块级走的是”模块需求→模块设计→模块实现→模块验证”。站在自己那一级，他什么都要管，一样不少。报告库 13 号因此把模块工程师叫”迷你系统工程师”。

但分形有一处不对称，恰恰是最要紧的地方。

模块级比系统级少的是这几样，不是遗漏，是结构：**对外的那部分架构**、确认、部署、集成方案。对外的那部分架构说的是，模块的边界、接口与职责来自上层架构的分配，模块工程师不能改；确认要落到真实或模拟的使用条件下，由系统级来兜；部署与集成方案都发生在系统级。

少掉的这几样在别处；归他自己的那一块也很清楚：模块**自身的架构**——它的基本概念或属性与管控原则（内部结构、内部依赖、内部接口、实现策略是实现它们的载体）——由模块工程师自己定义。在自己的边界内，他就是模块这一级的架构师。边界内自治，边界外听架构的。

这里补一句，免得和另一种说法相撞。报告库 18 号把工程师描述为”在单一学科内把分配到的需求做成部件/子系统的执行者”。从

学科分工那一面看，工程师是把分配到的需求做成部件的人；从闭环那一面看，模块这一级又是完整的一圈。两种说法说的是同一件事的两面：模块工程师不必跨学科整合，但在他自己那一级上，需求、设计、实现、验证一样不缺。

系统工程师是把系统工程落到项目里的那个人。他不是头衔，是角色。按 24765 的角色词条模式，系统工程师指实践系统工程学科的人，与其正式头衔无关。这一句是类推：24765 库内无正本，据报告库 13 号 §2.1，所本为同书 systems security engineer 词条的句式。15288:2023 §3.50 给的是”系统工程”这一途径的定义，正本逐字 transdisciplinary and integrative approach。你叫架构师还是 CTO，只要在实践系统工程，你就是系统工程师。

他的职责，报告库 13 号收成这样几个动作：转化（把需要、期望与约束翻成规定需求，再导向方案）、统筹（跨学科整合，让各专业域协同成一个系统）、把关（证明”做对了”且”做了对的东西”）、守护（让系统在生存周期内持续正确、可控地演化）。这几个动作是本书的凝练，不是标准的原文。

同一副骨架，粒度不同。这份清单，哪一种粒度都要用，一项不能少：模块工程师要科学素养，系统工程师要，架构师也要。区别不在能力种类，在粒度与责任——模块工程师把它用在”我这个元素”上，系统工程师用在”整个系统”上，架构师用在”**架构的成立与守持**”上。

系统架构师是这条谱系上的另一个角色——工程师角色体系中专责”架构定义”的那一个。这一章先把他放进谱系里点个名，第 18 章再专门把他从头到尾量一遍。

> **判据**：我背的账，是专责还是终责？我清楚自己站在哪一级吗？

## 案例进展②｜万辉：伪工程

恒信的万辉，团队上了敏捷、上了持续集成、上了测试，方法论和工具都在。

但按这份清单量：他有②方法论素养、③实践能力，缺①科学素养——容量评估被砍，他不追问”凭什么能行”；缺④职业伦理——为进度悄悄牺牲可靠性，没让任何人知情。更致命的是，持续集成那套东西依赖张三调试的三条隐藏命令、李四记得的两个环境变量、王五没写进文档的部署顺序——⑥可复现意识同样站不住。

这些缺项，最先落到他身边每个人头上。

容量评估被砍那天，小白问过一句。

“不做容量评估，月末高峰顶得住吗？”

万辉摆摆手：“先上再说，容量评估下次再补。”

黄程直到上个月那场事故——数据库连接池耗尽、合同审批系统宕机四十分钟、法务和财务几百个审批卡住——才知道评估早被砍了。他把这句话补完：

“我不管你们的工具链怎么搭。我就问一句：卡死那天，谁提前告诉过我评估被砍了？容量评估问的是’凭什么能行’，脚本只在他那台机器上跑得顺，问的是’换个人接得住吗’——按判据算，一笔缺的是科学素养，一笔缺的是可复现意识；没让利益相关方知情，缺的是职业伦理。”

玉杰的委屈在发版。上线那晚，部署脚本在万辉那台机器上跑得顺，换台机器就缺三条命令和两个环境变量——那套环境是万辉亲手搭的，可跑通它要什么，分记在三个人手里，没进任何文档；玉杰翻遍 wiki 也问不出个所以然。万辉一出差，发版就只能等他回来，或等他远程补一句口令。玉杰嘀咕：“这哪是持续集成，是他一个人的发版。”

万辉是有②③、缺①④⑥的人——方法论和工具都在，人这一侧却站不稳；这就是伪工程：形状在，底座不在。

他说的和做的对不上。判词从”初挂”推进到”为什么缺”：他有面子的那一半——方法能说清；复现那一半却站不住。里子缺的是①科学素养与④职业伦理——关键时刻护不住。

把他放进粒度谱系看，万辉的名片上印着**架构师**——谱系里的系统架构师。而一个架构师，恰恰最不能缺底座：系统级的判断全部长在这份清单上，底座缺一块，增量全是空转。**他缺的不是画图，是底座**——这一笔，第 18 章接着算。

---

## 9.4 工程师怎么做成

边界划清了，剩下的问题只有一个：从哪一步开始，按什么次序走。

这一节给出一条路线：路上有几道关口，先过哪一道，每道关口上做什么、怎么算过了；走岔了会是什么样子；最后落到一件看得见的东西上。

路线是这样：先能亲手做出来；再过”从我会做到别人会做”；再过”从照做到会取舍”；最后过”知道什么不能牺牲”。这几道关口过

完，一个工程师就立起来了。再往前走，还有一条横向的路：把责任范围从模块扩到系统。

### 9.4.1 亲手做出来

清单上的每一项，都要从”做”里长出来：科学素养要在做里被验证，方法论素养要从做过的做法里提炼，工程判断要在真实的取舍里练。不上手，前几项都长不出来。

所以起点不是读书，也不是听课，是上手。而且起点不必高——照猫画虎做出第一件东西，也算站在起点上了。第③项实践能力那条强度光谱（阅读、看演示、跟做、仿做、独立做、救火）说的就是这一站的深浅：先站上去，再往里走。

这一站最常见的错，是拿”学过”当”做过”。学过的人会说，做过的人能拿出来。

> **判据**：我最近一次亲手把东西做出来，是什么时候？

### 9.4.2 从我会做到别人会做

这道关口上，人的分岔最明显。有人停在这里，成了手艺的极致；有人过去了，成了工程师。水忠就停在这里——不是他不会做，是他的做没法被人接住。

过关靠的是同一件事的两面。

**写下来**：把你做对的那件事写成方法——步骤、依据、坑。这是②方法论素养。**交出去**：写完主动交给别人用，看他能不能照做。这是⑥可复现意识。

只写不交，方法躺在文档里；只交不写，接的人还得靠问。两样都做完，你的能力第一次离开你本人——它不再只长在你身上。

这也是水忠那句”心里有数”的分水岭。判断在心里，是手艺；判断在纸上、别人照着能做出质量相当的成果，才是工程。

> **判据**：如果我明天走人，团队还能交付同等质量吗？

### 9.4.3 从照做到会取舍

新手问”这一步怎么做”——照单执行；熟手问”这一步有几种做法”——开始会选；资深问”这里要不要再选一次”——会权衡；工程师问”这个取舍守不守得住”——知道什么不能牺牲。判断的依据从”怎么做”升到”守什么”，物种才从执行者变成工程师。

这条上升的线，底下垫的是①科学素养和⑤工程判断。**①科学素养**——会取舍的前提是知道依据成不成立，不然取舍凭的是手感。**⑤工程判断**——在约束里做对的取舍，还要能担责。

只升尺度、不补依据的人，会变成”很有主见但说不清依据”的那种人；只补依据、不升尺度的人，会一直停在”照做”上。两样都得动。

> **判据**：规则没有覆盖的地方，我能做对的取舍吗？

### 9.4.4 知道什么不能牺牲

前面几道关口练的是”能做事”和”会取舍”，这一道练的是方向。判断越准、能力越强，方向错了破坏越大，这就是能力不中性的意思。

这一站的标志很朴素：**第一次为一件”做了也没人反对”的事说不**。那个”不”的第一个对象，常常是自己的冲动——“这个功能月底要上，容量评估下次再补”，说不；“先上再说，反正没人查”，说不。

说不是动作，护住是结果：利益相关方知情，有记录，有机制兜底。一个人是不是真的过了这一站，不看他说得多硬气，看他护住的那件事有没有留下痕迹。

> **判据**：我上一次为”没人反对的事”说不，是什么时候？利益相关方知道吗？

### 9.4.5 从模块工程师走到系统工程师

几道关口过完，一个人已经是一个工程师了。再往前走，有两条方向正交的路。

一条是把责任范围往外扩——从模块工程师走到系统工程师；另一条是往管理走——技术经理、项目总监。两条没有高低，只有方向不同。这一章讲前一条，因为它是本书的行程；后一条也正当，只是不在这本书的路线里。

拓宽不是技能叠加，是视角换轨加责任接管：从对”我的模块”负责，切到对”整个系统”负责；优化目标从”单点最优”变成”系统最优”；而系统最优，常常要求某个模块让利。

把视角摆在一起看：

| 维度 | 模块视角 | 系统视角 |
| --- | --- | --- |
| 关注对象 | 我的模块内部：内聚、性能、实现 | 模块之间、系统与环境 |
| 完成标准 | 模块验证通过，任务完成 | 验证加确认，看整个生存周期 |
| 协作方式 | 接口按文档走，其余不管 | 涌现行为、系统级权衡 |
| 优化目标 | 单点最优 | 系统最优（可以牺牲单点） |
| 责任 | 专责，元素实现 | 终责，系统成败 |

换轨有几个低成本的入口：主动认领跨模块任务（集成测试支持、跨模块缺陷的根因分析——被迫同时看多个模块和接口）；去需求评审现场坐着，哪怕旁听，看”我的模块需求”怎么从”系统需求”分配下来。还有一个入口：把”验证”升级成”验证加确认”，主动问一句”模块验证通过，但系统真的做对了吗”。

换轨有一个标志，很朴素：第一次因为系统牺牲了自己的模块，还觉得值。反过来，如果永远把”我的模块”护在最前，那只是模块工程师做得久，不是系统工程师长得成。

要分清两件事：**升职是组织给你换位置，拓宽是你自己换轨道**。被任命的粒度，也不等于实际承担的粒度：水忠的名片是模块工程师，干的是系统级的活；反过来，挂着系统工程师头衔只做单点开发的人，也不是系统工程师。**粒度看的是你背的账，不是名片**。

> **判据**：我愿意为系统牺牲自己的模块吗？我是在单点最优，还是系统最优？

### 9.4.6 从失败看缺项

路线不只往长成的方向走，也可能走岔。工程师走岔的样子，常见的是这几种。

只会做，不会说。活干得漂亮，判断全在心里，从不成文。交付依赖他一个人，他一休假，产线就停转。缺的是⑥可复现意识。

只会说，不会做。方法论讲得头头是道，流程、规范、最佳实践张口就来，但落不了地。会做的人判断在手上，这种人的判断在嘴上。缺的是③实践能力。

守不住底线。症状不是”不会做”，是”做的时候没守住”。泰坦尼克号按”不会沉”配的救生艇，挑战者号在工程师的反对声中升空，737 MAX 的安全需求在进度与市场的压力下没有被认真对待，一路让了位；这些事故的共同点不是不会做，是守不住（事故经过见第 2 章 §2.3.3）。缺的是④职业伦理。

能力失控。最危险的一种，因为他看起来最优秀：能力越强、这一项越缺，方向错了破坏越大。缺的仍是④职业伦理那一项，只是败法不同：不是守不住，是方向偏了。

每一种都有可观察的标记：看只会做的，看他休假时产线停不停；看只会说的，看他说的能不能换成任何一个具体的决定；看守不住底线的，看他上次为进度砍了什么、利益相关方知不知道；看能力失控的，看他最得意的那个决定方向对不对。

这几种死法死于不同的地方——说不清、做不了、没底线、方向偏——但缺的是同一份清单上的空格。

> **判据**：我最近一次失手，缺的是哪一项？

### 9.4.7 拿产品数据检验

路线走完了，最后一个问题：凭什么说一个人真的长成了。

清单上的能力长在人身上，看不见摸不着。面试会看走眼，领导会凭印象打分，连他自己也容易高估。能力要能被检验，就必须落到东西上。

落到什么上？产品数据。第 7 章立过：开发产品就是开发产品数据；第 6 章那棵树的叶子，就是产品数据。看一个工程师，不看嘴上说

得多满，看他能不能开发出高质量的产品数据——换个人拿着它，做得出、接着种得动。

每一项能力，都有一条别人看得见的到达证据：

| 能力 | 到达证据（谁能看见、看见什么） |
| --- | --- |
| ① 科学素养 | 你留下的预测对账记录：改动前写的预测、实际偏差、以及”哪条原理越了界”的结论 |
| ② 方法论素养 | 别人照你写的步骤做，结果稳定；他卡住的地方，就是你还没显性化到位的地方 |
| ③ 实践能力 | 最近一次你亲手落地的东西在跑——不是你说过，也不是你评审过 |
| ④ 职业伦理 | 你为一件”做了也没人反对的事”说不，且利益相关方知情、有记录 |
| ⑤ 工程判断 | 一个规则没覆盖的取舍：选了、写了依据、有人知道 |
| ⑥ 可复现意识 | 你休假两周，产线不停 |

他干的活也有：

| 他干的活 | 到达证据（谁能看见、看见什么） |
| --- | --- |
| 需求 | 一份规定需求：明示、可采（能当依据用）、三类内容齐全，而且进了基线 |
| 设计开发 | 一棵树：每条需求落到一个能负责它的节点上 |
| 产品数据 | 一份可验证、如实规定、换个人能接着种的产品数据 |
| 评审、验证与确认 | 三次判定各对其对象（本书凝练，见 §9.2.8），有证据、有记录 |

有一件事要说清，免得看起来数了两遍：产品数据既是这条路上的第三段，也是整份清单的物证。它既在路上被造出来，又被拿在手上量这份清单；两重身份，同一件东西。

按粒度分账，各人负责的那一套产品数据也不一样。这里用报告库 13 号 §7.1、§7.2 与 §7.3 的口径（本书的工作清单，不是标准的条文）：

| 粒度 | 负责的产品数据 | 与其他粒度的关系 |
| --- | --- | --- |
| **系统架构师** | 架构描述（两样内容：基本概念或属性与管控原则） | 主责；模块级边界由它分配 |
| **系统工程师** | 利益相关方需求、运行概念、系统需求、系统详细定义、技术方案、实现方案、集成方案、集成测试、验证、确认、部署、运维等系统级工件（不含架构描述） | 主责系统级；**批准**模块级 |
| **模块工程师** | 模块需求、模块详细定义、模块实现方案与实现报告、模块测试、模块验证、模块运维 | 主责模块级；对外边界听架构的，系统工程师批准 |

系统工程师是唯一对系统成败负终责的角色：系统级那一套几乎全由他主责，模块级那一套全由他批准；模块工程师干活，他验收。这就是终责的落点：谁负责的东西最多、又要给别人的东西签字，谁就是那个兜底的。

**粒度隔离就是模块化的意义**（报告库 13 号 §7.3 的凝练）：系统工程师不需要知道模块内部的细节，只需要模块的接口和验证结果；模块工程师在自己的边界内自治，把成果交上去。

还有一件事要分清：**产品数据以规定为主，不是记录**。它写在实现之前，声明”产品应该是这样”，被采购、制造、测试、运维照着执行；记录写在实现之后，只在被审计、复盘时查阅。两者都在产品数据里，但工程师的看家本领是前者：把”应该是这样”规定清楚，让别人照做，让验证有标尺。

所以，看一个工程师，不看他说得多满，只看他能不能开发出高质量的产品数据。能力再强，开发不出高质量的产品数据，这个工程师也不成立。高质量不是写得多，是叶子站得住：可验证、如实规定、换个人能接着种。

> **判据**：换个人拿着我负责的那套产品数据，能不能把产品做出来、继续演进？

## 案例进展③｜水忠与万辉：都还在长成

那么，谁是工程师？

**水忠**：判断是顶配，但不可复制。他有工程师的里子——做对与守住的那几项都在场；缺”换个人能接住”的面子——表达与复现都不在场。他是模块工程师的岗位，做着系统工程师的活，缺的是把一切写成方法的②⑥。

**万辉**：方法能说清，但守不住底线。他有工程师的面子——方法能说清；复现那一半却站不住；缺”关键时刻护住”的里子——①科学素养与④职业伦理不在场（③实践能力、⑤工程判断在场）。他的名片是架构师，缺的却是架构师最不能缺的底座。

两个人都不是完整的工程师。但两个人都值得被叫一句”在成为工程师的路上”, 因为他们都在用这份清单量自己, 都在补自己缺的那一项。

**判词和上一节量的是同一句话**。水忠缺的那一半，正是”换个人拿你的产品数据做得出、接着种得动”那一半；万辉缺的那一半，是”产品数据在，却不可信”那一半。一句话落到两个人身上，量出两种缺口。

### 9.4.8 认知反转

回到章首那场面试。会写代码的、有证书的、干了二十年的，都答不上徐漾那个问题。

现在我们知道，那个问题问的是**可复现**：换个人能交付吗？三个人都答不上来，因为他们都把”工程师”定义在了自己身上：会写代码的人、有证书的人、有经验的人。

定义在自己身上的, 都不稳; 定义在能力上的, 才稳。而能力最终只被一件事检验: 你开发出的产品数据; 换个人接着种得动, 才叫稳。

这一章还多打碎了一个定义: 把”工程师”定义在一个名词上, 同样不稳。工程师不是某一个人的称呼, 是一种统称; 同一种统称底下, 站着好几种粒度。

把模块工程师、系统工程师、系统架构师放进同一句话里叫”工程师”，就像把树叶、树干、树根都叫”树”——都是树，各管一段，账不是同一本。理解了粒度，你才不会拿模块工程师的尺子去量系统工程师，也不会拿系统工程师的账去问架构师。

人们把一个本该由能力定义的东西，交给了载体、头衔和年资去定义——这就是”工程师”这个词的全部误解。

这本书要带你走的，是从工程师到架构师的路。这条路的起点，就是这一章量的这份清单。

---

## 小结

这一章把前面七章收拢到一个人身上：工程师首先是一份能力清单。能力来自第 2 章那几层，各长一项，另加横切的工程判断与元的可复现意识；活来自第 3 到第 8 章，需求、设计开发、产品数据、评审验证确认，按第 3 章那条路径依次走一遍。这份清单是模块工程师、系统工程师、系统架构师共用的底座。

它划清了外面的边界：技术员照单执行，科学家求真，手艺人依赖个人，会写代码只是载体，头衔、证书、年限都不是判据。它也划清了内部那道边界：同一个统称底下，账不是同一本，模块工程师负元素实现的专责，系统工程师负系统成败的终责（不可委托），系统架构师负**架构的成立与守持**的专责；模块工程师是系统工程师的分形，而分形不对称。

它给出一条路线：先亲手做出来，再过”从我会做到别人会做”，再过”从照做到会取舍”，最后过”知道什么不能牺牲”；再往前走，是从模块工程师到系统工程师的拓宽，而拓宽是换轨，不是升职。走岔的样子也给了，每一步的到达证据也给了。

最后，它把能力从”人身上”挪到”纸面上”：能力最终体现在能不能开发出高质量的产品数据——换个人拿着它，做得出、接着种得动，能力才成立。

下一章，我们看全书概念的重心——架构：先把这个词说准——给它一个能对账的定义，并清掉几处最贵的误解。架构师，就是在这个底座上长出系统级判断的人（第 18 章会给全那份清单）。

散会时，黄程说了句实在话。

“招工程师，我就认一条判据——换个人，这活也接得住。水忠的判断换个人接不住，万辉的’先上再说’换个人也守不住：一个缺’别人也会做’，一个缺’守住底线’。这两笔账，最后都在产品数据上看得见。”

万辉没接话。他知道，这句自己也答不上来。

# 附录

## A 本章概念总图

工程师：一份能力清单（不是头衔、证书、年限）

一条判据（全章唯一）：

换个人、换团队，用同样的需求和约束，能不能交付质量相当的成果

├─ 清单里的能力（问"我有没有"；源＝第2章那几层）：

│ ① 科学素养（凭什么能行） ② 方法论素养（把工作组织成方法）

│ ③ 实践能力（亲手把方案变成现实） ④ 职业伦理（能力不中性，它定义方向）

│ ⑤ 工程判断（横切在上面） ⑥ 可复现意识（元能力）

├─ 清单里的活（问"我会不会干"；源＝第3到第8章，按第3章那条路径依次走一遍）：

│ 需求（从利益相关方的需要与期望，开发成能当依据用的规定需求；第3、4、5章）

```
   │        → 设计开发（由需求到方案的逐对生成；第3、6章）
   │        → 产品数据（这条路径上造出来的东西；第6、7章）
   │        → 评审、验证与确认（拿证据判"对没对"；第8章）
   ├─ 侧面：粒度（在哪一级被检验）／熟练度（照做→会选→会取舍→守底线）
／
   │          走岔的样子（只会做／只会说／守不住底线／能力失控）
   └─ 物证：产品数据（换个人拿着它，做得出、接着种得动）——
                它既是这条路上的第三段，也是量整份清单的那件物证
```

一份清单从哪来：能力＝第2章那套分层（工程科学／方法论／工具与实践／价值与伦理）各长一项，

另加横切的⑤与元的⑥（以 ABET 工程师成果七条对照凝练，本书主张）；

活＝第3到第8章，次序由第3章那条路径给定

清单不是什么：不是技术员／不是科学家／不是手艺人／不是会写代码的人／

不是头衔、证书、年限

同一个统称，不同的账：模块工程师（专责）／系统工程师（终责·不可委托）／

系统架构师（专责）；模块工程师是系统工程师的分形，

分形不对称（模块级没有对外的那部分架构／确认／部署／集成方案，

模块自身的架构归自己）

怎么长成（一条路线）：

亲手做出来 → 从我会做到别人会做 → 从照做到会取舍 → 知道什么不能牺牲

→ 横向的路：从模块工程师走到系统工程师（换轨标志：第一次为系统牺牲自己的模块还觉得值）

→ 证据：产品数据（按粒度分账）

往前追：能力来自第2章，活来自第3到第8章。

往后落：可复现意识长成第13章的基线，职业伦理长成第14章的治理；

第18章再加上系统级的增量，合成架构师的一份清单。

判词：水忠①③⑤顶配、缺②⑥（接不住）｜万辉有②③、缺①④⑥（守不住）：都还在长成

下一章：说准"架构"——定义与误解

## B 本章的能力级

**第 ③ 级为主**。

- **表征**：把”工程师”和挨着它的词说准、分开——技术员／科学家／手艺人／头衔、证书、年限／模块工程师、系统工程师、系统架构师／终责与专责。
- **图式**：一份能力清单的构造——能力（分层各长一项，另加横切的工程判断与元的可复现意识）＋活（需求、设计开发、产品数据、评审验证确认，按第 3 章那条路径依次走一遍）；同一个统称底下的粒度谱系；模块工程师是系统工程师的分形，且分形不对称。

· **心智模型**：能推演”缺哪一项会走岔成什么样”；能说出这份清单从哪来（能力来自第 2 章那几层，活来自第 3 到第 8 章）；能力不中性、职业伦理定方向；能力最终被产品数据检验；粒度决定账本。

· **解释框架**：本章不建（归第 17、18 章）。

用第 1 章 §1.4.4 的尺子量一下：这一章把工程师这个词从第 ① 级（表征）推到第 ② 级（图式）——清单的构造、统称底下的粒度谱系、模块与系统的分形关系，都成了认得出的结构；再往前一档是第 ③ 级（心智模型）：能推演缺哪一项会走岔成什么样。本章以第 ③ 级为主；第 ④ 级（解释框架）不建，归第 17、18 章。

能答全四问才算拥有——你能拿着这份清单，量别人，也量自己。

## C 自查 · 这些说法对吗？

1. “会写代码的人，就是工程师。”——**错**。写代码是载体，不是判据；工程师在规则留白处判断。
2. “有职称、有证书，就是工程师。”——**错**。证书只证明考过一次试；名义不定义能力。
3. “干了二十年，一定是好工程师。”——**错**。二十年也可能都停在同一套手艺里；可复现才是工程。
4. “工程师就是照着流程做的人。”——**错**。那是技术员；工程师在流程没有覆盖的地方判断。
5. “工程师只要技术好，伦理是管理者的事。”——**错**。职业伦理是清单里的一项，缺了会出事；而且能力越强，破坏越大。

6. “能力越强的工程师越值得信赖。”——**错**。能力不中性，职业伦理定方向。
7. “把经验写成文档，就算工程化了。”——**对了一半**。显性化只是第一步，还要可检验：换个人照着做，质量相当。
8. “我的判断，不需要别人接得住。”——**错**。可复现意识就是让判断被接住；接不住的只是手艺。
9. “工程师的追求是零错误。”——**错**。工程要的是错误可见，不是零错误（第 2 章）。
10. “拿到头衔就是工程师了，一辈子都是。”——**错**。头衔只是别人给你的位置；清单一项项亮起来，才算成为。
11. “从新手到工程师，能力慢慢堆出来就行。”——**错**。判断的依据要换尺度，从”怎么做”到”守什么”。
12. “模块工程师就是小一号的系统工程师。”——**错**。他是同构的分形，但模块级没有**对外的那部分架构**、确认、部署与集成方案——而**模块自身的架构**恰恰归他自己。
13. “系统工程师是比模块工程师更高的职级。”——**错**。粒度不是职级，是责任性质；终责与专责不是高低。
14. “挂着头衔、只做单点开发，也是系统工程师。”——**错**。粒度看的是你背的账，不是名片。

15. “一个工程师好不好，聊一聊就能判断。”——**错**。能力长在人身上，不可直接检验；最终看产品数据——换个人拿着它，做得出、接着种得动。
16. “产品数据就是记录。”——**错**。产品数据以规定为主，记录也在其中；写在实现之前的规定才是它的主干。
17. “清单上有能力就够了，干没干过不重要。”——**错**。能力问”我有没有”，活问”我会不会干”；两样都答得上来，才叫是个工程师。

## D 练习

1. **体检**：用这份清单给”你们公司最像工程师的人”做一次体检——逐项打勾，指出缺项，并说出缺这一项会走岔成哪一种。
2. 粒度定位：把你们团队的核心成员放进谱系（模块工程师／系统工程师／系统架构师），说出每个人背的是专责还是终责——有没有”名片一个粒度、实际另一个粒度”的人？
3. **起点**：你最近一次亲手把东西做出来，是什么时候？如果答不上来，先做这一件事。
4. 第一道关口：把你最近一次做对的事写成一份可交接的记录（步骤、依据、坑），交给一个没参与过的同事照做一次——他卡在哪一步？
5. 第二道关口：写下你最近一次重要取舍——当时哪些约束在打架、选了哪个、依据是什么、后果如何、重来会怎么选。

6. 第三道关口：写下你的不可越线清单（安全、合规、隐私、数据真实性各一条），再找出最近一次”为进度让路”的现场——利益相关方知情吗？有记录吗？
7. **证据**：拿你负责的那套产品数据，问一句”换个人拿着它，能不能做出来、继续演进”——答不出的部分，就是你的缺口。
8. 拓宽自测：你现在的视角是模块视角还是系统视角？你最近一次”为系统牺牲自己的模块”，是什么时候？
9. **清单对表**：把你手上这摊活按”需求 → 设计开发 → 产品数据 → 判定”摆一遍。看哪一段断了，就回头读哪一章。

## E 参考文献

### 标准

· R-103 ISO 9000:2026（第 5 版）—— §3.10.6 competence（ability to apply knowledge and skills to achieve intended results，应用知识和技能以取得预期结果的能力）：能力是干成事的本领，不是背下来的知识点，所以这份清单只能逐项自问；**§3.3.11 设计和开发**（set of processes that transform requirements for an object into more detailed requirements for that object）—— §9.2.3 引它说”一组过程，就是一套方法”（**本书读法**）；§3.11.2／§3.11.12／§3.11.14 评审、验证、确认—— §9.2.8 三条判定的出处；**§3.5.1+注 2**（规

定的需求＝明示的需求）、**§3.11.1 确定**（三判定的根术语）—— §9.2.8”规定需求”与”判定”的锚

- R-102 ISO/IEC/IEEE 15288:2023 —— §3.47 system element（系统元素：系统的离散部分，能加以实现以满足规定的需求）；§3.50 systems engineering（系统工程——一种跨学科、综合性的途径）；**§6.4.2 利益相关方需要与需求定义过程** —— §9.2.8”把需要与期望开发成规定需求”的口径依据
- R-04 ISO/IEC/IEEE 42010:2022 —— §3.2 架构定义（基本概念或属性，以及为实现和演化而设的管控原则）

> **术语注（跨标准用词）**：本书引 ISO 9000 原文处**照正本用词**；本章自撰正文一律作”利益相关方”（与全书一致）。

**经典文献**

- R-15 ECPD 1941／ABET 1979／INCOSE 2019 —— ABET 工程师成果七条；ABET 把工程定义为”运用判断力加以应用”（applied with judgment）（库外来源·库内无正本，文字转引自报告库 11 号 §7.2 与 12 号 §1.1；ECPD 1941 转引自 05 号 §5；INCOSE 2019 转引自 13 号 §2.1②）
- R-44 NSPE Code of Ethics —— 工程师的服务必须致力于保护公众的健康、安全与福祉（**库外来源**·库内无正本，转引自报告库 12 号 §1.1）

- R-69 INCOSE Systems Engineering Vision 2035 —— 系统工程师的能力分组与”从模块走向系统”的能力缺口（**库外来源**·库内无正本，转引自报告库 13 号 §8.2）
- R-20 ISO/IEC/IEEE 24765 —— module 与系统工程师（**库外来源**·库内无正本；本章”系统工程师”一句系按 24765 的角色词条模式类推，据报告库 13 号 §2.1）
- R-09 ISO/IEC/IEEE 15288:2015 —— 历史版本行（**库内无正本**；本章引用一律走 2023，即 R-102）
- R-18 CMMI —— 工程方法论举例（**库外来源**）

> 本章能力清单的回扣链：§2.2.4（试金石）、§2.3.3（分层与价值失守的事故）、§2.3.4（自动化）；第 6 章（那棵树）、第 7 章（产品数据）、第 8 章（三道门禁）；第 10 章（架构）、第 18 章（架构师）；附录 A T-25（工程师）、T-113~T-117（系统工程师／模块工程师／系统架构师／粒度／终责专责）、附录 C 第 9 章行、附录 E 第 9 章块。

# 第 10 章 架构：定义与误解

## 10.1 引言

架构是这个行业里最重的词之一，也是被用得最松的词之一。评审会上有人说”我们的架构是微服务”，共享盘里躺着一份”M3-架构 v1.0”，有人把拓扑图、部署图、ER 图统统叫架构，还有人把”这次改个字段”说成”架构变了”。

这些用法里说的多半不是架构，而是**架构描述**——架构被写下来、画出来、存起来之后的样子。按 ISO/IEC/IEEE 42010:2022 的区分，架构描述是**有形的**工作产品，架构本身**无形、抽象**（该标准 Introduction 逐字：Whereas an AD is a tangible work product, an architecture is intangible and abstract, understood through its concepts, properties and principles.）。两样东西被一个词罩着，误读就从这里长出来。

这一章把架构的定义说准，把这个定义意味着什么讲透，再把贴着这个词长出来的误读逐个清掉。

### 10.1.1 误解现场

架构评审会，冰链科技，周四上午。水忠把 U 盘插上投影，屏幕上是一张图：四层方块，从下到上——硬件层（温控单元、传感器、电池）、固件层、云端层（数据采集、追溯平台）、App 层；一根箭头从传感器一路画到 App，旁边标着”温度数据”。

“这就是我们 M3 的架构。”水忠讲了十分钟，把四层方块过了一遍。“硬件和固件是我这边，云端是董勐他们三个，测试的李旭，验证与确认归他。有问题的现在提。”

产品开发经理永兵指了指那张图：“先记一个问题——客户问’M3 是什么系统’，我们能拿这张图答吗?”

制造侧的逸人举手：“我有个问题。这图里没有’我们怎么造它’——哪道工序测什么、哪个批次装哪个固件?”

水忠：“架构图讲的是系统长什么样，不是怎么造的。”

逸人点点头，没再问。散会。图被存进共享盘，文件名”M3-架构 v1.0”。此后三个月，没人打开过它。

三个月里，三个问题先后找上门。

文正想调需求：“温度数据上传间隔从 1 分钟改成 5 分钟，影响什么?”没人答得上来——图上只有一根叫”温度数据”的箭头，没说采集间隔定在哪一层。

水忠想加”断电补传”，问云端团队：“重连逻辑在哪一端?”云端说这是固件的事。可固件就是水忠自己写的，他知道里面没有重连逻辑。两个团队在图里找依据，图里没有。

第三个问题最贵。文正带 M3 谈一笔冷链订单，客户问：“你们 M3 到底是什么系统?”水忠答”温控箱”，董勐答”追溯平台”，云端那边做 App 的又答”冷链管理 App”。三个答案，没有一个对得上。饭桌上安静了一会儿，那笔订单后来没了下文。

水忠把图翻出来，想维护一版新的。他对着那张图看了很久，忽然意识到一件让他后背发凉的事：**这张图从来没有描述过真正的 M3**。

如果你就坐在这间会议室里，可能觉得他们挺冤——图画了，评审开了，还要怎样?这一章要推翻的正是这个直觉。那张四层方块图在架构领域有个准确的名字：**架构描述**，而且只是架构描述里极粗糙的

一小块。水忠把这一小块当成了架构本身，于是图烂掉的时候，他以为架构也烂了——可 M3 的架构一直活着，只是从没有人把它写下来。

### 10.1.2 这一章要回答什么

读完本章，你应该能回答：

1. 架构到底是什么？为什么”画一张框图”不算做了架构？
2. 权威定义里的每一个词各管什么？两半之间是什么关系？
3. “基本”这两个字凭什么？什么算基本，什么不算？
4. 为什么说架构交不出去？交出去的那件东西叫什么？
5. 元素与关系在架构里住在哪？定义句里为什么没有它们？
6. “改起来贵的东西就算架构”，这句话错在哪？
7. 业务架构、数据架构、应用架构、技术架构，是一个架构还是四个？
8. 用了某个框架，算不算有了架构？

### 10.1.3 这一章的骨架

先把骨架立起来，后面三块都是给它添血肉。

**一句定义**：架构是一个实体在其环境中的基本概念或属性，以及管住该实体及其相关生存周期过程怎么实现、怎么演化的管控原则（ISO/IEC/IEEE 42010:2022 §3.2）。

这句话里并列着两样性质不同的东西：一头回答”它是什么”，一头回答”它必须怎么造、怎么变”。前半是描述性的，后半是规范性的——规范性是说它可以被违反。不理解这个并列，就理解不了这个定义。

**第二句**：架构是抽象的，交不出去；能交付、能评审、能版本化的，是架构描述。日常说”架构”时，一半以上的场合说的其实是描述。

**第三句**：“基本”不是形容词，也不是”改起来贵”；它的判据只有目的——去掉它，这个实体还能达成它的目的吗。

骨架立完，本章分三块走：架构是什么（定义与含义）、架构描述是什么（三个近邻概念的边界）、架构不是什么（把误读逐个清掉）。

---

## 10.2 架构是什么

### 10.2.1 架构这个词的来路

架构不是软件和系统领域的原住民。它最初是建筑学的词，指承重结构、空间布局。软件工程在 1970 年代借用了它：结构化设计把模块划分、层次关系叫”架构”。

1990 年代系统越做越大，“软件架构”成为一门显学，词义也一路漂移。把四站摆出来，能看清漂的是哪一头：

| 时点 | 表述 | 主体 | 界定方式 |
| --- | --- | --- | --- |
| 1996（Hilliard） | 系统在其环境中的**最高层级的构想**（the highest level conception） | 系统 | 偏主观的”构想” |
| 2000（IEEE 1471） | 系统的**根本组织**，体现于其**组件**之中 | 系统 | 把主观构思压成客观结构 |
| 2011（42010 第 1 版） | 系统在其环境中的基本概念或属性，**体现于其元素、关系以及设计和演化的原则之中** | 系统 | 把架构从具体结构上提到概念层 |
| 2022（42010 第 2 版） | **实体**在其环境中的基本概念或属性，以及管住该实体及其相关生存周期过程怎么实现、怎么演化的**管控原则** | **实体** | **删去**”体现于元素与关系” |

（前三站口径经库内 22 号、06 号等报告转引；42010:2011 全文与 1996／2000 两站正本均不在库，属**库外·不可核**，此处只作历史对照。）

2022 版这一站有两个动作值得记住，它们直接决定本章后面几节怎么讲。

一是主体扩域。从”系统”换成”实体”（entity of interest，所关注实体）：企业、组织、产品、过程、数据、应用都可以被架构。系统只是实体最常见的一种形态——本书讲的主要是系统。

二是删去了界定方式。2011 版用”体现于其元素、关系以及设计和演化的原则之中”给架构划界；2022 版把这一句从定义句里**删去**。这是**删去加重定位**：2022 版定义句里不再有它。那一层含义只保留在 42020:2019 §3.3 的注 1 里，措辞由”元素、关系”改成”组件、组件之间的关系、实体与环境的关系”，强度由定义句里的”是”降为注释里的”通常意指”（usually intended to be embodied in）。注释级，没有术语地位。

对读者来说，这一删一降带来一个必须当面交代的后果：元素与关系不再是架构的定义要素。它们有它们的位置，但那不是定义句——§10.2.5 专讲这件事。

> 术语注（本书读法）：本书把这条定义读作”**系统不可还原的涵义**”——**关于系统根本性质的那组基本概念或属性（不是”一组决定”**：决定只是认出并记下它的抓手，正位在 §5.2.12／§6.10）。这个读法是本书的凝练，不是任何标准的原文

> （42010:2022 正本全文没有”不可还原”的对应表述）。它落的是定义句里”基本概念或属性”那一半，管控原则是与它并列的第二样内容，不在”涵义”之内。欢迎检验、欢迎校准。

### 10.2.2 架构是基本概念或属性与管控原则

把定义句逐词读一遍。原文是这样的（ISO/IEC/IEEE 42010:2022 §3.2）：

> **Architecture: fundamental concepts or properties of an entity in its environment and governing principles for the realization and evolution of this entity and its related life cycle processes.**
> 中文（依中英对照版 §3.2 条，语序从中文习惯）：架构——一个实体在其环境中的基本概念或属性，以及用于该实体及其相关生存周期过程的实现和演化的管控原则。

这句话读起来像一串名词堆在一起，其实每个词都压着分量。

“基本概念或属性”（fundamental concepts or properties）是前半句。它回答”这个实体是什么”。架构不关心全部细节，只关心那些”如果改变，就会让这个实体变成另一种实体”的东西——判据在 §10.2.4。

原文用的是”或”（or），不是”和”（and）。中文常译成”概念和属性”，读起来像是必须同时具备的两半。“或”比”和”宽松：可以从”

核心概念是什么”切入（订单是一种交易意图的记录），也可以从”核心属性是什么”切入（数据全程可追溯），两条路都能走到架构。

这个析取式不是随手的并列——它是 42010 工作组有意识的选择，为的是不带偏见地兼容两种哲学（42010:2022 附录 A.2 逐字）：Architecture as concept，架构是存在于人脑中的系统构想；Architecture as property，架构是实体的属性或特性。标准对两派不表态（without prejudice）。

“属于一个实体、在它的环境中”（of an entity in its environment）交代了架构住在哪。现行版的主体是**实体**，不限于系统；而”在它的环境中”这半句常被读丢——同一个温控设备，放在实验室台架上，架构可以是”温度控制单元”；放在冷链运输里，必须包括”断链检测与补传”。环境变了，什么算基本就变了。

“及管控原则”（and governing principles）是后半句，也是这次改版升格上来的一块。2011 版把原则放在”设计和演化的原则之中”，是界定方式的一部分；2022 版把它提出来，与”概念或属性”**并列**，还给它加了一个限定词——**governing**。这个词的分量在下一段。

> 术语注（管控原则，三处同源不同位，混用会同时污染术语表与治理讨论）：- **定语位** governing principles＝**管控原则**，就是定义句里这一块：有约束力、可据以判定违规； - **关系位** a viewpoint **governs** a view＝**治理**／**管辖**，说的是架构视角与架构视图之间的关系（§10.3.2）； - **名词位** governance＝

> **治理**，是组织治理（第 14 章）；它的架构侧义项见附录 A T-118「架构治理」。
> 三处只共用”治理”二字，不是一回事。

“实现与演化”（realization and evolution）带的是两个宾语：该实体，以及它的相关生存周期过程。定义句写的是”该实体及其相关生存周期过程”的实现和演化，架构原则同时约束两件事：东西怎么做成，以及做它的那些过程怎么走。这一处常被读丢，而它是”架构管到退役与替换”的出处：第 13、17 章接的就是这条线。

两半的性质不同，这是本节最要紧的一句话。前半（概念或属性）是描述性的——它在说这个实体是什么；后半（管控原则）是规范性的——它在说这个实体必须怎么造、怎么变。规范性有一个可检验的标志：它可以被违反。第 15 章会拿”该内容能否被设计违反”当筛选判据；本章只记一句：把两半合成一个名词去叫，就会丢掉”能被违反”这一半的锋芒。

> 术语注（**本书判断**：正本内部一处口径不一，引用时须一并交代）：定义句（§3.2）是**两项并列**；而同一标准 §5.2.2 写的是 The architecture of an entity of interest **comprises the fundamental concepts or properties** of that entity considered in its environment——“comprises”只提前半，“管控原则”被排在紧随其后的 can pertain to any or all

> of the entity’s: … principles governing its design, use, operation and evolution 清单里；附录 A.2 谈两种哲学时同样只说“concepts or properties”。**本书一律照定义句 §3.2 写（两项并列）**，此处交代差异，不作调和。

这两半落到纸上，自然就是两份东西。前半是基本概念或属性，写成一份基本概念和基本属性清单；后半是管控原则，写成一份管控原则清单。后一份清单天然分两栏：管”怎么造对”的实现侧，与管”怎么变不散架”的演化侧。这与前面那句”实现与演化带两个宾语”是同一件事的两种说法：在定义句里它是一句话的两个宾语，落到纸上就是两栏。

三处要说清。其一，清单是”表达”，不是”架构”：按 §3.1，架构工作是一串动作——构想、定义、表达、记录、沟通、证明其正确实施、维护和改进——清单是”定义”这一步的产物，不等于定义句所讲的那个架构本身（§10.2.3 划这条界）。其二，“或”不是”和”：原文 concepts or properties 兼容两种哲学，概念与属性不必各列一张，合成一份更贴正本。

其三，清单可以按”方面体”分块：§3.9 的 aspect 是”实体性格或性质的一部分”，标准示例是功能／结构／信息，按它分块是 2022 版明确承认的清单化结构。方面体是标准给的分块维度，与 §10.4.4 那组”关注域视角”不是一回事：前者按实体的性格或性质切，后者按”看什么”切。

清单的用处是对账，不是罗列：列完要能回答——哪些关注点没有条目承接，哪些条目只服务单一关注点且不承重，哪些内容其实可以被设计自由处置。答不出这三问，清单越长，越说明”什么算基本”这条判据已经丢了。（这几份清单怎么长出来，是第 11 章的事。）

### 10.2.3 架构无形，有形的是架构描述

理解了定义的两半，还要理解这句话在说一个什么样的东西。标准的答案很直接（42010:2022 Introduction 逐字）：

> Whereas an AD is a tangible work product, an architecture is intangible and abstract, understood through its concepts, properties and principles.
> 架构描述是有形的工作产品；架构则是无形、抽象的，要通过它的概念、属性和原则来理解。

这句话在标准里的职能是划界：把架构与架构描述分开。它不是在给架构做本体论分类。这一点值得说清，因为”抽象”在中文里至少有三个所指（依 30 号研究报告 §6.1 的三分）：

| 用法 | 说的是什么 | 例子 |
| --- | --- | --- |
| **含义层**：概念／共相 | 词的意思是一般的、不指向某个时空个体 | “服务器”“三角形”“需求”这些概念都抽象 |
| **本体层**：抽象对象与具体对象 | 指称物有没有时空位置、有没有因果力 | 数、命题、类型是抽象对象；这张纸、这台机器、这个团队是具体对象 |
| **粒度层**：省略细节 | 工程口语里的”抽象层次” | 概览图比详图抽象，任何层次都能再抽象一层 |

三层混用会得出一个空洞的结论——“一切术语的指代都是抽象的”。那不是发现，那是把”含义总是抽象的”这个平凡事实当成了本体论结论。更要紧的是：粒度层的”抽象”（＝概略、高层级）恰是本章要清掉的误读之一（§10.4.6）。所以本书说这件事时，不用”抽象物”这个词，改说标准的另一面：架构没有形态，交不出去。

“交不出去”不是修辞，它有三条可当场检验的判据（依 30 号研究报告 §6.3）：

- **可交割性**：系统能交付、验收、上线、停机；架构描述能签收、入库、版本化；架构不能——交出去的从来不是架构。给客户签的那份东西，是架构描述。
- **同一性**：同一个架构可以有几份不同的架构描述——多载体一个内容，这是抽象对象的标志；描述这一侧恰好相反，每一份都是个

别的工作产品、个别可版本化的信息部件（§3.3、§3.14）。**架构的同一性不绑在载体上**。

- **因果力**：架构不导致任何事。它只能靠被理解、被援引、被遵守起作用；系统能算、能发指令、能故障。

“交不出去”还有一面：内容与载体总要分开说。同一个概念常常有两个指称——抽象的那一面是内容，具体的那一面是载体：

| 概念 | 内容（抽象指称） | 载体（具体指称） |
| --- | --- | --- |
| 架构 | 那个架构 | 描述它的架构描述 |
| 关注点 | 利害、立场 | 记录在案的关注点条目 |
| 架构视角 | 那套惯例 | 视角规格（可入库、可版本化） |
| 架构视图 | 所表达的内容 | 架构描述中的信息部件 |

书里后面几章反复用这张表：说”需求”时，指的可能是那个条件或能力，也可能是成文的那一条。分不清内容和载体，就会把”改了一份文档”当成”改了架构”。

最后一条界限要钉住：架构描述是工作产品，架构不是。标准对这两侧各有一个词——架构描述＝work product（§3.3），是可触的工作产品、是提供给利益相关方的信息的有形表示；架构＝intangible and abstract。日常说”我们的架构在这台服务器上”，说的其实是描述。

这套区分顺带解释了一件事：架构压的是复杂度。一个真实的企业系统可能有几百个服务、几百个接口、几十个数据对象，远超任何单个人的脑子；架构做的事是把它压成”十几个业务域、每域几个核心构件”这样一组**可以装进脑子的基本概念或属性与管控原则（图只是**

**它的表达，不是它本身**），于是处理任何一个具体问题时，你只需要关心系统的一小部分。ISO/IEC/IEEE 42010:2022 讲架构工作的动机，用的正是”帮利益相关方管理复杂度”这句。

### 10.2.4 基本是对目的必要且充分

定义里那个”基本”（fundamental）是整句最容易被一带而过、分量却最重的词。它来自拉丁语 fundamentum——地基：地基在最底层，承托一切，一动则整栋楼动。

地基仍是比喻，判据得自己立。而”什么算基本”这条判据，标准没有给。

两部标准都没有回答它：42010:2022 只说架构可以关乎什么（§5.2.2），42030 只说评估对的是规定的评估目标（§1／§3.4）。SEBoK 直接写明 fundamental 这个词”没有普适定义”，详细程度”取决于使用的语境和设计目的”，并要求架构描述”说明何为纳入、何为排除”（均系报告转引，SEBoK 库内无正本，属库外·不可核）。这是一处权威缺口，本书给出的判据是形式化，不是标准条文。

判据的形式是两条，参照系都是**目的**：

| 判据 | 问法 | 管什么 | 失效形态 |
| --- | --- | --- | --- |
| **必要性**（membership，入会） | 去掉 x，目的是否落空? | 成员资格：什么必须进 | **架构缺失**（under-specification） |
| **充分性**（closure，闭包） | x 齐备之后，剩余问题是否都不影响目的的可达? | 闭包：架构到哪儿停 | **架构越界写成设计**（over-specification） |

一处精确化不可省：这一对判据不是数学意义的充要条件。架构不构成目的达成的充分条件，达成还要靠实现、验证、运行；充分性只指一件事：目的所要求的结构条件已被完整表达。

必要性一侧可以落成三句更顺口的问法，三句各问一面，问的还是同一件事：

- 不可再分：去掉它，就不再是它了?
- 不可缺失：缺少它，这个实体就垮掉?
- 不可替代：换掉它，性质就变了?

三句共有一个锚：那条内容是否还服务于这个实体在其环境中的目的。目的变了，“基本”就变了。同一个温控算法，在心脏起搏器里是基本，在计算器应用里只是实现细节。

目的判据要跑两遍。一遍是**兑现侧**——实现这个目的需要它吗；一遍是演化侧——目的变了，它守不守得住。定义句里 “and evolution”就是第二遍的出处，只跑一遍必漏演进类的基本条目（比如”新增传感器类型不得破坏已有追溯链路”这类，只在演化侧才现形）。

判据之上还有一层顺序，不能倒。拿目的筛完之后，可违反性、可引用性、可判定性这几条才依次上场——它们的位次在必要性之后，作用是排序与检查，不是准入：

> ① **目的必要性**（总闸，参照目的做承重测试）→ ② **可违反性**（这条内容能不能被设计违反）→ ③ **可引用性** → ④ **可判定性** → **关注点闸门**（这次写哪些、写多细、给谁看）

这条判据从哪来？从治理推。治理的核心动作是裁量——某条内容合规还是不合规、准还是不准。既然是裁量，就得有一把不随人变的尺。这把尺不能是关注点：关注点互相冲突，而治理存在的理由恰恰是仲裁冲突的关注点。于是尺只能落在**目的**上。一句话说清：“基本”由目的界定，不由关注点界定。

由此得到两道闸门与一类来源：目的与关注点是闸门，环境是来源，各管一段：

| 闸门／来源 | 管什么 | 判据 |
| --- | --- | --- |
| **目的（承重性）** | 条目够不够格进”基本”这一层 | 拿掉它，这个实体还能达成它的目的吗 |
| **关注点（取舍与呈现）** | 够格的候选里，这次写哪些、写多细、给谁看 | 哪些关注点没有条目承接（缺口）；哪些条目只服务单一关注点且不承重（降级） |
| **环境（语境与影响）** | 第三类来源：实体周围的**语境与影响** | 定义中 in its environment——法规、组织与业务环境、物理条件、外部接口、资源 |

这也解释了为什么”监管合规”能进架构，而不是可选项：不是因为它有人在乎（那是关注点层的理由），而是因为它构成实体在环境中达成目的的条件。

一处方向性提醒，本章只记结论：目的的定义权在治理机构，不在架构师（42020:2019 §6.4.3／§6.4.4／§8.4.4 是它的正本位置）。架构师对目的有质询权与守护责，可以问”这条目的还算数吗”，不能自己改写。目的是被悄悄换掉的，架构就悄悄失效了——这是架构失效的第一病因，第 14 章展开。

这条判据还有一个常被忽略的后果：约束即解放。判据筛完，够格进架构的只有”去掉它目的就落空”的那几条；剩下的一大片——某个接口用 REST 还是 gRPC、某个界面怎么排——被判据主动放还

给做的人。所以好的架构不是”管得最多”，而是”管得最准”：约束只落在要命的地方，别处留着让执行者自己拿主意。M3 的产线正是这么干活的（案例进展③）。

### 10.2.5 元素与关系不在定义里

很多讲架构的材料，第一句就给一张图：几个方块，几根连线。方块是元素，连线是关系，看起来顺理成章。可翻开现行定义句，元素与关系根本不在里面——这一节把它们的去向交代清楚。

先说清事实。2011 版定义句里有”体现于其元素、关系以及设计和演化的原则之中”；2022 版删去了这一句。同时删去的还有 42020 的注——42010:2022 §3.2 的 SOURCE 行逐字写着 ISO/IEC/IEEE 42020:2019, 3.3, modified — The notes to entry have been removed.（42020:2019 §3.3 的注已被移除）。

再说它们的去向。那一层含义没消失，它搬到了 42020:2019 §3.3 的注 1，并且降了调。注 1 逐字：

> The fundamental concepts or properties of the architecture entity are **usually intended to be embodied in** the entity’s **components**, the relationships between components, and the relationships between the entity and its environment.

三点变化：措辞由 elements／relationships 改成 **components**；强度由定义句里的”是”降为”通常意指”；地位由定义句降为**注释**——注释级，没有术语地位。

在 42010 这一侧，元素与关系另有一个位置。§ 5.2.2 逐字：The architecture of an entity of interest **can pertain to** any or all of the entity’s——组成元素；元素之间的交互或相互关系；与环境（含环境中的其他实体）的交互或相互关系；行为与结构；管住其设计、使用、运行与演化的原则。语气是”可以关乎”，不是”体现于”。附录 A.2 说得更直白：In some cases, the fundamental elements are physical or structural components of the entity and their relationships.——在某些情况下，根本要素就是实体的物理或结构组件及其关系。

三句话合起来就是这一节的结论：元素与关系**不在定义句内**——最多是”通常意指体现于”（42020:2019 § 3.3 注 1，注释级）；它们是**架构可以关乎的对象**（§ 5.2.2），也是架构描述要交代的内容，但**不是定义要素**。把它们当架构本身，是误读的第一步。

元素这个词还要再分三层，否则一定混。同一份材料里出现”元素”，可能指三样不同的东西：

| 术语 | 出处 | 在哪一层 | 是什么 |
| --- | --- | --- | --- |
| **架构描述元素**（AD element） | 42010:2022 §3.4 | 描述层 | 架构描述的零件：利益相关方、关注点、方面体、架构视角、架构视图、视图组件、模型种类、对应关系…… |
| **架构落点**（**本书工作名**；旧名”架构构成物”已废止） | components／relationships 见 42020:2019 §3.3 注 1（附录 A T-168） | **实体侧**（不在定义句内） | 实体内部的构造（**注释级，没有术语地位**）——是架构”**可以关乎**”的对象，不是架构的构成物 |
| **系统元素**（system element） | 15288:2023 §3.47 | 实现层 | 可被实现的离散部件 |

还有第四个词要分清：架构实体（architecture entity，42020:2019 §3.6；附录 A 作”被架对象”，同物异名）指”被架构的那个事物”，≈ 42010 的所关注实体，不是”架构的构成物”。中文圈常把它误读成构成物，此处钉住。15288:2023 §5.9 另有一句可作旁证：架构实体与系统元素”代表两种不同的概念”（依中英文对照版；英文原文作 notions）。

为什么这一节的最后要强调”关系”？因为系统的涌现属性来自关系，不来自单个元素。第 2 章讲过：每个发动机都正常、每块机翼都达标，飞机未必能飞。放到架构这一层，它有一个很具体的推论：可靠性的瓶颈通常不在单个组件，而在组件之间的交互模式。单个传感器精度再高，也拦不住”传感器→固件→云端”这条链上某个环节的重传超时。

### 10.2.6 假判据冒充真判据

真判据只有一条：目的。“什么算基本”这个问题太要紧，市面上就长出几个代理来冒充它。

| 假判据 | 为什么会冒充判据 | 两类失真 |
| --- | --- | --- |
| **不可逆性**（改起来贵） | 可观测、直观、易沟通 | **漏判**：不贵但承重（“日志须可被外部审计员独立读取”）；**误纳**：很贵但与目的无关（框架版本、组织惯例技术栈） |
| **可判定性／可引用性／高层级／概要** | 都是”看起来像判据”的代理 | 同一形状的错误换了外衣——**代理冒充真判据** |

“改了它还是不是这个实体”这句问法看着像不可逆性，其实是承重测试的另一半：可逆的改动也可能承重（不贵，但去掉目的就不成立），不可逆的改动也可能无关（很贵，但与目的无关）。两类失真都真实发生。

代理不是没用，是不能当总闸。它们的正当用途有四样：**排序**（先定哪几件）、**定决策时机**（什么时候必须拍板）、**配治理强度**（多严的评审与留痕）、做回代检查（事后核这条决定是否真承重）。

这里有一处必须替标准说清的边界。42010:2022 §6.10.1 确实给了一张"哪些决策值得记录"的选择准则清单，里面有一条正是 that are likely to be costly to change（改起来可能很贵的）。那是决策记录的取舍准则，不是架构的准入判据。两者都用到"贵"，但一个管"记不记"，一个管"算不算架构"——混起来，就会把很贵但无关的东西记进架构，同时把不贵但承重的东西漏在架构之外。

（第 12 章会专门讲决策的时机与记录；本章只用它的反面，把"贵即架构"这条误读钉住。）

## 案例进展①｜"M3 到底是什么系统?"——水忠开始找那个不可还原的东西

章首那场会之后，水忠被"三个人三个答案"刺痛了。他把文正、李旭拉进会议室，想弄清一件事：我们到底守着一个什么样的系统。

文正先开口："M3 就是个温度记录仪加个箱子，能调温、能记录、能上传。你说的'系统'，不就是这些功能的集合吗?"

水忠摇头："功能能加，架构不能。按承重测试问一遍就分开了——去掉它，M3 还能达成它的目的吗。加个摄像头记录装卸，M3 还是 M3，目的是同一个；可要是把'全程可追溯'拿掉，它就从医疗冷链设备退成一只会调温的箱子。所以我不数功能，我数'少了什么它就不是它'。数目对不上不要紧，方向对得上就行。"

李旭盯着纸上那行字：“我测了两年，最难受的不是测，是说不清凭什么说’测过了’。判据得能追问到出处——你刚说的’少了什么不是它’，就是我要的那个靶子：M3 的架构在哪，我该验的东西就在哪，不用再对着功能清单一条条猜。”

水忠在纸上写下来：“断链即报废的疫苗，能从出库一路撑到接入冷库，全程温度可追溯。去掉它，M3 就是一只能调温的箱子。”

三个人都不说话了。他们第一次意识到，自己守着一个系统快两年，却从没讨论过它”不可还原”的部分是什么。

## 10.3 架构描述是什么

架构交不出去，可它必须被共享、被讨论、被检查、被传承。承担这件事的那件东西叫架构描述。这一块把它的身份说清，再把它的三个近邻概念——架构视角、架构视图、关注点——各归其位。

四者之间的关系是标准里最值得背下来的一条链：

> **利益相关方 →（has 持有）关注点 →（frames 框定）架构视角 →（governs 治理）架构视图 →（addresses 应对）关注点；架构视图组成架构描述；架构描述表达架构。**

（四关系名的文本依据为 24 号 §15.5／§16.5 与 30 号 §3.1 的转引；正本对应图为位图，不可逐字核。）

四个关系名各有各的名字，不可互换：**持有**说的是谁在乎什么；**框定**说的是规则框定问题，不是解决问题；**治理**说的是规矩管产物；应对说的是真正对关注点作出交代的，是视图的内容。

### 10.3.1 架构描述是一个工作产品

标准的定义很短（ISO/IEC/IEEE 42010:2022 §3.3）：

> **architecture description (AD): work product used to express an architecture.** 中文：架构描述——用于表达架构的工作产品。

两条注把它钉得更实：注 1 说工作产品是某个过程产出的制品；注 2 说架构描述是提供给利益相关方的信息的有形表示，被视为**信息部件**（information part）。

“信息部件”在标准里有独立定义（§3.14）：separately identifiable body of information that is produced, stored, and delivered for human and machine use——可单独识别，为供人与机器使用而被产出、存储和交付的信息体。三个动词值得记住：**产出、存储、交付**。这三件事架构本身一件也做不到，架构描述全都做得到。

为什么非要有它？因为涵义锁在一个人脑子里，就没有协作价值。M3 的架构再清晰，如果只存在于水忠一个人脑中，就没人知道数据所有权的边界、新工程师不知道断链检测为什么是刚性边界、客户也理解不了 M3 凭什么值得信任。架构必须被**表达**出来，才能被共享、被讨论、被检查、被传承。

架构描述要交代哪些内容，标准是逐条规定的（§6.2 至 §6.10 的条款标题级）：识别利益相关方 → 识别利益相关方角度 → 识别关

注点 → 识别方面体 → 纳入架构视角 → 纳入架构视图 → 纳入视图组件 → 记录对应关系 → 记录架构决策与理据。

把这张清单倒过来读，就是一份架构定义的操作顺序——从最后一项的”决策与理据”倒推回第一项的”谁”，每一步都有出处、有产出、可核对。（这条链怎么走，是第 15 章的事；本章只用它说明一件事：架构描述不是一张图，是一份有规定内容的制品。）

两处边界本章只点到，不展开。

一是”能冻结的是架构描述，不是架构”。变更控制、基线、入库，管的都是架构描述这一侧（第 7 章已划死这条界）。把”架构改了”和”架构描述改了”混成一句话，配置管理就没法谈。

二是架构描述是证物，不是权威（24 号研究报告 §11.2）。一份架构描述之所以能拿来判设计，不是因为它叫架构描述，而是因为它能回溯到架构；断在回溯这一段，它就从凭据退化成资料。第 15 章讲这个动作怎么做。

### 10.3.2 视角是一组惯例

“看到每个方面”这件事，标准用两个词分工：架构视角（viewpoint）是规则，架构视图（view）是产物。先看规则的定义（42010:2022 §3.8）：

> **architecture viewpoint: set of conventions for the creation, interpretation and use of an architecture view to frame one or more concerns.** 中文（中英对照

> 版逐字）：用于创建、解释和使用架构视图以框定一项或多项关注点的一组惯例。

这里有一处不对称，是本章要讲的要点之一：这句话里没有属概念。标准的惯例是”凡需属概念必明写”——架构描述写 work product（§3.3），架构视图写 information part（§3.7）；而 §3.8 是一个裸名词短语，没写它是哪一类东西。本书照正本写”一组惯例”，不替它补一个标准没写的属概念。（此处要小心：有些转述把视角说成”一种工作产品”，那是按体例推得的，正本没有。）

（另有一处用词要说明：标准 §3.8 注 3 的中译作”架构视角的约定记录在该视角的规格中”，定义句则作”一组惯例”——两处是同一个词 conventions，本章统一用”惯例”。）

一组惯例凭什么能管住一张图？三条注给了答案：

- **注 1**：“框定”关注点意指塑造、组织、表达那些关注点——用它来区分”视角框定关注点”与”视图应对关注点”两个阶段，类似”界定问题”与”解决问题”之别；
- **注 2**：视角是架构师所确定的、与架构描述目的相关的关注点的参考框架；
- **注 3**：惯例记录在该视角的规格里。规格本身是信息部件（§3.16），于是”一组惯例”落了地：有名字、有版本、有位置可放——这才是视角可入库、可版本化、可跨项目复用的依据。

治理的实质是可判定的符合性。视角不是视图的一部分（它通常独立存放在视角库里），也不产出内容（内容始终是架构师针对这个具体实

体做的）；它管的是”这份产物合不合这本规则书”。标准对这两侧各有一句话：架构描述包含或引用它所用到的每个架构视角（§6.6）——“引用”二字说明视角可以住在描述之外；而一个架构视角治理一张或多张架构视图（§5.2.7 逐字：An architecture viewpoint governs one or more of these architecture views.）。

> 术语注（重数，两层依据要分开）：**“一个视角可治理多张视图”是规范正文逐字**（§5.2.7）；**“每张视图恰有一个治理它的视角”**（视图视角 1:1）在资料性附录 A.5 里有明文：每张视图都必须有一个架构视角的规格、须受一个架构视角治理，即符合**一组惯例**；两视角造一图时，可把两个视角组合成单一视角，也可把多视角的用法直接记在该视图里。**规范正文只写到这一步：每个视角至少治理一张视图（§6.7）**。24 号与 30 号研究报告依 OMG 的一份缺陷记录（UAF14-37，**库外·不可核**）作出的判定是旁证，不是唯一依据。本书照此处理：**1:N 引规范正文，1:1 引资料性附录**，不把 1:1 写成”标准规定”。

两条硬要求各堵一个漏洞（§6.6 与 §6.7）：每个关注点至少被一个架构视角框定（防”无根关注点”）；每个被引用的架构视角至少治理一张架构视图（防”空视角”，§6.7）。

视角规格该写什么，标准是 shall 级的清单（§8.1）：a) 与视角相关的利益相关方角度；b) 该视角框定的一项或多项关注点；c) 与这

些关注点相关的一项或多项方面体；d) 持有这些关注点的已知典型利益相关方；e) 构建视图时使用的模型种类与图例；f) 捕捉视图及其视图组件内部关系的对应方法；g) 关于该视角的信息来源。另有一句 should：视角规格应当标识视图方法，并标识一种或多种模型种类。

> 术语注（一处计数差异）：本书另有几处**不同的归并口径**。附录 A 的视角规格条作六项：把正本的 a/b/d 并作”框定的关注点与服务对象”，并把 should 的视图方法一并列入；30 号 §3.3 作五项，32 号 §4.4 作六项。**正本的 shall 清单是上列七项，视图方法那一句是 should**。规格怎么写、四份可复用蓝本是什么，第 15 章展开；本章只钉住**“必须写什么”以正本七项为准**。

### 10.3.3 视图是一个信息部件

规则说完了，看产物（42010:2022 §3.7）：

> **architecture view: information part comprising portion of an architecture description.** 中文（中英对照版逐字）：包含架构描述的一部分的信息部件。

视图是有属概念的——它是信息部件，可单独识别、可产出、可存储、可交付。这与视角那句裸名词短语形成对照，也是”规则 vs 产物”这条分界的正本依据。

视图由**视图组件**组成（§5.2.8）。组件可以是模型式的，受**模型种类**约束；也可以不是模型（表格、清单、决策记录片段），由图例记

录其惯例。“图”只是视图组件里最显眼的一种，不是全部——把架构描述等同于”几张图”，丢掉的正是那些非模型的组件。

每张视图是刻意的”不完整”。一张视图只回答一组关注点：完整画出每一面，等于把所有关注点糊在一张图上，对任何人都没用。视图的价值恰在剥离掉你不关心的部分，只保留与这组关注点相关的本质。

那么一个实体需要几张视图？标准的答案异常清爽（§6.7 与附录 A.5）：由利益相关方及其关注点决定，不由方法论决定。三张也好、七张也好、十二张也好——只要每张视图都能说出”它在回答谁的什么问题”，且没有重要利益相关方的核心关注点被遗漏，就是完备的。这个答案把”需要几张视图”这个吵不清的问题，换成了”利益相关方是谁、关心什么”这个可以具体讨论的问题。

顺带把”框架”这个词的位置说清。标准里有个正式概念叫架构描述框架（ADF，§3.5）：在特定应用领域或利益相关方群体中确立的、用于描述架构的惯例、原则和实践。2022 版用 ADF 取代了 2011 版的”架构框架”，为的是把它与 42030 规定的架构评估框架区分开——一个规定”架构怎么写”，一个规定”架构怎么评”，此前共用一个词。

业界现成的 ADF 有几种常见的：4+1 视图模型（Kruchten，1995，软件领域最经典）、C4 模型（软件，四层缩放）、TOGAF（企业架构，业务／数据／应用／技术四域）、Zachman 框架（企业架构，六行六列的矩阵）。它们是”一种常见的视角组织方式”，不是本章的

骨架——本章的骨架是那条链：关注点 → 视角 → 视图。框架是对已有视角的打包，视角才是一手的东西；一个领域找不到现成的 ADF，按标准设计的正规路径是自建一个项目 ADF（第 15 章讲怎么写）。

最后留一句"别串尺子"。评架构好不好，对的是**目的**（归 42030）；评架构描述合不合适，对的是细化过程的适宜性（归 42020）。串了尺子，评估就退化成文档审查——报告里出现"这张图没画好"，就是串尺子的症状。

### 10.3.4 关注点站在哪个位置

链条的起点是利益相关方：对实体有利益、权利、份额或主张的角色、职位、个人或组织（§3.17）。不只是用户：开发者、运维、安全、业务方、监管、维修、甚至退役处置方，都是利益相关方（6.2 节的示例里都列着）。

每个利益相关方都有对实体的**关注点**（concern，§3.10）：matter of relevance or importance to a stakeholder——对利益相关方具有相关性或重要性的事项。

> 术语注（两册措辞不同、不可互替）：42010:2022 §3.10 作 matter of **relevance** or importance；42020:2019 §3.8 作 matter of **interest** or importance，该条 SOURCE 注写明取自 42020 且 **Notes have been modified**。引用时必须带标准名，不能混着用。

关注点不是需求，这是本节最要紧的一条界。关注点属于**人**（立场、利害，常常还没有解）；需求属于**系统**（含主体、条件、指标、验证方法的陈述句）。三条判据把它们分开：

| 判据 | 问法 | 结论 |
| --- | --- | --- |
| **换人测试** | 换个利益相关方，它还在不在？ | 在 → 关注点；不在 → 需求 |
| **改写测试** | 能问出通过／不通过吗？需要指标与验证方法吗？ | 需要 → 需求 |
| **粒度检查** | 停在哪一级？ | 太抽象挑不出架构视角、太具体进了设计——合适粒度 = **刚好足以挑出架构视角** |

还有一条位置要记牢：关注点在架构描述一侧，需求在规格一侧——关注点是挑架构视角的锚点，需求是规格说明的输入。混为一谈，会同时污染需求基线与架构描述的可追溯性（细办见第 15 章）。

关注点天然冲突。标准对它的性质只留了两句可核的话，都值得照抄（分属 §5.2.4 与 §5.2.3）：

> Concerns are based on current interests and influences of the stakeholders and are often subjective in nature.
> 关注点基于利益相关方当前的利益与影响，且往往本质上是主观的。

> Some stakeholders' concerns are contrary to the success of the entity of interest. 有些利益相关方的关注点与该所关注实体的成功相抵触。

“冲突不可消除、只能权衡”是本书自己的判断——标准给的是两个事实（关注点往往主观、有些关注点与实体成功相抵触），从事实到”只能权衡”这一步是本书跨的，此处署名。

怎么个冲突法：监管要更多加密与留存，会拖慢数据链路、增加存储成本；老板要快上线，会积累技术债、牺牲可维修性；司机要报警简单粗暴，与追溯要记录详尽精确互相打架。架构不是”找一个让所有关注点都满意的解”，而是在互相冲突的关注点之间做出有意识的权衡。

最后一句位置的话：关注点不是判据，是判据的现场。判据是目的（§10.2.4）；关注点承担的是另一件事——它提供”这次写哪些、写多细、给谁看”的输入，并充当发现缺口的雷达。用一句话记：关注点是原料，目的是准绳。（这句话的完整读法、以及”每一条关注点必须有一个出路”的裁决机制，是第 15 章的活。）

## 案例进展②｜四张图，四个对不上的”架构”

恒信的架构师万辉，最近也在为”架构”头疼。他把合同审批系统画成四张图：业务、数据、应用、技术，挂满一面墙。

运维玉杰先看了一遍，指着技术架构图问：“部署在哪、坏了从哪查、监控看什么——这三样在里面吗？运维要的不是服务清单，是出事以后我照着能不能查下去。这一面图上没有，我这条线就是空的，出了问题只能挨个问人。”

万辉：“技术架构图我按模板画的，服务边界、依赖关系都在。”

徐漾路过，站住看了五分钟：“万辉，合同审批里’一份合同’，在这四张图里分别是什么?”

万辉指着四张图：“业务架构里，是从发起、法务初审、财务校验、业务终审到归档的一条流程；数据架构里，是’合同主表＋审批记录＋附件’三个实体；应用架构里，是审批引擎里的一个流程实例；技术架构里……呃，我没画合同。可这四个说的都是同一份合同，怎么会指成四样东西。”

徐漾说：“就凭这最后一句——你的四张图是四个系统。ISO/IEC/IEEE 42010:2022 §5.2.2 写的是：同一个所关注实体的架构，可以透过一份或多份架构描述来理解。业务、数据、应用、技术是对同一个架构的四个关注域视角，必须能对得上。”

徐漾接着说：“检验的办法是拿一条真实场景横着穿：‘法务在 App 上审批一份五百万的合同’，业务图里走哪条流程、数据图里动哪些表、应用图里调哪个服务、技术图里跑在哪台机器上。一条场景走不完四张图，四张图就是四个系统。”

万辉那天把四张图撤下来重画。他后来说，他画的不是”四个架构”，是一个架构的四个切面——让它们对得上的不是模板，是一条真实场景。

## 10.4 架构不是什么

前面两块把架构和它的描述都立起来了。这一块从反面走一遍：把贴着”架构”这个词长出来的误读逐个清掉。清完，前面那些定义才算真正站住。

### 10.4.1 不是一张图

水忠的病不是”没画好图”，是**把图当成了架构**。这两种病看着像，治法完全不同：前者去改图，后者去改对”架构”的理解。

“架构”在日常里被用得非常泛滥。有人说”我们的架构是微服务”，有人说”架构图在共享盘”，有人说”这次上架改了架构”，还有人把拓扑图、部署图、ER 图统统叫”架构”。这些用法有一个共同的毛病：把架构当成了一样能画出来、能保存、能过期的东西。

标准知道这个毛病。它在附录里记了一件有意思的事（42010:2022 附录 A.2 逐字）：

> Empirical studies have discovered four metaphors for architecture found in organizations: architecture as blueprint; architecture as literature; architecture as language; architecture as decision.

——组织里实际存在四种关于架构的隐喻：架构即蓝图、即文献、即语言、即决定。这是实证研究的发现，而标准紧接着说：本文档的概念基础**不预设其中任何一种**，与四种都兼容。

这一句值得停一下。它说明两件事：第一，“架构是一张图”不是谁在胡闹，它是有实据的组织习惯；第二，**假的是把它当成全部**——蓝图只是四种隐喻之一，而且是最容易过时的那一种。图会过时，被图投影的那组基本概念或属性可以一直有效。一张图可以过时，架构可以一直有效——水忠团队就是被这个区分困住的典型。

### 10.4.2 不是一组决定

另一种常见的算式是”架构就是那组改起来贵的决定”。这个说法听起来比”一张图”高级，方向也对了一半——它至少指向了”内容”而不是”载体”。它仍然不成立，原因与上一节同源：标准把”架构即决定”同样列为四种隐喻之一，可见它是组织里真实存在的说法，但标准不预设它。

“决定”在标准里另有正位，三处都要一起看：

- **§5.2.12 给了它定义**：An architecture decision is a collection of choices made in the overall context of an architecture.（架构决策是在某个架构的总体语境中作出的一组选择。）它在标准里是一个独立的架构概念；
- **§6.10 是 shall 级的要求**：架构描述**应当记录**被认为对该实体架构至关重要的架构决策及其理据——注意这里的”至关重要”由组织与项目自定准则（§6.10.1 给了一张取舍清单）；
- §5.2.9 把它列为架构描述元素所承载的概念之一（架构决策与理据在内）；而 §3.4 注 1 的清单里没有它——这是正本自身的两处不一致，引用时把两处都交代出来，不要只引一处。

三处合起来说清一件事：“决定”是认出架构、并把架构记下来的抓手，不是架构本身。把抓手当本体，代价很具体：决定清单会越写越长（凡是贵的都往里加），而”这个系统是什么”始终没有正面回答。M3 那次评审会上，水忠的图里没有任何决定；等他把老规矩写下来

的时候，写下的也不是”决定的集合”，而是一句涵义加一组原则（案例进展③）。

### 10.4.3 不是设计

架构与设计最容易混，因为两者都落在”系统里需要决定的事”上。分界不在粗细，在**种类**（24 号研究报告摘要三）：

> **架构与设计的差别是种类（kind），不是粒度（granularity）。**

这条结论有一个很硬的证据：合规关系的存在本身就是证明。若架构只是”更粗的设计”，那么更细的那一层不可能对更粗的那一层”不合规”——自己对自己无所谓合规。

那条分界可以拧成一句可判定的判据：这份内容能不能被设计违反？能被违反的（设计边界、接口约定、数据所有权、向后兼容的承诺），属架构这一侧；不能被违反的（某个接口用 REST 还是 gRPC、某张表的索引怎么建），属设计这一侧。

顺着这条界看设计那一头的定义，会更清楚（ISO/IEC/IEEE 15288:2023 §3.13）：设计是”系统元素及其关系的**规格**，足够完整到足以支撑对架构的一次合规实现”。注意承担合规的是实现，不是设计本身。设计只是把架构与需求转成可实施的规格，它给出的结论只能是**条件性合规**（第 15、16 章展开）。

两个过程不是两个阶段。15288 允许迭代与并发，“先把架构定义做完、再开始设计定义”是瀑布式误读；真实形态是同一时刻在相邻两

个种类层上的两类工作（24 号研究报告 §8.2；正本侧见 15288:2023 §6.4 并发的表述、42020:2019 中英对照版）。

### 10.4.4 不是一堆”XX 架构”

市面上有业务架构、数据架构、应用架构、技术架构，还有需求架构、功能架构、逻辑架构、物理架构，甚至微服务架构、事件驱动架构——这些到底是不是多个架构？

**一个所关注实体只有一个架构**。那些”XX 架构”分属几套不同的维度，叠在一起才生出了混乱：

| 说法 | 实际是什么 | 判断维度 |
|---|---|---|
| 业务／数据／应用／技术 | 同一个架构的四个**关注域视角** | 按”看什么”切 |
| 概念／逻辑／物理 | 同一个架构从抽象到具体的**描述层次** | 按”多详细”切 |
| 微服务／事件驱动／分层 | **架构风格**——架构决策的套路 | 按”用哪套套路”切 |
| “用了 Spring Cloud” | **框架**——把套路物化成可复用的制品 | 按”借了谁的骨架”切 |

这四个是按”看什么”切出来的四组视角——不是四种架构，也不是 §3.9 的方面体。

混乱的根源在日常用语：把”架构描述”简称为”架构”，把”架构视图”也简称为”架构”，简称来简称去，听起来就好像系统有多个架构了。就像只有一副骨骼的人，医生正面拍 X 光、侧面拍 X 光、横

切面拍 CT——**这些是同一副骨骼的影像，不是多副骨骼**。同一个架构的若干视图也是这个关系。

> 术语注（这条断言的证据边界）：**正本支持的是”一个架构可以有多个架构描述”** —— §5.2.2 逐字 The architecture of an entity of interest can be understood through one or more distinct ADs, each created for a purpose relative to the architecture and stakeholder needs.（一个实体的架构可以透过一份或多份不同的架构描述来理解。）**“一个所关注实体只有一个架构”这句更强的话，是本书的读法**（依 06 号研究报告 §1.5），此处署名。它的用处是把”XX 架构”归位，不是拿来否认可选方案——分析阶段可以有多套候选架构，被采纳的只有一个。

### 10.4.5 不是框架与风格

有一种说法比”微服务架构”更常见，也更唬人：**框架**。“我们技术栈全是 Spring 全家桶”“M3 用的某某嵌入式框架”，紧接着一句：“用了 Spring Cloud，所以我们系统是微服务架构。”

这句话错在哪？Spring Cloud 是一套开源的**框架**：服务注册、配置中心、熔断降级，现成的轮子一堆。你引入它、按它的规则填进业务代码，一个微服务项目的骨架就搭起来了——但骨架搭起来，不等于

架构有了。服务边界划在哪、数据归哪个域、接口契约怎么定，这三样框架一个都没替你决定。你下载的是工具，工具替你做不了决定。

一句话分清：架构是”这一个”实体在其环境中的**基本概念或属性与管控原则**，框架是”这一类”的组织方式。架构属于实体，框架属于领域。试金石也简单：删掉它，系统还在吗？换掉 Spring Cloud，你的系统还是那个系统，对外功能可以一个字节不变；改动它的**基本概念或属性**（比如”数据所有权归哪个域”这条根本约定），这个实体就不是原来那个了——同一性问的是”**目的还在吗**”，不是”零件换了没”（忒修斯之船·T-144）。

风格也照这条尺子量：微服务、事件驱动、分层都是套路，换一套，系统还是那个系统。

顺带把挨着的几个词分开：**构架**是名词的动词化，本义”搭建承重骨架”（“把 XX 构架出来”，口语里指把主干结构从无到有立起来）；架构是本文讲的那组基本概念或属性与管控原则；框架是可复用的半成品。

三者的验收标准不同——“构架出来”的默认验收是骨架可见，而架构定型的灵魂是基本概念或属性与管控原则：图上有了框和线就说”构架好了”，问”为什么这么分、备选死在哪”却答不上来，这正是画图匠与架构师的分界。

### 10.4.6 不是”高层级”

最后一片误读最隐蔽：**以为”基本”就等于”高层级”**——画得越顶层、越概览，越像架构。

这是两个轴被叠成了一个轴。24 号研究报告专门否掉了两个说法，这是其中之一：

> 说法：架构描述中的元素是系统的高层级元素。否掉的理由：类型错误（“高”是**呈现粒度**，不是**基本性**）；即使指架构落点（components／relationships），“是”这个动词仍然错；深层是**把 fundamental 偷换成了 high-level**。

**“基本”与粒度正交**。共识算法很复杂，但可以是基本的；变量命名很简单，但永远不是。随手画的模块框图，再高层级也不是架构；起搏器的定时算法这类关键细节，再细也可能是架构。判别只看一条：**去掉它，这个实体还能达成它的目的吗**。

（另一个被否掉的说法是”架构描述＝设计描述的概要”。它与本节同源——都是把种类层的东西搬到粒度层去说；分界见 §10.4.3 与 §10.3.3。）

### 10.4.7 认知反转

回到章首那场评审会——水忠放图、讲十分钟、散会。那场会连错三次，一次比一次深。

**第一处，他把”架构描述”当成了”架构”**。那张图只是架构描述里一个粗糙的切片：画了元素，没画关系，更没画原则；真正的架构，是那组不可还原的涵义（基本概念或属性）与管住它的管控原则。图会过时，那组涵义一直活着。

第二处，他把”画了图”当成了”做了架构”。做架构不是画一张漂亮的框图，是识别出”去掉就不是这个系统”的那组基本概念或属性，并立起管住它怎么造、怎么变的原则。图三个月没更新，M3 却被正确造出来——因为那组东西一直有人在脑子里守着。图是投影，投影后面那个东西才是实体。

第三处，也是最深的一处：他以为”改起来贵的”才配叫架构。于是他既漏掉了一些不贵但要命的东西（那几条老规矩没有一条是因为”贵”才被立起来的），也差点把一些很贵却与目的无关的选择（某个框架版本）请进架构。

三处错，其实是同一处错的三种说法：

> **架构不是一张图，也不是一堆贵的决定；它是实体在其环境中的基本概念或属性，加上管住它怎么实现、怎么演化的管控原则。它交不出去——能交付、能评审、能版本化的是架构描述；而”基本”的判据只有一条：目的。**

收成第 1 章的一个词：水忠对”架构”，完成了从**见过到拥有**的跨越——“见过”是背得出”架构”、画得出四层方块图；“拥有”是能答全四问（它是什么、不是什么、从哪来、往哪去）。**四问答全，才算拥有。**

回看第 8 章。那一章的判据是”评审对目标”：评审对着系统的目标审，审的恰恰包括架构描述是否对得起系统的那组基本概念或属性。而”不可还原”给下一章埋下最重要的种子：这组东西不是画出来的，是设计出来的。下一章讲架构设计：把需求翻译成”**架构**”——定

本质、立管控原则，并把它表达成架构描述（元素与关系是架构”可以关乎”的对象，其规格归下游设计定义过程）。“哪些决策不可逆、什么时候必须拍板”，是再下一章的事。

## 案例进展③｜把 M3 的架构说清楚

一次复盘，水忠把下班后写的”老规矩”投影出来。那不是一张图，是一份清单：

**M3 不可还原的涵义**：断链即报废的疫苗，能从出库一路撑到接入冷库，全程温度可追溯。

**管控原则 · 怎么造 M3**（约束实现）：

- 温度数据归属追溯域——传感器只管采、云端只管传、追溯平台管存管看，这条边界谁都不许动；
- 断链检测是刚性边界：检测到断链，必须实时告警并记录，不得吞掉、不得延迟；
- 追溯链路上每个环节都可独立验证。

**管控原则 · 怎么变才不散架**（约束演化）：

- 新增传感器类型，不得破坏已有追溯链路；
- 固件升级必须向后兼容已出厂的设备；
- 云端采集协议变更，先过追溯完整性评审。

董勐看完说：“水忠，这些规矩——数据归属、断链刚性、兼容承诺——就是 M3 的架构吧？我那边接口怎么往外导数据，往后得照它们判。”

水忠：“是。按 ISO/IEC/IEEE 42010:2022 的定义，架构是’实体在其环境中的基本概念或属性，以及管住该实体及其相关生存周期过程怎么实现、怎么演化的管控原则’。上面那行’断链即报废的疫苗……全程可追溯’是前半句，下面这些规矩是后半句。我以前以为

架构是那张图；现在知道图只是给’长什么样’画的，这些字才是’少了它 M3 就不是 M3′的东西。”

逸人从产线接话，翻开实测记录：“这条’断链检测是刚性边界’落到我这儿最重——每台箱子下线前都得实测断链告警，测不过不能发货。我先把清单过了一遍：数据归属、断链实时告警、每跳可验证，三条产线都能照做，没有一条造不出来。”

水忠：“这就是管控原则约束实现与演化的意思——它替产线把’测什么’定死，剩下的你们自己拿主意。”

文正接着问：“那客户想看架构，我们给他看什么？”

水忠：“给疾控中心看追溯链路，给司机看状态，给维修看结构，给监管看合规。这四样是按’利益相关方 → 关注点 → 架构视角 → 架构视图’那条链挑出来的，没有一样是那张四层方块图。”

文正：“图是死的，视图是活的——每个利益相关方看到自己该看的那一面，合起来说的是同一个 M3。”

## 小结

这一章把”架构”和”架构描述”分开钉死。

**架构**是一个实体在其环境中的基本概念或属性，以及用于该实体及其相关生存周期过程的实现和演化的管控原则（ISO/IEC/IEEE 42010:2022 §3.2）。这两半并列，一头回答”它是什么”，一头回答”它必须怎么造、怎么变”；后一半是规范性的，可以被违反。

架构是抽象的，交不出去；能交付、能评审、能版本化的是架构描述。元素与关系不是定义要素——它们搬到 42020:2019 §3.3 的注 1 里，降成了”通常意指体现于组件及其关系”，注释级、没有术语地位；它们在 42010 里的位置是 §5.2.2 的”架构可以关乎”那五类，

以及附录 A.2 的”在某些情况下，根本要素就是实体的物理或结构组件及其关系”。

**“基本”不是形容词，也不是”改起来贵”**。它的判据是目的——去掉它，这个实体还能达成它的目的吗；跑两遍，兑现侧与演化侧。可违反性、可引用性、可判定性排在它后面，是排序与检查，不是准入。

架构描述则按”利益相关方 → 关注点 → 架构视角 → 架构视图”这条链组织：视角是一组惯例（规则），视图是一个信息部件（产物）；关注点属于人、需求属于系统，关注点不是判据，而是判据的现场。

全章要清掉的误读：不是一张图，不是一组决定，不是设计，不是一堆”XX 架构”，不是框架与风格，不是”高层级”。

下一章讲架构设计——把这组基本概念或属性设计出来：概念怎么幸存、属性怎么涌现、管控原则怎么立；再下一章讲架构决策：哪些决策不可逆、怎么判断、什么时候必须拍板。

---

# 附录

## A 本章概念总图

**架构，是一个实体在其环境中的基本概念或属性，加上管住它怎么实现、怎么演化的管控原则（ISO/IEC/IEEE 42010:2022 §3.2）**

**两半并列：前半描述性（它是什么）｜后半规范性（它必须怎么造、怎么变，可以被违反）**

**本书读法：系统不可还原的涵义（落前半句；管控原则是并列的第二样内容；**不是"一组决定"**—决定是抓手）**

**架构 无形、交不出去（intangible and abstract）—三条判据：可交割性／**

同一性／因果力（**同一性讲的是"多载体一个内容"：架构不绑在载体上，也不绑在元素与关系上**）

架构描述 有形的工作产品（work product）——可产出、可存储、可交付；被视为信息部件

利益相关方 --has--> 关注点 --frames--> 架构视角 --governs--> 架构视图 --addresses--> 关注点

视角 = 一组惯例（正本无属概念）｜视图 = 信息部件（正本明写属概念）

一个视角可治多张视图（规范正文 §5.2.7）；"每张视图恰有一个视角"见资料性附录 A.5（两视角造一图可合成单一视角）

"基本"（fundamental）：对目的必要且充分——必要性管入会（去掉它目的落空吗），充分性管闭包（到这层可以停了吗）

判据栈：目的必要性（总闸）→ 可违反性 → 可引用性 → 可判定性 → 关注点闸门

两道闸门：目的（承重性）·关注点（取舍与呈现）｜另有一类来源：环境（语境与影响）

两遍跑：兑现侧 + 演化侧

假判据：不可逆性（漏判不贵但承重／误纳很贵但无关）；可判定性·可引用性·高层级·概要

元素与关系：不是定义要素（2022 已从定义句删去）——搬到 42020:2019 §3.3 注 1（降为"通常意指体现于组件及其关系"，注释级）

42010 的位置：§5.2.2「架构可以关乎……」五类；附录 A.2「某些情况下根本要素就是组件及其关系」

元素三层：架构描述元素（描述层）｜架构落点＝components／relationships（实体侧，注释级）｜系统元素（实现层）

另有一词要分清：架构实体（附录 A 作"被架对象"）＝被架构的那个事物，不是架构的构成物

不是什么：不是一张图（蓝图只是四种隐喻之一）｜不是一组决定（决定是抓手，正位在 §5.2.12／§6.10）｜不是设计（种类不是粒度；判据＝能否被设计违反）｜不是一堆"XX 架构"（关注域／描述层次／风格／框架四件事）｜不是框架与风格（用框架≠有架构）｜不是"高层级"（基本与粒度正交）

## B 本章的能力级

**第 ③ 级为主**

用第 1 章 §1.4.4 的尺子量一下：这一章以第 ③ 级为主。架构不再是图上的一条线，而是能推演”改它会发生什么”的那组东西。

- **表征**——本章把哪几个词说准：架构（无形、交不出去）／架构描述（工作产品）／架构视角（一组惯例）／架构视图（信息部件）／关注点／方面体／管控原则（定语位 governing principles）；并把四个近邻词分开：架构描述元素／架构落点（components／relationships）／系统元素／架构实体
- **图式**——本章给出哪个可识别的结构：定义的两半；“利益相关方 → 关注点 → 架构视角 → 架构视图”这条链；判据栈与两道闸门；元素三层

- 心智模型——本章给出哪个能推演的机制：承重测试（去掉它，目的还达得成吗）＋两遍跑（兑现侧／演化侧）；代理的两类失真；目的漂移是架构失效的第一病因
- 解释框架——本章不建此级；由第 17、18 章承担

> 收束句：**四问答全，才算拥有**。

## C 自查 · 这些说法对吗？

1. “我们画了架构图，架构工作就完成了。”——**错**。画出来的是架构描述；架构是那组基本概念或属性加管控原则。图会过时，架构可以一直有效。
2. “架构就是那张框图，图过时了架构就过时了。”——**错**。过时的只是描述。M3 的图三个月没更新，那几条老规矩一直活着。
3. “我们有业务架构、数据架构、应用架构、技术架构四个架构。”——**错**。一个所关注实体只有一个架构；那四个是对同一架构的四个关注域视角，必须能对得上。一条场景走不完，就是四个系统。
4. “微服务架构是架构。”——不准确。微服务是架构风格，是决策的套路；你的系统是什么架构，取决于它守着哪些不可还原的涵义、立了哪些管控原则。
5. “架构就是技术选型。”——错。技术选型是可以换的实现；换掉框架，系统还是那个系统，改动服务边界就不是了。

6. ““基本”就是高层级、就是概览。“——错。”基本”与粒度正交：共识算法很复杂但可以是基本，变量命名很简单但永远不是。
7. “改起来贵、改不动的，就是架构。”——错。不可逆性只是代理，会漏判也会误纳；判据是目的——去掉它，这个实体还能达成它的目的吗。
8. “架构和架构描述是一回事，只是叫法不同。”——错。架构无形、交不出去；架构描述是有形的工作产品，能产出、存储、交付、签收、版本化。
9. “架构视角是一种工作产品。”——不准确。正本 §3.8 的定义句是裸名词短语，没有属概念；它是”一组惯例”，本书不替标准补它没写的属概念。
10. “每张架构视图恰好对应一个架构视角，这是标准规定。”——一半对。规范正文只写”一个视角治理一张或多张视图”（§5.2.7）；“每张视图恰有一个视角”见资料性附录 A.5（每张视图都必须有一个架构视角的规格，须符合一组惯例），OMG 技术缺陷单只是旁证。
11. “关注点就是需求，收得越全越好。”——错。关注点属于人、需求属于系统；三条判据（换人测试、改写测试、粒度检查）分得开。关注点不是判据，是判据的现场。

12. “定义里说了架构体现在元素与关系之中。”——错。那是 2011 版口径；2022 版已删去该层，它降为 42020:2019 §3.3 的注 1，注释级、没有术语地位。
13. “架构决策就是架构。”——错。决定是认出架构、并把架构记下来的抓手；架构决策在标准里另有定义（§5.2.12），是架构描述必须记录的内容（§6.10）。
14. “领域没有现成的视角标准，就只能等。”——错。标准故意不规定视角清单；缺领域 ADF，就按规格自建一个项目 ADF，这是标准设计的正规路径。
15. “架构评估就是审文档。”——错。评架构好不好，对的是目的（42030）；评架构描述合不合适，对的是细化过程的适宜性（42020）。串了尺子，评估就退化成文档审查。

## D 练习

1. 挑一个你正在做的系统，答”承重测试”：哪些东西”去掉它，系统就不再是它”？写下至少三条。
2. 对上一题的三条，逐条跑两遍：兑现侧（实现目的需要它吗）与演化侧（目的变了它守不守得住）。
3. 找一条你们团队常说”很重要”的决定，用假判据检验一遍：它是不是因为”贵”才被叫架构？去掉它，目的还达得成吗？

4. 列出你们系统的利益相关方与各自的关注点（至少五个，含监管、运维、维修、退役处置）；哪一条你从来没考虑过？
5. 拿一条”XX 架构”的清单（业务、数据、应用、技术），用一条真实场景横着穿过去；走不通的地方，就是”多个架构”的病根。
6. 找出你们团队所有叫”XX 架构”的图与文档，按四类归位：关注域视角、描述层次、架构风格、框架。有没有被当成四个架构的？
7. 用”架构视角 → 架构视图”分析你们某张架构图：它采用什么视角（规则）？回答谁的什么问题？刻意忽略了什么？
8. 检查一次架构评审：评审的标的是一张图，还是”那组去掉就不是它的基本概念或属性”？把不该评的剔出去，把漏掉的请进来。
9. 你们用过的框架替你们定了什么、没替你们定什么？把”没替你们定的”列出来——那些就是架构决策。
10. 你们团队有没有一份”从没写下来、但大家都在守”的规矩？写下来，标出哪些是”去掉就不是它”的，哪些只是习惯。
11. 用四问的”它从哪来”量一次”架构”：从建筑图纸到基本概念或属性，这个词经历哪几个阶段？每阶段指什么？
12. 找一份你们的架构描述，逐条对 §3.14 的三个动词：它能不能被**单独识别**、被**存储**、被**交付**给别人用？

（答案要点见书末附录，与训练教材题卡同源）

## E 参考文献

### 标准

- R-04 ISO/IEC/IEEE 42010:2022（+ GB/T 45630-2025 等同采用）—— 架构定义（§3.2：实体的基本概念或属性+实现与演化的管控原则）；架构描述=工作产品（§3.3）；架构描述元素（§3.4）；架构视角=一组惯例、定义句无属概念（§3.8）；架构视图=信息部件（§3.7）；关注点（§3.10）；信息部件（§3.14）；规格（§3.16）；架构可以关乎的五类（§5.2.2）；关注点往往主观、有些与实体成功相抵触（§5.2.3／§5.2.4）；视角治理视图（§5.2.7）；架构描述元素所含概念（§5.2.9）；架构决策与理据（§5.2.12／§6.10，含 §6.10.1 记录取舍准则）；架构描述要交代的内容（§6.2~§6.10）；视角规格的 shall 清单（§8.1）；架构描述框架（§3.5／§7.1）；四种隐喻与两种哲学（附录 A.2）
- R-05 ISO/IEC/IEEE 42020:2019（AWI 42020 修订在研，据 24 号／32 号；库外待核）—— 架构定义来源（§3.3，其注 1 承接 2011 版的”体现于元素与关系”并降调）；架构实体（§3.6）；关注点（§3.8）；治理与管理的分工（§5.3~§5.5）；治理的裁决动作（§6.4.3／§6.4.4／§8.4.4）
- R-85 ISO/IEC/IEEE 42030:2019（2019 首版；库内正本无版本状态记载，32 号记 AWI 修订在研（ISO 93814），属库外待核）—— 架构评估以规定的评估目标为基准（§3.4）、对照架构的预期目的

评估其质量与价值（§1 a)~d)、§4.5）；评估的对象是架构、不是架构描述的适宜性（§1 注）；三层评估（§4.3）

- R-102 ISO/IEC/IEEE 15288:2023 —— 架构定义（§3.5，主体为系统）；设计定义”足够完整到足以支撑对架构的一次合规实现”（§3.13）；系统（§3.46）；系统元素（§3.47）；架构实体与系统元素是两种不同的概念（§5.9，依中英文对照版，英文原文作 notions）

**经典文献**

- R-19 TOGAF（The Open Group，9.x 与 10；10 全文未取到）—— 企业架构的四域（业务／数据／应用／技术），本书按”一种常见的视角组织方式”引用
- R-32 Kruchten,“The 4+1 View Model of Architecture”（IEEE Software, 1995；经典文献，非标准）—— 软件领域最经典的视角集合，本书按”一种常见的视角组织方式”引用
- R-33 Simon Brown,“C4 model”（c4model.com；非标准，具体版本号未核）—— 四层缩放
- R-34 John Zachman,“A Framework for Information Systems Architecture”（IBM Systems Journal, 1987；经典文献，非标准）—— 企业架构分类矩阵

**内源（本项目研习报告，只列不作引用依据）**

- 30-架构描述核心概念研究报告（v1.4）、32-架构视角与关切判据研究报告（v1.2）、24-架构与设计边界研究报告（v1.8）、06-架构

概念研究报告（v5.8）、07-架构概念研究报告（v1.10）、22-架构、概念、属性研究报告（v1.8）

> 对照阅读：本章”架构不是设计”呼应第 6 章的框架定义与第 15、16 章的两类工作；“评审对象是架构描述”呼应第 8 章；“能冻结的是架构描述”呼应第 7 章。

# 第 11 章 架构设计：概念幸存、属性涌现、原则自加

> 本章概念：架构设计 / 基本概念 / 基本属性 / 管控原则
> 英文来源：architecture design / fundamental concepts / fundamental properties / governing principles（ISO/IEC/IEEE 42010; 42020; 15288）主案例：冰链科技 M3 二代（产品侧）｜镜照：恒信集团合同全生命周期平台（IT 侧）术语口径：本章「架构视角／架构视图／关注点／基本概念或属性／管控原则／生存周期／利益相关方」按 GB/T 45630-2025（等同采用 ISO/IEC/IEEE 42010:2022）定名；本书早期稿件的「治理原则」即「管控原则」(governing principles)、「视点」即「架构视角」、「关切」即「关注点」。对照表见附录 A。

---

## 11.1 误解现场

冰链科技决定给 M3 做一个大升级。物流车队反馈：有些断链是可以提前预判的——箱门虚掩、温度骤升的前兆、电池低电量，“等它真断了再告警，已经晚了”。

水忠提了个方案：加一个”边缘断链预判”能力，在箱内就把风险算出来，提前告警。

要升级就得先做架构设计。水忠把相关的人叫进会议室，准备了三样东西。

第一样，一张新的四层方块图。在”固件层”和”云端层”之间，多加了一个框，写着”边缘预判模块”。

“这就是新一代 M3 的架构。”水忠指着那个框。

第二样，一页技术选型。新传感器型号、边缘芯片型号、预判算法跑在哪个芯片上、数据上报协议用什么——一页纸列得清清楚楚。

“选型都定了，可以直接下单打样。”

第三样，一页”老规矩”。这是第 10 章复盘后整理的：温度数据归属追溯域、断链检测是刚性边界、固件升级必须向后兼容。

“架构的规矩，都在这里了。”

文正看完三样东西，问了三个问题。

第一个问题：“预判逻辑，到底跑在硬件上还是跑在云端?”

水忠一愣——那个框悬在固件层和云端层之间，他画的时候没想过这个问题。

第二个问题：“‘提前告警’要提前多久？提前 30 秒、5 分钟还是 1 小时？1 小时前告警可能是误报，30 秒前告警可能来不及。”

水忠更答不上——他列了技术选型，没列”这个能力到底要做到什么程度”。

第三个问题，也是让李旭憋了很久的问题：“这个能力加进来之后，M3 还是 M3 吗?”

李旭说，“你上次说，M3 的架构是’断链即报废还能撑到接入冷库、全程可追溯’。

现在加了个’预判’——这是 M3 长出了新的本质，还是只是多了一个功能?”

水忠沉默了。三样东西——图、选型、老规矩——加在一起，也没能回答那三个问题：预判逻辑跑在硬件还是云端？“提前告警”要提前多久？加进来之后，M3 还是不是 M3？

会议室里安静下来。水忠忽然说不清一件本该很基本的事：他刚才做的，到底算不算”架构设计”？如果算，为什么三样东西连这三个问题都答不上来？如果不算，真正的架构设计又该是什么？

产品开发经理永兵也在被叫来的人里，他最后开口：“咱们做了这么多年 M3，怎么被问到’它是什么’，一句话都说不上来。”

这一章，我们就把”架构设计”这件被画图、选型、定规矩淹没的事，单独拎出来。

你会发现三件反直觉的事（这正是本章三条本性的头两条：概念幸存与属性涌现；第三条原则自加，第二幕一并立起来）：

**架构设计不是画图、选型、定规矩——它是把需求翻译成”架构”的活动；架构只有两样内容（基本概念或属性＝本质、管控原则＝governing），而元素与关系是这两样的落点——它们属实体侧，其规格归下游的设计定义过程。本质不是凭空设计的——基本概念是幸存下来的，基本属性是涌现出来的。**

---

## 这一章要回答什么

读完本章，你应该能回答：

1. 架构设计到底是什么？为什么”画图、选型、定规矩”三样加在一起，仍不算架构设计？
2. “定义”和”设计”差在哪？为什么标准里只有”架构定义过程”，没有”架构设计过程”？
3. 架构设计的输入是什么？为什么”质量属性才是主要驱动”？为什么必须是规定需求？目的与任务为何居判据侧首位、以确认单锚

定？目的是什么、怎么得到、怎么判断目的到位、以目的检验怎么三分？

4. 架构设计要产出什么？为什么说它只产出”本质＋管控原则”两样，而元素与关系是这两样的落点、其规格归下游的设计定义过程？
5. 基本概念是怎么来的？为什么说它是”幸存”下来的，不是”设计”出来的？命名为什么是概念化的完成仪式？
6. 基本属性为什么不能直接设计？“设计实体的构成，让属性涌现”是什么意思？
7. 本质（概念/属性）与管控原则各自怎么”定”？落点那一层的元素与关系又怎么”定”？为什么说”关系是成本”、管控原则”是自加的”？
8. 架构设计和详细设计怎么分工？分界为什么是种类而不是粗细？为什么”先横后纵”、横向那层错了纵向挖得再深也是白挖？
9. 利益相关方和关注点，是”设计”出来的，还是”识别”出来的？为什么说”漏掉一个利益相关方，就是漏掉一整类关注点”？
10. 构思、定义、设计是一条什么链？为什么说”构思打开可能性、定义终结漂浮、设计转换到底”？“设计”为什么必须声明取广义还是狭义？

11. 架构生成六步是哪六步？为什么”没有背书的架构，只是架构师的个人意见”？
12. 创新和构思/定义/设计是什么关系?为什么说”架构式创新的主导动作恰是定义”?

---

## 这一章的骨架

这一章要立的是”架构设计”。先把骨架立起来——后面三幕，都是给它添血肉。

> **架构设计，是把需求翻译成”架构”的活动。而按 2022 定义，架构只有两样内容：实体的基本概念或属性（本质）＋管控原则（governing principles，有约束力）。设计还必须把这两样表达成可被评审、可被基线化的架构描述，并交代它们在实体构成上的落点（元素及其关系——它们是架构”可以关乎”的对象，其规格归下游的设计定义过程），并用目的判据回答”什么才配叫基本”。**

这一章的骨架是**一个动作 + 两样内容 + 一次表达 + 一条动词链**：一个动作（把需求翻译成”架构”——设计是动词，定义是完成时，没定形等于没设计）；两样内容（本质＝基本概念或属性；管控原则＝约束实现与演化及其生存周期）。

一次表达是把这两样写成架构描述（AD）——概念怎么定、属性怎么来、管控原则怎么立，都在设计里交代。至于元素与关系：**它们不是第三样内容**，而是架构”可以关乎”的对象，属**实体侧**，其规格归下游的设计定义过程（§11.2.4 专讲这件事）；一条动词链是构思打开可能性，定义终结漂浮，设计转换到底（§11.2.3 立，第三幕工程化为六步）。

骨架之上，还有三个”不是设计出来的”反直觉，比任何方法更先要立住：**基本概念是幸存下来的、基本属性是涌现出来的、管控原则是自加的**。三样东西（画图、选型、定规矩）把这几个本性盖住了大半辈子——第一幕先把它剥开。落点那一层另有一条纪律：**关系是成本，要克制**。

第一幕钉”架构设计是什么”（一个动作：翻译出本质与管控，并把它表达成架构描述），第二幕分开它和容易混的东西（三样东西不是架构设计 + 三个”不是设计出来的” + 落点那一层的纪律），第三幕回答”往哪去”（概念怎么定、属性怎么来、管控原则怎么立，动词链与六步怎么跑）。方法底座时刻，是水忠把三样东西换成”能定形的东西”的那一天——从”见过”走向”拥有”（章末详说）。

---

## 11.2 第一幕 架构设计是什么（定界）

> 这一幕把”架构设计”看清：先立一个动作（把需求翻译成”架构”），再听标准怎么说（设计是转化、定义是完成时），然后看

> 清定义给的两样内容（本质＋管控原则），再交代元素与关系的去向（它们是架构”可以关乎”的对象，不在定义句内，规格归下游的设计定义过程），最后划开它和详细设计的边界（横切 vs 纵挖）。

### 11.2.1 一个动作：把需求翻译成”架构”

章首那场会，水忠以为带齐三样东西就做完了架构设计。三样东西为什么都不算，第二幕专门磨；这一节先回答那个被淹没的问题：**架构设计到底在干什么？** 一句话：

> **架构设计，是把需求翻译成”架构”的活动。** 按 2022 定义，架构只有两样内容——实体的基本概念或属性（本质），以及管住这个实体及其相关生存周期过程怎么实现、怎么演化的管控原则。

注意这句话里的两个关键词。

第一个关键词是”翻译”。架构设计的输入是需求（这个实体要做什么、做到什么程度），输出是架构（这个实体在概念与属性上的根本设定，加上管住它实现与演化的管控原则）。

“翻译”不是照搬，而是**必然做选择**：同一份需求可以译出不同的架构；“要提前告警断链”可以译成”边缘计算”（箱内算），也可译成”云端预判”（上报后算）。

翻译的每一个选择，都是架构决策的种子。

第二个关键词是”架构”。架构设计关心的不是某个部件的内部实现（那是详细设计的事），而是这个实体在概念与属性上的根本设定——定义给的第一样内容。

定义给的第二样是管控原则——约束实现与演化的规矩，它同样不由部件内部决定。

两样之外还有一层**落点**：它们必须落得下去，落到实体的构成（成分及其关系）上。它们是架构”可以关乎”的对象（42010:2022 §5.2.2），不在定义句内，其规格归下游的设计定义过程。

这一节只钉住那个动作，两样内容各是什么、落点在哪一层，下一节 §11.2.4 一起收全。

### 11.2.2 标准怎么说：设计是转化，过程以产出物命名

“架构设计”不是我们的自造词，它有标准出处。

先看一个容易被忽略的前提——**“设计”这个词，在标准里是怎么定义的**。

ISO 9000:2026 §3.3.11 给”设计和开发”（design and development）下过一个定义：

> **设计和开发——将客体的要求转换为对该客体更详细要求的一组过程**。

注意它的动词：设计是”转化”（transform）。更要注意它的输入和输出：**都是需求**——输入是需求，输出是”更详细的需求”。特性与规范落成文档，就是产品数据（第 7 章）；这份产品数据转过身，就是下一层设计开发的输入（第 6 章的逐对接力）。

**设计不负责说”系统是什么”，负责说”系统怎么兑现承诺”——而兑现承诺的方式，本身又写成一份更详细的需求。**

这条”转化”链上，架构设计是**第一站**——把系统需求，转化为这个实体的基本概念或属性与管控原则（15288:2023 §6.4.4 目的句：生成系统架构的不同方案、选定、并以一致的视图和模型表达）。

> 措辞要对准：§3.3.11 那句”转换为更详细的要求”，落点在**下一站**——把架构转化为元素及其关系的规格，那是设计定义过程（15288:2023 §6.4.5）。把架构定义也说成”转化为更详细的要求”，就把架构读成了”更详细的需求”，那正是 2022 定义要挡掉的读法。

而”架构设计”这个词本身，标准里其实没有直接使用。ISO/IEC/IEEE 15288（系统生存周期过程）里，有两个并排的过程：**架构定义过程**（Architecture Definition Process，正本全名 System architecture definition process，6.4.4）和**设计定义过程**（Design Definition Process，6.4.5）。

架构定义过程的输入是系统需求，输出是架构描述；设计定义过程的输入是架构定义过程的输出，输出是各元素的详细规格。

为什么是”定义”，不是”设计”？这个命名玄机，值得单独说——它藏在”定义”和”设计”这对元概念的区别里。

### 11.2.3 “定义”与”设计”：划界 + 陈述，还是怎么做

“定义”（definition）和”设计”（design）是架构领域天天混用的两个词，先厘清这对元概念，才能看懂 15288 为什么用”定义”命名两个过程。

先把词源摆出来。

**definition ← 拉丁语 de-finis**——de（彻底地）+ finis（界限）→ “划出边界”。**定义 = 划界 + 陈述**：先确定它覆盖什么、不含什么，再写成精确的描述。

它的语言性质是陈述性的（descriptive），回答”它是什么”；评价判据是求真——准确、无歧义、可验证。

**构思（conceiving）← 拉丁语 con-capere**——con（收进来）+ capere（握取），“怀在脑中”，像受孕一样孕育可能性。构思回答”可能是什么”，评价判据是**发散**——从一个关注点生成多个候选架构概念，而不是直奔唯一解。**构思打开可能性**。

**design** 是另一回事。它回答”怎么做、怎么造”（how），语言性质是规范性的（prescriptive），给出方案。

评价判据是求优——满足需求、权衡取舍、可实现。

| | 定义 Definition | 设计 Design |
|---|---|---|
| 回答的问题 | 它是什么（what it is） | 怎么做 / 怎么造（how） |
| 语言性质 | 陈述性（描述事实） | 规范性（给出方案） |
| 评价判据 | 求真：准确、无歧义、可验证 | 求优：满足需求、权衡取舍 |
| 典型产出 | 术语表、需求规格、判据、边界 | 方案、接口契约、详细特性、图纸 |
| 生存周期位置 | 前端，框定问题空间 | 中段，在解空间里创造 |

但”设计”这个词，还有一重必须随身携带的身份——**一词两义**。

**广义设计**（ABET 工程教育认证）：engineering design = 为满足需要而对系统、部件或流程进行**迭代决策与创造的容器过程**——构思、定义、表达都被它装在里面。**狭义设计**（15288 §6.4.5 设计定义过程）：以**已生效的架构基线**为输入，把架构逐层细化为可实现元素与契约的**下游流程**。

说”设计”之前，先声明取哪一义：并列谈时是狭义（基线生效后的下游）；说”设计是动词（构思）“时是广义（把构思也装进去的容器）。不声明，这个词就在容器与下游之间漂移——你说”我们在设计”，可能是构思阶段，也可能是基线后的细化，听众无从判断。

**关键洞察**：设计本质上也是”定义”的一种——只是定义的对象从”问题/术语”换成了”解决方案”。

15288 里只有”架构定义过程”“设计定义过程”，没有”架构设计过程”，因为**过程以产出物命名，而产物必须被定义**。“生成候选架构”是 15288 §6.4.4 **目的句**里的第一个动作（生成备选、选定、表达）；42020 把它拆在 §8.4.5 综合潜在方案、§8.4.7 形成候选架构两处。设计定义过程（6.4.5）排在架构基线生效之后，才是狭义的”设计”。

三句话把这条动词链钉住：**构思打开可能性，定义终结漂浮，设计转换到底**。

构思无法评审、停在脑子里，描述才能评审、验证、基线化——定义才能进入流程。

水忠那天做的是一种”构思”，但没走完”定义”，拿不出”定形”的东西——他的架构设计没有任何可评审、可验证、可基线化的产物。

一句话：**设计是动词（构思），定义是它的完成时（定形）**——此处的”设计”是广义（容器）；在更细的三动词链里，它是”构思 → 定义 → 设计（狭义）“：构思打开、定义定形，狭义的设计在基线生效后接手、转换到底。

这解释了一件很多团队困惑的事：为什么”我们做过架构设计”和”我们有架构描述”是两回事——前者是动词，可能只发生在几个人的脑子里；后者是完成时，是能被检查、能进入配置管理（第 13 章）的资产。

**设计得再好，没定形，就等于没设计。**

### 11.2.4 要产出什么：只有两样，元素与关系去哪了

这一章的核心命题：架构设计要产出的，就是定义里那两样东西——外加把它们表达成架构描述、并交代它们在实体构成上的落点。还记得第 10 章那条架构的权威定义吗：

> **Architecture — fundamental concepts or properties of an entity in its environment and governing principles for the realization and evolution of this entity and its related life cycle processes.**（ISO/IEC/IEEE 42010:2022 §3.2；[SOURCE: ISO/IEC/IEEE 42020:2019, 3.3]）

> 中文：架构——一个实体在其环境中的基本概念或属性，以及为实现和演化该实体及其相关生存周期过程的管控原则（governing principles）。
> 引文口径：中译按本书定名取「基本」（GB/T 45630-2025 3.2 作「基础概念或属性」）；逐字引国标时照抄「基础概念或属性」。正本附录 A.2 说明 concepts or properties 是”不抱偏见”地容纳两种哲学（架构即概念／架构即属性）——概念**或**属性取一即可，不是并列两项。

第 10 章用这条定义回答”架构是什么”；这一章用同一条定义回答”架构怎么被设计出来”。定义只给两样，架构设计要产出的就是这两样。那么元素与关系去哪了？

**第一样，本质——实体的基本概念或属性**。它回答系统”怎么想、具备什么”。标准写的是 concepts **or** properties（“或”：概念与属性是两个认知入口，不必同时具备）；可一旦要设计，两个入口都要交代——概念回答”系统在概念层面是什么”（怎么想），属性回答”系统具备什么本质性质”（怎么被观察、怎么度量）。概念不是凭空设计的，是**幸存**下来的；属性不是直接设计的，是**涌现**出来的——第二幕各用一节讲。

**第二样，管控原则（governing principles）**。它有约束力地管住这个实体怎么实现、怎么演化、相关生存周期过程怎么走。标准只有这一句——不拆成”设计规则/演进规则”两条；“怎么把它造

对”与”怎么变都不出局”是这一句管控原则要同时约束的两个方向（§10.2.2）。管控原则不是抄来的，是自己**立**的——第二幕讲。

**元素与关系去哪了——实体侧那一层**。两样内容不是悬空的：它们要落到实体的构成上——组成成分（components）与成分之间的关系（relationships），也靠架构描述（AD）去交代。标准不在定义句里列出这两样（§10.2.5）；架构设计仍必须回答”这个实体由哪些成分组成、成分之间怎么连、哪些连接断不得”——它们不在定义句内，属**实体侧**，其规格归下游的设计定义过程（15288:2023 §6.4.5）。关系不是越多越好，要**克制**，第二幕讲。

所以”基本”二字凭什么裁决？**fundamental 的准入判据是本书形式化**：42010 §3.2 与 42030 §3.1 都只给定义句、不给这条判据，本书把它形式化为「必要性（membership）＋充分性（closure）」双判据，参照系是系统在其环境中的**目的**；**不是数学意义的充要条件**——充分性仅指”目的所要求的结构条件已被完整表达”。

IEV 831-01-01 Note 2 系二手派生表述，仅作佐证；本质两样与管控原则都在目的这条判据下裁出”什么配叫基本、什么要立成原则”，它将在 §11.3.6 被操作化为不变量判据。

> 术语注：上面把本质拆成”概念/属性”两个入口、把落点单列”元素/关系”，是本书为教学的展开——对账标准原文时记得：定义给的是 concepts **or** properties（一个 or 概念）加一条 governing principles（管控原则，约束该实体的实现、

> 演化及其生存周期过程；governing＝有约束力）；elements/relationships 不在定义句里——最多是注释级的”通常意指体现于”（42020:2019 §3.3 注 1）、以及”架构可以关乎”的对象（§5.2.2）；它们属实体侧，规格归设计定义过程（§10.2.5）。落位是本书的选择，别把它当定义原文。

水忠要做 M3 二代的架构，就是回答：M3 的本质怎么想、具备什么（概念/属性），管控原则怎么立（约束实现与演化），这两样又怎么落到 M3 的构成（成分及其关系）上。

### 11.2.5 横切 vs 纵挖：架构设计 vs 详细设计

理解了”定义 vs 设计”，再看架构设计和详细设计的边界，就顺理成章了。

它俩都是”把需求转化为特性”的设计活动，但**管的性质完全不同**（15288:2023 §6.4.4／§6.4.5）：

| 维度 | 架构设计 | 详细设计 |
| --- | --- | --- |
| 管的性质 | **适宜性、存续性、合意性**（对目的与关切而言） | **与技术及其他设计元素的兼容性、构建与集成的可行性** |
| 核心问题 | 这个实体在概念与属性上是什么、实现与演化守什么规矩 | 已划定范围之内，部件如何**实现** |
| 决策内容 | 本质、原则、边界、接口、权衡 | 算法、数据结构、逻辑流程 |
| 变更代价 | 高——牵一发动全身（**代价是后果，不是分界**） | 相对低，局限在部件内 |
| 典型产出 | 架构描述、视图、ADR | 详细设计说明书、类图、伪码 |

分界不在粗细，在**种类**（第 10 章 §10.4.3 已立）。有一条可判定的判据：**这份内容能不能被设计违反？**能被违反的（本质的设定、边界、接口约定、数据所有权、向后兼容的承诺），属架构设计这一侧；不能被违反的（某个接口用 REST 还是 gRPC、某张表的索引怎么建），属详细设计那一侧。

15288 把这条界写得更直白：**架构管的是适宜性、存续性与合意性；设计管的是与技术及其他设计元素的兼容性，以及构建与集成的可行性**。它还补了一句质量要求——**有效的架构尽可能不依赖于设计**。一个逼着设计只能有一种做法的架构，通常不是严谨，是越界。

**一句话记法**：架构设计决定”哪些墙该砌、哪里开门”；详细设计决定”每个房间怎么装修”。

还有一对更形象的分工——**横竖分工**。架构设计是**横向铺**：横跨全部关切与整个生存周期，管的是”对目的而言合不合适、撑不撑得住、值不值得”；详细设计是**纵向挖**：在已经划定的范围里往实现里挖，管的是”和技术兼不兼容、造不造得出来、能不能集成”。

**先横后纵**——横向那层定错了，纵向挖得再深也是白挖。

再往下走一层：详细设计（纵挖模块内部）之前，模块自己也有一个”框架定义”——内部结构、依赖方向、实现策略，**先定义再实现**；这块归模块工程师自己（模块内兼模块架构师）。系统有架构、模块也有架构——42010:2022 的主体是**实体**，模块也是一个实体；每一层都先定义”它是什么”，再把它做出来。层级不是分界，种类才是：同一个实体身上，管性质的那一层是架构设计，往实现里挖的那一层是详细设计。

注意，这一章讲的”架构设计”，和第 6 章讲的”框架定义”是两个层次的东西。

第 6 章那棵树的树干树枝（框架定义），管的是”结构是什么、承接了谁、往下分解出什么”；而架构，是那棵树里不可还原的部分（§10.2.2 说过）。

架构设计，正是要定出这组不可还原的**基本概念或属性**与管控原则——它在框架定义的上游，比框架定义更能决定后面的一切。

这条”横切 vs 纵挖”的分界，是架构设计最近的一道边界。第二幕的磨边弧，会把这条边界继续往外磨——分清它和图、选型、老规矩的分界，以及它到底吃什么。

## 概念卡片 · 架构设计

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 把需求翻译成”架构”的活动——定出实体的基本概念或属性（本质）、立起约束实现与演化的管控原则（定义给的两样内容），并把它们表达成架构描述、交代它们在实体构成上的落点；产品开发中最难撤销的一组内容（**不可逆性是代理判据，不是准入判据**——它会漏判不贵但承重者、误纳很贵却与目的无关者，§11.3.6） |
| **它不是什么** | ⑥⑦ | 对立=详细设计——架构横向铺（跨全部关切与整个生存周期：管适宜性、存续性与合意性），详细 |
| **它[illegible]来** | ④⑧⑨ | [illegible] |

## 案例进展①｜水忠的第一步：先搞清楚架构设计吃什么

会议散了，三个问题还悬在桌上。水忠把李旭、文正、董勐留下，没急着补图、补选型、补规矩——刚立的那句话还在耳边：架构设计是把需求翻译成”架构”的活动。那翻译这件事，第一步该干什么？

李旭替他开了个头：“先别管翻译成什么样。你得先说清——这个’翻译’，吃进去的是什么？”

水忠想了想：“是需求。但要翻译的需求，不是文正嘴里那句话，是……”

他停住了，“我得先搞清楚，架构设计到底吃什么。”

---

# 11.3 第二幕 架构设计不是什么：幸存、涌现、自加（磨边界弧）

> 这一幕把”架构设计”和它容易混的东西分开——三样东西不是架构设计；吃进嘴的不是需求原文；要产出的两样内容，没有一样是”直接设计”出来的——本质（概念幸存、属性涌现）、管控原则（自加）；落点那一层另有一条纪律：元素关系要克制，因为关系是成本。一条连续磨边弧，把边界一条条磨出来。

## 11.3.1 先看病：三样东西为什么都不是架构设计

上一节划开了架构设计与详细设计的分界——“最近的一道边界”。这一节往更远磨：水忠带进会议室的三样东西（图、选型、老规矩），是

三个更常被误当成”架构设计”的东西。现在把它们摆在一起，分清层次。

**图，是架构描述**。第 10 章讲过，架构描述是把系统不可还原的涵义，按”利益相关方 → 关注点 → 架构视角 → 架构视图”切给不同的人看。

水忠的图画的不是”怎么设计”，是”设计完之后长什么样、给谁看”。它是产物，不是动作。

**选型，是候选**。传感器选哪个、芯片选哪个、协议用哪个——这些是”选项”，是待检验、待权衡、待决策的素材。

选型表本身没有回答”为什么选它、不选它的代价是什么”。它只是把选择摆上了桌。

**老规矩，是管控原则的候选/雏形**。“温度数据归属追溯域”“断链检测是刚性边界”这些规矩约束的是”怎么把它造对”；另一些老规矩管”怎么变不破坏”（如固件升级必须向后兼容）。它们不是管控原则本身——是管控原则立法时从既有实践里读出来的候选，过四关才升格（§11.3.8 细讲，案例进展⑤）。

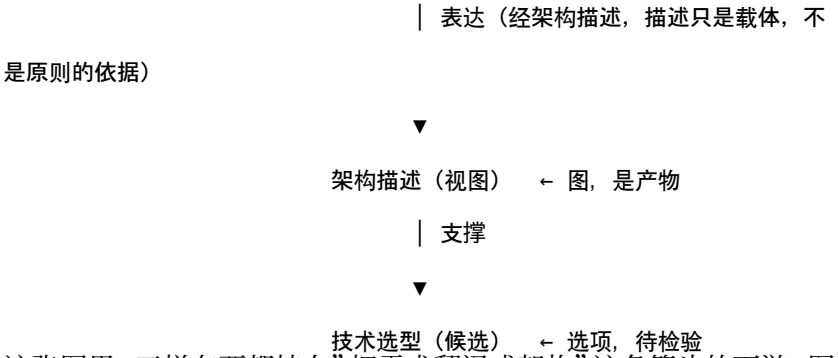
三样东西，没有一样回答了”架构设计到底在干什么”。

水忠画了图，但没做切分的权衡；列了选型，但没定义部分之间的连接；写了老规矩，但那是从既有系统里”读出来”的，不是”设计出来”的。

**三样东西——图、选型、老规矩——分别只是产物、候选、规则；架构设计是另一个更靠前的动作：把需求翻译成”架构”。**

把这三样东西在架构设计周围的层次摆开，是这个形状：

```
需求 ──架构设计──▶ 架构（不可还原的涵义）          ← 架构本体
                    ├── 基本概念或属性（本质）      ← 承重件
                    └── 管控原则（老规矩是候选）    ← 规则
```




这张图里，三样东西都挂在”把需求翻译成架构”这条箭头的下游：图是**产物**、老规矩是**规则（候选）**、选型是**选项**——没有一样是翻译本身。

图注两处要点：一，**管控原则与基本概念或属性并列，同属架构本体**，不是架构描述的下游；架构描述只是表达载体，不是管控原则的依据。二，**元素/关系**在现行标准里没有术语地位——42020 §3.3 Note 1 只在注里提 components 与 relationships（即本书的”元素”），2022 版定义句里已删去 2011 版的 embodied in its elements, relationships；本书把它作落点处理，图中从略。

### 11.3.2 吃进嘴的不是”全部需求原文”：输入六类

先立一条边界：**架构设计吃的，不是”全部需求原文”**。需求原文是原料，架构设计吃的是从原料里挑出来、归拢好的一组输入——一组更宽、也更讲究的东西。

本书把它归拢为六类（目的与任务为首类，需求含功能/非功能两类）：

| 类别 | 具体内容 | 典型例子 |
| --- | --- | --- |
| **目的与任务**⭐（判据侧首位） | 系统为什么存在（why）——使命、业务目标、商业论证、任务书 | 企业战略目标、产品定位陈述 |
| **需求** ⭐（含功能需求、非功能需求两类） | 功能：系统要做什么（what）；非功能：做到什么程度（质量属性） | 功能：用例；非功能：响应时间、可用性、扩容倍数 |
| **约束** | 不可谈判的限制 | 指定技术栈、法规合规、成本上限、工期 |
| **环境** | 系统所处的上下文；与目的构成”目的能否达成”的**第三类硬条件**（法规／物理条件／外部接口／资源） | 部署在边缘设备、必须对接遗留系统、团队能力 |
| **关注点** | 利益相关方关心的 | 老板关心成本、运维关心可观测性、安全官关心隔离 |
| **资产** | 可复用的输入 | 参考架构、既有代码、行业标准 |

六类分两组职能：**判据侧**（目的与任务 / 关注点 / 需求）供给”什么是根本”的判据，内部有严格位次——**目的与任务居首位、起决定作用**，是 fundamental 成员资格的裁决者；关注点是**原料**、规定需求

是**产物**、目的是**准绳**，三者”目的为纲、关切与需求为目”（与目的冲突的需求，要么澄清、要么拒绝）。

判据对目的量、不对关切点量（元素的资格问题）；目的对关切点账、不对着偏好量（目的的合法性问题）——两句成对用，缺一，要么目的独断、要么架构被偏好撑胖。

**选项空间**（约束 / 环境 / 资产）圈定”在什么范围内选”——约束砍候选、环境定舞台、资产提供可继承的不变量。目的与任务居判据侧首位，以《系统目的或任务理解确认单》锚定进入设计（§11.4.3 展开）——与第 10 章”概念层的目的”不冲突（第 10 章讲目的”是什么”，这里讲目的作判据输入进入架构设计）。

关注点一类还要有自己的**关切点台账**（登记与分流，42010 §6.4；细则见第 15 章 §15.4.5）。

注意分类里的一个隐藏信息：**功能需求不排第一位（非功能需求才是主要驱动）**。

把一个从业界到学界反复强调、却总被忽略的论断单独拎出来——

> **非功能需求（质量属性）才是架构的主要驱动**。（Bass / Clements / Kazman，Software Architecture in Practice）

功能需求告诉你”做什么”，质量属性告诉你”做到什么程度”，而**“做到什么程度”才真正决定结构形态**：

- 要 99.99% 可用 → 冗余与故障域设计；
- 要支持 10 倍扩容 → 无状态与分片；
- 要快速上市 → 最少部件数。

**功能相同的系统，质量属性不同，架构可以完全不同。**

水忠面对的”断链预判”需求，功能上就是”算风险、发告警”六个字；但”提前 30 秒还是 5 分钟”“误报率要压到多少”“算错了的后果谁来兜”——这些质量属性，才真正决定预判逻辑放硬件还是云端、要不要双算、要不要置信度门槛。

功能需求划出轮廓，质量属性决定体重——**轮廓决定长相，体重决定能不能站起来**。

### 11.3.3 还得是”规定需求”：吃未分配的系统需求

再立一条边界：**不是任意需求都能喂给架构设计**——是规定需求，而且是**尚未分配到部件**的那一层（系统的场合就是系统需求）。输入不只要”全”，还要”稳”。第二论断：

> **架构设计的输入，应当是规定需求（specified requirements）级别的产物——经过评审、被明示、可追溯的需求**。

规定需求这个概念，第 4 章讲过：被明示的、以成文信息记录的需求，是验证的参照。

为什么架构设计要吃规定需求，而不是”文正嘴里那句话”？因为架构设计定的是这个实体的基本概念或属性与管控原则——它们一旦错了，后面的属性、原则与全部设计都在错的地基上盖楼（§11.4.2 推论一）。

**在流沙上画架构，等于开局自损一目**——需求一变，本质与原则就得跟着动；而它们一动，后面全跟着动。

水忠若拿”要能提前预判断链”这句话直接去设计，会在第一版结构上反复返工——因为这句话既没定义”提前多久”（属性缺失），也没定义”判错了怎么办”（约束缺失）。

架构设计的输入，应该是把这些补全之后的规定需求。

“架构设计的输入是系统需求”——这句话里有个容易被忽略的词：**系统**。

需求是有层级的，架构设计和详细设计吃的不是同一层：

**需求层级（分解深度：浅 → 深）**

**利益相关方需求（用户 · 利益相关方）⟶ 系统需求（已验证 · 基线化）⟶ 部件需求（分配矩阵）⟶ 模块约束**

**需求工程　　架构设计（吃系统需求）**

**详细设计（吃部件需求）　　实现**

> 别把这幅图读成”架构粗、设计细”——它画的是**需求**的分解深度，不是架构与设计的粗细之分；两者的分界是种类，不是粒度（§11.2.5）。

- **架构设计**的输入 = **系统级需求的全部**（系统粒度、已验证基线），但不是需求原文的全部——它吃的是需求工程提炼后的系统需求，且吃的是”尚未分配到部件”的需求（**需求分配是架构设计自己的工作，不是输入**）；
- **详细设计**的输入 = **分配矩阵中指向该部件的需求子集 + 部件间接口契约**——它不需要全局视野。

15288 早就承认了这一点：架构定义过程（输入=系统需求）、设计定义过程（输入=架构定义过程的输出）。

**“下一层吃的，是上一层产出的。”** 设计链是接力关系，不是并行关系。

### 11.3.4 质量属性不是”分”下去的：涌现属性只能译

功能需求可以逐条分配，但质量属性是需求侧的目标，它在系统上以（基本）属性的形式涌现（emergent property）——“可用性 99.99%”是**整个系统**的属性，没法切碎了分给某个部件。

它只能被架构**机制**满足（冗余、故障转移、隔离），再**翻译成部件级的设计约束**：

> 系统级：“可用性 ⩾ 99.99%”（架构输入） 部件级：“本模块必须无状态，状态存外部存储”（架构翻译后的详细设计输入）

**质量属性不能”分”，只能”译”**。所以非功能需求必须在架构阶段由架构师亲自解读，不能简单分配给某个模块——根本没有哪个模块”负责可用性”。

这也是第 12 章”设计链两端最高风险”的伏笔：需求组织与产品构件两端的撤销代价最高，中间层的作用是缓冲。

### 11.3.5 关注点不是设计出来的：是识别出来的

输入六类里有一类是”关注点”（concern）。第 10 章讲过关注点天然冲突、只能权衡；这里要补上最关键的一条方法论——**关注点是识别出来的，不是设计出来的**。

**关注点（concern）** 的正本定义：matter of interest or importance to a stakeholder——对利益相关方而言具有利益或重

要性的问题（ISO/IEC/IEEE 42020:2019 §3.8，中文逐字取库内中英对照本）。注意它的三个要素：

- **有主体**——必须挂在某位利益相关方（stakeholder）名下；没有"谁在关心"，就没有 concern；
- **有对象**——事项指向所关注的实体，可涉及其环境、生存周期、架构、需求、设计、实现与运行（42010:2022 §3.10 Note 2），不必限于该实体本身；
- **本质是"事项"**——标准不把 concern 定义为可验证的陈述；concern 是利益相关，不是 requirement。

> 出处注：42020 §3.8 是这条定义的原始出处；42010:2022 §3.10 从它引来，措辞改作 matter of relevance or importance to a stakeholder（对利益相关方而言具有相关性或重要性的事项），并注明"注已修改"；15288:2023 §3.10 亦取 42020 §3.8 的措辞。两册措辞不同、不可互替。42010:2011 的旧句法是 interest in a system relevant to one or more of its stakeholders，本书早期稿件曾据此行文，已不再使用。

**纠偏**：两份正本用的动词都是 **identify（识别）**，不是 design（42010 §6.4 shall identify the concerns；42020 §8.4.3 f) Identify stakeholders and their concerns）。concern 是利益相关方的**利**

**益**，已存在于其头脑、组织和契约里；架构师的工作是**发现、提取、陈述**，而不是发明。

一个”设计”出来的、没有利益相关方真正关心的 concern，不是架构洞察，是自嗨。

识别关注点，有一条可以走完的路径——五步：

1. **识别利益相关方**——谁在关心。组织图、合同、监管、用户群体、开发运维团队。**漏掉一个利益相关方，就是漏掉一整类关注点**；
2. **提取关注点**——关心什么。访谈、既有文档、场景分析、**遗留系统教训复盘**（上一代系统在哪翻的车，就是最真实的 concern）；
3. **划界与陈述**——写成条目（这本身就是一次”定义”）。合并重叠、区分含糊、挂到 stakeholder 名下；
4. **组织编排**——stakeholder × concern 矩阵，分组，**标记冲突组 = 权衡点 = 架构决策的主战场**；
5. **验证完备性**——正向追溯（每个视图/决策都追得到 concern 吗）+ 反向追溯（有 concern 没被任何视图覆盖吗）+ 迭代更新。质量特性一侧可对照 ISO/IEC/IEEE 25010 质量模型清单（功能适用性/性能效率/可靠性/可维护性……）逐项过一遍。

一个好的关注点，四条判据：1. **有主**——挂 stakeholder；2. **有界**——一句话说清覆盖范围；3. **可权衡**——能与别的 concern 形成取舍；4. **可追溯**——驱动视图/决策。

> 术语注：GB/T 45630-2025（42010）的 stakeholder 与 ISO 9000:2026／9001 的 interested party 是两个定义不同的英

> 文词，中文分译两名——本章述 42010 口径一律作「利益相关方」，涉 ISO 9000／9001 口径处仍作「相关方」（interested party）。

关注点与需求是两回事，三条判据把它们分开：**换人测试**（换个利益相关方它还在不在？在→关注点，不在→需求）、**改写测试**（能问出通过/不通过、要指标与验证方法吗？需要→需求）、**粒度检查**（太抽象挑不出架构视角、太具体进了设计；合适粒度＝**刚好足以挑出架构视角**）。

每条关注点必须**四类分流、悬空不是选项**：**承重型**（不满足目的就达不到——**产出架构目标与可登记的"基本"条目**）、**约束型**（**不产出"基本"条目**、只限可行域——转成边界条件**附在既有条目上**）、**偏好型**（只影响方案品味——下沉设计或运维）、**交付型**（影响成本、进度、组织接口——进项目基线）。空着的就是漏项，治理要查。

主张者随附**来源·后果·度量**三件套，**不齐退回补件，而不是驳回**——把"随口说"变成"要举证"。细则见第 15 章 §15.4.5。

清单级判据还有一条——**完备**：矩阵无空行空列。

水忠要给 M3 二代做架构设计，第一步不是画图，是先问：**谁在乎这个"边缘断链预判"？**

司机在乎"别误报烦我"；冷链客户在乎"真要断的时候得留出补救时间"；维修在乎"预判错了会不会误动作"；监管在乎"告警记录要不要留痕、留多久"。

这些 concern 天然冲突——"别误报"和"真断要提前告警"就是一对。**架构设计本质上就是在冲突的关注点之间做权衡；权衡的前提是先看到每个方面**。

## 案例进展②｜恒信镜照：这次，万辉没有先画图

恒信把合同审批系统升级成”合同全生命周期平台”（“全生命周期”是该平台专名，本章其余用语按国标作”生存周期”），架构设计落到万辉头上。上一回（第 10 章）他刚学会一件事：拿一条真实场景横穿四张图，检查它们是不是同一个架构。

这回他照老习惯又要先画图——手停在绘图板边，忽然想起徐漾上次那句他接不上来的追问：这四张图，是从谁在乎什么出发的？他放下笔，决定这次反过来：**先不听图怎么说，先听人在乎什么**。

他列了名单挨个聊：法务在乎审批合规，财务在乎对账准确，业务在乎走单快——这些他早知道。

真正让他愣住的是名单后面那些人：监管在乎追溯留痕，运维在乎出了事能查，合同相对方在乎查进度方便，未来的维护者在乎代码能看懂——前三类他从来没聊过，最后的”未来维护者”他甚至没当对方是利益相关方。

聊完，他把所有”在乎”摊在桌上，第一次做出 stakeholder（利益相关方）× concern 矩阵——有四格是空的：有利益相关方，却没有视图打算回答他们在乎什么。

“我以前的四张图，是从’我要画什么’出发的。”万辉说，“这次是从’谁在乎什么’出发的——矩阵里那些空格，比图上的线条更早告诉我，该从哪下手。”

徐漾点头：“**架构设计的第一步，不是画，是听——听那些’在乎’的人，到底在乎什么**。”

——这正是 §11.3.5 要说透的：**关注点，是识别出来的，不是设计出来的**。

### 11.3.6 基本概念不是设计出来的：概念是幸存下来的

先讲本质里的概念这一面——它是最根本、也最容易被误解的一处。

**“基本概念”不是凭空“设计”出来的，而是“选出来 + 逼出来 + 幸存下来”的。**

90% 的情况不是发明概念，而是**从久经检验的概念库中选择**；剩下 10%，是需求和质量属性**逼**着生成的。

> 术语注：“90%/10%”是本书的**概数**，不是实测统计——它只负责翻掉“概念是发明出来的”这个误解：大多数概念从成熟库里选，少数被需求/属性逼出来。别当精确比例对账。

生成出来的只是“候选”，要经过检验活下来，才配叫“基本概念”。

基本概念有四副面孔，分别回答“系统在根本上是哪一种存在”：

| 面孔 | 是什么 | 例子 |
| --- | --- | --- |
| **组织范式** organizing paradigm | 系统在根本上是“一种什么形态” | 分层栈、管道、事件驱动、微服务 |
| **核心抽象** central abstractions | 领域里“永恒”的实体与关系 | ERP 的订单-物料-库存；银行的钱-账户-流水 |
| **根本机制** governing mechanism | 系统赖以工作的基本机理 | 消息传递、事件溯源、状态机、共享内存 |
| **根隐喻** root metaphor | 用“什么眼光”看这个系统 | 城市（分区/市政）、有机体（免疫）、机器（管线） |

它们共同回答一个终极问题：**“这个系统在概念层面，到底是什么、怎么想?”**

基本概念从哪来？有五条路径，架构师大部分时候走第一条，其余四条是备用的”生成器”：

| 路径 | 方法 | 说明 |
| --- | --- | --- |
| ① **模式选择**（最常见） | 从架构模式库/参考架构里”选” | 分层、管道-过滤器、事件驱动、微内核、微服务……先识别问题类型再去货架选。**选型本身就是设计**——选哪个概念、为什么选，就是第一条 ADR |
| ② **领域抽象** | 从需求中”提” | 问”剥掉实现细节，这个领域里什么永远成立？“——答案即核心抽象（DDD 的 bounded context、aggregate） |
| ③ **质量属性反推** | 约束”逼” | 例同§11.3.2（99.99%→冗余、10 倍→分片）；“不可篡改”→只追加日志。**质量属性驱动架构的”驱动”机制就在这里** |
| ④ **类比迁移** | 从成熟领域”借” | 操作系统给了微服务”进程/调度/隔离”；生物给了安全”免疫/纵深防御”——借来的概念已 |
| ⑤ **第一原理**（兜底） | 从系统目的”推” | 货架没有、领域太新、被原领域检验几十年类比失效时，回到系统存在的**目的**：它最根本 |

> 一个提醒：这里的”选型”选的是架构模式 / 概念（分层、事件驱动、微内核……），属于架构设计本身；§11.3.1 里那种传感器、芯片、协议的具体技术选型仍是”候选”——待检验、待权衡的选项，两者不是一回事。

注意①和③的组合：水忠面对”边缘断链预判”，概念库里有”边缘计算”“云端计算”两个现成范式（路径①）；而”误报率”“提前量”这些质量属性（路径③）会反过来逼他重新想：预判是”新机制”，还是既有追溯机制的新输入？——回到李旭那个问题：“M3 还是 M3 吗？”

基本概念这块，最深的机制是**幸存者逻辑**。

基本概念 = 对”不变性”的承诺——它之所以”基本”，因为去掉它这个实体就不再达成它的目的（呼应第 10 章”不可还原”）。“最难改”是它的常见后果，不是它入选的理由——那是代理判据，§11.3.6 会当场拆掉它。

而它的诞生，是一个**假设-检验循环**，不是一次”设计”就完成：

1. **提出**——按五条路径生成**候选**概念（这是”设计”）；标准把这一步叫**综合**——42020 §8.4.5（在解空间中综合潜在方案）、§8.4.7（形成候选架构）；
2. **检验**——用原型（spike）、权衡分析（ATAM）、架构评审去检验候选；
3. **淘汰/幸存**——大多数候选会死掉，活下来的才升级为”基本概念”，被**命名**、被写进架构描述、成为后续所有决策的锚。

> **所以严格说：基本概念不是”设计”出来的，是”幸存”下来的——设计只负责摆上桌，检验负责决定谁配得上”基本”二字。**

检验的最高判据，是**目的判据**：fundamental 由系统对其环境中的**目的**单独裁决（判据定义同 §11.2.4）。它操作化为**不变量判据**：在演化与重实现中必须守住、否则系统就沦为另一个系统的概念与属性。判别问句只有一句：「**这个东西变了，系统还能不能达成它的目的——它还是不是为那个目的的这个系统？**」——章首李旭问的”M3 还是 M3 吗”，正是它的日常版。

它的形式是**必要性（membership）＋充分性（closure）**双判据（形式化与出处的对账见 §11.2.4），且**不是数学意义的充要条件**——充分性仅指”目的所要求的结构条件已被完整表达”。

裁一个候选（概念或属性）配不配叫”基本”，走三**道测试**：① **反事实承重测试**——删掉它，目的会怎样？**跨**（目的落空）→概念级；**变形**（目的不变、实体性质变了）→属性级基本；**只是变难**→不进架构层。② **反事实替换测试**——换一个实现，这个概念／属性还在不在？在 → 基本；不在 → **那只是当前实现的一部分**（“实现混进本质”由此挡掉）。③ **目的判据两遍跑**——兑现侧（实现目的需要它吗）＋演化侧（目的变了它守不守得住）；只跑一遍，演进类的基本条目必漏。

目的判据是**成员资格判据**——裁决”谁是 fundamental”；它之下还有**从属**判据：改动代价（Ralph Johnson）。代价判据只在根本

集合**之内**决定哪些提前锁死、锁到什么程度、哪些保留自由度——**不产生新架构成员**，只做优先级与时机的排序。

**顺序不可颠倒：先过目的关**（是不是根本的），**再过代价关**（值不值得现在定死）。两种错位都是病——用代价判据越过目的判据，把难改但无关紧要的东西误当架构；跳过代价校验，把根本的东西不分轻重全锁死，演化窒息（完整展开见 § 12.3.1）。

两个**假判据**要避开：**不可逆性**（可观测、直观、易沟通，却两类失真——**漏判**不贵但承重者，如”日志须可被外部审计员独立读取”；**误纳**很贵却与目的无关者，如框架版本、组织惯例技术栈），以及**可判定性／可引用性／高层级**（都是”看起来像判据”的代理，同一形状的错误换了外衣）。代理不是没用——正当用法是排序、配治理强度、做回代检查；**判据只有一个，就是目的**。

这个问句的后半——“还是不是为那个目的的这个系统”——问的是**同一性**：人造系统的同一性是目的论的（同一性目的论·忒修斯之船，见附录 A T-144）。李旭问”M3 还是 M3 吗”，问的其实是”M3 的目的还在吗”。

这里有一个标志性现象：**概念设计完成的标志，是它有了名字**。“事件溯源”“六边形架构”“双写分离”——一旦结构被命名，它就从想法变成了**可共享、可讨论、可传承**的概念。**命名就是概念化的完成仪式**。

水忠那天的”边缘断链预判”，没名字之前只是一句需求；被命名为”边缘预警输入”、写进架构描述后，它才成为可讨论、可传承、可

作后续决策锚的概念——就像”数据所有权归追溯域”，在第 10 章后成为人人遵守的规矩。

### 概念卡片 · 基本概念

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 系统在其环境中”怎么想”的根源——组织范式 / 核心抽象 / 根本机制 / 根隐喻（42010 定义里的 concepts——本质·怎么想） |
| **它不是什么** | ⑥⑦ | 对立=详细实现——概念是”去掉就不是它”的层，改它等于换一个系统；实现是”改了还是它”的层；辨析=≠技术选型——选型是候选，概念是幸存下来的承诺；命名是完成仪式 |
| **它从哪来** | ②③⑨ | 90% 从概念库选、10% 被需求/属性逼出——候选要过检验才配叫”基本”（§11.3.6）；四副面孔承自软件架构与领域建模数十年的范式沉淀，“命名是 |
| **它到哪去** | ④⑤⑧ | 由基本概念组织系统去响应核心提问：这哪类生 |

### 11.3.7 基本属性不是设计出来的：属性是涌现出来的

再看本质里的属性这一面。它和基本概念的区别，是设计里最要紧的一处：

- **概念** =“系统是什么”（范式、抽象、机制）——可以**切**、可以**选**；
- **属性** =“系统具备什么本质特征”——**不是”设计”出来的，是”涌现”出来的**。

> **你设计的是实体的构成，属性是它运转之后的宏观效果**。所以”设计属性”的正确姿势是**逆向工程**：先定目标属性（来自需求与关注点），再设计结构让属性涌现，最后验证它真的涌出来了。

一个类比把这件事钉死——材料科学。钢铁的强度、韧性、导热性是宏观属性，由微观晶格结构决定：**不能直接设计”强度”，只能设计微观结构让强度涌现**。

架构同理：**不能直接设计”可用性”，只能设计冗余机制让可用性涌现**。属性的”设计”，本质是**构成的设计 + 涌现的验证**。

水忠想要”提前告警断链”（属性：预判及时性），不能直接画一个”及时性”模块；只能设计”双路采样 + 边缘计算 + 置信度阈值”这套构成，再验证”提前量”真的涌出来了。

基本属性的设计，有一条四步流水线：

| 步骤 | 内容 | 说明 |
| --- | --- | --- |
| **① 场景化（QAS）** | 属性从”形容词”变”可测指标” | 刺激→环境→响应→度量（S/E/R/M）。例：“进程崩溃时（刺激）、正常负载下（环境）、30 秒内切换备用（响应）、P99 恢复 ⩽ 30 秒（度量）”。**场景化后的属性才是能进入设计、能用于验收的标尺** |
| **② 战术选择（tactics）** | 从战术库挑战术并组合 | 战术是属性的实现单元：可用性→冗余/心跳/故障转移；性能→缓存/并发/异步；安全→认证/授权/加密 |
| **③ 构成落位（placement）** | 机制放对位置 | 属性是涌现的，**不能”切”给部件，只能”放”进构成**，放的位置不同效果天差地别；战术要落到概念划定的构成上——概念与属性在这里握手 |
| **④ 验证度量** | 属性必须可测 | 负载测试验性能、故障注入验可用性、渗透测试验安全。**没度量过的属性承诺等于没有** |

**贯穿四步的是权衡**：属性之间天然冲突——性能 vs 安全、可用性 vs 成本、一致性 vs 延迟。

**“基本属性”的设计实质就是排优先级**（ATAM 的活：把属性冲突摆到桌面上，逼出取舍）。

水忠的”预判及时性”走一遍四步：场景化——“冷链温度持续偏离（刺激）、正常运输（环境）、提前 5 分钟告警附置信度（响应）、误报率 ⩽ 5%（度量）”；战术——边缘计算 + 双路冗余；落位——预判逻辑放固件层、置信度模型放云端（可更新）；验证——历史运输数据回放测误报率。

**四步走完，属性才从一句”要能提前告警”变成了一个可验收的承诺**。

不是所有属性都配叫”基本”。**基本属性 = 生命线属性：系统可以平庸但不能没有的那些**。涌现是来源，生命线是筛选；判断方法叫关键性分析（criticality analysis）：

· 飞机的安全、银行的账务准确——生命线，可以平庸但不能没有；

· 界面的美观——不是，可以平庸，不能没有的是别的东西。

**识别生命线属性，决定权衡时的优先级与资源投向**。

> 术语注：“基本属性 = 生命线属性”是本书对 fundamental properties 的**操作化解释**——标准只定义”基本”不替你筛选；“生命线属性”与”关键性分析”（criticality analysis，源出安全工程）是本书给的筛选法，欢迎校准。

水忠的 M3，生命线属性是”断链即报废时还能撑到接入冷库、全程可追溯”（第 10 章那条不可还原涵义，对应的属性是可用性 + 可追溯性）。

“预判及时性”能不能成为”基本”属性，要看它是不是生命线——如果预判错了导致误动作、反而破坏追溯，那它就不是生命线，甚至要压住。

### 概念卡片 · 基本属性

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 系统具备的本质特征——可观察、可度量的表现；涌现属性（emergent property），由实体的构成决定 |
| **它不是什么** | ⑥⑦ | 对立=基本概念——概念定”系统是什么”（可切可选）；属性定”具备什么”（涌现，只能让实体的构成催生）；辨析=≠质量属性——质量属性是需求侧目标（非功能需求），属性是构成运转后涌现的结果；QAS 场景化是两者的桥 |
| **它从哪来** | ②③⑨ | 系统级质量属性没法像功能需求那样逐条分配，只能靠设计实体的构成催生——干是单列一块，用四步流水线 |
| **它到哪去** | ④⑤⑧ | 回答”属性到底是什么”——属性（AS）怎么变成 |

## 案例进展③｜冰链：水忠第一次把”概念 + 属性”摆在一起

水忠开始认真做 M3 二代的架构设计。他按”本质＋管控原则＋落点”一样一样来。

**本质 · 基本概念**。他问：“这个系统，在概念层面到底是什么?”

原来的答案——“断链即报废还能撑到接入冷库、全程可追溯”——是一个”机制型”概念（根本机制 = 断链检测 + 追溯）。现在加了预判，他面临两个候选：

- 候选 A：**边缘预判作为新的根本机制**——M3 从”断了才告警”变成”能预判断链”，本质变了；
- 候选 B：**边缘预判作为追溯机制的一个新输入**——M3 的本质还是”可追溯”，预判只是让追溯更早生效。

李旭问的”还是不是 M3”，翻译成概念设计的语言，就是选 A 还是选 B。

水忠用路径③（质量属性反推）检验：客户真正在乎的是”疫苗不能断链”（可追溯），不是预判本身——用目的判据说：预判只是让追溯更早生效的手段，对目的不构成新本质。

于是候选 B 幸存——**“边缘预判”没有成为新的基本概念，它成为追溯机制的一个新输入，被命名为”边缘预警输入”**。

**本质 · 基本属性**。水忠把”要能提前告警”场景化：提前量 ⩾ 5 分钟、误报率 ⩽ 5%、预判失效时回退到”断链即报”原逻辑（降级安全）。

再选战术：边缘计算（及时）+ 双路冗余（防误报）+ 置信度门槛（宁可漏报不可误动）；落位：预判引擎放固件层、置信度模型放云端（可更新）；验证：历史运输数据回放。

“双路冗余，就是多一路采样、多一颗传感器——物料、批次、组装工序全要跟着走。”逸人翻了翻选型页，“这一路加不加，采购、批次、量产都得重新排。”他接着算账，“新增料号，BOM 多一行、备件库多一行、工序多一步，起订量、交期都得重谈——不是多一颗料的钱，是整条供应链重排的账。”

“那就把它摆上桌。”水忠说，“属性设计本来就是排优先级：‘宁可漏报不可误动’要双路，成本约束要省冗余——两头都得顾。你这个制造约束，是这步权衡里必须算进去的一张牌。”

“概念 + 属性，这两样想清楚，图怎么画都是对的；这两样不想清楚，图画得再漂亮也是空壳。”水忠对董勐说。

### 11.3.8 管控原则不是抄来的：原则是自加的

管控原则立法之前，必须先分清三个天天被混用的词：

| | 来源 | 性质 | 例子 |
|---|---|---|---|
| **约　　束** constraints | 外部强加 | 不可谈判，**接受** | 法规、指定技术栈、成本上限 |
| **决策** decisions | 架构师选择 | 具体结构选择 | “缓存用 Redis” |
| **原　　则** principles | 架构师**自加** | 指导决策的规则 | “状态必须外置”“禁止循环依赖” |

关键区别：**约束是被迫的，原则是自愿的自我限制**。

架构师主动给自己上的纪律——因为踩过坑、或预见会踩坑。§10.2.4 说过”约束即解放”；这里要补上”原则”这个自愿的维度：

**原则是”决策的决策”（meta-decision）——它不是替你做决策，是让你不用重新想。**

注意”自愿”说的是**来源**——纪律是架构师主动给自己上的；它一旦立法，就带着**管控原则（governing principles）的约束力**约束整个团队：可判定、可入评审、进基线（第 13 章），不是可选的建议。自愿对立法权而言，约束对守约者而言——两回事。

> 与第 10 章对账：第 10 章”约束即解放”里的”约束”，泛指架构给团队的全部规则——主要指这里的”原则”（自愿自加的规矩）；这一节把外部强加的单独叫”约束”。两处不冲突。

注意第 10 章定义里的管控原则——现行版原文是 **realization and evolution**（实现与演化）：它要同时管住”怎么把它造对”与”以后怎么变不破坏”（演化这一面 §11.3.9 展开）。

管控原则从哪来？三个生成源：

| 生成源 | 机制 | 示例 |
| --- | --- | --- |
| **概念 → 执行细则** | 概念回答”怎么想”，原则落地成可判定的规矩 | 概念”无状态” → 原则”任何服务不得在本地保存会话状态（必须外置）” |
| **属性 → 必须句式** | 属性回答”要什么”，原则翻译成”因此必须怎么做” | 属性”可用性 99.99%” → 原则”任何单点故障不得导致整体不可用（禁止单点）” |
| **教训 → 禁止句式** | 失败的反向提炼；**原则的典型句式是”禁止”——大部分原则是禁止重蹈覆辙** | 循环依赖拖垮过项目 → 原则”模块间只允许单向依赖（禁止循环依赖）” |

**标准句式：**`必须/禁止/应当 + 规则 + 理由（rationale）`。理由不可省——没有理由的原则是教条，有理由的原则是纪律。

四条有效性判据：

1. **可判定（decidable）**——能判定”它违反了吗”。“使用标准接口”可以判定，“追求优雅”不可判定；
2. **有理由（rationale）**——每条都能答出”为什么”。答不出，删掉；
3. **少而精（few）**——一般 5~15 条。原则太多等于没有原则；
4. **可入评审（enforceable）**——能变成架构评审的检查项。**原则写出来不是给人看的，是给评审用的。**

水忠第 10 章写的那些”老规矩”——“温度数据归属追溯域”“断链检测是刚性边界”——用这把尺子重新审视：有理由吗？能判定违反吗？少而精吗？能入评审吗？

有些能，有些不能。**“老规矩”要升格为管控原则，得先过这四关**。

管控原则怎么立？两条路径对撞：

- **回顾式提炼（bottom-up）**：先做了一批决策，然后问”这些好决策背后共同的规矩是什么？“——原则是好决策的**公约数**；
- **前瞻式预设（top-down）**：从概念和属性出发，**先立规矩再让决策遵守**。

实践里两者必须对撞：先预设一批（来自概念/属性），再在决策实践中提炼修订（来自教训），最终沉淀出少量真正站得住的。

**原则是宪法 + 判例的互驯**：宪法（原则）指导判例（决策），判例反过来修宪。**僵死的原则和没有原则一样有害**（“禁止用 NoSQL”在数据模型根本变化后就该重写）。

还有一个必须守住的分寸：**别用原则代替思考**。

原则是自加的约束，所以它永远面对一个诱惑——拿原则挡掉所有权衡。正确用法：**每个决策先过原则（快检），没覆盖的地方才进入完整权衡（慢想）**。

原则负责”不需要重新想的事”，权衡负责”必须重新想的事”。如果一条原则让团队不再权衡，它就已经从纪律退化成教条了。

### 11.3.9 管控原则还要管住演化：怎么变而不破坏对

管控原则还要管演化这一面——它和第 10 章管控原则里的”演化”半句直接对应（§10.2.2 的“实现与演化”带两个宾语），本质一句话：

> **管控原则对演化的约束，是它对实现约束的”时间维”延伸。**
>
> 管控原则既管”怎么把它造对”（实现），也管”**怎么变而不破坏对**”（演化及其生存周期）——同一条原则，一个管当下，一个管未来。

两条边界必须先立：**第一，准则不产出”基本”条目**——概念与属性**产出可登记的”基本”条目**，准则约束的是”别人怎么做”；把准则拿去追关切，会得出”准则无据”的错误结论。**第二，约束型关切（法规、环境、人因类要求）不产出”基本”条目**，只作边界条件**附在既有条目上**。

为什么必须有它？因为**架构在无人管束时必然退化**（架构的熵增）：

- **架构漂移**（architectural drift）：实现一点点偏离设计意图（“这次赶时间”累积的）；
- **架构侵蚀**（architectural erosion）：结构性问题累积，依赖纠缠、边界模糊，直到”没人敢改”；
- **技术债**（technical debt）：捷径欠的债不断滚利息；
- **目的漂移**（purpose drift）：目的悄悄变化而架构未跟随——方向与架构漂移相反（架构漂移：实现偏离架构，架构对、执行歪；目的漂移：目的偏离架构，执行对、判据过期），更危险、无症状，发现时不变量集合已整体过期。目的变更 = 架构重审的强制触发器

（第 14 章展开）；防架构漂移靠合规评审，防目的漂移靠以目的为参照的定期检视。

管控原则对演化的约束，就是给”变”上的制度保险——**系统不是不变，而是变的方式受约束**。

管控原则在演化方向要管什么？八类，全部是”变”的句式：

| 准则类别 | 示例句式 | 防的是什么 |
| --- | --- | --- |
| **兼容性** | 接口变更必须向后兼容或走版本化 | 变，但不能打断已存在的东西 |
| **可替换性** | 部件必须可独立替换 | 变，但不要牵一发动全身 |
| **渐进性** | 变更必须小步、可独立发布 | 变，但不要大爆炸式豪赌 |
| **可逆性** | 关键变更必须留回退路径 | 变，错了要能回来 |
| **受控性** | 一切变更必须走配置管理 | 变，但不能失控 |
| **审查关口** | 超规模的变更必须走架构评审 | 变，但不能悄悄侵蚀 |
| **留痕** | 变更后 ADR 与架构描述必须同步 | 变，但不能让架构失忆 |
| **技术债** | 走捷径必须记债、定还期 | 变，但不能债滚债 |

**判别一条管控原则更贴实现还是更贴演化**：这条规矩是关于”怎么建”还是”怎么改”的？**主语是”变”的，贴演化那一面**。

管控原则在演化方向的约束怎么生成？四条源：

1. **时间维化**——拿每条管控原则问一句”这条规矩在变更时怎么执行”（“禁止循环依赖”→“新增依赖必须先检查依赖方向”）；
2. **面向未来的质量属性**——可维护性、可扩展性、可移植性；
3. **历史教训**——不兼容升级搞挂过生产 →“接口变更必须双版本并行期”；
4. **配置管理纪律的翻译**——见下。

管控原则对演化的约束和配置管理（第 13 章）是同一套制度的两层皮：**配置管理管”物”（configuration item 受控），管控原则管”形”（架构在受控下变形）**。

基线是”演进的地板”（变更从基线出发、到新基线为止），版本是”演进的足迹”（每次演进必有版本足迹）。

**好的管控原则会给自己留一条”宪法修正案”通道**——架构重写（技术断代、业务剧变时）也是演进的一种，走更重的流程，否则原则会逼出地下变更。

### 11.3.10 元素关系不是越多越好：关系是成本

再看落点：元素与关系。这一层属**实体侧**（其规格归下游的设计定义过程），不是架构的第三样内容——它是”架构可以关乎”的对象里最要紧的一处。它有一个反直觉的第一原则：

> **关系是成本**。n 个要素如果全互联，是 $n^2$ 条关系——**每一条关系，都是未来变更时的一处牵动**。所以关系设计的第一原则是”**克制**”：能没有的关系就不要有。

好的架构，**落点**上往往关系稀疏——关系少，独立性才强，可演进性才高（呼应管控原则在演化方向约束里的”可替换性”）。

设计关系，第一步是”定性”——给每条关系分类：

| 类型 | 例子 | 典型强度 |
| --- | --- | --- |
| **结构关系** | 包含（part-of）、聚合、泛化（is-a）、依赖 | 强（编译期绑死） |
| **行为关系** | 调用、消息传递、事件、数据流、控制流 | 中 |
| **接口关系** | 提供/消费（provide/require）、契约 | 契约本身弱，但承载强交互 |
| **部署关系** | 逻辑要素 → 物理资源（allocation） | 视场景而定 |
| **时序关系** | 顺序、并发、同步/异步 | 中 |

**关系的”强度”是设计的自由变量**——同一对要素之间，可选强关系（同步调用、共享状态）或弱关系（事件、消息、契约）。

关系设计的艺术，就是**在满足需求的前提下，把每条关系往弱的方向推**：

| 光谱位置 | 关系类型 | 代价/收益 |
| --- | --- | --- |
| **强** | 同步调用 · 共享状态 · 编译期依赖 | 变更牵动整片 |
| **中** | 消息 · 事件 · 异步 | 解耦但保持连接 |
| **弱** | 契约 · 运行时发现 · 事件总线 | 可替换性最高 |

设计关系，走六步动作：

1. **识别必要关系**——从行为/数据/控制流需求里提取必要连接，每记一条都问：“真的必须吗?”

2. **分类定性**——标出每条关系的类型（结构/行为/接口/部署/时序）。

3. **定方向——依赖方向是头号决策**——依赖图必须无环（DAG），允许 A→B 就不允许 B→A；循环依赖是架构腐蚀的头号元凶，要配**环检测**。
4. **降耦合（核心动作）**——强关系降级为弱关系：同步调用 → 异步消息、直接调用 → 事件、共享状态 → 消息传递——每降一级，牵动范围小一圈。
5. **契约化**——把剩下的关系写成**显式契约**：接口规格、协议、时序约束、错误语义——否则就是隐式耦合（实现之间悄悄握手）。
6. **检查**——环检测、扇入扇出检查、层次合规检查、架构评审。

五条判据：

- **有方向**：依赖无环（DAG），方向即权力——被依赖方负责稳定，依赖方负责变化；
- **够稀疏**：关系少而必要（邓巴数式直觉：一个部件能稳定维护的关系数量有限）；
- **尽量弱**：能异步不同步、能事件不调用、能契约不共享；
- **要显式**：依赖抽象不依赖实现（DIP）——建立在”接口”上，不建立在”具体类”上；
- **不穿越**：迪米特法则（Law of Demeter）——“**talk to your friends, not strangers**”：只与直接关联者说话，不要隔着一层关系去够另一个要素。

**关系不是独立存在的——它是概念与属性在设计侧的落点**：概念”管道” → 关系”数据流沿管道流动”；属性”可用性 99.99%” → 关系”主备切换 + 心跳”。

**概念定”有哪些成分”，属性定”成分要有什么性质”，关系定”成分怎么连起来才具备这些性质”**。

## 案例进展④｜恒信镜照：万辉的准则与关系

恒信的万辉在合同审批系统上学到”图要对得上场景”之后，重做合同全生命周期平台的架构设计——先立本质与管控原则，再看落地。

**管控原则**。他从徐漾那学到”原则是自加的约束”，先列了五条候选：①合同数据所有权归合同域；②审批链每步必须留痕；③对外接口变更必须向后兼容或走版本化；④新增功能不得要求重写既有模块；⑤走捷径必须记债、定还期。

徐漾检查，先教判别法：“主语是’变’的规矩，贴管控原则里约束演化的那一面。”③的主语”接口变更”、⑤的主语”走捷径”，都在”变”上——它们是管控原则约束演化的部分；剩下的①②④，有理由、能判定、可入评审——过四关。

**管控原则·约束演化**。③（兼容性）、⑤（技术债）正是 §11.3.9 八类表里的两行；他想到合同历史上那类编号事故的教训（第 12 章会专门展开），把教训时间维化：“接口变更必须双版本并行期”“已归档合同的历史数据口径不得重排”。

**落点·元素与关系**。他画了合同主表、审批记录、附件三个实体的关系图，然后做了一件事——把”审批引擎直接调合同主表”改成”审批引擎通过合同服务（契约）访问合同数据”。

关系从”强”降到了”契约弱”。徐漾看了说：“你以前是画’谁调用谁’，现在你在设计’依赖朝哪个方向、弱到什么程度’——这就是关系设计。”

---

## 11.4 第三幕 架构设计往哪去（运用）

> 这一幕把”两样内容＋它们的落点”用起来：本质与管控原则不是并列吃需求的——基本概念是唯一直接面对原始世界的层，属性被概念限幅，管控原则从概念/属性提炼，元素与关系是它们在实体侧的落点；一条内部输入链把它们串起来，三个推论给出体检指标；最后看水忠第一次”设计”出了 M3 二代——三样东西，换成了能定形的东西。

### 11.4.1 输入是一条链：基本概念直面世界

要产出的”本质＋管控原则＋落点”，吃的不是几份并列的需求，而是分两个来源：

- **外部原始输入**：目的与任务、需求（功能/非功能）、约束、关注点、环境、资产——从外部进入架构活动；
- **内部派生输入**：前面产出的东西再喂给后面的——形成一条**内部输入链**。

**环境是第三类硬条件**（法规／物理条件／外部接口／资源，与目的共同裁决”目的能否达成”）；在判据栈里，目的之下还坐着**关切闸门**——

它不管够不够格进架构，只管这次写哪些、写多细、给谁看。架构师把力气放在收全关切、写好视角规格、打通映射表上。

> **只有”基本概念”直接面对外部原始输入；越往后，输入里”内部成分”占比越高。**

外部原始输入（目的与任务 · 需求（功能/非功能）· 约束 · 关注点 · 环境 · 资产）

│（唯一直接消费方）

▼

基本概念（本质·怎么想）—限幅→ 基本属性（本质·具备什么）

│ 落点 │ 落点 · 提炼

└→ 元素与关系（落点·实体侧） ←—管控原则（约束实现与演化）

（从概念/属性提炼＋外部教训·信条·实践）

| 产出 | 外部原始输入 | 内部派生输入 | 一句话说明 |
| --- | --- | --- | --- |
| **基本概念**（本质·怎么想） | 目的与任务、功能需求、质量属性、约束/法规、关注点、环境、资产 | —（最底层） | **唯一直接面对原始世界的层**，后面都吃它的输出 |
| **基本属性**（本质·具备什么） | 非功能需求、法规合规、业务目标、关注点 | **概念**（限幅） | 属性受概念约束：概念定上限，属性定验收线 |
| **管控原则**（约束实现与演化） | 历史教训、组织信条、行业最佳实践、生存周期预期、配置管理规范、运维现实 | **概念 + 属性**（提炼） | 原则是概念/属性的”必须·禁止·应当”句式化，还要管住演化（§11.3.9） |
| **元素与关系**（落点·实体侧） | 行为/数据流需求、部署环境、接口/协议标准 | **概念 + 属性**（落点） | 元素与关系是两样内容在实体构成上的落点、属性的实现通道；其规格归下游的设计定义过程 |

> 两组职能的落点（与 §11.3.2 同口径）：判据侧供”判据”、选项空间圈”舞台”；目的与任务居判据侧首位，以《系统目的或任务理解确认单》锚定进入（确认单见 §11.4.3）。

> 术语注：**元素／关系**在现行标准里**无术语地位**（仅 42020 §3.3 Note 1 注释级提及，其原文作 components 与 relationships；2011 版定义句里的 embodied in its elements, relationships 已删）；**管控原则属架构本体**，架构描述只是它的表达载体，不是它的依据。

### 11.4.2 三个推论：输入链带来的体检指标

这条输入链，带出三个推论，每一个都是架构设计里的”体检指标”。

**推论一：只有概念是”翻译”，其余是”翻译的翻译”。**

如 §11.4.1 那条链所示：属性把概念当”已知前提”，管控原则与落点吃的是提炼物。**概念设计错了，后面跟着错，而且错得越来越远**——概念阶段值得最重的人力。

水忠为什么先花两节在”基本概念”上？因为如果”边缘预判是不是新的基本概念”没想清，属性、管控原则、元素与关系全都会在错误的地基上盖楼。

**推论二：内部链是一条”失信放大器”。** 每一层的输出质量 = 上一层输入质量 × 本层加工质量。

**输入链断裂的症状**：管控原则不是从概念/属性提炼的，而是抄来的——原则和架构互相矛盾（概念说”无状态”，原则却是抄来的”共享缓存”）。

评审时发现原则跟概念对不上，基本可以断定链断过。

**推论三：反向校验是免费的架构体检**。因为输入有内部依赖，可以**从后往前核查**：逐条问”这条关系服务于哪个属性？这个属性被哪个概念限幅？这个原则从哪个概念/教训提炼?”

答不出来的，就是”无源之水”。**ADR 的价值在此再次显现：它是输入链的书面凭证**。（第 12 章会讲决策记录六问怎么写。）

## 案例进展⑤｜冰链：收口，水忠第一次”设计”出了 M3 二代

水忠把设计做完，在纸上收口。这一次他带进会议室的不是图、选型和老规矩，而是能”定形”的东西。章首那页老规矩先过四关筛选：升格为管控原则（约束实现：数据所有权、断链刚性；约束演化：固件升级向后兼容），另补”可独立验证”——预判机制要能单独验收，才守得住可追溯这条底线。

1. **本质 · 基本概念**：M3 二代仍是”断链即报废还能撑到接入冷库、全程可追溯”——“边缘预警”是追溯机制的新输入，不是新本质（候选 B 幸存，已命名）；
2. **本质 · 基本属性**：预判及时性（提前 ⩾ 5 分钟、误报率 ⩽ 5%、失效降级安全）+ 既有生命线（可用性、可追溯性）——全部场景化；
3. **管控原则（约束实现）**：老规矩过四关升格——数据所有权、断链刚性、可独立验证，每条带理由、可判定、少而精、可入评审；

4. **管控原则（约束演化）**：新增传感器不得破坏追溯链、固件升级必须向后兼容、预判逻辑变更必须先过追溯完整性评审——把前一条原则放到演化上做”时间维化”（§11.3.9）；
5. **落点 · 元素与关系**（实体侧，规格归下游设计定义过程）：传感器 → 固件（预判引擎）→ 云端（置信度模型）→ 追溯平台，依赖方向单向、接口契约化——“边缘预警输入”通过契约接入，不绕过追溯域。

逸人把那页元素关系对着产线看了半天：“传感器 → 固件 → 云端 → 追溯平台，依赖单向不绕——落到产线，物料跟着传感器走、批次跟着固件走、追溯跟着平台走。方向要是绕了，我的批次台账第一个乱。”

文正看完，问了一个和第 10 章那次一样的问题：“客户想看新一代 M3 的架构，给他看什么？”

水忠这次没掏图。他想了想，说：“看这几样。图是描述，描述会过时；**这些是设计，设计一旦定形，就是后面所有决策的锚**。”

李旭补了一句，给这一章做了个注脚：“上次我们问’M3 是什么’，这次我们问’我们怎么把它设计出来’——**两件事，一件比一件难，一件比一件更像在做事**。”

---

### 11.4.3 架构生成六步：动词链的工程化

构思 → 定义 → 设计这条动词链不是概念推演，是可以跑起来的流程——六步摆出来，架构生成就有骨架。六步之上另有一层权威：**架构集合目标（42020 §6.4.3）、架构治理决策（§6.4.4）与架构目标**

**及关键成功准则（§8.4.4）的确立属治理决断；「关切准入」的四关裁决是本书对它的形式化**（四关见 §15.4.5）。

但开跑之前先有一张**入场券**：《系统目的或任务理解确认单》——目的或任务原文引用（业务方原话）+ 架构师复述理解 + 业务方确认签字（属理解确认，非第 8 章的确认 validation）。

它是判据侧首类输入「目的与任务」（§11.3.2）的**确认形态**：判据必须已确认、可判定，签字后的目的才是设计真正消费的输入。

它同时是「理解」职责的产品数据：**纯理解、零架构**——复述里不出现元素、方案、需求等架构语言。一张单子，既是输入（判据锚点），也是产品数据（签字生效）——**签字之前，后续一切产物都没有判据锚点**。目的模糊或关注点冲突时，向上游回溯澄清，不带着模糊目的硬架。章首水忠没答”预判让 M3 长出新本质还是只多一个功能”就开工——这张入场券逼他先把目的说清。

入场券上的目的还要写清**“舍掉了什么”**（读不出”舍了什么”，等于没写——一个什么都想要的目的，等于没有目的），并做一次**同层锚定检查**：目的与目标锚的是**同一个所关注实体**吗？部门级、集团级的诉求只能作约束或环境条件进场，不能当架构目标的来源。

**入场券要锚定的「目的」，本身要先说清——是什么、怎么得到、怎么判断到位**。

目的是系统存在的理由——在其环境中、为利益相关方、解决什么问题或抓住什么机会、创造什么价值。两个界定：① 目的在环境侧不在系统侧——功能是「系统做什么」（向内看），目的是「系统为世界做什么」（向外看），同一功能可服务不同目的。

② 目的分层——**目的**（为什么存在）→ **目标**（要实现的结果，ISO 9000:2026 §3.7.11 objective）→ **需求**（必须满足什么可验证条款）；评审、确认沿链向上追溯到目的（第 18 章架构师守目的同链）。

目的怎么得到——三条来路，最后收口到本节那张入场券（“关切是原料，目标是产物，目的是准绳”，定式见 §11.3.2）：

| 来路 | 内容 | 出处/方法 |
| --- | --- | --- |
| ① 战略承接 | 企业战略 → 业务架构 → 产品战略 → 系统目的（实例化投影） | 保证目的有所承（正当性）；目的漂移时逐层查 |
| ② 业务或任务分析 | 定义问题空间（痛点/机会/约束）→ 刻画任务/业务需求 → 目的与任务陈述 | 15288 业务或任务分析流程；ConOps |
| ③ 关注点反推 | 关键利益相关方关切聚类提炼，**校验与补漏**目的 | 42010:2022 EoI 可从关注点分析中涌现；「**关切 → 目的**」**这条推导线不存在**——目的由治理机构在关切基础上**确立**（42020 §6.4.3／§8.4.4），标准点名的是「关切／要求／质量属性 →（identify and define）→ 架构目标」（§8.4.4 f)） |
| 收口 | 落到《系统目的或任务理解确认单》：原文引用 + 复述 + 业务方签字 | 本节入场券；签字前只是草稿，不能作判据 |

得到的目的是不是够用，三条到位判据——**可作判据**（能裁决任何概念/属性是否 fundamental;「成为行业领先」这类裁决不了的，打回精化）、**可分解**（能向下拆出目标与需求）、**可追溯**（能向上指回战略/业务任务）。

以目的检验架构要三分——**确认**（预期用途）与**评审**（所规定目标）以目的为依据，**验证**以规定需求为参照、不直接以目的为参照;「目的 → 规定需求 → 实现」的参照链各站各位（42030:2019 管评估，家族分工见 § 10.3.3；三把尺子见第 8 章）。

目的分层往下，目标还有运行层形态——**操作目标/MOE**：目的在运行环境的可度量投影，运行层确认的显性判分线；但 MOE 达标只是确认的必要条件，MOE 全绿 ≠ 确认通过（§ 8.2.7/ § 8.3.5）。

| 步 | 做什么 | 工具 / 纪律 | 对应动词链 |
| --- | --- | --- | --- |
| ① 定驱动因素 | 从关注点里筛出值得架构的问题（ASR） | 三标准：影响面广 / 难逆转 / 约束硬；6±3 个 | 构思的瞄准 |
| ② 生成备选 | 打开可能性 | **备选 ⩾2 个**，否则没有权衡可言；来源：模式库 / 参考架构 / 第一性原理 | 构思的主体 |
| ③ 权衡选定 | 终结漂浮 | 效用树 + ADR（含被否备选死因） | 定义的动作义 |
| ④ 分配与接口 | 把边界落到元素与契约上 | partition / align / allocate + ICD（第 6 章的一棵树、三个动作是它的落地） | 定义的深水区 |
| ⑤ 视角规格 + 表达与基线 | 把视角规格写齐（六项），视图逐张声明治理它的视角、记跨视图对应关系；再翻译成一致视图并冻结 | **六项规格见第 15 章 §15.4.1**；视图声明与对应关系硬要求见 §15.4.3；AD + ADR 进配置管理（第 13 章） | 表达 + 记录 |
| ⑥ 评审与背书 | 验证关口 + 组织合法化 | 评审四看；TR2/TR3 通过才放行（第 8 章展开架 | 认证恰当实现 |

三条纪律把”生成 ≠ 画图”钉死：**驱动先于图**（没有 ASR 就画图是装饰）、**决策先于图**（视图只是 ADR 的投影）、**基线先于图**（不受变更控制的 AD 只是草图堆，还算不上架构基线）。

⑤这一环有三条重数关系要记牢：**每张架构视图恰有一个治理它的架构视角（1:1）；一个架构视角可治理多张视图（1:N）；架构视角与关注点是多对多**。需要一张图同时交代两类关注点时，正确做法是**拆视图、拆视角**，不是给一张图挂两个视角。

这里有一个措辞陷阱：**不能说”架构构思 + 架构定义 + 架构设计”**。第一，“架构定义”是 15288 §6.4.4 的**流程名**，“生成候选架构”就在这个流程里（目的句首句），还含表达与文档化——并列等于把”定义”读窄；第二，设计定义（§6.4.5）的输入是**已生效的架构基线**，把设计画进生成过程，等于把回环的出口画进入口。

第三，生成过程的直接产出是 AD（架构描述），候选词里没有”表达”，架构就停在脑子里。正确说法：**生成 = 构思（①②）+ 定义动作义（③④）+ 表达（⑤），补上认证（⑥）才闭环；设计（狭义）在门外，基线一生效就接手**。

**“没有背书的架构，只是架构师的个人意见。”**六步里最容易被跳过的是②（直接从需求跳到唯一解画图，架构层的 Cowboy Coding）和⑥（只评审不背书，评审变汇报）。评审四看——概念陈述 / 选定 + ADR 备选死因 / 视图·基线受控 / 回炉痕迹闭环——缺一看，材料就是半成品。

六步的入场券与产出（确认单、**关切点台账、架构目标（含关键成功准则）、视角规格集（项目 ADF）**、基本概念和基本属性清单、架

构描述、决定清单、管控原则、系统接口契约、系统级验证方案），正是**系统架构师**的产品数据清单（§18.4.6 会给全）。关切点台账、架构目标与视角规格集三样的细则见第 15 章 §15.4.1、§15.4.3。

评审背书的拍板权在系统工程师，架构师给理由（第 18 章委托—协商—验收）。

一句话分工：**架构集合目标、架构治理决策与架构目标及关键成功准则的确立属治理决断**（42020 §6.4.3／§6.4.4／§8.4.4），不在系统工程师手上——治理决策在架构委员会（第 18 章）；**清单冻结**走的是治理闸门甲（§15.4.6）。

### 11.4.4 系统基本概念和基本属性清单：目的判据的裁决产物

六步走完，目的判据把”什么配得上基本”裁出来了（§11.3.6），裁决结果得落在纸上——《**系统基本概念和基本属性清单**》。

这是一份 fundamental **成员资格登记表**，每条注明三件事：① 概念/属性的**定义**；② 对目的的**必要性论证**（不变量判别问句的答案）；③ 到目的/需求的**追溯链**。另加一列**代理校验**——可违反性／可引用性／可判定性（三者只是代理，用来排序、配治理强度与做回代检查，不能当准入判据）。答得出必要性论证才配进清单，答不出的，就是输入链上”无源之水”（§11.4.2 反向校验）。

清单进基线之前还有一道**双向覆盖检查**：**正向查漏**（每条承重关切都追得到目标与元素吗）＋**反向查滥**（每个元素都追得到一条关切吗——没有的＝无据扩张）。**冻结主体是治理机构（闸门甲，第 15 章 §15.4.6）**，不是架构师自冻。

它是**目的判据的裁决产物**——“什么是根本”的登记汇总、评审的第一张核对单。评审先从”这条概念配得上基本吗”问起。**清单本身基线化，增删走变更控制**——命名取”清单”后缀，正因它是一份**受控枚举**，不是随意增删的笔记。

清单还有一层作用：**锁住本质，正是为了放执行自由**——只守这组不变量，详细设计与实现可以放手展开。这是”约束即解放”（§10.2.4）在清单上的落点。它也真拦得住变更：某次以”省成本”为由提出”取消追溯、只留结果数据”，按清单核对，追溯是基本概念、必要性论证答不过，变更在实现前就被挡下。

一句话分工：**确认单是锚（目的进场），清单是纲（裁决什么是根本的），架构描述、决定清单、接口契约是目（表达、留痕、落地）**。锚不定则纲不举、目不张。

### 11.4.5 创新：链尾的评分席

把构思、定义、设计讲完了，还有一个词总在旁边打转：创新。它和这条动词链是什么关系？

**创新不在动词链上**。ISO 56000:2020 定义创新是”创造或重新分配价值的新对象或变更对象”——注意它是**名词**，是动作的结果，不是动作本身。“我们去创新吧”和”我们去成功吧”是同一类语病：你能做的是构思、定义、设计、开发；**创新是对动作结果的追认**——链上的活干完了，够新又成了，才被追认为创新。

判据只有一条：**新 × 成，缺一归零**。只有新没成，是点子、是发明；只有成没新，是优化、是复现。发明是新想法的诞生（熊彼特），创新是新组合的**兑现**。

创新有四种进来方式——Henderson & Clark 的四分型告诉我们，主导动作随创新类型在三个动词之间滑动：

| 创新类型 | 换什么 | 主导动作 |
| --- | --- | --- |
| 突破式（radical） | 概念与连接全换 | 构思 + 定义 + 设计（全链） |
| **架构式（architectural）** | 元件不动、连接重排 | **定义**（重新画边界） |
| 模块式（modular） | 元件换新、连接不动 | 构思（元件概念）+ 设计 |
| 渐进式（incremental） | 元件微调、连接不动 | 设计（部件级） |

第二行是”创新主要靠构思”最锋利的反例：**架构式创新的主导动作恰恰是”定义”**——不用新造概念，只要把边界重新画、元素重新分组，就能颠覆一个行业。电动车滑板底盘就是例子：电池、电机都成熟，重新分块。

创新可以工程化吗?**可以，但要精确到对象**。流程（Stage-Gate）、方法（TRIZ）、组织（ISO 56001）、组合（漏斗下注）都能工程化；**不可工程化的是单点结果的确定性**——工程化管漏斗的形状与流量，不管哪一滴水通过。灵感出现时，好体系不让它被流程闷死，给它足够的入场券，死也死得有证据（ADR）。

冰链的 M3 二代，预判机制是新输入、接口不动——这是渐进式/模块式的活，主导动作落在构思与设计；真正”重新画边界”的那类变化，主导动作在定义——它不被叫”创新”，往往只是因为它没换元件；但元件不动、连接重排，恰恰是架构式创新的定义。

**口诀：链上的活干完了，够新又成了，才被追认为创新**。

## 认知反转

回到章首那间会议室。水忠带了三样东西——图、选型、老规矩——以为这就是架构设计。

现在我们知道了，那场会的问题，不是”没准备充分”，而是连错了六处，一层比一层深。

**第一处，他把”产物”当成了”设计”**。图、选型、老规矩都是产物/候选/规则，没有一样是”把需求翻译成架构”这个动作——设计是动词，定义是完成时，拿不出定形的东西就等于没设计。

**第二处，他跳过了输入**。第一步不是画，是问”谁在乎什么”——关注点是识别出来的，不是设计出来的；漏掉一个利益相关方，就是漏掉一整类 concern。水忠那张新图，没回答任何一个”谁在乎什么”。

**第三处，他以为设计是几样并排画进一张图的东西**。其实要产出的是两样内容（本质＋管控原则），另加一层落点去向（元素与关系，实体侧），而且不是并列——基本概念直面世界，属性被概念限幅，管控原则从概念/属性提炼，元素与关系是落点。**只有概念直面需求，后面都吃它的输出**——概念错了，后面全错，越来越远。

**第四处，也是最深的一处，他不相信概念会死、属性会涌**。真相是：**概念是幸存下来的**——“边缘预判”能不能成为基本概念，是它过不过检验决定的（质量属性反推：客户要的是可追溯），检验不过候选就死，哪怕画进了图里；属性是涌现的，只能设计实体的构成去催生。

**第五处，他不相信规矩是自加的、关系是成本**。管控原则不是抄来的——约束被迫、决策自选、原则自加；元素关系也不是越多越好——关系是成本，能没有的关系就不要有。

**第六处，他以为”做设计”是一件做完就完的事**。设计不是一次动作，是一条链：构思打开可能性、定义终结漂浮、设计（狭义）转换到底；本质与管控原则都定形后，还要评审背书——**没有背书的架构只是架构师的个人意见**。链尾坐着创新的评分席：链上的活干完了，够新又成了，才被追认为创新。

这一章的六处反转，其实是同一个反转的六种说法：

> **架构设计不是画图、选型、定规矩——它是把需求翻译成”架构”：定出基本概念或属性（本质）、立管控原则、落点在元素及其关系上（实体侧，规格归下游设计定义过程）；这当中，概念幸存、属性涌现、管控原则自加，没有一样是”画”出来的；落点那一层则守着另一条纪律——关系是成本，要克制**。

把章首和章末两张桌子并排看，水忠完成了从”见过”到”拥有”的跨越。**画图、选型、定规矩，只是见过；把本质与管控定形、表达成架构描述，才是拥有**。用第 1 章那把”见过 → 拥有”的四问尺子量一遍：本质（概念/属性）、管控原则、元素与关系，每一样都要答得全”是什么、不是什么、从哪来、往哪去”——答得全才算真正拥有。

回看第 10 章的结尾：架构不是一张图，也不是一堆贵的决定——它是实体在其环境中的基本概念或属性，加上管住它怎么实现、怎么演化的管控原则，“基本”的判据只有一条，目的。这一章补上这些内容是怎么被设计出来的——**先想清楚定什么、立什么**。

架构设计，就是在信息最不完整的时候，把这个实体最难撤销的那两样内容——本质与管控原则——想清楚、定形，并表达成架构描述。

而”哪一样最难撤销、难撤销到什么程度、该什么时候拍板”——那是第 12 章《架构决策》的事。

这一章回答”架构怎么被设计出来”，下一章回答”设计出来的东西里，哪些最不能反悔”。

用第 1 章 §1.4.4 的尺子量一下：这一章还是**第③层（心智模型）**——把需求翻译成架构，是一次能推演、能证伪的翻译，不是画图。

---

## 小结

这一章把”架构设计”从画图、选型、定规矩的日常误解里拎了出来。

**架构设计是把需求翻译成”架构”的活动**——它定出实体的基本概念或属性、立起约束实现与演化的管控原则；设计是动词，定义是完成时，没定形就等于没设计。

架构设计的**输入**是六类——目的与任务（判据侧首位）、需求（规定需求：功能+非功能）、约束、环境、关注点、资产；其中**非功能需求才是主要驱动**；质量属性以（基本）属性涌现——不能分，只能译。**关注点是识别出来的，不是设计出来的**。

架构设计的**工作**，是把定义里那两样内容做出来，再表达成架构描述、并交代它们在实体构成上的落点：**本质**——基本概念（怎么想——幸存，命名是完成仪式）与基本属性（具备什么——涌现，只能设计结构让涌现）；**管控原则**（约束实现与演化及其生存周期——自加，立了就有约束力）；**落点**——元素与关系（两样内容落在哪些成分

上、怎么连——它们属实体侧，其规格归下游设计定义过程；关系是成本）。

这几样不是并列的：**只有基本概念直接面对外部输入**——属性被概念限幅，管控原则从概念/属性提炼，元素与关系是落点——概念错了，后面全跟着错、越来越远。

反向校验（从关系追到属性、追到概念、追到管控原则）是免费的架构体检。

这条链再往里看一层：**构思打开可能性、定义终结漂浮、设计转换到底**；工程化就是架构生成六步——定驱动、生成备选、权衡选定、分配接口、视角规格＋表达基线、评审背书。**没有背书的架构，只是架构师的个人意见**。创新不在链上，坐在链尾的评分席：新 × 成，缺一归零。

六步的入场券与产出里，有两样是判据的载体：《系统目的或任务理解确认单》锚定目的（进场）、《系统基本概念和基本属性清单》登记”什么配得上基本”（裁决）——**确认单是锚、清单是纲**（冻结在治理机构，闸门甲），架构描述、决定清单、接口契约是目；评审先过清单这一关。

确认单要锚定的**目的**，本身要先说清——目的是系统存在的理由（环境侧/为利益相关方），分目的→目标→需求三层，经战略承接/业务任务分析/关注点反推三条来路得到，过到位判据三（可作判据/可分解/可追溯）才够用；以目的检验三分——确认与评审以目的为依据，验证以规定需求为参照。

第 10 章说架构是实体在其环境中的基本概念或属性加管控原则、“基本”的判据只有目的；这一章补上这些内容是怎么被设计出来的——先想清楚定什么、立什么。

下一章，我们把”难撤销”这把尺子造出来：哪些内容最难反悔、怎么判断、什么时候必须拍板。

物料表上新标了一串依赖关系：谁先、谁后、谁不能绕。以前他只看”料能不能按时到”；这一章之后他明白——产线排不排得动，先看料表背后那条谁也绕不开的链。

---

## 本章概念总图

**架构设计 = 一个动作（把需求翻译成"架构"）+ 两样内容（本质 + 管控原则）+ 一次表达（架构描述）+ 一条链（只有概念直面世界）**

**第一幕 · 是什么**

**翻译必然做选择：同一需求 → 不同架构**

**设计是动词，定义是完成时—过程以产出物命名（15288 只有"架构定义过程"）**

**要产出什么：只有两样—本质（基本概念/属性）+管控原则（governing principles）；再表达成架构描述**

**元素与关系不是第三样内容：架构"可以关乎"的对象，属实体侧，规格归下游设计定义过程（15288 §6.4.5）**

**与详细设计：横向铺（关切·生存周期） vs 纵向挖（实现）—先横后纵**

**第二幕 · 不是什么：三个"不是设计出来的" + 落点那一层的纪律**

三样东西不是架构设计：图=产物 / 选型=候选 / 老规矩=管控原则的候选（过四关升格）

输入不是全部需求原文: 六类（目的与任务 · 需求 · 约束 · 环境 · 关注点 · 资产; 判据侧/选项空间, 目的居首位; 环境 = 第三类硬条件, 关切闸门定这次写哪些） · 规定需求 · 吃未分配的系统需求

关注点不是设计出来的：是识别出来的（五步 · 四判据 + 完备 · 三测 · 四类分流不许悬空 · 举证三件套）

基本概念不是设计出来的：是幸存下来的（四副面孔 · 五条生成路径 · 候选→检验→命名 · 三道测试）

基本属性不是设计出来的：是涌现出来的（QAS → 战术 → 落位 → 验证; 生命线; 不能"分"只能"译"）

管控原则不是抄来的：是自加的（约束被迫 / 决策自选 / 原则自加；必须·禁止·应当+理由）

管控原则还要管住演化：怎么变而不破坏对（防漂移 / 侵蚀 / 技术债；准则不产出"基本"条目）

落点上关系不是越多越好：关系是成本（克制 · 往弱推 · 契约化）

第三幕 · 往哪去

输入是一条链: 只有基本概念直面世界, 属性被概念限幅, 管控原则从概念/属性提炼, 元素与关系是落点

推论：只有概念是"翻译", 其余是"翻译的翻译"；链是失信放大器；反向校验免费体检

三动词链：构思打开可能性 → 定义终结漂浮 → 设计转换到底（设计一词两义: 容器 vs 下游）

架构生成六步（前置入场券：《系统目的或任务理解确认单》）：定驱动 ASR → 生成备选 ≥2 → 权衡效用树+ADR → 分配接口 ICD → 视角规格 + 表达基线 → 评审背书 TR2/TR3

目的入口（入场券背后）：目的=系统存在的理由（环境侧·为利益相关方）→ 分层目的/目标/需求 → 三条来路 + 到位判据三 → 收口确认单；以目的检验三分（确认/评审以目的为依据、验证以规定需求为参照）

关切与目标：关切是原料、目标是产物、目的是准绳——「关切 → 目的」这条推导线不存在

系统基本概念和基本属性清单：目的判据的裁决产物——评审第一核对单 · 受控枚举 · 双向覆盖检查 · 治理机构冻结

创新坐链尾评分席：新×成，缺一归零；架构式创新主导动作是定义

见过→拥有：画图 / 选型 / 定规矩 = 见过；把本质与管控定形、落进可描述的架构 = 拥有

## 本章判据

> 本章判据多为本书的提炼与主张，不是标准条文——凡有标准出处者正文已随文注明；每条当场可检验，欢迎检验、欢迎校准。

1. **架构设计 = 翻译**：把需求翻译成”架构”——定出实体的基本概念或属性、立起约束实现与演化的管控原则；画图、选型、定规矩都只是这个翻译的产物/候选/规则，不是翻译本身。**违反即崩：**

**水忠带了三样东西（图、选型、老规矩）进会议室——没有一样是翻译这个动作，架构设计等于没做。**

2. **设计是动词，定义是完成时**：定义 = 划界 + 陈述（de-finis）；过程以产出物命名——构思无法评审，描述才能评审、验证、基线化。没定形，就等于没设计。**违反即崩：构思停在脑子里，拿不出可评审、可基线化的产物——“我们做过架构设计”被当成”我们有架构描述”。**
3. **横切先于纵挖**：架构设计横向铺（跨全部关切与整个生存周期，管适宜性、存续性与合意性），详细设计纵向挖（往实现里挖，管兼容性与可行性）——先横后纵；分界不在粗细，在种类（§11.2.5）。**违反即崩：横向那层定错了，纵向挖得再深也是白挖——墙砌错了要拆楼，装修错了刷一遍就行。**
4. **非功能需求才是主要驱动**：功能需求告诉”做什么”，质量属性告诉”做到什么程度”——后者才真正决定结构形态。**违反即崩：功能相同的系统，质量属性不同，架构可以完全不同——“提前 30 秒还是 5 分钟”不定，预判逻辑放硬件还是云端就无从定。**
5. **输入是六类、且必须是规定需求**：六类里目的与任务居判据侧首位（以《系统目的或任务理解确认单》锚定进入），需求必须是系统级规定需求（已验证、未分配），在流沙上画图等于自损一目；需求分配是架构设计自己的工作，不是输入。**违反即崩：拿”要**

**能提前预判断链”这句话直接设计，水忠在第一版架构上反复返工——需求一变，本质与原则跟着动，后面全跟着动。**

6. **质量属性不能分，只能译**：涌现属性是整个系统的属性，只能被架构机制满足、再翻译成部件级约束。**违反即崩：把”可用性 99.99%“切碎了分给每个模块——根本没有哪个模块”负责可用性”，只能设计冗余机制让属性涌现。**
7. **关注点是识别出来的，不是设计出来的**：识别五步（利益相关方→提取→划界→编排→验证）；好 concern 四判据（有主/有界/可权衡/可追溯）+ 完备（矩阵无空行空列）；与需求分家靠三测（换人／改写／粒度——合适粒度＝刚好足以挑出视角）；每条必须四类分流（承重/约束/偏好/交付），**悬空不是选项**；主张者随附来源·后果·度量，**不齐退回补件而非驳回**（§15.4.5）。**违反即崩：从”我要画什么”出发——先定画哪几张图、再往回填关注点，利益相关方真正在乎的被漏在矩阵外；改从关注点出发，矩阵里的空格才会暴露出来（案例进展②）。**
8. **概念幸存、属性涌现**：基本概念 90% 选 + 10% 逼，候选→检验→幸存，命名是完成仪式；基本属性不能直接设计，只能设计结构让属性涌现并度量验证。**违反即崩：候选画进了图里，检验不过照样死——“边缘预判”没成为新本质，只是追溯机制的新输**

**入；“及时性”画不进去，只能设计”双路采样 + 边缘计算”让结构催生**。

9. **管控原则是自愿的自我限制，立了就有约束力**：约束被迫、决策自选、原则自加；句式 = 必须/禁止/应当 + 规则 + 理由；可判定/有理由/少而精/可入评审；立法后要把它放到演化上检一遍（时间维，§11.3.9）。**违反即崩：老规矩过不了四关——没有理由、不能判定、多如牛毛、入不了评审，它就不是纪律，是教条**。
10. **关系是成本**：关系稀疏才可演进；依赖无环、往弱推、契约化；五判据（有方向/够稀疏/尽量弱/要显式/不穿越）。**违反即崩：n 个要素全互联是 $n^2$ 条关系，每条都是未来变更的一处牵动——循环依赖是架构腐蚀的头号元凶**。
11. **只有概念直面世界**：本质（概念/属性）、管控原则、元素与关系的输入是外部原始输入 + 内部级联；概念错了后面全错——反向校验（这条关系服务哪个属性？这个属性被哪个概念限幅？）答不出的就是无源之水。**违反即崩：概念设计错了，后面跟着错、错得越来越远——概念说”无状态”，原则却是抄来的”共享缓存”，评审对不上账**。
12. **构思打开可能性，定义终结漂浮，设计转换到底**：三动词链是接力——构思生成 ⩾2 候选、定义画边界定形、设计（狭义）基线

生效后转换到底；“设计”一词两义必须声明。**违反即崩：只有一个候选就”定义”——唯一解 = 未审视的解，没有权衡可言。**

13. **生成六步，缺一步不闭环**：（前置入场券：《系统目的或任务理解确认单》）定驱动 → 生成备选 → 权衡选定 → 分配接口 → **视角规格＋表达基线** → 评审背书；三纪律（驱动/决策/基线先于图）；⑤还须写齐视角规格六项、视图逐张声明治理视角、记跨视图对应关系（第 15 章 §15.4.1、§15.4.3）。**违反即崩：没有背书的架构只是架构师的个人意见——评审是验证关，绕过去等于没验证。**
14. **创新 = 新 × 成，坐链尾评分席**：创新不在动词链上；够新又成了才追认；架构式创新主导动作恰是”定义”。**违反即崩：把”点子”当”创新”——只有新没成是点子/发明，没落地不算创新（奥斯陆手册）。**
15. **目的定义在先，以目的检验三分**：目的是系统存在的理由——环境侧、为利益相关方、解决什么问题创造什么价值；目的→目标→需求分层，评审、确认沿链上溯到目的；以目的检验三分——确认（预期用途）与评审（所规定目标）以目的为依据，验证以规定需求为参照、不直接以目的为参照；目的到位三判据（可作判据/可分解/可追溯）。**违反即崩：拿「成为行业领先」这类裁决不了的目的当判据，什么都裁不出来；「验证过需求」不等于「确认过用途」——MOE 全绿也可能没过确认关。**

## 自查 · 这些说法对吗？

1. “架构设计 = 画架构图 + 做技术选型 + 定老规矩。”——**错**。这三样是产物/候选/规则；架构设计是把需求翻译成”架构”的动作。
2. “我们做了架构设计 = 我们有架构描述。”——**不准确**。设计是动词（构思），定义是完成时（定形）；构思停留在脑子里，定形才能评审、验证、基线化。
3. “架构设计就是选技术栈。”——**错**。选型是待检验的候选；架构设计是在信息最不完整时定出本质、立管控原则、把它们表达成架构描述并交代落点——选型只是其中很小一部分素材。
4. “功能需求决定架构形态。”——**不准确**。功能需求告诉”做什么”，质量属性才真正决定结构形态——99.99% 可用和 99% 可用是两个架构。
5. “架构设计的输入 = 全部需求原文。”——**错**。输入是系统级规定需求（已验证、未分配）；需求分配是架构设计自己的工作。
6. “关注点是可以’设计’出来的。”——**错**。关注点是识别出来的，不是设计出来的——它已存在于利益相关方的头脑/组织/契约里，架构师的工作是发现、提取、陈述。
7. “基本概念是架构师发明出来的。”——**错**。90% 是从概念库选的，10% 是被需求/属性逼出来的；候选要过检验才配叫”基本”，命名是概念化的完成仪式。

8. “直接设计可用性，把 99.99% 分配给每个模块。”——**错**。质量属性是需求侧的目标，涌现在系统上的是（基本）属性——不能分，只能译；你设计的是实体的构成，属性是它运转后的宏观效果。
9. “原则越多越严谨，把能想到的规矩都写上。”——**错**。原则要少而精（5~15 条）、可判定、有理由、可入评审；没有理由的是教条，太多等于没有。
10. “定了’怎么造对’的规矩就够了，‘怎么变’不用管。”——**错**。管控原则同时约束实现与演化及其生存周期；只立”造对”不立演化，架构在无人管束时必然退化（漂移/侵蚀/技术债）。
11. “关系越多系统越强，大家尽量互联互通。”——**错**。关系是成本（$n^2$）；关系稀疏、依赖无环、往弱推、契约化，才可演进。
12. “架构设计要产出的是几样并排的东西。”——**错**。基本概念直面外部输入，属性被概念限幅，管控原则从概念/属性提炼，元素与关系是落点——概念错了后面全错；反向校验是免费的架构体检。
13. “构思 = 等灵感。”——**错**。构思有方法（模式库 / 参考架构 / 第一性原理）；构思是三动词链第一环，产出 ⩾2 个候选。
14. “架构生成可以说成’架构构思 + 架构定义 + 架构设计’。”——**错**。“架构定义”是 15288 流程名（覆盖全程），“架构设计”排在基线生效之后；生成是 构思 → 定义动作义 → 表达 + 认证。

15. “创新 = 灵感 = 构思。”——**错**。创新是结果义（新×成）；构思只供给”新”，“成”要靠定义与设计兑现。
16. “架构设计的输入就是需求，目的和任务跟设计没关系。”——**错**。六类输入里目的与任务居判据侧首位、起决定作用——它是 fundamental 成员资格的裁决者；以《系统目的或任务理解确认单》锚定进入设计（§11.3.2）。
17. “《系统目的或任务理解确认单》是设计做完之后补的文档。”——**错**。它是架构设计的入场券、判据侧首类输入「目的与任务」的确认形态——签字之前，后续一切产物都没有判据锚点（§11.4.3）。
18. “目的、目标、需求是一回事，定了目标就定了目的。”——**错**。目的是系统存在的理由（环境侧/为利益相关方），目标是”要实现的结果”（ISO 9000:2026 §3.7.11 objective），需求是可验证条款——三层各站各位、逐层具体化；以目的检验三分——确认与评审以目的为依据，验证以规定需求为参照；MOE 全绿 ≠ 确认通过（§11.4.3 目的深化）。

## 练习

1. 挑一个你正在设计或维护的系统，按”两样内容＋它们的落点”回答：它的本质——基本概念（组织范式/核心抽象/根本机制/根隐喻）、基本属性（生命线属性）——各是什么？管控原则（约束实

现与演化）怎么立？本质的落点在哪些元素与关系上？——答不出任何一样，那一处就是你的设计缺口。

2. “把需求翻译成’架构’”——挑一个最近的真实需求（比如”要能提前告警”），试两种翻译（A 边缘计算 / B 云端计算），各列一条好处、一条代价。为什么必然做选择？
3. 找出你们系统最近一次架构变更，用”两样内容＋它们的落点”判断：它动的是基本概念、基本属性、管控原则，还是元素与关系？如果动的是基本概念（系统本质变了），团队有没有意识到这一点？
4. 列出你们系统的利益相关方和关注点，做一张 stakeholder × concern 矩阵——哪个格子是空的（有利益相关方没关注点，或有关注点没利益相关方）？哪个格子之间在冲突？
5. 挑一条你们系统里的”老规矩”（想立成管控原则的），用四判据检查：可判定吗？有理由吗？少而精吗？可入评审吗？——过不了关的，它不是原则，是教条。
6. 把你们打算立的管控原则逐条做”时间维化”：这条规矩在变更时怎么执行？——产出一条约束演化的原则（用”变”字句式）。
7. 画出你们系统关键模块之间的依赖关系，检查：有环吗？有多少条关系是不必要的？把一条强关系降级成弱关系（同步调用 → 异步消息 / 事件），变更的牵动范围小了多少？

8. 挑一个属性（如可用性、响应时间），走四步流水线：场景化（QAS）→ 战术 → 构成落位 → 验证度量。注意——属性必须落到”度量”上，否则承诺等于没有。
9. 用”反向校验”给你们的架构做一次体检：随机挑一条管控原则，向上追——它从哪个概念/属性提炼来的？答不出，它就是”无源之水”（输入链断过）。
10. 区分三个词：约束、决策、原则——各找一条你们系统里的实例。判断：哪条是外部强加的（接受），哪条是自己选的（决策），哪条是自愿的自我限制（原则）？
11. 用第 1 章的四问法量一个本章概念——“基本概念”或”基本属性”：它是什么、不是什么、从哪来、往哪去？哪一问你答得最薄？最薄的那一问，往往就是这块设计真实的缺口。
12. 把一个架构决策拆成三动词链：构思阶段你生成过几个候选？定义阶段被否掉的备选”死因”记录了吗（ADR）？设计（狭义）阶段是等基线生效才动的吗？
13. 用六步给最近一次架构活动做体检：六步里哪一步被跳过了？（最容易被跳过的是②生成备选和⑥评审背书——直接从需求跳到唯一解画图，是架构层的 Cowboy Coding。）

14. 判断你所在组织最近一次被叫”创新”的事：够新吗？成了吗？主导动作是构思、定义还是设计？——元件不动、连接重排的架构式创新，主导动作是定义。
15. 写一份《系统目的或任务理解确认单》：业务方原话（只引不改）+ 你的复述理解 + 确认记录——检查你的复述里有没有出现架构语言（出现，就是越界了）；再问一句：这份确认单是你架构设计的输入，还是设计做完后的补记？
16. 给你的系统做一次”目的体检”：按三条来路（战略承接 / 业务或任务分析 / 关注点反推）逆推系统目的，再用到位判据三检查——可作判据？可分解？可追溯？答不出的，目的还没到位，别急着写确认单。

（答案要点见书末附录，与训练教材题卡同源）

## 参考文献

**标准** - R-04 ISO/IEC/IEEE 42010:2022 —— 架构定义（本质 = concepts or properties；管控原则 = governing principles，有约束力、约束实现与演化及其生存周期；**该定义不给”什么算基本”的准入判据**，判据的形式见本书形式化；落点读法见 ch10 §10.2.5；出处链 IEV 831-01-01 Note 2 系二手派生表述，仅作外部佐证）；architecting 七动词链（构思→定义→表达→记录→沟通→认证恰当实现→维护改进）= 能力增量的母词 - R-05 ISO/IEC/IEEE 42020:2019 —— 架构过程（§8.4.5 在解空间中综合潜在方案／

§8.4.7 形成候选架构：候选架构概念由此生成，后接评估、选择）；架构治理（§6.4.3 架构集合目标／§6.4.4 架构治理决策）- R-09 ISO/IEC/IEEE 15288:2015 —— 架构定义过程（6.4.4，内部第一件事=生成候选架构）vs 设计定义过程（6.4.5，基线生效后的下游） - R-103 ISO 9000:2026 —— §3.3.11 设计和开发定义（“将某一客体的要求转换为该客体更详细要求的一组过程”）；§3.7.11 objective 定义与注（目标=要实现的结果 · 分战略/战术/操作）；§3.11.14 确认（针对特定预期用途的要求）；§3.11.2 评审（针对既定目标） - R-06 ISO/IEC/IEEE 25010:2011 —— 质量模型（属性清单有标准可查）- R-07 SEI ADD / ATAM —— 属性驱动设计（11.3.7 四步流水线的出处）/ 架构权衡分析（效用树） - R-15 ECPD / ABET —— engineering design 容器定义（“设计”一词两义的广义出处） - R-20 ISO/IEC/IEEE 24765 —— design 双义（过程与结果）- R-01 ISO 1087:2019 —— definition = 区分概念的表述（§3.3.1，“画边界”的跨域印证）- R-75 ISO 56000:2020 —— 创新 = 创造或重新分配价值的新对象或变更对象（结果义：新×成）- R-76 ISO 56001:2024 —— 可认证的创新管理体系（与 9001 同构）- R-77 ISO 56002/03/05/06/07 —— 创新管理家族 - R-85 ISO/IEC/IEEE 42030:2019 —— 架构评估框架（以目的检验架构：AE = 就所规定的评估目标对架构所作的判断，§3.4；42010 管描述、42020 管过程、42030 管评估；§10.3.3 术语注同源）

**经典文献** - R-35 Len Bass 等《Software Architecture in Practice》—— 非功能需求（质量属性）才是架构的主要驱动 - R-55 Evans《领域驱动设计：软件核心复杂性应对之道》（2003）—— DDD：bounded context / aggregate - R-56 迪米特法则（Law of Demeter, 1987）—— 最少知识原则——“talk to your friends, not strangers” - R-67 依赖倒置原则（DIP, Martin）—— 面向接口编程、依赖倒转（SOLID 之一）- R-68 邓巴数（Dunbar, 1992）—— 群体规模上限约 150 人——“邓巴数式的直觉” - R-78 Henderson & Clark 1990（Administrative Science Quarterly）—— 创新四分型（架构式创新主导动作=定义）- R-79 熊彼特《经济发展理论》1912 —— 发明≠创新（创新=新组合的兑现）- R-80 奥斯陆手册（OECD）—— 创新须”已被提供给潜在用户或投入使用” - R-81 Parnas 1972 —— 信息隐藏（partition 按”会一起变的”分组） - R-82 Perry & Wolf —— 架构腐蚀（drift / erosion）- R-83 Cooper（Stage-Gate）—— 创新流程可工程化 - R-64 IPD（集成产品开发）—— 技术评审点 TR / 决策检查点 DCP（从这条路径推出的主流方法）；TR2/TR3 作架构评审放行点（六步第⑥步评审背书） - R-84 Altshuller（TRIZ）—— 构思不靠等灵感（“有方法”的证据）- R-45 INCOSE 系统工程手册 —— MOE/MOP/TPM 三层度量（MOE=效能度量=运行环境中的

成功指标≈操作目标层·运行层确认的显性判分线；MOP=性能度量=系统级；TPM=技术性能测量=关键部件级）

> 对照阅读：本章”架构设计＝两样内容＋落点交代”承接第 10 章的架构定义（基本概念或属性＋管控原则）与”系统不可还原的涵义”读法，呼应第 6 章”需求与方案结对”、第 12 章”架构决策”（本质与管控原则里哪些最难撤销、何时拍板）；视角规格、关切裁决与两个闸门的细则见第 15 章。

# 第 12 章 架构决策：对不可逆的提前投资

> 本章概念：架构决策 / 不可逆性 英文来源：architecture decision / irreversibility 主案例：恒信集团合同审批系统（IT 侧）｜镜照：冰链科技冷链温控箱（产品侧） 术语口径：本章「基本概念或属性／管控原则／生存周期」按 GB/T 45630-2025（等同采用 ISO/IEC/IEEE 42010:2022）定名；本书早期稿件的「治理原则」即「管控原则」（governing principles）。对照表见附录 A。

---

## 12.1 误解现场

恒信合同审批系统，黄程提了一个”小需求”。

黄程推开万辉工位边的转椅，把手机屏幕转过来：“万辉，合同编号得改。电子签章平台那边的规则是’合同类型-年份-流水号’，跟咱们库里对不上。帮我们把编号规则改了?”

万辉瞄了一眼：“改个编号格式的事，小事。今天就能上。”

路过的产品开发经理满海听了半句，也没停下脚步：“改个字段而已，测试跑一遍就上，别拖了。”

当天下午，万辉改了合同主表的编号字段，顺手把合同状态枚举加了两项——“作废”“草稿”——方便法务标状态。测试环境跑了一遍，正常。上线。

三个月后，连环三爆。

**第一爆，财务对账**。合同编号按新规则重排后，与财务已归档的纸质合同对不上。审计抽查时，黄程查了三天，查不出”去年 12 月那批 500 万合同”对应的电子记录。

**第二爆，台账**。下游台账系统按旧编号规则解析，全部乱掉——同一份合同在台账里识别成了三条。

**第三爆，追溯**。法务要查”半年前那份作废合同当初是谁批的”，历史审批记录的关联键断了，审批链断在中间。

会议室里，万辉还觉得冤：“我就是改了个字段。字段嘛，不是想改就改?”

徐漾看着他，说了一句让整间会议室安静下来的话：

“万辉，你改的不是一个字段。你改的是这套系统的**数据模型**——它是这套系统里最不可逆的决策之一。你却把它当成了’改个变量名’。”

如果你坐在这间会议室里，你大概也会站在万辉那边。改个编号规则、加个状态枚举——听起来就是”改个字段”。大多数人对”架构决策”的直觉正是如此：架构是画图的事，改字段是码农的事，两码事。

这一章要推翻的，正是这个直觉。**架构决策不是某次评审会上”画架构”才发生的；它藏在每一个”改个字段”里**。

合同的数据模型怎么组织、编号规则定成什么样、状态枚举分几类——这些选择，选错了要花大代价才能改对。而”想改就改”，恰恰是最危险的一种对待方式——**把高不可逆的决策，当成低可逆的决策来处理**。

这一章我们做三件事——识别它、磨掉三种错觉、把尺子用起来；三幕的具体骨架，见下面「开篇 · 本章枢纽」。

你会发现两件反直觉的事：**架构决策不可避免——不是做不做，是早晚；而不可逆的决策，必须在成本低的时候提前做**。

---

## 这一章要回答什么

读完本章，你应该能回答：

1. 什么算”架构决策”？为什么”加一个字段”也可能是架构决策？
2. 为什么说架构决策不可避免？“我们没做架构决策”到底是什么意思？
3. 怎么判断一个决策的不可逆程度？“不可逆”是技术属性还是经济属性？三层判断框架是什么？代价判据和目的判据是什么关系？
4. 数据所有权/数据模型为什么几乎最不可逆？改它要按什么待遇对待？
5. 高不可逆的决策应该什么时候做？“推迟决策”怎么反而是一种架构操作？
6. 一个决策怎么被记下来？决策记录六问是什么？“不可逆性标注”标在哪？
7. 审查深度该按什么分级？给低不可逆的决策开三场评审会值得吗？
8. 从需求到实现，每一层的架构决策为什么不同？需求有架构吗？

---

## 这一章的骨架

先把本章要钉的两个词、和它们牵出的一条判断链立起来——后面三幕，都是给这层骨架添血肉。

> **架构决策 = 选错了要花大代价才能改对的选择。** 它藏在每一个”改个字段”里；而判断它值不值得慎重，靠的不是”这是不是技术难题”，而是一把经济学的尺子——**不可逆性**。

> 这里有一句必须先说清：不可逆性本身**只是代理，不是准入判据**。它会**漏判**——不贵但承重的东西（如”日志须可被外部审计员独立读取”）；也会**误纳**——很贵但与目的无关的东西（框架版本、组织惯例技术栈）。
> 代价关因此只能在**基本（fundamental）之内**排序与决定何时锁死；被识别的架构决策**不因此进入架构**。它的正当用途只有四样：排序、定决策时机、配治理强度、做回代检查。

这一章的骨架是**一个词 + 一把尺子 + 一条判断链**：一个词（架构决策——用三标志辨认：影响半径大 / 持续时间长 / 修改成本高）；一把尺子（不可逆性——三层判断框架：影响半径 / 信息成本 / 修改风险）；一条判断链（**识别 → 判断 → 抉择 → 记录 → 分级 → 定位**，识别属第一幕、分级并入第三幕）。

第一幕把”架构决策”这个词钉死——识别它，别让它在”改个字段”里蒙混过关；第二幕把三种错觉磨掉——它是”不是什么”的

边界；第三幕把尺子用起来——判断、抉择、记录、分级、定位，一条链走到底。

---

## 12.2 第一幕 架构决策是什么（定界）

> 这一幕把”架构决策”看清：先看病（“改个字段”为什么炸了三个月），再钉定义（选错了要花大代价才能改对）+ 三标志判据，然后用三个思想实验让三标志长进直觉里——最后落在识别它的第一道动作：改字段前三问。

### 12.2.1 先看病：“改个字段”为什么炸了三个月

万辉的病，不是”改错了字段”，而是把”数据模型”当成”字段”——把一件架构决策，当成一件普通实现。两个东西长得像（都要改代码、都能改）、其实不同层（一个是架构决策、一个是实现决策），糊在一起用，就出事了。

先看万辉到底改了什么。合同审批系统的数据模型里，“一份合同”是三个实体：合同主表（编号、类型、状态、金额、签署方）、审批记录（审批链、各节点意见）、附件（合同文件）。这三个实体之间的关联、编号怎么编码、状态枚举怎么划分——这套数据模型，决定了：

- **财务怎么对账**：台账按合同编号解析，编号规则变了，解析全乱；
- **法务怎么追溯**：审批记录挂在合同主表上，关联键变了，历史审批链断掉；
- **下游怎么对接**：台账、报表、审计，全都依赖这套编号和状态口径。

换句话说，万辉改的”一个字段”，牵一发动全身。这正是架构决策的第一特征——**影响半径大**。它不像”改一个函数内部的排序算法”，改了只影响一个点；它像地基里的一根桩，动了它，整栋楼的承重都要重新算。

但这里有个更隐蔽的陷阱，值得单独挑出来——**万辉不是”改错了”，他甚至没意识到自己在做一个决策**。他以为自己在”执行”一个字段修改，实际上他在”决定”数据模型的结构。

这个”没意识到”，才是最危险的：**没有被识别出来的架构决策，不会得到它应有的审查，却依然承担架构决策的全部代价**。认不出它是架构决策，就不会有人认真论证、认真评审、认真记录——但它改错的后果，一样都不少。

### 12.2.2 权威定义：选错了，要花大代价才能改对

“架构决策”这个词，在标准里有一个不带修饰的定义；架构领域另有一条更直白的操作表述：

> **架构决策 = 在架构的总体语境中做出的一组选择**。（a collection of choices made in the overall context of an architecture，ISO/IEC/IEEE 42010:2022 §5.2.12）
> 操作表述（**本书凝练，不是标准原文**）：**这个选择选错了的话，后来要花很大代价才能改对**。

“花很大代价才能改对”反过来就是它的**代理判据**：如果你做了一个选择，后来发现错了，改一下只要几分钟——这不叫架构决策，这就是

普通的实现选择。真准入判据是目的必要性（本章 L125 与判据块会说清这层关系）。

“花很大代价”这个说法，听起来有点虚。用什么度量？架构领域给出了三个标志，也就是三把尺子：

| 标志 | 含义 | 反面（非架构决策） |
| --- | --- | --- |
| **影响半径大** | 改了它，不止改一处，要联动改很多东西 | 改它只影响一个文件、一个函数、一个模块内部 |
| **持续时间长** | 一旦选定，后续无数个其他决策都建立在这个决策之上 | 今天做了，下周就可以换掉，不影响其他东西 |
| **修改成本高** | 选错了要改对，代价大到你会选择”算了，就忍着吧” | 选错了，改对只需要一次提交 |

> 术语注：本节”三标志＝识别架构决策的判据”是**早期稿的旧读法**，已降为**代理**——真准入判据是目的必要性（见下）；三标志管的是”要不要按架构待遇办、排在什么时候”。本表第二行的标尺取**生存周期**（life cycle）义，它是代理三标志里**对应时间的那一维**，在 §12.4.1 的三层框架里落在”影响半径”的**时间耦合**一格。

> 与第10章对账：§10.2.4 的”不可还原”三判据（去掉/缺少/换掉）识别的是系统最核心的架构——去掉就不是它的那层；这里的三标志量的是**代价**，位置在代价关，不在目的关。

用三把尺子量量万辉改的”编号规则”：

- **影响半径**：编号规则一变，财务、台账、审计、电子签章全部要跟着改——**大**；
- **持续时间**：编号一旦定下来，三年后归档的合同还按它编码——**长**；
- **修改成本**：改了之后才发现要重排历史数据、重对审计账——**高到想回到改之前**。

三把尺子全中。所以尽管万辉只敲了几行代码，他做的仍然是一个架构决策。

这里已经能看到第 1 章示范的判据主义——不给没有判据的定义。但要说准：**真准入判据是目的必要性**——拿掉它，目的是否落空；三标志不是判据本身，是它的**代理**。

**“基本”这条判据，操作化成三道测试**（判据形式见第 10 章 §10.2.4、第 11 章 §11.3.6；代理的两类失真与关切裁决见第 15 章 §15.3.2、§15.4.5）：

1. **承重测试**：去掉它，目的会怎样？——目的**跨**（概念级基本）／目的不变但实体性质**变形**（属性级基本）／只是**变难**（不进架构层）；
2. **替换测试**：换一个实现，这个概念／属性还在不在？——在，是基本；不在，那只是当前实现的一部分；

3. **两遍跑**：兑现侧（实现目的需要它吗）＋演化侧（目的变了，它守不守得住）。

**注意：这三道测试与三标志不是同一把尺。**前者量”进不进架构”（对目的量），后者量”代价多大、什么时候锁”（对代价量）。量错了尺子，就会拿”改起来贵”当成员资格。

三标志的准确位次，一句话写清、可被当场检验：**三标志管”要不要按架构待遇办、排在什么时候”，不管”是不是架构”**。”是不是”由**目的必要性**单独裁决——直引第 10 章 §10.2.4 与第 11 章 §11.3.6：拿掉它，系统还能不能达成其目的。三标志只是这条判据的快速代理。

谁在这条判据上有位置？**指向这个架构决策的利益相关方是谁、他持有的是哪条关切**——这一步要在识别的同时点出来。恒信的数据模型，利益相关方是财务、法务、审计、对接方；他们的关切是”账要对得上”“追溯链不能断”。

**没有利益相关方与关切的决策，不会有主人。**

这个定义不是凭空冒出来的。“架构决策”这个词，近年才成为工程界的高频词，它的兴起和两件事有关。

一是 **ISO/IEC/IEEE 42010**（架构描述标准）把”架构决策”当作**架构描述（AD）必须记录**的核心内容之一（§6.10.1：An AD shall record architecture decisions considered essential to the architecture of the entity of interest）——标准给的分寸是：架构决策是”**在架构的总体语境中做出的一组选择**”（§5.2.12）。**选择不是架构本身**：架构是实体在其环境中的基本概念或属性与管控原则（§3.2），决策是认出并记下那个架构的抓手。

二是**架构决策记录**（Architecture Decision Record，ADR）这种做法的流行——2011 年，软件架构师 Michael Nygard 提议用”一页卡、回答几个固定问题”的方式记录架构决策，后来被大量团队采纳。

从那时起，“架构决策”从一种模糊的说法，变成了一个**可记录、可管理、可审查的工程对象**。这一章讲的，就是这个对象：怎么识别它、怎么判断它、怎么做出它、怎么记下它。

### 12.2.3 三个让人”哦”的例子：书架、厨房、订单服务

“影响半径大、持续时间长、修改成本高”——这三个词还是抽象。架构文献里流传着三个经典例子，每一个都是三标志的完美演示，也每一个都能让你拿身边的事套一遍。

**例子一：书架怎么分类**。你有一千本书，按颜色、按作者首字母、还是按主题？选了按颜色——以后每次找”Python 相关的书”，都要把所有颜色层翻一遍；选了按主题——“计算机”那一层秒找到。选完再想从按颜色改成按主题，五百本书全部重新摆——这就是修改成本。“书架按什么规则分类”，就是一个架构决策。

**例子二：商用厨房的布局**。灶台和冰箱之间的距离。灶台紧挨冰箱：效率高，但散热互相影响；灶台和冰箱在对角线：各自独立，但每次拿食材走八步。不管选哪个，一旦灶台接好燃气、冰箱接好水电，再想挪位置，就得拆墙、重铺管线、停工几天——这就是不可逆。厨房布局，也是一个架构决策。

**例子三：软件里订单服务的边界**。“库存扣减”放在订单服务里，还是库存服务里？放订单服务：前期开发快，但三年后采购入库、盘

点调整都要改订单服务，订单服务开始管库存的事，职责膨胀；放库存服务：多一次远程调用，但库存的所有操作（扣减、入库、盘点、调拨）都在一个地方，职责干净。“库存扣减放在哪个服务里”，还是一个架构决策。

注意这三个例子的共同点：**它们都不是”技术难题”，而是”选择难题”**。书架分类不需要任何高深知识，厨房布局也不是物理难题，订单服务边界更是没有标准答案。

架构决策的难，不在”怎么做出来”，而在”选哪个、为什么选、选了之后要承受什么”。这也是为什么它常常被当成”反正怎么选都行，那就随手选一个”——随手选，就是不做决策地做决策，代价迟早会来收账。

把三标志套一下日常常见决策，你会看到一条光谱——不是所有决策都同样”架构”：

| 决策 | 影响半径 | 持续时间 | 修改成本 | 是不是架构决策? |
| --- | --- | --- | --- | --- |
| 系统拆几个服务，每个服务管什么 | 大（所有后续开发都在这个框里做） | 系统整个生存周期 | 极高（数据迁移+团队重组+接口重定） | ✅是 |
| 订单服务的数据库选 MySQL 还是 PostgreSQL | 中（影响一个服务的表结构设计+运维） | 系统生存周期（迁移成本高） | 中-高 | ✅是 |
| 订单服务内部用哪个 ORM 框架 | 小（只影响内部实现） | 中（换框架要重写数据访问层但接口不变） | 中（机械劳动，风险可控） | ⚠边界 |
| 订单列表排序用冒泡还是快排 | 极小（只影响一个查询的性能） | 短（随时可以优化） | 极低（换算法一次提交） | ❌不是 |
| 变量名叫 orderList 还是 orders | 极小 | 中（命名规范一旦确立会持续使用） | 低（全局替换即可） | ❌不是 |

光谱的意义在于：**“架构决策”和”实现决策”之间没有一条自动分界线，分界线靠三标志划**。划错了——把架构决策当实现决策——就是万辉的翻车现场。

### 概念卡片 · 架构决策

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 在架构的总体语境中做出的一组选择（42010:2022 §5.2.12）——选错了后来要花很大代价才能改对；用三标志辨认：影响半径大 / 持续时间长 / 修改成本高 |
| **它不是什么** | ⑥ | 对立：实现决策——架构决策改不起（三标志），实现决策改得起（一次提交、局部影响） |
| **它不是什么** | ⑦ | 辨析：架构决策 ≠ 架构风格——微服务/事件驱动是决策的”套路”，具体系统选了哪些决策才是架构决策本身；也不等于”画架构图时做的决定”——它藏在每个”改个字段”里 |
| **它从哪来** | 三标志③（代理） | 改起的决策 |

## 案例进展①｜恒信：一次”改个字段”，三次撞墙

万辉那天晚上把编号变更回滚了一半，没完全回滚——因为回滚也要成本。他把徐漾拉进会议室，第一次认真复盘”改个字段”这件事。

玉杰那晚负责执行回滚，只回滚了一半就停了。他把运维记录摊在桌上：“代码能回滚，可台账、财务已经按新编号开始解析了，它们不跟代码一起回滚——要全回滚，得把已经按新编号解析过的数据迁回去，账重对一遍。”

“我到现在也没想明白，”万辉说，“我改编号规则之前，查过所有引用这个字段的地方，十二处，全改了。测试也过了。为什么还会炸?”

徐漾没有直接回答，反问：“你改之前，有没有问过——‘合同编号’这个字段，除了代码里十二处引用，还有谁在依赖它?”

万辉：“还有谁？台账系统、财务对账、电子签章……它们都是对接方。”

“对。它们不在你的代码里，但它们是这份数据模型的’下游’。”徐漾说，“你说的十二处引用，是’字段’层面的依赖；台账、财务、审计，是’数据模型’层面的依赖。你只排查了字段层面，就把一个数据模型级的决策，当成字段级的操作给做了。”

小白负责对接方接口，她插了一句：“合同编号在我们接口文档里是定义得最死的字段，台账、财务对接时都拿它当关联键——这字段从来不只是我们系统自己在用。改之前要是有人问我一声，我第一句就让他先问对接方。”

万辉沉默了一会儿：“那我怎么知道，一个改动是’字段级’还是’数据模型级’?”

“你问自己三个问题，一分钟就能判断——**改了它，要联动改多少东西？它定了之后，多少别的决定要建立在它上面？改错了，改回**

**来的代价有多大？** 三个问题里有两个答’大’，你就该停下来，把这次改动当成一个架构决策来处理——先论证、先评审、先记录。而不是’今天就能上’。”

万辉把这三个问题记在笔记本上。他后来管这个叫”改字段前的三问”——而他每次问完都会发现，真正值得动手改的’字段’，比想象中少得多。

把方法论的那个瞬间点破：改字段前三问，就是第 1 章示范的判据主义在落地——给一个改动下判断之前，先给它一把可检验的判据。万辉这一章做的，也是一次概念校准：把”改个字段”，从”实现的细节”，修正为”架构的决策”。

---

## 12.3 第二幕 架构决策不是什么（磨边界弧）

> 这一幕把”架构决策”和三种错觉分开——判据主义在这里正式展开：概念靠正例活、靠反例定型。三条磨过去：不可逆不是技术属性（是经济属性）、不可避免不是可做可不做、被动做不是没得选。

### 12.3.1 错觉一：不可逆性是经济属性，不是技术属性

先说最容易被技术直觉带偏的一个错觉。前面反复说”修改成本高”“不可逆”，这两个词容易让人误解成”技术上改不了”。这句话必须澄清：

> **不可逆性不是技术属性，是经济属性。**一个决策是否不可逆，取决于修改它需要多少人、多少时间、冒多大风险——而不是”技术上能不能改”。

> 术语注：‘不可逆性是经济属性而非技术属性’是本书的**凝练**——经济学”撤销成本随时间上升”的观察是共识，把不可逆归结为”经济属性”、用”能改 vs 值得改”来表述，是本书的说法，不是任何标准的条文；欢迎检验、欢迎校准。

几乎任何软件决策，技术上都能改。数据库能迁移、接口能重新设计、数据结构能重构——没有改不了的。但”能改”和”值得改”是两回事。**“不可逆”的真正含义是：修改成本已经高到一个临界点，以至于理性的决策者会选择”忍受现有架构的不完美”，而不是”掏出修改成本”**。

万辉的编号规则，技术上当然能改回去——但改回去的代价是：三个月的账重新对、历史数据重新排、台账重新解析、审计重新自查。当这个代价摆上桌，理性的选择往往是”算了，将就着用吧”。于是”能改”变成了”没人愿意改”——这才叫不可逆。

这解释了为什么”不可逆”是架构决策的核心属性，而不是普通决策的属性：**普通决策改得起，所以选错了问题不大；架构决策改不起，所以选错的那一刻，代价就开始复利了。**

“不可逆性”这个词，是从经济学和决策理论里借来的。经济学很早就发现：有些决策一旦做出，撤销的成本会随时间上升——你越晚发现选错了，改错的成本越高。

而金融学里”期权的价值”这个概念（为”保留选择权”定价），正是理解”一个决策应该什么时候做”的关键。这两个概念，会在第三幕 §12.4.7 再次出现——到那时你会看到，“提前投资”不是一句口号，它有完整的经济学逻辑。

补一句位次，别让这把经济学的尺子喧宾夺主：**不可逆性这把代价尺子，是从属的经济校验。**

它不是与目的判据并列的”另一半”——什么是 fundamental 由**目的**单独裁决；代价判据只在这组根本**之内**决定哪些要提前锁死、锁到什么程度、哪些保留自由度——做优先级与时机的排序，**不产生新的架构成员。**

（这里的 fundamental＝对系统在其环境中的目的而言必要且充分——锚为**二手派生表述** IEV 831-01-01 Note 2，只提供这一读法，不是准入判据的条款，见第 10、11 章。）

顺序不可颠倒：**先过目的关**（是不是根本的），**再过代价关**（值不值得现在定死）。

两种错位都是病——用代价判据越过目的判据，把难改但无关紧要的东西误当架构；跳过代价校验，把根本的东西不分轻重全锁死，演化窒息。

### 12.3.2 错觉二：架构决策不可避免——不是可做可不做

如果万辉提前知道”编号规则”是个架构决策，他会怎么做？他可能会说：“那我们不做了。”——这是另一个流行的误读，值得单独用一节来说。

想象你要盖一栋三十层的办公楼。有两种做法：

**做法 A：直接开工。**找几个施工队就干。地基挖到一半发现下面是软土层，得加深——返工。墙砌到五层发现电梯井位置和管线打架——拆墙重来。封顶后发现消防通道宽度不达标——改不了。

**做法 B：先出结构图、管线图、消防图。**各专业对图协调，确认无误后按图施工。

直觉上，做法 A 是”没有架构”，做法 B 是”有架构”。这一节要说的反直觉结论是：**做法 A 不是”没有架构”，它只是把架构决策推迟到了施工过程中。**

地基要多深、承重墙在哪、管线怎么走、电梯井在哪——这些决策在做法 A 里一个也没少做。只是它们发生在”挖到软土层之后”“砌到五层之后”“封顶之后”——晚到修改成本已经爆炸。**做法 A 和做法 B 的区别，从来不是”有没有架构”，而是”架构决策发生在成本低的时候，还是成本高的时候”。**

| 盖楼 | 做系统 |
| --- | --- |
| 地基 | 数据模型——改一张核心表的字段类型，成本可能波及几十个服务 |
| 承重结构 | 服务边界——拆分或合并服务，比改内部逻辑贵一个数量级 |
| 管线 | 接口契约——改了上游接口，所有下游全得适配 |
| 消防规范 | 安全合规——事后改造安全模型，往往意味着架构级推翻重做 |

这也是第 1 章反事实检验的一次使用：把”画架构图”这个动作去掉，架构决策一个也没少——它们只是换了个时间，在更贵的时刻重新发生。

“做法 A 不是没有架构”这个结论，推出一条这一章最硬的判断：

> **架构决策不是”做不做”的问题，而是”什么时候做”的问题。你只有两个选择——在成本低的时候主动做，或在成本高的时候被动做。**

注意这句话的两个部分。前半句，“不是做不做”——很多人以为，我们跳过架构、直接开写代码，就等于”没有架构决策”。错。**写第一行代码时，架构决策就已经在发生了**：数据存在哪个库、功能拆成几个模块、接口怎么定义、状态枚举分几类——这些决策一个也跑不掉。

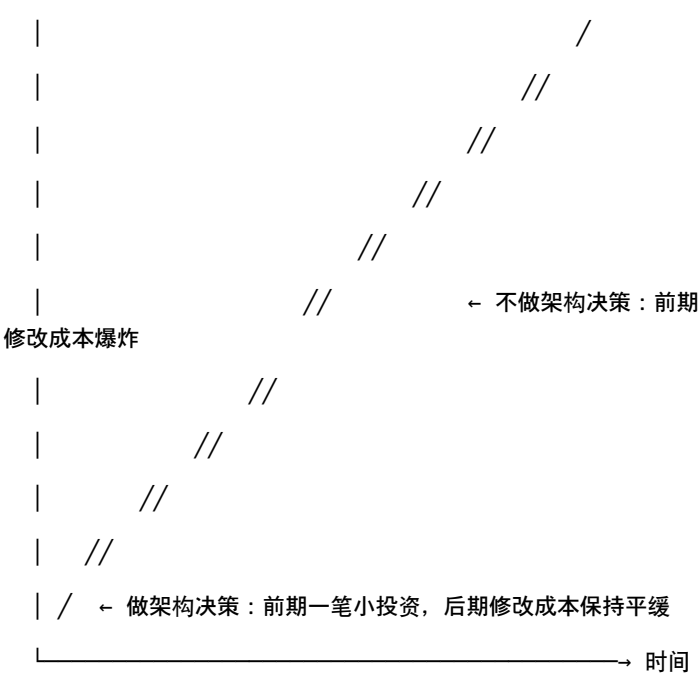
不画架构图，不等于没有架构决策，只是做这些决策的人没意识到它们的不可逆性。

后半句，“主动做 or 被动做”——这就是全部的选择空间。**“我们不做架构决策”是一个伪选项：你只是选择在哪个时刻做它们**。主动做，是在成本低、信息充分、还有回旋余地的时候做；被动做，是等代码写完了、数据存满了、团队定形了，被事故逼着做。

万辉的编号规则，就是”被动做”的教科书：不是他没有做数据模型的决策，是他把决策推迟到了”三个月后、历史数据都进去了、下游都对接了”的时候——然后用一场事故的价格，把决策重新做了一遍。

### 12.3.3 错觉三：被动做不是没得选

“成本低的时候做”和”成本高的时候做”，差别有多大？这张修改成本曲线图，是整个架构决策思想的核心图景：




曲线上方那条陡峭的线，是”不做架构决策”的人生：前期一分钱不花（不用开评审会、不用论证、不用写决策记录），后期每一次改动都越来越贵——因为改动要重算的东西越来越多。

曲线下方那条平缓的线，是”做了架构决策”的人生：前期多花了一点分析和决策的时间（那是一次性的小投入），后期每次改动都在一个被划定的解空间里进行，成本可控。

这张图要说的经济学，就是：**前期的一小时决策，买的是后期的一个月返工**。这还不是最关键的——最关键的，是那些”改不了”的

决策。曲线再陡，至少还能改；有些决策错过窗口期之后，不是贵，是**几乎没法改**——数据已经迁移完了、对外接口已经发布了、历史合同已经归档了。

这也是为什么”架构决策”要和”普通决策”分开对待：普通决策错了可以改，架构决策错了，是拿着整个系统的生存周期去还。

既然主动做这么划算，为什么还有那么多团队像万辉一样被动做？这个问题值得停下来——因为答案不是”他们傻”，而是四个非常人性化的原因，每个人都中过招。

**第一，乐观偏差**。人天然低估未来的改动频率和成本。改编号规则的时候，万辉脑子里想的是”改一下就完事”，不会去想”三年后档案里还躺着十万份按旧规则归档的合同”。**未来被想象得太便宜，于是现在的决策被做得太随便**。

**第二，责任分散**。没有人被指定为”架构决策的所有者”。数据模型是谁的？前端说归后端，后端说归数据库，数据库说归业务——于是没有人对”编号规则”这个决策负责，它默认成了”顺手就能改的实现”。**没有所有者的决策，不会得到它应有的重视**。

而”编号规则”这类架构决策的所有者，正是**系统架构师**的专责：可以委托给模块架构师/模块工程师，但责任仍在自己（第 18 章委托—协商—验收）。

**第三，反馈滞后**。架构决策的代价，往往在几年后才显现——而且那时做决策的人可能已经转岗、升职、离开公司。“今天的决定，三年后的代价”——代价没有主人，决定就没有压力。

**第四，也是最隐蔽的，“隐形的决策”无处显形**。大多数架构决策不是在一次正式的”架构评审”上做的，而是散落在无数个”改个

字段”“加个接口”里。没有人把它们收集起来、命名出来，它们就永远是一堆”顺手的改动”，永远得不到架构决策该有的待遇。

这四条合在一起，指向同一个结论：**被动做不是没有架构，是”没有显式的架构”**。而让架构决策显式化的第一步，就是认出它——给它一个名字、把它从”顺手改动”里挑出来。

这正是第一幕三标志的用途：不是让你把每件事都当成架构决策隆重处理，而是让你**先把那些高不可逆的挑出来，不让它们混在低可逆的堆里被随手处理掉**。

## 案例进展②｜冰链镜照：温度记录的数据所有权，被”随手一改”

几乎在恒信翻车的同时，冰链也出了一档子事——数据所有权。

第 10 章讲过，M3 的架构里有一条铁规矩：“温度数据归属追溯域，传感器只管采、云端只管传、追溯平台管存管管看——数据所有权边界谁都不许动。”这条规矩，是水忠当晚写下、复盘时当众过了一遍的。

然后来了个客户。一家大型连锁药店，订购了 2000 台 M3，提了一个要求：“我们要自己导出原始温度数据，接入我们的质管系统。”

董勐的第一反应是：“简单。给他们的质管系统开个直连接口，把原始温度数据直接推过去。”

水忠听到这句话，拦住了：“董勐，你知道这个’简单’，改的是 M3 的什么吗?”

“不就是加个数据导出接口吗?”

“数据从哪来?”水忠问。

“追溯平台啊。”

“那’直连’的这条通道，绕过谁?”

董勐顿了一下：“绕过……追溯域。传感器→云端→客户质管系统，不经过追溯平台。”

“对。你这一’简单’，把第 10 章我们当众立下的管控原则——数据所有权归追溯域——给改了。传感器的原始数据开始直连客户系统，追溯平台对这批数据失去控制。三年后追溯出问题，谁负责？追溯平台看不见、管不着、审计对不上。”

董勐后背一凉。他忽然意识到，自己刚才准备动手的”加个接口”，是冰链这套系统里不可逆性最高的一档决策——“数据所有权归属追溯域”，正是第 10 章立为管控原则、这一章补上”几乎最不可逆”诊断的那一条。他差点把它当成了”随手一改”。**把高不可逆的决策，当成低可逆的决策随手处理——这正是第二幕要磨掉的第三种错觉**。（怎么量出”极高不可逆”，第三幕 §12.4.5 那张矩阵会给全。）

“所以这不是’接口问题’，是’架构决策’。”董勐把”直连”方案划掉，“客户要导出数据，可以。但要走追溯域的受控通道，每条导出记录留痕。这么一来，导出还是能导出，但数据所有权这条线，谁都没动。”

水忠点头：“对。**不可逆的决策，不是不能做——是要在看清代价之后，用对的方式做**。客户需求可以满足，但满足的方式，不能是悄悄改掉系统里最不可逆的决定。”

---

## 12.4 第三幕 架构决策往哪去（运用）

> 这一幕把尺子用起来：先给不可逆性一把三层判断框架（影响半径 / 信息成本 / 修改风险），再回答什么时候做（抉择）、怎么留下来（记录）、审查资源怎么分（分级）、放在哪一层做（定位）。

### 12.4.1 怎么判断不可逆性：三层判断框架

前两幕把”架构决策”和”不可避免”讲清楚了。现在进入最实操的一步：**怎么判断一个具体决策的不可逆程度?**

先回到第二幕 §12.3.1 那个澄清：“不可逆”不是”技术上改不了”，而是”修改成本高到理性的人选择忍受”。这把判断不可逆性的任务，从”能不能”变成了”多贵”。而这个”多贵”，可以从三个维度分别评估——这就是架构领域常说的**不可逆性的三层判断框架**。

> 术语注：‘三层判断框架’（影响半径 / 信息成本 / 修改风险）是本书对架构领域常见判断维度的**凝练与命名**，不是任何标准给出的固定条款——不同团队可能用不同的维度组合；本书据此组织本章的判断尺子，欢迎检验、欢迎校准。

判断一个架构决策的不可逆性，看三层：**修改的影响半径、修改的信息成本、修改的风险**。对照第一幕的三标志：三标志是识别用的快筛，三层是判断不可逆程度的细化——首项同名”影响半径”，三标志的”

修改成本”对应三层的”修改风险”。下面三节各讲一层，§12.4.5 用一张矩阵把它们叠起来，§12.4.6 用万辉的案例完整走一遍。

给”不可逆性”一把判据主义的量尺：三层框架，就是不可逆性的可检验量尺——拿它量任何决策，都能当场落一个”高 / 中 / 低”。

> 位次注：这三层量的仍是**代价**，全部落在代价关之内；它判”多不可逆、给什么待遇”，不判”进不进架构”——后者只由目的必要性裁（§12.2.2）。

### 12.4.2 第一层：修改的影响半径

改这个决策，需要联动修改多少东西？影响半径决定修改成本的下限——半径越大，成本的下限越高。

影响半径有三个来源，分别对应三种”耦合”：

| 耦合维度 | 低耦合（易修改） | 高耦合（难修改） |
| --- | --- | --- |
| **结构耦合** | 数据归属明确，改一个域不影响其他域；内部接口，单一消费者 | 数据跨多个服务共享，改所有权涉及数据迁移；公开接口，几十个外部消费者 |
| **时间耦合** | 可渐进式迁移——v1 接口保留三个月，下游逐步迁到 v2 | 必须一次性同步上线——如数据库表结构变更，所有读写方必须同步升级 |
| **组织耦合** | 修改只需一个团队内部协调 | 修改需要跨团队对齐、同步排期——技术耦合可以用工具解决，组织耦合只能开会 |

万辉的编号规则，三样全占：结构上它跨财务、台账、审计（结构耦合高）；改的时候不可能让历史数据渐进迁移，只能一次性同步（时间耦合高）；牵扯三个部门排期（组织耦合高）。所以它的影响半径，注定很大。

### 12.4.3 第二层：修改的信息成本

有些决策修改的成本，不在”动手改”，而在”重新搞清楚应该改成什么样”。

这一层有两个来源：

**第一，决策的知识深度**。当初这个决策是靠一条长推演链做出来的——跨多层、多域分析权衡后才得出的结论。修改它，意味着要**重**

**走一遍推演链**，重新确认所有下游依赖仍然成立。追溯链越长，修改的信息成本越高。

合同的数据模型，当初是法务、财务、业务三方反复对齐后定的：哪几个字段是主键、编号怎么编码、状态枚举怎么划分——每条后面都站着一段业务理由。万辉改的时候，这些理由没有一条被重新审视过。

**第二，决策的隐含上下文**。当初做决策的背景信息没有被记录下来，只存在于当事人的脑子里。当这些人离开、忘记或转岗后，修改一个决策的真实成本是：

> **修改一个决策的真正成本 = 重新发现当初为什么做这个决策的成本 + 实际修改的成本**

大多数时候，真正贵的是前者。万辉不是不知道编号规则改了会乱，他是不知道**当初为什么这么定**——而”不知道为什么要这么定”，才是他敢”今天就能上”的原因。

把决策记下来，压低的正是一个决策的**信息恢复成本**——写下来，后来人才不用靠猜”当初为什么这么定”。（第三幕 §12.4.8 会讲具体怎么记。）

### 12.4.4 第三层：修改的风险

有些决策不是不能改，而是一旦改错了，后果严重到无法接受。

风险也有两个来源：

**第一，爆炸半径**。被修改的东西，是否处于业务关键路径、数据关键路径、安全关键路径的交汇点？

**修改内部管理后台的按钮颜色 → 错了 → 难看，不急**

**修改核心支付流程的状态机 → 错了 → 资金损失，紧急**

**修改用户认证的鉴权模型 → 错了 → 安全漏洞，灾难**

**第二，数据完整性风险**。涉及历史数据迁移的修改，自动升级一个风险等级：

**修改新写入数据的表结构 → 低风险（新的按新结构写）**

**修改已有历史数据的表结构 → 中风险（迁移脚本可能出错、可能丢数据）**

**修改多系统共享的核心数据 → 极高风险（数据迁移 + 多系统同步上线 + 回滚策略复杂）**

到这里，一个规律已经浮出水面——**为什么数据所有权/数据模型，几乎是架构里最不可逆的决策？**因为它在三层框架里全都占满：影响半径大（数据被多个系统依赖）、信息成本高（当初为什么这么分域，只有少数人知道）、修改风险极高（历史数据迁移，一步错全盘错）。

这就是为什么第 10 章把”数据所有权”立成一条管控原则——它几乎最不可逆，这一章给那个诊断补上了完整的论证。

### 12.4.5 综合判断：一张矩阵，一眼定级

把三层叠起来，一张判断矩阵就可以落地了：

| 决策示例 | 影响半径 | 信息成本 | 修改风险 | 综合判定 |
| --- | --- | --- | --- | --- |
| 服务边界划分 | 大（影响团队组织+接口契约） | 高（涉及业务域理解） | 高（数据迁移） | **高不可逆** |
| 对外接口契约 | 大（外部消费者不可控） | 中 | 高（破坏外部集成） | **高不可逆** |
| 数据所有权归属 | 大 | 高 | 极高（数据迁移） | **极高不可逆** |
| 架构模式（微服务 vs 单体） | 极大 | 高 | 极大 | **几乎不可逆** |
| 数据库选型 | 大 | 中 | 中（有迁移工具但成本高） | **中-高不可逆** |
| 编程语言选择 | 大（全代码库） | 低（社区标准明确） | 中（重写成本高但可渐进） | **中不可逆** |
| 内部工具库选型 | 中 | 低 | 低（可逐步替换） | **低不可逆** |
| 日志格式 | 小 | 低 | 低 | **高度可逆** |

这张矩阵的用法，不是把每个决策都拿去做完整评估——那是浪费。

它是一把**分诊尺**：先粗判一个决策落在哪一档，再决定给它什么待遇。

高不可逆的，认真做、多人审、记录在案；低不可逆的，快速决定、错了再改、不设关卡。

### 概念卡片 · 不可逆性

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 修改成本高到理性的决策者宁愿忍受现状、也不掏钱去改的程度 |
| **它不是什么** | ⑥⑦ | 对立：可逆性——可逆决策改得起（一次提交、局部影响），不可逆决策改不起（数据迁移、接口重定、团队重组）；辨析：不可逆 ≠ 技术上不能改——几乎任何软件决策技术上都能改；“不可逆”是经济属性，不是技术属性 |
| **它从哪来** | ②③⑨ | 从经济学与决策理论借来——经济学很早就发现：有些决策一旦做出，撤销成本随时间上升，越晚发现选错、改错成本越高；金融学的“期权价值” |
| **它到哪去** | ④⑤⑧ | 能翻译“径大以策错（为何留选择权定义何时该行动和修改符合）是理解”什么时 |

### 12.4.6 完整演练：把三层框架套到”编号规则”上

理论说完了，用万辉的案例完整走一遍，让三层框架不再是抽象名词。

**第一层，影响半径**。万辉改”编号规则”，需要联动修改的东西清单：代码里引用编号的 12 处（字段层面）；台账系统的编号解析（结构耦合）；财务对账口径、审计归档口径（结构耦合）；三个部门同步排期（组织耦合）；历史数据必须一次性重排，无法渐进（时间耦合）。**半径：大**。

**第二层，信息成本**。当初编号为什么这么编码？——法务、财务、业务三方协商的结果，没人写过文档（隐含上下文）；这 12 处引用各自承载什么业务语义，没人能一次说清（知识深度）。**信息成本：高**。

**第三层，修改风险**。合同编号处于对账关键路径和审计关键路径的交汇点（爆炸半径大）；要动历史合同数据，重排即可能破坏审计链（数据完整性风险极高）。**风险：极高**。

三层合计，综合判定落在”**高-极高不可逆**”。对照第三幕 §12.4.9 的待遇表：应该”完整审查 + 至少一人独立审查 + 决策记录”。而万辉实际给它的待遇是：零审查、零记录、“今天就能上”。**差距，就是那场三个月的连环爆炸**。

这一节想说明的不只是”万辉错了”，而是这把尺子的可操作性：任何一个人，拿分诊三问（联动改多少？信息成本高吗？风险多大？）过一遍，一分钟就能给一个决策分诊。**分诊不需要完美——只要能把”高不可逆”从”低可逆”里挑出来，就已经赢了九成**。（分诊三问与改字段前三问共享第一个筛选问句”联动改多少”——一个筛”是不是架构决策”，一个量”不可逆到什么程度”。）

注意别把三组问句混起来：**改字段前三问**（第一幕）回答”这是不是架构决策”，**分诊三问**回答”这个决策多不可逆”，**决策记录六问**（§12.4.8）回答”这个决策怎么留档”——识别、判断、记录，各管一段。

### 12.4.7 抉择：什么时候做——定节奏与显式推迟

前面给了”不可逆性”这把尺子。现在用它来回答那个最实际的问题：

**面对一个决策，到底什么时候做?**

直觉上，“越早做越好”——架构决策早做，不是成本低吗? 这个直觉对了一半，另一半是反的。架构决策的时间策略，不是一刀切”早做”，而是**按不可逆性分级**：

| 决策类型 | 特征 | 操作 |
| --- | --- | --- |
| **高不可逆** | 影响半径大 + 信息成本高 + 修改风险高 | **早做**，充分分析，多人审查 |
| **中不可逆** | 影响半径中等 | 做初步分析，预留调整空间 |
| **低不可逆** | 修改成本低，可随时改对 | **晚做**，快速决定，错了再改 |

注意最后一行的”晚做”——它不是偷懒，是一条有名字的策略。架构经济学里有一个概念叫**期权的价值**（option value）：**一个决策如果现在信息不够，推迟它本身就是一种价值——等到信息充分时再做，比现在瞎猜然后返工要便宜得多**。

**提前决定：服务边界（因为改这个很贵）**

**推迟决定：具体缓存策略（因为现在信息不够，改这个也不贵）**

**必须早做：数据所有权归哪个域（因为一旦错了，数据迁移成本极高）**

这就是”提前投资”这四个字的准确含义。它不是说”所有架构决策都要尽早做死”——恰恰相反，**好的架构投资，是把有限的深度分析资源，集中投到真正不可逆的决策上**——上面三类决策，各自一种节奏。

每个架构决策，操作上都先过两关：**先过目的关**——它守住的是不是对目的必要的不变量（判别问句见 §11.3.6；不是，就不配被锁进架构）；**再过代价关**——按不可逆性排优先级与时机（上面这张表）。目的关决定”值不值得当架构”，代价关决定”什么时候做、做多重”——两关的次序，正是 §12.3.1 那句”先过目的关、再过代价关”。

再补一层**目的与目标的对账**——这两样在决策里的待遇不一样：**目的变更触发重审**（下一节 §12.4.8 的复审条件第一条）；**目标不可砍——目标砍了，关切就断供**。

要改目的，须**回定义期**并**过闸门**（第 15 章 §15.3.5 闸门甲／乙）。

把”推迟”从一种无奈，变成一种操作，是架构决策里最反直觉的一招。

多数时候，“推迟”给人的印象是坏的——拖延、不做决定、踢皮球。但架构语境里的”推迟”，是**显式地、有意识地、记录在案地”决定现在不做决定”**。两者的差别在于：

- 被动的拖延：**不做**决策，也不记录，等事故来逼；
- 显式的推迟：**做**一个”现在不做”的决策，记录下”等什么信息、什么时候再评估”，然后主动去收集那个信息。

这在高频迭代的场景里尤其重要。假设团队收到一个小变更，标准流是”快速分析 → 实施”。嵌入不可逆性预判之后，变成：

```
收到一个变更 → 30 秒不可逆性预判 →
    低不可逆 → 快速实施
```

**高不可逆但信息不足 → 显式推迟：标记"需完整设计推演"，升级到更重的分析路径**

30 秒的预判，能把一个”高不可逆但被当成小变更”的改动拦下来——就像董勋拦下那个”直连接口”一样。而”显式推迟”之所以是决策而不是逃避，因为它回答了三个问题：**现在为什么不做？等什么信息？什么时候再评估？** 回答不了这三个问题，才是真拖延。

**架构师的核心能力，不是做出正确的决策，而是识别哪些决策如果不正确，后果很严重**。这一章给它补上后半句：识别出”后果严重”之后，**对信息不足的高风险决策主动推迟、对低风险决策不投入、对高风险决策足额投入——这本身就是最好的架构决策**。

这条能力，正是第 18 章架构师能力清单的第四项增量——判断不可逆性。

### 12.4.8 记录：决策记录六问，把决策变成资产

讲了半天”主动做”“显式推迟”“看清代价”，可这些东西做没做、怎么验证？答案是一个极其朴素的动作——**把决策记下来**。

架构决策的记录，有一套公认的格式——**六问为纲，来源与弃项为补**：

| 问题 | 回答什么 |
| --- | --- |
| **决策名称** | 这是一次什么决策? |
| **背景** | 当初为什么必须做这个决策? 面对什么问题? **哪个利益相关方、持哪条关切?** |
| **来源** | 本决策**上承哪条关切**（在关切台账里指得出那一行)? |
| **选项** | 考虑过哪几个方案? 各有什么好处和代价? |
| **选择** | 选了哪个？为什么? |
| **影响** | 选完之后，哪些后续工作被它决定了? |
| **弃项** | **舍掉了哪条关切、哪个备选**? 为什么不是它? |
| **不可逆性** | 这个决策现在反悔，要付出多大代价? |
| **批核与例外** | **谁批的**?有无例外、例外如何登记、限期何时复审? |

> 术语注：‘决策记录六问’是本书在 ADR（架构决策记录）惯例基础上的**凝练**——Nygard 原版字段为 标题/状态/背景/决定/后果；本书省去”状态”、补入”选项”与”不可逆性”两问，

把权衡过程与可逆性对账直接写进记录。六问不是 ADR 原版字段，是本书的格式，欢迎检验、欢迎校准。
表里”来源”“弃项”“批核与例外”三栏，是把 ADR 接回关切台账与治理：42010:2022 **§6.10 条款名＝Recording of architecture decisions and rationale**（记录架构决策与理据，**已核逐字**）；“谁批的”是**治理机构**，不是架构师；例外要**留痕**（理由、范围、期限、批准人）。
该条正文一手逐字（§6.10.1）：

> An AD shall record architecture decisions considered essential to the architecture of the entity of interest within the scope and intended purpose of the AD. 架构描述应记录在 AD 范围与既定目的之内、被认为对该被关注实体（entity of interest）的架构必不可少的架构决策。

正是这条条款把”弃项”栏立成硬要求——上引该条第二句 The AD should record alternative decisions considered and rejected and the rationale for those choices.（AD 宜记录已考虑并否决的备选决策及其取舍理据）：被否备选的”死因”就是理据，也正是评审对质三问的第三问。

> 栏数说明：核心仍是**六问**（名称／背景／选项／选择／影响／不可逆性），“来源”“弃项”“批核与例外”是接台账与治理的**补栏**，不改变六问的骨干。

记”不可逆性”之前，先过目的判据：这个决策守住的是不是对目的必要的不变量（判别问句：这东西变了，系统还能不能达成它的目的，见 §11.3.6）——不是，则不配被锁进架构；是，再用三层框架量代价。六问的”不可逆性”栏，记的是代价关那一档。

用恒信”合同编号规则”这个决策来演示一遍——如果万辉当初写下了这么一段，三个月后的连环爆炸，至少会提前被识别：

**决策名称：合同编号规则与电子签章平台对齐**

**背景：电子签章平台要求编号为"合同类型-年份-流水号"，**

**与系统现有编号规则不一致，法务提出对齐需求。**

**利益相关方与关切：法务（可追溯）、财务（月底对账）、**

**审计（抽查可核）、台账对接方（解析口径稳定）。**

**来源：上承关切台账"审计抽查可核"一行（黄程提出）。**

选项:

**A．全局重排编号，历史合同同步迁移**

**好处：编号统一，下游口径一致**

**代价：历史数据迁移，与纸质归档对照会乱，审计风险高**

**B．新旧编号双轨，历史合同保留原编号 ← 选了它**

好处：不动历史数据，审计可追溯，下游渐进适配

代价：一段时间内两套编号并存，报表需兼容

选择：B。因为历史合同已与纸质归档绑定，全局重排的审计与追溯风险，远大于双轨并存的一段过渡成本。

弃项：舍掉 A（舍"编号统一"这条关切，代价是报表需兼容两套口径）；舍掉"先不动、等电子签章平台改规则"。

影响：台账/报表需兼容两套编号；电子签章对接按新编号；历史合同的查询与追溯维持原口径；三个月后评估是否并轨。

不可逆性：中-高。已归档历史数据的口径不可逆（重排即破坏审计链）；新编号规则一旦被下游长期依赖，再改成本上升。

批核与例外：双轨并存属对"编号唯一"惯例的例外，由治理机构批，登记理由（审计风险）、范围（历史合同）、期限（三个月），到期复审判并轨或续期。

注意这段记录的价值——它不在当下。当下万辉当然知道为什么选 B；它的价值在一年后：换个人接手，看到”不可逆性：中-高”和那句”重排即破坏审计链”，就不会再犯”全局重排”的错。

**决策记录是把一次决策，从”当事人脑子里的知识”变成”组织的资产”**。§10.3.1 说架构必须被表达出来，才能被共享、被讨论、被

检查、被传承；决策记录，就是架构决策的”表达”——被写下来的决策，才可能被共享、被检查、被传承。

记住它和识别用的改字段前三问不同——前三问答”是不是”，六问答”怎么留档”。别撞名，也别忘了各管一段。

**决策记录与视角一样，是资产；具体方案不是**。规则很直白：**视角是资产、视图是产物**——决策记录与视角可入库、可跨项目复用，具体方案不可复用（第 15 章 §15.4.1）。

六问记录之外，还有一个配套动作——**在架构的关键节点上标注不可逆性**：

**合同数据模型（合同主表 / 审批记录 / 附件）：**

**不可逆性：极高**

**理由：数据所有权归合同域，财务 / 台账 / 审计 / 电子签章均依赖**

**修改代价：历史数据迁移 + 4 个下游系统适配 + 审计口径重建**

标注的价值，和上面六问记录是同一件事——价值都在**一年后换人接手**时兑现（论证以六问那段为母本，这里不再展开）；落到具体节点上的作用，是把第 10 章案例进展③里水忠定下的”老规矩”变成一块自带警示牌：换人看到”不可逆性：极高”和”理由”，就知道不要轻易动。

ADR 里还应记两样，常被漏：**被否掉的备选死因**——它回答”为什么不是它”，是评审对质三问的第三问（第 8 章），也正是上表”弃项”栏；**复审条件**——写下”什么情况出现就必须重审这条决策”（如新增数据源、权限模型变更），过复审条件不重审，这条 ADR 就过期了。

复审条件里最高的一条是**目的变更 / 目的漂移**——系统目的变了，fundamental 的判据本身换了，架构必须重审：**目的变更**（战略调整、产品定位改变，显式事件）是强制触发器；**目的漂移**（目的悄

然变化而架构未跟随）靠以目的为参照的定期检视（两类扳机的分工见第 14 章）。

两者都与架构漂移方向相反（架构漂移=实现偏离架构，架构对、执行歪；目的漂移=目的偏离架构，执行对、判据过期）。

示例”新增数据源、权限模型变更”之外，第一条就写”目的变更”。复审条件建议作六问之外的可选第七项，不破坏六问框架。

**目的与目标的对账，在这一节收一次口：目的变更触发重审（复审条件第一条）；目标不可砍——目标砍了，关切就断供；要改目的，须回定义期、过闸门**（第 15 章 §15.3.5 闸门甲／乙）。

### 12.4.9 分级：把审查资源花在刀刃上

记录之外，还有一个更硬的落地机制——**审查深度按不可逆性分级**。

不是所有架构决策都值得同样的审查投入。给一个低不可逆的决策开三场评审会，是浪费；给一个高不可逆的决策走”自查”，是灾难。分级原则是：

| 不可逆性 | 最低审查深度 | 审查参与者 |
|---|---|---|
| **极高** | 完整审查（语义 + 影响） | 多人交叉审查 |
| **高** | 完整审查 | 至少一人独立审查 |
| **中** | 语义符合性 | 自查 + 同行复核 |
| **低** | 快速自检 | 自查即可 |

这条机制和一个更朴素的原则是同一件事：**把有限的审查资源，集中在真正不可逆的决策上，而不是均匀撒在所有决策上**。万辉最大的问题，不是没审查——是他根本没意识到编号规则属于”高不可逆”，于是它连”自查”都没走，直接”今天就能上”。

回想第 8 章的三个看门人：评审、验证、确认是把关的动作；这一节的按级审查，是把关动作在”架构决策”上的具体配置——先分诊，再派尺子。

## 案例进展③｜恒信：决策被写下来之后

编号风波过去一个月，恒信来了一个新需求：合同要支持”电子签章 + 线下签章”混合模式，法务要求在合同上加一个”签章方式”字段。

这次，万辉没有直接动手。他先做了一分钟的三问：“改了它，要联动改多少东西？——财务对账、台账、审计、电子签章对接，四五处。定了之后，多少决定建立在它上面？——合同状态流转、审批链、归档规则。改错了改回来代价大吗？——涉及已签合同的历史数据。”三问里两问半答”大”。

他停下来，填了一份六问决策记录，把”签章方式”放进了”选项”栏里讨论，而不是直接当作”字段”加进去。评审会上，他把”背景、选项、选择、影响、不可逆性”讲了一遍，黄程当场提出：“混合模式下，‘电子签章的合同’和’线下签章的合同’，归档口径得分开，审计才好查。”——万辉把这条补进了”影响”栏。

玉杰在旁边听了全程，补了一句：“以前你们改字段，我负责的是最难看的那段——数据不跟代码一起回滚，炸了也只能硬扛。这次’影响’栏写得清楚，归档口径、对账兼容都点名了，我照着部署，心里有数。”

会后，徐漾路过万辉的工位，看到那份记录，说了句：“你现在把’改字段’当成’做决策’了。三个月前你要是有这一步，编号那次炸不了。”

万辉：“那次之后我才明白——**不是所有改动都值得这么隆重，但你不先判断它值不值得，就没有资格说’它不值得’**。”

### 12.4.10 定位：设计链各层的架构决策

讲到这里，说的都是”一个决策”——怎么判断、怎么做、怎么记录。现在把镜头拉远：**从一条完整的设计链看，每一层的架构决策为什么长得不一样？**

> 与第 11 章对账：§11.3.3 从需求粒度把链切成”需求工程→架构设计→详细设计→实现”；这里从承接对象切成”需求组织→业务流程→系统功能→产品构件”。两种切法互为投影——一个看每层”吃什么”（粒度），一个看每层”谁承接、做什么决策”（对象）。

第 4 章讲过需求怎么被开发出来，第 6 章讲过设计怎么落地成一棵”树”。把它们连起来看：从”零散的需求”到”一个能用的系统”，中间隔着几层工作。每层的工作，都是在把上一层的东西，翻译成下一层能用的东西。而翻译的过程中，必然要做决策。一个做医药系统的朋友曾经这样类比，记忆深刻：

| 层 | 类比 | 在做什么决策 |
| --- | --- | --- |
| 需求原始表述（第 0 层·输入边界，不决策） | 患者说”我头疼、吃不下饭、睡不好” | 不决策，只是表述 |
| 需求组织 | 医生分诊：头疼→神经内科，吃不下饭→消化内科 | 把散乱的自述归类到科室 |
| 业务流程 | 神经内科医生：偏头痛还是紧张性头痛？持续多久？ | 把笼统主诉拆成可诊断的症状单元 |
| 系统功能 | 确诊并开检查单：偏头痛，做脑电图+MRI | 把症状转化为可检验的医学判断 |
| 产品构件 | 开药方：普萘洛尔 40mg 每天两次 | 把判断转化为可执行的处方 |

各层的决策所以不同，是因为**每一层面对的需求抽象层次不同、向下传递的对象不同**。

有一个贯穿各层的共性，值得先钉死：**每一层的架构决策，本质上都在回答同一件事的两半——“这一层的实体，它的基本概念或属性是什么、管住它实现与演化的管控原则是什么？”**（42010:2022 §3.2 只给这两样。）只不过每一层的**被架实体**不同：

| 层 | 被架实体 | 该层的架构决策问题（定义句两半） |
| --- | --- | --- |
| 需求组织 | 需求体系 | 按什么维度分类才承重（承重／约束／偏好／交付）？哪些分类规矩要立成管控原则？ |
| 业务流程 | 业务流程 | 业务行为单元按什么范式划分？哪些规矩管住它的实现与演化？ |
| 系统功能 | 系统功能框架 | 功能框架在概念或属性上是什么？功能之间怎么归并属实现方式 |
| 产品构件 | 产品构件 | 构件这个实体在概念或属性上是什么？边界与构件间交互是设计定义的产出 |

> 一句话把这条共性钉准：**架构决策回答的是”这一层的实体是什么、守什么规矩”，不是”它由哪些元素组成、元素之间怎么连”**——后者是设计定义的产出（15288:2023 §3.13）。四层里前两层常常整层都是架构工作；后两层则要分开看：**这一层**

> **的架构**与**这一层的设计**是两类工作，不是粗细两版（第 16 章 §16.2.1）。

这张表还可以换一种读法——**判据传导链**。每一层的”目的”，由上一层为它定义（本层要承接的使命）；本层判定”什么是根本”（fundamental＝对目的必要且充分）的标尺，就是这一层的这个目的。

这条”必要且充分”是二**手派生表述**（IEV 831-01-01 Note 2），只提供这一读法，不是准入判据的条款（见第 10、11 章）。

于是每一层的架构决策，都是拿”上一层的目的”当 fundamental 标尺做出来的。**断链处——本层做决策时够不到上一层的目的是什么——就是过度工程（为不服务于上层目的的能力提前建能力）与欠工程（上层目的要求的能力本层没接住）的发病点**。

判据传导链不只在这一条链里成立——它向上还能接到战略对齐（业务战略→产品战略→架构战略→技术实现，见 §18.4.5）：每一层的目的，同样是下一层的 fundamental 标尺。

判据传导链还带出**层级关**：判定在**本层被架实体**（entity of interest）这一级上进行——本层只在本层的被架实体上裁 fundamental，被架实体内部的细节由下一层的被架实体去裁，本层不越级。

用恒信的合同审批系统把这张表填满，你会看到每一层的架构决策都活生生：

- **需求组织层**——“合同审批”的需求怎么组织（按哪几个**关注面**切：承重／约束／偏好／交付?）

- **业务流程层**——审批流程这个实体在概念上是什么（业务行为单元按什么范式划分）；“发起→初审→校验→终审→归档”拆到什么颗粒度，是它的设计
- **系统功能层**——“可会签”“全程留痕”是功能框架的本质还是可选特性（承重测试）；审批、会签、抄送、台账归并成几个功能，是它的设计
- **产品构件层**——审批引擎作为实体，它的基本概念或属性是什么；它和电子签章对接、台账之间的边界划在哪，是设计定义的产出

### 12.4.11 需求有架构吗？

一个经常把工程师绕晕的问题：**需求不是回答”要什么”吗？架构不是回答”怎么做”吗？需求有架构吗？**

分两层回答。上面的”零散需求清单”这一层，**需求本身没有架构**——刚收集上来的表述散乱、重叠甚至矛盾，它就是一张待整理的清单。

再往下一层：**需求的方案，本身就是需求侧的一份架构描述**（AD）——它按**关注面**切（不是按部门／模块切），由该侧视角治理。

这里有两层之分，别混：**架构与系统元素不在同一层**——架构没有形态、交不出去，写它的架构描述（AD）才是交得出去的具体物；而”组成它的那些元素”在实现层。

需求的方案是后者的一份——可签收、可入库、可版本化。

> 术语注：**旧名”架构构成物”已废止**（2026-10-11 全库改称「实体构成」，「架构层」改称「实体侧」）——它容易与下面的”架构

> 实体”混成一个。标准里的”架构实体（architecture entity）“从来不是”架构的构成物”，它指**被架构的那个事物**——即 42010:2022 的”被关注实体（entity of interest）“。**架构实体与系统元素各是一个概念，不是一回事**。
> 15288:2023 §5.9 一手逐字：
>
> > Note that an architecture entity (an entity being architected or one subject to the system architecture definition process) and a system element (a discrete part of a system that fulfils specified requirements) represent two different notions. 注意，架构实体（被架构的实体，或受系统架构定义过程作用的实体）与系统元素（系统中满足规定要求的离散部分）代表两个不同的概念。
>
> 42020:2019 §3.6 给”架构实体”的定义同此义——thing being considered, described, discussed, studied or otherwise addressed during the architecting effort（架构工作中被考虑、描述、讨论、研究或以其他方式处置的那个事物）。

所以”需求有架构吗”的答案分两句：**需求本身没有架构；需求的方案有架构，而且是需求侧的一份架构描述**。

**切法只有一条硬规矩：按关注面切，不按模块或功能切**。关注面是”谁在乎哪一面”（承重、约束、偏好、交付四类分流）；按模块切会让视图退化为模块说明书（30 号 §4.6）。

它里面的架构决策——按哪几个关注面分类、每条需求写到什么粒度、冲突的需求怎么处理——**不决定”系统怎么做”，但决定了”系统设计从哪里出发、按什么逻辑组织、怎么验证说全了没有”**。这些决策的影响贯穿整条设计链——但它们仍是目的判据与代价判据合判的产物，不因”贯穿全链”就自动进架构。

这一章不止一次提到”需求怎么组织”的决策。它是设计链上第一个架构决策，也是几乎所有下游决策的起点——需求分类分错了，后面每一层都在错的框架里打转。第 4 章说”需求是开发出来的”，这一章给它加一个注脚：**需求不只是被开发出来的，还是被”架构”出来的——组织需求的方式，本身就是一次架构决策**。

> 与第 10 章对账：§10.4.4 的结论是”**一个所关注实体只有一个架构**”，概念／逻辑／物理只是同一个架构的**描述层次**。本节说的”需求侧的架构描述”，指的是**另一个被架实体（需求体系）的架构描述**，不是同一个架构的又一层——两者不矛盾。
>
> 术语注（待核）：本书语境下的”需求架构”指**需求侧的架构描述**，不是与系统架构并列的另一层架构。本条与仓库数据文件模板的两档命名（`02-相关方需求架构定义`／`05-系统需求架构定义`）**口径尚未统一，属待核**；本次不擅自改模板命名。

### 12.4.12 链的两端，承受最高风险

把各层决策摆在一起，你会看到一张非常对称的图景——**这条链的两端，不可逆性都极高；而中间两层，反而是修改成本最低、最能吸收不确定性的地方**。

| 层 | 在做什么决策 | 改错后果 | 不可逆性 |
| --- | --- | --- | --- |
| 需求组织 | 需求怎么分类、怎么组织 | 整条链从这里出发——根歪了，整棵树歪 | **极高** |
| 业务流程 | 业务怎么组织、产出什么业务行为 | 流程结构不合理、颗粒度失当 | 中-高 |
| 系统功能 | 系统功能怎么组织 | 功能混乱、质量关注点遗漏 | 中 |
| 产品构件 | 构件边界和契约 | 系统无法稳定演化——数据迁移、接口重定、团队重组 | **极高** |

为什么两端最高、中间缓冲？逻辑很直白：

- **需求组织层（链的起点）**：它定义了整条链的出发点和完备性判据——根歪了，后面全歪。改它虽然不碰代码，但要重做下游所有层；
- **产品构件层（链的终点）**：它是最终落地的东西——改构件边界，不是改文档，是改代码 + 数据 + 团队。数据库要拆、接口要重定义、团队要重新划分。这是量级的差异；

- **中间两层**：改它们的成本在”文档层面”，它们的作用正是**缓冲和吸收不确定性**——在文档层面把问题充分暴露和解决，避免带到代码层面。

这个结构说明了一件事：**设计链上的分析工作，不是为分析而分析。它的价值，就是在”产品构件”这最后动刀之前，把问题在便宜的层面充分暴露和解决。** 需求组织的决策贵（影响全链）、产品构件的决策更贵（落地即资产）；中间的每一层分析，都是在用”文档的代价”换”代码的代价”。

万辉的编号规则之所以炸，换到这条链上说，是他跳过了前面所有层的分析，直接在”产品构件”这个最贵的层面动了刀——连”需求组织”这一层都没走。**他把一条本应该在链条上游便宜地解决的决策，拖到了链条下游最贵的地方才被迫解决。**

## 案例进展④｜恒信：把决策放回它该在的层

万辉后来养成了一个习惯：任何一个变更进来，先问一句”这个变更，落在哪一层?”

有一次，业务方提了一个需求：“合同审批要支持’多人会签’，三个人以上联合审批。”万辉没有再直接改审批流代码，而是先看它落在哪一层：

- 落在**需求组织层**吗？——不是。这不是”需求怎么分类”的问题，是需求内容变了；
- 落在**业务流程层**吗？——是。审批流从”单人审批”变成”多人会签”，是业务流程结构的改变，是业务流程层的架构决策；

- 落在**系统功能层**吗？——一半。审批引擎要支持会签节点，是功能能力的变化；
- 落在**产品构件层**吗？——不是。不需要动构件边界，审批引擎还是那个引擎。

“多人会签”于是先在业务流程层被认真设计——流程怎么定义会签节点、会签和单人审批怎么共存、历史单人审批记录怎么兼容。分析完了，才落到功能层和构件层去实施。

徐漾在旁边看着，评价了一句：“你以前改一个需求，直接从’需求’跳到’改代码’，中间两层你根本没走。现在你知道先把决策放到它该在的层——**在便宜的层做对的决策，贵的层才不用付返工的学费**。”

万辉把这句话记了下来。他后来在团队里立了个规矩：每个变更，先填一行”这个变更落在哪一层、要动的是哪一类决策”——不是走形式，是逼自己在动手前，先把决策分诊清楚。

---

## 认知反转

回到章首。万辉把”合同编号规则”当成”改个字段”，当天改完当天上线，三个月后连环爆炸。现在我们知道了，那场爆炸，不是一次”改字段失误”，它连错了四层，一层比一层深。

**第一层，他把”架构决策”当成了”实现决策”**。数据模型——合同编号怎么编码、状态枚举怎么划分——是这套系统里不可逆性最高的决策之一，三标志全中：影响半径大、持续时间长、修改成本高。他却用”改个字段”的方式处理了它：不识别、不评审、不记录，直接”

今天就能上”。**没有被识别出来的架构决策，不会得到它应有的审查，却依然承担架构决策的全部代价**。

**第二层，他不知道架构决策不可避免**。他以为”这次改动不需要做架构”是一个选项——错了。数据模型的决策他早晚要做：不是在那次”改字段”时做，就是在三个月后那场事故里做。**“我们没做架构决策”从来不是一个选项，你只是在选择”在成本低的时候做”还是”在成本高的时候做”**。他选择了后者，于是多付了三个月的账。

**第三层，他没有一把判断不可逆性的尺子**。三层框架——影响半径、信息成本、修改风险——没有一样是他改字段前会想的。如果他先问分诊三问：“联动改多少东西？信息成本高吗？风险多大？”——他会发现答案全是”高”，然后停下来，把这次改动当成架构决策来对待。

**第四层，也是最深的一层，他错过的不是一次决策，是一整条链**。从需求组织到产品构件，每一层都有它该做的架构决策；而万辉的做法，是跳过中间所有层，直接在”产品构件”这个最贵的层面动刀。**在便宜的层做对的决策，贵的层才不用付返工的学费**——这句话，是这一章所有论证的收束。

这一章的四次反转，其实是同一个反转的四种说法：

> **架构决策不是评审会上的事，它藏在每一个”改个字段”里；它不可避免——不是做不做，是早晚；而不可逆的决策，必须在成本低的时候、用对的方式提前做**。

用第 1 章的话收一次束：对”架构决策”，你原来只是”见过”——背得出定义、知道”提前投资”这个说法；这一章把它推到”拥有”——

能答全四问（它是什么、不是什么、从哪来、往哪去），能用改字段前三问识别它、用三层框架判断它、用决策记录六问留下它。**能答全四问才算拥有——你能用改字段前三问，认出不可逆的决策**。

回看第 10 章的结尾。第 10 章说：架构是实体在其环境中的基本概念或属性，加上管住它怎么实现、怎么演化的管控原则；而”基本”的判据只有一条——目的。这一章换一把尺子看同一批内容——**哪些最不能反悔**：识别不可逆（三标志）、判断不可逆到什么程度（三层框架）、在正确的时机用正确的方式做（按不可逆性分级、显式推迟、决策记录六问）、放到正确的层面做（设计链各层）。

**第 10 章回答”什么算架构”，这一章回答”架构怎么被决定”**。

而”提前投资”这个词，给下一章埋下了钩子。投资一旦做出，就需要被管理：一个决策做出之后，系统一直在变，谁来保证”当初那个决定”不被悄悄改掉、不被版本搞乱、不会被遗忘？

这就要说到**配置管理**——基线把”已经批准的决定”冻结成参照，版本记录每个决策的演进，变更控制守住基线的门。第 10 章用**承重测试**找出了”不可还原的涵义”（目的关）；这一章换一把尺子——**不可逆性**（代价关）——看这些内容在变更里各自落在哪一档。两把尺子不是同一把（§12.2.2 已对过账）。下一章，我们要让它在变化中”说得清”。

用第 1 章 §1.4.4 的尺子量一下：这一章把**第 ③ 层（心智模型）**推到了”决策”这一格——**不可逆性就是那个能推演”现在不定、以后要多贵”的变量**。

## 小结

这一章把”架构决策”这个词钉死，并给了它一把可操作的尺子。**架构决策**是”选错了要花大代价才能改对”的选择，用三标志辨认——影响半径大、持续时间长、修改成本高；它藏在每一个”改个字段”里，识别它（改字段前三问），是正确对待它的第一步。

第二幕磨掉三种错觉：**不可逆性是经济属性**不是技术属性；**架构决策不可避免**——不是做不做的问题，是”成本低时主动做”还是”成本高时被动做”的问题，“我们没做架构决策”是伪选项；**被动做不是没得选**——它只是没有显式的架构。

第三幕把尺子用起来。**不可逆性**用三层框架判断——影响半径（结构/时间/组织耦合）、信息成本（知识深度/隐含上下文）、修改风险（爆炸半径/数据完整性）；数据所有权/数据模型在其中几乎最不可逆。

“提前投资”的意思不是”所有决策都早做”，而是**先过目的关、再过代价关**：什么是 fundamental 由目的单独裁决（第 10/11 章），代价判据只是**代理**，只在这组根本之内按不可逆性决定节奏。

节奏分三档：高不可逆早做、低不可逆晚做（保留期权价值）、信息不足的高风险决策显式推迟。决策要用六问记录下来（名称/背景/选项/选择/影响/不可逆性），另补来源、弃项与批核三栏；复审条件第一条写”目的变更”，审查深度按不可逆性分级。

最后，在整条设计链上，每一层的架构决策都不同——需求组织（链起点）和产品构件（链终点）承受最高不可逆性，中间层的作用是缓冲和吸收不确定性。

**需求本身没有架构；需求的方案，本身就是需求侧的一份架构描述——按关注面切，不按模块切。**

对”架构决策”这四个字，你从”见过”走到了”拥有”：能答全四问，能用一条判断链（识别→判断→抉择→记录→分级→定位）走完任何一次改动。

到这里，把一条需求变成决定的路，走完了——从一棵树，到一次架构决策，每一站都有它的尺子。可决定做出来，不是终点：系统还在变，得有人守着它。

下一章，我们讲”决定做出来之后怎么被守护”。架构决策一旦做出，系统还在变，谁来保证”当初那个决定”不被悄悄改掉？**配置管理**——用基线冻结、用版本追踪、用变更控制守护。第 10 章用承重测试找到”不可还原的涵义”（目的关），这一章用不可逆性（代价关）把它排开，下一章，让它”说得清”。

追溯平台的接口清单上，多了一条：受控导出。以前他以为数据所有权就是”归对域、不许改字段”——归位就完事；这一章之后他知道，所有权是一条线，导出外给也动它——数据所有权不可逆，接口不是想开就开。

---

## 本章概念总图

**架构决策 = 选错了要花大代价才能改对的选择**

**三标志：影响半径大 · 持续时间长 · 修改成本高**

**识别：改字段前三问（联动多少？定后多少决定建立在它上？改错代价多大？）**

**反例：改个变量名（可逆、局部、改得起）—不是架构决策**

三种错觉（磨边界弧）：

错觉一：不可逆不是技术属性，是经济属性——"能改"和"值得改"是两回事（代价判据从属于目的判据：先过目的关再过代价关）

错觉二：不可避免不是可做可不做——盖楼思想实验：不是做不做，是早晚

错觉三：被动做不是没得选——被动做 = 没有显式的架构；曲线：前期一小时买后期一个月

不可逆性：三层判断框架

影响半径（结构/时间/组织耦合）→ 信息成本（知识深度/隐含上下文）→ 修改风险（爆炸半径/数据完整性）

最不可逆：数据所有权 / 数据模型 / 服务边界 / 对外接口契约

矩阵分诊 → 完整演练；分诊三问 = 判断不可逆程度（区别于识别的改字段前三问）

判断链：识别 → 判断 → 抉择 → 记录 → 分级 → 定位（识别属第一幕、分级并入第三幕）

抉择：先过目的关（是不是根本）再按不可逆性定节奏（高不可逆早做 · 低不可逆晚做 · 期权价值 · 显式推迟 = 记录"现在不做"）

记录：决策记录六问（名称/背景/选项/选择/影响/不可逆性）+ 来源（上承哪条关切）/弃项/批核与例外 + 不可逆性标注 + 复审条件（目的变更=重审触发器）；视角是资产、视图是产物——记录可复用，具体方案不可

分级：审查深度按不可逆性分级，资源集中在刀刃上

定位：设计链各层——需求组织（极高）→ 业务流程（中-高）→ 系统功能（中）→ 产品构件（极高）；判据传导链：每层目的=下一层 fundamental 标尺

需求本身没有架构；需求的方案＝需求侧的一份架构描述（按关注面切，不按模块切）

架构（没有形态、交不出去）／实体构成（元素及其关系·实体侧）／实现层（系统元素·具体物）—AD 是交得出去的具体物

（旧名"架构构成物"已废止；标准的"架构实体"＝被架构的那个事物，≠实体构成，与系统元素是两个概念）

## 本章判据

> 十条判据是本书的主张，不是任何标准的条文（“代价从属”关系为本书凝练；三标志与三层框架都只是**代理**，真准入判据是目的必要性）——它们都当场可检验：欢迎检验、欢迎校准。判据 3 的”目的判据锚”（对目的而言必要且充分）系二**手派生表述**（IEV 831-01-01 Note 2）——42010:2022 正本未引 IEV（书目仅列 IEC Electropedia），它不是任何标准的准入判据条款。

1. **架构决策三标志**：这个决定”影响半径大吗 / 持续时间长吗 / 修改成本高吗”——两问以上答”是”，就该按架构决策对待；三问全否的，是实现决策。识别用改字段前三问。**违反即崩：万辉把”合同编号规则”当”改个字段”——三标志全中，却按实现决策”今天就能上”，三个月后连环三爆。**
2. **架构决策不可避免**：“我们没做架构决策”是伪选项——架构决策不是做不做的问题，是”成本低时主动做”还是”成本高时被动

做”的问题。**违反即崩：盖楼”做法 A”看起来没有架构——地基、承重、管线一个也没少做，只是都拖到成本爆炸时才做。**

3. **不可逆性是经济属性（从属代理）**：不可逆 ≠ 技术上不能改；它的含义是修改成本高到理性的人选择忍受。“能改”和”值得改”是两回事。**代价判据从属于目的判据**——什么是 fundamental 由目的单独裁决（第 10/11 章），代价只是**代理**：它只在这组根本之内排优先级与时机，**不产生新成员**；两类失真各有一例——**漏判**（“日志须可被外部审计员独立读取”：不贵却承重）、**误纳**（框架版本、组织惯例技术栈：很贵却与目的无关）。先过目的关、再过代价关，两种错位都是病（§12.3.1）。**违反即崩：拿”技术上能改回去”就说可逆——编号规则技术上当然能改，代价是三个月的账重对、历史数据重排，于是”能改”变成了”没人愿意改”。**
4. **三层判断框架**：改一个决策，从”影响半径（结构/时间/组织耦合）→ 信息成本（知识深度/隐含上下文）→ 修改风险（爆炸半径/数据完整性）“三个维度评估不可逆性；一分钟版用分诊三问。**违反即崩：只量”改起来多贵”一个维度——万辉只看到十二处引用改完，没量信息成本与修改风险，三爆一样不少。**
5. **数据所有权几乎最不可逆**：数据模型/所有权在三层框架里全占满——影响半径大、信息成本高、修改风险极高。改动它，要按最高不可逆对待。**违反即崩：把数据所有权当”加个导出接口”——董勐差点直连客户系统绕过追溯域，三年后追溯出问题没人负责。**

6. **按不可逆性定节奏**：高不可逆→早做、充分分析、多人审查；中不可逆→初步分析、预留调整空间；低不可逆→晚做、快速决定、错了再改（期权的价值）。**违反即崩：一刀切”越早做越好”——把低不可逆的缓存策略也拉去开三场评审会，分析资源被均匀撒掉，真不可逆的决策反而没有分析。**
7. **显式推迟是决策**：信息不足的高风险决策，“决定现在不做、记录等什么信息、何时再评估”——是架构操作，不是拖延。**违反即崩：回答不了”现在为什么不做？等什么信息？何时再评估？“的”再等等”，就是被动拖延——等事故来逼。**
8. **决策记录六问＋来源与弃项**：六问为 名称 / 背景 / 选项 / 选择 / 影响 / 不可逆性，另补三栏——**来源**（上承哪条关切）、**弃项**（舍掉了哪条关切、哪个备选）、**批核与例外**（谁批的、例外如何登记、限期何时复审）——把决策从个人知识变成组织资产，降低未来的信息恢复成本（与识别用的改字段前三问各管一段）。**违反即崩：写不出”背景”和”选项”的记录——当初为什么这么定没人知道，换人接手只能靠猜，万辉”今天就能上”的事故重演。**
9. **审查资源分级**：审查深度按不可逆性分级，把资源集中在真正不可逆的决策上，而不是均匀撒在所有决策上。**违反即崩：给低不可逆的决策开三场评审会是浪费；给高不可逆的决策走”自查”是灾难——万辉连自查都没走，直接”今天就能上”。**
10. **设计链两端最高风险**：需求组织（链起点，极高）和产品构件（链终点，极高）承受最高不可逆性；中间层的作用是缓冲和吸收不

确定性——在便宜的层解决，别拖到贵的层。**违反即崩：跳过中间所有层，直接在”产品构件”这个最贵的层面动刀——万辉连”需求组织”都没走，把本该上游便宜解决的决策拖到下游最贵处才被迫解决。**

## 自查 · 这些说法对吗？

1. “改一个字段，就是改一个字段，跟架构没关系。”——**错**。数据模型、编号规则、状态枚举往往是系统里不可逆性最高的架构决策；“改个字段”可能是改架构。
2. “我们直接写代码，不做架构决策。”——**错**。架构决策不可避免——写第一行代码时它们就在发生；“不做”只是选择在成本高的时候被动做。
3. “这个决策技术上能改回去，所以它是可逆的。”——**错**。不可逆性是经济属性不是技术属性；“能改”和”值得改”是两回事——修改成本高到不想改，就是不可逆。
4. “架构决策越早定越好。”——**不准确**。按不可逆性分级：高不可逆早做，低不可逆晚做（保留期权价值）。一刀切”早做”会浪费分析资源。
5. “推迟决策就是拖延，是坏事。”——**错**。显式的推迟——记录”现在不做、等什么信息、何时再评估”——是架构操作；被动的拖延才是问题。

6. “不可逆的决策就不能动，一动就是灾难。”——**错**。不可逆决策不是不能做，是要在看清代价后用对的方式做（像董勐的受控导出）。
7. “写决策记录是浪费时间，当下我知道为什么这么选。”——**错**。记录的价值在一年后——降低”重新发现为什么”的信息恢复成本，把决策变成组织资产。
8. “需求只是’要什么’，需求没有架构。”——**半对**。需求本身没有架构；但需求的方案，本身就是需求侧的一份架构描述（按关注面切，不按模块切），它含架构决策，是整条链的起点。
9. “所有架构决策都应该走完整评审。”——**错**。审查深度按不可逆性分级——低不可逆的快速自检即可，把审查资源集中在高不可逆的决策上。
10. “判断一个决策的不可逆性，看’改起来多贵’一个维度就够了。”——**错**。三层框架——影响半径（结构/时间/组织耦合）、信息成本（知识深度/隐含上下文）、修改风险（爆炸半径/数据完整性）——三个维度各打高/中/低，综合判定。
11. “数据所有权是 DBA 的事，不是架构决策。”——**错**。数据模型/所有权在三层框架里全占满——影响半径大、信息成本高、修改风险极高——几乎最不可逆，改动它要按最高不可逆对待。

12. “决策记录写了六问就行，不用在关键节点标注不可逆性。”——**错**。标注=等级+理由+修改代价，让”老规矩”落到具体节点时自带警示牌——换人接手看到”不可逆性：极高”就知道不要轻易动它。
13. “越难改的决策越重要，贵的东西就该优先锁死。”——**错**。难改 ≠ 重要：什么算 fundamental 由目的单独裁决，代价只在这组根本之内排优先级与时机——难改但无关紧要的不配当架构；先过目的关、再过代价关（§12.3.1）。
14. “复审条件写’新增数据源、权限模型变更’就够了。”——**错**。第一条该写”目的变更”——系统目的变了（含目的漂移的悄然变化），都是架构重审的触发器（§12.4.8、第 14 章：变更=显式事件强制触发，漂移=以目的为参照定期检视）。

## 练习

1. 挑一个你最近做过的改动，用”三标志”检验：它影响半径大吗？持续时间长吗？修改成本高吗？它是个架构决策，还是个实现决策？
2. 找出你们系统里一个”数据模型级”的决定（主键怎么定、状态枚举怎么划分、字段口径），思考：如果重做，要联动改多少东西？
3. 用”三层判断框架”评估你们系统里一个具体的决策——影响半径、信息成本、修改风险，各打一个高/中/低，得出综合判定。

4. 找出一个”当初不知道为什么要这么定、现在也不敢改”的决定——它大概率是信息成本极高的不可逆决策。把它写下来，补上”背景”。
5. 用六问格式，为你们最近一次重要的架构决策写一份记录。如果你找不到”背景”和”选项”——说明这个决策从没被认真记录过，这本身就是发现。
6. 列出你们系统里五个决策，按不可逆性排序。检查一下：你们投入的审查资源，是跟着这个排序走的吗？
7. “推迟决策”实操：找一个”信息不足但不可逆性高”的决策，写三句话——现在为什么不做？等什么信息？什么时候再评估？
8. 把一个”高不可逆”的决策（数据所有权、服务边界、接口契约），对照审查分级表：你们当前对它投入的审查深度，匹配吗？
9. 从需求到实现，画出你们这条链（不需要四层，三层也行），标出每一层”不可逆性最高”的决策，看它们是不是集中在两端。
10. “需求的方案有架构”——你们的需求是怎么组织的（按什么分类、什么粒度）？这个组织方式本身，是被认真决策过的，还是顺手定的？
11. 复盘一次”被动做架构决策”的经历（像万辉那样，被事故逼着改）：如果能回到当时，你会在哪个时刻、用哪把尺子拦住它？

12. 挑一个高不可逆的决策，走两关：先过目的关（它守住的是对目的必要的不变量吗?），再过代价关（按三层框架排优先级与时机）——两关的次序错了吗?
（答案要点见书末附录，与训练教材题卡同源）

## 参考文献

**标准** - R-04 ISO/IEC/IEEE 42010:2022（“基本”判据的锚系二**手派生表述** IEV 831-01-01 Note 2——本书采用，本标准不给该准入判据）—— 架构定义：**实体**在其环境中的基本概念或属性，以及该实体及其相关**生存周期**过程的实现和演化的**管控原则**（governing principles）；架构决策是**架构描述（AD）必须记录**的内容之一（§5.2.12／§6.10.1）

**经典文献** - R-36 Nygard，“Documenting Architecture Decisions”（2011）—— 架构决策记录（ADR）的起源（六问为本书适配；被否备选死因与复审条件为本书补充）

> 对照阅读：本章”决策记录与不可逆性”呼应第 6 章”需求与方案结对”、第 8 章”评审对目标”、§10.2.4”不可还原三判据”与 §10.3.1”架构必须被表达出来”、第 11 章”架构设计（基本概念或属性＋管控原则；元素与关系是它们的落点、其规格归下游设计定义过程）“。

# 第 13 章 配置管理：让一直在变的东西说得清

> 本章概念：配置 / 版本 / 基线 英文来源：configuration / version / baseline 主案例：冰链科技 M3 冷链温控箱（产品侧）｜镜照：恒信集团合同审批系统（IT 侧）

---

## 13.1 误解现场

M3 冷链温控箱在市场上跑了一年多，散在全国的物流车上，约五千台。第 10 章水忠立下的老规矩里有一条：“固件升级必须向后兼容已出厂的设备。”

这年春天，一个 bug 冒了出来：箱温降到 −20℃ 以下时，传感器偶发丢读数——一秒的数据，偶尔缺一格。李旭在测试里复现了，水忠修好，发了一版新固件 v2.3，通过 App 推送给已出厂的设备。三个月里分批升级了大约三千八百台。一切正常。

然后疾控中心来审计一批疫苗运输记录。

审计员问了一个以前从没人问过的问题：“这批疫苗四月运输，全程温度记录正常。你们要证明一件事——这批记录，是在哪一版固件下采集的？你们从这批之后，采集逻辑、断链判定升级过没有？”

水忠觉得这问题简单：“查一下这批设备当时跑的固件版本。”

结果查不清。升级记录只有一句”已推送 v2.3”，没有”哪台设备、哪个时间、从哪一版升到哪一版”的明细。

更麻烦的是：修丢读数的 bug，动的是采集算法——v2.1 采的温度记录和 v2.3 采的，断链判定的逻辑不一样。一条温度记录到底算不算”断链”，取决于它是哪个固件采的。可这批设备当时在哪个版本上跑的，没人能说清。

追溯链在”固件”这一环断了。

逸人接话：“出厂时每批设备烧的哪版固件，产线台账上都有；可它们出了厂，App 推送升过几回级、哪批升了哪批没升——这边没账。中间那根线，两头没接上。”

水忠觉得冤：“我们有版本号啊。v2.1、v2.2、v2.3，都标着呢。升级也走流程了，李旭也验证过了。怎么就不算配置管理了?”

文正：“你摸摸看。版本号是有了，可哪一版才是我们对着验收、客户手里那批对得上账的?升级了哪些设备、哪个批次跑在哪一版上、版本之间采集逻辑变没变——这些有记录吗?没有。我们把’版本号’当成了’基线’，以为标了号就万事大吉——其实光有版本号没用，得有一版是大家拍板认下、拿它当基准的，那才算数。”

产品开发经理永兵也在场，他听了全程，低声说了一句：“咱们把箱子卖出去的时候，跟客户讲的是’全程可追溯、断链必告警’。今天连自己都追溯不清，这四个字还怎么站得住?”

水忠愣住了。

如果你坐在冰链这间会议室里，你大概也会觉得冤：版本号不是有吗?流程不是走了吗?——大多数人对配置管理的直觉正是如此：**版本管理做好了，就等于配置管理做好了。**

这一章要推翻的，正是这个直觉。**版本只是时间轴上的刻度，基线才是把”已经批准的决定”冻结下来、让一切变化得以言说的参照。**

而”配置”——那个被各种”参数文件”“个性化设置”遮蔽的概念——是这一切的地基。

这一章我们做三件事：把”配置”从”参数文件”里解放出来，看清它其实是”实体被记录下来的样子”（第一幕）；把”版本”和”基线”这两个时间轴上的概念钉死，看清它们一横一竖的分工（第二幕）；最后把三词用起来——回答”配置管理到底为了什么”、看清它怎么守护第 12 章那些”不可逆决策”、以及为什么非做不可（第三幕·守护与价值）。

---

## 这一章要回答什么

读完本章，你应该能回答：

1. 配置是”参数文件”吗？为什么说配置是实体”被记录下来的样子”，而不是参数本身？
2. 配置项、配置、配置信息三个词差在哪？为什么一份文档既是”配置信息”又是”配置项”？
3. 什么是”变与不变的分离”？为什么”改一个参数就要改代码、重编译、重发布”是变与不变没分离的信号？
4. 版本、变体、修订差在哪？“版本顺着流、基线垂直切”是什么意思？为什么说”配置提供内容、版本提供位置、批准提供资格”——三者缺一不可才成一个基线？

5. 为什么说”没有基线，变化就不可言说”？变更控制是怎么守住基线的门的？
6. 版本管理做好了，配置管理就做好了吗？配置管理的四大活动是什么？为什么一个都不能少？
7. 配置管理的核心目的三问是什么？为什么说让这三个问题任何时刻都有答案，就是配置管理的全部工作？
8. 配置管理和第 12 章的架构决策是什么关系？为什么说决策记录是决策的”配置信息”？
9. 什么是契约链？为什么说基线是每一级契约的”冻结端”、版本是”交付端”？

---

## 这一章的骨架

先把本章要钉的三个词、和它们牵出的分工立起来——后面三幕，都是给这层骨架添血肉。

> **配置提供”内容”，版本提供”位置”，批准提供”资格”——三者齐备，才成一个基线**。配置（configuration）是实体被记录下来的形态——那台 M3 “被记录下来的该有的样子”；版本（version）是时间轴上的演进刻度——它”走到第几步”；基线（baseline）是被批准冻结的参照——它”哪一版被盖章”。三词各有各的活，谁也不能替谁。

> 术语注：这句口诀（内容 / 位置 / 资格）与”一横一竖”的分工，是本书对 ISO 10007（配置/配置项/配置信息/基线）与 ISO/IEC/IEEE 24765（版本）定义的凝练编排，不是任何标准的原文——标准分别定义了三个词，“谁提供内容、谁提供位置、谁提供资格”的对应是本书的主张，欢迎检验、欢迎校准。

这一章的骨架是三**个词** + **一横一竖** + **一把口诀** + **一个纲领**：三个词（配置 / 版本 / 基线，各管内容 / 位置 / 资格）；一横一竖（**版本顺着流、基线垂直切**——横管流动、竖管冻结）；一把口诀（配置提供内容、版本提供位置、批准提供资格——三者缺一不可才成基线）；一个纲领（**让一直在变的东西说得清**）。

判据主义是这一章的方法底座时刻——正是第 1 章那把**“不给没有判据的定义”**的尺子：版本要能精确对应到”哪一次完整构建”，基线要三特征三问全”是”。三个词，都得过这把尺子。

第一幕把三个词各钉在一层（是什么），第二幕把它们和容易混的东西分开（不是什么——一横一竖磨边界弧），第三幕回答”往哪去”（配置管理怎么把第 12 章那些不可逆决策守护住）。

---

## 13.2 第一幕 配置、版本、基线是什么（定界）

> 这一幕把三个词各钉在一层：配置是实体被记录下来的形态（内容，含三个隐形过滤器与配置三问）、版本是时间轴上的演

> 进刻度（位置）、基线是被批准冻结的参照（资格，含三种正式基线与运行/部署配置）。三词各有各的活——先看清它们，后面磨边和运用才有得磨。

### 13.2.1 配置：实体被记录下来的形态

要讲清”配置”，绕不开配置管理里三个纠缠在一起的概念：**配置项**、**配置**、**配置信息**。这三兄弟最大的坑，是它们彼此定义的——打开权威标准 ISO 10007，你会看到：

- “配置”用”配置信息”定义：在配置信息中规定的、相互关联的功能与物理特性；
- “配置项”用”配置”定义：配置中满足最终使用功能的实体；
- “配置信息”又指”描述配置项所需的资料”。

三个术语互相引用，首尾相连，形成一个循环定义。你要懂一个，就得先懂另外两个；你谁都不懂，谁都没法下手。

解耦的方法，是往这个环里引入一个**不依赖任何配置术语的锚点**——“实体”（entity）：任何可被识别和区分的一个具体存在，实物（设备、板卡、部件）或信息制品（文档、代码、模型）。然后让三个概念全部锚定在”实体”上，彼此不再互引：

| 概念 | 锚定实体 | 回答的问题 | 一句话 |
| --- | --- | --- | --- |
| **配置项** | 被组织选定、纳入配置管理范围的实体 | 管哪些? | 被管起来的那个东西 |
| **配置** | 实体被记录在配置信息里的关联功能+物理特性 | 它是什么样? | 它被记录下来的该有的样子 |
| **配置信息** | 记录实体功能与物理特性的资料 | 拿什么核对? | 记录它的那些资料 |

用体检打个比方：配置项是”人”，配置是”这个人的身高体重”，配置信息是”他的体检档案”。三者围着同一个产品转，但一个是对象、一个是属性、一个是记录——**不是同一层的东西，谈不上真循环**。

把它落到冰链。M3 这台产品，哪些是配置项？——温控硬件板卡、固件、云端追溯平台、App、设计图纸、需求规格书……只要组织选定”它值得被单独标识、单独管版本、单独管变更”，它就是一个配置项。

配置项清单（Configuration Item List）回答了”我们管哪些东西”。第 6 章那棵”设计树”——枝干是框架定义、叶子是详细定义——它们就是两个配置项，分别纳入基线。

框架改了走重一点的变更，内容改了走轻一点的变更，这就是枝干和叶子分开管版本的原因。

M3 的”配置”是什么？——固件里”传感器采集算法 + 断链判定逻辑 + 告警逻辑”这一整套相互关联的特性组合在一起、记录在配置信息里、塑成的那个样子。

注意这里的两个词：**相互关联**，**被记录**——这是定义里的两个关键词，展开见 §13.2.2 的三个隐形过滤器。

配置信息是什么？——记录这些特性的资料：固件源码、固件说明、采集算法规格、告警阈值定义、变更记录。它**独立成立**：你先有记录，才谈得上”被记录的样子”，所以它不回头引用”配置”。

这里有一层，要和第 7 章的产品数据接上：**配置信息——记录实体功能与物理特性的资料——正是产品数据里被纳入管理的那一部分**。产品数据是全集（需求、设计、制造、质量、服务、管理六类），配置信息是被基线化、被变更控制的那一档。

一条需求可以是产品数据而不是配置信息（还是草稿）；一旦它被批准、纳入基线，就成了配置信息。第 7 章说”产品数据是产品的信息存在”，这一章做的，就是让这个信息存在”一直在变，还说得清”。

> **最大的坑：文档既是”配置信息”，又可以是”配置项”**。设计图纸是描述 M3 的”配置信息”；但图纸本身也要被管理、要管版本、要走变更——那时它就是”配置项”。两个层面，不冲突。很多团队在这件事上吵一架：图纸到底是配置信息还是配置项？答案：都是。一个是从”它描述谁”看，一个是从”它自己要不要被管”看。

“configuration”这个词，拆开看，本身就是答案。

- 英文：con-（共同、一起）+ figurare（塑造、成形）——**共同塑成的形态**；
- 中文：配（配合、匹配）+ 置（放置、安设）——**搭配并安置成形**。

两种语言，同一个意思：**配置的本质是”组合成形”**。若干部分（组件、特性）经过”配”——互相配合、匹配、协同；再经过”置”——被放置、被记录进一个整体；共同塑成一个整体形态。它天然是”多体”概念：con-、配，都暗示必须有多个参与者。单个孤立的东西，没有配置可言。

回看 ISO 10007 的定义，你会发现定义里的每个词都对应着拆字：

- “相互关联”——对应”配”：特性之间互相配合、协同；
- “被配置信息规定/记录”——对应”置”：特性被放置、记录进整体；
- “组合塑成的形态”——对应”成形”：as-designed（应该长这样）与 as-built（真长成这样）所描述的”形”。

理解了”组合成形”，再看一个常见场景就豁然开朗了。很多产品有一个”选配”的概念：同一台车，可以选 V6 发动机、自动变速箱、豪华内饰。选配不是”随便从清单里勾几个”——它是有约束的选择：

- **requires（必配）**：选了 V6 发动机，必须配自动变速箱；
- **excludes（互斥）**：选了经济版内饰，不能选豪华版座椅；
- **mandatory / alternative（必选 / 多选一）**：某些项必须有且只能有一个。

为什么要有这些约束？因为配置是”在可行域里找解”——四股力量压下来：

- **需求多样**（每个人要的形态不同）；
- **成本预算**（全选=成本爆炸）；
- **技术约束**（互斥与兼容不可兼得）；

· **商业差异**（一个平台长出多 SKU）。

选择就成了”需求的表达（想要什么）+ 约束的妥协（能给什么）+ 价值的取舍（值得给什么）“。

还有一个隐藏的第五股力量：**质量与可维护**。特性越多，测试组合指数爆炸，缺陷面越大。配置不只是商业手段，它本身就是复杂度治理——**选择，是给系统的复杂度上保险**。

到这儿，“配置”的全貌其实已经浮出来了。本书把配置管理浓缩成一句很耐嚼的话：**从”多”到”一”，就是 configuration 的全部旅程**。

多个组件、多个特性，经过”配”与”置”，塑成一个整体形态——这是**即时结果**（as-designed / as-built 所描述的”形”）；这个形态被正式批准、冻结，成了**基线**——这是**正式结果**；最终落地成一个可运行、可交付、可复现的产品实例——这是**交付结果**。

第一幕的”形”、“基线”与”实例”（运行/部署配置，见 §13.2.5），一个”从多到一”的旅程，就把这一章的主题串起来了。

### 13.2.2 三个隐形过滤器：不是所有特性都是配置

现在把”配置”的定义再压实一步。一个实体有无数特性——颜色、尺寸、导热系数、成本、负责人、生产批次、库存状态……“配置”是这些全部特性里的一个子集。这个子集是怎么被切出来的？ISO 10007 的定义里，藏着三个隐形过滤器：

**第一，类别边界：只认功能特性 + 物理特性**。成本、责任人、生产批次、库存状态这些管理属性，不属于配置。它们由”状态记录”这个活动来管（第三幕会讲），但不算”配置”本身。为什么？——因为配置回答的是”产品是什么样”，不是”产品值多少钱、谁负责”。

**第二，记录边界：只有被配置信息规定/记录下来的特性才算配置**。定义里的“defined in configuration information”是这个意思。

某个没人记录过的导热系数，客观上存在，但它不构成配置——因为没被写下来，就无从核对、无从追溯。**特性客观存在，配置是”被管理起来的那部分特性”**。

注意，这个”记录”既包括设计规定的（as-designed——图纸上写着的），也包括实际实现的（as-built——实物上量出来的），前提都是”被认定并记录下来”。

M3 设计图上规定”断链告警阈值 ±1℃“，这是 as-designed 配置；产线上实测某批设备的实际阈值并记录进台账，这是 as-built 配置。两个都是配置，都参与追溯。**一个特性没有对错，只有”记没记、管没管”——记了、管了，它才是配置的一部分**。

**第三，关联边界：是相互关联的特性整体，不是零散清单**。“这里一个阈值、那里一个开关”凑不成配置；它们要作为一套相互关联的整体，才成为”形态”。

三个过滤器一起，把”实体的全部特性”和”配置”切开。这也是为什么”参数文件”的说法那么有误导性——它让人以为配置是一堆零散的键值对；实际上配置是一套**关联成形的整体形态**。参数是零件，配置是装好的整机。

> 术语注：三个隐形过滤器是本书对 ISO 10007 定义（“defined in configuration information”与”相互关联的功能与物理

> 特性”）的操作化读法，不是标准的原话——原文见参考文献 R-08。

三个过滤器合起来，就是这一章判据主义的第一把尺子：

> **配置三问：它是功能/物理特性吗（类别边界）？它被记录了吗（记录边界）？它和其他特性关联成整体了吗（关联边界）？——三问全”是”，才是配置。**

> **概念卡片 · 配置**

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 实体被记录在配置信息里、相互关联的功能特性与物理特性——实体”被记录下来的样子”（ISO 10007） |
| **它不是什么** | ⑥⑦ | 对立：实体的全部客观特性——特性客观存在，配置是”被管理起来的那部分特性”（三个隐形过滤器切出来的子集）；辨析：配置 ≠ 配置项（对象 vs 属性：配置项是”人”，配置是”人的身高体重”）；配置 ≠ 配置信息（属性 vs 记录：配置是”体检结果”，配置信息是”体检档案”）；配置 ≠ 参数文件（参数是零件， |
| **它** | ②③⑨ | **管** |

### 13.2.3 版本：时间轴上的演进刻度

先看一个老朋友：版本。你可能每天都在用，但很少有人停下来问——版本到底在干什么？

想象一条时间轴。产品从诞生到退役，是一个**连续**的过程：改了采集算法，过两周又修了个 bug，下个月加了告警功能……变化是连续的，但”沟通”必须是离散的。你没法在报障电话里跟客户说”我们的固件现在是’改过 17 次、其中第 9 次动了采集算法、第 12 次改了断链判定’的固件”——这句话没法说。

版本，就是把”连续的时间”切成”可指认的格子”的那把刀。ISO/IEC/IEEE 24765 的定义很朴素：版本是配置项的**初始发布，或其后基于完整构建的重新发布**。一句话——**版本是同一个实体在时间轴上的演进刻度，回答”走到第几步了”**。

版本号有一套通行规则，软件界叫语义化版本（SemVer），主.次.补丁号三段：

| 段 | 名称 | 什么时候升级 |
|---|---|---|
| 主版本 | major | 不兼容的变更——v1 → v2，可能破坏现有使用 |
| 次版本 | minor | 向后兼容的新功能——v1.0 → v1.1，老用户无痛升级 |
| 补丁号 | patch | bug 修复——v1.1 → v1.1.1 |

工程图纸用另一套叫法：Rev A / Rev B / Rev C——**修订**（revision）。“修订”和”版本”是同一个思想的两个粒度：修订偏”改了几次”（内部变更记录，更细），版本偏”发布到第几代”（对外整体编号，更粗）。

版本解决什么问题？四个：

1. **追踪演进**——知道走到哪一步，每一步改了什么有据可查；
2. **回溯**——出问题能回到旧版本（回滚、复现客户环境）；

3. **并行**——v1 还在服役、v2 已开发，多代共存需要不同名字区分；

4. **复现**——报障时说”你跑的是 v2.1”，版本号是沟通的最小公约数。

版本也带一条判据（判据主义的老规矩——不给没有判据的定义）：一个编号，能精确对应到”哪一次完整构建”吗？能，是版本；不能（比如只有文件名），它还不算版本。

### 概念卡片 · 版本

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 配置项的初始发布或其后基于完整构建的重新发布（ISO/IEC/IEEE 24765）——时间轴上的演进刻度，回答”走到第几步” |
| **它不是什么** | ⑥⑦ | 对立：变体（variant）——版本回答”先后到了哪一步”（时间维度，先后因果）；变体回答”同时有哪几种形态”（空间维度，并存差异）；二者正交；辨析：版本 ≠ 修订（版本偏对外整体编号、较粗；修订 Rev A/B/C 偏内部变更次数、更细）；版本 ≠ 基线（版本是刻度、流动的；基线是冻结的参照） |
| **它从哪来** | ②③④⑤⑧⑨ | 管理[illegible]（版本号[illegible]）；发布[illegible]；[illegible]；[illegible]；

### 13.2.4 基线：被批准冻结的参照

版本之外，还有一个词：基线。它在”让变化说得清”这件事上，比版本更重要——却更容易被忽略。

ISO 10007:2017 的定义是：**基线——经批准、确立了产品在某一时点的特性、并作为生存周期各项活动参照的配置信息**。

拆开看，三个特征，一个都不能少：

**特征一：被批准（approved）**。基线不是自动生成的东西——不是”版本号到 v1.0 了所以它是基线”，而是组织**正式批准、盖章确认**的结果。从”流动状态”升级为”受控状态”，需要一个有权威的动作。

谁批准？通常是 CCB（变更控制委员会）或对应的管理者。批准意味着：这不再是”随时能改的一版”，而是”全项目认可的参照”。

**特征二：被冻结（frozen）**。基线一旦建立，任何人都不能随意改。它是配置管理里少数”神圣不可侵犯”的东西——不是不能改，而是改必须走一条正式的路（第三幕讲的变更控制）。随意改基线，等于把”参照系”拆了，那跟没有基线没有区别。

**特征三：作参照（reference）**。它是生存周期所有活动的基准——后续的变更评估、验证、验收、审计，全部拿它对照。“这版需求和基线的差别在哪？”“这版固件和产品基线的差别在哪？”——没有基线，这些问题无从问起。

一句话总结三个特征：**基线是被批准冻结的参照**。批准是资格，冻结是状态，参照是用途。

把三个特征翻成判据主义的话：**不给没有判据的定义——“基线”不是靠感觉认的，是三条可检验的判据验出来的**。问全三句——**基线三问**：被批准了吗？被冻结了吗（任何人不能随意改）？生存周期拿它

作参照吗？——三问全”是”，才是基线；答不出，它只是一版没被批准的版本。

### 13.2.5 三种正式基线：功能、分配、产品

基线不是只有一种。沿着产品诞生的顺序，一个典型产品会建立三种正式基线——它们分别对应配置家族（需求配置 → 设计配置 → 实物配置，即按”意图→方案→实物”划分的三环）的三个层面：

| 基线 | 对应环节 | 内容 | 什么时候建立 |
| --- | --- | --- | --- |
| **功能基线**（Functional Baseline） | 需求配置 | 批准的功能需求——“要做什么” | 需求被批准时 |
| **分配基线**（Allocated Baseline） | 设计配置 | 功能分配到硬件/软件/人——“由谁实现” | 功能分配定案时 |
| **产品基线**（Product Baseline） | 实物配置 | 最终产品的完整定义与实物记录——“实际是什么” | 产品定型/交付时 |

三种基线，回答三个问题：要做什么、由谁实现、实际是什么。它们层层递进——功能基线冻结”要做什么”，分配基线冻结”由谁实现”，产品基线冻结”实际是什么”。这也呼应了第 6、11 章讲的设计链：从需求到构件，每一层都有自己”该被冻结的决定”。

再往下，产品交付之后，还有一层——**运行/部署配置**：环境参数、功能开关、灰度比例。它通常不立正式的三大基线，但内核完全相同：同一套代码，配不同的环境参数，就”组合成”不同的运行形态。

恒信的”测试环境/预发环境/生产环境”，是同一套代码的三个运行配置；玉杰那个日期文件夹，缺的正是”哪个包 + 哪套环境参数 = 哪个可复现的运行形态”。这也是为什么部署管理永远绕不开配置管理——**部署的本质，就是给某个版本配上某个环境，塑成一个可复现的形态**。

但还有一层边界要划：**目的、目标要不要也纳入配置管理？**要受控，但走的是另一条分级。功能/分配/产品基线（连同运行/部署配置）按**生存周期成熟度**切——“要做什么、由谁实现、实际是什么、交付后什么形态”；另有一条**正交的变更权威轴**，按”谁批准、谁变更、变了意味什么”切。

在这条轴上，**目的**（连同架构原则、治理红线）入**治理基线**——治理层批准，一变就是”系统该不该存在”的危机；**目标**（需供双方约定的成功判据）入**协议级**——双方共同确认、变更要重谈承诺；**需求及以下**入产品 CM 基线——CCB 批准、CCB 变更，是正常工程动作。这套分级（治理级/协议级/产品 CM 级）是本书的梳理主张、非 ISO 术语；§14.3.3 会专讲这条轴。

### 概念卡片 · 基线

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 经批准、确立了产品在某一时点的特性、作为生存周期各项活动参照的配置信息（ISO 10007:2017）——被批准冻结的参照 |
| **它不是什么** | ⑥⑦ | 对立：流动状态——基线把”流动”升格为”受控”：线左版本自由流动，线右任何变化必须走变更控制；辨析：基线 ≠ 版本（版本是刻度，基线是有”资格”的那一版——把里程桩当收费站）；基线 ≠ 发布（基线是参照，发布是交付给用户的动作——发布的东西通常取自基线，但两者不是一个概念）；基线 ≠ 备份（备份是为了恢复，基线是为了比较和决策） |
| **它从哪来** | ②③⑨ | 经过评审和批准的变更请求，按照既定流程实施后，由配置管理员或指定人员在配置库中建立。基线不能随意创建，必须满足明确的触发条件，如阶段评审通过、重大里程碑达成或发布前夕。 |

## 案例进展①｜冰链：那页设置，到底是不是配置

误解现场那场审计之后，水忠和几台在役设备较上了劲。他抱着一个已出厂的 M3，打开 App 设置页，翻来覆去地想文正的话——“版本号不是基线”。

“我明白版本号和基线是两回事了，”水忠对文正说，“可我心里还有个坎。我们 App 上那个设置页，客户改’上报间隔’、改’告警阈值’——这到底是不是配置?”

文正没有直接答，反问：“你改那个’告警阈值’，改的是什么?”

“改的是……固件里的一个参数啊。客户在 App 上调，服务端把值推下去，固件读到新值就生效。”

“你这一句话里，其实藏了三样东西。”文正说，“客户改的’上报间隔’这个**参数**——它是配置信息里的一个条目，被记录在案，没错。但’配置’不是这个参数，是这一整套东西：**上报间隔、告警阈值、断链判定逻辑、采集算法，它们关联在一起，塑成的’M3 的行为形态’**。

参数是零件，配置是装好的整机。你改一个参数，是让整机的’形态’变了——变了之后，这台设备采出来的温度记录，含义都跟着变。”

水忠醍醐灌顶：“所以审计员问’这批记录是哪一版固件采的’，问的不是参数，是**形态**——哪一套采集逻辑+判定逻辑组合成的行为，采出了这批记录。”

“对。这就是为什么追溯温度记录，必须知道固件版本。因为你改的任何一个参数，都可能改变’这台设备当时是什么样’。参数是变了就变了的开关；配置是’这台设备被记录下来的样子’——你永远要知道，它当时是哪副样子。”

## 13.3 第二幕 配置、版本、基线不是什么（磨边界弧）

> 这一幕把三个词和它们容易混的东西分开——判据主义在这里磨弧：配置不是参数文件、版本不是基线（一横一竖）、变体不是版本；磨到最后一道，是”没有基线，变化就不可言说”。

### 13.3.1 配置不是参数文件：先看病

冰链的 App 里，有一个设置页。里面有几个开关和数字框：温度上报间隔（默认 5 分钟）、超温告警阈值（默认 ±1℃）、断电告警开关、蜂鸣器音量。

李旭给新来的测试员讲解时，指着这页说：“这就是咱们的配置。客户能改的，都在这页。客户改了个参数，我们不用动固件、不用重编译，服务端一推就行。”

这句话，在座的人没人觉得有问题。参数、配置，不是一个意思吗？

这一章的第一个病根，就藏在”不是一个意思吗”里。**把”配置”当成”那页设置里的参数”，是配置管理领域流传最广的误解——它把一个重量级的概念，压缩成了一个轻量级的词：参数文件。**

用第 1 章的话说，这又是一次”翻译伪朋友”（病三）式的近亲：参数（parameter）和配置（configuration）长得像（都跟”可变的东西”有关）、其实不是一层（参数是单个的值，配置是一整套”被记录的形态”），糊在一起用，就出事了。

先把话说清楚：李旭指的那页设置，确实是”配置信息”的一部分——那些参数被记录下来了。但**参数文件只是记录”形态”的图纸，**

**形态本身才是配置**。这个区别，第一幕已经立过——参数是零件，配置是装好的整机；第二幕磨边弧的第一道，就把它钉成边界。

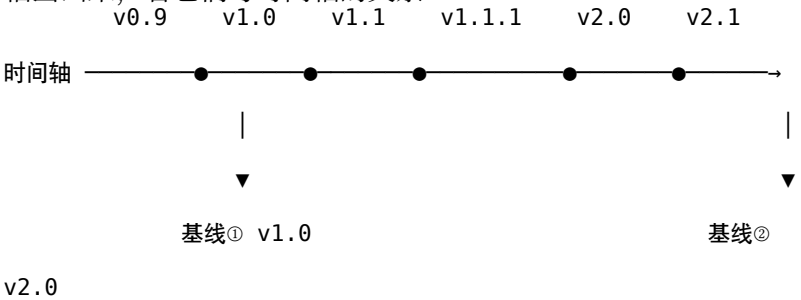
这背后，其实还藏着一个更普适的机制——**变与不变的分离**。李旭那句话里已经有了答案：客户改”上报间隔”，不用动固件、不用重编译，正因为**可变的部分（参数）被外置了，不变的核心（固件逻辑）只写一遍**。

反过来，如果改一个参数就要改代码、重编译、重发布，说明可变的部分被焊死在了不变的核心里——**这就是变与不变没分离的信号**。

### 13.3.2 版本不是基线：一横一竖

现在时间轴上场了。第一幕把”配置”钉在了”实体被记录的形态”上；版本和基线，是时间轴上的两个概念——产品在持续变化：固件从 v1.0 走到 v2.3，合同需求改了六版，图纸从 Rev A 画到 Rev C。变化本身不可怕，可怕的是变化说不清。让变化”说得清”，版本和基线各管一半——而它们恰恰是最容易被混成一对的。

版本和基线，经常被当成一回事——“我们不是有版本吗，怎么还要基线?” 它们当然有关系，但**方向完全不同**。要看懂，先把时间轴画出来，看它们与时间轴的关系：



（被批准、被冻结）　　（被批准、被冻结）

“横”与”竖”，不是上下方向，而是**与时间轴的关系**：

- **横 = 与时间同向（版本）**。版本像**尺子上的刻度**，沿着时间轴排列——v0.9 在左、v2.1 在右，告诉你”走到哪一步”。刻度是尺子的一部分，跟着尺子走。它只描述**位置**，不改变时间轴的性质——刻度左边、右边，一样自由。版本是**流动**的。
- **竖 = 与时间垂直（基线）**。基线像**横在路中间的闸门**，垂直插入时间轴，把时间**切断**：闸门左边是”自由区”（版本随便流动），右边是”受控区”（任何变化必须停下来接受批准）。闸门不是尺子的一部分，是**从外面插进来的障碍**——它制造”分界”，改变时间轴的性质：无约束 → 有约束，质变。基线是**冻结**的。

> 生活类比：**版本是高速公路上的里程桩**——沿路排开，告诉你走了多远；**基线是收费站**——横在路中间，把路截断，过站要检查。一个顺着路，一个挡住路。横管流动，竖管冻结。

这个区分，直接回应了误解现场水忠的困惑：“我们有 v2.3，所以基线有了。”——v2.3 是刻度，它只回答”走到第几步”；它没有回答”哪一版是’被批准、被冻结、作参照’的”。**把版本当基线，等于把里程桩当成收费站：你确实知道路走了多远，但没人拦着检查，任何车都能随意改道。**

判据主义在这里给出第二把尺子：版本是刻度还是基线，不靠感觉认，靠一横一竖验——**它顺着流（版本），还是竖着切（基线）？** 问

一句”这一版有没有被批准、被冻结、拿它作参照”——答不出，它只是刻度。

### 13.3.3 变体不是版本：空间与时间正交

版本之外，配置管理里还有一个经常和”版本”搞混的词——**变体**（variant）。两者的区别，值得用一节说清，因为它们一个管时间、一个管空间，方向完全不同。

冰链的产品线，不是只有一台 M3。同一条产线，可以长出：标准版（-20℃~8℃ 通用）、医疗加强版（多一路独立温度探头）、工业版（更宽的 −40℃~15℃ 范围、防爆外壳）。

这三个形态**同时存在、并存销售**——它们不是”先后走了几步”，是”同时长了几副样子”。这就是变体：**空间维度上的形态差异，回答”有几种形态”**。

版本呢？——每一副形态内部，还在随时间演进：医疗加强版从 v1.0 升到 v2.4。这就是版本：**时间维度上的演进刻度，回答”走到第几步”**。

所以变体和版本是**正交**的，一张小表就能看清：

| | 回答的问题 | 维度 | 举例 |
|---|---|---|---|
| 变体（variant） | 有几种形态? | 空间（并存差异） | 标准版 / 医疗加强版 / 工业版 |
| 版本（version） | 走到哪一步? | 时间（先后因果） | v1.0 → v2.4 |
| 修订（revision） | 内部改了几次? | 时间（更细粒度） | Rev A / B / C |

一个配置可以有很多版本（医疗加强版 v1.0、v2.4）；一个版本下也可以有多个配置（v2.4 的标准版、医疗加强版、工业版）。而**基线，是**

**变体 × 版本的交叉点被盖章的结果**——“医疗加强版 v2.4”这个形态，被批准、被冻结，就成了基线。判断口诀一句话：

> 问”有几种形态” → 变体；问”改到第几版” → 版本；问”这个版本的这个形态” → 基线。**配置提供”内容”，版本提供”位置”，批准提供”资格”——三者齐备，才成一个基线。**

这里把两个”形态”说开：**变体是配置的产出——同一核心经不同选配，长出不同形态。** 于是各归其位：配置提供的是”内容”（实体被记录下来的该有的样子），变体呈现的是”形态”（同一条产线长出的那几副样子）——**配置提供内容、变体呈现形态，两者不打架。**

把这句话补全成动态的，就是配置管理的完整主循环：**配置变了 → 版本升了 → 批准冻结 → 新基线形成，旧基线退役。** 变体管”几副样子”，版本管”每副样子走到第几步”，基线管”哪一步被盖章定格”——三个概念各管一段，谁也不抢谁的活。这个主循环，就是第三幕”变更控制”的种子。

### 13.3.4 没有基线，变化就不可言说

磨边弧磨到最后一道，也最深的一道。前面几道都是”两个词怎么分”，这一道问的是：**没有基线，这个世界还剩什么？**

> **从基线出发，一切变更都可描述；离开基线，一切变化都不可言说。**

为什么”离开基线就不可言说”？——因为要描述一个变化，你必须有一个”从哪变起”的原点。“从 v1.0 变到 v1.1”这句话之所以有意义，是因为 v1.0 是一个所有人都认的、被冻结的、不会动的原点。没有这个原点，“变了”就没有参照——像在茫茫大海上说”我在动”，却没有岸标。

第 4 章讲过需求管理的基线，结论是”基线是变更的出发原点”。这一章把这个原点推广到所有配置项——固件、硬件、代码、图纸、数据。任何”变”，都必须回答”从什么变到什么”；而这个”从什么”，只能由基线提供。

回到冰链。水忠困惑”我们有版本号啊”——版本号给的是”走到哪一步”（v2.3），但没给”哪一步是被批准、被冻结、全项目拿它对照的原点”。于是当审计员问”这批记录是在哪个固件下采集的”，水忠找不到那个原点——**他有一条连续流动的版本链，却没有一个可以参照的冻结点。变化在发生，但每一步都查无实据。**

把这道边界放回磨边弧里看：前面三道是”谁是谁”，这一道是”缺了谁就什么都没了”。配置、版本、基线三词各有各的活；缺了基线那一层，版本也管不住，配置也说不出——**磨边弧磨到最后，磨出的是”三词缺一不可”的资格边界。**

## 案例进展②｜恒信镜照：版本靠”另存为 v2”

几乎在同一段时间，恒信也踩进了同一个坑——只是这一次，连版本号都没有。

合同审批系统上线半年后，运维只有玉杰一个人。每次发布，他的做法是把构建产物扔进一个以日期命名的文件夹：“2026-05-12”

“2026-05-18”。合同管理模块要改一个字段校验，玉杰把新版打进去，文件夹里新旧两个包并存——他懒得删旧的。

两个月后，一个合同状态流转的 bug 暴露了。万辉：“回滚到上上个版本。”玉杰打开”2026-05-18”文件夹——里面有两个包，一个新一个旧，文件名分别是”合同系统_final.zip”“合同系统_final_2.zip”，分不清哪个是哪个。

万辉：“我们有版本号啊。” 徐漾：“你把’文件名的版本’当成了’版本’，把’版本’当成了’基线’——三层都差一点。真正的版本号，要能精确对应到’哪一次完整构建’：这份产物是哪个代码提交编译出来的、包含哪几条变更。

你这’final_2′，既不知道它是哪次构建，也不知道它有没有经过批准，更不知道它和上一版差在哪——它连’版本’都算不上，遑论’基线’。”

万辉看着那个文件夹：“所以出问题要回滚，我连’回滚到哪’都指不出来。”

“对。**版本是回滚的地址，基线是变更的参照**。你只有日期文件夹，既没地址、也没参照——两个月前还能说的’变了’，两个月后就’不可言说’了。”

后来徐漾帮恒信补了发布基线：每次发布，一个包 = 一个版本号 + 对应代码提交号 + 对应需求基线 + 变更记录，一次一张发布记录。从此”上上个版本”这个词，在恒信变成了一句可执行的指令，而不是一种猜测。

万辉把这套文件翻出来看时，自己都笑了——第 4 章合同需求靠”v2.doc”“最终版 v3.doc”管版本，现在是系统部署靠”final_2.zip”管版本，**同一个病，换了个文件后缀再犯一遍**。

---

## 13.4 第三幕 配置管理往哪去：守护与价值（运用）

> 这一幕把三词用起来，分三组推进——**运用**（13.4.1$_{13.4.3)}$：怎么守住基线的门（变更控制）、四大活动怎么循环、核心目的三问怎么答；**守护·意义**（13.4.4$_{13.4.6)}$：配置管理守护的到底是什么——第 12 章那些不可逆决策、契约链、两把易混场景的尺子；**价值**（13.4.7~13.4.9）：为什么非做不可（三条铁律）、带来什么（四个维度）、值多少钱（经济价值）。

### 13.4.1 变更控制：守住基线的门

基线被冻结之后，产品还要变。变，但不能乱变——这就是配置管理的核心循环：**冻结 → 变更 → 再冻结**。

基线建立**之前**：开发流，自由演进——怎么改都行，反正还没批准； 基线建立**之后**：任何变化都是”对基线的偏离”，必须走变更控制——一条正式的路：

**变更申请 → 影响评估 → CCB 批准 → 实施 → 建立新基线**

**（CR） （影响谁） （批不批） （改） （盖章，闭环）**

每个环节都有明确的问题：

- **变更申请（CR）**——要改什么？为什么改？连基线之外的”顺手改动”都要先说出来；

- **影响评估**——改了这一处，会波及哪些配置项？哪些版本、哪些批次、哪些下游要跟着变？这一步，靠的是配置信息（记录）+ 追溯关系（影响地图）。第 12 章万辉改编号那种事，在这里就会被拦住——影响评估一查，发现要联动财务、台账、审计；
- **CCB 批准**——值不值得改？风险谁承担？由变更控制委员会（Change Control Board）裁决。CCB 不是走形式，它是”谁来为这个变更背书”的机制；
- **实施**——改。改完重新验证；
- **建立新基线**——批准之后，新的一版升级为新的基线，旧的基线退役。循环闭合。

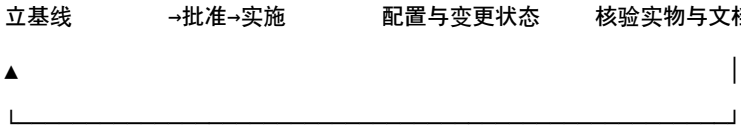
这就是”版本顺着流、基线垂直切”在动态里的样子：**版本一直在流，基线一次次竖切。每一次变更，都是”从旧基线出发、经控制、落成新基线”**。配置管理，本质上就是”冻结 → 变更 → 再冻结”的循环，基线是每次循环的支点。**基线是变更控制的前提，变更控制是基线的守护者**。

### 13.4.2 配置管理四大活动：标识、控制、记录、审计

变更控制只是配置管理的一环。完整的配置管理（Configuration Management，CM），ISO 10007 归纳为五大活动，其中”规划”是组织级的策划（明确职责、流程、工具、剪裁规则）；落到日常执行，是四个持续循环的活动：

① 配置标识 ⟶ ② 配置控制 ⟶ ③ 状态记录 ⟶ ④ 配置审计

选配置项　　变更申请→评估　　记录并报告　　对照基线




| 活动 | 解决的问题 | 典型产出 |
| --- | --- | --- |
| **配置标识** | 说不清”产品是什么” | 配置项清单、基线定义 |
| **配置控制** | 变更失控、改了没人知道 | 变更请求记录、受控版本演进 |
| **状态记录** | 设计与实物对不上账 | 状态报告、版本台账 |
| **配置审计** | 复现不了、责任推诿 | 审计报告、一致性核对结论 |

四个活动，各有各的战场。配置标识管”是什么”，配置控制管”怎么变”，状态记录管”现在什么样”，配置审计管”实物和记录对不对得上”。它们循环咬合——标识立了基线，控制管着变更，记录把变更和状态写下来，审计回来检查记录和实物是否一致，发现不一致，重新回到标识去修正。

配置审计本身还分两支，值得当场点破：**功能配置审核**对照需求验”做对没有”——实质就是验证，借的是测试与文档评审这些证据手段；**物理配置审核**对照基线核”实物与记录一致”——这才是配置管理日常自带的动作。本章讲的是后者；前者做完，“实物=需求”才算有证据。

这套活动的产物落进角色，就是后记四角色清单里**模块工程师（团队）**的《版本与基线记录》（状态报告、版本台账、变更记录都在这一份里）；§13.4.4 纳管的架构决策（ADR）与管控原则，则是**系统架构师**的”决定清单”“管控原则”。

把四个活动套回冰链的固件事故，一眼就能看出缺在哪：

| 活动 | 该有的样子 | 冰链实际 |
| --- | --- | --- |
| 配置标识 | 固件是配置项，立过固件基线 | 固件是配置项，但从没立过基线 |
| 配置控制 | 固件升级走 CR → 评估 → 批准 → 新基线 | “走了流程”，但没有影响评估（没问采集逻辑变没变） |
| 状态记录 | 哪台设备、哪个时间、升到哪一版，有台账 | 只有一句”已推送 v2.3” |
| 配置审计 | 抽查实物，核实记录与实物一致 | 从来没做过 |

四个活动，四个洞。水忠说”我们配置管理做得挺好啊”——那是错觉：**不是做了版本号就等于做了配置管理，四大活动一个都不能少**。

### 13.4.3 配置管理的核心目的：三问，永远答得出

四大活动，围着同一个核心目的转。ISO 10007 把配置管理的目的归成一句话：

> 在产品持续变化的过程中，始终保证它的配置是**已知的、受控的、与需求一致的、可追溯的**。

落到日常，就是随时能回答三个问题——这一章里，它们叫**配置管理三问**：

1. **它应该是什么？**——as-designed：设计规定的样子，由需求/设计配置和基线回答；

2. **它实际是什么？**——as-built：实物真正长成的样子，由实物配置和状态记录回答；
3. **它是怎么变成这样的？**——变更历史：每一次变更从哪来、到哪去，由变更控制和审计回答。

审计员问冰链的那句”这批记录是哪一版固件采的”，正是第三问——它是怎么变成这样的。水忠答不上来，因为状态记录缺位、变更历史散乱。**配置管理的全部工作，就是让这三个问题在任何时刻都有答案。**其中第一个问题靠标识、第二个靠状态记录、第三个靠变更控制——四个活动，就是这三个问题答案的生产线。

> 别把几把”三问”搞混：**配置三问**验的是一个东西”算不算配置”（它是功能/物理特性吗？被记录了吗？关联成整体了吗？）；**基线三问**验的是”这一版够不够格当基线”（被批准了吗？被冻结了吗？生存周期作参照吗？）；**配置管理三问**验的是”整个产品的配置管理做到位没有”（应该是什么？实际是什么？怎么变成这样的？）。版本另有一**问**：能精确对应到哪一次完整构建吗？——一把管单个概念，一把管单版资格，一把管整体目的，各量各的，别混用。

## 案例进展③｜冰链：一次”合规的固件升级”

误解现场之后，冰链把固件升级重建了一遍。正好撞上下一次升级——修一个新的告警延迟 bug，要再发一版固件。

这次，水忠手里多了一张变更申请单。

**影响评估**。水忠没直接发固件，先把”这一版改了什么”写清楚：改了告警延迟判定逻辑（阈值判断改为双窗确认）。

然后他查了追溯关系：告警延迟逻辑，影响哪些配置项？——固件、追溯平台的断链判定、App 告警展示。影响哪些批次？——在役的全部 M3。

他在评估单上写了一行：“固件行为变化，历史温度记录的采集与判定逻辑随之变化，需知悉在役设备批次分布。”

逸人把产线批次台账翻出来：各批设备出厂配的哪版固件、各批多少台，记得清。他把批次分布补进评估单——固件行为变了，哪些批次的历史温度记录会改判定，这次有据可查。

**CCB 批准**。这次升级影响全量在役设备，文正按新规矩把它提到变更控制会：水忠汇报、李旭评估验证方案、董勐确认云端兼容、逸人报批次分布、文正拍板。批准后，固件 v2.4 升为**新固件基线**。

**状态记录**。升级不再是”已推送 v2.3”一句笼统的话。每一台设备，从哪一版升到 v2.4、升级时间、升级结果（成功/失败待重试），进版本台账。以后任何一台设备报障，能查到它当前跑的是哪一版、升级历史是什么。

**配置审计**。三个月后，李旭抽查二十台在役设备：实物固件版本 vs 台账记录，十九台对得上，一台台账写 v2.4、实物还是 v2.3（升级失败未重试）。记录修正，补推送。

那批疫苗的批次追溯，这次对上了。疾控再问”这批记录是哪一版固件采的”，水忠打开台账：这批设备当时在 v2.3，采集逻辑是修订前的判定——答案有据可查。

水忠后来对文正说：“我以前以为，配置管理就是版本号管好。现在我懂了——**版本号只是刻度，基线是参照，流程是守护。三样都有，变化才说得清**。”

### 13.4.4 配置管理：架构决策的守护者

读到这儿，一个细心的读者应该已经发现：配置管理讲的东西，和第 12 章架构决策讲的东西，有某种亲戚关系。第 12 章说架构决策是”选错了要花大代价才能改对”的决定，要识别它、判断它的不可逆性、在成本低时提前做。这一章说版本、基线、变更控制——让一直在变的东西说得清。

这两者不是两门课。第 12 章回答”什么决策值得认真做”，这一章回答”认真做出的决策，怎么在持续的变化中被守护”。第 12 章的结尾留了一个钩子：**“决定做出来之后怎么被守护”**——本幕来兑现。把两张图叠起来看，你会发现它们几乎一一对应：

| 第 12 章 · 架构决策 | 第 13 章 · 配置管理 | 同构关系 |
| --- | --- | --- |
| 决策记录六问（名称/背景/选项/选择/影响/不可逆性） | 配置信息 | 都是”把决定写下来”——决策记录是决策的配置信息 |
| 不可逆决策识别（三标志） | 基线冻结 | 识别出”不可逆”，所以把它冻结成参照，不让它被随手改掉 |
| 推迟决策（现在不做，等什么信息） | 变与不变的分离 | 推迟=把”可变的部分”外置，核心不变、变化的部分可替换 |
| 变更影响分析 | 变更控制 | 改一个决策/配置项前，先评估波及范围 |
| 按不可逆性分级审查 | 配置项分级管理 | 高不可逆的配置项，更重的变更流程、更强的控制 |

这张表值得逐行细看。

**决策记录 ↔ 配置信息**。第 12 章说，决策记录六问把一次决策从”当事人脑子里的知识”变成”组织的资产”；配置信息做的是同一件事——把”这台 M3 是什么样”从”水忠的脑子”变成”可核对的资料”。

第 12 章留过一句话：“决策记录降低的，正是未来的信息恢复成本。”配置管理给这句话配了一个组织级的动作：**让每一条关键信息，都落在某一份配置信息里，而不是某个人的脑子里**。

**不可逆决策识别 ↔ 基线冻结**。 第 12 章的三标志——影响半径大、持续时间长、修改成本高——本质是在问”这个决定，值不值得

当回事”。配置管理用基线给出了”当回事”的物理动作：**高不可逆的决策，一旦批准，就冻结成基线；此后谁要动它，必须走变更控制，而不是顺手一改**。

第 12 章万辉改编号、冰链董勐想给客户开直连绕过追溯域——如果他们面对的是”已冻结的基线 + 必须走的变更控制”，那两场事故都不会发生。

**推迟决策 ↔ 变与不变的分离**。第 12 章的”显式推迟”——信息不足时决定现在不做、保留选择权——落到配置管理里，就是第二幕那个朴素的机制：**变的东西不写死在不变的东西里**。

核心资产（代码、逻辑、功能）只写一遍，变化的部分（参数、开关、选项）外置。推迟决策不是不决策，是让”还可能变的东西”保持可替换——这恰好就是”变与不变分离”的工程形态。

**变更影响分析 ↔ 变更控制**。第 12 章说改一个决策前要问”联动改多少、信息成本多高、风险多大”；配置管理的变更控制把这三问流程化了——CR 里必须附影响评估，影响评估靠追溯关系。**第 12 章的三问是个人尺子，配置管理的变更控制是组织流程——尺子变成流程，才是守护**。

其中架构基线尤其要纳管：架构描述（AD）连同架构决策记录（ADR）一起进配置管理（ISO 10007），评审通过后定格，改动走基线变更流程。**不受变更控制的 AD 只是草图堆，还算不上架构基线**（第 11 章）——描述不受控，不等于架构不存在，等于它没被定格。

架构描述还是最贵的一类配置项——它腐化了，评审对目标就失去参照，带崩的是整个评审体系（第 8 章：评审对目标，架构描述就是那个”目标”）。

影响评估沿追溯链往回追，还有一端要补：**上溯到”目的”**。

上游变更（编号规则、数据所有权这类）不止要评估波及哪些配置项，还要确认它仍在服务系统目的（=守住第 14 章不变量链的目的环）——目的变更会传导为原则与基线更新，走变更控制（这正是第 14 章治理交给配置管理的接口）。

目的本身变了而配置项照旧受控，基线守住的是一套过期的”根本”。**基线与变更控制（ISO 10007）恰好守不变量链的第三层（架构演化：变更受控、不变量守住、变体在允许范围内）与第四层（架构资产：架构描述、架构视角库、可追溯链完整可信）——对应第 14 章治理对象表三、四行。**

## 案例进展④｜恒信：回到编号事故的现场

恒信补了配置管理之后，有一次复盘，徐漾把大家带回第 12 章那场编号事故。

“万辉，如果当时我们有这套东西，”徐漾说，“你改编号规则，会怎么走?”

万辉想了想：“先填 CR。影响评估一查——编号规则这个配置项，被财务、台账、审计、电子签章四个下游引用；对应的需求条目、设计决策、发布版本，全挂在追溯关系上。评估单上会写：‘该变更影响全部历史合同数据，不可逆性高。’——这一行字写出来，我当天就不敢’今天就能上’了。”

“对。”徐漾说，“第 12 章你学的是’改字段前三问’——一把个人尺子。配置管理把它变成了流程：CR 必填影响评估、高不可逆必走 CCB。**个人尺子靠自觉，流程靠结构**。你不是自觉不够，是一个人扛不住那么多’顺手改动’的记忆。”

万辉低头看着那份评估单，说：“我以前觉得配置管理是文员的活、是文档的活。现在我明白——**配置管理是第 12 章那些不可逆决策的保险丝**。决策做对了，靠它守住；决策做错了，靠它让你早发现、早止损。”

徐漾点头，补了一句这一幕最重要的话：

“还有一层。编号规则、数据所有权、固件升级——这些’最不可逆的决定’，往往跨部门、跨项目、跨时间。谁来定它们的准入、谁有资格批准它们的变更？——这就不是配置管理一个工具能回答的了。**那是治理的问题：谁掌舵，谁划桨**。配置管理是治理的一只手，下一章，我们讲那只手的头脑。”

### 13.4.5 契约链：每一对相邻环节，都是局部甲乙方

“守护不可逆决策”还有一个更宏观的形态，值得单独讲——**契约链**（chain of contracts）。

回想第 6 章的设计链：需求 → 方案 → 实现，一层层往下。如果给这条链加上”谁要求谁、谁兑现谁”的视角，你会发现——**链上每一对相邻环节，都是一组局部的”甲方-乙方”关系**：

- 需求方对方案方是”甲方”：要求”按这个需求做”；
- 方案方对实现方是”甲方”：约束”按这个设计做”；
- 每一环的乙方，就是下一环的甲方。

这条契约链在每一对结对处的具体形态，就是第 6 章的**候选声明 + 骨架先行**：叶子”声明我归谁”（向方案方提出要求）、方案方”验证对不对”（验收），先约好的枝梢清单就是这一环契约先谈好的接口——局部甲乙方不是抽象关系，在结对处就长在”候选声明 + 骨架先行”里。

这是系统工程里成熟的思想：ISO/IEC/IEEE 15288 把每一级接口定义为”需方-供方协议”（acquirer-supplier agreement）。而配置管理，给这条契约链提供了每一环都需要的两个东西：

> **基线，是每一级契约的”冻结端”；版本，是每一级契约的”交付端”。**

- 甲方把要求冻结成**基线**，乙方按基线开发；
- 乙方产出的**版本**，交付给甲方验收；
- 甲方验收通过，又成为下一环的”基线”。

用恒信举个例：合同审批需求被徐漾批准，立**功能基线**——这是对设计方的要求；设计方产出的设计，立**分配基线**——这是对实现方的约束；实现方产出的系统，发布**产品基线**。每一环的机制完全相同：要求逐级冻结为基线，兑现逐级交付为版本。

这就是为什么”基线对谁、版本对谁”经常把团队绕晕。日常说法是”按基线开发，提交版本”——甲方批准基线、乙方提交版本。这个说法完全成立。

但要注意：**它不是某一级专属，而是每一级都重复出现的通用机制**。需求阶段有功能基线，实现阶段有产品基线——基线和版本在每一层都在，只是因为视角不同，看起来”属于”不同的人。

一个边界要分清：**开发链内部的”甲乙方”是技术契约**（要求-兑现，管”对不对得上”）；完整的外部甲乙方还多一层商务契约（法律合同、付款、责任，管”赔不赔得起”）。机制同构，勿混为一谈——把”合同”和”契约链”当成一回事，是常见的越界。

### 13.4.6 两个易混场景：把尺子用起来

配置管理的边界问题，有两个经常被拿出来问的场景，值得用这一章的尺子现场裁决。

**场景一：从一堆需求里选子集作为设计开发的输入——这是配置吗?**

是。这正是”需求配置”——配置家族的第一环（意图层）。给某个版本、某个客户选一组需求，就是配置一个”需求变体”；选择结果一旦冻结，就是**功能基线**，从此进入配置管理的领地，后续变更走变更控制。

它和”产品配置”的区别，只是对象（意图 vs 形态）和阶段（开发前 vs 开发中/后），内核完全相同。第 4 章徐漾给恒信建的第一版需求基线，干的正是这件事。

**场景二：同一设计号的多个实体，选一个作为产品部件——这是配置吗?**

取决于配置管理的粒度。假设 M3 有一批同型号的温控板卡（同一个 PMN 号），装进这台设备时，选 3 号还是选 17 号？——看你的管理粒度：

| 粒度 | 性质 | 说明 |
| --- | --- | --- |
| **类型级**（按型号管） | **不是配置** | 选谁，这台设备的”特性形态”都不变——只是分配和序列号追溯 |
| **实例级**（按序列号管） | **是配置** | 选谁决定这台产品的 as-built 配置——安全关键行业按单件管理 |

判定标准一句话：**选 A 还是选 B，产品的特性/形态变了吗？**变了——它们本就是不同配置，标识系统要升级；没变——只是分配问题。

医疗冷链这种安全关键行业，往往按实例级管理：“这台设备装的是序列号 S-02 的板卡”是这台产品 as-built 配置的正式条目——因为出了事要能精确追溯到装的是哪一块板。

### 13.4.7 为什么必须做：三条铁律

读到这里，你可能会想：这些流程、台账、审计，是不是”大公司才配得上的负担”？小团队、小产品，值得吗？——这一节正面回答这个问题。配置管理不是锦上添花，它背后是三条绕不过去的铁律：

**第一，产品开发是协作，协作需要共同事实。**M3 是硬件 + 固件 + 云端 + App——四个域：水忠一人管硬件和固件，董勐等三人管云端与 App。固件改了采集逻辑，如果云端不知道，追溯平台就会拿旧逻辑解释新数据——灾难。多团队各说各话的前提，是有一个各说各话都对得上的”共同事实”——那个共同事实，就是配置基线。

**第二，产品开发是长周期，长周期需要组织记忆。**一台 M3 的生存周期可能七年。七年里，水忠可能升职、转岗、离职。固件当初为什么

这么设计、采集逻辑改过几轮——这些知识如果只在水忠脑子里，人一走，知识就蒸发。**配置信息是组织唯一带不走的资产**——它不随人员流动而消失。

**第三，产品要担责，担责需要可答的账**。冷链断链，要回答”是谁的责任”；疫苗出问题，要回答”这批追溯记录是否可信”。安全、合规、合同，都是必答题。

答不出账，不是”管理不善”，是**没资格承担这份责任**。医药冷链、航空、军工的资质门槛里，配置管理都是硬性项——ISO 9001 要求”产品标识与可追溯性”，正是这个意思。

三条铁律回答的是”为什么做”；至于”做多重”，是另一件事——**配置管理是可剪裁的**。

小团队不是不做，是做”够用的那一档”：至少版本可指认（每个包对应一次完整构建）、变更可追溯（每次改动留一行记录）。大团队、安全关键行业，才配得上完整的 CCB、全流程审计、单件序列化。

剪裁的原则一句话：**配置管理的复杂度，应该与产品的责任等级匹配**。一个内部工具和一个医疗冷链温控箱，配的流程本就不该一样——但”版本可指认、变更可追溯”这条底线，谁也不能省。

### 13.4.8 带来什么：四个维度

三条铁律说”必须做”，那”做了带来什么”？四个维度：

| 维度 | 内容 |
| --- | --- |
| **质量** | 做对（实物=记录，配置审计保证；实物=需求，验证保证）· 测准（测试有明确的版本对象）· 修快（快速定位缺陷来自哪次变更） |
| **效率** | 少返工 · 少扯皮 · 快交付（多版本并行不打架，回滚有地址） |
| **合规** | 可追溯 · 可追责 · 过审计（ISO 9001 / 航空 / 军工 / 医疗的硬性要求） |
| **信任** | 客户能复现、能修复、敢用——“你说的每一版，我都对得上” |

### 13.4.9 经济价值：省钱、赚钱、保钱

“便于管理”是手段，不是价值。配置管理的价值，是**消除的浪费 + 解锁的机会**——落到钱上，三个方向：

**省钱（消除浪费）**。

- **返工费**：缺陷发现越晚，修复越贵（约 1 : 10 : 100 : 1000）。配置管理让每一次变更留痕、让每个版本可回溯——缺陷早暴露，返工早买单；
- **报废费**：状态记录避免”造了没人要的、造了重复的”——这版产品对应哪个批次、卖给谁，一查便知；
- **召回费**：可追溯 = **精确召回**。批次追溯查得清，出问题只召回受影响的那一批；查不清，只能全量召回——差一个数量级的钱。冰

链那次批次追溯，查清和查不清，是”精确召回受影响的那一批”和”召回全部在役五千台”的差别。

**赚钱（解锁机会）**。

- **多变体 = 多收入**：同一核心 × 不同配置 = 多形态交付——配置管理是产品线增长的安全带。没有配置管理，一个变体就翻一次车，产品线长不大；
- **资质 = 订单**：航空、军工、医药的标书里，配置管理是市场准入的门票，没它递不进标书；
- **交付速度 = 赢单**：多版本并行不打架，不违约、不丢单。

**保钱（兜住风险）**。

- **责任清晰**：事故了，哪次变更、谁批的、影响哪些产品——账可答，责任不推诿；
- **知识资产不流失**：配置信息是组织唯一带不走的资产。

把三个方向叠起来看，一句话：**配置管理不是在管文件，是在管”这个产品现在是什么、过去怎么变成这样、将来还能变成什么样”——它保的是产品全部历史与未来的可答性**。

这三笔账（省钱·赚钱·保钱）合起来，就是配置管理”一直在变还说得清”的全部价值。而”谁有资格批准基线、谁有资格裁决变更”——这把尺子不在配置管理手里，在治理手里。这一幕的运用、守护与价值，到这里收束。

---

## 认知反转

回到章首。疾控审计员问”这批记录是哪一版固件采的”，冰链查不清；水忠觉得冤，因为”我们版本号不是有吗”。现在我们知道，那场查不清，不是一次”记录事故”，它连错四层，一层比一层深。

**第一层，他把”配置”当成了”参数”。**他以为配置就是 App 上那页设置、固件里那几个开关。其实配置是”实体被记录下来的形态”——采集算法、断链判定、告警逻辑，关联在一起塑成的 M3 的行为。参数是零件，配置是装好的整机。**把配置当参数，等于拿零件清单当整机图纸。**

**第二层，他把”版本”当成了”基线”。**他有 v2.1、v2.2、v2.3——那是时间轴上的刻度，回答”走到第几步”。他没有的，是”被批准、被冻结、作参照”的基线——回答”哪一版被盖章”。**版本顺着流、基线垂直切：里程桩顺着路排，收费站横在路中间。他有里程桩，没有收费站。**

**第三层，他不知道”没有基线，变化就不可言说”。**追溯一条温度记录，必须知道”从什么基线出发、经哪些变更、落在哪一版”——没有基线，就没有”从什么变起”的原点，一切变更查无实据。**离开基线，一切变化都不可言说。**

**第四层，也是最深的一层，他错过的不是一个流程，是一整套守护。**配置标识、配置控制、状态记录、配置审计——四大活动循环咬合，才让产品”任何时刻都说得清”。而这一切，守护的正是第 12 章那些”不可逆决策”：决策做对了，靠基线把它冻住；决策错了，靠变更控制让它早止损。

这一章的四次反转，其实是同一个反转的四种说法：

> **版本只是刻度，基线才是参照；配置是”被记录的样子”，配置管理是”让这个记录永远可信”的机制——让一个一直在变的产品，在任何时刻都能被准确地说出来。**

把四次反转收成第 1 章的一个词：水忠对”配置管理”，完成了从**见过到拥有**的跨越。“见过”，是他背得出版本号、分得出 v2.1 和 v2.3；“拥有”，是他能答全四问。

- **它是什么**——配置是实体被记录的形态、版本是时间轴上的刻度、基线是被批准冻结的参照；
- **不是什么**——配置不是参数文件、版本不是基线、变体不是版本、离开基线变化就不可言说；
- **从哪来**——硬件工业的组件选配，走到 ISO 10007、24765 的定稿；
- **往哪去**——变更控制、四大活动，守护第 12 章那些不可逆决策。

第 1 章说，能答全四问才算拥有——水忠现在答得出了。你也应该答得出：**能答全四问才算拥有——你能说出版本与基线一横一竖，让一直在变的东西说得清。**

回看第 10 章的结尾。第 10 章说：架构，是对不可逆决策的提前投资；第 12 章把它拆成识别、判断、记录的可操作动作。这一章把投资之后的”守护”补齐了——**不可逆决策被批准之后，用基线冻结成参照，用版本记录演进，用变更控制守住基线的门。**第 12 章回答”哪

些决策值得认真做”，这一章回答”认真做出的决策，怎么在持续变化中不散架”。

而”守护不可逆决策”这件事，还有一个更大的问题没回答：**基线由谁批准、变更由谁裁决、高不可逆的准入权在谁手里？**——这不是配置管理这个工具能回答的，这是治理的问题。下一章，我们讲”谁掌舵，谁划桨”——**配置管理是治理的一只手，治理是那只手的头脑**。

用第 1 章 §1.4.4 的尺子量一下：地图四层讲的是**个人脑内**那张图；这一章守的是它的**外化形态**——把图变成组织能共用、能追溯的那一份。

## 小结

这一章把”配置管理”讲成了一件可以说清的事。**配置**是”实体被记录下来的形态”——配置项是对象、配置是形态、配置信息是记录，三者都锚定在”实体”上；变与不变的分离，让核心只写一遍、变化的部分外置。

**版本**是时间轴上的演进刻度，回答”走到第几步”；**基线**是被批准冻结的参照，回答”哪一版被盖章”——**版本顺着流、基线垂直切**，一个管流动、一个管冻结。**没有基线，变化就不可言说**：任何变更必须能回答”从什么变到什么”，而”从什么”只能由基线提供。

变更控制（冻结→变更→再冻结）守住基线的门，四大活动（标识/控制/状态记录/审计）循环咬合，让产品在任何时刻都能回答三问——应该是什么、实际是什么、怎么变成这样的。

配置管理不是”便于管理”的装饰，它守护的正是第 12 章那些”不可逆决策”：决策记录是决策的配置信息，基线冻结是不可逆决策的物理动作，变更影响分析是第 12 章三问的流程化。

测试记录上，多了一栏：基线。以前他只知道”测过了”；这一章之后他知道——测的是哪一版、对照哪条基线，写不清楚，“测过了”就没法交代。

它经济价值的全部秘密，藏在三句话里：省钱（返工、报废、精确召回）、赚钱（变体、资质、速度）、保钱（责任、知识）。而”谁有资格批准基线、谁有资格裁决变更”——这个问题超出配置管理本身，是治理的领地。

下一章，我们讲治理。配置管理给了”怎么守住不可逆决策”的手，治理回答”谁掌舵、谁划桨”的头脑——**配置管理是治理的一个抓手，第 14 章把它放回它该在的位置**。

---

## 本章概念总图

**第一幕 · 三词各亮一层（是什么）**

**配置 = 实体被记录下来的形态（配 + 置 = 组合成形）—提供"内容"**

**配置项 = 被纳入管理的实体（对象）/ 配置 = 被记录的关联功能+物理特性（形态）/ 配置信息 = 记录特性的资料（记录）**

**三个隐形过滤器：类别边界 / 记录边界 / 关联边界 — 配置三问**

**版本 = 时间轴上的演进刻度（横，与时间同向）—提供"位置"—回答"走到第几步"**

**主.次.补丁号（SemVer）；修订 Rev A/B/C = 更细粒度的内部次数**

基线 = 被批准冻结的参照（竖，垂直切断时间）—承载"资格"—回答"哪一版被盖章"（批准授予资格，基线承载资格）

三特征：被批准 / 被冻结 / 作参照

三种正式基线：功能（要做什么）→ 分配（由谁实现）→ 产品（实际是什么）

运行/部署配置：代码 + 环境参数 = 可复现的运行形态

第二幕 · 一横一竖磨边界（不是什么）

配置不是参数文件：参数是零件，配置是装好的整机

变与不变的分离：核心只写一遍，变化的部分外置

版本不是基线：里程桩 vs 收费站—版本顺着流，基线垂直切

变体不是版本：变体管空间（有几种形态），版本管时间（走到第几步），二者正交

没有基线，变化就不可言说：从基线出发，一切变更都可描述；离开基线，一切变化都不可言说

第三幕 · 守护（往哪去）

变更控制：冻结 → 变更 → 再冻结（CR → 影响评估 → CCB → 实施 → 新基线）

四大活动：配置标识 → 配置控制 → 状态记录 → 配置审计

核心目的三问：应该是什么 / 实际是什么 / 怎么变成这样的—任何时刻都答得出

配置管理 = 架构决策的守护者：决策记录↔配置信息；不可逆识别↔基线冻结；推迟↔变与不变分离；影响分析↔变更控制

契约链：基线=每一级契约的冻结端，版本=每一级契约的交付端

三条铁律：协作要共同事实 / 长周期要组织记忆 / 担责要可答的账

价值：省钱（返工/报废/精确召回）· 赚钱（变体/资质/速度）· 保钱（责任/知识）

## 本章判据

> 这十三把尺子是本书的判词与编排，不是任何标准的条文——概念出处见各概念卡（ISO 10007 / 24765 / 15288），“三问/判词/循环”的提法本身是本书主张：当场可检验，欢迎检验、欢迎校准。

1. **配置三问**：它是功能/物理特性吗（类别边界）？它被记录了吗（记录边界）？它和其他特性关联成整体了吗（关联边界）？三问全”是”，才是配置。参数文件只是记录形态的图纸，形态才是配置。**违反即崩：把”那页设置”当成配置——客户改个参数就以为配置变了，可没被记录的形态，追溯时无据可查**。
2. **变与不变的分离**：核心资产只写一遍，变化的部分外置——“改一个参数就要改代码、重编译、重发布”，是变与不变没分离的信号。**违反即崩：客户一改”上报间隔”就得改代码、重编译、重发布——可变的部分焊死在不变的核心里，改一次发一次**。
3. **配置项/配置/配置信息三分离**：配置项是”被管的那个东西”，配置是”它被记录下来的该有的样子”，配置信息是”记录它的那些资料”——都锚定在”实体”上。**违反即崩：把”被管起来的实体”当”被记录的形态”——审计问配置是什么，答成”就是我们管的那些板卡和文档”，对象和形态对不上账**。

4. **版本是刻度**：版本回答”走到第几步”；一个版本号必须能精确对应到”哪一次完整构建”（文件名不算）。**违反即崩：文件名加个 final_2 就当版本——玉杰回滚时，分不清哪个包对应哪次构建**。
5. **基线三特征**：被批准（有权威的盖章）+ 被冻结（任何人不能随意改）+ 作参照（生存周期拿它对照）——三问全”是”才是基线。**违反即崩：版本号到 v2.3 就当基线——没被批准、没被冻结，疾控审计时找不到可参照的冻结点**。
6. **版本顺着流、基线垂直切**：版本沿时间轴排开（流动），基线垂直插入切断时间（受控）；把版本当基线，等于把里程桩当收费站。**违反即崩：只有里程桩没有收费站——版本一直在流，却没有一版被批准冻结，改了什么查无实据**。
7. **没有基线，变化就不可言说**：任何变更必须能回答”从什么基线变到什么基线”；答不出，就还没有基线。**违反即崩：审计问”这批记录是哪一版固件采的”——版本链连续流动，却找不到”从什么变起”的原点**。
8. **变更控制循环**：变更申请 → 影响评估 → CCB 批准 → 实施 → 建立新基线——基线建立后，任何变化必须走这条线。**违反即崩：“先推 v2.3 再说”——没有 CR、没有影响评估，改采集算法动了断链判定，批次追溯全错**。

9. **配置管理四大活动**：配置标识（是什么）→ 配置控制（怎么变）→ 状态记录（现在什么样）→ 配置审计（实物与记录一致吗）——一个都不能少。**违反即崩：只做版本号（标识的一半），缺状态记录和配置审计——“已推送 v2.3”一句笼统的话，哪台设备在哪一版说不清**。
10. **配置管理三问**：它应该是什么（as-designed）？它实际是什么（as-built）？它是怎么变成这样的（变更历史）？——任何时刻都答得出。**违反即崩：只能答”应该是什么”，答不出”实际是什么”和”怎么变成这样”——疾控一问”这批记录是哪版固件采的”，当场卡壳**。
11. **配置管理守护架构决策**：决策记录↔配置信息、不可逆决策识别↔基线冻结、推迟决策↔变与不变的分离、变更影响分析↔变更控制——第 12 章的尺子，第 13 章给它们配了流程。**违反即崩：决策做对了没有基线冻结、做错了没有变更控制早止损——万辉改编号”今天就能上”，三个月连环三爆**。
12. **契约链**：需求对方案、方案对实现，每一对相邻环节都是局部甲乙方——基线是每一级契约的冻结端，版本是交付端。**违反即崩：基线版本各归一方——需求方只管冻结、开发方只管交付，验收时对不上”哪版需求对应哪版交付”**。

13. **配置 × 版本 × 基线三分工**：变体管空间（有几种形态）、版本管时间（走到第几步）、基线管资格（哪一版被盖章）——配置提供”内容”，版本提供”位置”，批准提供”资格”；三者齐备，才成一个基线。**违反即崩：只有配置和版本、没有批准——变体形态一直在变，没有一版被盖章定格，主循环转不起来**。

## 自查 · 这些说法对吗？

1. “配置就是系统设置里那些参数。”——**错**。参数是零件，配置是装好的整机——配置是”被记录的形态”，参数只是配置信息里的条目。
2. “版本管理做好了，配置管理就做好了。”——**错**。版本只是刻度，配置管理是标识+控制+状态记录+审计四活动；缺了基线和变更控制，版本也管不住。
3. “我们有版本号，所以有基线。”——**错**。版本是刻度，基线是资格——被批准、被冻结、作参照，三特征全有才是基线。
4. “基线一旦建立就永远不能改。”——**错**。基线可以变更，但必须走变更控制，改完建立新基线。“冻结”不是”死锁”。
5. “没有基线，变化也说得清——反正有 git 历史。”——**错**。git 历史是记录，基线是被批准、被冻结、被全项目认可的参照；没有批准动作，历史只是流水账。

6. “配置管理是文档的事，跟代码、硬件没关系。”——**错**。任何配置项——代码、固件、硬件板卡、数据、图纸——都可立基线、走变更控制。
7. “小团队、小产品，配置管理是大公司的负担。”——**错**。配置管理可剪裁——小团队至少要有”版本可指认、变更可追溯”；负担的大头来自”查不清”的返工和事故。
8. “变更控制就是多开会，拖慢交付。”——**错**。变更控制是分级别的：高不可逆走 CCB，低可逆快速走。真正的拖慢，是没有控制导致的事故和回滚。
9. “配置管理管的是’现在是什么’，跟历史没关系。”——**错**。核心目的三问里有一问就是”它是怎么变成这样的”——变更历史是配置管理的半壁江山。
10. “变体就是版本，改个名字而已。”——**错**。变体管空间（有几种形态）、版本管时间（走到第几步），二者正交；“医疗加强版 v2.4”是变体×版本的交叉点，被批准冻结才成基线。
11. “改一个参数就要改代码、重编译、重发布——这是正常的，参数本来就长在代码里。”——**错**。这是”变与不变没分离”的信号——核心资产（代码、逻辑、功能）只写一遍，变化的部分（参数、开关、选项）应该外置。

12. “配置项、配置、配置信息三个词，是同一件事的三个叫法，换个角度而已。”——**错**。配置项=被管起来的实体（对象）、配置=实体被记录的形态（属性）、配置信息=记录形态的资料（记录）——三者锚定实体，不是同一层的东西。
13. “基线和版本各有归属——需求阶段的功能基线归需求方，实现阶段的产品版本归开发方。”——**错**。契约链每一级都重复同一机制：每一级契约都有基线（冻结端）和版本（交付端），只因视角不同看起来”属于”不同的人。

## 练习

1. 找出你们产品里”客户/用户能改的参数”，逐个用”配置三问”检验：它是配置，还是配置信息里的一个条目？
2. 列出你们产品五个配置项（代码/固件/文档/数据/硬件），给每个标一个”该不该纳入配置管理”的判断。
3. 检查你们当前最大的配置项：有版本号吗？版本号能对应到”哪一次完整构建”吗？有基线吗（被批准+被冻结+作参照）？
4. 找一次你们最近做过的”回滚”或”追溯”：回滚到哪一版？追溯查得到哪一批？查不到的地方，缺的是版本、基线、还是状态记录？
5. 画出你们产品的”版本链”（时间轴），在上面标出应该竖切的几个”基线点”——它们该在哪些里程碑建立？

6. 挑一个最近的变更，对照”变更控制循环”五步走一遍：哪一步你们跳过了？跳过的那一步，代价是什么？
7. 用”配置管理四大活动”诊断你们团队：标识、控制、状态记录、审计，各打一个”有/缺/半吊子”，看缺的补上要做什么。
8. 把第 12 章”决策记录六问”写的一份决策记录，对照第三幕的”同构表”，标出它对应的配置信息、基线、变更控制各在哪里。
9. “契约链”：画出你们开发链上三层（需求→设计→实现），标出每层”基线（冻结端）“和”版本（交付端）“分别是什么。
10. 回到万辉的编号事故（第 12 章），用配置管理的视角复盘：如果当时有 CR + 影响评估，评估单上会写什么？评估单能否拦住那次事故？
11. 找出你们行业里一个”精确召回 vs 全量召回”的真实案例——算一下，配置管理（批次追溯）在这个案例里值多少钱？
12. 用第 1 章的四问法量”配置”或”基线”：它是什么、不是什么、从哪来、往哪去——哪一问你现在答得最薄？回本章概念卡片或 §1.2.1（基线的”凝结”故事）把它补上。

（答案要点见书末附录，与训练教材题卡同源）

## 参考文献

**标准** - R-08 ISO 10007:2017 —— 配置管理主标准（对应国标 GB/T 19017）：配置/配置项/配置信息定义、基线定义、五大活动 - R-20 ISO/IEC/IEEE 24765 —— 版本定义（初始发布或重新发布） - R-09 ISO/

IEC/IEEE 15288:2015 —— 契约链（需方-供方协议）- R-21 SemVer —— 语义化版本号（主版本·次版本·补丁号三段升级规则）- R-03 ISO 9001:2015 —— §8.5.2 标识和可追溯性

> 对照阅读：本章”没有基线，变化就不可言说”呼应第 4 章”基线是分界线”（需求管理视角）、“枝干和叶子是两个配置项”呼应第 6 章（设计文档的配置化）、“决策记录六问”呼应第 12 章（本章同构表的另一半）。

# 第 14 章 治理：谁掌舵，谁划桨

> 本章概念：治理 / 管理 英文来源：governance / management
>
> 主案例：恒信集团合同审批系统（IT 侧）｜ 镜照：冰链科技 M3 冷链温控箱（产品侧）

---

## 14.1 误解现场

恒信的合同数据，又对不上了。

审计前，财务对账发现：同一份合同，三个系统，三个样子。法务的合同审批系统里，编号是”HX-2026-0418”；财务的台账里，同一份合同编号成了”2026-04-018”。

业务的系统里干脆没有编号，靠合同名模糊匹配。审计要求”合同、台账、系统三方对账”，对不上。

这不是第一次。第 12 章那场编号事故之后，恒信一直在跟数据打架。

只是这次，集团终于拍板：“要治一治数据。”

集团数据部牵头，招标选型，上了一套”数据治理平台”。供应商演示得漂亮：自动清洗、去重、补全、画血缘、定时对账。

最后一页 PPT 写着：“一揽子解决贵集团数据治理问题，数据从此是资产。”

产品开发经理满海也在选型会现场。演示一页页翻过去，他在审批栏里签下”同意”——数据干净了，合同系统用起来才省心。

五百万的预算批了。平台买回来，数据部还专门成立了一个”数据治理委员会”。

运维玉杰接手跑平台，第一天就发现：这台五百万的机器，大部分时间是闲着的。他嘀咕了一句，当晚把负载曲线存进了运维台账。没人在意。

法务、财务、业务、IT，各来一位部门负责人，每季度开一次会。

徐漾参加了第一次委员会会议。议程三项：季度数据质量报告（清洗了多少条、对上了多少条）、平台使用汇报、下一季度清洗目标。

四十五分钟开完。万辉在门口小声说：“这不就是数据清洗周报会吗？”

徐漾没接话。他想起第 13 章最后，自己跟万辉说过的那句话——“那是治理的问题：谁掌舵，谁划桨。配置管理是治理的一只手。”现在他站在”治理”两个字面前，却发现自己根本不知道它是什么。

他翻了翻那五百万的合同，又看了看季度会议纪要，总觉得哪里不对。三个系统里对不上的合同编号，第一季度结束仍然对不上——因为**没有任何一个人，被授予”裁决合同编号标准”的权力**。

平台能清洗一万条，却裁决不了”法务的编号是标准，财务必须对齐”。而后者，才是五百万买不来的东西。

你大概也见过这种场景：公司说要搞”数据治理”，买了一套工具、成立了一个委员会，然后一切照旧。

问题不在工具不好，而在——**我们把”治理”这个词，用在了一件”管理”的事上**。

工具清洗数据，是管理；谁有权裁决跨部门的标准、谁来担责、偏航了谁来纠——才是治理。

这一章要把这两个词拆开。它们不是大小之分，是方向之分。

我们做三件事：第一幕看清”治理”这个词真正的源头——掌舵，不是统治；第二幕把它和最容易混的三个东西分开——不是管理、不是越早越好、不是例行；第三幕把它用起来——治理怎么运转（边界、

过滤器、三活动），以及怎么一眼认出”假治理”（术语膨胀与五问判据）。

---

## 这一章要回答什么

读完本章，你应该能回答：

1. 治理和管理差在哪？为什么说”管运营不是治理”？
2. “掌舵”这个词到底带出了什么？“有舵无桨到不了岸，有桨无舵原地转圈”——治理和管理为什么是缺一不可、授权与报告的纵向关系？为什么”统治”译不出”治理”？
3. 什么时候才需要治理？为什么五人团队引入正式治理委员会是”形式过早”？
4. 为什么说治理是持续系统、不靠事件启动？只开季度会的委员会，为什么可能是橡皮图章？委员会为什么是”过滤器”，而不是”处理器”？
5. 怎么一眼认出”假治理”？为什么”数据治理平台”常常是在干管理的活？
6. “框架坏了”和”框架不适合了”是两种什么样的失效？为什么”不适合了但还没坏”最危险？委员会的三活动（感知/定规/裁例外），缺了一环会先出什么症状？
7. 治理四问是什么？为什么说”谁来治、依何治、如何治、治得怎样”是任何治理都逃不开的框架，缺一问治理就悬空？

8. 为什么说”委托必有治理”？责任可以委托、担责不可委托（ISO/IEC 38500 §6.1）——下放责任之后，治理为什么反而更不可回避？
9. 为什么说”跨边界才是治理”？权力 100%、精力 1% 的处境，为什么恰恰是治理该长出来的地方？
10. 为什么治理层”永不碰桨”？委员会直接去设计系统、定具体方案，为什么反而越治越乱？

---

## 这一章的骨架

先把本章要钉的两个词、和它们牵出的三层立起来——后面三幕，都是给这层骨架添血肉。

> **治理 = 掌舵，管理 = 划桨**。治理问”这条船往哪开、由谁来掌、偏航了谁来纠”，管理问”怎么把事做好、做对、做快”——一个是决策系统，一个是执行系统。它们不是大小之分，是方向之分。

这一章的骨架是**一个分界 + 一把尺子 + 一个机制**：一个分界（掌舵 vs 划桨——决策系统与执行系统）；一把尺子（**治理四问**——谁来治〔角色〕？依何治〔规则〕？如何治〔机制〕？治得怎样〔担责〕？四问有答，才谈得上治理）；一个机制（**持续治理**——治理机构持续行使四活动循环，

评估持续进行、发现偏差即调整指导；定期评审只是传感装置，不是治理本身）。

> 术语注：**担责**是术语位（accountability 的定译），**问责**是动词位（治理机构对谁问责），**述责**是动作层（承担者把账摆出来）。三者不混用——本章说”担责不可委托”，说的是那项义务；说”向治理层问责”，说的是那个动作（§14.3.3）。

这一章从头到尾都在演示第 1 章的一个底层方法：**意义即用法——用”它做什么”定义”它是什么”**。不先问”治理是什么”，先问”治理做什么”：它掌舵，不是管事情。

第一幕钉”治理”这个词本身（掌舵——从词源、翻译到字源，再落到跨域与架构治理），第二幕把它和最容易混的东西分开（不是管理、不是越早越好、不是例行），第三幕回答”往哪去”（它怎么运转、以及怎么一眼认出”假治理”）。

---

## 14.2 第一幕 治理是什么（定界）

> 这一幕把”治理”看清：先立”用它能做什么定义它是什么”（意义即用法），再拆词源（govern=掌舵）与翻译史（统治→治理），然后看”治”与”理”两个字藏着的方法论（外导内顺），最后

> 落回它为什么能跨领域（治理四问——谁来治/依何治/如何治/治得怎样），并落到跨域最锋利的实例——架构治理（42020）。

### 14.2.1 先看病：我们买的，到底是舵还是桨

回到误解现场。恒信买的那套平台，清洗、去重、对账，样样能干——没有人怀疑它的本事。

徐漾怀疑的是另一件事：**这台机器干的活，和”治理”是同一件事吗？**

徐漾没问”治理是什么”，他问的是”治理做什么”。**用”它做什么”定义”它是什么”（意义即用法），是本书定义概念的底层方式——这一章从头到尾，都在演示它。**

要回答这个问题，得先弄清”治理”这个词从哪来。而这个词的源头，藏着第一个反转。

### 14.2.2 舵手不是船主：govern 的本义是掌舵

英文 governance 的词根，是希腊语的 **kybernân**（κυβερνᾶν）——本义是”掌舵、领航船只”。

这个词进入拉丁语成为 gubernare（引导、掌管），再进入英语成为 govern。

关键词是”掌舵”。请想清楚这个画面：**舵手不是船的主人。** 他不拥有这条船，不站在船头指手画脚，只是站在船尾，根据风向和水流调整方向。

船主的意志靠他实现，但实现的方式是”引导”，不是”命令”。

一个同根词把这个本义留了下来：**cybernetics（控制论）**。控制论的核心思想是”通过反馈调节方向”，而不是”通过强制实现服从”。

机器根据输出偏差修正自己的行为，就像舵手根据偏航修正航向。

这是 govern 语义基因的最佳佐证：**它天生是”纠偏”，不是”支配”**。

现在做一次思想实验。如果我们当年把 governance 译成”统治”——把”掌舵”译成”谁骑在谁头上”——会发生什么？

“统”是统一控制，“治”是治理，两个字加在一起，预设了一个高高在上的主权者，和一群被支配的臣属，靠强制力自上而下压下来。

一个本来在描述”怎么把船开好”的词，被塞进了一层原文根本没有的压迫色彩。

**翻译不是换一个词，是换一整套对世界的假设。**

### 14.2.3 从”治理危机”到”国家治理现代化”：一场有意的定译

governance 作为一个独立的政治概念，起点是 **1989 年世界银行的报告**《撒哈拉以南非洲：从危机到可持续增长》。

报告第一次提出”治理危机”（crisis of governance），并把 governance 定义为”行使政治权力以管理国家事务”。

从那以后，governance 迅速取代 government，成为国际组织话语里的关键词。

中文世界对这个词的翻译，走过四个关键节点：

```
1989 世界银行"治理危机" → 1995 首译"治道" → 2000 俞可平《治理与善治》定译 → 2013 国家治理现代化
```

> 术语注：时间线中”1995 首译’治道’“出自 1995 年论文《GOVERNANCE：现代”治道”新概念》（署名”知贤”）——学界公认的 governance 开山译作，可核；确切锚点是 1989 世行”治理危机”与 2000 俞可平定译”治理”。

2000 年，俞可平主编的《治理与善治》把 governance 定译为”治理”；2013 年，十八届三中全会提出”推进国家治理体系和治理能力现代化”。

“治理”从此进入中国官方话语的最高层。

俞可平说得非常清楚：强调”国家治理”而非”国家统治”，“不是简单的词语变化，而是思想观念的变化”。

他给出了统治（rule）与治理（governance）的五个实质区别：

| 维度 | 统治 rule | 治理 governance |
| --- | --- | --- |
| 主体 | 单一：政府公共权力 | 多元：政府 + 企业 + 社会组织 |
| 性质 | 强制、命令 | 协商为主、强制为辅 |
| 来源 | 国家法律 | 法律 + 契约 + 共识 |
| 向度 | 自上而下，单向 | 多向互动：上下 + 平行 |
| 范围 | 政府权力所及领域 | 整个公共领域（更宽） |

翻译选”治理”，就是在语言上完成这场切割——让 governance 在中文里拥有一个独立词位，和 rule 划清界限。

**这不是翻译的口味问题，是概念的分界线问题**。

### 14.2.4 “治”与”理”：治水疏导，治玉顺纹

翻译选”治理”，不只是为了避开”统治”，还借了中文古典的正资产。

两个字，各有来历。先给结论：**治理 = 外导内顺——治从外导、理从内顺**；下面拆开看。

**治——从治水来，核心是”疏导”**。《说文解字》里”治”本是一条河的名字，但这个字真正的生命力，来自它的字形：三点水。

中国人对”治”的全部想象，都来自治水。

鲧治水，用”堵”——九年不成，被殛于羽山；大禹改用”疏”——疏导河川入海，功成。

这是中国政治文明的第一课：**对付有活力、会流动的东西（水、人心、社会、市场），压制必败，疏导方成**。

“治”字从此带上这个基因：治国、治病、治学，全是”把一个失序的领域引回有序”。

它还能当名词用：治世对乱世，“天下大治”——治，本身就是秩序达成的状态。

**理——从治玉来，核心是”顺纹”**。”理”字左边的”王”，其实是”玉”（作偏旁时玉写作王，如珠、琳、琅）。

《说文解字》：“理，治玉也。”

玉不能像石头那样硬凿——那样会碎。匠人必须先读懂玉石内在的纹理脉络，顺着纹理下刀，才能剖出美玉。

所以”理”第一层是动词”治玉”，第二层升华为名词——**事物内在的脉络本身**：纹理、肌理、条理、道理。

这个词族的扩张壮观得惊人：

| 词 | 含义 |
| --- | --- |
| 地理 | 大地的脉络 |
| 物理 | 万物的脉络 |
| 心理 | 心的脉络 |
| 肌理 | 肌肉的纹理 |
| 天理（宋明理学） | 宇宙最高本体 |

一个从剖玉石来的字，最后统治了中国形而上学七百年。

“治”加”理”，合成”治理”：**外导内顺**。治是从外面疏导（对流动之物因势利导），理是从里面顺着（循内在脉络顺而治之）。

合起来的含义是——**干预必须遵循被干预对象的内在规律**。治水要懂水性，治玉要懂纹理，治国要懂民心，治数据要懂业务。

> 术语注：“外导内顺”（治从外导、理从内顺）与”干预遵循对象内在规律”，是本书对《说文解字》字源（治=治水、理=治玉）的凝练概括，不是古籍原话——字源公开，概括是本书的读法。

一个词，把”怎么干预”的方法论都写进去了。

### 14.2.5 为什么能跨领域：治理四问

现在回到那个最实用的问题：为什么”治理”能从国家一路用到数据、架构、流程？

因为”统治”锁死了一种关系——主权者和臣属。你只能说”统治国家”，不能说”统治数据”。

但”治理”不需要主权者，它只需要四样东西：

**谁来治（主体与角色） → 依何治（规则与标准） → 如何治（机制与流程） → 治得怎样（担责与结果）**

这四样，任何领域都有。治国家要回答这四问，治一张数据表也要回答这四问。跨领域时，“如何治（机制）”落到具体领域即显现为领域的差别——所以这四问落到具体领域，对应着角色、规则、机制、担责。

> 术语注：“治理四问”（谁来治/依何治/如何治/治得怎样）是本书对各类治理实践（ISO 37000、ISO 38500、COBIT、国家治理话语）的凝练，不是任何一份标准的原文——它是一把通用尺子，落哪个领域就换哪组对应词：谁来治=角色、依何治=规则、如何治=机制（落到具体领域即显现为领域的差别）、治得怎样=担责。

标准锚一：治理的权威定义在 **ISO 37000:2021 §3.1.1**——“一种以人为基础的系统——组织为实现其宗旨，通过该系统受到指导、接受监督，并被要求就其宗旨的实现作出说明、承担后果”。定义句里有四样东西，缺一样就不完整：**以人为基础**（human-based）说的是构成——这套系统由人的行为做成，不是文件堆、不是 IT 系统，所以译”以人为基础”不译”以人为本”（后者是价值观口号）；三**职能**（受到指导／接受监督／被要求作出说明、承担后果）说的是它做什么；**为宗旨**说的是它为了什么；“**被**要求”是承重墙——丢了它，治理就从”有一种外部力量把组织置于须交账的地位”退回成”组织自己要自觉”。

标准锚二：**ISO/IEC 38500:2024** 同口径——§3.3 把治理表述为”指导、监督和担责”三职能构成的系统（这里第三成分的英文是名词 **accountability**，定译取”**担责**”；“问责”留作动词位，§14.3.3 展开）；§6 给出四项活动：利益相关方参与→评估→指导→监督。三**职能与四活动不是一回事**——三职能回答”治理是什么”，四活动回答”治理怎么转”，§14.3.2 会专门划这条线。

治理四问是对这两条定义的凝练：谁来治=角色落点、依何治=指导的规则、如何治=三职能怎么落地成机制、治得怎样=三项治理结果的达成（下一段）。

再说一层：**治理的锚是宗旨，不是任何一条规则**。ISO 37000 给了十一条治理原则，分三层：**首要的一条是宗旨**（Purpose）——其他十条都宜在它的背景下理解；**基础的四条**是价值创造、战略、监督、担责；**赋能的六条**是利益相关方参与、领导力、数据和决策、风险治理、社会责任、长期生存能力和绩效。

十一条原则的读法只有两句：**宗旨是唯一的”首要”；全部原则都宜适用，而且同时适用**——治理不是一张打勾清单。

治理做得对不对，标准只认三个结果：**有效的绩效 / 尽责的管理 / 合乎伦理的行为**。“有没有董事会”“有没有制度文件”都不算数，“宗旨有没有真的在做决策时被引用”才算数。

所以宗旨还得过一道真伪关：**不能用来做决策的宗旨，不是宗旨，是标语**。恒信把”数据从此是资产”写进 PPT 那一刻，宗旨就已经被降级成口号了。

**从国家到数据表，尺度差了十个数量级，结构一模一样**。

全球治理是这套逻辑最好的试金石。政治学家罗西瑙（James Rosenau）1992 年有本书，书名就叫《没有政府的治理》（Governance without Government）。

地球上不存在一个能”统治”全球的世界政府，只有主权国家、国际组织、非政府组织、跨国企业，通过规则和协商协调事务。

说”全球统治”一句话就露馅了；说”全球治理”，精确成立。

一句话收住第一幕：

> **“统治”回答的是”谁骑在谁头上”；“治理”回答的是”这条船往哪开、由谁来掌舵、偏航了谁来纠”**。翻译选”治理”，就是把”掌舵逻辑”搬进了中文——这也是为什么从联合国到企业

> 数据中台，用的都是同一个词：因为它们做的，确实是同一件事的不同尺度版本。
> 一句话收住标准的定义：**治理就是以宗旨为中心的一套以人为基础的系统，通过”指导—监督—担责”这条不可断的链条，用方针与授权划出参数，用可追溯的决策与理由证明自己配得上信任**。

最后补一栏名字，免得后面读串：**“担责”是那项义务的名字**（对他人说明并解释责任履行情况的义务），**“问责”是治理机构的那只手**，**“述责”是承担者把账摆出来的动作**。本章说”担责不可委托”，说的是义务；说”向治理层问责”，说的是动作。

这一栏在中文里值得单独记一笔：**中文没有一个天然能与”责任”切开的 accountability 单词**——日语也没有，只好造了「説明責任」这个复合词。所以这一对概念的分界，不能靠词形承载，只能靠定义句和第一次出现的括注承载。

> **概念卡片 · 治理**

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 由被治理对象之外的人或机构，依据规则与担责，对”方向与框架”持续纠偏——掌舵，不是统治；回答”谁来治（角色）→依何治（规则）→如何治（机制）→治得怎样（担责）“。属是 system、种差才是全部信息：**以人为基础**（要素是人，不是文件堆、不是 IT 系统）+ 三职能 + 为宗旨 |
| **它不是什么** | ⑥⑦ | 统治（rule）——主体单一/强制命令/自上而下（统治）vs 主体多元/协商为主/多向互动（治理），后者预设治理四问（谁来治/依何治/如何治/治得怎样）、治理（偏[illegible]）、依[illegible]（治理[illegible]） |
| **它[illegible]来** | ④⑤⑨<br>820 | [illegible]治理[illegible]<br>[illegible]治理[illegible]<br>[illegible]治理[illegible] |

### 14.2.6 治理落到架构域：架构治理

治理能跨域，最锋利的一个例子在架构域——上一节说治理四问是通用尺子，这一节看它落到架构域长什么样。组织级的架构治理（ISO/IEC/IEEE 42020:2019 §6.1 架构治理流程）目的只有一句话：**建立并保持架构集合中的架构，与企业目标、政策和战略一致**。

这句话是标准的原文照译；正本句尾还有一句，一并补上：and with related architectures（**以及各相关架构之间的一致**）——这一句在转述里最容易被漏掉，而它管的恰恰是架构集合内部的横向一致。治国家要回答四问，治”所有系统的架构”也要回答四问——领域换成架构、规则换成架构原则、角色换成架构师、担责换成符合性监控。

先按上一节那条定义拆一遍：架构治理的四要素是**主体**（治理机构及其授权的架构委员会）、**对象**（架构决策／决策遵从／架构演化／架构资产）、**手段**（规则、评审、契约、例外、监视）、**目的**（保持架构与宗旨的一致）。**四要素缺一即不成立**：无主体则担责落空，无对象则治理变成泛泛管理，无手段则治理只剩口号，无目的则治理退化为流程合规。

先回答”治理到底治什么”。架构治理守护的不是”文档”，是一条**不变量链**——「目的 → 不变量 → 根本决策 → 遵从与演化」，四层：

| 治理对象 | 具体内容 |
| --- | --- |
| 架构决策 | fundamental 的概念与属性是否仍与目的一致（决策对不对） |
| 决策遵从 | 实现/设计是否偏离已确立的基本概念或属性与管控原则（有没有被执行） |
| 架构演化 | 变更是否受控、不变量是否被守住、变体是否在允许范围内（怎么变） |
| 架构资产 | 架构描述、架构视角库、可追溯链是否完整可信（决策的载体） |

> 术语注：「架构视角库」里的**架构视角**，是 42010:2022 §3.8 说的一组惯例（用来创建、解释与使用架构视图以框定关注点的那套规则）；惯例记载在各视角自己的**规格**里，所以视角有版本、可入库复用（§3.8 注 3）。架构视角是**规则**不是产物——“库”里存的是规格，不是视图；视图按 §3.7 是架构描述的一个**信息部件**。

三大支柱在这条链上各就各位：**架构委员会守第一层**（决策对不对），**合规评审守第二层**（有没有被执行——活动性质属验证：对照已确立架构查符合性，评审是证据手段；此处取机制名义），**架构合同守第二、**

**三层的契约化**；支柱之外，还有一道把链冻住的机制——**基线与变更控制（ISO 10007）守第三、四层**（§13.4.4）。

这里的”不变量”，就是第 11 章说的基本概念——对不变性的承诺（§11.3.6）；它由**目的**裁断（§10.2.4 的目的判据）。目的变了，不变量集合就过期。

顺着这条链，“为什么治”也清楚了——三种侵蚀力量，每一种都打在链上：① **局部优化侵蚀全局根本**——每个单步都”情有可原”，累积成架构腐化（不变量不会自己守住自己）；② **目的漂移使判据失效**——目的悄悄变而架构没跟上，fundamental 集合整体过期、无症状；③ **决策不可逆放大错误成本**——治理把错误拦在便宜的时候。**不变量不会自己守住自己，目的不会自己宣告变更——这就是架构治理存在的原因**。

在这里，治理的方向权与内容权分开了：**方向权（目标、组织政策、战略）属组织层，内容权（原则、架构政策、指令、符合性判断、决策依据）属架构师**。所以架构师是治理在架构域的”技术权威与执行主体”。

顺着方向权再往下分一层：方向权的实质是**目的的所有权**——系统目的与任务的定义权和担责在业务方（赞助人/产品管理；15288 业务或任务分析流程），架构师是目的的”理解与守护者”：对目的有**质询权**（用可行性、成本、一致性反哺），**没有改写权**。

这层所有权落到纸面，就是业务方签字生效的《系统目的或任务理解确认单》；目的变更由业务方行使，架构师负责触发重审（§14.3.7）。

他产出的不是某个系统的开发文档，而是一整套组织级、跨项目跨版本的产品：架构原则、政策与治理指令、架构委员会章程、治理决策记录（带理由）、合规评估报告、架构健康度指标……**系统级文档是”作品”，这些制度是”船规”。**

治理与管理的分界在这里也照搬了 42020 的分法：**架构治理适用于整个组织**（设定方向、评估、监控符合性），**架构管理适用于架构集合**（实施治理指令）。

这句话在正本里是两处注，两处同文（§6.2 注 2 与 §7.2 注）：

> Architecture governance outcomes are applicable across the organization while architecture management outcomes are applicable across the entire collection of architectures.

说的是两者**结果**的适用范围；§5.3 另有一句说治理过程”适用于组织内的全部架构工作”。

域内还要再分一次——**产品架构治理与企业架构治理，不是同一种**。产品架构治理的对象是产品（硬件／软件／接口／构型），服务的是产品线的使命，**有时可以落在一条产品线内**：裁决权就在一个总师体系里，管理就够。企业架构治理的对象是企业四域（业务／数据／应用／技术），服务的是组织整体的宗旨与战略，**天然跨部门**——没有哪个部门能裁决另一个部门。

但架构治理真正的分量，不在范围大小，在**决策的长期承诺**：**换一颗芯片与调整一次架构，差别不在这次花多少钱，而在把未来十几年**

**的可能性框定了**。所以架构决策必须由治理层作、必须留下理由——不是因为更难，而是因为**它的后果比决策者的任期还长**。

顺着这条线再问一层：**企业架构（这里指企业运营体系架构）为什么必须治理？**追问到根上，是一条五环因果链：**企业战略随环境调整 → 运营体系目的承接战略随之改变 → 架构由目的评判（目的一变，“基本”的标准就换）→ 运营体系在演化运行中也会偏离目的、架构不会自纠 → 目的的所有权在治理机构、不在执行者手里**。

五环每一环都指向同一件事：架构与目的一致，不会自己保持——必须由治理机构对架构持续行使指导、监督、担责，把它保持在目的上。治理的目的，就是让架构持续符合运营目的。落到组织上：治理机构之下设架构委员会，架构师在其中开发与维护架构、做符合性判断、**向治理机构报告偏差**；架构师不定义运营体系目的——目的归治理机构。

而且架构治理不是架构师的额外职责：42010:2022 §3.1 给架构师工作（architecting）下定义时，原文就写着 certifying proper implementation of, maintaining and improving an architecture（认证架构的恰当实现、维护与改进架构）——前者就是符合性监控（治理），后者就是度量与演化治理。第 18 章会讲，架构师把这份制度背在身上，是什么样子。

回头再看这条定义——整条链是七动词：构思、定义、表达、记录、沟通、**认证恰当实现、维护与改进**（42010:2022 §3.1）。治理用到的，正是链尾这两项：认证 = 符合性监控（查实现符不符合架构），维护与

改进 = 演化治理（对抗架构腐化）。前三个动词展开在第 11 章，记录与沟通、以及整条链作为能力的母词在第 18 章——治理不是另起炉灶，是这条链在组织级的制度化（本书凝练）。

## 案例进展①｜恒信：五百万买来的，是舵还是桨

先回到恒信那五百万。委员会第一次会后，徐漾把那份五百万的采购合同带回了工位，一页页翻。

他圈出平台的功能清单：数据清洗、去重、补全、血缘分析、对账报告。

万辉凑过来：“你在找什么?”

“我在找一样东西。”徐漾说，“功能清单里有一行，写’清洗规则可由管理员配置’。你注意——是’管理员配置清洗规则’。”

可我想找的，是’谁有权定义合同编号的标准、财务和法务谁对齐谁、标准变了谁担责’——这几个问题，清单里一个都没有。

万辉想了想：“那是制度问题，平台解决不了。”

隔壁工位的小白搭了句话：“平台那套清洗规则，我调过接口——配置项全是’怎么洗’：按什么格式、去重、补全。没有一项是’洗完之后听谁的’。”

“对，平台解决不了。”徐漾放下笔，“可我们的委员会也解决不了——季度会议开了四十五分钟，议程里全是’清洗了多少条’，没有一条是’该裁决什么’。”

**我们把治理委员会，开成了清洗工具的周报会；把五百万，花在了桨上。舵，还是没人掌。**

“那’舵’是什么?”

徐漾把掌舵的比喻推给他：“你想——一条船，方向偏了，谁来纠?不是划桨的人，是掌舵的人。”

我们三个系统对不上的合同编号，就是船偏了航。划桨的人（平台）能多划几下、把数据洗得更顺，但方向为什么偏、往哪纠、按谁的规则纠——这是掌舵的人（委员会）的活。

**可我们委员会一年四次坐在船上开会，一次也没碰过舵。**

万辉沉默了一会儿，说：“所以五百万买的是桨，而且我们还把它叫成了舵。”

---

## 14.3 第二幕 治理不是什么（磨边界弧）

> 这一幕把”治理”和它容易混的东西分开——三条边连续磨过去，每条都靠反例定型：不是管理（掌舵 vs 划桨）、不是越早越好（三条件）、不是例行（持续系统、两种失效）。

### 14.3.1 先看病：我们做的”管理”，为什么不是”治理”

第一幕说清楚了”治理”是什么。但恒信的麻烦才刚开始。

因为真正难的不是定义，是**分清治理和它最像的那个邻居：管理**。

误解现场那句话值得再拿出来：平台清洗数据，是管理；谁有权裁决跨部门标准，才是治理。

很多人听到这里会说：“这不是一回事吗？管好数据不就是治理数据吗？”——这句话，正是这一章要连根拔起的第二个误解。

### 14.3.2 掌舵与划桨：决策系统与执行系统

治理和管理的分野，公司治理学的奠基人 Bob Tricker 1984 年用一句话划清楚了：

> “If management is about running the business, governance is about seeing that it is run properly.” 管理是经营企业，治理是确保企业经营得当。

他还用了一个贯穿全书的比喻：**governance holds the rudder（治理掌舵），management holds the oars（管理划桨）**。

顺着这个比喻想，就是一句耐嚼的话：**有舵无桨，到不了岸；有桨无舵，原地转圈**。治理和管理，缺了哪一个，另一个都走不远。

顺带把”管理”这个词也拆清：ISO 9000 把管理定义为”指挥和控制组织的协调活动”；management 来自拉丁语 manus（手）+ agere（驱动），意为”用手驱动”——和划桨的 oars 同义基因，亲手把事做成。它跟两个词要分清：**管理 ≠ 统治**——管理对物、事、流程下手，统治对人、权力下手；**管理 ≠ 运营**——运营偏日常运行维护，是管理的一部分（第三幕伞结构里看得更清）。

治理和管理都有”谁管、管什么、依据什么、怎么管、监督问责”五个要素——所以它们的区别不在”有没有要素”，而在**要素的层次和权力关系**：

- **治理关注决策系统**——解决”什么事情值得做、谁有权决定、谁担后果”；
- **管理关注执行系统**——解决”怎么把事情做好”。

用 ISO/IEC 38500 的说法，治理层持续行使的是 §6 模型四活动——**利益相关方参与**（engage stakeholders）→ **评估**（evaluate）→ **指导**（direct）→ **监督**（monitor）；管理层跑的是 **PDCA 循环**（计划

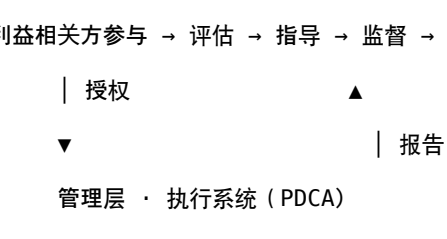
Plan → 执行 Do → 检查 Check → 改进 Act，属 ISO 9000 族执行循环）。

这里要辨清两件事。**第一，定义与执行不是一回事**：**定义看 §3.3 三成分**（指导／监督／担责，回答”是什么”），**执行看 §6 四活动**（利益相关方参与／评估／指导／监督，回答”怎么转”）。担责是贯穿性职能，不单列为循环里的一步。

**第二，中文”监督”一个词接了两个英文词**：**oversight**（治理机构查组织表现是否符合**自己设定的期望**，含 assurance——确信治理系统本身设计得当并按预期运行）与 **monitor**（通过测量获取绩效与进展信息、周期性核对）。两者是**包含关系**：oversight ⊃ monitor。同一句里要分开时，写”监督（oversight，含 assurance）“对”监测／复核（monitor，测量层面）“。

**评估与监督还各占一个时点**：**评估判断”该不该”（批准之前），监督复核”是不是”（批准之后）**。评估看向外与向前，监督看向内与向后。

两层之间不是并列，是**授权与报告**的纵向关系：




最要紧的是那句”治理是对管理本身的管理”。治理不是”另一种管理”，它**定义管理的边界，然后监督管理有没有在边界内好好干**。

管理者自己定的规则，自己执行，自己检查——这套闭环里的每一步，都是管理；而”规则该不该这样定、边界划得对不对、管理者有

没有越界”，是管理自己回答不了的问题，得有人站在管理之上问——那就是治理。

> **分界判据**：把这一节的比喻（掌舵/划桨、决策/执行、边界/框架）收成一句话——**治理是跨层级的远程校正机制，专职守护不变量（标准、原则、基线）；管理只在框架内调变量、达成运行目标**。治理与管理之别在职能层级与对象（治理定框架、管理在框架内执行），不在触发方式——两者都持续运转（§14.3.6）。掌舵划桨是画面，守不变量、调变量是实质：§14.3.4 的五要素表、第三幕的伞结构，都是这句话的展开。

最后给”掌舵的这套东西”配一张零件表。标准没有列过它，下面这张是**从标准内容推导的**——读表、建表、给人家的治理开诊断单，都用得上：

| 类 | 元素 | 它回答 |
| --- | --- | --- |
| **主体** | 治理机构、治理小组、人员、利益相关方 | 谁 |
| **意图** | 组织宗旨、组织价值观 | 为了什么 |
| **规矩** | 治理方针、合规义务 | 依据什么 |
| **结构** | 决策结构、委员会、角色与权限、报告要求 | 怎么搭 |
| **关系** | 授权 → 报告 → 担责；利益相关方参与 | 怎么连 |
| **行为** | 领导力、合乎道德的行为 | 怎么动 |
| **保证** | 保证过程、审计、“使自己确信”的机制 | 怎么确信 |

三点读法。① **原则不是要素，结果也不是要素**——十一条原则是”按什么规范搭”，三个结果是”跑出什么”，塞进同一张表，这套零件就散了。

② **“保证”这一类最容易被整个跳过**。治理机构不只要看报表，还要**确保内部控制体系被建立**，并**使自己确信治理系统被恰当设计、按预期运行**——工具是内部审计与独立审核。只查业务、不查治理本身，等于一把尺子量别人、不量自己。

③ **这张表是诊断单**。拿恒信逐行过一遍：主体有（委员会）、意图有（数据是资产）、规矩有（清洗规则）、结构有（季度会）——**关系断**（没有授权、没有裁决权）、**行为空**（没人敢越过部门说话）、**保证无**（没人确认过这套治理本身转不转）。前四行都在，后三行全空，合起来就是那四十五分钟的季度会：**零件买齐了，传动轴没装**。

### 14.3.3 委托必有治理：责任可委托，担责不可委托

讲到这里，一个该问的问题浮出来了：**治理到底管什么，为什么组织离不开它?**

先把问题的起点补上：为什么会有治理？——因为**委托**。合作规模一旦超过单个头脑能看管的上限，委托就是唯一选择：老板一个人看不完所有事，只能把具体责任下放给管理层，管理层再授权给下一层。不委托，组织长不大。ISO 37000 §3.2.4 给”授权”下的定义就是这个动作：把权力和责任从一方分配给另一方；ISO/IEC 38500 §7.2.5 说的是同一条实践：治理机构把具体责任下放给管理层。

但委托不等于甩手。标准的另一半是 **§6.1：担责不可授权**——治理机构可以授权他人，但仍对整个组织的作为、不作为和渎职负责。

**责任可以委托，担责不可委托**。下放后的执行一旦脱离指导、监督、担责，组织目的就无法保证；而最终担责仍落在治理机构。所以委托必有治理：由治理机构对被下放的部分持续行使指导、监督、担责（§3.3）。

**担责这条义务长什么样？** ISO 37000 §3.2.2 把它定义为：**对他人说明并解释责任履行情况的义务**——注 1 说这个义务包括**告知和解释**履行的方式，注 2 说**不履行责任的后果可由责任方承担**。

中文最容易丢的不是后半句，是前半句。把这条义务拆成三个部件就看清了：

| 部件 | 它是什么 | 落在中文哪里 |
|---|---|---|
| **held** | 被动，且有人持有——有人抓着你 | “**被**要求” |
| **-able** | 一种**状态**，不是行为、不是美德 | —— |
| **account** | 账（不是”错”） | “**说明**”＋“**承担后果**” |

所以一句要紧的话：**担责不是你的品格，是你的处境**——你被放在一个”随时要交账”的位置上。**“held”这一半是中文最容易丢的**——“担责”“履责”这两个词的语法主语都是被问责者本人，那个”有人抓着你”的位置就在词形里消失了。

**它的核心动作是把账摆出来，不是低头认错**。标准给治理机构列的披露清单里，最硬的一条是”**解释和证明**”——举证责任在治理机构这一侧，而且**失败也要列进披露范围**（含不履行责任的后果），方向还是**双向的**：治理机构既对整个组织负责，也对授权给它的人负责。

那”后果”呢？它没被删掉——**惩罚是条件 e 的保证机制**（见下一节授权五条件）。两者分工很清：**平时让治理保持在”随时交得出账”的状态，出事时后果才落得下来**。把担责做成追责，第一后果是信息

消失（坏消息不上报），恰好摧毁它要建立的信任——标准给担责记的产出是**信任与合法性**，不是惩罚。

注意：委托不是治理的前提。**无委托时，治理同样是组织常态（§3.3）**——同一个人兼任治理与管理时，两个角色仍必须分清（这正是这一章从头到尾在做的分界）。委托只是让治理成为不可回避的必需，而不是治理的开端。

这正是为什么恒信那三个系统对不上的编号，靠"多清洗几遍"永远解决不了——清洗是数据部的执行（管理），"合同编号以谁为准"的最终担责却没人接住：集团把数据管理下放给了数据部，但裁决标准的担责没有落在任何具体的人或机构上。**该下放的责任下放了，不该下放的担责悬空了**。

把这一节收成一条链：**规模 → 委托 → 担责不可委托 → 治理**。规模迫使委托，委托让担责悬空，悬空的担责只有治理能接住——治理机构对被下放部分持续行使指导、监督、担责，让"老板不在场"不至于变成"目的不在场"。

再往深一层看，治理守护的本质是什么？**治理是目的的"远程存续机制"——具体责任下放了，但目的的所有权不能下放**。原则库把目的冻成规则，基线把目的冻成参照，评审把目的请回现场，追溯矩阵让每个决策都能被目的认领。第 13 章的基线、第 8 章的评审，都是这只"目的在场"的具体形态。

这句"基线把目的冻成参照"里，藏着一个第 13 章已划开的分级：目的走的基线和需求走的基线，**不是同一条**。目的（连同架构原则、治理红线）进的是更高、更少动的那条——**治理基线**：由治理层（企业/

战略）批准，一变就是”系统该不该存在”的危机，而不是一次规格调整。

在这条轴上，**目标**（需供双方约定的成功判据）在中间进**协议级**——双方共同确认、变更要重谈承诺；**需求及以下**才进产品 CM 基线——CCB 批准、CCB 变更，是正常工程动作。

> 术语注：「**治理基线」是本书梳理主张**——ISO 10007 只定义了”基线”，没有”治理基线”这个标准词；本书把”基线”按”谁批准、谁变更、变了意味什么”再切一层，分出治理级（目的/原则/红线）、协议级（目标/成功判据，需供双方约定）与产品 CM 级（需求/设计/实现）。它与第 13 章的功能/分配/产品基线（按生存周期成熟度切）正交，不是一回事。

### 14.3.4 同一个要素，两个层次

现在把治理和管理放在同一张表里，看同一个要素在两个层次的回答完全不同：

| 五要素 | 治理层（决策系统） | 管理层（执行系统） |
| --- | --- | --- |
| 谁管 | 治理机构（董事会/委员会/理事会） | CEO/经理/团队负责人 |
| 管什么 | 方向、规则、边界 | 执行、运营、交付 |
| 依据什么 | 章程、法律、战略意图、利益相关方期望 | 治理层制定的政策、标准、SOP |
| 怎样管 | §6 四活动循环：利益相关方参与→评估→指导→监督 | PDCA 循环：计划→执行→检查→改进 |
| 监督管理 | 监督管理层，对授权给它的人与整个组织担责 | 监督执行层，向治理层担责 |

这张表最关键的是第三行：**管理的”依据什么”，恰恰是治理的产出。**

治理层制定政策、标准、边界框架，管理层在这些框架里做事。

于是形成一条清晰的权力链：

**所有权 → 治理 → 管理 → 执行**

**（定方向）（定框架）（干执行）（动手做）**

顺着这条链，恒信的问题第一次可以被精确地说出来：那三个系统，是”执行”和”管理”的事；“合同编号以谁为准”是”治理”的事——它要裁决标准、担起担责。

恒信把这一层整个缺掉了：所有权在集团，管理在数据部，执行在平台，**中间没有治理层，没有任何人有权裁决”标准”**。

工程侧的职责，在这条链上各占一席：**系统工程师**对系统成败负**终责**（accountability，不可委托——对外扛雷的账他背）；**系统架构**

**师**对**架构的成立与守持**负**专责**（responsibility，可委托可追责——工作可以委托出去，责任仍是自己的）。

**模块架构师/模块工程师**（通常同一个人或团队）在模块内自治、边界外听架构的。

委托—协商—验收贯穿：系统架构师把模块架构委托给模块架构师，系统工程师把架构正确性委托给系统架构师——**拍板权在系统工程师，理由权在架构师**。

### 概念卡片 · 管理

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 在既定框架内，把事做好、做对、做快的执行活动——划桨（ISO 9000 定义：指挥和控制组织的协调活动） |
| **它不是什么** | ⑥⑦ | 治理——治理管”做什么、该不该做”（决策系统，掌舵）；管理管”怎么做”（执行系统，划桨）；管理在框架内，治理管框架本身；管理 ≠ 统治（管理对物/事/流程，统治对人/权力）；管理 ≠ 运营（运营偏日常运行维护，是管理的一部分，见第三幕伞结构） |
| **它从哪来** | ②③⑨ | 管理随公司制/职业经理人阶层的兴起而独立成域——职业管理者接住所有者授权，专职把事做好（management） |
| **它到哪去** | ④⑤⑧ | 这四章是在为治理就位铺路：先把框架搭对、搭牢，治理才有意义 |

## 案例进展②｜冰链镜照：这不是”技术问题”

几乎同一段时间，冰链在做一场事故复盘。

第 13 章那场”断链追溯查不清”之后，水忠把固件升级流程重建了一遍：影响评估、CCB 批准、版本台账、配置审计，全配齐了。

这次复盘，他觉得自己可以理直气壮地收尾：“技术问题，修了；流程漏洞，补了。这事翻篇。”

文正问了一个他不爱听的问题：“水忠，你说’技术问题’。我帮你说完整——传感器在 -20℃ 丢读数，是硬件和固件的 bug，这叫技术问题；追溯查不清，是管记录的环节漏了，这叫流程问题。这两条，你都修了。”

“但第三个问题，没有人问过：**温度采集从固件到云端到追溯平台，中间的数据所有权、变更边界，到底谁说了算？**”

水忠一愣：“采集逻辑是我的，云端是董勐的，追溯平台是云端的，App 是……各管各的呗。”

“各管各的——这句话就是问题。”文正说，“第 12 章你拦过董勐，说他不能绕开追溯域给客户开直连。你拦下来了，可那条数据所有权的铁规矩，执行全靠你这个人——**哪天你不在，谁来拦？凭什么拦？**”

这次事故里，固件改了采集算法，没有一个人有权力站出来说’采集算法是追溯数据的地基，不能这么改’——大家都觉得’那是水忠的事’。

技术 bug 是这次事故的引信，但让引信能点着的，是没人掌舵。

水忠沉默了。他第一次意识到：他修好的那个流程，是划桨的流程；而”谁有资格定义数据边界、谁来裁决跨域变更”——那个他没修的东西，才是让这场事故根本不会发生的那层。

“技术问题”这个词，把一件**框架级**的事，归类成了**执行层**的事。这一归类，就没人再往上面看一层了。

### 14.3.5 不是越早越好：治理三条件

案例看完，一个现实问题冒出来：**是不是所有团队都需要正式治理机构？**冰链那次对话里，水忠真正想问的是”我们这种团队，值得为一个’框架’较劲吗?”

答案不是”是”。治理不是越早引入越好——恰恰相反，**很多团队引入正式治理，是形式过早**。

判断要不要引入正式治理机构，有一套三条件模型：三**个条件同时满足才需要，任一缺失就不必设**。

条件一：分离　　所有者 ≠ 管理者（出钱定方向的人不管日常）　┐

条件二：静默积累　　偏差不会立刻爆雷，而是悄悄积累　　├→ 三个都满足 → 需要正式治理机构

条件三：框架级　　问题出在框架本身，超出任何管理者权限　┘

任一缺失 → 不必设正式治理机构

**条件一，分离**。出钱定方向的人，和管日常的人，不是同一拨。

五人创业团队、个人工作室，老板自己定方向自己管日常，没有分离，不必设正式治理机构。

**条件二，静默积累**。偏差不会立即爆雷，而是悄悄积累。安然、雷曼的偏离都”安静地积累”了几年才爆。

如果偏差每次立即可见（实时告警、瞬时故障），管理自己的反馈环就能抓住，不必设正式治理机构。

**条件三，框架级**。问题出在框架本身，不在执行层。方向偏了、标准过时了、原则冲突了、权限越界了——这些超出管理权限，因为管理只在框架内部干活，改不了框架本身。

一个 bug、一次部署失败、一次需求延期，是执行层问题，管理权限内就能解决。

条件一和条件二合击的效果最危险：出钱定方向的人看不到（责任被委托在下层），执行的人不觉得有偏（且偏差太慢），偏离就安静地积累下去——直到治理的传感装置检测到，或者直到爆雷。

三个条件全满足时，局面是：**有人需要从外部、在管理层权限之上、对框架本身纠偏——这就是治理。**

**授权要有效，还得过五道关**（ISO 37000 §4.2.2）。这五条是担责能不能落地的先决条件，缺哪条，担责在哪条上就变味：

| 条件 | 不成立时变成什么 |
| --- | --- |
| a) 预期结果经协商、明确、达成一致 | **事后加罪**——“没人说过要这样” |
| b) 所需资源可使用 | 有责无资源 |
| c) 权力与责任层级匹配（含自主权） | **有责无权** |
| d) 定期报告产出、结果、过程，并**举证**行动合理适当 | 只看结果，不看依据 |
| e) 不履行或不守界限的**后果可执行** | **无牙之担责** |

配一条禁令：**任何人都不宜为他们无权处理的事项负责，也不宜为他们没有提出或商定期望的事项负责。**

条件 c 是这一章那条分界的背面。**有责无权，等于没责**：让一个团队对数据质量负责，却不给它裁决跨部门标准的权力，那个责任从第一天起就是空的。反过来也散架——**有权无责，等于悬空**：集团手里握着”以谁为准”的裁决权，却从没把它分给任何人，于是那三个系统的编号永远对不上。

用五组场景验证这个模型，全部成立：

| 领域 | 分离 | 静默积累 | 框架级 | 治理机制 |
|---|---|---|---|---|
| 国家 | 权力属人民但由官员代行 | 权力膨胀、腐败 | 宪法失效 | 宪法法院、选举 |
| 组织 | 资本属股东但由 CEO 经营 | 战略偏移 | 内控失效 | 董事会、审计 |
| 流程 | 标准由设计者定、执行者走 | 流程僵化 | 流程原则过时 | 委员会、豁免审批 |
| 架构 | 原则由委员会定、项目组写码 | 技术腐化 | 架构标准失效 | 合规评审、架构合同 |
| 数据 | 规则由所有者定、操作者用 | 数据孤岛 | 数据权属失效 | steward、质量审计 |

形式各异，内核相同。把表拉到恒信：法务、财务、业务三个部门，标准各自定、数据各自用——**分离**；

三个系统的编号对不上，不是一天爆出来的，是积了半年——**静默积累**；“合同编号以谁为准”，没有哪个部门负责人有权拍板，要超越所有部门——**框架级**。

**三条件全满足，恒信需要治理**。但它买的平台、开的季度会，没有一个在治理。

### 14.3.6 不是例行：治理是持续系统

三条件回答”要不要治理”，这一节回答”治理怎么转”。这也是这一章最容易被委员会开成橡皮图章的一条。

先看管理。管理是**持续系统**：在治理划定的框架内，按 PDCA（计划→执行→检查→改进）持续达成运行目标。周例会、季度计划、发布排期——日历说该做了就做，管理从不设”启动”，它是常态。

治理同样是持续系统。ISO/IEC 38500 §6 模型把它展开为四活动循环：**利益相关方参与**（识别并参与利益相关方）→ **评估**（考虑并作出有依据的判断）→ **指导**（传达期望的目的与结果）→ **监督**（复核，作为适当决策与调整的依据）——循环往复。**治理不靠事件启动：评估持续进行，发现偏差即调整指导**。所谓”启动”，只是评估发现偏差后的一轮处置，不是治理要等一件事发生才从零开始。

所以治理与管理的分界，不在触发方式——两者都持续运转，标准也不设”启动”——而在**职能层级与对象**：治理定框架（定方向、定规矩、监督管理），管理在框架内执行（达成运行目标）。这正是 §14.3.2 分界判据的直接推论。

那”只开季度会看报告”的委员会，为什么还是橡皮图章？因为治理的持续运转被简化成了”到点开会”——只走了四活动里监督/评估的一个常规载体（看报告），评估发现偏差后从不启动”指导”的调整。**传感是治理的一环，不是治理的整体**。把传感当治理本身，等于装了烟雾报警器，就以为消防队已经出动了。

一句话区分：**治理是持续系统，定期评审只是它的一只传感器——传感在例行地跑，传感发现偏差后的那次处置，才是治理真正在动**。只开季度会、开了也不处置，就是橡皮图章。

这里，第 8 章立过的一个东西正好派上用场——复盘。第 8 章说过，复盘（retrospective）不是评审：评审管把关，复盘管改进，它绕回起点，把上一轮的教训变成下一轮的目标。

现在把它接上：**复盘是例行的——定期开；但复盘只负责"发现"，它是传感装置。真正把复盘变成治理的，是复盘发现偏差之后，评估据此作出的那轮处置**。

一个扎心的推论，值得每个委员会主任刻在桌上：**如果一个治理委员会只在季度会议上工作，它把持续系统开成了例行会议——它就是橡皮图章**。治理需要双载体：定期评审（监督/评估的常规载体，捕捉缓慢漂移）+ 临时召集（评估发现突发偏差后的处置）。只有前者没有后者，评估发现了偏差也不处置，就是橡皮图章。

### 14.3.7 两种失效：坏了要修，不适合了要演

治理要盯的"偏差"，不止一种。区分它们，比想象中重要——因为治理对它们的发现方式和响应方式完全不同。

**"坏了"是内部失效——框架自身有缺陷**。例如：

- 架构原则互相矛盾（"买不造"和"避免供应商锁定"同时存在，却没说明怎么调和）；
- 数据标准同一个字段三个系统三种定义；
- 流程审批出现循环依赖。

框架的内部逻辑断了。它有症状：矛盾、冲突、故障、违规——评估在红旗亮时就能抓住，治理的响应是**修**：修复缺陷，恢复一致性。

**“不适合了”是外部过时——框架没错，但世界变了。**架构为单体设计，业务却要微服务化；数据治理为结构化表格设计，AI 来了要管非结构化训练数据；流程为瀑布设计，组织却转了敏捷。

架构域里，“不适合了”最典型的成因是**目的变了**——分两种扳机。

**目的变更**：战略调整、产品定位改变，是显式发生的强制扳机——fundamental 的判据本身换了，一旦发生必须强制触发架构重审，因为不变量集合整体过期。

**目的漂移**：目的悄悄变而架构没跟上——产品定位从”专业工具”滑向”大众应用”、业务方在评审里一点点改口径。它没有症状、事后才发现，只能靠定期评审**以目的为参照**检视。

一个显式、一个无声，正好是持续治理里”评估”要接住的两种偏差：目的变更靠评估以显式扳机抓住、强制重审；目的漂移只能靠评估以目的为参照的定期检视发现。

目的漂移所以最危险，在于**目的漂移就是同一性漂移**——人造系统的同一性是目的论的（同一性目的论，见附录 A T-144），目的悄悄换了，“它还是不是它”已经换了答案，架构却还在服务旧目的：一切按旧标准看都合规，判据却整体过期。所以目的漂移是**架构失效的第一病因**（§10.2.4）。

框架的内部逻辑没断，但它在解决昨天的问题。它**没有症状**——一切按旧标准看都合规。

只有定期评审以目的为参照回头看，才抓得住，治理的响应是**演**：更新框架，适配新环境。

| | 坏了（内部失效） | 不适合了（外部过时） |
|---|---|---|
| 问题在哪 | 框架自身 | 框架与环境的匹配 |
| 有没有症状 | 有——矛盾、冲突、故障、违规 | 没有——按旧标准全合规 |
| 治理怎么发现 | 评估在红旗亮时抓住 | 定期评审以目的为参照检视 |
| 治理怎么响应 | 修——修复缺陷 | 演——更新框架 |

两种失效经常连锁成一条死路：

**环境变了 → 框架"不适合了"（无症状）→ 团队开始绕过 → 框架"坏了"（有症状）→ 爆雷**

**↑ ↑**

**定期检视·定期评审（早拦截） 红旗评估·红旗（晚拦截）**

逻辑是这样的：当框架不适合新环境，团队按框架做就会慢、会错、会做无用功。没人想故意违规，但为了把活干完，大家开始”灵活处理”——这次跳过评审、那次用临时方案、下回绕过标准。每一次绕过都”合理”，但这些绕过会积累成框架的内部损伤。**“不适合了”最终导致了”坏了”**。

最危险的状态，是”不适合了但还没坏”：没有症状（红旗不会响）、管理看不出来（管理在框架内部，看不到框架与环境的匹配度）、团队不会主动报（每次绕过都是”临时处理”）。

这个状态只能被一件事发现：**定期评审时，有人从外部视角问一句”我们的框架还在解决今天的问题吗？“**——这就是持续治理里监督/评估这只传感器存在的全部理由。

第 8 章的复盘，就是这个传感装置之一：它定期回头看，把”坏了”（红旗）和”不适合了”（过时）都翻到桌面上来——但它只负责”发现”。

发现之后怎么处置，才是治理的事。**传感是例行的，处置永远不是例行的——这正是上一节那条线的落点**。

> 一句话：**坏了要修，不适合了要演，治理两个都要管。定期检视抓”不适合了”（早拦截、无症状），红旗评估抓”坏了”（晚拦截、有症状）**。

目的漂移/目的变更被评估发现之后，接下来”怎么办”——那是第三幕的事：治理层评估、裁决、重审判据，执行层才动手改架构。§14.4.4 用一个判例，把这串动作逐段拆给你看。

## 案例进展③｜恒信：季度会议为什么是橡皮图章

第三季度委员会开会前，数据部把徐漾请去参谋——他们隐约觉得，这个会”哪里不对”。

徐漾把问题摆上桌：“我翻了前两次的纪要。第一次会议四十五分钟，第二次四十六分钟。议程永远三个：清洗量、平台汇报、下季度目标。我问一个问题——过去半年，评估发现过哪些偏差？有没有一次变成了调整指导？”

数据部的人面面相觑。

“没有吧。”徐漾说，“**因为你们把治理委员会，开成了只看报告的例行会**。你们等日历说’该开会了’，就开；可治理是持续系统——评

估持续进行，发现偏差就该调整指导。合同编号对不上，评估过；财务要求改字段定义，评估过；某个部门想绕过标准，也评估过。”

评估到的东西，一件都没变成”指导”。会照开，舵没动。

这就是橡皮图章——不是你们不努力，是你们把一个持续系统，开成了只看报告的形式。

数据部负责人有些不服：“定期开会不好吗？我们就是靠季度会监控数据质量啊。”

业务方的黄程接了一句：“季度会我场场到，可从没听谁在会上说过’哪一栏编号是标准’。我们财务月底对账，等的是一句有人拍板的话，不是又一份对账报告。”

“监控没问题。”徐漾说，“但你要分清：季度会里看报告，是治理的**监督/评估**环节；看了报告之后该不该调整指导、怎么处置，才是**治理在动**。传感是例行的，处置永远不是例行的。你把传感当成了治理本身——等于装了烟雾报警器，就以为消防队已经出动了。”

数据部负责人沉默。那天晚上，他把”传感 ≠ 治理”五个字，写在了会议纪要的封面。

---

## 14.4 第三幕 治理往哪去（运用）

> 这一幕把”治理”用起来：先补一把尺子（三条判据与一个门槛——判”算不算治理”与判”该不该上治理”是两件事），再讲清它怎么落地运转（跨边界、过滤器、三活动循环——感知、定规、裁例外），最后讲清怎么一眼认出”假治理”（术语膨胀的病根、最终公式的处方、五问判据的尺子）。

### 14.4.1 三条判据，一个门槛：别把”跨部门”当判据

第二幕讲清了”治理是什么、不是什么”，最后一个实用问题：**治理怎么落地？**一个委员会到底怎么运转，才能既不做橡皮图章、又不变成事事都要审批的瓶颈？

落地之前先补一把尺子。前面到处在用”这是治理还是管理”，但那一问其实混了两件不同的事——**判”算不算治理”**与**判”该不该上治理”**。两件事各有各的答法，混着用会得出荒唐结论。

**先说判”算不算治理”。**三条判据，任一条不成立即不属治理：

| 判据 | 治理 | 管理 |
| --- | --- | --- |
| **层级** | 在管理层之上，定方向与框架 | 在治理定下的框架内执行 |
| **对象** | 目的、原则、基线、例外、担责关系 | 方案、模型、代码、配置、部署 |
| **不可自裁** | 判据本身的变化只能由治理裁决 | 框架内的取舍由管理自行决定 |

用法是**逐段定性**，不是整件事贴一个标签。拿 §14.4.4 那个判例试一遍——目的漂移后调整架构：评估发现漂移属治理；追认漂移为新目的、或拉回原目的属治理；根本集合重审与原则基线批准属治理；架构调整与实施迁移属管理。**整条链上只有前段是治理，后段是裁决之后的执行输出。**

**再说判”该不该上治理机构”。**这一个是**理由**，不是判据：**跨出单管理层权限的议题，须提交治理机构。**它回答的是”要不要升级”，不是”算不算治理”。

两条不可互换：**判据管分类，理由管升级。**

> **把”跨部门／跨管理层”当判据用，会推出一个错结论：不跨层的架构治理不算治理。**产品架构治理常常就落在一条产品线内——裁决权在一个总师体系里，管理就够得着。它照样是治理，只是**机构可就近、治理强度可低**。反过来也成立：一件事即使不跨层，只要动的是判据本身，它仍属治理。

拿恒信过一遍这两把尺子。“三个系统三种字段定义”这件事——如果三个系统归同一个团队，负责人直接拍板统一，那是数据管理；如果分属三个部门，得往上报，那是**门槛意义上的升级**（制度上没人有权替另外两个部门改）。

但**升级了不等于就是治理**：上会之后要做的是什么？定”编号标准以谁为准、谁对齐谁、新系统准入要过什么检查”——这些是判据意义上的治理（对象是标准，不是字段）；而”字段长度定 50 还是 100、用哪个工具对齐”——那是判据意义上的管理。

**这一刀切开，恒信那五百万才算被真正诊断清楚**：他们缺的从来不是数据处理能力，是**判据本身没人有权动**。

### 14.4.2 跨边界才是治理：权力 100%，精力 1%

第二幕讲清了”治理是什么、不是什么”，最后一个实用问题：**治理怎么落地？**一个委员会到底怎么运转，才能既不做橡皮图章、又不变成事事都要审批的瓶颈？

先回答三个层层递进的挑战。

**第一，同一个问题，归管理还是归治理，分界线在哪？**——跨边界才是治理。“三个系统三种字段定义”，如果三个系统归同一个团队，

数据负责人直接拍板统一，一个管理者权限内就能解决——这是数据管理，不是治理。

如果三个系统归三个不同团队，A 说”我们对接监管不能改”，B 说”我们跟供应商一致改不了”，C 说”历史遗留改成本太高”——三个管理者谁都没权限命令另外两个改。

这时需要的不是更好的管理，而是**一个跨管理边界的权威**：裁决哪个定义作标准、定新系统准入规则、让拍板的人担责。

准确的说法是：**当”修框架”的权力在一个管理者手里时，是管理；当”修框架”需要超越任何单个管理者权限时，才是治理。**

**第二，企业里一切不都归老板吗？治理不是多此一举吗？**——权力上完全对，但这里有一道裂缝：**权力覆盖 100%，精力覆盖不到 1%**。

一个企业一年做几万个决策，老板不可能每个都亲眼看、亲口定。他必须授权，而授权的那一刻，委托就重新出现——每一层授权都在下层复制了问责悬空的可能。

所以治理不是在老板权力之上加一层，而是在老板精力够不到的地方替他延伸：

> **治理 = 把”老板如果有时间会做的决定”制度化，不需要老板亲自在场。**

而且更深一层：**最高管理者自己也在问责链上**。 在权力链里，他坐在治理机构之下——他上面还有治理机构（董事会、审计、股东会），这些就是治理最高管理者本人的机制。最高管理者也可能追求任期安全、做账美化业绩、过度扩张——这些偏离同样”安静地积累”。

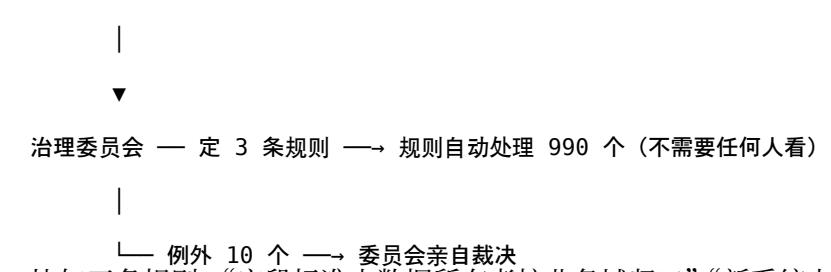
**第三，委员会比老板更没精力，岂不是更糟？**——如果委员会是”多一层人来审管理层的每个决定”，那确实更管不过来。但委员会不是这么干的。

### 14.4.3 委员会是过滤器，不是处理器

这是这一章最反直觉、也最值钱的一条。

假设一个企业一年产生 1000 个跨边界争议——三个系统字段对不上、流程谁审批谁、新系统要不要过标准。**没有治理时**，1000 个争议全部到老板桌上，每个都要亲自裁决；老板精力 1%，于是要么爆掉，要么大部分被”先这样吧”放行。

**有治理时，委员会不处理这 1000 个**。它先定几条规则：




比如三条规则：“字段标准由数据所有者按业务域归口”“新系统上线必须过数据标准合规检查”“跨部门数据变更需审批”。三条规则写出来，990 个争议**不需要任何人看**——规则已经说了怎么办。只有 10 个规则覆盖不了的例外，才到委员会。

**所以委员会一年实际干的活：定几条规则 + 处理 10 个例外，大约 20 小时的活，不是 1000 小时**。老板花 1 小时定 1 条规则，替自己挡住了 1000 次 escalation——这就是**力量倍增器**（force multiplier）。

委员会不是处理器，是过滤器的滤芯。**滤芯不需要处理流过的全部水，它只需要在那里，水自己就被过滤了**。

这正好接回持续系统：管理在框架内持续干活（活很多），治理靠规则过滤、只裁例外（没例外就少开会）。**如果委员会天天开会审一堆事，只有两种可能：规则没定好，或者委员会退化了——它不再定规则、裁例外，而是变成了一个慢速的管理层。**

### 14.4.4 三活动循环：感知、定规、裁例外

委员会到底做什么？一句话说不完——治理不是”制定规则”或”裁决例外”二选一，它是一个三**活动循环**：

**感知 → 定规 → 裁例外 → 感知 → …**

- **感知**——持续检测被治理对象的内外部状态。环境变了（“不适合了”）、框架断了（“坏了”）、规则被绕过了（例外发生），都得先被检测到。这就是前面说的传感装置：定期评审（例行检视）+ 红旗响应（突发报警）。
- **定规**——根据感知到的情况，制定新规则或修订旧规则。这就是力量倍增器——3 条规则替委员会挡住 990 个 escalation。
- **裁例外**——规则覆盖不了的情况，做判断、给裁决。这是评估发现例外后的核心工作。

三者是循环，不是并列清单：感知发现环境变了 → 定规更新规则 → 新规则需要被监督 → 感知继续检测 → 裁例外积累了重复模式 → 反馈给定规修订规则。

任何一个环节缺位，治理就失效：

- **缺感知则盲**——不知道环境变了、不知道例外发生了，委员会变成橡皮图章；

- **缺定规则乱**——990 个 escalation 全到桌上，委员会变成审批瓶颈；
- **缺裁例外则僵**——规则覆盖不了的情况没人裁决，团队只能绕过规则。

例外的正式形态，值得单独说——**豁免（dispensation）**：有期限、有补救计划的受控偏离。豁免通道的设计价值不在"放水"，而在**没有豁免通道的治理会逼出隐瞒**——偏离从显式转入地下，治理反而失明。每条豁免登记在册、到期强制复审；若每个项目都在申请豁免，说明标准本身有问题——**豁免数量是治理框架的健康指标**（呼应 §14.4.3 过滤器：豁免就是那 10 个上会的例外）。

精确的表述是：

> **治理 = 持续感知被治理对象的内外部状态，据此制定/修订规则，并裁决规则覆盖不了的例外。**

> 这里有两把尺子，别混：**ISO 38500 §6 四活动（利益相关方参与→评估→指导→监督）是治理系统的骨架**——管"治理系统本身怎么转"（读偏差、给纠偏、再确认，对应 kybernân 反馈控制，见 §14.3.2）；**感知→定规→裁例外是委员会日常工作的三件事**——管"委员会具体干什么"。两者可粗对应：感知≈评估、定规≈指导（定规后还要监督执行）、裁例外≈评估的

> 例外分支。一个管系统，一个管动作——不是同一件事的两套说法。

三个动作，全是对”方向与框架”动手，没有一个是对”具体业务”动手。治理层永远不碰桨。

> **判例 · 目的漂移事件链**：某产品从”专业工具”滑向”大众应用”（目的漂移），发现后调整架构——这算治理吗？拆成五段定性：① **侦测**——定期以目的为参照检视，捕捉无症状漂移＝治理（感知）；② **显式化与裁决**——业务方行使目的所有权，追认新目的或拉回原目的＝治理（定规）；注意：拉回则根本不动架构，可见”调整架构”不是漂移的必然响应；③ **根本集合重审**——哪些不变量保留/过期/新增，批准原则与基线更新＝治理（动的是判据集合本身）；④ **架构调整**——候选、权衡、修订架构描述＝管理；⑤ **实施迁移**——动手改＝管理。结论：**漂移触发的是一次治理事件，架构调整是裁决之后的执行输出**——治理管①~③（感知/定规/裁决判据），④⑤是执行，治理层碰了④⑤就退化成了慢速管理层（判据 12）。

## 案例进展④｜恒信：把委员会改造成过滤器

数据部把徐漾的批评听进去了，但不知道怎么改。徐漾给了他们一个具体处方。

“委员会每年只干三件事。”徐漾在白板上写，“第一，**定规**——把这一年所有重复出现的争议，总结成几条规则。你们最典型的争议就是合同编号。我建议现在定一条：‘合同编号标准由法务归口，财务、业务的数据必须对齐法务编号；定义变更走跨部门变更审批。’这一条写出来，多少个对账纠纷就不用再上会了。”

“第二，**设感知**——季度会保留，但从’开会的’改成’看表盘的’：数据质量报告、跨部门争议清单、规则被绕过的次数，每个季度看一眼。它不是治理本身，它是烟雾报警器。”

“第三，**裁例外**——只有规则覆盖不了的，才上会。比如三个月后，财务说’监管要求编号必须带财务段位，法务的编号格式改不了’——这是一个规则没预料到的例外，值得开一次临时会，全体到场裁决。会不是季度开，是**有事才开**。”

万辉问：“那平台呢？五百万那套？”

“平台是执行层的桨。”徐漾说，“它清洗、去重、对账，干的是数据管理的活——这是好事，该干。但你要记住它的位置：**平台在管理层的伞下，不在治理层的伞下**。如果哪天有人拿平台的演示 PPT 来说’我们已经治理了’，你就知道，他又把桨当成舵了。”

数据部负责人看着白板上三个词，沉默了很久。然后他说了一句很实在的话：“我以为委员会就是定期开会的。原来定期开会只是报警器，真正干活的，是定规和裁例外。”

### 14.4.5 术语膨胀：用治理的名字，干管理的活

现在回到误解现场那个最扎眼的现象：恒信花五百万买”数据治理平台”，干的却是清洗的活——**用治理的名字，干管理的活**。

先把这个现象放到第 1 章的尺子下量一下。第 1 章说过的词汇病之一——病二”同词异义”：同一个词，不同的人指不同的概念。治理的膨胀就是病二的最新一版：同一个”治理”，有人指掌舵层（定规则、裁例外），有人指执行层（清洗、设计、编码）。

平时各说各的相安无事，一到”该不该立项、谁说了算”就露馅——同一个词挂了两套所指。

这不是恒信一家的问题，也不是数据一个领域的问题。这是近些年”治理”这个词全行业的流行病：**数据治理、架构治理、流程治理，全被膨胀着用**。

先看数据。许多人把”数据治理”的内涵扩大了——既包括治理，也包括依据规则开展的数据质量提升工作。后者在 DAMA 框架里的正式名称是**数据质量管理**（Data Quality Management），属于”数据管理”这个伞下面的执行领域，跟数据治理是**平级**关系，不是包含关系。

再看架构。TOGAF 官方对照表白纸黑字：架构治理管”控制与合规”（标准、政策、决策），架构管理管”执行与交付”（设计、代码、部署）。ADM 的 B-D 阶段（业务/数据/技术架构设计）全是管理的活，治理只在审查点介入。

再看流程。BPM 文献同样明确：“Process governance is a component of BPM”——流程治理是 BPM 的一个组件，不是 BPM 的全套。

三个领域，画成同一张表：

| | 伞（大概念） | 治理（掌舵层） | 管理（划桨层） |
|---|---|---|---|
| 数据 | 数据管理（DAMA，11 个知识领域） | 数据治理（定标准·定权属·监督管理） | 数据质量、元数据、架构、安全、集成… |
| 架构 | 企业架构（TOGAF） | 架构治理（委员会·合规评审·架构合同） | 架构设计/建模、技术选型、系统构建/部署 |
| 流程 | 流程管理 BPM（ABPMP） | 流程治理（归属·政策·跨部门裁决） | 流程设计 / BPMN 建模、执行/自动化、监控/优化 |

注意”伞”那一列——**每个领域的伞都是”管理”，不是”治理”**。这跟”治理在管理之上”恰好反过来了：企业层面治理 ⊃ 管理（董事会在 CEO 之上），但领域层面**管理 ⊃ 治理**（数据管理是伞，数据治理是 1/11；BPM 是伞，流程治理是组件之一）。

为什么？因为企业层面的治理对象是”整个组织”，而领域层面的”XX 治理”对象是”XX 这个东西”。

对象不同，层级关系就不同。

也正是对象不同，**分域治理才有一套可直接套用的模子：X 治理 = 组织治理在”X”这个域上的展开——沿用同一套治理原则，每条补上”对这个域的治理含义”与”结果”。三条不变：原则不另立、治理机构不另设、最终担责不转移。只有对象换了。**

组织治理的产物随对象而变：架构域出**架构决策＋理由**，数据域出**用途审批＋权属**，流程域出**流程清单与分级＋符合性检查**。同一个模子，长出不同的账本。

对象不同，三域各自的**判据所有者**也不同——架构域是业务方（目的所有权，§14.2.6），流程域是流程负责人，数据域是数据所有者；三域都满足治理三条件，所以都必须有独立的治理安排——同一张骨架，只换判据所有者、不变量集合、执行系统三样填充。

膨胀的根源，三个领域完全一样：**“治理”听起来比”管理”高级 → 厂商和顾问膨胀用词 → 一词跨两层 → 角色混乱**。

“数据治理平台”卖的是 ETL 清洗，“架构治理”包含设计编码，“流程治理”等于 BPM 全套——全都是用治理的名字干管理的活。

后果是：团队分不清哪些决策该上委员会、哪些自己拍板就行——要么事事上报变成瓶颈，要么事事自己搞绕过治理。

### 14.4.6 最终公式：XX 管理 = XX 治理 + XX 开发与运营

膨胀的病根，是”管理”和”治理”两个词在伞和子流之间错位。要根治，得把伞的命名理顺。先排除一个错误答案：

> “XX 管理 = XX 治理 + XX 管理”——**否决**。这就是 A = A + B，自包含。

那执行部分总得有个统称。ITIL/COBIT 用”运营”，DAMA 用一堆专名不统称，TOGAF 干脆换伞名。最干净的方案，是借 DevOps（Development + Operations）造一个公式：

**XX 管理 = XX 治理 + XX 开发与运营**

- **开发** = 设计 + 构建（从无到有造东西：建模、ETL、架构设计、流程自动化）；
- **运营** = 运行 + 维护 + 优化（造完之后让它持续跑好：DBA、运维、质量修复、调优）。

这正好是 PDCA 循环里 P（计划/设计）+ D（执行/构建）+ C（检查/监控）+ A（改进/优化）的完整映射。三个领域全部验证：

| 领域 | XX 治理 | XX 开发与运营 |
| --- | --- | --- |
| 数据 | 标准 · 权属 · 审计 ·steward | 建模/ETL（开发）+ DBA/质量修复/BI（运营） |
| 架构 | 委员会·合规·合同 | ADM 设计/编码（开发）+ 运维/调优（运营） |
| 流程 | 归属·政策·裁决 | BPMN 建模/自动化（开发）+ 执行/监控/优化（运营） |

架构领域尤其顺——TOGAF 的核心方法就叫 **ADM（Architecture Development Method，架构开发方法）**，“开发”是官方用语。这个公式干净地解决了全部问题：无自包含、无术语膨胀、执行层完整覆盖、三领域通用。

推到国家层面，层级反转过来了：国家的核心活动不是”造产品”而是”行使权力、提供公共服务”，所以治理是伞。但”国家治理 + 国

家管理 = 国家治理”又是 A + B = A 的自包含，伞名不能是加数之一，于是最终落到：

> **国家发展与运行 = 国家治理 + 国家管理**

“发展与运行”和”开发与运营”完美平行——开发→发展、运营→运行，造产品→建国家、跑系统→跑国家。两行结构一模一样，区别只在尺度。**治理在两个层面都是那个方向盘：定规则、裁例外、监督管理。**

### 14.4.7 一眼认出”假治理”：五问判据

讲完这么多，你需要一把能随身带的尺子。判断一个组织现在做的到底是治理，还是挂着治理名字的管理，五个问题：

**① 有没有"分离"？　　出钱定方向的人和管日常的人，是同一拨吗？—同一拨，不需要治理**

**② 治理在持续运转吗？ 治理机构是否持续行使四活动（利益相关方参与/评估/指导/监督）、评估发现偏差后是否调整指导？—只开会看报告、从不处置，是橡皮图章**

**③ 委员会在"定规、裁例外"吗？ 还是天天审具体方案、批具体预算？—后者是慢速管理层**

**④ 有没有"传感装置"？ 定期评审 + 红旗响应，两样都有吗？—只有红旗没定期，抓不住"不适合了"**

**⑤ 治理层碰不碰桨？　　委员会在不在设计、编码、清洗、部署的现场？—碰了，就是退化**

五个问题，从”要不要治理”问到”是不是治理”。任何一问问出”没有”，就要警惕——你面对的可能不是治理，是又一个挂羊头卖狗肉的季度会。

最后补一句提醒：**五问是体检表，不是分类器**。它判的是”这套治理转不转得动”，不是”这件事算不算治理”——后者归 §14.4.1 的三条判据。两把尺子各量各的，别串。

## 案例进展⑤｜恒信：把桨放回桨位，把舵装回头脑

第三季度，恒信按新规则运转了一个季度。徐漾在一个下午收到两份消息。

第一份来自数据部负责人，附了一份报告：法务编号归口标准上线后，三个系统的合同编号对账率从 62% 升到了 98%。报告结尾写了一行字：“原来我们缺的从来不是清洗，是一个有人担责的’编号标准’。”

第二份来自万辉，是一张采购评估表的批注。供应商又来推销”新一代数据治理平台”，万辉在评估结论栏写了四个字：**“建议驳回。”**旁边是手写的理由——“清洗能力，现有平台已覆盖；治理能力，工具采购解决不了。此单采购对象实为数据清洗平台，属数据管理执行层，不应以’治理’立项。”

评估表里还夹着运维玉杰附的一份运行数据：现有平台的清洗任务负载常年不到三成。玉杰附了两行说明——这套清洗任务他常年跑着，最清楚它的底细：“再买一台，跑的还是同一套清洗，多一台机器只是多一套监视。桨还是那把桨，不会多出半个舵。”这份数据，正好垫在万辉那句”清洗能力已覆盖”下面。

徐漾回了他一行字：“你知道你刚才干了什么吗?”

万辉：“拒绝了一次伪治理的采购。”

“不止。”徐漾说，“你顺便做了一次真正的治理——你在裁决’以什么名义立项、什么东西归到哪一层’。这就是委员会最该干的活：**定**

**边界、裁例外**。第 13 章你学会了’谁有资格批准基线、谁有资格裁决变更’；这一章，你补上了问题的另一半——**谁有资格批准一个’治理’项目的立项**。”

万辉看着那条回复，忽然想起半年前自己改编号规则时的那句”今天就能上”。他笑了笑，把那行字存了下来：

> “桨要放回桨位，舵要装回头脑。**先分得清谁在掌舵、谁在划桨，船才不会在原地转圈**。”

---

## 认知反转

回到章首。恒信买了五百万的”数据治理平台”，成立了”数据治理委员会”，数据还是对不上。如果只看表面，你会说”治理做了，工具也买了，委员会也成立了，怎么还是没用？“——现在我们知道，那场”治理”，连错四层，一层比一层深。

**第一层，他把”治理”当成了”管理”**。他以为治理就是把数据管好——清洗、去重、对账，让数据干净。其实治理是掌舵，管理是划桨：**平台清洗一万条，是划桨；谁有权裁决”法务的编号是标准，财务必须对齐”，才是掌舵**。把治理当管理，等于买了把好桨，然后宣布”我们装上了舵”。

**第二层，他把”统治”误读进了”治理”**。治理的源头是希腊语”掌舵”——舵手不是船主，靠引导不是靠命令。

这个被翻译刻意选中的词，把”主权者-臣属”的关系，换成了”领域+规则+角色+担责”。**治理不是更高的统治，是更柔的掌舵**。

你命令不了水往哪流，你只能疏导它；你命令不了三个部门改定义，你只能裁决标准、立规则、让大家都向它对齐。

**第三层，他不知道治理是持续系统**。委员会每季度开会、四十五分钟开完、看报告就散会——只走了四活动里监督/评估的一个载体，评估发现偏差后从不调整指导。

**治理是持续的四活动循环：利益相关方参与、评估、指导、监督；评估持续进行，发现偏差即调整指导**——合同编号对不上了，调整指导：谁的标准、怎么对齐；规则覆盖不了的新情况，调整指导：定新边界。定期评审是烟雾报警器，不是消防队。只开会、不处置，就是橡皮图章。

**第四层，也是最深的一层，他把”治理”这个词，用在了管理的活上**。这是术语膨胀——数据治理、架构治理、流程治理，全行业同一个病：治理听起来高级，于是整个伞都被叫作治理，执行层的工作全被贴上治理的标签。**而真正值钱的，恰恰不是那个标签，是标签下面那层”定规则、裁例外、担起担责”的安排**。

这一章的四次反转，其实是同一个反转的四种说法：

> **治理不是”更高级的管理”，是”另一个方向的工作”——一个掌舵（定方向、定规则、裁例外、监督管理），一个划桨（把事做好）。治理长在责任下放造成的担责悬空处，靠持续的四活动循环纠偏、在框架级定方向；从国家到数据表，这是同一件事的不同尺度版本**。

用第 1 章的话收一次束：对”治理”这个词，你原来只是”见过”——背得出”治理≠管理”、说得上”掌舵划桨”。这一章把它推到”拥有”——能答全四问（它是什么、不是什么、从哪来、往哪去），能用治理四问、持续系统、五问判据，去量一个委员会、一次采购、一场例会。**能答全四问才算拥有——你能分清掌舵与划桨，认出谁在守不变量。**

而这一切，靠的都是第 1 章那句”意义即用法”：不先问”治理是什么”，先问”治理做什么”——它做什么，就定义它是谁。你用”它做什么”给”治理”下定义的那一刻，就再也不会把委员会开成清洗周报会。

回看第 13 章的结尾。第 13 章说：配置管理是治理的一只手——基线由谁批准、变更由谁裁决、高不可逆的准入权在谁手里，是治理问题。

这一章把那只手的头脑装上了：**委员会定规、裁例外、设感知，正是给”谁有资格批准基线、谁有资格裁决变更”找到了一个负责任的答案**。配置管理给了守护不可逆决策的手，治理给了这只手方向。

而最后，两条案例线都各自走到了这一步——恒信守住了跨部门的编号标准，冰链重建了固件升级的边界。第 17 章，两条线合流：从一条需求，走到上线运维，走一遍完整的变更闭环——**把这一整本书的概念，放在同一条时间线上，看它们怎样配合着，让一个系统从出生活到退役**。

用第 1 章 §1.4.4 的尺子量一下：这一章守的同样是**外化地图**——只不过守的不是数据本身，而是”谁有权改这张图”。

## 小结

这一章把”治理”从流行词里捞了出来。**治理是掌舵，不是统治**——govern 的本义是希腊语”掌舵领航”，舵手不是船主；中文把它定译为”治理”，借了”治”（治水疏导）和”理”（治玉顺纹）的古典正资产，完成与”统治”的概念切割，也留下了治理四问（谁来治/依何治/如何治/治得怎样）这个跨领域通用公式。

标准的定义句只有一句，但四样东西一样都不能少：**以人为基础**（这套系统的要素是人，不是文件堆、不是 IT 系统——所以 governance 是人做的一套安排，不是买来的一台机器）；三**职能**（受到指导、接受监督、被要求交账）；**为宗旨**（它为了什么）；那个”**被**”字（丢了它，治理就从”有外部力量把组织置于须交账的地位”退回成”组织自己要自觉”）。

十一条原则里，**宗旨是唯一的”首要”**——不能用来做决策的宗旨，不是宗旨，是标语。治理做得好不好，标准也只认三个结果：有效的绩效、尽责的管理、合乎伦理的行为。

**治理不等于管理**——治理是决策系统（掌舵：定方向、定规则、裁例外、监督管理），管理是执行系统（划桨：把事做好）；治理长在责任下放造成的担责悬空处，靠治理机构持续行使指导、监督、担责把它接住。收成一句话：**治理是跨层级的远程校正机制，守护的是不变量（标准、原则、基线）；管理在框架内调变量、达成运行目标**。

而在”这是治理还是管理”这一问上，尺子有两把：**判”算不算治理”看三条判据（层级／对象／不可自裁，逐段定性）；判”该不该上治理机构”看一个门槛（跨出单管理层权限）。判据管分类，理由管**

**升级**——把”跨部门”当判据用，会推出”不跨层的架构治理不算治理”的错结论。

**治理不是越早越好**——分离、静默积累、框架级三条件同时满足才需要**正式**治理机构；**治理不是例行工作**——治理是持续系统，治理机构持续行使利益相关方参与/评估/指导/监督四活动，评估持续进行、发现偏差即调整指导。

定期评审与第 8 章的复盘都是传感装置（评审以目的为参照抓漂移、复盘翻教训——两类传感并列），把传感当治理整体，就是橡皮图章；**委员会是过滤器不是处理器**——定规则替它挡住 990 个争议，只裁 10 个例外。治理的运转是三活动循环：感知、定规、裁例外。

最后——**“治理”这个词被全行业用滥了**：数据治理、架构治理、流程治理的伞其实都叫”管理”，治理只是伞下一个掌舵的子域（这是第 1 章病二”同词异义”在行业语里的重演）。真正的公式是”XX 管理 = XX 治理 + XX 开发与运营”，治理层永不碰桨。而分域治理其实只有一套模子：**X 治理 = 组织治理在”X”域上的展开——原则不另立、治理机构不另设、最终担责不转移，只有对象换了**。

第 13 章说：配置管理是治理的一只手。这一章把那只手的头脑装上了——**基线由谁批准、变更由谁裁决、高不可逆的准入权在谁手里，由治理回答**。配置管理给了”怎么守住不可逆决策”的流程，治理给了”谁有资格定这些流程”的权威。

第 12 章问”哪些决策值得认真做”，第 13 章问”认真做的决策怎么被守护”，这一章问最后一个问题——**“守护不可逆决策的人，自己由谁守护。”**

下一章，两条案例线合流：从一条需求走到上线运维，走一遍完整的变更闭环——把全书所有概念放进同一条时间线，看它们怎样配合着，让一个系统从出生活到退役。

架构文档的扉页，添了一行：不可逆的决策，过治理。以前他以为架构师说了就算；这一章之后他明白——他掌舵，但船不是他的。

---

## 本章概念总图

**治理 = 掌舵（决策系统）：定方向 · 定规则 · 裁例外 · 监督管理**

**定义（ISO 37000 §3.1.1）：一种以人为基础的系统——组织为实现其宗旨，通过该系统受到指导、接受监督，并被要求就其宗旨的实现作出说明、承担后果**

**四成分：以人为基础（要素是人，不是文件堆、不是 IT 系统，故译"以人为基础"不译"以人为本"）+ 三职能（受指导/受监督/被要求交账）+ 为宗旨 + "被"字（丢了它就退回成"组织自己要自觉"）**

**属是 system、种差才是全部信息；三职能（是什么）≠ 四活动（怎么转）**

**担责三名字：担责（术语位·那项义务）/ 问责（动词位·治理机构那只手）/ 述责（动作层·把账摆出来）**

**词源：govern ← gubernare ← kybernân（掌舵领航）——舵手不是船主**

**翻译：统治（rule）→ 治理（governance）的切割——五维度：主体多元/协商为主/契约共识/多向互动/范围更宽**

**字源：治（治水·疏导）+ 理（治玉·顺纹）= 外导内顺——干预遵循对象内在规律**

**通用公式：谁来治（角色）→ 依何治（规则）→ 如何治（机制）→ 治得怎样（担责）——任何领域都有这四样**

**宗旨是唯一的"首要"原则：11 条治理原则分三层（首要 1=宗旨 / 基础 4=价**

值创造·战略·监督·担责 / 赋能 6），同时适用、不是打勾清单

判得对不对只认三个结果：有效的绩效 / 尽责的管理 / 合乎伦理的行为——"有没有董事会""有没有制度文件"都不算数

宗旨真伪判据：不能用来做决策的宗旨，不是宗旨，是标语

治理落地到架构域 = 架构治理（42020）：四要素（主体/对象/手段/目的，缺一即不成立）· 治什么=目的→不变量→根本决策→遵从与演化链（四层）· 三种侵蚀力量（局部优化/目的漂移/决策不可逆）· 目的变更=重审触发器 · 豁免=安全阀 · 方向权（含目的所有权在业务方）属组织层、内容权属架构师——系统级文档是"作品"、治理产出是"制度"（第18章）

产品架构治理 vs 企业架构治理：前者可落在一条产品线内（管理够得着），后者天然跨部门；架构治理的分量在"后果比决策者任期还长"

治理 vs 管理：决策系统 vs 执行系统

治理 = 掌舵（四活动：利益相关方参与→评估→指导→监督）| 管理 = 划桨（PDCA：计划→执行→检查→改进）

评估判断"该不该"（批准之前），监督复核"是不是"（批准之后）；oversight ⊃ monitor（中文"监督"接两个英文词，需分时写"监测／复核"）

机制版：治理 = 跨层级远程校正 · 守护不变量（标准/原则/基线）| 管理 = 框架内调变量 · 达成运行目标

"有舵无桨到不了岸，有桨无舵原地转圈"

委托必有治理：责任可委托、担责不可委托（ISO/IEC 38500 §6.1）——委托不是治理的前提，无委托时治理同样是组织常态

担责的部件：held（被要求）＋-able（一种状态）＋account（账）——它是你的处境不是你的品格；核心动作是把账摆出来，不是低头认错（产出是信任与合

法性）

授权有效五条件：结果协商一致 / 资源可用 / 权力与责任匹配 / 定期报告并举证 / 后果可执行；配一条禁令—不宜为无权处理、也没商定期望的事项负责

权力链：所有权 → 治理 → 管理 → 执行

治理系统的七类元素（从标准推导）：主体/意图/规矩/结构/关系/行为/保证—原则不是要素、结果也不是要素；"保证"最常被整个跳过

什么时候需要正式治理机构（三条件）：分离 + 静默积累 + 框架级—三者同时满足才需要

任一缺失 → 不必设正式治理机构（治理职能仍在）；五人团队引入正式治理 = 形式过早

治理是持续系统：四活动循环（利益相关方参与→评估→指导→监督）持续运转，不靠事件启动—评估持续进行，发现偏差即调整指导

定期评审与第8章的复盘，都是监督/评估的常规载体（两类传感装置：评审以目的为参照抓漂移、复盘翻教训）；传感是治理的一环、不是治理整体

治理与管理之别在职能层级与对象（治理定框架、管理在框架内执行），不在触发方式—两者都持续运转

只开季度会看报告、评估后从不处置的委员会 = 橡皮图章

两种失效：坏了（内部失效·红旗评估·修）vs 不适合了（外部过时·定期检视·演）—架构域"不适合了"的典型=目的变更/目的漂移（目的变更=重审强制触发器，目的漂移靠以目的为参照的定期检视）

最危险：不适合了但还没坏—只能靠定期评审从外部问一句"框架还在解决今天的问题吗"

传感装置（含第8章复盘）只负责"发现"，发现之后怎么裁决才是治理

治理怎么运转：

判"算不算治理"看三条判据（层级/对象/不可自裁，逐段定性）；判"该不该上治理机构"看一个门槛（跨出单管理层权限）——判据管分类，理由管升级，两把尺子不可互换（把"跨部门"当判据会推出"不跨层的架构治理不算治理"的错结论）

跨边界才是治理：修框架的权力超越任何单个管理者 = 治理（介入门槛）

权力 100%，精力 1%——治理长在缺口上；最高管理者自己也在担责链上

委员会是过滤器不是处理器：1000 争议 → 定 3 规则 → 990 自动 + 10 上会（力量倍增器）

三活动循环：感知 → 定规 → 裁例外（缺感知则盲 / 缺定规则乱 / 缺裁例外则僵）；四活动模型=系统骨架；裁例外=豁免通道（安全阀，没有豁免逼出隐瞒）

术语膨胀：用治理的名字干管理的活（接第1章病二"同词异义"：同一个"治理"挂两套所指）

伞是"管理"不是"治理"：数据管理 ⊃ 数据治理；企业架构 ⊃ 架构治理；BPM ⊃ 流程治理

分域模子：X 治理 = 组织治理在"X"域上的展开——三条不变（原则不另立·治理机构不另设·最终担责不转移），只有对象换了

最终公式：XX管理 = XX治理 + XX开发与运营（国家发展与运行 = 国家治理 + 国家管理）

治理层永不碰桨：设计、编码、清洗、部署都是执行层的活

## 本章判据

> 本章判据是本书的主张，不是任何标准的条文（判据 8 的目的变更触发器承接第 10/11 章目的判据，其”对目的而言必要且充分”的读法系二**手派生表述**）——它们都当场可检验：欢迎检验、欢迎校准。
> 条数说明：本章判据由 13 条增为 **14 条**（2026-09-23 依 28 号新增判据 9「判据与门槛分家」，原判据 9~13 顺延为 10~14）。**新增而非折入既有条目**，因为它是”分类判据”本身的一条规矩——折进判据 2 的注里，读者翻到判据表就看不见它；而 §14.4.1 整节、判据 14 的收尾句、以及第 17 章与训练教材的引用都指向它。（**本书惯例是”改内容不加减条目”**，本条属有意的例外。）

1. **掌舵 vs 统治**：治理是”这条船往哪开、由谁来掌舵、偏航了谁来纠”，不是”谁骑在谁头上”——治理靠引导，不靠命令。**违反即崩：靠命令压着三个部门统一编号——红头文件发了，没人裁决、没人担责，编号照样对不上；命令压不动水，只能疏导。**
2. **治理四问**：谁来治（角色）？依何治（规则）？如何治（机制）？治得怎样（担责）？——四问有答，才谈得上治理；统治对非国家领域答不出这四问（主权者-臣属锁死了主体形态）。注一：此四问≠第 1 章四问法。注二：四问判”算不算治理”，与”该不该上治理机

构”是两把尺子（§14.4.1）。**违反即崩：四问里答不上”谁来治”——三个部门都说编号标准不是自己的事，标准就永远悬着**。

3. **掌舵 vs 划桨**：治理=决策系统（做什么、谁有权决定、谁担后果）；管理=执行系统（怎么把事做好）。机制地说——治理是跨层级的远程校正机制，专职守护不变量（标准、原则、基线）；管理在框架内调变量、达成运行目标。**违反即崩：把治理当管理做——平台多洗一万条数据，跨部门标准没人裁决，编号还是对不上**。
4. **“有舵无桨到不了岸，有桨无舵原地转圈”**：没有执行的治理是空转，没有治理的执行是转圈——两者是授权与报告的纵向关系，不是并列。**违反即崩：恒信五百万买的是桨、委员会四次开会没碰过舵——桨在划，船在原地转圈**。
5. **委托必有治理**：责任（具体责任）可委托，担责不可委托（ISO 37000 §4.2.2／§6.5.2、ISO/IEC 38500 §6.1）——治理机构可以授权他人，但仍对整个组织的作为、不作为和渎职负责；下放后的执行若脱离指导、监督、担责，组织目的就无法保证，最终担责仍落治理机构。委托不是治理的前提，无委托时治理同样是组织常态。**违反即崩：集团把数据管理下放给数据部，却没把”合同编号以谁为准”的担责落到任何人——该下放的责任下放了，不该下放的担责悬空了**。

6. **治理三条件**：分离 + 静默积累 + 框架级——三个同时满足才需要**正式**治理机构；任一缺失，不必设正式治理机构（治理职能仍在，可由同一人兼任治理与管理，两角色仍须清晰）。五人团队引入正式治理委员会=形式过早。**违反即崩：五人团队没有分离，硬拉一个季度委员会——没有委托可守，例会就成了挂在墙上的形式。**
7. **治理是持续系统**：治理机构持续行使利益相关方参与、评估、指导、监督四活动（ISO 38500 §6）——评估持续进行，发现偏差即调整指导，不靠事件启动。定期评审只是监督/评估的常规载体（传感装置），把传感当治理整体就是橡皮图章；第 8 章的复盘是另一只传感装置（评审管把关、复盘管改进，两类传感并列不混），复盘后评估作出的处置才是治理。**违反即崩：委员会只在季度会看报告——评估发现偏差也不处置，编号半年对不上，舵一次没动。**
8. **两种失效**：坏了（内部失效，红旗评估抓，修）vs 不适合了（外部过时，定期检视抓，演）——最危险的是”不适合了但还没坏”，只能靠定期评审从外部问一句。**架构域”不适合了”最典型是目的变了：目的变更=重审强制触发器，目的漂移只能靠以目的为参照的定期检视。违反即崩：环境已变、框架没症状、团队开始绕过——没人评估、没人看见，直到绕过积累成爆雷。**

9. **判据与门槛分家**：判”算不算治理”看三条判据（层级／对象／不可自裁，逐段定性）；判”该不该上治理机构”看一个门槛（跨出单管理层权限，这是升级的理由）。**判据管分类，理由管升级，不可互换**——把”跨部门”当判据用，会推出”不跨层的架构治理不算治理”的错结论（§14.4.1）。**违反即崩：把”跨部门”当判据——一条产品线内的架构治理被判成”不算治理”，于是没人给产品线的架构决策留理由**。
10. **跨边界才是治理（介入门槛）**：修框架的权力在一个管理者手里=管理；需要超越任何单个管理者权限=升级到治理机构。权力覆盖 100%、精力覆盖不到 1%——治理长在缺口上。**违反即崩：数据部拍板”统一编号”——法务、财务、业务没有义务听，标准落地不了，因为裁决权没越过管理边界**。
11. **委员会是过滤器**：1000 个争议，定 3 条规则自动处理 990 个，只裁 10 个例外——委员会是力量倍增器；天天审具体事的委员会退化了。**违反即崩：委员会不定规则——1000 个争议全上桌，老板精力 1% 要么爆掉、要么被”先这样吧”放行**。
12. **三活动循环**：感知→定规→裁例外——缺感知则盲（橡皮图章），缺定规则乱（全到桌上），缺裁例外则僵（团队绕过）。裁例外=豁免通道（有期限、有补救计划的受控偏离，没有豁免会逼出隐瞒）。

**违反即崩：缺裁例外——规则覆盖不了的新情况没人裁决，团队只能绕过规则。**（过滤器模型见 §14.4.3，判例见 §14.4.4）

13. **治理层永不碰桨**：委员会不设计、不编码、不清洗、不部署——碰了，就是退化成了慢速管理层。**违反即崩：委员会天天审清洗规则、批 ETL 任务——它成了慢速管理层，舵没人掌。**
14. **术语膨胀五问**：有分离吗？治理在持续运转吗？委员会在定规裁例外吗？有传感装置吗？治理层碰桨吗？——任何一问是”没有”，就要警惕”假治理”（五问是体检表，判”转不转得动”，不判”算不算治理”——后者归判据 9）。术语膨胀的根，是第 1 章病二”同词异义”：同一个”治理”，挂了两套所指。**违反即崩：五问里问出一问”没有”——“数据治理平台”五百万立项，问”治理层碰桨吗”就露馅：它从头到尾在清洗。**

## 自查 · 这些说法对吗？

1. “数据治理就是把数据管好——清洗、去重、保证质量。”——**错。**清洗去重是数据质量管理，属执行层（管理）；数据治理定标准、定权属、监督管理（掌舵）。用治理的名字干管理的活，正是术语膨胀。
2. “治理就是管理层再高一级，CEO 就是最高治理者。”——**错。**CEO 是最高管理者不是最高所有者；他上面还有董事会、审计、股东会——那些才是治理 CEO 的机制。顺带分清三个名字：**担责**

是那项义务（术语位）、**问责**是治理机构那只手（动词位）、**述责**是把账摆出来（动作层）——三者不是一个词。

3. “我们买了治理平台、成立了委员会，治理就算落地了。”——**错**。工具是桨，委员会是舵手——平台清洗是管理；委员会如果只开季度会看报告、从不处置，是例行工作，不是治理。
4. “委员会要每季度开例会，治理才有效。”——**错**。治理是持续系统（四活动循环），定期会议只是监督/评估的常规载体（传感装置）；评估发现偏差即调整指导，只开会看报告、从不处置，是橡皮图章。
5. “五人小团队也应该搞正式治理委员会。”——**错**。治理三条件（分离/静默积累/框架级）任一缺失就不必设正式治理机构；五人团队没有分离，引入正式治理委员会是形式过早——但治理职能仍在，可由同一人兼任治理与管理，两角色仍须清晰。
6. “治理就是老板把所有事都管起来。”——**错**。权力覆盖 100%、精力覆盖不到 1%——治理是把”老板如果有时间会做的决定”制度化，不需要他亲自在场。
7. “治理委员会要审每一个跨部门决定，层层把关。”——**错**。委员会是过滤器不是处理器：定规则自动处理 990 个，只裁 10 个例外——委员会天天审具体事，说明规则没定好。

8. “治理层级越高，管的事越多越好。”——**错**。治理层永不碰桨——委员会不设计、不编码、不清洗、不部署。治理碰执行，就是退化。
9. “数据治理、架构治理、流程治理，是把管理升了级。”——**错**。领域层面”管理”是伞（数据管理 ⊃ 数据治理），治理是伞下的一个领域；企业层面才反过来（治理 ⊃ 管理）。两个层级关系不一样。
10. “架构治理包含架构设计——治理层也要参与设计。”——**错**。TOGAF 白纸黑字：治理管控制与合规（标准、政策、决策），设计、编码、部署全是管理的活。治理在设计审查点介入，不参与设计本身。
11. “治理就是管好那些架构文档——文档受控了，架构就治理好了。”——**错**。治理的对象不是”文档”，是「目的→不变量→根本决策→遵从与演化」链（架构资产只是四层之一）——目的漂移了、决策偏离了目的，文档再整齐也白搭（§14.2.6）。
12. “目的变了，架构要不要重审，看情况再说。”——**错**。目的变更是架构重审的强制触发器——fundamental 的判据本身换了，不变量集合整体过期；目的漂移（悄悄变）只能靠以目的为参照的定期检视抓（§14.3.7）。

## 练习

1. 找出你们组织/团队最近一个挂着”治理”名头的事（数据治理/架构治理/流程治理/治理委员会），用”治理四问”（角色/规则/机制/

担责）逐个检验：它答得上四问吗？再用标准那句定义对一遍成分——**以人为基础**（是人说了算，还是文件说了算）、**三职能**（指导/监督/担责都在吗）、**为宗旨**（它服务哪条宗旨）、**“被”字**（那个须交账的地位，是制度给的还是靠自觉）？

2. 用”掌舵 vs 划桨”把你所在组织最近五个”大决策”分类：哪些是治理（定方向/定规则/裁例外），哪些是管理（执行/优化/交付）？分完再补一刀：按三**条判据**（层级/对象/不可自裁）逐段定性，再用**介入门槛**（跨出单管理层权限）说明它该不该升级——**判据管分类，门槛管升级，别把”跨部门”当判据**（§14.4.1）。
3. 你们组织符合治理三条件（分离/静默积累/框架级）吗？——符合几个？还差哪个？还差的那个，是不是”管理足够”的原因？
4. 你参与的某个委员会/评审会/例会，用”持续系统”诊断：它是否持续行使利益相关方参与/评估/指导/监督四活动？评估发现偏差后有没有调整指导？只开会看报告，是传感装置不是治理。
5. 找一个”坏了”和”不适合了”的真实案例各一个（可以是你们团队的）：前者为什么能被评估在红旗亮时抓住，后者为什么往往要等到爆雷？
6. 画出你们组织的”权力链”（所有权→治理→管理→执行）：每一层的人分别是谁？有没有一层是空的（比如有管理没有治理）？

7. 用”过滤器”模型改造一个你们现在”事事上报”的流程：你能定哪几条规则，让 90% 的争议不再上会?
8. 用”三活动循环”诊断一个治理/委员会机制：感知、定规、裁例外，哪一环最弱？缺的那一环，会先出现什么症状?
9. 用”术语膨胀五问”检验你们组织的”数据治理/架构治理”：它是真治理，还是挂着治理名字的管理?
10. 把你们组织某个领域的伞结构写出来：XX 管理 = XX 治理 + XX 开发与运营——三个加数各自对应哪些人、哪些活动？伞名有没有跟子流重名（自包含)?
11. 回到恒信的编号事故（第 12 章），用治理视角复盘：如果当时有”跨部门编号标准由法务归口”的规则，万辉改编号会走什么路径？规则能不能从根上拦住那次事故?
12. 挑一个你们系统的架构决策，沿”目的→不变量→根本决策→遵从与演化”四层各自问一遍：这层谁在守护、被什么侵蚀？再按§14.4.4 那个判例（目的漂移事件链）的拆法走一遍：目的漂移后，哪个动作算治理（侦测/裁决/重审判据)、哪个算管理（调整架构/实施迁移)?

（答案要点见书末附录，与训练教材题卡同源）

## 参考文献

**标准** - R-04 ISO/IEC/IEEE 42010:2022 —— 架构描述；“architecting”七动词链定义（**§3.1**)：构思→定义→表达→记录→

沟通→认证恰当实现→维护与改进（治理用链尾两项：认证 = 符合性监控、维护改进 = 演化治理，正文 §14.2.6 实引）；§3.7 架构视图＝信息部件、**§3.8 架构视角＝一组惯例**（正本定义句无属概念，不补；可版本化依据在 §3.8 注 3——惯例记载在该视角的规格里，正文 §14.2.6 治理对象表随用） - IEV 831-01-01 Note 2 —— “基本（fundamental）”对系统在其环境中的目的而言必要且充分的判据：目的漂移使整个不变量集合过期（正文 §14.2.6 + 判据 8 锚；同 §10.2.4）。出处口径：该条系**二手派生表述**——42010:2022 正本未引 IEV（书目仅列 IEC Electropedia），它不是任何标准的准入判据条款，本书只取”对目的而言必要且充分”这一读法 - R-08 ISO 10007:2017 —— 配置管理（对应国标 GB/T 19017）：架构描述基线化与变更控制，守不变量链第三、四层（正文 §14.2.6 治理对象表三、四行 + §13.4.4 实引） - R-05 ISO/IEC/IEEE 42020:2019 —— 架构治理流程 §6（§6.1 目的句：建立并保持架构集合中的架构与企业目标/政策/战略**以及各相关架构**一致）；治理 §6 vs 管理 §7（治理结果适用于整个组织、管理结果适用于架构集合—— §6.2 注 2／§7.2 注同文，并见 §5.3）；六项任务中”定架构目标”（§6.4.3）与”查对治理指令的遵守”（§6.4.5）最常被整个跳过（正文 §14.2.6） - R-17 ISO/IEC 38500:2024 —— IT 治理（组织的 IT 治理，对齐 ISO 37000）：§3.3 治理定义＝以人为基础的系统、包括指导/监督/担责

三职能；§4 三项治理结果的 IT 展开；**§6.1 担责不可授权**；§6 治理模型四活动=利益相关方参与/评估/指导/监督（2024 新增利益相关方参与）；§3.2 评估=考虑并作出有依据的判断、§3.8 监督=复核作为适当决策与调整的依据（正文 §14.3.2 评估/监督之别）；§7.2.5 授权；§7.2.7 展示担责的机制 - R-94 ISO 37000:2021 —— 组织治理指南（治理本体权威）：§3.1.1 定义（以人为基础的系统／三职能／为宗旨；中译定稿见 §2.2）；§3.2.2 担责定义与两注（注 1 告知与解释；注 2 后果由责任方承担）＋§6.5.3.2 披露八项（含”解释和证明”与”不履行责任的后果”）、§6.5.1 方向双向；§3.2.3 责任、§3.2.4 授权（正文 §14.3.3）；§3.2.9 治理方针、§3.2.10 组织宗旨（宗旨真伪判据见注 2”宗旨指导表现目标”）、§3.2.11 组织价值观；§3.3.1–3.3.6 角色族（治理机构／治理小组／人员／利益相关方）；§4.1 治理核心五条（设定并致力于宗旨与价值／确定价值创造方法／指导和参与战略／监督按期望执行／表达责任感）；§4.2.2 授权五条件与那条禁令；§4.2.3 治理与管理的区分（“定参数是治理，在参数内选最优是管理”）；§4.3.1 委员会；§4.3.2 注 3 内部审计（保证）；§5 三项治理结果（有效的绩效／尽责的管理／合乎伦理的行为）；§6 十一条原则三分类（首要 1=宗旨；基础 4=价值创造·战略·监督·担责；赋能 6）＋§6.4 监督（内部控制体系与 assurance）；§6.3.3.1.2 d 指导的止境（管”需要履行什么责任”，不管”具体如何履行”）；表 1 原则

分组（正文 §14.2.5／§14.3.2／§14.3.3／§14.3.4／§14.3.5 实引） - R-19 TOGAF / DAMA-DMBOK / ABPMP CBOK —— 治理伞结构：架构治理（TOGAF）、数据管理 11 领域（DAMA）、流程治理（ABPMP）；“管理”是伞、“治理”是子域；分域治理三条不变（原则不另立／治理机构不另设／最终担责不转移，正文 §14.4.5 实引）

**经典文献** - R-46 World Bank, Sub-Saharan Africa: From Crisis to Sustainable Growth（1989）—— “治理危机”一词的起点 - R-40 俞可平《治理与善治》(2000) —— governance 定译为”治理”；“不是简单的词语变化，而是思想观念的变化” - R-41 Rosenau, Governance without Government (1992) —— 全球治理：“没有政府的治理”试金石 - R-37 Tricker, Corporate Governance（1984）—— 掌舵 vs 划桨 - R-57 ITIL 4 / COBIT 2019 —— IT 服务管理 / IT 治理框架（PDCA 对照与”运营”用语；治理模型以 ISO 38500:2024 §6 四活动为准） - R-60 许慎《说文解字》(东汉) —— “治”=治水疏导（三点水）、“理”=治玉顺纹——治理字源

> 对照阅读：本章”数据所有权、变更边界，到底谁说了算”呼应第 12 章、“谁有资格批准基线”呼应第 13 章、“复盘是传感装置”呼应第 8 章、§14.2.6 架构治理呼应第 18 章（架构师把治理制度背在身上）。

# 第 15 章 架构定义落地：从目的到架构视图

本章主题：架构定义怎么从一句目的，一路落成一组有规格、有治理、可对账的架构视图——链条九步 / 关切台账与裁决 / 基本判据的三道测试 / 视角规格六项 / 视图的三条硬要求 / 两把评估的尺子 / 两个闸门 演示对象：恒信万辉（图很全、链条断了）与冰链水忠（老规矩没写成规格） 对应章节：承接第 10 章（架构）、第 11 章（架构设计）、第 12 章（架构决策）、第 14 章（治理）；下接第 16 章（架构定义到设计定义）、第 17 章（整合）、预演第 18 章（架构师） 术语口径：本章「架构视角／架构视图／关注点／基本概念或属性／管控原则／生存周期／利益相关方」按 GB/T 45630-2025（等同采用 ISO/IEC/IEEE 42010:2022）定名——第 10、11、12 章已于 2026-09-23 同步迁移（第 10 章原「视点／视图／架构框架」、第 11 章原「治理原则」）；全书其余章节的迁移随批走（每批把该章所在篇铺齐）。对照表见附录 A。

---

## 15.1 误解现场

恒信合同平台的架构评审会，开在评优会之前。满海把电子签章改造的验收单压在桌角，说了句开场白：“今天不看进度，只看一件事——**这份架构定义，能不能接住平台接下来的三年**。”

万辉站起来，打开投影。十五张图，从业务流程到部署拓扑，一张比一张细。他讲了四十分钟，讲完，会议室安静了几秒。

满海先问：“‘数据归属唯一’这条，在哪张图上？”

万辉翻了翻：“数据架构图里有实体关系。”

满海摇头：“我要的不是实体关系，是**谁对哪个对象负责**。你图上没有这一行。”

玉杰接着问：“我最想看的是’一份合同从发起到归档，跑在哪、监控什么’。你这儿还是服务清单——**部署在哪、坏了从哪查，图上找不到**。”

黄程也开口了，说的是业务方那句问了三年的话：“这版能不能保证月底账目对得上？图上看不出来。”

徐漾最后问了一个问题。他没有指着任何一张图，只把手里那支笔转了一圈：

“万辉，你这十五张图，**每一张是按哪个视角画的？它框定谁的什么关切？这条关切又是从哪条目的来的？**”

万辉张了张嘴。他答得出每张图画了什么，答不出这三个问题里的任何一个。

“那这就不是一份架构定义，”徐漾说，“**这是十五张图**。”

---

## 这一章要回答什么

读完本章，你应该能回答：

1. 架构定义落地是一条什么链？为什么它的起点是目的、终点不是图？

2. 为什么目的必须写清”舍掉了什么”？同层锚定检查查的是什么？
3. 为什么目的之后要立刻写”基本概念或属性 + 管控原则”？它们和目标、关切是什么关系？
4. 关切和需求差在哪？三条判据是什么？为什么关切台账里不许有”悬空”的一行？
5. 关切凭什么可以被裁决？裁决靠的是原则，还是可指认的事实？
6. 谁裁决关切？架构师和治理机构各管哪几关？
7. 没有现成的视角标准怎么办？一份视角规格必须写明哪六项？
8. 三个领域的视角集差别在哪？为什么说”差别在行、共性在列”？
9. 为什么每张视图都必须声明治理它的视角？一张视图能有几个治理视角？
10. 视图之间对不上，是图不好，还是架构不成立？
11. 架构评估的依据到底是目的还是目标？评架构与评描述差在哪？
12. 发现缺口谁补、什么时候补？两个闸门为什么一个都不许动？

---

## 这一章的骨架

第 10 章讲架构是什么，第 11 章讲架构设计怎么产出它，第 12 章讲架构决策怎么定。三章加起来，还差最后一段路——**从一句目的，到一组能被人用、能被人查、能被人评的架构视图，中间要走几步、每步由谁负责**。

这一章立的就是这段路。骨架是**一条链 + 三张对账表 + 两个闸门 + 三个反直觉命题**：

- **一条链**——九步：目的 → 基本概念或属性与管控原则 → 架构目标 → 利益相关方关切 → 架构视角 → 架构视图 → 架构评估 → 描述评估 → 双向对账；
- **三张对账表**——判据链（关切 → 目标 → **基本条目**）、描述链（架构 → 视角 → 视图）、过程链（概念化 → 细化 → 评估），各管一条链的对账；
- **两个闸门**——闸门甲（定义完成，不许再砍目标）、闸门乙（评估启动，不许再改目的）；
- **三个反直觉命题——图不是产物，架构视图才是**（图只是视图组件的一种形态）；**关切不是判据，是判据的现场**（判据只有一个，就是目的）；**标准闭不上的缝，只能由责任来闭**（裁决要靠人，不靠原则）。

第一幕把这条链逐步拆开（是什么），第二幕把最像的四样东西剥下来（不是什么），第三幕把它走完（往哪去）。

三幕走完，这一站内部就走通了。但这一站不是终点——**这份架构描述要交给下一段工作，有人拿它去定元素、定接口、定取值**。交接怎么交、交出去什么、对方接住什么，是下一章的事；而这条链上每一个要签字的地方签的是谁的名字，要到第 18 章才点。

---

## 15.2 第一幕 架构定义落地是什么：一条九步的链（定界）

> 这一幕先立链条，再看每一步——九步里，四步归定义、两步归表达、两步归评委、一步归对账。

### 15.2.1 一条链：从目的到架构视图

“架构定义落地”不是一个动作，是一条链。链的九步摆出来是这样：

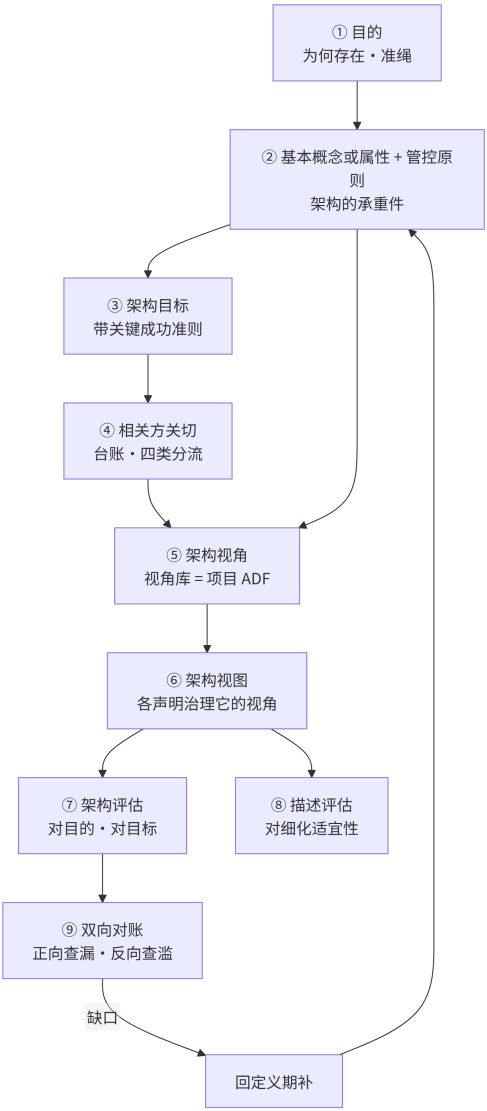



图 19　图 15-1

这张图有四件事值得先记住：

**第一，目的在链头，图不在链尾**。链尾是”对账”，是一个动作，不是一张图。图只是第⑥步产出的视图组件的一种形态——**把图当成终点的人，会在链上只走一步，还以为走完了全程**。

**第二，第②步和第③步是两回事**。基本概念或属性是”承重件”——定义句里真正承重的那一半；架构目标是”判定量”（怎么知道目的达成了）。把两者混成一张表，是最常见的重复计数来源。

**第三，第④步在第③步之后**。关切是原料、目标是产物——真实顺序是先收集关切、再由治理机构把关切确立成目标。倒过来写（先定目标、再找关切来背书），就是把裁量藏起来了。

**第四，第⑦步和第⑧步是两把尺子，不是一把**。评架构对目的（ISO/IEC/IEEE 42030），评描述对细化过程的适宜性（ISO/IEC/IEEE 42020 §10.4.6）。串了尺子，评估就退化成文档审查。

### 15.2.2 第一步：目的是准绳，不是口号

链头这一步，恒信做得最差，冰链做得最好——但两家的做法都值得看一眼。

**目的要写到什么程度才算写清？**六件事：它为什么存在、锚定的是哪个实体、舍掉了什么、为什么舍、什么条件下要重审、由谁赋予并签字。冰链 M3 的目的写得像一句人话——“让疫苗在冷链运输中一路不失活、安全送达”（第 3 章 §3.2.2）；恒信合同平台的目的也写得不差——“让合同审批合规、月底账目清楚对得上”。

问题出在**第三件事**上。**目的声明书里最见功力的，是”舍掉了什么”这一栏**。读不出”舍了什么”，等于没写——因为一个什么都想要的目的，等于没有目的。

M3 舍掉了什么？舍掉了”箱体自己发电”、舍掉了”全程实时上传”（做的是断点补采）。恒信的平台舍掉了什么？舍掉了”每个业务域的极致深度”、舍掉了”无需配置即开箱可用”。**每一条”舍”，都是后面架构目标不去追、基本条目不去长的地方**。

写完目的，还有一道机械检查——**同层锚定**：目的和目标锚的是同一个实体吗？M3 的目的是设备级的；“提高签派可靠率”是运营级的，“扩大空域容量”是空域级的——后两者只能作为约束或环境条件进场，不能当架构目标的来源。**拿 A 尺子量 B 东西，是目的这一关最常见的失守方式**。

这里还有一条第 18 章会展开、本章先立住的界线：**目的的定义权和问责在业务方，不在架构师**。架构师对目的只有四件事可做——理解、澄清、质疑、守护；**没有改写权**。所以第②步到第⑨步的全部工作，都是”对着一份别人签过字的目的”做的。

### 15.2.3 第二步：把承重件一次写完

目的之后紧跟的，不该是一堆图，也不该是一张目标表，而是**基本概念或属性 + 管控原则**——也就是 42010 那句定义的全部承载物：

> **架构**：实体在其环境中的基本概念或属性，以及该实体及其相关生存周期过程的实现和演化的管控原则（governing principles）。（ISO/IEC/IEEE 42010:2022 §3.2）

**为什么要紧跟目的？** 三个好处，一个比一个实在：

- 目的之后立刻看到”这份架构由什么承重”，就不会把目的与目标混为一谈；
- 承重件一旦列出，第③步的目标、第⑤步的视角都可以**对着它**检查——越界没有、漏项没有；
- 概念属性与管控原则并列摆开，一眼能看出**哪些是承重物、哪些是约束**。

**哪样是承重物，哪样不是，有一条硬分界：概念与属性产出可登记的”基本”条目，准则不产出。**准则约束的是”别人怎么做”，它本身不长出任何东西来。**把准则拿去追关切，会得出”准则无据”的错误结论**——这是第⑤步对账时最容易踩的坑。

冰链水忠那份”老规矩”就是典型：接口字表冻结、降级定义、告警语义，全是准则；“空中/路上自主获取的状态”“独立性”“跨代接口契约”那种东西才是概念。水忠的老规矩一条都不缺承重件支撑，可他从没把承重件单独写出来过——**所以他每次讲架构，都得从头讲一遍故事**。

### 15.2.4 第三步：目标与关切——原料与产物

第③④两步是一对，先把三句话记住：

> **关切是原料，目标是产物，目的是准绳。判据对目的量，不对关切量**——回答”什么够格进架构”。**目的对关切点账，不对着偏好量**——回答”这个目的合不合法”。

**两句必须成对用**。只用第一句，目的可以独断；只用第二句，架构会被偏好撑胖。

**关切与需求不是一回事**。关切属于人——立场、利害、忧虑的方向，常常还没有解；需求属于系统——含主体、条件、指标、验证方法的陈述句。三条判据把它们分开：

| 判据 | 问法 | 结论 |
| --- | --- | --- |
| **换人测试** | 换个利益相关方，它还在不在? | 在 → 关切；不在 → 需求 |
| **改写测试** | 能问出通过／不通过吗？需要指标与验证方法吗? | 需要 → 需求 |
| **粒度检查** | 停在哪一级? | 太抽象挑不出视角、太具体进了设计——合适粒度＝**刚好足以挑出视角** |

**关切台账里不许有”悬空”的一行**。每条关切必须四选一：

| 类型 | 判别 | 出路 |
| --- | --- | --- |
| **承重型** | 不满足，目的就达不到 | 产出架构目标与可登记的”基本”条目 |
| **约束型** | 不产生元素，只限可行域 | 转成边界条件，挂在相关元素上 |
| **偏好型** | 只影响方案品味 | 下沉设计或运维 |
| **交付型** | 影响成本、进度、组织接口 | 进项目基线 |

**空着的就是漏项，治理要查**。这条规则看着琐碎，它是整套机制里最省事的一道保险——**因为”悬空”是架构缺失唯一无法伪装的状态**。

### 15.2.5 第四步到第六步：视角是规则，视图是产物

第⑤步是全部关键：**视角不是挑的，是按规格写出来的**。

先分清两个东西。**架构视角**是”创建、解读与使用架构视图、以框定一项或多项关注点的一组惯例”——它是规则；**架构视图**是架构描述里的一部分——属概念是**信息部件**，它是产物。两者的属概念**不同位**：后者是 42010:2022 §3.7 明写的 information part；前者那句定义（§3.8）是**裸名词短语，正本没写属概念——正本凡需属概念必明写**（§3.3 架构描述＝work product、§3.7 架构视图＝information part），所以照正本写「一组惯例」，**本书不替它补一个标准没写的属概念**。

> **术语注（架构视角的定义句）**：42010:2022 §3.8 逐字——set of conventions for the creation, interpretation and use of an architecture view (3.7) to frame one or more concerns (3.10)（创建、解读与使用架构视图（3.7）、以框定一项或多项关注点（3.10）的**一组惯例**）。（引文一手：42010:2022 §3.8。）
>
> 那么，一组惯例凭什么能入库、能版本化、能跨项目复用？**依据不在属概念，在 §3.8 注 3**：The conventions of an architecture viewpoint are documented in a specification (3.16) of that viewpoint. In some communities and architecture description

> frameworks, “view specification” and viewpoint are synonyms.（架构视角的惯例记载在该视角的**规格**（§3.16）里；在某些团体与架构描述框架中，“视图规格”与视角是同义词。）
> 规格本身也是一个**信息部件**（§3.16）——惯例因而有名字、有版本、有位置可放，这才是视角成其为组织资产的那一步。

两者之间那个关系，标准里写作 **governs**，中文作”治理／管辖”。**治理在这里的实质是可判定的符合性**：视角是规则书，视图是按规则书做出来、并且声明”我服从哪本规则书”的产物。

三条边界必须记牢：

- **治理 ≠ 包含**。视角不是视图的一部分，它通常独立存放在视角库里，可以治理多张视图、也可以治理别的实体的视图——**这正是架构描述框架（ADF）能成为行业资产的前提**；
- **治理 ≠ 生成**。视角不产出内容，内容始终是架构师针对这个具体实体做的；
- **一张视图恰有一个治理视角**（1:1），一个视角可以治理多张视图（1:N）。**若一类视图要同时交代两类关切，正确做法是拆视图、拆视角，而不是给一张图挂两个视角**。

第⑥步还有一层容易被忽略的落差：**标准里的”视图组件”不止模型**。基于模型的视图组件（框图、表格、矩阵）由**模型种类**管辖；非模型的视图组件（清单、决策记录片段）只由**图例（legend）**记录约定。

**换句话说，“图”只是视图组件里最显眼的那一种——把它当成视图的全部，就会漏掉架构描述里一半的硬货。**

### 15.2.6 第七步到第九步：两把尺子，一张对账表

链尾三步，是这一章最容易跟第 8 章串味的地方，必须分清。

**第⑦步「评架构」对的是目的。**ISO/IEC/IEEE 42030 的官方摘要把评估基准写得很直白：assess the quality of architectures with respect to their **intended purpose**（对照其**预期目的**评估架构质量）。同一条摘要里还有一句同样重要的话：determine whether architecture entities address their **intended purpose**（判定架构实体是否达成其预期目的）。

**第⑧步「评描述」对的是细化过程的适宜性**，归 42020 §10.4.6（评估架构细化）。这里有一条常被引错的界碑——42030 摘要的注里写着：This document addresses the evaluation of an architecture and **not** an evaluation of the architecture description’s suitability.（本文件针对的是架构的评估，**而非**架构描述适宜性的评估。）

**两把尺子混用的后果很具体**：拿”描述符合不符合格式”当评估，评估就退化成文档审查——**报告里出现”这张图没画好”这类结论，就是串尺子了。**

**依据是目的，判定靠目标。**目的作准绳，但目的常常是一句话，量不了；真正上手量的，是目的拆出来的若干条**架构目标**，每条带关键成功准则。两层各有各的崩法：目标齐全但追不到目的 → **无据扩张**；目的清楚但目标缺项 → **架构缺失**。

这里还有一处中文里的撞车，必须点名：标准里有两个 purpose。**intended purpose**（预期目的）是被架构的实体的属性，是 42030 的评估基准；**process purpose**（过程目的）是过程自己的属性，42020 把它定义为”实施该过程的高层级目标”。**引用时凡出现”目的”，必须标明是哪一个。**

**第⑨步「对账」是双向的**——正向查漏（哪条关切没有视图承接）、反向查滥（哪张视图上的东西追不到任何关切）。这一正一反，正是第 24 号研究所称的**双向覆盖检查**：**未响应＝遗漏，无依据＝凭空扩张**。

---

## 15.3 第二幕 架构定义落地不是什么：四处最像的地方（磨边界弧）

> 这一幕把四样最容易被当成”落地”的东西剥下来：画图交付、关切清单、视角照抄、文档审查。

### 15.3.1 不是”画图交付”：图是视图组件，不是架构

回到章首那十五张图。万辉的错不在图画得不好——**他的错是把”交付十五张图”当成了”交付一份架构定义”**。

判据在这里：**删掉图，还剩下什么？**如果删掉图之后，基本概念或属性、管控原则与关切、目的之间的关系一起消失了，那交的就不是架构定义，是画册。

再往里看一层，链上有两个界，跨不过去：**架构视角** →（**governs**）→ **架构视图** →（**represent 描述**）→ **架构落点** →

（**realize 实现**）→ **系统元素**。图上那个”审批引擎”方框，是**描述层**的视图组件，它代表的是**实体侧**的构成，而不是实现层那个真实的构件。**同形，不同界——这就是”架构＝粗粒度的设计”这一误读最难根除的原因**：打开两份文档，看到的都是方框和箭头。

> **术语注（架构实体 ≠ 架构落点 ≠ 系统元素）**：中文圈常把 architecture entity 读成”架构的构成物”，这是误读。15288:2023 §5.9 逐字写着——Note that an architecture entity (an entity being architected or one subject to the system architecture definition process) and a system element (a discrete part of a system that fulfils specified requirements) represent two different notions.（注意，架构实体（被架构的实体，或受系统架构定义过程约束的实体）与系统元素（系统的一个离散部分，用于满足规定要求）代表两个不同的概念。）（引文一手：15288:2023 §5.9。）42020:2019 §3.6 给”架构实体”的定义是 thing being considered, described, discussed, studied or otherwise addressed during the architecting effort——**“被架构的那个事物”**，与 42010:2022 的 EoI（所关注实体）同指。那么本链上的”**架构落点**”是什么？它是**本书工作名**（见附录 A **T-168**，**非 ISO 术语**；**2026-10-11 由旧名「架构构成物」改名**——旧名是 2011 版 embodied-in 的换词存活、读起来像”

> 架构由它构成”）：指**架构”可以关乎”的对象**，住在**实体侧**（components 与 relationships，42020:2019 §3.3 注 1，注释级、无术语地位）、由视图经 represent 指代、再由实现经 realize 兑现为**实现层的系统元素——它既不是架构实体，也不是系统元素**。三样东西分住三层：视图组件在描述层、架构落点在实体侧、系统元素在实现层。**本章的”同形，不同界”，界的正是这三层**。

### 15.3.2 不是”关切清单”：关切不是判据，是判据的现场

有了台账，是不是就能裁”什么够格进架构”了？不是。

**关切的多源、会漏、会歪、会打架，使它无法承担判据**。关切是**发现物**——谁在乎有事实可查；而判”够不够格”是**裁量物**——立哪几条、定多严，权力在此。**真正随意的从来不是关切，是目标**。

同理，另外几个候选判据也得让位：

| 假判据 | 为什么会冒充判据 | 两类失真 |
| --- | --- | --- |
| **不可逆性** | 可观测、直观、易沟通 | **漏判**：不贵但承重（“日志须可被外部审计员独立读取”）；**误纳**：很贵但与目的无关（框架版本、组织惯例技术栈） |
| **可判定性／可引用性／高层级／概要** | 都是”看起来像判据”的代理 | 换一件外衣的同一个错——**代理冒充真判据** |

**代理不是没用，是不能当总闸**。它们的正当用法是排序、配治理强度与做回代检查；**判据只有一个，就是目的**。

### 15.3.3 不是”视角照抄”：三域的差别在行，共性在列

“要哪些视角”这个问题，很多团队的做法是找一份行业框架照抄。三**个领域的目录摆出来，会看到一件有意思的事：**

| | 实物产品 | 企业运营体系 | 作战任务系统 |
|---|---|---|---|
| **权威框架** | ISO/TS 14812（明文四条）+**无统一视角目录** | TOGAF 四域 + ArchiMate | DoDAF 八视角／NAF 网格／UAF 11×11 |
| **主干「行」** | 功能 → 物理 → 通信 → 企业 | 业务 → 数据 → 应用 → 技术 | 能力 → 作战（逻辑）→ 服务／系统 → 资源 |
| **「列」轴** | 复用 NAF／UAF 的关注面 | 用途 × 内容两轴 | NAF 七列／UAF 十一列 |

**结论：差别在”行”（谁的事），共性在”列”（从哪个侧面看）**。任一领域自建视角，都在往这两个轴上落格。

还有一条更该知道的负面发现：**实物产品领域没有一份现成的权威视角目录**。INCOSE 系统工程手册给的是过程，不是视角清单；能取到明文定义句的只有 ISO/TS 14812 那四条（而且带着 ITS 领域限定词，只能作结构性迁移，不能当通用标准原句照抄）。

**所以”没有标准”不是死路——标准早就给了正解：你自己建一个 ADF**。

### 15.3.4 不是”文档审查”：评估的对象是架构，不是描述

第 8 章讲过评审、验证、确认三个看门人，那里评审的尺子是”适宜、充分、有效”。本章第⑦⑧两步用的是另一套尺子。

两者不能互相顶替：

| | 架构评审（第 8 章） | 架构评估（本章第⑦步） |
|---|---|---|
| 对什么判 | 目标、规定需求、预期用途三把尺子 | **目的**（42030 摘要 b／d 条） |
| 谁做 | 评审组、同行、委员会 | 评估方（宜独立于设计方） |
| 产出 | 评审结论与行动项 | 评估计划与评估报告（42030 §8） |

**评架构对目的，评描述对细化**；两者都不是”图好不好看”。**报告里出现”这张图没画好”，说明的不是图有问题，是尺子串了。**

### 15.3.5 不是”评估现场补图”：缺口回定义期，两个闸门都不许动

第⑨步对账一旦查出缺口，最自然的反应是当场补一张图。**这是把两个过程搅在一起了。**

正确归属只有一句：**缺口回定义期补，并经闸门重审——评估只出结论，不动手。**两个闸门就立在这条界上：

- **闸门甲（定义完成）**：**不许再砍目标**——目标砍了，关切就断供；要砍得走治理重审，并回改目的声明书；
- **闸门乙（评估启动）**：**不许再改目的**——评估基准一变，整轮评估作废重来；目的要改，先改声明书，再过闸门甲。

**这两条一立，那个自由裁量区就被两侧闸门夹住了**——裁量只允许发生在定义期，而且必须留痕。

### 15.3.6 不是”标准说了算”：标准闭不上的缝，只能由责任来闭

这一段是本章的收口，也是本章最硬的一句。

翻遍 42010 家族，会发现**两处该有判据的地方是空的**：一处是”什么算基本”——定义句只说架构住在概念、属性与原则里，一个字没说哪些算基本；另一处是”关切凭什么被裁决”——两条关切定义都只说”利害／重要性”，**没有任何过滤器**，也不说由谁判去留。

**这两处空白不是标准的疏漏，是范围使然**：标准管的是描述的结构与表达，不管过程与方法；而”什么算基本”“谁来裁”恰恰是过程与方法的事。

于是答案只能从另一侧找：**判据的来源是架构治理**。治理的核心动作是裁量——某条内容合规还是不合规；既然是裁量，就必须有一把不随人变的尺。这把尺不能是关切（关切互相冲突，治理存在的理由恰恰是仲裁冲突），**只能落在目的上**：

> **治理需要一把高于个体关切的尺，那把尺就是目的。**

而裁决主体在标准里是有据可依的——42020 把三个动作写在架构治理过程名下：Establish architecture collection objectives（确立架构集合目标）、Make architecture governance decisions（作

出架构治理决策）、Establish architecture objectives and critical success criteria（确立架构目标与关键成功准则）。

**“确立目标”与”作决策”是治理过程名下的动作，不是架构师名下的动作**。

---

## 15.4 第三幕 架构定义落地往哪去：把这条链走完（运用）

> 这一幕讲手艺：视角规格怎么写、视角从哪来、视图有哪三条硬要求、对账怎么做、关切分四关怎么裁、缺口长什么样。

### 15.4.1 视角规格六项：写一次，用很多次

“视角”这个词最大的误会，是把它当成一种立场或一种眼光。**它不是立场，是约定；不是心情，是规格**。

一份合格的视角规格必须写明六项——这六项同时也是它对视图的约束力：

| 视角必须规定 | 对视图的约束力 |
| --- | --- |
| 名称、出处（来自哪个 ADF）、版本 | 视图据此声明受谁治理 |
| 框定哪些关切、服务哪些典型利益相关方与角度 | 视图不得越界，也不得漏掉本视角该管的关切 |
| 采用的模型种类、记号与建模约定 | 视图的每个视图组件必须落在这几类模型里 |
| **对应方法** | 本视图与其他视图的挂钩方式照此办理 |
| **视图方法：建造类／解读类／分析类** | 他人能按同一套方法复现、读懂、检查这张视图 |
| **本书归并口径**：必需的模型、元素、关系、约束与属性 | 视图该有什么、可以没有什么，一望即知 |

> **出处注（本书归并口径）**：前五项逐条对得上 42010:2022 **§8.1 的 shall 清单 a)~g)**——a) 利益相关方角度；b) 本视角框定的关注点；c) **方面体**；d) 典型利益相关方；e) 模型种类与图例；f) 对应方法；g) 参考信息来源（“名称、出处、版本”另见附录 B 模板 B.2.2／B.2.3／B.2.13）。**“视图方法”是 §8.1 的 should 项（§8.3），不是 shall。末一项“必需的模型、元素、关系、约束与属性”是本书的归并口径，§8.1 正本无此项**——别把它当标准原文引；标准在这一节规定的是 a)~g)，其中 c) 方面体未单列进本表。

后两项最容易漏，也最要命：**缺了对应方法，跨视图一致性无从建立；缺了视图方法，这张图只能由画它的人解释**。

再补一条硬门槛：标准对”规格”（specification）的定义里写着 **complete, precise and verifiable**（完整、精确、可验证）三个词——**“说不清、没法验证”的视角，根本不构成一份规格，也就不构成一个合规的视角**。顺带钉一处属概念：**规格本身也是一个信息部件**（§3.16）——三个词是对”信息部件”的质量要求，不是另一个属概念。

**怎么验一份视角是否成立？五问**：① 视图声明了治理它的视角吗（名称、出处、版本）？② 所用模型种类与记号在规格内吗？③ 视角框定的关切覆盖且不越界吗？④ 与其他视图的对应关系符合对应方法吗？⑤ 按视图方法能复现与检查吗？**五问全过，治理成立**。

### 15.4.2 视角从哪来：四份蓝本，加一个自建的纪律

自建一个 ADF，不必从白纸开始。业界有四份可用的蓝本：

| 蓝本 | 视角集 | 适用场合 |
| --- | --- | --- |
| **RM-ODP** | 企业／信息／计算／工程／技术（五视角） | 分布系统与开放性要求强的系统 |
| **ISO/TS 14812** | 功能／物理／通信／企业（四视角） | 实物产品——**其抽象结构即 RM-ODP 在实物产品语境的落法** |
| **GERAM／ISO 15704** | 功能／信息／资源／组织（四视图） | 企业与制造系统建模 |
| **UAF／NAF 的 P 行** | 资源分类／结构／交互／功能／状态／路线图／数据模型／约束 | 作战任务与体系类系统的资源侧 |

**四份可以混用**，而且有据：42010 的符合性允许多份 ADF 并存，也允许定义项目自有 ADF。**唯一不能省的是：你用的视角必须写出规格，视图必须声明受哪个视角治理。**

自建也有纪律，而且这条纪律来自标准之外的经验：**自建视角有成本**——它意味着额外的描述、评估、管理与使用工作量。**先穷尽复用，再自建；缺的那几个才自己写。**

### 15.4.3 视图的三条硬要求

一份能用的架构描述，落在视图这一级只有三条硬要求：

1. **声明治理视角**——名称、出处、版本三样齐全，**一张视图恰一个**；
2. **模型种类落在该视角规格之内**——基于模型的视图组件由模型种类管辖，非模型的由图例记录，**两类都不能”随手画”**；

3. **跨视图的对应关系可判定**——对应方法规定怎么挂钩，挂钩不上的地方要能被查出来。

**第三条第最容易糊**。它有一个很硬的判读：**视图之间对不上，不是”图不够好”，是视图之间的对应关系断了——那一刻，架构不成立**。因为对应关系承载的正是”这些图说的是同一个架构”这件事。

### 15.4.4 对账：正向查漏，反向查滥

三张对账表各管一条链：

| 链 | 上游 → 下游 | 对账表 |
|---|---|---|
| **判据链** | 关切 → 目标 → 基本条目 | **四级映射表** |
| **描述链** | 架构 → 视角 → 视图 | **视角库 + 描述符合性检查表** |
| **过程链** | 概念化 → 细化 → 评估 | **阶段闸门清单** |

**四级映射表是整块拼图的承重件**，它的样子是这样：

| 关切点 | 架构目标 | 关键成功准则 | 基本条目 | 视图落点 | 依据决策（ADR） |
|---|---|---|---|---|---|
| 疾控要能交出全链温度数据 | 数据可被第三方独立验证 | 断链定位到环节级 | 温度记录带断电保持 | 信息视图·追溯矩阵 | ADR-07 |

三条规则，一条比一条严：**每一行必须打通**（任一格空着＝这一行还没成立）；**每个元素至少一行**（没有行的＝无据扩张）；**每行至少一条 ADR**（没有的，说明它不是”决定的”，是”惯例滑进来的”）。

**反向查滥排在正向查漏之后，不是因为它次要，而是因为它更难**。漏项看得见，滥项看不见——它长得像”更周全”。三种最常见的滥项：技术选型越界（把器件型号写成架构约束）、惯例滑入（把组

织习惯的技术栈写成架构原则）、交付关切升格（把采购价上限写成”基本”条目）。

### 15.4.5 关切分四关裁：举证前置，退件不是驳回

关切进不了架构，通常不是因为它不重要，而是因为**没人说得清它能不能往下走**。所以裁决的对象不是”关切真不真”，是”**关切能不能往下走**”。

四关、四个责任人：

| 关口 | 裁什么 | 程序性判据 | 责任人 |
| --- | --- | --- | --- |
| **立案** | 能不能进台账 | ① 来源可指认（谁在乎）② 后果可指认且可度量 | **架构师** |
| **进架构** | 能不能转化为架构目标 | ③ 与目标可对应 ④ 类型定性 ⑤ 出路非空 | **治理机构** |
| **排序** | 排不排得上、何时做 | 资源与优先级 | **治理机构** |
| **复核** | 目的变更后重审 | 目的声明书 ↔ 关切台账 ↔ 四级映射表逐行对上 | **治理机构** |

**最硬的分水岭是第②条**：能写出期望值与可接受区间的，是关切；写不出的，是愿望。**愿望不立案，理由不是”不重要”，而是”无法判定，因而无法承接”**——这是程序性理由，经得起复议。

**防”随意说”最有效的一招，是举证责任前置**：主张者随附来源、后果、度量三样，**不齐退回补件，而不是驳回**。这一招把”随口说”变成”要举证”——**整个机制不需要任何人做价值裁判**。

它同时还是评估依据正确性的保障：42030 附录 A 的结构是 stakeholder values → qualities → measures（利益相关方价值 → 质量 → 度量）——**关切只有走到”可测”这一步，才有资格参与架构评估**。不设这道关，评估依据就是流沙。

### 15.4.6 两个闸门与十二步最小清单

把前面的手艺收成一份能贴在墙上的清单——**最小可执行十二步**：

| # | 动作 | 产出 | 责任 | 闸门 |
|---|---|---|---|---|
| 1 | 写目的声明书（含舍弃清单） | 目的声明书 | 治理机构 | 立项 |
| 2 | 建关切台账（出路不许空） | 关切台账 | 架构师 | 立项 |
| 3 | 同层锚定检查 | 层级核对记录 | 架构师 | 立项 |
| 4 | 确立架构目标与关键成功准则 | 目标清单 | 治理机构 | 概念化出口 |
| 5 | 建视角库（按六项规格写） | 视角规格集（项目 ADF） | 架构师 | 细化入口 |
| 6 | 出视图、声明治理视角、记对应关系 | 架构描述 | 架构师 | 细化出口 |
| 7 | 冻结基本条目清单 | 基本概念和基本属性清单 | 治理机构 | **闸门甲** |
| 8 | 双向覆盖检查（正查漏、反查偏） | 缺口清单 | 架构师 | 闸门甲前 |
| **10** | **[illegible]** | **[illegible]** 909 报告 | 评审（架构师独立于设计方） | **闸门乙后** |

**注意责任栏的分工**：第 1、4、7、10 步在治理机构与评估方手上，第 2、3、5、6、8、9、11、12 步在架构师手上。**取舍不是架构师的活，举证与对账是**。

### 15.4.7 缺口长什么样：三种缺口，三种补法

缺口不是一种病，是三种。看两个案例加一份公开示例，三种缺口各有各的长相：

| 缺口 | 现场长相 | 补的是什么 | 什么时候补 |
| --- | --- | --- | --- |
| **补不变量** | 视图定义了数据字与更新率，没定义”既有字义不可变” | 一条**冻结清单＋兼容规则**（不变量） | 定义期，闸门甲前 |
| **补可检出性** | 视图定义了失效与降级，没定义”哪些失效必须被监控、多久检出” | 一张**监控覆盖矩阵**（错误可检出性） | 定义期，闸门甲前 |
| **补判据本身** | 准则写着”扩展点稳定”，却没定义”什么算破坏性变更” | **一条可判定违反的判据** | 定义期，闸门甲前 |

三种缺口的共同点：**它们都不在评估阶段解决**。评估只出结论——“此处缺口会导致哪条目标无法判定”；补齐的动作，以及因此需要的变更，全部回定义期，走闸门重审。

第三行最值得盯：**准则不可判定＝拦不住任何东西**。一条写着”应当追求稳定”的准则，和一句祝福没有区别。**准则必须写到能判出违反，才算立住了**。

---

## 案例进展①｜恒信：万辉补的三关

评审会散场那天下午，徐漾没走。他把万辉那十五张图在桌上摊开，按顺序摆好，然后抽出三张白纸。

“第一张，写关切。”他说，“不许写需求，只写’谁在乎、在乎什么、不满足会怎样’。”

万辉写了半页就卡住了——他发现”月底账目对得上”这条后面，写不出可度量的后果。

“写不出就先别写，”徐漾说，“**这不是不重要，是还接不住**。”

第二张纸写视角。万辉第一次试着给他那些图贴标签：合同流程图画的是”业务过程”、数据图画的是”数据对象与归属”、部署图画的是”物理与部署”。贴到第四张，他停住了：**有一张图，贴不上任何一个视角**。那是张”系统总体架构图”——什么都有，什么都不全。

“那张不是视角的产物，”徐漾说，“那是目录页。”

第三张纸写对账表。六列：关切、目标、成功准则、基本条目、视图落点、决策。第一行填上，第二行就开始空；到第七行，万辉自己停笔了——“**视图落点这一列，有一半是空的**。”

玉杰凑过来看了一眼那张表，指着”平台运维可诊断”那一行：“这条我以前提过三次，没人记。”他顿了顿，“现在它有行了。”

那天下午，恒信多了三张纸：一份关切台账、一份视角清单、一张填了七行的对账表。**图还是那十五张，但从这一天起，它们知道自己归谁管、为谁画、为什么画**。

## 案例进展②｜冰链：水忠的老规矩，原来是一份没写规格的视角

冰链这边，是李旭先看出问题的。

M3 二代的接口评审上，水忠照例把老规矩念了一遍：数据字字义不许改、扩展只许追加、降级是能力分级的连续谱。李旭听完问了一句：“这些规矩，除了你，还有谁按同一套办法查得出来?”

水忠愣了一下。

“我不是说规矩不对，”李旭说，“我是说——**你这份规矩，别人拿着它，能不能画出同一张图、看出同一个错?**”

文正把话接了过去：“字表冻结、追加式扩展、未知字忽略……这些是**约定**，不是经验。约定的东西能写成规格。”

那天下午，水忠做了件他从没做过的事：把”接口”写成了一份视角规格。“名称、出处、版本”三行先卡住他——出处写什么？文正说：“写’冰链数据总线约定 v2′，它就是我们项目的 ADF。”

写到”对应方法”那一栏，他停了很久：“安全视角的失效状态，怎么跟通道划分挂钩”——这句话他在脑子里想过无数遍，从没写下来过。写下来才发现，**有些地方他自己也没想清楚**。

董勐在”模型种类”那栏旁边补了一句：“那张数据字表，得说清楚是给人读的还是给机器校验的。”逸人补了最后一栏：“扩展规则得能让产线查——哪批箱子升了、哪批没升，它得对得上。”

规格写完，水忠把它打印出来，钉在工位挡板上。**那几条老规矩还是一样的字，但从此它不靠他一个人守了**。

# 案例进展③｜两个最不像的系统，同一句定义，两副承重件

走到这里，把两条线摆在一起看一次。这一章讲的链条，两个世界都走通了，但**承重的地方不一样**：

| 面向 | 冰链 M3（设备类） | 恒信合同平台（平台类） |
| --- | --- | --- |
| 第一条基本概念 | 空中/路上自主获取的状态（对外部世界的**认知**） | 可配置性（对自身能力的**承诺**） |
| 命门概念 | 独立性 | **数据归属唯一 + 语义唯一** |
| 承重件偏哪一侧 | **偏属性**（精度、时延、误警漏警、降级） | **偏概念与准则**（归属、注册、可配置优先、配置不得越界） |
| 最危险的失效 | 漏警 | **配置出来的孤岛**（配置里悄悄多一份副本） |
| 最易漏的关切来源 | 维修单位、监管（沉默方） | 数据治理、审计、法务、二次开发方（**同为沉默方**） |
| 对应关系的死点 | 物理 ↔ 安全 ↔ 通信（通道对齐） | **权属 ↔ 注册 ↔ 配置（归属与写入权对齐）** |
| 缺口补法的性质 | 补不变量 | 补**判据本身**（可判定性） |

**一句话对照：M3 是”测得准、说得清”，恒信平台是”改得动、守得住、散不开”。**

合起来是一条结论：**同一句定义句（基本概念或属性 + 管控原则），在设备类产品上承重件偏属性，在平台类产品上偏概念与准则。**

这印证了一件事——**清单不能照抄，只能按各自的目的推。**

还有一条更该记住的：两个系统最易漏的关切来源，**都是沉默方**。维修单位、监管、审计、法务、二次开发方——他们不主动提需求，后果却最重。**台账上少了他们，不是少了几行字，是少了几条命门。**

---

## 认知反转

回到章首那间会议室。万辉交了十五张图，被问了三个问题，一个也答不上来。

如果你读完了前面十五章，你会先替他辩解一句：图画得挺全，决定也写了，怎么就不算架构定义？

这辩解站不住，因为它连错了四层。

**第一层，把”画图”当成”落地”。**图是链条第⑥步里视图组件的一种形态。**万辉走了一步，交了一份看起来像全程的东西。**

**第二层，把”关切”当成了可以跳过的一步。**他跳过的不只是一张台账，是**为谁画**这个问题。一张没有对应关切的图，被追问时只能回答”图上有什么”，答不出”它替谁解决什么”——这正是黄程那句”这版能不能保证月底对账”的来历：**他问的不是图，是台账上他那行。**

**第三层，把”视角”当成了画图习惯。**视角不是习惯，是规格；规格不写下来，图就只能由画它的人解释。水忠的老规矩为什么值钱？

因为它本来就是一份没写出来的视角规格——**写下来之后，它才从一个人的手艺，变成一群人的约定。**

**第四层，也是最深的一层，以为判据在自己心里。**万辉心里其实有一套”什么重要”的直觉，他的图也照着那套直觉排了轻重。**可直觉不是判据：判据要对着一份别人签过字的目的，逐条量。**

这一章真正要立的，是那句最短的话：**图不是产物，架构视图才是；关切不是判据，是判据的现场；标准闭不上的缝，只能由责任来闭。**

用第 1 章的话收一次束：对”架构定义落地”，你原来只是”见过”——见过模板、见过图的样式、见过别人家的架构文档。这一章把它推到”拥有”——能说出链条九步各是什么、每一步归谁、三项对账各查什么、两个闸门各拦什么。**能答全这四问才算拥有。**

再用第 1 章 §1.4.4 那四层量一次：这一章站在**第 ③ 层（心智模型）**——它给你的不是新概念，是一张能推演的作业图：拿到任何一份架构定义，你都能顺着链条走一遍，指出它在第几步断的。

链条走完，这一站内部就通了。但架构不会自己活下去——**这份架构描述马上就要交出去，交给下一段工作。**那是下一章的事；而这条链上每一个要签字的地方签的是谁的名字，是第 18 章的账。

## 小结

这一章讲的是架构定义的”最后一公里”——从一句目的，到一组能被人用、能被人查、能被人评的架构视图。

它立了一条九步的链：目的（准绳）→ 基本概念或属性与管控原则（承重件）→ 架构目标（判定量）→ 关切（原料）→ 架构视角（规则）→ 架构视图（产物）→ 架构评估（对目的）→ 描述评估（对细化）→ 双向对账。

它把四样最像的东西剥了下来：不是画图交付、不是关切清单、不是视角照抄、不是文档审查；也没忘两句界线——缺口回定义期补、标准闭不上的缝由责任来闭。

它给了一套手艺：视角规格六项（尤重对应方法与视图方法）、四份可混用的蓝本（RM-ODP／ISO TS 14812／GERAM／UAF-NAF 的 P 行）、视图的三条硬要求、三张对账表、四级映射表、关切裁决四关、十二步最小清单、两个闸门。

它还给了三种缺口的长相：补不变量、补可检出性、补判据本身——**它们都回定义期补，一个都不在评估现场解决。**

最后，它落了一条对照：**同一句定义句，设备类产品承重件偏属性，平台类产品偏概念与准则**。M3 是”测得准、说得清”，恒信平台是”改得动、守得住、散不开”。清单不能照抄，只能按各自的目的推。

链条走完，还剩最后一个问题——这条链上每一个要签字的地方，签的是谁的名字？下一章回答它。

## 本章概念总图

**架构定义落地 = 一条链（九步） + 一套裁决（四关） + 两把尺子（评估） + 两个闸门**

**第一幕 · 是什么：一条九步的链**

① **目的** 准绳（写清"舍掉了什么"；同层锚定；定义权在业务方）

② **承重件** 基本概念或属性 + 管控原则 = 定义句的全部承载物（概念属性承载，准则不承载）

③ **目标** 判定量（带关键成功准则）

④ **关切** 原料（台账；四类分流：承重/约束/偏好/交付——悬空即漏

项）

⑤ 视角　　规则（一组惯例——照 §3.8 正本，正本无属概念；可复用、可入库的依据是 §3.8 注 3；governs 视图 1:1）

⑥ 视图　　产物（信息部件；视图组件 = 模型 + 非模型；声明治理视角）

⑦ 评估　　对目的（42030）——依据是目的，判定靠目标

⑧ 描述评估　对细化适宜性（42020 §10.4.6）——与⑦两把尺子，不许串

⑨ 对账　　正向查漏、反向查滥（无响应＝遗漏；无依据＝扩张）

第二幕 · 不是什么：四处最像的地方

不是画图交付　图 = 视图组件的一种形态；两个界跨不过去（represent / realize）

不是关切清单　关切是发现物、目标是裁量物；判据只有一个＝目的（不可逆性等只是代理）

不是视角照抄　三域差别在行、共性在列；实物产品无现成目录 → 自建 ADF 是正解

不是文档审查　评架构对目的、评描述对细化；"这张图没画好"＝串尺子

不是现场补图　缺口回定义期；闸门甲不许砍目标、闸门乙不许改目的

不是标准说了算 标准闭不上的缝，只能由责任来闭（判据来源于架构治理）

第三幕 · 往哪去：把这条链走完

视角规格六项　名称出处版本 / 框定关切与服务对象 / 模型种类记号 / 对应方法 / 视图方法 / 本书归并项

+ 规格三词门槛：complete · precise · verifiable；五问验"治理成立"

视角从哪来　　四蓝本（RM-ODP / ISO TS 14812 / GERAM / UAF-NAF 的 P 行）＋ 自建的成本纪律（先复用后自建）

视图三硬要求　　声明治理视角（1:1） / 模型种类在规格内 / 对应关系可判定（对不上 = 架构不成立）

对账　　三张表：四级映射表（判据链）/ 视角库 + 检查表（描述链）/ 闸门清单（过程链）

四级映射表三规则：每行打通 / 每元素至少一行 / 每行至少一条 ADR

关切四关　　立案（架构师）/ 进架构·排序·复核（治理机构）；举证前置：来源·后果·度量

两个闸门　　甲＝定义完成不许砍目标；乙＝评估启动不许改目的

缺口三种　　补不变量 / 补可检出性 / 补判据本身——一律回定义期

两句元规则

判据对目标量，关切点管不漏，取舍要留痕。

标准闭不上的缝，只能由责任来闭。

## 本章判据

> 本章的判据是本书的主张，不是任何标准的条文——标准出处（42010／42020／42030）见正文；凡属形式化（关切四关、举证三件套、四级映射表、两个闸门、三处接缝），已在文中标明。它们都当场可检验：欢迎检验、欢迎校准。

1. **目的门槛**：目的声明书写不清”舍掉了什么”，等于没写。**违反即崩：一个什么都想要的目的，等于没有目的——后面的目标会替它把什么都想要一遍**。
2. **同层锚定**：目的与架构目标必须锚在同一个实体上。**违反即崩：拿部门级目标去问设备级架构**。
3. **判据唯一**：够不够格进架构，只对**目的**量（承重测试），不对关切量、不对偏好量。**违反即崩：用”这个很贵、改不了”当选拔标准，于是漏判了便宜却承重的条目，误纳了昂贵却无关的条目**。
4. **两遍跑**：目的判据要跑两遍——兑现侧（实现目的需要它吗）＋演化侧（目的变了它守不守得住）。**违反即崩：只跑一遍，演进类的基本条目必漏**。
5. **承重测试三档**：删掉它——目的**跨**（概念级基本）／目的不变但实体性质**变形**（属性级基本）／只是**变难**（不进架构层）。**违反即崩：把”变难”当”承重”，架构被撑胖**。
6. **替换测试**：换一个实现，这个概念／属性还在不在？在 → 基本；不在 → 那只是当前实现的一部分。**违反即崩：把当前技术选型写进架构，下一代产品推翻重来**。
7. **准则不追关切**：概念与属性产出可登记的”基本”条目，准则不产出——准则只约束”别人怎么做”。**违反即崩：拿准则去追关切，得出”准则无据”的错误结论，把好准则删掉**。

8. **约束型只挂边界**：适航、环境、人因一类约束型关切不产出”基本”条目，只作边界条件附在既有条目上。**违反即崩：把审定要求当承重件处理，架构被撑胖；当它不存在，产品取不到证**。
9. **关切三测**：换人测试（换个利益相关方它还在不在）、改写测试（能不能问出通过／不通过）、粒度检查（刚好足以挑出视角）。**违反即崩：拿关切当需求收集，收回一堆”希望系统好用”的伪需求**。
10. **台账不许悬空**：每条关切必须是承重型、约束型、偏好型、交付型之一，出路不许空。**违反即崩：悬空的那几行，就是下一次事故的入口**。
11. **举证前置**：主张者随附来源、后果、度量三样；不齐**退回补件**，不是驳回。**违反即崩：没有这一招，“随意”的来源就变成裁决者本人**。
12. **视角规格六项齐备**：尤查”对应方法”与”视图方法”。**违反即崩：规格不全的视角，产出的东西不构成架构视图——它只是一张图**。
13. **视图声明治理视角**：一张视图恰有一个治理视角（1:1）；要交代两类关切，就拆视图、拆视角。**违反即崩：一张图挂两个视角，符合性无从判定**。
14. **对应关系可判定**：跨视图的挂钩方式必须写进规格、能被查出来。**违反即崩：视图之间对不上时，那不是图不够好，是架构不成立**。

15. **两把尺子不串**：评架构对目的（42030），评描述对细化适宜性（42020 §10.4.6）。**违反即崩：评估报告里出现”这张图没画好”，评估已退化成文档审查**。
16. **缺口回定义期**：评估只出结论，不动手；缺口一律回定义期补，并经闸门重审。**违反即崩：在评估现场补图，两个过程搅在一起，闸门形同虚设**。

**工作检查单**（每次交架构定义前，逐条自问）：

- □ 目的声明书写了”舍掉了什么”吗？谁签的字？
- □ 目的与目标同层吗（锚的是同一个实体吗）？
- □ 承重件（基本概念／属性）与准则分开了吗？准则有没有被拿去追关切？
- □ 关切台账每条都有出路吗？沉默方（监管、运维、退役、审计、法务、二次开发方）在台账上吗？
- □ 每条关切的”后果”写得出度量吗？写不出的，退回补件了吗？
- □ 视角库里的每个视角，六项规格齐吗？缺的那几项谁补？
- □ 每张视图声明治理视角了吗？有没有一张图挂两个视角？
- □ 跨视图的对应关系查过吗？查得出”对不上”吗？
- □ 四级映射表每行都打通了吗？每个基本条目至少有一行吗？每行至少一条 ADR 吗？
- □ 这次是评估还是评审？评的是架构还是描述？
- □ 发现缺口，是当场补的，还是回定义期补的？

## 自查 · 这些说法对吗？

1. “架构定义落地，就是把架构图画全、画细。” —— **错**。图是第⑥步里视图组件的一种形态；链条九步，画图只占其中一步。
2. “目的写得越宏大越好，格局大才带得动团队。” —— **错**。目的声明书最见功力的是”舍掉了什么”；读不出舍了什么，等于没写。
3. “目的定完就直接画图，中间那几步可以并行省掉。” —— **错**。省掉的是承重件、目标、关切与视角规格——链条断在中间，尾部再漂亮也接不住。
4. “关切就是需求的另一种说法，收集得越全越好。” —— **错**。关切属于人（立场、利害），需求属于系统（可验证的陈述）；三条判据分得开，且关切要分流，不是全收。
5. “关切多了就先都记下来，反正台账不嫌长。” —— **错**。台账不许有悬空的一行；没有出路的条目，本身就是漏项。
6. “他说不出来后果有多严重，说明这条关切不重要，可以驳回。” —— **错**。要退回补件，不是驳回——理由不是”不重要”，是”无法判定，因而无法承接”。
7. “关切是判据的来源，什么够格进架构要问关切。” —— **错**。关切是发现物，目标是裁量物；判据只有一个——目的。关切管的是取舍与呈现。

8. “不可逆性可以作为’什么算基本’的判据。” —— **错**。它只是代理，会漏判（不贵但承重）也会误纳（很贵但无关）。
9. “没有行业视角标准，就只能等标准出来。” —— **错**。标准故意不给清单，正解是自建 ADF——按规格写出你要用的视角。
10. “三个领域的视角集差别很大，说明视角不可迁移。” —— **错**。差别在行（谁的事），共性在列（从哪个侧面看）；自建视角就是在往这两轴上落格。
11. “视角就是一种立场、一种眼光。” —— **错**。视角是一**组惯例**（§3.8 正本定义句无属概念，本书不替它补）；立场是利益相关方角度，两者不是一回事。
12. “一张图可以同时挂业务视角和技术视角，反正看的人自己会分。” —— **错**。一张视图恰有一个治理视角；要交代两类关切，拆视图、拆视角。
13. “图都画完了，架构定义就算落地了。” —— **错**。落地要过对账这一关：正向查漏、反向查滥，缺口回定义期补。
14. “视图之间对不上，是图画得不够细。” —— **错**。那是对应关系断了——视图之间对不上时，架构不成立，不是图不成立。
15. “架构评估就是审文档，看格式全不全、图规范不规范。” —— **错**。评架构对目的（42030），评描述对细化适宜性（42020 §10.4.6）；串了尺子，评估退化成文档审查。

16. “评估的依据是目标，目的太虚了没法量。” —— **错**。依据是目的、判定靠目标——两层各有各的崩法：目标追不到目的＝无据扩张，目的没有目标承接＝架构缺失。
17. “评估会上发现缺口，当场补一张图最快。” —— **错**。缺口回定义期补；评估只出结论。当场补图，等于把闸门拆了。
18. “准则写得宽一点好，太具体容易限制创新。” —— **错**。准则不可判定违反，就拦不住任何东西——“应当追求稳定”和一句祝福没有区别。
19. “这套流程走完一遍，架构就再也不用动了。” —— **错**。目的变更触发重审（复核关），配置管理让变化说得清（第 13 章）——链条是活的。
20. “标准没规定，说明这块怎么做都行。” —— **错**。标准闭不上的缝（“什么算基本”“关切由谁裁”），只能由责任来闭——判据的来源是架构治理。

## 练习

1. **给一份真实的架构文档做”链条体检”**：拿你们项目最近一份架构描述，顺着九步走一遍，标出它在第几步断的——断在目的、承重件、目标、关切、视角，还是对账？
2. **写一次目的声明书的”舍掉了什么”**：找一个你们在做的系统，写三条”这次不做”，并说明为什么舍。写不出来，说明目的还没写清。

3. **建一张关切台账**：把你们项目的利益相关方列全，含沉默方（监管、运维、退役处置、审计、法务、二次开发方），每条给出”后果的度量”。写不出度量的，标”退回补件”。
4. **视角规格试写**：挑一个你们团队靠经验在守的”规矩”（接口约定、命名规则、降级策略），按六项规格写成一份视角规格——重点补”对应方法”与”视图方法”两项。
5. **给图贴标签**：把你们现有的架构图逐张贴上”治理视角”标签。贴不上的那张是什么？它可能是目录页，不是视图。
6. **双向覆盖检查**：拿一张对账表，正向查漏（哪条关切没有视图承接）、反向查滥（哪张图上哪一件追不到任何关切）——两边各挑出至少一条。
7. **找出一个”不可判定的准则”**：在你们的规范文档里找一条”写得正确但判不出违反”的准则，把它改写到能判违反。
8. **分清两把尺子**：找一份你们做过的”架构评审/评估”记录，看它用的哪把尺子——评的是架构对目的，还是描述对格式？串了的话，重写结论部分。

## 参考文献

**标准** - R-04 ISO/IEC/IEEE 42010:2022 —— 架构定义（实体在其环境中的基本概念或属性＋管控原则）；架构视角与架构视图（规则与产物、governs 关系、1:1 与 1:N）；视角规格（§8.1 与附录 B）；架构描述

框架 ADF（§7.1）；架构描述成分清单（§6.2~§6.10）；本章术语定名依 GB/T 45630-2025（等同采用）- R-05 ISO/IEC/IEEE 42020:2019 —— 架构过程六类，按正本 §5.3~§5.5 分三组：**§5.3 架构治理与架构管理**（两过程同组：治理 §6／管理 §7）／**§5.4 架构概念化、评估与细化**（三个核心过程：概念化 §8／评估 §9／细化 §10）／**§5.5 架构使能**（单独一节：使能 §11）；架构治理决策（§6.4.4）、确立架构目标与关键成功准则（§8.4.4）；评估架构细化（§10.4.6）—— **评描述归此，不归 42030** - R-85 ISO/IEC/IEEE 42030:2019 —— 架构评估框架：assess the quality of architectures with respect to their intended purpose（对照其预期目的评估架构质量）；评估三层（评估综合／价值评估／架构分析）与两个工作产品（评估计划／评估报告）- R-88 ISO/TS 14812:2022 —— 实物产品领域明文命名的四条视角（功能／物理／通信／企业），本章四蓝本之一 - R-89 RM-ODP（ISO/IEC 10746 系列，ITU-T X.901）—— 企业／信息／计算／工程／技术五视角，本章四蓝本之一 - R-90 GERAM／ISO 15704（含 ISO 19439:2006）—— 功能／信息／资源／组织四视图，本章四蓝本之一 - R-91 DoDAF 2.02 —— 八视角（全景／能力／作战／服务／系统／数据与信息／标准／项目）；裁剪纪律：“Fit-for-Purpose”，不规定模型 - R-92 NAF v4.1 —— 行（关注主题）× 列（关注方面）网格，每一格即一个视角；视角选择须裁剪 - R-93 OMG UAF 1.3（ISO/

IEC 19540）—— 十一视角 × 十一方面；自述其视角命名取 42010 语境

**经典文献** - R-45 INCOSE 系统工程手册 —— 实物产品领域**没有**现成的权威视角目录（只给过程）；自建视角规格须有合理理由（额外工作成本） - R-70 SEBoK —— 架构描述应说明”何为纳入、何为排除”；fundamental 一词”没有普适定义”，详细程度取决于使用语境与设计目的 - R-19 TOGAF —— 企业架构四域（业务／数据／应用／技术）；四域与 42010 视角的术语边界（框架内自述，非完整视角定义）

> 本章回扣链：第 10 章（架构·架构描述·架构视角与架构视图）、第 11 章（架构设计·系统基本概念和基本属性清单）、第 12 章（架构决策·不可逆性降级为代理）、第 14 章（治理·架构治理与例外）、第 17 章（整合·接缝与缝合）；第 8 章（评审验证确认·三个看门人）与本章第⑦⑧步的分工见 §15.3.4；附录 A T-152~T-160、附录 C 第 15 章行、附录 E 第 15 章自查速查。

# 第 16 章 架构定义到设计定义：架构与设计的交接

本章主题：架构定义与设计定义是两类工作、两个过程——不是粗细两版，也不是两个阶段；交接的入口动作、两条转化、五条判据、回退通道与递归换手点；设计定义的形态与出口；可实现性关切在架构视角上的归位 演示对象：恒信万辉（把架构描述交出去，被三个”怎么落”问住）与冰链水忠（老规矩第一次被交给别人当约束用） 对应章节：承接第 10 章（架构）、第 11 章（架构设计）、第 12 章（架构决策）、第 15 章（架构定义落地）；回扣第 6 章（设计开发落地·需求与方案结对）；下接第 17 章（整合）、预演第 18 章（架构师） 术语口径：本章「架构定义」「设计定义」取 ISO/IEC/IEEE 15288:2023 两个过程之名（§6.4.4／§6.4.5）；「系统设计描述」「设计描述符」「设计特性」取 INCOSE SEH 5.0 术语表（系统设计说明系同物异译）；「实现」「兑现」取 ISO/TS 14812 §3.1.15.2／§3.1.15.4。design 一词有宽窄两义，凡引用均标口径（§16.2.1）。**本章「架构定义」「设计定义」的过程条款（目的句、四组活动、两条转化、分配、设计演化原则）与 §5.9 分工句、§3.13 design**

> **定义句均直核 ISO/IEC/IEEE 15288:2023 正本；INCOSE SEH 5.0 为同版同年旁证——其中入口动作一句、设计描述符与原子元素两个术语及其判据、横向与纵向集成、“衍生需求纳入系统需求基线”一句，均系 SEH 5.0 表述（正本无此术语／此句）。** 对照表见附录 A。

---

## 16.1 误解现场

恒信合同平台的系统设计评审，开在架构基线冻结之后。

万辉交出了他的架构描述——十五张图的事已经过去了，这一次是真正对过账的一版：视角有规格、关切有台账、元素有清单。签字那天他还说了一句：“架构这一站，走完了。”

接手的是设计组。小白坐在投影前，讲合同签署链的系统设计描述：配置引擎怎么配、签署链怎么编排、扩展点怎么留。讲了二十分钟，玉杰先举手。

“我问一个运维的问题。”玉杰说，“你的设计里，**‘哪些失效必须被监控、多久检出’——这句话在哪儿？**”

小白翻到监控那一节：“我们设计了监控组件。”

“组件我看见了。”玉杰说，“我问的是判据：**漏了什么算漏，多久算迟。**”

黄程接着问：“业务方那句话——‘月底账目对得上’。你的设计里，**这句话落在哪个元素上？**”

小白指着事件总线：“编排层保证顺序。”

“保证到什么程度？”黄程问，“错了怎么发现？”

徐漾最后问的是万辉：“**你这版设计里，新增了一条‘与外部对账平台直连’。这条在架构描述的哪张视图上？**”

小白愣了一下：“这个……是设计细化出来的。”

“细化出来的，”徐漾重复了一遍，“还是**新长出来的**？”

万辉和小白对视了一眼。两个人都想到了同一句话，那句话在两家公司都有人说过，说得理直气壮：

**“架构不就是粗一点的设计吗？我们只是把它做细。”**

徐漾把笔放下：

“这句话我听过十年。它错在两处——**第一，架构和设计不是粗细之别，是两类工作；第二，做细的时候长出来的新东西，如果没人认领，它就不在任何人的账上。**”

---

## 这一章要回答什么

读完本章，你能回答：

1. 架构定义与设计定义，凭什么说是”两类工作”而不是”粗细两版”？
2. 同样一个 design，为什么标准里有两个意思？这个双义与”架构＝粗粒度的设计”这句误解是什么关系？
3. 有效的架构为什么要”尽可能不依赖于设计”？这条要求保护的是什么？
4. 从架构走到设计，第一步的入口动作是什么？为什么它不是”把架构发下去”？
5. 所谓”两条转化”，转的是哪两样东西？转不动的那一样，是设计错了还是架构错了？
6. 分配为什么必须对着”架构目标”而不只是对着系统需求？
7. 一个设计元素，怎么算”追得到出处”？追不到出处的，是什么病？

8. 接口为什么只能细化、不能新增？真需要新增时，正确的去处是哪里？
9. 设计描述符是什么？为什么”元素级描述符”必须与”系统级描述符”对得上？
10. 设计定义什么时候算做完？“停在〈待定〉”意味着什么？
11. 这个过程的出口为什么是”能拿去采购或实现”，而不是”评审通过”？
12. 架构与设计既然是上下游，为什么说”不是接力”？那三条回退通道各通向哪里？
13. 递归到哪一层可以停？“原子元素”的两个条件，为什么是信息充分性判据而不是深度判据？
14. “可实现性”有八项，为什么不必各配一个架构视角？新建一个视角的判据是什么？
15. 设计定义该由谁审、按什么动作审、能给出几档结论？

---

## 这一章的骨架

第 10 章说架构是什么，第 11 章说架构怎么被造出来，第 12 章说架构决策怎么定，第 15 章说架构定义怎么落地成一组能用的视图。四章合起来，把”架构”这一侧说完了。

**但架构不是自己活着的。**它必须被交给下一段工作——有人拿着它，去定出元素、接口、特性与取值，一直到能拿去采购或实现。这一段工作叫**设计定义**，这一章就讲它的交接。

骨架是**一场划界 + 两条转化 + 一道出口 + 一条回路**：

- **一场划界**——架构定义与设计定义是**两类工作**（作用面不同、目的句不同、独立性要求不同），但**不是两个阶段**（并发、迭代、递归）；
- **两条转化**——架构实体与关系 → 设计元素；架构特性 → 设计特性。两条都转不动的地方，就是交接处真正的病灶；
- **一道出口**——设计定义**到"能开发为规范、用于采购或实现"为止**；停在〈待定〉即未过出口；
- **一条回路**——设计评估回馈架构、衍生需求回流系统需求基线、迭代迫使架构更改：**它不是接力，是一条带着回路的递归。**

三幕走完，这一章要立住三句反直觉的话：

**设计定义不是架构的细化，是架构第一次被真正兑现的机会**——合规的承担者在实现侧，而实现能不能合规，取决于设计给出的规格够不够。（出处：ISO/IEC/IEEE 15288:2023 §3.13 `design, noun` 定义句；读法见 §16.2.2 术语注。）

**架构定的不是方案，是方案必须守的约定**——有效的架构尽可能不依赖于设计，为的是把设计的权衡空间留出来。（ISO/IEC/IEEE 15288:2023 §5.9）

**架构到设计不是接力，是递归里的一个换手点**——换手点按信息充分性判定：什么时候能自信地决定"买、造还是重用"，什么时候交出去。

# 16.2 第一幕 架构定义到设计定义是什么：两类工作，两条转化（定界）

> 这一幕先划界，再看交接——把两个过程的目的句并排读一次，把交接的动作、判据与回路摆出来。

## 16.2.1 两类工作，不是粗细两版

先看两句并排的目的。这两句都来自标准，措辞的差别比任何解释都有说服力。

> **架构定义（系统架构定义过程）**：目的是**生成系统架构的不同方案，选择一个或多个能够满足利益相关方关切和系统需求的方案，并以一致的视图和模型进行表达**。（ISO/IEC/IEEE 15288:2023 §6.4.4 目的句）

> **设计定义过程**：目的是**提供关于系统和其元素的足够详细的数据和信息，以便根据系统需求和架构实现解决方案**。（ISO/IEC/IEEE 15288:2023 §6.4.5.1 Purpose 句；原文：The purpose of the design definition process is to provide sufficient detailed data and information about the system and its elements to realise the solution in accordance with the system requirements and architecture.）

两句都在说”做方案”，可它们要的东西不一样：**一个要的是”生成、选择、表达”，另一个要的是”足够详细的数据和信息”**。

差别写在另一段话里，这段话是本章最该背下来的一句：

> The system architecture definition process focuses on defining an architecture that addresses the stakeholder concerns and is applied iteratively and concurrently with the business or mission analysis, stakeholder needs and requirements definition, and system requirements definition processes to determine the best solution to address stakeholder concerns. The design definition process, on the other hand, is driven by requirements that have been vetted through evaluation with the architecture and more detailed analyses of feasibility. Architecture focuses on suitability, viability, and desirability, whereas design focuses on compatibility with technologies and other design elements and on feasibility of construction and integration. An effective architecture is as design-agnostic as possible to allow for maximum flexibility in the design trade space. （系统架构定义过程侧重于定义一

> 种能够回应利益相关方关切的架构，并与业务或任务分析、利益相关方需要与需求定义、系统需求定义三个过程迭代地、并发地应用，以确定回应利益相关方关切的最佳解决方案。设计定义过程则不同，它由需求驱动——这些需求已经通过与架构的评估以及更详细的可行性分析得到审查。**架构管的是适宜性、存续性与合意性；设计管的是与技术及其他设计元素的兼容性，以及构建与集成的可行性。有效的架构尽可能不依赖于设计，以便在设计的权衡空间中实现最大的灵活性**。）（一手：ISO/IEC/IEEE 15288:2023 §5.9。）
> 同版同年旁证（中文对读，INCOSE SEH 5.0）：“系统架构定义过程侧重于适用于系统解决方案的**基本概念、特性结构、行为和特性**……**设计定义过程则侧重于制定整体系统设计，确保其细节充分以便于后续的实现**。”

四句话，四件事：**分工不同**（架构管适宜性、存续性与合意性；设计管与技术及其他设计元素的兼容性、构建与集成的可行性）；**驱动不同**（架构定义与业务或任务分析、需求定义迭代并发地应用，设计定义由已经过架构评估与可行性分析审查的需求驱动）；**产物不同**（一个产出架构描述，一个产出系统设计描述）；**还有一条质量要求——有效的架构尽可能不依赖于设计**。

**这一条要单独说**。它听起来别扭：架构不依赖设计，那它依赖什么？答案是：**它依赖目的、关切、需求与原则，唯独不依赖”将来会**

**选哪个设计”**。好的架构把约束立在该立的地方，把没定的事留给设计去权衡。**一个逼着设计只能有一种做法的架构，通常不是严谨，是越界**。

那么，为什么到处都有人把两者说成”粗细之别”？**根源在一个词的双义上**。

design 这个词，在标准里有两种用法：

| 口径 | design 指什么 | 出处 |
| --- | --- | --- |
| **宽义** | 定义**架构**、系统元素、接口及其他特性的**过程或结果** | ISO/TS 14812 §3.1.15.1（页面自标源 ISO/IEC/IEEE 24765:2017） |
| **窄义** | 架构定义**之外**的那一段工作（本章用此义） | ISO/IEC/IEEE 15288:2023 §6.4.5 |

**宽义的 design 定义句里，就明写着 define the architecture**。所以”架构是设计的一部分”这句话，在宽义下完全成立。问题出在下一步：有人从这个成立的宽义句，推出了”架构是设计的**粗粒度版本**”——**这一步是偷换，不是推论**。

**宽义说的是”设计这个词覆盖面很广”，不是”架构和设计是同一种工作的两档精度”**。引用时凡出现 design，都要先问一句：这是宽义还是窄义。

### 16.2.2 交接的两端：一边交出去什么，一边接住什么

划完界，看交接。

**交出去的一端是架构描述**。它交代三层东西，层次不能并成一行：**定义给的两样**——基本概念或属性、管控原则；**这两样的落点**——元

素与关系、接口（架构”可以关乎”的对象，属实体侧，其规格归下游的设计定义过程）；**表达它们的手段**——视角与视图。它的出口是”表达清楚”——供项目、组织与各利益相关方使用。

**接住的一端是系统设计描述**。它交代”做成什么、够细到能采购或实现”：元素规格、设计特性与设计描述符、分配、设计原理、可追溯性映射、设计评估报告。它在文档里有一句自我声明：**“我服从哪一份主导架构描述”**——因为它的第一条判据就是**与主导架构描述保持一致**。

两者摆在一张表里，差别一目了然：

| | 架构描述（AD） | 系统设计描述（DD） |
|---|---|---|
| 说什么 | **是什么、凭什么** | **做成什么、够细到能采购或实现** |
| 形态 | 视角 + 视图（每张视图声明治理它的视角） | 元素规格 + 设计特性与描述符（声明其主导架构描述并与之一致） |
| 表达手段 | 视角规格规定的**模型种类** | 方法 · 框架 · 图示形式 · 符号 · 其他模型形式 |
| 出口 | 供项目、组织与各利益相关方使用 | 开发为**规范**，用于**采购或实现** |

**一句话：AD 交代”凭什么”，DD 交代”做成什么”**。前者到”表达清楚”为止，后者要到”能拿去采购或实现”为止。

> 术语注（“合规实现”的出处与读法）：**“合规实现”一语的一手定义句在 ISO/IEC/IEEE 15288:2023 §3.13 design,**

> **noun**——specification of system elements (3.47) and their relationships, that is sufficiently complete to support a compliant implementation of the architecture (3.5)（系统元素及其关系的规定，其完整程度足以支撑对架构的一次合规实现）。三个要点：① 主语是**信息／规格**（一份规定），不是动作；② 关系词是 **support**，不是 be——设计**支撑**一次合规实现，不等于它就是那次实现；③ “合规实现”的承担者是 **implementation**——“把设计翻译为硬件部件、软件部件或两者的过程，或该过程的结果”（ISO/TS 14812 §3.1.15.2，术语学旁证）。本章枢纽那句”合规的承担者在实现侧，而实现能不能合规，取决于设计给出的规格够不够”，落的就是这条定义句。（一手：15288:2023 §3.13；过程侧一手：同标准 §6.4.5.1 Purpose 句；术语学旁证：ISO/IEC/IEEE 24765:2017 `design`、ISO/TS 14812 §3.1.15.2。）

### 16.2.3 交接的动作：入口、两条转化、一道出口

**入口动作，标准是明写的**，写在设计定义过程第一组活动里：

> **检查系统架构，以确定适用于系统设计的基本属性和概念，以及应指导设计及其演化的原则。**（中文逐字取 INCOSE SEH 5.0 中文版——**同版同年旁证**；该句是 SEH 对 15288:2023

> **§6.4.5.3 a) 准备设计定义**这一活动的转述，**非正本原句**。正本该活动的六项任务为：1) 定义设计定义策略／2) 确定为构成系统的每个系统元素所需的技术／3) 确定设计中需要表达的系统特性类别／4) 定义设计演化的原则／5) 识别并规划支持设计定义所需的使能系统或服务／6) 获取或取得这些使能系统或服务的使用权。）

**注意这句话的动词：检查**。不是”接收”、不是”分发”、更不是”照做”。从架构出发不是一句比喻——**它是设计方主动去架构里取三样东西：适用于本层设计的基本属性和概念、指导设计的原则、以及适用于本层的约束**。取不出来的地方，就是交接处第一个缺口。

取完之后是**两条转化**，这是整段工作的承重件：

**架构实体与关系　—转化→　设计元素（硬件、软件、结构、环境……）**

**架构特性　　—转化→　设计特性（功能·行为·尺寸·形状·材料·关键质量特性·数据处理结构）**

> 术语注（架构实体 ≠ 系统元素）：15288:2023 §5.9 逐字写着——Note that an architecture entity (an entity being architected or one subject to the system architecture definition process) and a system element (a discrete part of a system that fulfils specified requirements) represent two different notions.（注意，架构实体（被架构的实体，或受系统架构定义过程约束的实体）与系统元素（系统

> 的一个离散部分，用于满足规定要求）代表两个不同的概念。）上面那条转化的左端，**架构实体是”被架构的那个事物”**——42020:2019 §3.6 给它的定义是 thing being considered, described, discussed, studied or otherwise addressed during the architecting effort（在架构工作中被考虑、描述、讨论、研究或以其他方式处理的事物；15288 正文举的例：企业或项目目标、能力与效果、运行活动、资源功能），**不是”架构的构成物”，更不是系统元素**。一个是”被架构的”那一头，一个是”构成系统的”那一头——同一个系统之上两词各指一头，不可互换。所以”架构实体与关系 → 设计元素”这句里，转化的左端是**被架构的事物及其关系**，右端才是**构成系统的元素**。（一手：15288:2023 §5.9；42020:2019 §3.6。读法与第 10 章 §10.2.5 一致。）

两条转化之外，还有一刀**分配**：

> 将**系统需求分配给系统元素**，以确保满足所有系统需求和**架构目标**。（**正本 §6.4.5.3 b) 1) ＋注 4**；中译取 SEH 5.0）

**“和架构目标”这四个字不能丢**。只对着系统需求做分配，会分出一个”每条需求都有主”的设计，却可能整体偏离架构要守的那几件事；对着架构目标分，才会发现”目标某一条找不到承担者”这种病。

**出口判据在标准的另一句里**：

> **记录系统设计……称为系统设计描述……系统设计描述将开发为规范，以用于采购或实现组成设计的系统元素。（正本§6.4.5.3 b) 8) ＋注 12**，英文逐字 …in a form that can be used to either procure or realise the system elements that comprise the system；中译取 SEH 5.0）

**“开发为规范”是这道出口的名字。** 一份设计描述如果停在”这里待定”“取值后续确认”, 它就没过这道出口——**它不叫”做得还不够细”, 叫”还没做完”**。

### 16.2.4 怎么算做完、做对：五条判据

交接的质量，落在五条判据上。这五条都能当场检验：

| # | 判据 | 标准原话 |
| --- | --- | --- |
| 1 | **完备** | “提供关于系统和其元素的**足够详细的数据和信息**，以便根据系统需求和架构实现解决方案”；并”将开发为**规范**，以用于**采购或实现**” |
| 2 | **一致** | “设计应与**主导架构描述**保持一致”；并”与组织或项目采用的设计指南和标准保持一致” |
| 3 | **可追溯** | “维持系统设计的**双向可追溯性**”——“设计特性与**架构实体、接口和分析结果**之间的关系” |
| 4 | **描述符匹配** | “将系统元素及其特定的设计描述符与**整体系统的设计描述符相匹配**是系统设计的关键部分” |
| 5 | **接口只细化不新增** | 只对”**在系统架构中识别和定义的接口**”做”**细化或进一步定义**” |

第 2 条有一个现成的别名，叫**“一致”落到哪——它永远落在跨视图的对应关系上**。设计跟某一张视图冲突，是冲突；设计的两个部分各自跟不同的视图对上了，但两个部分之间对不上，也是不一致。

第 4 条最容易漏，也最容易被当成”细节问题”打发掉，所以下一节单独给它一个位置。

### 16.2.5 一条必须处理的缝：设计会不会”长出新东西”

第 5 条判据说”接口只细化不新增”。可现实里，设计阶段确实会长出架构里没有的东西。**这不是纪律松，是标准的另一句话必须一起读**：

> **系统架构识别系统元素，尽管架构描述可能仅识别那些在架构上具有重要意义的元素，但随着更详细设计考虑的出现，其他元素也会逐渐显现**。（中文逐字取 INCOSE SEH 5.0——同版同年旁证；**15288:2023 正本无此句**，正本**邻近句**＝§6.4.5.3 b) 6) 注 10：设计阶段**可以识别并定义架构定义过程未处理的额外接口**）

于是”设计不得引入架构没有的东西”这句话**过强**——它会把正常的显现当成违规。**正确的规则要配一条回流通道**：

> 当这些衍生功能影响目标系统时，**相应的衍生需求应当纳入系统需求基线**。（中文逐字取 INCOSE SEH 5.0——同版同年旁证；正本相关位＝15288:2023 §6.4.3.3 注 13（需求记录构成**系统需求基线**）、§6.4.5.1 注 2（设计定义回馈架构定义过程））

**三句话把这条缝说清楚**：

- 长出新元素，**是正常的**——架构只识别”在架构上有重要意义的”那些；
- 新长出来的东西**必须被认领**——认领的方式是进需求基线，不是就地留在设计里；
- **没被认领的新东西，就是本章章首那条”与外部对账平台直连”**——它不算违规，它算**无主**。

**“无主”比”违规”更危险**：违规有人拦，无主没人拦。

### 16.2.6 一条回路：不是接力，是递归里的一个换手点

如果架构与设计是接力，那么架构棒交出去，设计方跑完就算。**标准不是这么写的**。

> ……它们之间存在迭代，逐步演化的系统需求通过识别的约束和功能及质量要求来**塑造架构**。例如，由于详细的电气建模显示某个系统元素的负载超过了其分配的功率预算，这可能需要**更改系统架构，从而迫使设计变更**……**设计定义过程也可能会识别出需要重新考虑系统需求定义或系统架构定义过程中的决策和权衡**。

> **递归**指的是**在系统的不同层级上重复应用生命周期的过程**……在这个递归过程中，**上一层级的产出会作为下一层级的输入（在系统定义时往下，在系统实现时往上）**。

> 所有系统生命周期过程在系统层级的**每个层次上都是同时进行和迭代的**，并且在每个层级上这些过程都是**递归应用**的。

> **出处说明**：上列三句的**中文逐字取 INCOSE SEH 5.0**（同版同年旁证）；**这一组命题的正本规范位**＝15288:2023 **§3.31 `recursion`**（repeating the application of the same process or set of processes to successive levels of system elements in the system structure）与 **§5.8.2 Process iteration, recursion, and concurrency**（迭代／递归／并发的正本定义段，含 Iteration is not only appropriate but also expected. 与”迭代地、递归地使用生存周期过程”一段）。

三条并排读，会得到本章第三个反直觉命题：**架构定义与设计定义，不是同一物的粗细两版，但也不是两个阶段**。它们是**同时进行、互相迭代、逐层递归**的两类工作。

**回退通道有三条，一条比一条正式**：

| 通道 | 名义 | 标准原话 |
| --- | --- | --- |
| **迭代反馈** | 设计逼着架构改 | “可能需要**更改系统架构，从而迫使设计变更**”；设计也会”识别出需要重新考虑系统需求定义或系统架构定义过程中的决策和权衡” |
| **评估反馈** | 设计评估回馈架构过程 | 评估结论”可作为**对系统架构定义过程的反馈**” |
| **衍生需求回流** | 新长出来的东西进基线 | “相应的**衍生需求应当纳入系统需求基线**” |

**再往下就是递归，而递归有一个止点**。标准的判据是这样的：

> 如果一个系统元素**只需要用黑盒（也称为不透明箱）表示法来描述它的需求**，并**能够自信而明确地定义其在现实世界中的解决方案**，那么它可以被视为**原子元素**。在这种情况下，可以在不进一步详细说明元素的情况下，自信而明确地做出关于是**购买、制造还是重用**该元素的决策。（**INCOSE SEH 5.0——同版同年旁证；正本无此句**——black box／atomic／opaque box 在 15288:2023 零命中。正本**邻近句**＝

> **§5.2.2**：…the appropriate system life cycle processes are applied recursively to an SoI to resolve its structure to the point where understandable and manageable system elements can be implemented (made, bought, or reused).——生存周期过程**递归**应用于所关注系统，直到把结构分解到”可理解且可管理的系统元素可被实现（制造、购买或复用）“的程度。）

**读这条判据要抓住两个词：黑盒、自信**。它问的不是”分到第几层”，而是”**关于这个东西，我手上的信息够不够我决定买、造还是重用**”。

**所以它是信息充分性判据，不是深度判据**。标准里没有”分到第几层停”的规定。两条容易搞混的相邻规则要说清：**层宽启发式 7 ± 2 是可管理性规则**（分几块管得过来），**不是停止规则**；**横向集成**（深入研究之前先把这一层的完整性做够）与**纵向集成**（各层级之间保持一致）管的是另一件事。

**换手点一到，交出去的东西就从”设计”变成了”规格”**。交给谁？交给下一层——由它去决定这一层是自己造、外面买、还是拿现成的重用。**这就是递归里”上一层级的产出作为下一层级的输入”落到地上的样子**。

---

# 16.3 第二幕 架构定义到设计定义不是什么：四处最像的地方（磨边界弧）

> 这一幕把四样最容易被当成”交接”的东西剥下来：细化、下游阶段、接口自由、可采购性清单。

## 16.3.1 不是”细化”：细化只是形状，不是判据

回到章首那句话：“架构是粗一点的设计，我们只是把它做细。”

**“做细”确实是这个过程的形状**——设计的产物比架构详细，这一点没错。错在把形状当判据。**判据不是”细了没有”，是”转没转过去”**：

- 架构里的每一个**实体与关系**，在设计里有承担者吗？
- 架构里的每一条**特性**，在设计里有落点吗？
- 架构识别的每一条**接口**，被细化了吗（而不是被替换了）？
- 系统需求与**架构目标**都分下去了吗？

**一个”更详细”的设计，完全可以在这四问上全军覆没**。篇幅变长不解决任何问题——**细化是形状，转化才是判据**。

## 16.3.2 不是”下游阶段”：并发、迭代、递归

第二样最像的，是把它当成流程图上的一根箭头：架构做完 → 设计开始。

**这根箭头在标准里不成立**。三条逐字（§16.2.6）摆在那里：**同时进行、互相迭代、逐层递归**。

有一处的工程后果最值得记住：**详细的电气建模显示某个元素的负载超过了它分到的功率预算——于是架构得改，设计跟着改**。这条

回路如果在组织里不存在（“架构都冻结了，你怎么还改”），那么实际发生的就不是”不改”，而是**偷偷改**：改在设计里，账上不认。

**所以”并发、迭代、递归”不是给过程图加复杂度，它是在给回退通道要一个合法的位置。**

### 16.3.3 不是”接口自由”：只细化不新增

第三样最像的：设计阶段发现两个部件之间需要一条新接口，顺手加上。

**“顺手加上”就是无主。** 正确的处置是把两种情况分开：

| 情况 | 处置 |
| --- | --- |
| 架构**已识别**这条接口，设计把它写细（时序、语义、错误处理、字表） | 正常——这就是”细化或进一步定义” |
| 架构**未识别**这条接口，但两支元素确实要连 | **不是就地补上**：要么回架构补一次识别（架构确实漏了），要么把由此产生的衍生需求纳入系统需求基线（新需求出现了） |

**判据一句话：接口的”存在”归架构，“长相”归设计。**两者混在一起，就会出现章首那一幕——设计里多出一条线，问它从哪来，答”细化出来的”。

### 16.3.4 不是”需求清单的副本”：转化与分配是两件事

第四样最像的，是把设计定义当成”系统需求的详细版”：需求一条条抄下来，逐条写方案。

**抄需求会漏掉一半工作。** 设计定义要转的是**两样**东西：

- **分配**——系统需求 → 系统元素（“谁来实现它”）；

· **转化**——架构实体与关系 → 设计元素、架构特性 → 设计特性（“架构说的那件事，在这一层长成什么”）。

**只有分配的活儿，会做出一个”每条需求都有主”的设计，却答不上”架构要守的那几件事，各由谁守”。** 反过来，只有转化的活儿，会出现一批”结构对得上、需求没人认”的元素。

**两条都做完，才叫转过去了。**

---

# 16.4 第三幕 架构定义到设计定义往哪去：把交接走完（运用）

> 这一幕讲手艺：四组活动怎么走、描述符怎么定、判据怎么用、审核给几档结论、八项可实现性关切怎么归位。

## 16.4.1 过程四组活动：准备、开展、评估、管理

标准把设计定义的过程活动分成四组。逐组读一遍，会发现**难点都不在”设计”这两个字上**。

① **为设计定义做准备**。取设计驱动因素（功能与非功能需求、约束都可以是驱动因素）；确定设计中需要的**技术与系统特性的类别**（质量模型在此帮忙分类）；**检查系统架构**，取出适用于本层的基本属性和概念、以及指导设计与其演化的原则；定设计工作的实施方法；确保支持元素、资源与能力就位。

② **开展系统设计**。 识别和评估设计方案（改现有的、组合可用的、全新创造，或三者组合）；**把系统需求分配给系统元素**；必要时用

综合技术构思（头脑风暴、类比、形态学图表）；把**架构实体转化为设计元素、把架构特性转换为设计特性**；做**设计分析**（参数化设计、权衡分析）；**分析与细化接口**；**记录系统设计**。

③ **评估系统设计**。评估整体适用性与优劣，**判据是”设计应与主导架构描述保持一致”**；评估设计对各利益相关方的价值与可能的负面影响；把分析与评估综合成整体评估决策——它既是**选择首选设计**的基础，也可作为**对架构定义过程的反馈**，还能为验证过程提供信息。

④ **管理设计定义的结果。维持双向可追溯性**（对需求、限制、目标的满足情况，以及设计特性与架构实体、接口、分析结果之间的关系）；记录并管理方案选择与设计决策的**理由**；维护设计配置，与配置管理过程结合做变更管理；保持设计完整性（通常会指定一个专门职责，例如**系统设计权威**）；**进行设计评审**。

**四组里，①③④三组都在做同一件事：把”设计”这件事的账记清楚**。这也解释了为什么”设计阶段”最常出的问题不是想不到方案，而是**想了没记、记了没追、追了没反馈**。

### 16.4.2 五类可指名的方法

方法不必自创，标准里点得出名字的有五类：

| # | 方法 | 要点 |
| --- | --- | --- |
| 1 | **两条转化（核心）** | 架构实体／关系 → 设计元素；架构特性 → 设计特性 |
| 2 | **分配** | 系统需求 → 系统元素，须同时满足系统需求**和架构目标** |
| 3 | **方案产生** | 改现有／组合可用元素／全新创造／组合；辅以综合技术（头脑风暴、类比思维、形态学图表） |
| 4 | **设计分析** | 参数化设计、权衡分析；接口的分析与细化 |
| 5 | **设计描述符法** | 设计描述符＝**设计特性**＋**其可能的值**；由自上而下的**分配与配置**与自下而上的**选择与测量**相结合确定 |

同处还给了**设计方法的四个来源**：需求与约束；业务或任务分析确定的业务问题、机会与解决方案空间；**由架构定义过程所界定的系统架构**；设计策略（例如最大限度地重用）。

**注意第三个来源**。架构不是设计方法之外的”输入文档”，它本身就是方法的来源之一——**架构定下的基本概念或属性与管控原则，直接决定这层设计可以用什么手法**。

### 16.4.3 描述符：一条最容易被当成细节的判据

**设计描述符 = 一组设计特性 + 它们可能的值**。 它由两个方向合起来确定：自上而下的**分配与配置**，自下而上的**选择与测量**。

为什么它重要？因为”完备”这条判据最终**落在描述符上**：

> 将系统元素及其特定的设计描述符与**整体系统的设计描述符相匹配**是系统设计的关键部分。

**这句话说的是”对得上”，不是”填满了”**。元素级的描述符，必须能对回系统级那份描述符——**对不上的地方只有两种可能：一种是元素自己造了一个指标（无据），一种是系统级的指标没人接（缺项）**。

**这也解释了”停在〈待定〉“为什么是硬伤**。描述符的值没定，分配与配置那一半就是悬空的；自下而上的测量也就无从做起。〈**待定**〉**不是留白，是欠账**。

### 16.4.4 判据怎么用：九查与三档结论

设计定义是一件**工作产品**，对它的判定动作有**两个名字**——**评审与验证**（确认要等它在使用中兑现，见下）。先看**评审**：

> **评审**：为达成既定目标，对客体的**适宜性、充分性或有效性**所作的确定。（ISO 9000:2026 § 3.11.2）

**能叫验证吗？能；能叫确认吗？不能**。验证对”规定的要求”——设计定义是**人工制品**，15288:2023 §6.4.9 的验证对象明写含 artefact（如系统需求、架构描述、设计描述）；确认对”针对特定预期用途或应用的要求”，**且确认必须在使用中兑现**（使用条件可以真实、也可以模拟）——设计定义还没做出来，谈不上”用”。**所以：设计定义能评审、也能验证，做不了确认**。

评审怎么查？九查，一查一病：

| # | 查什么 | 不合格的形态 |
| --- | --- | --- |
| 1 | **实体转化** | 有架构实体没有承担者；**或引入架构未授权的元素与依赖** |
| 2 | **特性转化** | 属性无承担者；**或出现追不到架构特性的”自设指标”** |
| 3 | **分配** | 目标找不到承担者；或元素无任何需求／目标挂靠 |
| 4 | **接口** | **新增架构未识别的接口** |
| 5 | **一致** | 与某张视图冲突；或跨视图对应关系断裂 |
| 6 | **可追溯** | 追不到出处／追不到去向 |
| 7 | **完备** | **停在〈待定〉** |
| 8 | **准则守** | 违反管控原则且未走例外留痕 |
| 9 | **查滥** | 技术选型当概念／交付关切升格／把自由度当架构资产 |

**结论只有三档**：**合规**／**条件性合规**（附补项、责任方、期限）／**不合规**（退回设计定义）。

**三档里真正的工程价值在中间那档**。“条件性合规”把”这份设计行不行”这个是非题，变成了一张**带责任方和期限的欠账单**——它不拦进度，它把账记上。

### 16.4.5 可实现性关切：八项不必配八个视角

还剩最后一块，是设计方最常向架构方提意见的地方：**可实现性**。

产品研发里的可实现性常被列成八项：**物料可采购性、可制造性、可测试性、可交付性、可部署性、可操作性、可服务性、可退役性**。设计方会问：这八项是不是应该各有一个架构视角，各出一张”XX 性架构视图”？

**答案是不必，而且”各配一个”是错的。**

**判据只有一条：是否真需要一套独立的模型种类，且能否独立判定**。需要 → 才新建视角；只是同一套模型种类的内容侧重不同 → 那只是**同一视角下的另一张视图**。

按这条判据归位，八项的落点是这样的：

| 处置 | 关切 | 说明 |
| --- | --- | --- |
| **并入既有视角（4 项）** | 物料可采购性、可交付性、可部署性、可操作性 | 企业视角（政策、资金、工作安排、管辖结构）与功能视角已能受理 |
| **扩既有视角规格（2 项）** | 可制造性、可测试性 | 需要扩的是模型种类（工艺路线、测试点、诊断覆盖率），视角本身不变 |
| **新建视角候选（2 项）** | 可服务性、可退役性 | 需要独立模型种类（维修任务与可达性、拆解顺序与回收路径）；先扩规格、代价不够再新建 |

**为什么”各配一个视角”是错的？**因为**切视角的维度是”看什么”（关注面），不是”谁在看”（利益相关方名单）**。

> 视角的实质是”约定 + 模型种类”：它 `aggregates some modelKind`、`frames some concern`、`governs some architectureView`。

八类利益相关方的关切在四个视角上**重叠分布**——比如可服务性同时落在功能（维修过程）、物理（可达性）、企业（服务组织与合同）三处。**但八类利益相关方都必须有受理口**，这是硬要求：

> validate that architectures **address the concerns of stakeholders**（ISO/IEC/IEEE 42030:2019 官方摘要 a 条）

**“必须有人受理”与“必须各建一个视角”是两件事**。前者是义务，后者是手段——**手段要论证，义务不用**。

还有一条口径要记住：**“XX 性视图”可以先有，“XX 性视角”须经论证**。一张图挂在什么视角下，是描述的选择；一个视角成不成立，看它有没有自己的模型种类与规格。

### 16.4.6 一条纪律：先穷尽复用，再自建

新建视角不是免费的——它意味着额外的描述、评估、管理与使用工作量。

**所以顺序是：① 并入既有视角 → ② 扩既有视角规格 → ③ 新建视角 → ④ 改用别的标准手段**（例如采用某种架构类型、或扩项目 ADF）。**四步的代价依次升高，能用低代价解决的不上高代价**。

**这条纪律同样适用于视角本身**：复用一份成熟的 ADF，永远比自建一套更省——**自建要有理由，理由要写下来**。

---

## 案例进展①｜恒信：交出去的那一版，被查了九条

架构基线冻结后第二周，恒信做了一次设计评审。徐漾没有参加讨论，只在最后拿走了一份东西——**九查表**。

查完，他在表上留了三行：

**第一行，查 1（实体转化）**：设计里多了一条”与外部对账平台直连”。**处置：不是删掉，是回架构补一次识别，或把衍生需求纳入需**

**求基线**。万辉在会上说了一句实话：“这条我第一反应是违规，其实不是——它是**我的架构没识别**，我该谢谢它。”

**第二行，查 3（分配）**：架构目标里有一条”配置出来的孤岛必须可检出”。设计里的配置引擎只管生成配置，**检出这件事在谁身上，表上是空的**。

**第三行，查 7（完备）**：扩展点的”兼容性承诺表”里，“什么算破坏性变更”那一栏写着〈待定〉。

玉杰在旁边看完，说了句让小白记了很久的话：

“**判据没定，不是没做完，是没法判**。你这一栏空着，将来谁都可以说自己没破坏兼容。”

评审结论栏填的是**条件性合规**——三项补项，各带责任方与期限。小白拿着那张表回工位，第一次觉得”条件性合规”这四个字不算难听：**它不拦我，它把账记上了**。

## 案例进展②｜冰链：水忠的老规矩，第一次被别人拿去用

冰链这边发生了一件小事。

M3 二代的接口视角规格写完之后，水忠把它挂在了项目 ADF 里——就是李旭那句话的作用：“**你这份规矩，别人拿着它，能不能画出同一张图、看出同一个错？**”

这一版开始，设计组真的去用了。董勐拿着那份规格，做数据传输链的设计定义。做到第三天，他来找水忠：

“你这条’降级是能力分级的连续谱’——**我这层的降级状态该分几级？**”

水忠反问：“你觉得该分几级?”

“我想分五级。”

“**五级是你定的，还是我这条规矩定的？**”

董勐愣了一下，翻那份规格。规格里写的是**连续性**——降级不许出现”跳变”——但**没有规定级数**。

“那就是我定的。”董勐说。

“对。”水忠说，“**它管的是形状，不管你的取值**。我要是连级数都替你定了，你这层的权衡空间就没了。”

那天下午，董勐在自己的设计描述里补了两栏：**“本层取值：五级”**，以及**“依据：接口视角规格第 4 条（连续性要求）”**。

**一栏是设计元素的设计特性，另一栏是可追溯性映射**。文正后来看到这两栏，说了一句话：“**架构没管你的取值，但你的取值有了出处**。”

## 案例进展③｜两个世界，同一条缝

把两条线并起来看一次。这一章讲的交接，两个世界都走通了，但**卡住的地方不一样**：

| 面向 | 冰链 M3（设备类） | 恒信合同平台（平台类） |
| --- | --- | --- |
| 交出去的一端 | 四视角 + 自建五条（安全／环境／制造／保障／人因） | 四视角 + 自建六条（数据治理／配置与版本／安全与合规／AI 治理／运维／集成与扩展） |
| 交接的第一个动作 | 检查架构，取”独立性”与”降级连续性” | 检查架构，取”数据归属唯一”与”可配置优先” |
| 最容易断的一步 | 接口：**通道划分** ↔ **总线分区** ↔ **失效状态** 三处对不上 | 分配：**权属** ↔ **注册** ↔ **配置** 三处对不上 |
| 设计里的”自设指标”风险 | 器件型号进架构元素位（技术选型当概念） | **把”配置自由度”当架构资产**（可配项追不到承重关切） |
| 出口卡在哪 | 描述符取值停在〈待定〉（环境适应性） | 描述符取值停在〈待定〉（配置覆盖度） |
| 缺口的性质 | 缺**不变量清单**（字义冻结与追加规则） | 缺**判据本身**（什么算破坏性变更、哪些环节必须声明 AI 参与） |
| 换手点在哪一层 | 收发机组件可以买了——**黑盒可定义** | 配置引擎可以外购——**黑盒可定义** |

**一句话对照：M3 的交接卡在”接口对不上”，恒信的交接卡在”判据没立”。**

合起来是一条结论：**同一条交接链，设备类产品最先断在跨视图对应关系上，平台类产品最先断在判据的缺失上。** 前者要靠对账表，后者要靠补判据——**两者的补法不一样，但都回同一个地方：架构定义期。**

---

## 认知反转

回到章首那间评审室。小白被问了三个问题，万辉被问了一个。

如果你读完了前面十五章，你会替他们说一句：设计确实比架构详细，说”做细了”不算错吧?

这句话站不住，因为它把四件事混成了一件。

**第一层，把”粗细”当成了”种类”。**架构与设计不是同一支笔的两档笔尖——**目的句不同、作用面不同，还有一条独立性的要求**：有效的架构尽可能不依赖于设计。**这一条要求的存在，本身就证明了它们不是同一件事的两档精度**——同一件事不会要求”尽量不依赖自己”。

**第二层，把”接收”当成了”交接”。**真正的第一步是**检查**架构：取出适用的基本属性与概念、取出指导设计的原则。**万辉交出去的是文档，设计方该拿的是约束。**交文档不算交接，取到约束才算。

**第三层，把”长出来的”当成了”细化出来的”。**设计会长出新元素，这很正常；不正常的是**没人认领**。章首那条”与外部对账平台直连”，处置起来一点都不难——回架构补一次识别，或者把衍生需求纳入需求基线。**难的是先认出它无主。**

**第四层，也是最深的一层，以为交接是一次性的**。标准写得很清楚：**并发、迭代、递归**。电气建模一算，负载超了预算，架构就得改。**这条回路如果组织里不认，实际发生的不是"不改"，是偷偷改**——改在设计里，账上不认，等到集成那天一起结账。

这一章真正要立的，是那三句最短的话：**设计定义不是架构的细化，是架构第一次被真正兑现的机会；架构定的不是方案，是方案必须守的约定；架构到设计不是接力，是递归里的一个换手点**。

用第 1 章的话收一次束：对"架构与设计的交接"，你原来只是"见过"——见过设计文档、见过接口表、见过"架构冻结"四个字。这一章把它推到"拥有"——能说出两类工作的分界在哪、入口动作是什么、两条转化各转什么、五条判据怎么查、三条回退通道通向哪里、换手点凭什么判定。**能答全这四问才算拥有**。

再用第 1 章 §1.4.4 那四层量一次：这一章站在**第 ③ 层（心智模型）**——它给你的是一张交接图：拿到任何一份系统设计描述，你都能顺着这张图走一遍，指出它在哪一步转丢了。

交接走完，这条链上的每一站就都到齐了。下一章把它们放回同一张时间线上。

## 小结

这一章讲的是架构交出去的那一段路：从一份架构描述，到一份能拿去采购或实现的系统设计描述。

它先划了界：架构定义与设计定义是**两类工作、两个过程**——目的句不同（生成·选择·表达 vs 足够详细的数据和信息）、作用面不同（架构管**适宜性、存续性与合意性**——实体在其环境中的基本概念或属性

与管控原则；设计管**与技术及其他设计元素的兼容性、构建与集成的可行性**——要足够详细的数据和信息），并且有一条独立性的要求：**有效的架构尽可能不依赖于设计**。但它同时说明：两者**不是两个阶段**，而是并发、迭代、递归。

它立了交接的动作：入口是**检查架构**，取适用的基本属性与概念、以及指导设计及其演化的原则；中间是**两条转化**（实体与关系→设计元素、特性→设计特性）与**分配**（需求→元素，须满足架构目标）；出口是**开发为规范，用于采购或实现**——停在〈待定〉即未过出口。

它给了五条判据（完备、一致、可追溯、描述符匹配、接口只细化不新增）、一条必须处理的缝（长出来的新东西要有人认领）、三条回退通道，以及一个按**信息充分性**判定的换手点——原子元素的两条件一到，就去做买／造／重用的决策。

它把审核的办法落成九查与三档结论，其中真正的工程价值在”条件性合规”这一档：**它不拦进度，它把账记上**。

最后，它给八项可实现性关切归了位：**受理口是义务，视角是手段**——四项并入既有视角、两项扩规格、两项作新建候选；**“XX 性视图”可以先有，“XX 性视角”须经论证**。

这一章的三句话：**设计定义不是架构的细化，是架构第一次被真正兑现的机会；架构定的不是方案，是方案必须守的约定；架构到设计不是接力，是递归里的一个换手点**。

这条链上的四个站点到齐了。下一章，把它们放回同一张时间线上。

# 本章概念总图

架构定义到设计定义 = 一场划界（两类工作） + 两条转化 + 一道出口 + 一条回路

第一幕 · 是什么：两类工作，两条转化

划界　　架构定义（生成·选择·表达） vs 设计定义（足够详细的数据和信息）

一条质量要求：有效的架构尽可能不依赖于设计（为设计的权衡空间留位；15288:2023 §5.9）

design 双义：宽义含 define the architecture；窄义＝架构之外的这一段

两端　　交出去＝架构描述（凭什么）；接住＝系统设计描述（做成什么）

入口动作　　检查系统架构，取出适用于设计的基本属性和概念、指导设计与其演化的原则（"检查"，不是"接收"）

两条转化　　架构实体与关系 → 设计元素；架构特性 → 设计特性

分配　　系统需求 → 系统元素，须满足系统需求**和架构目标**

出口　　开发为规范，用于采购或实现；停在〈待定〉即未过出口

五条判据　　完备 · 一致（对主导架构描述） · 可追溯（双向） · 描述符匹配 · 接口只细化不新增

一条缝　　设计会长出新元素（正常）→ 认领方式 = 衍生需求纳入系统需求基线；无主比违规更危险

一条回路　　三条回退通道（迭代反馈／评估反馈／衍生需求回流）

递归　　并发·迭代·递归；换手点 = 原子元素两条件（黑盒可定义 + 自信定解）→ 买／造／重用决策

判据性质＝信息充分性，不是深度；7±2 是可管理性规则，不是停止规则

第二幕 · 不是什么：四处最像的地方

不是"细化" 细化是形状，判据是转化——篇幅变长不解决任何问题

不是"下游阶段" 并发·迭代·递归；回路不合法，改动就会转入地下

不是"接口自由" 存在的归架构、长相的归设计；新增即无主

不是"需求副本" 分配（谁实现）与转化（长成什么）是两件事，缺一条都转不过去

第三幕 · 往哪去：把交接走完

四组活动 准备（检查架构·取驱动因素）／开展（转化·分配·分析·记录）／评估（对主导架构描述）／管理（追溯·理由·配置·评审）

五类方法 两条转化／分配／方案产生／设计分析／设计描述符法

设计方法的四个来源：需求与约束／业务问题与机会／**系统架构**／设计策略

描述符 设计特性 × 其可能的值；自上而下分配配置 ＋ 自下而上选择测量

完备最终落在描述符上；〈待定〉不是留白，是欠账

九查三档 九查逐项对病；结论＝合规／条件性合规（带责任方与期限）／不合规

动作名 = 评审与验证（ISO 9000 §3.11.2／15288 §6.4.9）；设计定义做不了确认

可实现性 八项不必配八视角；判据＝是否真需要独立模型种类且能独立判定

并入 4（采购·交付·部署·操作）／扩规格 2（制造·测试）／新建候选 2（服务·退役）

切视角的维度是"看什么"，不是"谁在看"；受理口是义务，视角是手段

三句元规则

设计定义不是架构的细化，是架构第一次被真正兑现的机会。

架构定的不是方案，是方案必须守的约定。

架构到设计不是接力，是递归里的一个换手点。

## 本章判据

> 本章的判据是本书的主张，不是任何标准的条文——标准出处（15288:2023 §5.9／§6.4.4／§6.4.5·§6.4.5.1／§3.13、14812 §3.1.15.x、42030、ISO 9000 §3.11.x）见正文，INCOSE SEH 5.0 为同版同年旁证；凡属形式化（三类缺口、九查三档、可实现性归位四步、换手点两条件）已在文中标明。它们都当场可检验：欢迎检验、欢迎校准。

1. **两类工作**：架构定义与设计定义的分界不在粗细，在**作用面与目的句**——一个要”生成、选择、表达”，一个要”足够详细的数据和信息”。**违反即崩：把架构当粗设计，于是设计的”细化”永远停在形状上，转化一件没做**。

2. **design 标口径**：凡引用 design，先标明宽义还是窄义——宽义含 define the architecture，窄义指架构之外那一段。**违反即崩：拿宽义句推出”架构是粗粒度设计”，一步偷换，后面全错**。
3. **架构不依赖设计**：有效的架构尽可能不依赖于设计（ISO/IEC/IEEE 15288:2023 §5.9）。**违反即崩：架构把设计的权衡空间预先钉死，设计方只剩照做，产品失去演化余地**。
4. **入口是检查**：“检查系统架构，取出适用于设计的基本属性和概念、以及指导设计及其演化的原则”——不是接收文档、不是照做。**违反即崩：把架构当输入文档收下就开工，约束一条没取到，设计自由生长**。
5. **两条转化缺一不可**：架构实体与关系 → 设计元素；架构特性 → 设计特性。**违反即崩：只做分配（谁实现），做出一个”每条需求都有主”却没人对应架构目标的设计**。
6. **分配对着两个量**：系统需求 → 系统元素，须同时满足系统需求**和架构目标**。**违反即崩：架构目标找不到承担者，而分配表看起来是满的**。
7. **一致落在跨视图对应关系上**：“设计应与主导架构描述保持一致”——一致不是一个整体印象，是一条条对账。**违反即崩：两个部分各自跟一张视图对上了，彼此之间对不上**。

8. **接口只细化不新增**：接口的”存在”归架构，“长相”归设计。**违反即崩：设计里多出一条线，问它从哪来，答”细化出来的”**。
9. **新增必有认领**：设计会长出新元素，这是正常的；不正常的是没人认领——衍生需求应纳入系统需求基线或回架构补识别。**违反即崩：无主的新东西比违规的更危险——违规有人拦，无主没人拦**。
10. **描述符匹配**：元素级设计描述符必须能对回整体系统的设计描述符。**违反即崩：对不上的地方只有两种——自设指标（无据）或指标无人接（缺项）**。
11. **出口是规范**：“开发为规范，用于采购或实现”；停在〈待定〉即未过出口。**违反即崩：把〈待定〉当留白，欠账记成进度**。
12. 三**条回退通道合法**：迭代反馈、评估反馈、衍生需求回流——回路必须在组织里合法。**违反即崩：回路不合法，改动转入地下，集成那天一起结账**。
13. **换手点是信息充分性判据**：原子元素两条件——**只需黑盒表示法描述需求、能自信而明确地定义其现实世界中的解决方案**；达标即可做买／造／重用决策。**违反即崩：拿”分到第几层”当停止规则，切到没人能对结果负责的深度；或该买不买、该重用不重用，一路自制到底**。

14. **九查对九病**：实体转化·特性转化·分配·接口·一致·可追溯·完备·准则守·查滥。**违反即崩：只查”全不全”，不查”有没有多”，滥项全部漏网**。

15. **审核只有三档**：合规／条件性合规（附补项、责任方、期限）／不合规；动作名是**评审**或**验证**，不是**确认**（§16.3.2 已对账）。**违反即崩：把”确认”拉进来当判据——拿”在真实用途中表现如何”当设计定义的合格证明；设计定义还没做出来，谈不上”用”**。

**工作检查单**（每次交接设计定义前，逐条自问）：

- □ 这一次是”接收文档”还是”检查架构取约束”?
- □ 架构的每条实体与关系，在设计里有承担者吗?有没有承担者拿不出出处?
- □ 架构的每条特性，转成设计特性了吗？有没有”自设指标”?
- □ 系统需求全部分配了吗？**架构目标**也逐条有承担者吗?
- □ 架构已识别的接口，是被细化了，还是被替换了?有没有新增的接口?
- □ 新增的元素与依赖，认领了吗（回架构补识别／进需求基线)?
- □ 跨视图的对应关系逐条对过账吗?“一致”是怎么判的?
- □ 元素级的设计描述符与系统级描述符对得上吗?〈待定〉还剩几处、谁负责补齐?
- □ 这次动作叫什么？是评审，还是被当成了验证／确认?
- □ 结论给了几档?“条件性合规”的补项写清责任方与期限了吗?

· □ 实现方那边提出的”XX 性”关切，是要求新建视角，还是并进既有视角／扩规格就够了？

## 自查 · 这些说法对吗？

1. “架构就是粗一点的设计，设计就是细一点的架构。” —— **错**。两类工作的目的句、作用面与独立性要求都不同；design 的宽义句不能推出”粗细两版”。
2. “有效的架构应该把设计的做法尽量定死，免得设计走偏。” —— **错**。标准反过来写：**有效的架构尽可能不依赖于设计**（ISO/IEC/IEEE 15288:2023 §5.9），为的是把设计的权衡空间留出来。
3. “架构做完了才轮到设计开始，这是阶段划分。” —— **错**。两者是并发、迭代、递归；不是接力，是一条带三条回路的链。
4. “架构描述发给设计方，交接就算完成了。” —— **错**。第一步是**检查**架构，取出适用的基本属性与概念、指导设计的原则。交文档不算交接，取到约束才算。
5. “设计就是把系统需求一条条实现掉。” —— **错**。设计要转两样：分配（需求→元素）与转化（架构实体→设计元素、架构特性→设计特性）。
6. “分配只要对上系统需求就行，架构目标是架构自己的事。” —— **错**。标准原话是满足所有系统需求**和架构目标**——少了后半句，目标就会找不到承担者而无人发觉。

7. “设计里出现架构没提过的新元素，说明设计违规了。” —— **错**。架构只识别在架构上有重要意义的元素，其他元素会随设计深入逐渐显现；要管的不是它出现，是**它有没有被认领**。
8. “多出来的那条接口就地补上最快，反正功能要跑通。” —— **错**。接口的存在归架构、长相归设计；就地补上就是无主——回架构补识别，或把衍生需求纳入基线。
9. “描述符取值先写〈待定〉，等内容定了再补。” —— **错**。完备这条判据最终落在描述符上；〈待定〉不是留白，是欠账——“判据没定，不是没做完，是没法判”。
10. “设计做完了，让它通过验收测试就算合格了。” —— **错**。设计定义是工作产品，只能**评审**；确认要落在实现出来的客体上，且须在使用中兑现。
11. “设计只要不与架构描述冲突，就算一致。” —— **错**。一致永远落在**跨视图的对应关系**上：与某张视图冲突是不一致，各部分彼此之间对不上也是不一致。
12. “设计评估是设计组自己的事，不必回馈架构。” —— **错**。评估结论”可作为对系统架构定义过程的反馈”——这是标准写明的三条回退通道之一。

13. “架构要不要改，等设计做完再说。”—— **错**。详细的电气建模一算负载超预算，架构当场就得改；回路不合法，改动就会转入地下。
14. “分到能被采购的层级就可以停，这是个经验数。”—— **错**。换手点按**信息充分性**判定：只需黑盒描述需求、且能自信而明确地定义现实世界中的解决方案——此时才做得出买／造／重用的决策。
15. “分层的宽度按 7 ± 2 控制，所以分到七层就该停。”—— **错**。7 ± 2 是可管理性规则，不是停止规则；标准没有”分到第几层停”的规定。
16. “八项可实现性关切，每一项都该配一个架构视角。”—— **错**。切视角的维度是”看什么”（关注面），不是”谁在看”（利益相关方名单）；判据只有一条——是否真需要一套独立模型种类且能独立判定。
17. “既然不必各配视角，那这些关切就不用管了。”—— **错**。**受理口是义务，视角是手段**——42030 摘要 a 条要求架构必须回应利益相关方的关切。
18. “新增一个视角没什么成本，多一个更保险。”—— **错**。自建视角意味着额外的描述、评估、管理与使用工作量；顺序是先并入、再扩规格、再新建、最后才改标准手段。

## 练习

1. **给一次交接做体检**：拿你们最近一份系统设计描述，对着入口动作问一句——你们是”检查架构取约束”，还是”收到文档就开工”？取到的那几条约束，写在哪一栏？
2. **走一遍两条转化**：挑架构描述里的一个元素与一条特性，各自往下追一层：它转成哪个设计元素、哪条设计特性？转不动的那一处，是架构没立住，还是设计没转？
3. **分配对着两个量查一次**：把系统需求与架构目标分两列，逐条找承担者。哪一条找不到？找不到的原因写下来。
4. **找一条无主的接口**：在你们的设计里找一条架构没识别过的接口或依赖，按两种情况给出处置（回架构补识别／进需求基线），并说明选哪条、为什么。
5. **描述符对账**：挑一个元素，把它的设计描述符与系统级描述符对齐一次。哪几格对不上？哪几格还停在〈待定〉？谁负责补齐？
6. **做一次九查**：拿九查表把一份设计过一遍，重点查第 1、第 9 两项（有没有加戏、有没有滥项）——然后给一个三档结论，条件性合规的补项写清责任方与期限。
7. **给一项可实现性关切找家**：挑你们团队最常抱怨的一项（可制造性、可测试性、可服务性皆可），按四步判定它的落点：并入既有视角、扩规格、新建视角，还是改用别的标准手段？

8. **验证一下换手点**：挑一个已经在采购或重用的元素，用黑盒两条件检查它——当初决定”买”的时候，是不是真的只需要黑盒表示法、且能自信地定义它的解决方案？

## 参考文献

**标准** - R-04 ISO/IEC/IEEE 42010:2022 —— 架构定义（基本概念或属性＋管控原则）；架构视角与架构视图；架构描述框架 ADF；本章术语定名依 GB/T 45630-2025（等同采用） - R-05 ISO/IEC/IEEE 42020:2019 —— 架构过程**六个**：架构治理（§6）／架构管理（§7）／架构概念化（§8）／架构评估（§9）／架构细化（§10）／架构使能（§11）；分组＝**§5.3 架构治理与架构管理**（同组）／**§5.4 架构概念化·评估·细化**（三个核心过程）／**§5.5 架构使能**；架构治理决策（§6.4.4）、确立架构目标与关键成功准则（§8.4.4）；评估架构细化（§10.4.6）；架构实体 §3.6＝被架构的那个事物（见 §16.2.3 术语注） - R-85 ISO/IEC/IEEE 42030:2019 —— 架构评估框架：评估用途 a 条 validate that architectures address the concerns of stakeholders；以预期目的为评估基准 - R-88 ISO/TS 14812:2022 —— 实物产品领域四条明文视角（功能／物理／通信／企业） - R-104 ISO/IEC/IEEE 24765:2017 —— **术语学旁证**（“合规实现”一语的一手出处已改挂 R-102／15288:2023 §3.13）；保留理由只剩**宽义 design 的术语学来源**——其 design 定义句含 define

the architecture；其”合规实现”句系经 ISO/TS 14812 §3.1.15.1 `design` 注 1 转引（页面自标源即本标准），**非一手** - R-105 ISO/TS 14812:2025（术语批次，**术语学旁证**）—— §3.1.15.2 `implementation`（把设计翻译为硬件部件、软件部件或两者）、§3.1.15.4 `realization`（系统概念的实现）；链条 concept → design → implementation → realization；“合规实现”的承担者依本条读作 implementation（一手见 R-102 §3.13） - R-102 ISO/IEC/IEEE 15288:2023 —— **架构定义过程 §6.4.4**（目的句：生成不同方案、选择一个或多个、以一致的视图和模型表达）、**设计定义过程 §6.4.5**（目的句＝§6.4.5.1 Purpose：提供关于系统及其元素的足够详细的数据和信息，以按系统需求和架构实现解）、**§5.9**（as design-agnostic as possible 的规范位＝“有效的架构尽可能不依赖于设计”；two different notions 辨正句＝架构实体与系统元素代表两个不同的概念）、**§3.13 `design, noun`**（sufficiently complete to support a compliant implementation of the architecture＝“合规实现”的一手出处）。**确属正本、已直核**：设计定义四组活动（§6.4.5.3 a)~d)）、**两条转化**（b) 2)／b) 3) 逐字 Transform architectural entities and relationships into design elements.／Transform architectural characteristics into design characteristics.）、分配（b) 1)＋注 4)、设计演化原则（a) 4)）、出口（b) 8)＋注 12)、双向追溯（d) 4)＋注

19)、接口细化 (b) 6)＋注 10)、递归（§3.31)。**属 SEH 5.0 专有、正本零命中（故不标标准条款）**：设计描述符、原子元素两条件（black box／atomic／opaque box)、横向与纵向集成、入口动作一句（系§6.4.5.3 a) 该活动的转述、非正本原句)、“衍生需求纳入系统需求基线”一句（正本相关位＝§6.4.3.3 注 13／§6.4.5.1 注 2)。SEH 5.0 为后一组表述的同版同年中文对读旁证 - R-103 ISO 9000:2026 —— 评审（§3.11.2)、验证（§3.11.12)、确认（§3.11.14)、质量与质量特性（§3.5.2／§3.10.2）

**经典文献** - R-45 INCOSE 系统工程手册（SEH 5.0 中文版，2023-07，与 15288:2023 **同版同年**）—— **同版同年旁证 · 中文对读本**：①**其转述对象在正本**——架构定义与设计定义的分立说明＝15288:2023 §5.9 逐字、设计定义过程的四组活动与判据＝§6.4.5.3（见 R-102)；②**其专有表述在正本零命中——设计描述符、原子元素两条件**（black box／atomic／opaque box)、**横向集成与纵向集成、“相应的衍生需求应当纳入系统需求基线”、入口动作一句**：上列术语与句式的规范位**不在** ISO/IEC/IEEE 15288:2023，引用时须标 SEH 5.0；③SEH 是②类表述的同版同年**中文对读旁证**，也是「系统设计描述（系统设计说明)」「设计描述符」「原子元素」等术语的中文来源 - R-70 SEBoK —— 架构描述须说明”何为纳入、何为排除”；详细程度取决于使用语境与设计目的

本章回扣链：第 10 章（架构·架构描述）、第 11 章（架构设计·构思定义设计）、第 12 章（架构决策·不可逆性）、第 15 章（架构定义落地·视角与视图）、第 6 章（设计开发落地·需求与方案结对）；与第 17 章的分工见 §16.4.1（整合是”把这条链放回同一张时间线”）；与第 18 章的分工见 §16.2.1（架构师是”把这条链背在身上的人”）；附录 A T-164~T-167、附录 C 第 16 章行、附录 E 第 16 章自查速查。

# 第 17 章 整合：从一条需求到上线运维

> 本章主题：所有概念是同一系统的不同切片——顺次切片 + 横切切片，切片靠接缝缝合 主案例：双线合流（恒信·电子签章 × 冰链·断链补采）对应章节：收束第 2~16 章（全链贯通）；承第 15 章（架构定义落地）、第 16 章（架构定义到设计定义）；下接第 18 章（架构师）术语口径：本章「利益相关方／管控原则／生存周期／架构视角／架构视图」按 GB/T 45630-2025（等同采用 ISO/IEC/IEEE 42010:2022）定名——第 10、11、12、15、16 章已同步迁移，本章随之铺齐（除本行外：利益相关方 9 处；管控原则 3 处〔站点④作业链、概念总图、站点④切片回顾〕；生存周期 1 处〔R-09 描述行〕；架构视角与架构视图各 1 处〔随第 15 章九步链〕）；对照表见附录 A。

---

## 17.1 误解现场

恒信集团，电子签章上线两周后，黄程先找的是运维玉杰（系统工程师团队·运行侧）。

“编号对不上，是不是部署出问题了？”黄程问。

玉杰查了一遍：部署记录、版本基线、配置项，全对得上。“不是部署的事。系统这两周跑得挺稳。”

“那问题出在哪儿?”

玉杰想了想：“出在’编号从哪来’。上线前我盯的是系统活着没活着——谁部署、什么版本、监控正不正常；没人盯’签章编号怎么落进台账’这条链路。”

黄程点点头，把徐漾和万辉叫到会议室，摊开两张表：“同一天签的电子合同，两头对不上。月底对账就两天了，这两张表对不上，财务那边过不了关。”

万辉皱眉：“编号标准不是上季度刚定下来吗? 法务归口，全集团统一。规则没变啊。”

“规则没变，”黄程说，“**来源**变了。以前编号是台账登记的时候生成的；现在电子签章由第三方 CA 服务商在’签章完成’那一刻直接带回来一个编号，写进了业务库，绕过了台账。规则还是那条规则，可是生成编号的那台机器，换了主人。”

徐漾回去翻了一整天文档。需求文档里没有”签章编号要不要进台账”这一条；设计树里，“CA 对接”和”台账登记”是两片叶子，中间没有第三片叶子；测试用例里没有”签章→台账”的集成场景；基线的变更记录里，写的是”新增 CA 对接模块”，没有”编号来源变更”。它哪儿都不在。

他合上电脑，说了这章最重的一句话：

“我们不是哪一步做错了。我们把每一步都做得太对了——需求评审、设计评审、验证、确认、基线、变更控制、治理委员会，全都有。可我们把这些环节用成了一个个**独立的箱子**：需求是需求的，设计是设计的，测试是测试的。”

“系统不认这套分工，它是一条从需求到运维的连续时间线——**我们的编号来源，正好掉在两条切片中间的那道缝里**。”

同一天，三千公里外，冰链科技的实验室里，水忠和文正对着同一道缝发愣。

M3 刚推完固件 v2.5——断链自动补采。固件测过了，云端测过了，App 测过了，验收过了。疾控的审计员复查批次追溯，问了一句："这批补采的记录，采集时间戳是哪来的？"水忠答设备本地时钟，云端工程师答服务器时间归档——**一个箱子，两个时间**。管批次台账的逸人翻开记录，补了一句："批次上全是 v2.5 采的没错，可记录里的时间，两种都有。"漂移了几个小时，断链判定链路还是查不清。

"固件没问题，"水忠说。

"云端也没问题，"文正接上，"**两个都没问题，中间那条缝有问题**。"

十一章下来（第 3~14 章的十一个概念章），两个团队已经治好了各自的词汇病：等级不再是质量，版本不再是基线，治理不再是管理。他们以为学到的是一把一把的刀，磨得锃亮，摆满了一排。可当他们把刀各自挥向同一个系统的时候，发现刀与刀之间，是有缝的。

这一章，我们不学新的概念。我们要回答一个你读到这里一定会冒出来的问题：

**这十一章的东西，我怎么把它们装到同一个系统上？**

## 这一章要回答什么

读完这一章，你应该能回答：

1. 从一条需求到上线运维，一条完整的时间线上，这本书的每一个概念分别在哪个位置接力？
2. "所有概念是同一系统的不同切片"是什么意思？切片之间靠什么连接？

3. 为什么每个切片各自合格，系统还是可能失败?
4. 什么是顺次切片、什么是横切切片?为什么不能把横切切片当成顺次的一站?
5. 两个最不像的系统（业务流程 vs 硬件×软件），为什么走的是同一张时间线?
6. 为什么说”整合不是最后一步，是每一步”？一条变更的接缝，怎么列成一张缝合清单?
7. 为什么隐含需求是最深的接缝？场景里没被开发成规定需求的东西，后面会发生什么?
8. 为什么”不属于任何一片”等于”没人负责”？为什么技术系统上线不等于工程系统成功?
9. 遇到概念困惑，为什么先问”它在另一个世界的版本长什么样”？“概念是刀、系统是大象”怎么用?

---

## 这一章的骨架

这一章要立的是”整合”的切片与接缝。先把骨架立起来——后面三幕，都是给它添血肉。

> **所有概念，是同一系统的不同切片；切片靠四条接缝缝合；概念是刀，系统是大象。**

> 术语注：这套”切片 / 顺次横切 / 四条接缝 / 缝合清单”的命名与判据，是本书对前十一章的整合主张（把它们读成同一系统的一条时间线），不是任何标准的条文——它顺着第 13 章”版本顺着流、基线垂直切”长出来，欢迎检验、欢迎校准。这套”缝合清单”，在后记四角色里归**系统工程师（团队）**产出——全称”系统集成与缝合清单”。

这一章的骨架是**一把刀、四条缝、一头大象**：一把刀——**切片**：十一个概念是切同一头大象的十一把刀，切法只有两种：顺着时间切，和横着时间切；四条缝——**承接 / 参照 / 记录 / 问责**：切片之间靠它们缝合，不靠默契；一头大象——**概念是刀，系统是大象**：整合不是第十一个概念，是视角的转换。

第一幕把”整合”看清：切片是什么、刀从哪儿下。第二幕把它和容易混的东西分开：接缝断了不是部件坏了、不属于任何一片等于没人负责、技术上线不等于工程成功。第三幕把它用起来：一张时间线，七个站点，把切开的缝，缝回去。

---

## 17.2 第一幕 整合是什么：十一把刀，切同一头大象（定界）

> 这一幕把”整合”看清——先看病（每一片都合格，接缝断了），再立反直觉命题（所有概念是同一系统的不同切片），然后数十

> 一把刀（切片只有两种：顺次与横切），最后把两条案例线，放到同一张时间线上。

### 17.2.1 先看病：每一片都合格，接缝断了

把恒信的编号事故和冰链的时间戳事故放在一起看，你会发现它们不是两个不同的病，是同一个病在两种世界的两副面孔。

恒信这边，**接缝断在部门和职责之间**。签章编号由 CA 服务商生成——这条依赖属于谁？需求分析师觉得这是设计的事，架构师觉得这是集成测试的事，测试觉得这是需求没写的事，配置管理觉得这是变更记录的事。

每个人在自己的切片里检查了一遍，都找不到”编号来源”这一项——因为**它本来就不在任何一片里，它活在片与片之间**。

冰链这边，**接缝断在系统部件之间**。固件记设备本地时钟，云端按服务器时间归档——两个切片各自正确，中间缺一个”时间戳口径”的约定。这个约定，在需求里是隐含需求，在设计里是跨叶子的接口，在测试里是集成用例，在配置里是接口基线。它同样不属于任何一片。

前十一章，我们治的是词汇病：同一个词各指各的，同一种东西叫了不同的名字。这一章的病不一样——**词汇病治好了，概念病还在**：我们把”需求”“设计”“评审”“架构”“配置”“治理”这些概念，当成了彼此独立的房间，以为把每一间打扫干净，系统就成立了。

可系统不是一间一间的房。**系统是一头大象，概念是切大象的刀**。切法不同，肉还是同一头。你把大象切成十一块，把每一块都磨得光亮，然后宣布”十一块都合格了”——你依然拼不回一头大象，因为你切的时候，把肉和肉之间的筋腱切断了，没人把那道筋腱缝起来。

恒信和冰链的团队，就是那把切完之后把刀收进刀鞘、再不回头看的刀手。

### 17.2.2 反直觉命题：所有概念是同一系统的不同切片

这一章的反直觉命题，一句话：

> **所有概念，是同一系统的不同切片。**

前十一章的每一章，都让你以为自己在学一件新的东西。第 3 章学标尺，第 6 章学种树，第 7 章学数据，第 8 章学看门人，第 13 章学版本和基线——十一章下来，你可能觉得自己收了一排工具，每个工具一格抽屉。现在我们要把这个印象翻过来：

**不是十一个抽屉，是一头大象。**需求、设计、数据、评审、验证、确认、架构、架构设计、架构决策、配置、治理——它们是**切同一头大象的十一种切法**。每一章讲的都是同一个系统，只是从不同的角度下刀：

- 第 3、4、5 章从”**这头大象要成为什么**”下刀——它的标尺；
- 第 6 章从”**标尺怎么被造出来**”下刀——它的骨架；
- 第 7 章从”**它造出来的是什么**”下刀——它的血肉；
- 第 8 章从”**怎么知道它造对了**”下刀——它的看门人；
- 第 10、11、12 章从”**哪些决定决定了它是它**”下刀——它的不可还原涵义；
- 第 15、16 章顺着同一刀走到底——站内的链条怎么走完、这一站的产出怎么交给下一站；

· 第 13 章从”**它一直在变，怎么还说得清**”下刀——它的记忆；

· 第 14 章从”**谁有资格决定它往哪走**”下刀——它的舵。

肉是同一头。十一把刀切下去的角度不同，于是每一章看见的纹理都不一样——这既是前十一章的魅力，也是前十一章的陷阱：**看得越细，越容易忘记自己在切同一头大象**。

注意，这不是”盲人摸象”的老故事。盲人摸象错在**摸**——每个盲人只摸到一部分，把部分当全体。我们这本书不是这样：每一个概念都**正确**地描述了系统的一个真实剖面，没有一个是错的。

真正的病不在”摸错”，在”**切了就不管**”——切片是分析的手艺，**缝合是整合的手艺**，我们学了十一章切片，这一章学缝合。

医学院的解剖课，学生把一具身体一层层切开、标注、归档，是为了最终能闭上眼睛，在脑海里看见完整的人——**切开永远是为了整体，不是为了把切片本身装进镜框**。工程里没有”解剖完摆回去”这一步：系统是切完还要继续活的，你一边切它一边在变，切口不会自己长好。

所以工程里的缝合，比解剖难得多——**切片是减法，它回答”这头大象由什么组成”；缝合是加法，它回答”这些切片怎么共同活成一头大象”**。十一章教你减法，这一章补上加法。

所以，“整合”不是第十一个概念，不是把所有概念再来一遍的”总流程”。它是一个**视角的转换**：从”我在做需求/我在做架构/我在管配置”，换到”我切的是同一头大象，我知道每一刀从哪儿下、切的是哪一块、怎么拼回去”。而要缝合，先要知道切的方向——这引出最后一个关键区分：**切片有两种**。

### 17.2.3 十一把刀：切片有两种——顺次，与横切

顺着时间切的，是**顺次切片**：需求→设计→把关→运维，一站接一站，前一站的产出是后一站的输入。它们排成一条时间线，像接力赛的棒次。其中”设计”站的产出形态，是**产品数据**（第 7 章）——数据不另占一站，挂在站点②下。

横着时间切的，是**横切切片**：架构（含架构设计、架构决策与架构定义落地）、配置管理、治理。它们不排在时间线上的任何一站，而是**垂直贯穿每一站**——从需求到运维，每一站都有不可逆的决定要做、都有产出要被记录、都有边界要被裁决。

你大概已经认出来了：这个”顺/横”的区分，正是第 13 章讲配置管理时的那句话——**“版本顺着流，基线垂直切。”** 那章我们用里程桩和收费站比喻一条时间线；现在把那个比喻放大到整本书：**顺次切片是流上的里程桩，横切切片是垂直切断流的收费站。**

前十一章里，第 3~8 章教你顺次切片，第 10~14 章教你横切切片——第 9 章工程师是角色章，不占任何切片站点，单独站在这两套切法之外。

第 15、16 章不是第十二、十三把刀——切的是同一块，把架构这一站**走完**：站内的链条走到底，再把产出交给下一站。**把两套切法当成一条流水线，是整合里最常见的错位。**

### 17.2.4 一张时间线：双线合流

现在，我们把这两套切法放到同一条时间线上。

第 14 章结尾，我们预告过两条线合流：恒信守住了跨部门的编号标准，冰链重建了固件升级的边界。这一章，它们各自要上一个新变更，走一遍完整的闭环：

- **恒信**：电子签章（CA 电子合同）合规需求，从黄程的一句话，走到上线运维；
- **冰链**：断链自动补采，从疾控的一纸要求，走到在役批次的固件升级。

一个是跨部门的业务流程系统，一个是硬件×软件的冷链温控箱——**全书最不像的两个系统**。可它们走的是同一张时间线：

**一条变更闭环 · 同一条时间线，两条线并排走**



功能→分配→产品基线 → ⑤ 配置 ← 固件 v2.5 基线

（横切） 让变化说得清 版本顺着流·基线垂直切

编号来源谁有资格裁决 → ⑥ 治理 ← 固件升级边界谁批准

（横切） 谁掌舵谁划桨 委员会是过滤器

– – – – – – – – – – – – – – – – – – – – – – – – – – – – – –

上线≠工程系统成功 → ⑦ 运维 ← 批次在役升级·退役

旧流程退役·档案可查 从出生活到退役 技术系统≠工程系统

顺次切片（时间上接力）：① 需求 → ② 设计 → ③ 把关 → ⑦ 运维

横切切片（垂直贯穿每一站）：④ 架构（含架构设计与架构决策）· ⑤ 配置 · ⑥ 治理

这张时间线，就是这一章的活地图。下一步，先别急着上路——第二幕先把”整合不是什么”磨清楚，第三幕再沿着这张图，一站一站走。

---

## 17.3 第二幕 整合不是什么：接缝与一象一刀（磨边界弧）

> 这一幕把”整合”和它容易混的东西分开——磨边弧三条边一条条磨过去，每条都靠反例定型：接缝断了不是部件坏了、不属于任何一片等于没人负责、技术上线不等于工程成功；磨完三条边，回到正题，指名道姓那四条接缝（缝合靠什么）；最后做跨域织线（方法底座时刻）——概念是刀，系统是大象。

### 17.3.1 磨边一：接缝断了，不是部件坏了

上一幕立了”整合不是第十一个概念、不是总流程”——但”不是”具体长什么样？这一节磨的，就是整合具体”不是”的几件事。把两场事故摆在一起，最容易犯的第一个错：**把接缝断了，当成部件坏了**。

恒信这边，编号标准没错——错的是”编号从哪来”这条依赖，掉进了”CA 对接”和”台账登记”两片叶子之间。冰链这边，固件没问题、云端没问题——错的是”时间戳口径”这个约定，落在了固件和云端两个切片中间。两场事故，部件全好，全死在部件之间。

**如果只看部件，你会修错地方**：恒信去改编号格式，冰链去查固件逻辑——都治不了病。第 2 章那句”部件全对 ≠ 系统对”的涌现属性，在接缝上的样子，就是这样。

### 17.3.2 磨边二：不属于任何一片，等于没人负责

第二个错，是把”不属于任何一片”当成”不归我管”。

恒信的”编号来源”，需求文档里没有、设计树里没有、测试用例里没有、基线的变更记录里也没有——它哪儿都不在。冰链的”时间戳口径”，固件叶子写”设备本地时钟”，云端叶子写”按服务器时间归档”，中间没有一个枝梢说”两端时间戳要对齐”——它同样不属于任何一片。

**“不属于任何一片”，等于”没人负责”**。凡是落在所有切片之间的依赖，必须有一个责任人；没有责任人，那里就是将来爆雷的位置。恒信的编号、冰链的时间，都不是修好哪个部件能解决的——因为部件没坏，坏的是没人认领。

### 17.3.3 磨边三：技术上线，不等于工程成功

第三个错，是把”技术上线”当成”工程成功”。

恒信的电子签章上线了——服务器部署了、接口通了、验签能验了。可如果法务不能真用它审合同、财务对账对不上、业务方签完章还要手工补一遍台账，那技术系统上线了，工程系统并没有成功。

第 2 章讲过：工程系统 ⊃ 技术系统。**承诺的是整个社会技术系统能运转，不是设备跑起来**。冰链的 v2.5 推送了、固件测过了、云端测过了，可疾控审计问一句”补采记录的时间戳是哪来的”，还是答不清——技术上线只是把电点亮，利益相关方真的用起来，工程系统才开始活。

### 17.3.4 四条接缝：切片靠什么缝合

磨完三条边，回到正题：缝合靠什么？章首那两场事故，失败都在接缝上。走到这里，我们可以把这四条接缝指名道姓地讲出来了。它们贯穿全书，只是以前散在各章里：

**第一条接缝：承接——切片之间靠”契约链”缝合**。每站的产出，就是下一站的输入：规定需求被设计树承接，设计叶子被验证对照，预期用途被确认对照。第 13 章讲过，每一对相邻环节都是”局部甲乙方”，基线是冻结端、版本是交付端。

**切片不是一排独立房间，是一条传送带上的接力点**——前一站不产出可承接的东西，后一站就无从下手。恒信断在”编号来源”，就是因为”编号从 CA 来”这个事实没有被任何一站承接。

**第二条接缝：参照——切片靠”回到同一个系统”缝合**。每个切片都在回答关于同一个对象的三个问题：它应该是什么、实际是什么、

怎么变成这样（第 13 章配置管理三问）。需求回答”应该是什么”，验证回答”实际是什么”，配置记录回答”怎么变成这样”。三个切片各答一问，凑齐了才对得上同一个系统。

**切片之间的缝隙，靠回到系统本身来合拢**——遇到两个切片对不上，先别互相指责，把两个答案放回同一头大象上对。

**第三条接缝：记录——切片靠”组织记忆”缝合**。决策记录（第 12 章）、配置信息（第 13 章）、追溯矩阵（第 4 章）、评审记录（第 8 章），所有切片都在往同一条记忆里写。**缝靠记录缝合，不靠默契**——口头约定的接口、没写进基线的变更，就是恒信和冰链两次事故的共同死因。切片之间如果只靠人脑记住接缝，人走茶凉，缝就断了。

**第四条接缝：问责——切片靠”谁有资格负责”缝合**。所有权 → 治理 → 管理 → 执行（第 14 章权力链）。切片不是自动接力的，是靠”谁有资格批准、谁担责”接力的。恒信”编号来源”接缝断掉，本质是没人被授予裁决”编号从哪来”的资格；冰链”时间戳口径”补上，本质是有人（利益相关方确认）为这条接口担了责。**缝是缝出来的，不是自动合的；每一条缝，都要有一个责任人**。

四条接缝，四句自问：

- **承接**：这一站的产出，下一站能直接拿来用吗？
- **参照**：几个切片对同一头大象的描述，对得上吗？
- **记录**：这条接缝，人走了还查得到吗？
- **问责**：这条接缝，有名字、有主人吗？

四问都答得上，接缝是缝好的；任何一问答不上，那道缝就是未来的事故现场——恒信的编号，冰链的时间，都不是技术事故，是四问里有一问答不上来的事故。

### 17.3.5 跨域织线：概念是刀，系统是大象

磨完四条接缝，还差最后一步——把”概念”本身横过来看。

你有没有发现：恒信和冰链，一个业务流程系统、一个硬件×软件系统，用的却是同一批词——需求、设计、把关、决策、配置、治理、运维。它们为什么能用同一批词？因为**概念是刀，系统是大象**。刀不挑哪头大象：同一个概念，在恒信是一副样子，在冰链是另一副样子，骨相却完全相同。

恒信的”编号标准”和冰链的”数据所有权”，就是同一个概念的两种世界版本。恒信这边，合同编号由谁归口、来源在哪——是法务定的流程规则；冰链这边，温度记录以谁的时间为准、谁能动采集算法——是数据链定的所有权边界。

一个落在流程世界，一个落在硬件世界，问的都是同一件事：**这份数据的归属和口径，由谁说了算**。第 12 章说这是最不可逆的决策之一，第 10 章把它立成铁规矩——两个世界，同一个位置。

这就是第 1 章’迁移’构件在整本书层面的兑现——本章把它叫**概念跨域**，也是这一章的方法底座时刻。遇到任何概念困惑，先问一句：**它在另一个世界的版本长什么样？**

恒信看不懂”固件基线”，就看合同版本号下面那条被批准冻结的参照；冰链看不懂”跨部门编号标准”，就看温度记录的数据所有权边界。换一个世界，概念还是那个概念，站的还是同一个位置。概念不

是软件世界的私有财产，也不是硬件世界的行话——它是”让一个系统从出生活到退役”这件事的通用语言。

---

## 17.4 第三幕 整合往哪去：一张时间线，从切片回到系统（运用）

> 这一幕把尺子用起来——一张时间线，两种系统，七个站点，一站一站走完一条变更的完整闭环；然后把切开的缝缝回去（缝合清单），看两个最不像的系统在同一张时间线上各就各位（概念跨域的可见证据），收束到”让一个系统从出生活到退役”。

### 17.4.1 一张时间线，两种系统

走之前先说清一件事：为什么敢把两个这么不同的系统放在同一张时间线上？

因为**这张时间线不描述任何具体技术**。它描述的是同一件事——一个系统，从它”要成为什么”被确定的那一天起，到它”退役”那一天止，中间所有被一个变更唤醒的活动。

下面，我们沿着这张时间线走一遍。每到一个站点，你会看到两个团队各自怎么做、那站切片回顾哪一章——更重要的是，你会看到**接缝在每一站是怎么被缝上的**。恒信的编号事故和冰链的时间戳事故，都不是最后一刻才发生的；它们从需求那一站起，就埋在切片之间，只是当时没人知道——那道缝，该谁来缝。

电子签章和断链补采，一个是加一个模块，一个是改一段固件，看起来八竿子打不着；但它们在这张时间线上占的是同一个位置、走的是同一条路。前十一章的所有概念，在这张图上各就各位——**这就是”概念跨域成立”的检验：换一个世界，概念还站得住。**

现在开始。每一站都从”这站切的是大象的哪一块”讲起，然后两个团队上场，最后切片回顾。

### 17.4.2 站点① 需求：标尺从哪来

**这一站切的是：这头大象要成为什么。**

恒信，黄程先开口。不是开会，是在走廊里：“新法规下来了，电子合同得有电子签章，要有 CA 验证。这事你们做一下。”

如果是三年前的恒信，这句话会直接变成一张开发任务单。但那是第 4 章之前的事了。徐漾没有急着写 SRS，他先问了一句：“**先别管系统要做什么，先讲清楚它怎么被用**。”于是他写了一份 ConOps——不是条款，是故事：

> 黄程收到一份电子合同。他打开，系统验证这份文件的签章是真实有效的（CA 验签）；他审条款，改了一处，重新签；归档的时候，这份电子合同和纸质合同进同一个台账，有同一个编号体系，财务下月对账能对上。

故事讲完，才轮到条款。规定需求一条条落下来：验签算法、防篡改、归档格式……最后徐漾在那份 SRS 里，单独划了一条：“**电子签章的合同必须进台账，编号与既有编号标准一致**。”这条没人提过，是他

从故事里挖出来的——**隐含需求不会自己开口**，它藏在”法务收到一份电子合同”这个场景里。

“编号要和纸质合同一致”这条，后来成了整件事的命门。它写进需求的时候，恒信没人觉得它特殊；可它恰恰是掉进接缝的那条线——**所有后续切片都要承接它，承接的人却不知道它在这儿**。

> **走查说明**：这一站的每一步都是”正确做法”的演示——真实那场事故里，徐漾没来得及写 ConOps、“编号进台账”也没写进 SRS，黄程的走廊一句话被直接当成口头约定，接缝在第 1 站就断了。案例进展① 会把这条缝一处一处补回来。

冰链这边，疾控的要求是：“断链之后，数据要能补回来。”文正同样先写故事：

> 一辆 M3 冷链车在隧道里断了 30 秒链。出隧道后，设备自动重试采集，把断档期间的温度数据补传回来。平台上看不到 30 秒的空白，能看到补采的记录——补采的时间，就是设备当时真实记录的时间。

故事里有三个词后来都成了条款：断链判定阈值、补采窗口、**时间戳口径**。第三个词，疾控没提，文正是写故事写到”补采的时间就是设备当时真实记录的时间”这句时，才意识到它必须被规定——**设备的时**

**间，和服务器的时间，不是同一个时间**。他把它写进了需求规格，标注为需要利益相关方确认。

但请注意：这条”时间戳口径”写进需求的时候，冰链的云端工程师和固件工程师各读各的。固件工程师读到”设备本地时钟”，云端工程师读到”按什么时间归档”——**一句话落进了两个切片，各自读出了各自的理解**。需求这一站，把它写出来了；是谁往下传，还没人认领。

> **切片回顾 · 站点①** 这一站挂回第 3、4、5 章：需求是**开发**出来的，不是**管理**出来的——黄程的走廊一句话是”需要”，不是”需求”；ConOps 讲”故事”，SRS 讲”条款”，先讲清怎么被用，才谈得上必须做什么；规定需求 = 验证的参照，提供不出客观证据的，还不是规定需求。起点判据（完整、无歧义、不冲突、隐含需求已识别）在这里把关。任务理解这一层，要落到业务方签字的《系统目的或任务理解确认单》（后记四角色，系统架构师清单的入场券）。 **接缝提示**：这一站最容易断的两条缝，恰是恒信的”编号进台账”和冰链的”时间戳口径”——它们都是**隐含需求**，藏在场景里。隐含需求不被这一站开发出来，后面所有的切片都够不着它——验证验证不到，配置记录不了，治理也裁决不了。**切片之间最深的接缝，常常从第一站就埋下了**。

### 17.4.3 站点② 设计：标尺怎么被造

**这一站切的是：标尺怎么被造出来。**

需求定了，轮到设计。第 6 章讲过：设计不是写，是种——一棵树，枝干管结构，叶子管内容。

万辉这次没写六十页散文。他种了一棵树：框架定义是电子签章域的枝干——**签约发起 / CA 对接 / 验签 / 台账登记**；详细定义是挂在枝梢上的叶子，每片叶子有编号、描述、指标、验收标准、映射目标。分解、分配、分析，三遍功课做完。

但这棵树种完，徐漾来挑刺了。他指着两片叶子：“这片是’CA 对接’，管签章验证；那片是’台账登记’，管编号落账。**中间呢？CA 返回的签章编号，要不要进台账？怎么进？**”

万辉愣住了。需求规格里那条”电子签章的合同必须进台账”，他分配的时候，挂到了”台账登记”那片叶子上。可”CA 对接”叶子管的是”拿到签章结果”，“台账登记”叶子管的是”编号落账”——**两个枝梢各干各的，中间的过渡，不在任何一片叶子上。**

“这正是我上次写散文时埋的雷，”万辉说，“散文是一个字挨着一个字，接缝藏在段落里看不见；树是一片一片叶子，接缝长在叶子之间，反而显出来了。”

他补了第三片叶子：“签章编号映射”，负责”CA 返回的编号 → 台账编号体系”。这片叶子不是从任何需求里搬下来的，它是**从接缝里长出来的**——一棵树的完整性，不只看每片叶子是否健康，还看叶子和叶子之间有没有东西把它连起来。

冰链的设计树，是同样的故事换了一层皮。固件一棵树、云端一棵树、App 一棵树，三棵树靠接口对接。分解分配分析做完，文正把”

时间戳口径”那条需求拿给两端看——**固件叶子写”设备本地时钟打戳”，云端叶子写”按服务器时间归档”**。两个枝梢各自正确，中间没有一个枝梢说”两端时间戳要对齐”。

水忠问：“这条谁负责?”没人答得上来。最后，云端的董勐开了口：“那就固件负责打设备时间，云端负责原样存，展示的时候统一换算”——**接口约定，在那一刻才真正被造出来**。它应该长在设计树的枝梢上，但因为没人认领，它是在会议上口头长出来的，后来也没写进任何一片叶子。

> **切片回顾 · 站点②** 这一站挂回第 6、7 章：设计文档是一棵树，不是散文；方案不是**凭空**想出来的——骨架分解出来、候选构思出来——框架定义（树干树枝）管结构、详细定义（叶子）管内容；语义四查（覆盖完整、不超出、逻辑一致、**约束继承**）、分配三自检（全覆盖、1:1、N:1 合理）、符合性分析（每个枝梢语义+指标都过）。设计站的产出形态，就是第 7 章说的**产品数据**——开发产品=开发产品数据。 **接缝提示**：设计这一站，接缝长在**两片叶子之间**。约束继承查的是”需求里的约束有没有传下来”，但”跨枝梢的接口”不在任何一片叶子上——它要靠人工补一片叶子，或靠分析时专门查”两片叶子之间的过渡”。恒信的”签章编号映射”和冰链的”时间戳对齐”，都是接缝里

> 长出来的叶子。**符合性分析查的是每片叶子的健康，接缝的缝合，要靠人盯着树的结构**。

### 17.4.4 站点③ 把关：三个看门人

**这一站切的是：怎么知道它造对了**。

恒信的电子签章走到验收。万辉想发一份测试总结报告，徐漾拦住了：“三件事，分开办。”

- **评审，对目标**。不是逐条核对需求，而是抬头看：电子签章要达成什么？法务能离线验证签章真实性、财务对账不断、业务方在真实流程里签一份合同不卡壳。评审会开到一半，黄程问了一个需求里没有的问题：“客户发来的 PDF 是打印件，扫回系统，能验吗？”——这是对目标的追问，不是对条款的核对。
- **验证，对规定需求**。验签用例、防篡改用例、归档用例，对照 SRS 一条条给客观证据。测试全绿。
- **确认，对用途**。最后一步，不是让测试员替业务方签字。黄程带着法务部的人，在真实的审批流程里，真的审了一份电子合同——从收到、验真、审批、归档，走一遍。**确认的参照在场景里，不在文档里**。

三个看门人依次签字，才轮到”验收通过”四个字。但徐漾心里记着一件事：那一站补出来的”签章编号映射”叶子，**验证用例里其实没有**——测试测了”CA 返回签章结果”，测了”台账登记编号”，没测”CA 返回的编号落进台账”。三个看门人都站在门口，可”签章→台账”那条缝，谁也没把它作为一道门。

冰链的验收，李旭把确认做到了隧道里。验证：断链判定、补采窗口、温度精度，对照规定需求，实验室全过。确认：一辆真的 M3，装上 v2.5，在真的隧道里断了 30 秒链，开回来，平台能对上补采的记录。**只做验证不做确认，测试全过、没人用；这一站，他们把确认当成真实的一关来过**。

可同样的问题也在：确认的时候，他们验的是”补采功能能跑通”，没验”补采时间戳和原始记录时间戳对齐”——因为那条缝，在上一站是口头约定的，没进任何叶子，自然也没进任何确认场景。**三个看门人守的都是门，门与门之间的墙，不是守门的职责**。

> **切片回顾·站点③** 这一站挂回第 8 章：评审对**目标**、验证对**规定需求**、确认对**预期用途**——三把不同的尺子；验收不是一句话，是三个看门人依次签字（评审签”方向对”、验证签”做对了”、确认签”能用”）；只做验证不做确认 → 隐含需求系统性漏失 → 测试全过、没人用。**接缝提示**：把关这一站是**切片之间第一个显式的缝合点**——它对照的，是站点①的规定需求；它确认的，是站点⑦的真实用途。它天生是站在”两个切片之间”看东西的。但它的视野里只有”已被写下来的东西”（规定需求、设计叶子）；**落在切片接缝里的东西（口头约定的接口、没长成叶子的依赖），不写进任何一片，三个看门人就都看不见**。缝没被记录，就没人守。

### 17.4.5 站点④ 架构（含架构设计与架构决策）：不可逆的提前投资

**这一站横着切：从需求到运维，每一站都有不可逆的决定。**

注意这里的切法变了。站点①到③是顺次——上一站的产出喂给下一站。站点④开始，我们要把刀横过来：**架构（含架构设计与架构决策）不排在时间线的某一点，它垂直贯穿每一站。**

需求里有不可逆的决定（业务边界怎么划），设计里有（模块接口怎么定），运维里有（升级策略怎么选）——每一站动身前，都要问两问：这个决定守的，是不是”目的不走样”？它又有多不可逆？——**前一问定它够不够格进架构，后一问定给它什么待遇、排在什么时候。**

恒信电子签章里，藏着两个高不可逆的决策。第一个，引入第三方 CA 服务商——**对外接口契约**，签了五年合同，换服务商等于重做一遍验签。第二个，签章编号和台账编号的关系——**数据模型**，一旦确定，全集团的历史合同都绕着它转。

万辉这次没有再说”改个编号的事，今天就能上”。他记得第 12 章那三个月的连环三爆。

这次他把两关分开走。**先过目的关**：这两个决定守的，是不是”合同合规、月底账目清楚对得上”这条目的之下的不变量？是——去掉它们，目的就落空。**再过代价关**：用三层判断框架（影响半径／信息成本／修改风险）量它们多不可逆，再按量出的档给待遇。两关都过，才**在需求这一站就把它们识别出来，提前投资**——写决策记录，六问俱全：名称、背景、选项、选择、影响、不可逆性。

以”CA 服务商”那个决策为例，两关是这样走的：

- **目的关**：去掉它，电子合同验不了签、合规过不了、台账也就无从对账——目的是否落空？落空，因此它够格被锁进架构（承重判据见第 10 章 §10.2.4、第 11 章 §11.3.6）。
- **代价关·影响半径**：验签、归档、台账、法务流程全线耦合——任何一个环节要换接口，都是整条链重走一遍。
- **代价关·信息成本**：CA 的验签协议、证书体系，全集团没人懂，知识锁在服务商手里，换一家等于把这段知识重买一遍。
- **代价关·修改风险**：签章是合规动作，改错了是法律风险，不是返工风险。

代价关三问走完，结论明确——**高不可逆，早做、充分分析、多人审查**。于是这份决策记录在需求评审会上就拿出来，请法务、财务、业务三方各审了一遍——而不是等签完合同、上线半年后，才发现接口契约锁死了整个演进空间。

“这次不一样，”万辉说，“上次我是把’数据模型’当’字段’，随手一改。这次我在**需求还在改得起的时候**，就先把目的关过了，再把它当成最高不可逆对待了。”

冰链的决策，比恒信更硬。断链补采的时间戳口径，往深里问一层，是**温度记录的数据所有权**——这批记录以谁的时间为准、谁来定标准，直接决定批次追溯的可信度。

水忠和文正也是两关走下来的。先过目的关：这批记录以谁的时间为准，直接决定批次追溯可不可信——拿掉它，目的就落空，它够格进架构。再过代价关，三层判断框架走一遍：影响半径——所有批

次追溯；信息成本——固件云端两端，中间没有一个人两头都懂；修改风险——数据完整性，历史记录能不能对得上。结论：**几乎最不可逆，按最高级别对待**。

他们做了一个很小、但很贵的决定：把时间戳口径钉死成一条规定需求，标注”需利益相关方确认”，再让固件、云端两端坐在一起确认。这个决定花了一个下午——可它买的是后来不用返厂改固件、不用重构云端归档的整个成本。**在便宜的层做对的决策，贵的层才不用付返工的学费**。

站点④还有一条**作业链**，第 15 章把它拆成了九步：目的 → 基本概念或属性与管控原则 → 架构目标 → 利益相关方关切 → 架构视角 → 架构视图 → 架构评估 → 描述评估 → 双向对账。

三者分工要分清：**这一章回答”架构这一站在整条时间线上做什么”，第 15 章回答”这一站内部怎么走完、每一步由谁签字”，第 16 章回答”这一站的产出怎么交给下一站”**。关切的分流与裁决要过治理机构的手（站点⑥），评估要由独立于设计方的评估方做（第 15 章 §15.4.6 十二步最小清单）——**横切切片之间也靠接缝缝合，而不是各走各的**。

架构站点，走的就是第 11 章的架构生成六步（定驱动→备选→权衡→分配接口→视角规格＋表达与基线→评审背书），收口在评审背书（第 8 章）与基线（第 13 章）。

> **切片回顾·站点④** 这一站挂回第 10、11、12 章：架构 ≠ 一张框图，是系统不可还原的涵义——“基本”的准入判据是**必要性**

＋**充分性双判据**（去掉还是不是它 / 缺了系统垮不垮 / 换掉性质变不变，只是必要性一侧的问法；不可逆性一类是**代理**，管排序不管准入）。架构设计有四件反直觉的本性——概念幸存、属性涌现、管控原则自加、关系克制。这层不可还原涵义落成纸面，就是系统架构师产出的《系统基本概念和基本属性清单》（后记四清单，§18.4.6）。

架构决策**不可避免**——不是做不做，是早晚，在成本低的时候主动做，或在成本高的时候被动做；不可逆性是**经济属性**，不是技术属性。

**两关的次序不能颠倒：先过目的关（它守的是目的之下的不变量吗），再过代价关（按不可逆性排优先级与时机）**。代价判据只是**代理**——它会漏判（不贵但承重的，如”日志须可被外部审计员独立读取”），也会误纳（很贵但与目的无关的，如框架版本、组织惯例技术栈）；三层判断框架只在”基本”这组根本**之内**定待遇，不产生新的架构成员。

最不可逆的往往是数据所有权、数据模型、对外接口契约；决策记录六问把决策从个人知识变成组织资产。**接缝提示**：横切切片的缝合方式，是把”先过目的关、再过代价关”这两问**带进每一个顺次站点**。需求站的业务边界、设计站的接口契约、运维站的升级策略——这两问不是集中在某一次”架构评审会”上，而是散落在全程每一站里。**谁在每个站点的动身前先问目的、再问代价，谁就是在为这条横切线缝合**。章首那场事故里，

> 两问都没人问——“编号来源”于是被当成”编号格式”，悄悄变了，没人认出它是数据模型层面的不可逆决策；这一次重走，冰链在需求站就先确认了目的、再把时间戳口径钉成不可逆决策。**同一道缝，断与合，差别只在这两问**。

### 17.4.6 站点⑤ 配置：让变化说得清

**这一站也横着切：从需求到运维，每一站的产出都要被记录、被冻结、被说得清**。

恒信的电子签章，这站没再走”另存为 v2”的老路。**基线，就是被批准、被冻结、作参照的定格**——需求定稿，立**功能基线**；设计分配定稿，立**分配基线**；实现交付，立**产品基线**。三站三切，每切一次，都立成一条说得清的基线。

然后变化来了。上线前两周，CA 服务商通知：验签 API 升级，旧接口三个月后停用。搁在以前，这就是开发群里一句”接口换了，改一下”。

这一次，它走的是变更控制：**变更申请 → 影响评估 → CCB 批准 → 实施 → 建立新基线**。影响评估单上写着：“改验签接口，涉及 CA 对接叶子；不影响编号映射叶子；不可逆性中——对外接口契约的一部分。”

CCB 批了，改完，产品基线升了一版。状态记录同步更新，配置审计按季度抽查——**标识、控制、状态记录、审计，四大活动转起来了，不再是只有”变更控制”一道手续**。

“关键不是我们多走了流程，”万辉说，“是**每一个变化，都能说清’从什么变到什么’了**。上次部署包叫 final_2.zip 的时候，回滚

都找不到上上个版本；这次随便哪个版本，都能说清它改了什么、基于哪条基线。”

冰链这边，固件 v2.5 的基线，是给疾控审计准备的答案。上次审计员问”这批记录是哪版固件采的”，水忠查不清，因为升级记录只有一句”已推送 v2.3”。

这次，v2.5 的基线下有批准（CCB 批的）、有冻结（入库的参照固件）、有记录（从 v2.4 到 v2.5 改了哪些，逐条在案）。补采功能上线后，审计员再问，水忠能指着一份文件说：**这批记录是 v2.5 采的，补采逻辑是这一条，变更记录在这**。

“上次我们说’版本只是刻度，基线才是参照’，”文正说，“这次刻度底下，终于有参照了。**版本号还是那个版本号，可它现在能回答问题了**。”

> **切片回顾 · 站点⑤** 这一站挂回第 13 章：版本是时间轴上的演进刻度（顺着流），基线是被批准冻结的参照（垂直切）——“版本顺着流、基线垂直切”；没有基线，变化就不可言说；变更控制循环：CR → 影响评估 → CCB 批准 → 实施 → 新基线；配置管理三问：应该是什么 / 实际是什么 / 怎么变成这样；配置管理是架构决策的守护者（决策记录 ↔ 配置信息，影响分析 ↔ 变更控制）。**接缝提示**：配置管理是**时间线上的缝合线**——它把每一个顺次站点的产出，固化成”可描述的参照”，让切片之间的交接有据可依。站点①的需求、站点②的设计、站点③的

> 证据、站点④的决策记录，全都汇进这一条记录里。**恒信和冰链上次的失败，本质都是”切片之间的约定没有被记录”；配置这站，就是给所有接缝上档案**。缝可以不缝，但每一条缝都要留档。

### 17.4.7 站点⑥ 治理：谁掌舵谁划桨

**这一站还是横着切：每一站的边界，都要有人有权裁决**。

恒信电子签章走到上线前，冒出一个跨部门问题：签章编号要不要纳入编号标准、由谁归口？法务说”签章编号就该进标准，法规要的”；财务说”进标准行，但生成时机得对齐台账，否则对不了账”；业务说”别给我增加签章步骤”。三方各执一词，单边谁都定不了——这是**管理**管不了的，因为它不在任何单个管理者的权限里。

三年前的恒信，会开个季度会，四十五分钟，橡皮图章一盖，各回各家。现在的恒信，委员会改造过：**治理在持续运转，评估发现偏差即处置**。这次就是评估发现的那个偏差。

委员会把三方拉齐，做了一次裁决：**签章编号纳入编号标准，由法务归口；台账在签章完成时登记，财务对账口径对齐**。一条规则立下去，往后”签章编号进不进台账”不再需要开会——**规则替委员会挡住了 990 个争议，要裁的，正是这次这个漏网例外**。

“这次裁决的不是数据，是**边界**，”徐漾说，“谁有资格定’编号从哪来’——这是治理问题。上次没人有这个资格，编号来源悄悄变了也没人拦得住；这次有人裁决了，边界立住了。”

冰链的治理问题，同样藏在固件升级的边界里。M3 的老规矩：固件升级不能毁已出厂的箱子。可这次断链补采，恰恰要动采集算法

——第 13 章那次事故，就是修一个丢读数 bug 时动了采集算法，把断链判定逻辑改了。这一次，谁能批准”动采集算法”这个升级？

不是水忠一个人。单靠嵌入式工程师拍板，那是管理——他有权决定固件怎么改，但没权决定”改了对已出厂批次追溯的影响能不能承受”。这个决定越过单个管理者，需要一个框架级的裁决：**涉及数据所有权边界的变更，由谁批准**。

冰链没有正式委员会，但他们立了一条治理规则：凡是动采集算法、影响批次追溯的变更，必须过数据所有权的裁决关——谁认可，谁担责。规则靠治理机构持续行使评估与监督来守护，不是每季度开一次会；评估发现”有人想绕过去”这种偏差，即启动裁决。

> **切片回顾 · 站点⑥** 这一站挂回第 14 章：治理 ≠ 管理——治理是掌舵（定方向、定规则、裁例外），管理是划桨（把事做好）；治理是**持续系统**，治理机构持续行使利益相关方参与/评估/指导/监督四活动（定期评审只是监督/评估的常规载体——传感装置，把传感当治理整体就是橡皮图章）；**目的变更**（战略调整、产品定位改变）= 显式发生的强制扳机，一发生即强制重开闭环；**目的漂移**（目的悄悄变而没人宣告）靠以目的为参照的定期评审捕捉（两类扳机见 §14.3.7）；委员会是**过滤器**不是处理器（定 3 条规则自动处理 990 个，只裁 10 个例外）；配置管理是治理的一只手，基线由谁批准、变更由谁裁决，由治理回答。**接缝提示**：治理是**人与规则之间的缝合线**——它回答”谁有资

> 格为接缝负责”。恒信的编号来源、冰链的采集算法边界，都是落在切片之间的依赖；谁来认领它们、谁有权裁决它们，不是技术问题，是授权问题。**前几站的缝合靠手艺（识别、补叶、记录），这一站的缝合靠资格（谁有权为边界负责）**。手艺缝合的缝，会随人走；资格缝合的缝，才立得住。

### 17.4.8 站点⑦ 运维：从出生活到退役

**这一站，是顺次切片的终点，也是整头大象的归处。**

恒信电子签章，上线了。技术系统跑起来了——服务器部署了、接口通了、验签能验了。

部署是玉杰盯着上的。他说：“服务器、接口、验签都通了，可这只能证明技术系统醒了。能不能算成功，得看业务那边这一周用起来的样子。”

但第 2 章讲过：**工程系统 ⊃ 技术系统**。承诺的是整个社会技术系统能运转：法务真能用它审合同、财务对账真能对上、业务方真愿意用（而不是签完章再手工补一遍台账）。

上线第一周，徐漾盯着后台：电子签章的合同，有没有真的从”CA 返回”一路走到”台账登记”、走到”财务对账”？——那一条跨切片的链路，才是这一次上线真正的验收。

他又请三个利益相关方各回答一个问题：法务的人——“你能真用它审合同了吗”；财务的人——“下月对账，电子签章的合同能对上吗”；黄程——“你愿意用它，而不是签完章再手工补一遍流程吗”。

三个问题都答”能”，工程系统才真正接上了电——**技术系统上线只是把服务器点亮，利益相关方真的用起来，系统才开始活。**

与此同时，旧流程开始退役：纸质签章不再是主流程。但退役不等于销毁——历史合同档案要随时可查，靠的是站点⑤的配置管理。**一个系统从出生走到这，不是结束，是开始运营：它从此在时间线上活着，直到下一条需求把它再次唤醒，再走一遍这张时间线。**

冰链的 v2.5，通过 OTA 推向在役批次。不是所有箱子一起升——M3 有标准版、医疗加强版、工业版（第 13 章的变体），各版本交叉着在役；固件升级按边界分批走，老批次按老规矩处理，能升的升，不能升的不硬升。

批次是逸人在管。他说：“标准版能升，医疗加强版要过数据所有权那道裁决关，工业版固件锁死，不能硬升——不是我不想升，是边界在那儿。”他翻开批次台账，“每批升不升、什么时候升、升完基线在哪，都记着。”

M3 从第一批出厂，到一批批在役升级，再到逐步退役替换——**它的一生，就是这张时间线在一批批箱子上反复走。**

冷链运输能正常跑，不只是箱子能测：运输的人会用、监控的人能看、追溯的人查得清，人、流程、组织、信息都在转——那才是第 2 章说的工程系统。

再往后，还有最后一站常常被整个行业忘掉：**退役**。系统退役不是失败，是生命的完成——但退役同样要说清：这一批 M3 为什么退役、固件档案归档到哪、历史批次追溯还查不查得到。

第 13 章的配置管理三问，在退役这一天问的，是一个系统的完整一生：**它应该是什么、它实际是什么、它怎么变成这样**。从出生活到

退役，每一站都在，最后一站尤其不能缺席——退役不是把系统丢掉，是把它的一生交给档案。

水忠站在仓库里，看着一箱箱 M3 装箱上车，忽然说了一句：“我们修了这么多章的概念，其实是在干同一件事——**让一个系统，从出生活到退役，每一步都说得清、接得上、有人负责**。”

文正点头：“前十一章我们以为在学十一样东西。今天走完这一趟，才明白——**是一条时间线，走了十一遍**。”

> **切片回顾 · 站点⑦** 这一站挂回第 2 章：工程 = 科学原理 × 判断创造，在约束下，用系统方法，造出有用且安全的东西；工程系统 ⊃ 技术系统——承诺的是整个社会技术系统能运转，不是设备跑起来；系统工程是元工程，涌现属性：**部件全对 ≠ 系统对**。站点③的三个看门人、站点⑤的配置记录、站点⑥的治理边界，在这里汇合——上线不是终点，是系统从出生活到退役的中途一站。**接缝提示**：运维这一站，是**所有切片的总缝合点**。恒信的”签章→台账→对账”链路、冰链的”固件→云端→追溯”链路，就是把前几站的接缝一根根串起来验。**第 2 章那句”部件全对 ≠ 系统对”，在这一站第一次有了完整的画面：部件就是切片，系统就是接缝缝合之后的那头大象**。而”上线成功 ≠ 工程成功”，则是这张时间线的最后一课——系统要一直活到退役，中间每一次变更，都要再走一遍。

## 案例进展①｜恒信：把编号来源缝回去

恒信没有重做电子签章。徐漾把”编号来源”那条线追了出来——它穿过六个站点，本应在六个站点各被缝上一次；而那场事故里，六处一处都没缝。他把六处，一处一处补回来，列了一张**缝合清单**：

- **需求站**：补一条规定需求——“签章编号必须经台账登记，纳入法务归口的编号标准”。原先这是走廊里的一句假设，谁也没写成条文；现在它从隐含需求，变成了验证够得着的规定需求。
- **设计站**：把”签章编号映射”补成一片叶子，挂在”CA 对接”和”台账登记”之间——负责”CA 返回的编号 → 台账编号体系”。原先它是评审会上口头约定的，现在它有编号、有描述、有指标、有验收标准。
- **把关站**：补一个集成验证场景——“签章 → 台账 → 对账”，三个看门人重签。原先签章和台账各测各的，中间那条链没人验；现在它成了一等公民。
- **架构站**：把”编号来源变更”识别为数据模型层面的高不可逆决策——写决策记录六问，按最高级别审查。原先它是”改个编号”被随手放过；现在它被看清了：改的是数据模型，不是字段。
- **配置站**：把”编号来源变更”写进基线的变更记录。原先”编号从哪来”这个事实不在任何记录里；现在它是这条基线的显式变更历史。

· **治理站**：委员会裁决——“编号来源由法务归口，台账在签章完成时登记，财务对账口径对齐”。原先谁也没有资格定这个边界；现在边界立住了，往后不用再开会。

小白把”签章 → 台账”的集成用例补了出来，她标得格外醒目：“CA 返回的编号落进台账，这一行以前不在任何用例里；现在它是验收清单上的硬要求。”

玉杰接的是”把关站”和”配置站”两处：集成场景补出来后，他盯着”签章 → 台账 → 对账”这条链重跑了一遍；变更记录写进基线后，他逐项核对。他后来反复说的一句话是——“以前这条链在监控里是隐形的——我能看见部署、看见服务，看不见’编号从哪来’。现在，我能看见它了。”

两周后，黄程又拿来两张表。台账的编号，和电子签章系统里的编号，对上了。

“六处，一处没少，”徐漾说，“但你我都清楚——**这六处其实只要缝上任何一处，都不会炸**。缝得多，是因为六站都把它漏了。”

万辉看着那张缝合清单，忽然说了一句：“上次我们把这叫’流程’——需求、评审、测试、配置、治理，走一遍，感觉全做过了。**现在才明白，那不止是几道工序，是切同一头大象的几个切面；真正的活，是切面之间那道筋，有人把它连起来**。”

## 案例进展②｜冰链：把时间戳口径钉成接口

冰链同样没推倒重来。文正把”时间戳口径”追到三个切片，逐一缝合：

· **需求站**：需求规格补一条规定需求——“补采记录的时间戳 = 设备本地时间，原样存储；展示时由平台统一换算时区”。原先这句话两头各读各的；现在它是验证的参照。

- **设计站**：在固件和云端两棵树之间，补一片”时间戳对齐”接口叶子——固件打设备时间，云端原样存、统一换算。原先它是会议上的口头约定；现在它进了接口基线。
- **把关站**：补一个真实确认场景——再跑一趟隧道。固件、云端、App 联起来测”补采时间戳和原始记录时间戳对齐”，三个看门人重签。

需求、设计、把关——三站各补一处。至于架构站的尺子、配置站的基线、治理站的边界，在重走时间线的时候已经立住了；事故里真正漏掉的，就是当初没人认领的这三处。**一处不漏，才叫缝合；漏一处，就是下次事故的入口**。

疾控再来复查，审计员翻开批次追溯，问的还是那句话：“这批补采的记录，时间戳是哪来的?”水忠指着接口基线，一字一句：“**设备本地时间，原样存储，展示统一换算——v2.5 基线，变更记录在这**。”

董勐点开追溯平台：“平台这边原样存的，补采记录和原始记录同一把时间戳，展示层才换算。”

逸人翻开批次台账，逐批核对后指给审计员看：“补采的记录落在 3 号批次；升了 v2.5 的箱子和没升的分列，哪批升级、哪批没升，都对得上。”他顿了顿，“固件版本跟着箱子走——水忠那儿是接口基线，董勐那儿是追溯平台，我这儿是批次台账。这个缝，三头各自留了档。”

审计员点点头，过了。出门水忠说：“上次我们俩说，‘两个都没问题，中间那条缝有问题’。**这次我们把中间那条缝，变成了一站**。缝不再是缝，是接口——接口有人签、有人记、有人验。缝上了，就回到系统里了。”

## 案例进展③｜两个最不像的系统，同一张时间线

走到这里，回望一眼：恒信，一个跨部门的合同审批系统，软件、流程、组织；冰链，一个冷链温控箱，硬件、固件、云端、供应链。**全书最不像的两个系统。**

可它们走完了同一张时间线。七个站点，一个不多，一个不少；每一站的切片回顾，挂回的是同一批章节；断的缝、合的缝，断在同一个地方、合在同一个地方。

这就是这本书开篇那个承诺的兑现：**一个概念，两个世界的例子。**概念不是软件世界的私有财产，也不是硬件世界的行话——它是”让一个系统从出生活到退役”这件事的通用语言。

| 概念 | 恒信 · 业务流程系统 | 冰链 · 硬件×软件系统 |
| --- | --- | --- |
| 规定需求 | “签章编号必须进台账”——可验证 | “补采时间戳 = 设备本地时间”——可验证 |
| 框架定义 | 电子签章域：枝干管结构、叶子管内容 | 固件/云端/App 三棵树的枝干与叶子 |
| 确认 | 法务在真实流程里审一份电子合同 | 真车在隧道里断链 30 秒再跑一趟 |
| 基线 | 合同版本号下面被批准冻结的参照 | 固件 v2.5 入库时被批准冻结的参照 |
| 数据所有权 | 合同编号标准归法务归口 | 温度记录的时间标准归数据链统一 |
| 治理 | 跨部门”编号来源”谁有资格裁决 | 动采集算法的固件升级谁有资格批准 |

同一个概念，两个世界各长一副样子，骨相却完全相同——**这就是”概念跨域成立”的可见证据**。

这本书没有教你一套行业方法论——它教你的是识别切片的手艺，和缝合切片的手艺。这两样手艺，跟行业无关。

## 案例进展④｜收束：让一个系统从出生活到退役

现在，回到章首那两场事故。

恒信的编号对不上，不是编号标准错了——标准没错，是”编号从哪来”这条依赖掉进了切片之间的缝。

这条缝，在站点①就该被需求开发当隐含需求挖出来，在站点②就该被补成一片叶子——签章编号映射，在站点③就该被三个看门人验——签章→台账集成场景，在站点④就该被不可逆决策的尺子量过——编号来源归数据模型，在站点⑤就该被记进基线——编号来源的变更，在站点⑥就该被治理裁决——编号来源归法务口。

它要是在任何一个站点被缝合，都不会拖到上线两周后爆雷。

冰链的时间戳对不上，同理。它该在站点①被写成规定需求，站点②被补成接口叶子，站点③被确认场景验到，站点④被不可逆框架量过（时间戳口径 = 数据所有权），站点⑤被记进 v2.5 的基线，站点⑥被数据所有权的边界管住。

**六个站点，有一站缝上，事故就不发生。恒信是六站都没缝上，冰链是三处已立、三处漏——六个切片各自太”合格”了，合格到每个切片都相信自己不是那个缝接缝的人**。

而如果六站都缝上了呢？那就是这一章走完的完整闭环：一条需求，从法务的走廊一句话、疾控的一纸要求，被开发成规定需求，种成设计树，被三个看门人验过，被不可逆决策的尺子量过，被基线记

档，被治理授权，最后上线、运营、在役升级、退役——**所有概念，在同一条时间线上接力，让一个系统从出生活到退役**。

回望第 1 章，我们发了一把四问的尺子（它展开成九格）。你用它在十一章里量了十一个概念，量得很细，每个概念都填满了格子。现在你发现：**那不是十一头小象，是同一头大象的十一种切法；十一个格子不是十一个房间，是切同一头大象时，你依次在它身上落下的十一刀**。尺子没换，量错的是你以为它量的是十一个东西——它从头到尾，量的是同一头。

这就是第 14 章结尾预告的那句话，现在你亲眼看着它走完了。也是这本书全部十一章合起来的唯一一句话：

> **系统不是十一个房间，是一头大象；概念不是十一把工具，是一个系统的十一种切法。切得再细，也要缝得回来。**

---

## 认知反转

回到章首。恒信的编号对不上、冰链的时间戳对不上——如果只看表面，你会说”需求做了、设计做了、测试做了、配置管理做了、治理也做了，怎么还是出事？“现在我们知道，那两场事故，连错四层，一层比一层深。

**第一层，把”每个概念做对”当成”系统做对了”**。每个切片各自合格，不等于系统合格——系统死在切片的接缝上。

**需求的正确解决不了”编号从哪来”没有主人，测试的正确发现不了”签章→台账”没有场景，配置的正确记录不了”时间戳口径”口**

**头约定的变更**。切片是对的分析，缝合才是对系统的分析；我们把切片磨得锃亮，把缝合扔给了默契。

**第二层，把”概念”当成”房间”**。需求是需求的房间，架构是架构的房间，配置是配置的房间——每间都打扫干净。

**可概念不是房间，是切同一头大象的刀**。房间之间是走廊，走过即可；切片之间是肉和筋腱，切了就要缝。把概念当房间，等于切完大象，把刀收进抽屉，宣布大功告成——**你甚至不记得大象长什么样了，你只记得自己那间房里的十一个抽屉**。

**第三层，把”整合”当成最后多出来的一步**。以为整合是全书收尾时把概念汇总一下的”总流程”。

**可整合不是一步，是每一步都在做的事**：需求那站的隐含需求开发、设计那站的接口补叶、把关那站的集成场景、架构那站的一问、配置那站的变更留档、治理那站的边界裁决——全是缝合。**没有一个切片天生带接缝，接缝都是每一站人为缝出来的**。

**第四层，也是最深的一层，以为读完十一章，学到的是十一样东西**。需求、设计、评审、架构、配置、治理……你以为收了一把又一把刀。**其实你学到的是同一件事：让一个系统从出生活到退役，每一步都说得清、接得上、有人负责**。

切片会过时、会退役——需求会变、架构会演、基线会被新基线替代、治理规则会被新例外改写——但**系统活在时间线上**，每一次变更，都把它重新唤醒，再走一遍这张时间线。十一章不是十一个终点的十一个入口，是同一趟旅途的十一个站点，而你现在，站在了旅途上。

用第 1 章的话收一次束：对”整合”（也就是对整本书），你原来只是”见过”——背得出”所有概念是同一系统的不同切片”，十一个概念也各答得出”它是什么”。

这一章把它推到”拥有”——能说出切片是什么、整合不是什么（接缝断了不是部件坏了、不属于任何一片等于没人负责、技术上线不等于工程成功）、四条接缝靠什么缝合、一张时间线怎么把系统从出生活到退役。**能答全这四问——切片是什么、整合不是什么、四条接缝靠什么缝合、一张时间线怎么把系统从出生活到退役——才算拥有。你会拿着这条时间线，一站一站把接缝缝上。**

概念这条链，到这里就收束完了——**这一章的反转，是全书概念章里的最后一个反转**：从第 3 章到第 16 章，每一个概念都在这一章被装回同一头大象。第 18 章要量的不是概念，是”架构师”这个人——那条线，不在这张概念总图上。

> **概念是刀，系统是大象。前十一章教你怎么把刀磨快，这一章教你怎么别把大象切散——切了，就缝回来；缝不回来，系统就会从接缝里漏掉，像恒信的编号，像冰链的时间。**

用第 1 章 §1.4.4 的尺子量到底：这一章到了**第 ④ 层（解释框架）**——前十五章的概念，在同一张图上合成了一家的说法。

## 小结

这一章把前十一章合起来了。**所有概念，是同一系统的不同切片**——需求、设计、数据、评审、验证、确认、架构、架构设计、架构决策、

配置、治理，都是切同一头大象的刀，切法不同，肉还是同一头；把它们当成彼此独立的房间，是这一章的误解现场，也是整合最常见的死法。

我们把一条变更的完整闭环走了一遍：**顺次切片**（需求→设计→把关→运维，时间上接力）与**横切切片**（架构（含架构设计与架构决策）、配置、治理，垂直贯穿每一站）各就各位，恒信与冰链两条案例线在七个站点上并排合流——最不像的两个系统，走的是同一张时间线，**概念跨域成立**。

切片之间靠四条接缝缝合：**承接**（契约链）、**参照**（回到系统三问）、**记录**（组织记忆）、**问责**（谁有资格负责）——接缝要有人负责，缝合发生在每一站，而不是最后一步。

回望全书：第 2 章问”我们到底在做什么”（工程系统 ≠ 技术系统），第 3、4、5 章立标尺，第 6、7 章造标尺（第 7 章=产品数据，设计站的产出形态），第 8 章把三关，第 10、11、12 章看住不可逆的涵义，第 13 章让变化说得清，第 14 章为系统掌舵，第 15 章把架构这一站走完，第 16 章把架构交到设计手上——**七站走完，一头大象从出生活到退役**。这一章没有教新概念，它教的是那件最容易被十一章淹没的事：切了，就缝回来。

到这里，一条变更从出生活到退役的闭环走完了。但书还没走完——因为还有一个”物种”没有回答：**架构师**。下一章，正文的最后一章，我们回答这本书的书名；再往后，书末的附录会把全书的尺子

收进五个抽屉（术语表 / 概念档案集 / 误解清单 / 参考文献总表 / 练习答案）。

如果只带走一句话，带走这一句：

> **概念是刀，系统是大象。切得再细，也要缝得回来。**

运维台前，钉上了那张缝合清单。以前他盯”系统活着没活着”；这一章之后他知道——接缝要写下来、要有人认领，谁部署、谁验收、谁留痕，一行一行都对得上，才叫缝上了。

---

## 本章概念总图

第 17 章没有新概念——它的”概念总图”，是**全书概念总图**。把全书读成一张图：

**全书概念总图 · 把全书读成一张图（第17章 = 无新概念，收全书）**

**工程：我们到底在做什么（第2章 · 外框）**

**工程 = 科学原理 × 判断创造，在约束下，用系统方法，造出有用且安全的东西**

**工程系统 ⊃ 技术系统：承诺的是整个社会技术系统能运转**

**涌现属性：部件全对 ≠ 系统对**

**顺次切片（时间上接力，一站喂给下一站）**

**① 需求：标尺从哪来、要全 第3章 需求、设计开发、质量（定需求）+ 第4章 需求工程（开发造血·管理防腐）+ 第5章 需求的完整性（三类需求·三个载体）**

② 设计：标尺怎么被造　第6章 需求与方案结对（一次 Y=F(X) 求值·每份文档一棵树；分配生成·分析迭代·分解分析）

数据：开发产品 = 开发数据　第7章 产品数据（设计站的产出形态 · 两域三态）

③ 把关：三个看门人　第8章 评审对目标 / 验证对规定需求 / 确认对预期用途

⑦ 运维：从出生活到退役 第2章 工程系统回归（上线 ≠ 工程成功）

横切切片（垂直贯穿每一站，不是时间线上的一站）

④ 架构（含架构设计与架构决策）：不可逆的提前投资

第10章 架构（不可还原涵义·必要＋充分双判据）

+ 第11章 架构设计（本质＋管控原则＋落地；架构生成六步）

+ 第12章 架构决策（先过目的关、再过代价关；三层框架只是代理）

+ 第15章 架构定义落地（站内九步链：目的→基本概念或属性与管控原则→目标→利益相关方关切→视角→视图→两把尺子评估→双向对账）

+ 第16章 架构定义到设计定义（这一站的产出怎么交给下一站：入口·两条转化·一道出口）

⑤ 配置：让变化说得清　第13章 版本顺着流·基线垂直切·没有基线不可言说

⑥ 治理：谁掌舵谁划桨　第14章 决策系统·持续系统·委员会是过滤器

反直觉命题：所有概念是同一系统的不同切片

—切法不同，肉还是同一头

—切片靠承接/参照/记录/问责四条接缝缝合，让系统从出生活到退役

## 本章判据

> 本章的判据是本书的主张，不是任何标准的条文——它们都当场可检验：欢迎检验、欢迎校准。

1. **反直觉命题**：所有概念是同一系统的不同切片——“每个切片都合格”不构成”系统合格”；系统的成败，常常长在切片的接缝上。**违反即崩：恒信编号对不上、冰链时间戳对不上——需求、设计、测试全做了，系统还是死在切片之间。**
2. **顺次 vs 横切**：需求→设计→把关→运维是顺次切片（上一站产出喂给下一站；设计站产出的是第 7 章的产品数据；运维站回归第 2 章工程系统外框）；架构（含架构设计与架构决策）、配置、治理是横切切片（垂直贯穿每一站）——**横切的，不能当成顺次的一站来做。违反即崩：恒信把”编号来源”当顺次字段随手一改，横切切片的尺子只在改的那一站过——其他站全放手，上线两周后爆雷。**
3. **接缝四缝**：承接（契约链，基线=冻结端版本=交付端）、参照（回到系统三问：应该是什么/实际是什么/怎么变成这样）、记录（组织记忆：决策记录/配置信息/追溯/评审记录）、问责（所有权→治理→管理→执行）——**切片之间靠四条接缝缝合，不靠默契。违**

**反即崩：冰链”时间戳口径”靠会议口头约定、没写进接口基线——靠默契的缝，人一走就没人记得。**

4. **接缝要有人负责**：凡是落在所有切片之间的依赖（编号来源、时间戳口径、跨模块接口），必须有一个责任人；“不属于任何一片”等于”没人负责”，等于将来爆雷的位置。**违反即崩：恒信”编号来源”需求、设计、测试、基线都没有——没人认领，爆雷正中：上线两周后对不上账。**
5. **每一站都是一次缝合**：需求站挖隐含需求、设计站补接口叶、把关站验集成场景、架构站问不可逆、配置站记变更、治理站裁边界——整合不是最后一步，是每一步。**违反即崩：把整合留到最后一步——恒信六站一处都没缝，全堆到上线后补，两周爆雷。**
6. **隐含需求是最深的接缝**：场景里没被开发成规定需求的东西，后面所有切片都够不着它——验证验证不到、配置记录不了、治理裁决不了。**违反即崩：恒信”编号进台账”藏在场景里没开发成规定需求——验证验证不到、配置记录不了，上线两周后才发现。**
7. **概念跨域成立**：遇到一个概念，先问”它在另一个世界的版本长什么样”——恒信的编号标准就是冰链的数据所有权，换一个世界，概念还是那个概念。**违反即崩：把”基线”“治理”当本行业行话，换个系统就认不出——恒信看不懂冰链的固件基线，冰链看不懂恒信的编号标准。**

8. **上线不是终点**：技术系统跑起来 ≠ 工程系统成功；系统从出生活到退役，每一次变更都重新走一遍完整闭环。**违反即崩：冰链 v2.5 推送了、测过了，疾控审计一问”补采时间戳哪来的”还是答不清——技术上线了，工程系统没接上。**
9. **两个最不像的系统走同一张时间线**：概念是通用语言，不是行业行话——这是检验你学没学会的终极判据。**违反即崩：信了”硬件那套不适用我们”——恒信和冰链最不像的两个系统，走的却是同一张时间线。**
10. **缝合清单要写得出来**：一条变更的接缝，要能被列成一张清单——每条缝缝在哪两个切片之间、靠四条接缝里的哪一条、责任人是谁。写得出来的，才是缝得上的；写不出来的，就是埋在系统里的雷。**违反即崩：事故时恒信”编号来源”六处一处都列不出来——需求/设计/测试/决策/配置/治理没一处有它；事后补成清单，两周一处不落地缝上。**
11. **概念是刀，系统是大象**：遇到任何概念困惑，先问”它切的是同一头大象的哪一块”——切片的意义，永远在被切之前的整头大象；切开是为了缝合，不是为了收藏切片。**违反即崩：把切片收进抽屉——切完把刀收进刀鞘再不回头，大象拼不回来，系统从接缝里漏掉。**

## 自查 · 这些说法对吗？

1. “需求、评审、测试、配置管理、治理全都做了，系统肯定没问题。”——**错**。切片各自合格 ≠ 系统合格；接缝没人看，系统死在切片之间的缝里。恒信的编号来源、冰链的时间戳口径，都掉在所有切片之间。
2. “整合是项目最后多出来的一道工序。”——**错**。整合不是一步，是每一步都在缝合：需求挖隐含需求、设计补接口叶、把关验集成场景、配置记变更、治理裁边界。
3. “架构决策是需求评审之后的一个环节。”——**错**。架构（含架构设计与架构决策）是横切切片，从需求到运维每一站都有不可逆决定；把它当顺次的一站，等于只在一站做决策，其他站放手。
4. “配置管理只在发布前才需要。”——**错**。配置横切全程：功能基线→分配基线→产品基线，变更控制贯穿每一站；没有基线，中间任何一站的”变”都不可言说。
5. “我们这是业务流程系统，硬件那套的质量方法不适用。”——**错**。恒信和冰链是最不像的两个系统，走的是同一张时间线——概念跨域成立，这是全书的立书之本。
6. “评审过了、测试也全过了，为什么还要确认?”——**错**。评审对目标、验证对规定需求、确认对预期用途，三把尺子缺一把都不行；只验证不确认，隐含需求系统性漏失，测试全过、没人用。

7. “编号标准没变，来源变了，所以不算变更。”——**错**。编号来源是数据模型/数据所有权层面的变更，高不可逆——必须走决策记录、评审、变更控制、治理裁决。恒信就是栽在”规则没变，来源变了”。
8. “把每个概念学懂，就等于把系统做对了。”——**错**。概念是切同一头大象的刀；学懂概念是把刀磨快，还得知道刀从哪儿下、切下的部分怎么拼回去——缝合的手艺，概念书不教你，这一章教你。
9. “上线成功就是工程成功。”——**错**。技术系统上线 ≠ 工程系统成功；法务真能用、财务对得上、业务愿意用，整个社会技术系统转起来，才是工程系统在运转。
10. “读完这本书，你学了十一样独立的东西。”——**错**。你学了同一件事：让一个系统从出生活到退役，所有概念在同一条时间线上接力。十一章不是十一个终点，是同一趟旅途的十一个站点。
11. “概念是行业行话，换了行业就不成立了。”——**错**。概念是通用语言，不是行业行话——恒信的编号标准就是冰链的数据所有权，换一个世界，概念还是那个概念。
12. “接缝在心里记得清就行，不用写成清单。”——**错**。缝合清单写得出来，才缝得上——每条缝缝在哪两个切片之间、靠四条接缝里的哪一条、责任人是谁；写不出来的，就是埋在系统里的雷。

13. “不属于任何一片的东西，不归我管。”——**错**。“不属于任何一片”等于”没人负责”——落在所有切片之间的依赖，必须有一个责任人；没人认领，那里就是将来爆雷的位置。

## 练习

1. 画出你们最近一次真实变更（或一次事故复盘）的完整闭环时间线，标注每一站对应的概念；在每一站之间标出”接缝”，然后圈出你们最弱的一环。
2. 用”顺次/横切”把这本书的第 3~14 章分类（跳过第 9 章工程师；第 2 章工程是外框；第 15、16 章切的是④架构这一站，归横切那一行）；再把你所在组织的岗位、流程按同样方式分类——横切的那几条线，各自是谁在贯穿？有没有一条横切线没人贯穿？
3. 找一次你们”每个切片都做了但系统失败”的经历（没有就用本书的电子签章事故），指出接缝断在哪两个切片之间、断在四条接缝（承接/参照/记录/问责）里的哪一条。
4. 用第 13 章的配置管理三问（应该是什么/实际是什么/怎么变成这样）复盘你们系统当前的一个模块——三个答案都对得上吗？哪一问答不上，哪里就在漏。
5. 用第 12 章三层判断框架（影响半径/信息成本/修改风险），评估你们最近一次变更里最不可逆的一个决定——它被提前投资了吗，还是等爆雷了才做？

6. 选一条你们系统的真实需求，走一遍第 3 章起点判据（完整/无歧义/不冲突/隐含需求已识别）——哪一条最弱？有没有像”编号进台账”一样，藏在场景里没被开发出来的隐含需求？
7. 把你们系统的某个模块拆成一棵树（框架定义 + 详细定义），用语义四查 + 分配三自检查一遍——有没有两片叶子之间缺一片”接缝叶”？
8. 用第 8 章三个看门人复盘一次真实上线：评审对目标、验证对规定需求、确认对预期用途，各自在门口站住了吗？“确认”是不是真在场景里做的？
9. 你们组织最近一次跨部门/跨团队变更，用第 14 章治理四问诊断：谁来治〔角色〕？依何治〔规则〕？如何治〔机制〕？治得怎样〔问责〕？——落到这次变更就是：谁裁决？依什么规则？机制是什么？谁担责？答得上来吗？答不上来的那一问，就是接缝。
10. 找一个”概念跨域成立”的实例：把一个概念（基线/确认/治理/不可逆性）从你们行业搬到你完全陌生的行业，它还能成立吗？成立在哪个位置？
11. 角色扮演”接缝人”：你负责把一条变更从头接到尾，列出你必须主动缝合的每一处接缝（至少 5 处），并说明每处靠四条接缝里的哪一条缝合、责任人是谁。

（答案要点见书末附录 E，与训练教材题卡同源）

## 参考文献

**标准** - R-09 ISO/IEC/IEEE 15288:2015 —— 系统生存周期（从概念到退役的完整框架）——本章时间线（从概念到退役）与站点⑦ - R-04 ISO/IEC/IEEE 42010:2022 —— 架构描述（利益相关方/关注点/架构视角/架构视图——“切片”的正式语法）——站点④ 架构 - R-05 ISO/IEC/IEEE 42020:2019 —— 架构过程（六过程：治理·管理·概念化·评估·细化·使能）——站点④ 架构 - R-08 ISO 10007:2017 —— 配置管理（基线/变更控制——“接缝留档”的机制）——站点⑤ 与接缝·记录 - R-02 ISO 9000:2015 / R-03 ISO 9001:2015 —— 需求、设计开发、质量的术语与过程——站点①②（需求、设计） - R-103 ISO 9000:2026 —— 评审（§3.11.2）/ 验证（§3.11.12）/ 确认（§3.11.14）——站点③ 三个看门人（第 8 章实引；本章按”引哪版挂哪号”登记） - R-11 IEEE 1028-2008 —— 评审——站点③ - R-17 ISO/IEC 38500:2024 / R-19 TOGAF —— 治理——站点⑥

> 对照全书这条认知链：你现在站在这条链的收束点，回看每一站，都在这张时间线上有自己的位置。

# 第 18 章 架构师：把整条链背在身上的人

> 本章主题：架构师这个角色——它首先是什么 / 十项能力清单 / 与系统工程师的分工 / 与相邻角色的边界 / 怎么长出来 / 怎么失败 演示对象：恒信万辉（有”架构师”头衔、缺底座）与冰链水忠（无头衔、做架构工作） 对应章节：收束第 9 章（工程师）、第 10 章（架构）、第 11 章（架构设计）、第 12 章（架构决策）、第 14 章（治理）、第 16 章（架构定义到设计定义）、第 17 章（整合）

---

## 18.1 误解现场

年度评优会上，恒信把”年度架构师”颁给了万辉。颁奖词是：**“全年产出架构图最多、架构文档最全。”**

会后，徐漾把万辉叫住，问了一个让他愣住的问题：“当选架构师，意味着你现在是什么？”

万辉张了张嘴。他发现自己答不上来。

办公室里三个人接话，给出三个答案：

- 开发说：“架构师就是更高级的工程师——升职了嘛。”
- 测试部的同事说：“架构师是画架构图的人，图画得最多那个。”
- 运维说：“架构师是什么都管的人，出了事都找他。”

三句话，三个”架构师”。和两年前那两间会议室里为”架构”的吵架一模一样——只是这次，被误解的是万辉自己。

如果你读完了前面十六章，你会立刻发现这三句话都错在哪：**“更高级的工程师”把职级当成了能力；“画架构图的人”把画图当成了架构；“什么都管的人”把权力当成了责任**。但纠正他们之前，我们要先把一个更根本的问题回答清楚——这个问题，也是这本书从第一页就在问的：

**架构师，到底是个怎样的物种？**

这一章，是全书最后一次回答这个问题。它不再新增任何概念——前面十六章把尺子都发完了。它只做一件事：**把”架构师”这个人，放到那十六把尺子下面量一遍**。

---

## 这一章要回答什么

读完本章，你应该能回答：

1. 架构师首先应该是什么？为什么”架构师”头衔不等于”在做架构”？
2. 架构师的十项能力是什么？哪六项是底座、哪四项是增量？
3. 架构师和系统工程师是什么关系？为什么”背在身上”是专责不是终责？为什么架构师对系统成败的账不兜底？
4. 架构师和技术负责人、项目经理、领域专家差在哪？为什么画四张图不算架构师？
5. “被任命”为架构师，为什么不等于”成为”架构师？能力是怎么长出来的？

6. 架构师最常见的失败模式有哪些？“什么都管”为什么反而危险？
7. 架构师怎么验证自己的方案？为什么”代码用测试验证，架构用评审验证”？
8. 架构师的产品数据是什么？为什么说架构师主责**架构的正确性**、系统工程师主责系统级、模块工程师主责模块级？为什么框概分离不是角色的分工？架构师要不要检查实现符不符合架构分配？
9. 恒信的万辉和冰链的水忠，谁是架构师？为什么？
10. 架构师再往上是什么？为什么”再往上”不是升职，而是关注维度的扩张？
11. 架构师每天在做什么？“架构活动”（architecting）的七动词链是哪七个？为什么说它是十项能力清单的母词？
12. 架构师和工程师在五个维度上差在哪？为什么说”工具不变，棋盘变大”？
13. 架构师对目的负责到什么程度？为什么目的定义权在业务方、架构师只有质询权没有改写权？

---

## 这一章的骨架

这一章要立的是”架构师”的十项能力。先把骨架立起来——后面三幕，都是给它添血肉。

> **架构师，是把整条链背在身上的人**。这条链从第 9 章那个底座开始——先是一个工程师（六项底座），一项都不能少；再往上，是四项系统级的增量能力。底座六项 + 增量四项 = **十项能力清单**。

先分清两句话里的两条”链”，免得混。这一章标题的”整条链”，是第 17 章那条从一条需求到上线运维的**价值链**——架构师把它看在眼里（看在眼里，不是担在肩上）；本章还要摆出另一条链——**架构活动链**（architecting 的七动词：构思→定义→表达→记录→沟通→认证恰当实现→维护与改进），它是架构师每天在做的活的母词。价值链是”系统的旅程”，架构活动链是”架构师的工序”。

这一章的骨架是**一份清单 + 一组尺子（十项体检判据 + 两把分工尺 + 一把母词尺 + 一把目的判据尺，共十四项）+ 一个试金石 + 一个分寸 + 一个判词任务 + 三个反直觉命题**：

- **一份清单**——十项能力：底座六项（①科学素养 ②方法论素养 ③实践能力 ④职业伦理 ⑤工程判断·横切 ⑥可复现意识·元），增量四项（⑦识别后果严重的决策 ⑧对系统的理论 ⑨把架构说清楚 ⑩判断不可逆性）；
- **一组尺子**——十项体检判据（逐项自问）+ 两把分工尺（专责尺、符合性尺）+ 一把目的判据尺；
- **一个试金石**——“换个人，拿着你负责的架构描述和决策，能不能接住、继续演进?”；

- **一个分寸**——架构师是工程师角色体系中的一个**专门角色**，不是”最大的工程师”，也不背系统成败的账；他对架构正确性负**专责**，系统工程师对系统成败负**终责**（第 9 章那把粒度尺，在这一章收口）；
- **一个判词任务**——这一章给恒信的万辉和冰链的水忠下判词：谁是架构师；
- **三个反直觉命题——架构师首先是一个工程师**（六项底座一项都不能少）；**头衔不定义架构师**（职级也不定义）；**专责 ≠ 终责**（架构师不是”更高级的工程师”，也不是”什么都管的人”——他背专责、不背终责，见”一个分寸”）。

第一幕钉”架构师”本身（十项能力各是什么），第二幕把它和容易混的东西分开（角色边界 + 专责终责分野 + 失败四式），第三幕回答”怎么长成、怎么被检验、落判词”。

三幕走到最后，判词会把”架构师”从头衔钉成能力清单——那是第 1 章”见过→拥有”在本章的落点。

---

## 18.2 第一幕 架构师是什么：十项能力清单（定界）

> 这一幕把”架构师”看清：先立一句开场白（先是一个工程师），再逐项拆十项能力——每项给”是什么”和判据，最后收在六个更锋利的差异上。

### 18.2.1 一句开场白：把整个物种钉在底座上

这一章第一句话，不是”架构师要懂很多技术”，也不是”架构师要会画图”。它是这句：

> **架构师首先是一个工程师。**

这句话不是客套，是一个次序问题。第 9 章给工程师立过一把尺子——六项能力：科学素养、方法论素养、实践能力、职业伦理，加上横切的工程判断、可复现意识。六项能力分属工程的四层结构（工程科学、方法论、工具实践、价值与伦理），缺一层，“工程”就不是完整的工程；缺一项，“工程师”就不是完整的人。

架构师是工程师的一个方向，不是工程师的超越。**没有那六项能力，“架构师”三个字只是一张名片；有了它们，才谈得上往系统级长。**

这不是修辞，是职业定义。在 ISO/IEC/IEEE 12207/15288 里，架构师不是另一个职业，是工程师角色体系中的一种 **specialization**——它是工程过程中的一个角色，不是工程师之上的另一个物种。专责终责的分野，第二幕专门讲。

所以”首先是一个工程师”，是它全部的起点：**先有元素级的正确，才谈得上系统级的方向**。

增量长在底座上，**底座缺一块，增量全是空转**。

所以这一章不讲”怎么当架构师”的技术——那些第 10、11、12 章已经讲透了。这一章讲的是**一个物种的解剖学**：它由什么构成，和谁像但不一样，怎么长出来，怎么死掉。

### 18.2.2 底座六项：三栏点名，每项给判据

第 9 章把六项能力展开过，这里不重复论证，只点名——但每点一项，都给它一把当场能用的判据：

| 底座能力 | 它是什么 | 判据（怎么检验） |
| --- | --- | --- |
| ① 科学素养 | 判断”系统靠什么成功”——理论不是空的 | 我对这个系统的理论是什么？靠什么成功、最大压力在哪、做错什么最致命？ |
| ② 方法论素养 | 方法不靠人传人——架构不随人走 | 我的方法能被别人学会吗？还是只有我会？ |
| ③ 实践能力 | 落得了地——架构不只在图上 | 我最近一次亲手落地，是什么时候？ |
| ④ 职业伦理 | 能力不是中性的，它定义方向 | 压力面前，我会不会为进度悄悄牺牲可靠性？有没有让利益相关方知情？ |
| ⑤ 工程判断 | “识别后果严重的决策”不只是一句口号 | 冲突里，我依据什么做取舍？ |
| ⑥ 可复现意识 | 判断换个人也接得住——不是手艺型架构师 | 我的判断换个人能接住吗？我写决策记录吗？ |

每缺一项，架构师这个角色会发生什么：

- 缺①：判断不了”系统靠什么成功”，理论是空的；
- 缺②：靠人传人，架构随人走；

· 缺③：落不了地，架构停在图上；

· 缺④：判断越准、破坏越大——能力不是中性的；

· 缺⑤：“识别后果严重的决策”只是口号；

· 缺⑥：判断换个人就接不住，成了手艺型架构师。

这一章后面的部分，会在四个场景里反复回到这张表：讲角色边界时、讲成长路径时、讲失败模式时、给万辉和水忠下判词时。**整张表，就是”架构师是个怎样的物种”的第一层答案：一个先把六项能力背在身上的人**。

### 18.2.3 增量四项：十项能力清单，长在底座上

底座六项，是”先是一个工程师”。架构师在这个底座上，还要再长出四项系统级的能力。第 10、12 章把它们一件件摆过，这一章把它们收成一张清单：

| 增量能力 | 它是什么 | 判据（怎么检验） |
| --- | --- | --- |
| ⑦ 识别后果严重的决策 | 对不重要的事情不决策 | 影响半径大、持续时间长、修改成本高的，按决策对待 |
| ⑧ 对系统的理论 | 靠什么成功 / 最大压力 / 做错最致命 | 我的架构有原理吗？换个上下文还成立吗？ |
| ⑨ 把架构说清楚 | 利益相关方 → 关注点 → 架构视角 → 架构视图 | 每个利益相关方的关注点，都有一张图能落吗？ |
| ⑩ 判断不可逆性 | 高不可逆早做、低可逆晚做 | 我认得出不可逆决策吗？该早做的早做、该推迟的显式推迟吗？ |

**十项合起来，才是”架构师是个怎样的物种”的完整答案**。下面四项增量，各展开一节——每一节，都对应前面某章的一把尺子。

为什么增量恰好是这四项？按 ISO/IEC/IEEE 42010:2022 的架构定义（逐字引文见第 10 章 §10.2.2）——架构是实体在其环境中的**基本概念或属性**（本质），加上用于该实体及其相关生存周期过程的实现与演化的**管控原则**（governing）。

这两样落在哪？本书的读法是：本质落到实体的**构成（元素及其关系）**上，被描述和承接（第 10 章 §10.2.5 交代了它们的去向——**它们是架构”可以关乎”的对象，不在定义句内，不是定义要素**）；管控原则约束的，正是这些构成怎么实现、怎么演化。

这一定义定了分工——**工程师的技能围绕元素（把组成部件做对），架构师的增量围绕两样内容：本质与管控原则（元素、接口与关**

**系是设计定义的产出**——15288:2023 §3.13，不是这一角色的专责对象）：识别后果严重的决策，认的是本质的承重与演化风险；对系统的理论，说的是本质如何支撑成功；把架构说清楚，说的是本质与管控原则怎么被表达成架构描述；判断不可逆性，判的是这些内容难不难撤销。**元素是工程师的；本质与管控原则的成立与守持是架构师的。**

四项增量合起来，还对应 15288 给"架构定义流程"（§6.4.4）定的三件事：**生成**候选架构方案、**选择**一个、用一组一致视图**表达**。生成、选择、表达——这就是架构师每天在做的三件事，增量四项，是这三件事背后要长出的能力。

> 术语注：15288 §6.4.4 的三件事是实引（正本过程名 System architecture definition process）；42010 架构定义上文是精炼转述，逐字见第 10 章 §10.2.2。"元素是工程师的、本质与管控原则是架构师的"这类分工读法，"增量四项＝这三件事背后的能力"这一映射，以及**"元素、接口与关系是设计定义的产出（15288:2023 §3.13）、不属于这一角色的专责对象"**这一归属判断，都是本书对标准的**读法**。两处不能混：**标准不替本书分配工程师与架构师的职责**；**标准也不把 elements／relationships 写进架构定义句**——42010:2022 §3.2 只给两样，§5.2.2 说的是架构"**可以关乎**"它们。欢迎检验、欢迎校准。

### 18.2.4 ⑦ 识别后果严重的决策

第 10 章给出过一句近乎悖论的话：架构师的核心能力，不是”做出正确的决策”，而是”识别哪些决策如果不正确，后果会很严重”。这句话的反面，比正面更常见：

**大多数团队把精力花在无关紧要的决策上，而对真正致命的决策，从不当成决策来对待。**

万辉的例子最典型。合同编号规则，在第 12 章被徐漾点破前，万辉一直当它是”今天就能改的字段”。他改起来很快——数据模型改一下，代码改三处，两天上线。他没识别出这是一次影响所有历史合同的决策，直到对账对不上、台账对不上、追溯对不上，三个月连环三爆。

识别后果严重的决策，不是”想得多”，是”认得出哪类决策必须按决策对待”。判据第 12 章给过：影响半径大、持续时间长、修改成本高。这三条不是给你判断”哪个技术方案好”的，是给你判断”这件事值不值得开一次正式的会”的。**识别，依赖的是底座里的工程判断（⑤）——认不认得清后果严重，本身就是一种判断力。**

**对不重要的事情不决策，本身就是一种好的架构决策**——这句话的另一面是：对重要的事情不决策，是架构师的第一宗罪。

### 18.2.5 ⑧ 对系统的理论

第 10 章说，架构是关于重要的东西——“whatever that is”。真正深刻的是后半句：什么是重要的，本身需要判断；判断的依据，是你对这个系统的理论。

所谓理论，就是回答三个问题：这个系统靠什么成功？它会面临的最大压力在哪？什么东西做错了最致命？**理论长在底座的科学素养（①）上——靠什么成功的判断，本身就是对系统的一种科学理解**。

拿 M3 演一遍，第 10 章演过：水忠的理论是”M3 靠冷链可追溯取胜”，另一位架构师的理论是”M3 靠温控精度取胜”。同一个箱子，两种理论，画出的图、做的决策、投入的方向完全不同。**这不是两种画法，是两种世界观**。

一个没有理论支撑的架构，就像没有物理学原理的工程——你可能碰巧做对，但不知道为什么对，换个上下文就错。反过来，好的架构师不是能画出最漂亮架构图的人，而是**对自己系统的理论最清醒的人**：他知道自己的判断依据是什么，因此也知道在什么条件下这个判断会失效。

### 18.2.6 ⑨ 把架构说清楚

架构是系统不可还原的涵义，但涵义存在系统里，没有人看得见。架构师的工作之一，是把看不见的涵义变成利益相关方看得见的视图——第 10 章讲过那条链：利益相关方 → 关注点 → 架构视角 → 架构视图。

水忠给客户发过四样东西，没有一样是他那张四层方块图：给疾控中心看追溯链路，给司机看状态视图，给维修看结构视图，给监管看合规视图。**图是死的，视图是活的——每个利益相关方看到自己该看的那一面，而它们合起来，说的是同一个 M3**。

把架构说清楚，不是”文档写得好”，是**让每个利益相关方的关注点都有一张图能落**。利益相关方的关注点天然冲突（第 10 章：不可消

除，只能权衡），架构师不消灭冲突，他给每个冲突一个可见的落点——这样冲突才能被讨论，而不是被闷在各自脑子里。

### 18.2.7 ⑩ 判断不可逆性

第12章讲了一整章不可逆性：判断不可逆不能只看”改起来多贵”，要看三层——影响半径（结构/时间/组织耦合）、信息成本（知识深度/隐含上下文）、修改风险（爆炸半径/数据完整性）。还有配套的三个动作：高不可逆早做、低可逆晚做、该推迟的显式推迟。

董勐想给客户开直连接口，被水忠点醒后，他自己把方案划掉、改走”受控导出”——这是判断不可逆性的一个完整案例：他认出客户直连会绕开追溯域，而追溯域的数据所有权几乎是最不可逆的东西（第12章）。**判断不可逆性，是把架构决策从”今天能不能上”的日常节奏里拎出来，放进”值不值得为它多花三周”的另一个时间尺度。**

这一项，是十项里唯一一项不在”架构”章（第10章）里、而在”架构决策”章（第12章）里长出来的——它提醒我们，架构师的物种属性，从来不只是”会画架构”。**判断不可逆性依赖的底座，还是工程判断（⑤）——认得出哪些代价大到不能等，本身就是判断力。**

### 18.2.8 六个更锋利的差异

十项能力是清单，清单背后还有六条更锋利的差异——它们不在清单上，却决定了架构师每天做的是另一种活。

**第一，决策域：从”局部、可逆”换成”全局、难逆”。**工程师的决策大多是局部的、可逆的——改错了，重来成本可控。架构师的决策是全局的、难逆的——改错了，代价要大得多（第12章那句”对不可

逆的提前投资”，在架构师这里是日常）。所以架构师必须认得出哪些决策”值得按决策对待”，这正是增量第⑦项（识别后果严重的决策）的由来。

**第二，认知对象从”需求”换成了”关注点”。** 工程师面对的是规定需求——明示的、可验证的，有明确参照。架构师面对的是关注点——利益相关方对系统的兴趣点：目的、质量、成本、进度，甚至组织政治。

关注点比需求宽得多，可以模糊、可以冲突、没有标准格式。所以架构师的第一个动作不是”接需求单子”，是识别关注点、排优先序。第 10 章那条链的起点就在这：利益相关方 → 关注点 → 架构视角 → 架构视图，第一步永远是先问”谁押了注、在乎什么”。

**第三，验证方式从”测试”换成了”评审”。** 工程师的产出可以对规定需求做测试验证；架构没有”唯一正确解”，只有”在当前关注点与约束下最合理的权衡”——它无法用测试证实，只能靠评审（architecture review）验证：rationale 成不成立、权衡是否合理、有没有更优备选。

这正是第 8 章那句话的落点：**评审对目标，验证对需求，确认对用途**。架构没有逐条需求可对，它只能对目标——那个由利益相关方关注点汇聚成的”为什么要建这个系统”。

所以架构师必须把决策理由写下来：**代码用测试验证，架构用评审验证**——评审审的，正是那份写下来的理由。架构师最隐性也最要命的一条能力，是”经得起评审”的论证能力。

**第四，时间尺度：从”当下交付”换成”跨版本演化”**。工程师对当下负责——这个版本交付得对不对；架构师对演化负责——这套本质与管控原则明年、三年后还立不立得住（第 13 章基线、第 11 章管控原则对演化的约束，都在量这条时间轴）。所以架构师的判断里永远带着一个时间维度：今天的选择，是在给后年留活路，还是把后年堵死。

**第五，能力灵魂：从”契约与证据”换成”决策与理由（rationale）“**。工程师的立身之本是契约与证据——需求怎么说、验证怎么证；架构师的立身之本是决策与理由——为什么这么切、被否掉的备选死在哪。这也是为什么架构师必须写 ADR：理由不写下来，评审就无从审起（第 12 章六问）。

**第六，判断锚点从”规定需求”换成了”目的”**。工程师的判断有明确的参照——规定需求，对就是对、错就是错，验证能证伪；架构师的判断锚在目的上——他每天要回答的核心问题是”什么才是 fundamental 的”，而 fundamental 的判据，是对系统在其环境中的目的而言必要且充分（第 10 章）。

操作化为不变量判据：在演化与重实现中必须守住、否则系统就沦为另一个系统的那些概念与属性。判别问句就一句：**“这个东西变了，系统还能不能达成它的目的?”**（第 11 章）选错判据，架构要么漏掉根本（系统走着走着就不是它了），要么锁死细节（演化窒息）；三重过滤怎么防误判，回第 10 章。

**这个锚不是悬空的：立住它的是《系统目的或任务理解确认单》（§18.3.7）——原文引用、复述理解、业务方签字，纯理解、零架构；签字之前，架构师的一切产出都没有判据锚点。**

这六条更锋利的差异，第二幕磨角色边界时会一并磨开——决策域、认知对象、验证方式、时间尺度、能力灵魂、判断锚点的差别，正是架构师和那些”像但不一样”的角色分道扬镳的根源。

### 18.2.9 整条链：architecting 七动词链

十项能力清单回答”架构师这个物种长什么样”；还有一个纵切面的答案——架构师每天在做什么。ISO/IEC/IEEE 42010:2022 §3.1 给架构师的工作（architecting，本书译”架构活动”）下了定义：

> **在实体的整个生存周期内（本书语境里，这个实体就是系统），构思、定义、表达、记录、沟通、认证恰当实现、维护与改进架构的过程。**

七个动词，一条链：

| 动词 | 干什么 | 对应能力 |
| --- | --- | --- |
| 构思（conceiving） | 从关注点出发生成候选架构概念 | 关注点管理 |
| 定义（defining） | 画边界、做选择，收敛为确定方案 | 架构决策 + 质量权衡 |
| 表达（expressing） | 把架构翻译成一组各有其主的视图 | 抽象建模 |
| 记录（documenting） | 把视图与决策固化为架构描述（AD） | 抽象建模 |
| 沟通（communicating） | 让各利益相关方理解并达成共识 | 技术领导 |
| 认证恰当实现（certifying） | 验证实现与架构一致 | 技术领导 |
| 维护与改进（maintaining and improving） | 对抗架构腐化（drift / erosion） | 演化对齐 |

（certifying 一行的”验证实现与架构一致”，按第 8 章判例①族读：这是对照架构分配的符合性认定——本质是验证，评审/走查是它的证据手段。）

七动词**贯穿整个生存周期反复发生**，不是串行的七步。两条注意：

第一，这本书讲的”架构师”，是从事架构活动的人；42010 只给活动立词条，不给头衔立词条——**标准认的是活，不认职位**。所以”被任命为架构师”只是拿到了入场券，能不能在七动词上每天站住，才是尺子。

第二，这张动词链和前面那张十项能力清单，是**两套切法**：十项清单是”这个物种长什么样”的体检切法，动词链是”每天在做什么”的工序切法。本书凝练一句话：**动词链是能力的母词——每一项能力，都能在链上找到它管的那个动作**。两套切法并行，不互相吞并。

（第 11 章用的是这条链的前段：构思→定义→表达；第 14 章用的是后两个动词：认证=符合性监控、维护改进=演化治理。整条链，是这一章第一次整体摆出来。）

把这条链量到两个人的判词上，缺口就具体了：**水忠**做到了构思、定义、表达、认证，唯独缺**记录与沟通留痕**——老规矩换个人做不出来，正是缺⑥可复现意识；**万辉**画了四张图（表达），构思、定义会做、没长在自己身上——第 11 章的”本质＋管控原则＋落地”是接住点醒学会的；认证（架构符合性把关）更是他的功课——缺的正是底座①④⑥。

## 案例进展①｜万辉：有头衔，缺①④⑥（判词初挂）

十项清单第一次落到人身上，落在恒信的万辉身上——先量那个有”架构师”头衔的人。

评优会散场，话题还没凉。小白也凑过来，她指着那面挂满四张图的墙说：“图我们天天看，可真做接口时，碰上’数据放哪、归谁管’，图上找不着答案。”

运维的玉杰也翻出一份变更记录：“上回发布后我照例翻变更，‘改了什么’有，‘为什么改’没有——就一句’按上级要求’。他改完什么都不写，我们上线后翻变更，翻不到当时为什么。”

黄程也接话：“我们业务不看画得漂不漂亮，只看合同在审批链上别卡壳。”他顿了顿——当初把”改个编号”当小需求随口一提的，正是自己；三个月连环三爆。那个”编号”，是每份合同的唯一身份——台账、审计、签章，生存周期都靠它认人。“业务规则的每一行，都得有人想清楚它进系统哪块地。”

恒信的评优会之后，万辉把补课补到更深处。第 10 章他学会让四个切面对得上，可图下面依然没有”决定”的字。他把那四张图收起来，换成一份清单：合同审批系统的不可还原的涵义、管控原则（第 10 章水忠那份”老规矩”的恒信版）。画图容易，写决定难——他写到”合同编号的不可逆性”那一行，才明白徐漾当年点他的那句话是什么意思。

他补的第二件事，是给”年度架构师”这个头衔写了份自我体检：按十项能力清单，一项一项打勾。

科学素养——接近缺①（长期不碰实现，理论开始悬空）；方法论——有；实践——有。职业伦理——有一次”为进度悄悄牺牲可靠性”的记录（第 2 章），这让他知道④的账没还清；从这次起，每次取舍他先把代价摆到桌面上、让利益相关方知情——账开始还了；工程判断——正在长；可复现意识——以前没有，现在开始写决策记录（第 12 章六问）……

体检单上没打满勾。**判词在这里初挂：万辉有头衔，缺①④⑥。**但这张单子本身，就是他成为架构师的第一步——**他终于在用尺子量自己了。**

把粒度尺摆上来：万辉的名字印在”系统架构师”这一格，背的却是”专责”里最需要底座的那一份——架构正确性。专责可以委托、可以追责，但**架不住底座缺一块**——这第 18 章的分寸，他正在补。

---

## 18.3 第二幕 架构师不是什么：角色边界与分工（磨边界弧）

> 这一幕把”架构师”和它容易混的东西分开——六个相邻角色、四宗死法，一条条磨过去，每个都靠反例从他身上剥下来。

### 18.3.1 不是”更高级的工程师”

回到章首那场评优会。开发说”架构师就是更高级的工程师——升职了嘛”，这句话错在哪？

**错在把职级当成了物种**。职级是组织给你的一个位置（你管多少人、拿多少钱）；物种是能力决定的（你会做什么、能做什么）。第 9 章立过一句话：头衔不定义工程师。这句话在这里有第二层：**职级也不定义架构师**。

两者不是职级高低，是**两种决策域**——架构设计管系统级结构、组件划分、接口契约（架构师的主战场），详细设计管模块内部、算法、实现与验证（工程师的主战场）。

**同一个系统，工程师竖着做深，架构师横着铺广**。

两种决策域，可以凝成一句边界判据：**凡是”约束别人”的决策，归架构师；凡是”只管自己”的决策，归工程师**。架构师定边界、定

接口、立**管控原则**——那是约束着别人代码的决策；工程师在自己的模块内自治——内部结构、内部依赖、实现策略自己定，哪怕那已经是”模块内部架构”（§9.3.6 说的”模块级迷你架构师”）。

谁在越界，一眼可辨：**约束别人的，是架构的活；只管自己的，是工程的活**。反过来，两个角色互不替代：没有架构师，工程师各自为政（Cowboy Coding）；没有工程师，架构落不了地。

一个组织可以把任何人任命为”架构师”，就像可以把任何人印上”工程师”头衔。但”是否在做架构”、“是否具备十项能力”，不是任命决定的，是可检验的。更高级的工程师——如果”高级”指的是职级——可能什么都不做，只是工资高一点；架构师如果什么都不做，那他只是占着一个名头。

### 18.3.2 不是系统工程师：专责与终责的分野

这是全书最容易让人分不清的一对——因为架构师”首先是一个工程师”，而系统工程师又”实践着整个系统工程”。分不清的人，会把架构师当成”系统工程师的升级版”——于是章首那句话就来了：“架构师是什么都管的人，出了事都找他。”

不是。**架构师是工程师角色体系中的一个专门角色，系统工程师是另一个专门角色——两个角色，两本账**。

第 9 章那把粒度尺，在这里落定：系统工程师对**系统成败**负**终责**（accountability）——整个系统的账，出了事他兜底，这账**不可委托**；架构师对**架构的正确性**负**专责**（responsibility）——即对该实体在其环境中的基本概念或属性、以及管住该实体及其相关生存周期过程怎么实现、怎么演化的管控原则**是否立得住、是否被守住**负责；可以

委托，也可以追责。一句话：**架构师搞砸了，追责追到架构师；对外向客户、向监管扛雷的，是系统工程师**。

所以”把整条链背在身上”，要读准：**背在身上，是看在眼里，不是担在肩上**。架构师把整条链看在眼里——关心本质与管控原则、以及它们在实体构成上的落点，从出生到退役；但”系统成败”那本账，记在系统工程师头上。**看得见整条链，和兜底整条链，是两回事**。

两者的关系，是一个**委托—协商—验收**的回路，不是指挥—执行：

- **委托**：系统工程师把”架构正确性”委托给架构师，架构师以架构描述回馈，系统工程师评审背书；
- **协商**：架构能改变需求——选了一个架构方案，等于给系统加了一批它本来没有的要求（派生需求），要回流更新规定需求。所以两者是持续的谈判对手，目标不是”完美的正确解”，是各方都接受的”协商理解”（negotiated understanding）；
- **验收**：拍板权在系统工程师，理由权（rationale）在架构师——**你提方案 + 理由，我拍板 + 兜底**。架构师提方案必须带理由（42010），系统工程师做最终取舍（成本、进度、需求优先级，常跨出架构域）。

这一回路在模块级还有一条：**系统架构师把模块架构委托给模块架构师**（通常就是模块工程师自己/团队，§9.3.6）——系统架构师定边界、把验收，模块架构师定义模块内部的结构、内接口与准则。

两者还有常态张力：架构师偏向架构纯洁性与长远演化，系统工程师偏向交付可行与当前约束。调和机制不是谁压过谁，是**把理由摆出来对质**——系统级评审（TR）由系统工程师主持，架构师是材料主

笔和陈述方。这个分工，就是把”架构的远见”和”系统的责任”焊在一起的那道缝。

这一条，直接收束失败四式里的第二宗：**“什么都管”的架构师，就是把专责错当成了终责**——架构师的专责有边界，越过边界去背系统成败的账，既僭越，又失职。

### 18.3.3 不是技术负责人

技术负责人（tech lead）管的是**人**：代码评审、任务分配、进度、绩效、团队成长。架构师管的是**本质与管控原则**：这个实体在概念与属性上是什么、哪些边界/所有权/演化规矩必须守住。

两者可能重叠（一个人同时是技术负责人和架构师），但对象不同。技术负责人问”这个迭代能不能按时交付”；架构师问”这套本质与管控原则，能不能让未来的迭代不散架”。前者在时间尺度上工作，后者在**架构的尺度**上工作。

**判断一个人是技术负责人还是架构师，看他在压力下护住的是什么**：护住交付，是技术负责人；护住结构，是架构师。第 2 章那场事故里，万辉团队”为进度悄悄牺牲可靠性”——那是技术负责人的压力，压倒了架构师的职责。

技术负责人靠职位领导，架构师没有这个靠山——他要在没有正式职权（或职权不足）的情况下，凭专业可信度、清晰的愿景与有效沟通，让团队在技术方向上**自愿地、持续地**追随。

这是技术领导力，INCOSE 把技术领导力列为系统工程师的核心能力之一，注意它在”能力”清单里，不在”职位”清单里。**管理权可以任命，领导力不能任命**——架构从文档变成现实的桥，是团队的

自愿执行；可信度是唯一货币，积累慢、消耗快，只能靠”让别人因为你的判断而受益过一次”挣来。

### 18.3.4 不是项目经理

项目经理管的是**进度**：里程碑、资源、风险登记、利益相关方沟通。架构师管的是**架构**：本质（基本概念或属性）与管控原则（约束实现与演化）——模块边界、接口契约、演化方向是它的落点。

项目经理说”这个需求排在下个迭代”；架构师说”这个需求会破坏 M3 的追溯链路，必须先过完整性评审”。项目经理关心事情能不能按时做完；架构师关心做出来的东西能不能持续改。两者冲突时，常常是”进度 vs 架构”——这正是第 2 章价值层失守的现场。

**一句话：项目经理对日历负责，架构师对演化负责。**

### 18.3.5 不是领域专家

领域专家懂**业务**：合同的审批规则、疫苗冷链的法规要求、结算的流程。架构师懂**系统**：这些规则怎么被建模、这些流程的数据怎么流动、系统怎么响应变化。

领域专家说”法务要求合同必须有三个审批节点”；架构师说”这三个节点在系统里怎么表示、数据归谁、改起来要动哪里”。业务规则是领域专家的世界，架构是架构师的工地。**好的架构师不抢领域专家的活——他让领域专家的规则在系统里待得住、改得动。**

### 18.3.6 不是画图匠

这是最容易被混淆的一对，因为两者都”画图”。区别只有一条：**图是决定的表达，还是决定的替代。**

画图匠画的是”系统长什么样”——一张漂亮的分层图、四张对不上的视图。架构师画的（或者不画的）是”系统最重要的决定是什么”——数据所有权归谁、接口怎么切、哪些决策要提前做。第 10 章那句”架构≠一张框图”，在这里落成一个角色判据：

> **如果删掉你所有的图，你的决定还在，你是架构师；如果删掉你的图，你什么也没剩下，你是画图匠。**

水忠的”老规矩”不是图，是一份清单（M3 不可还原的涵义、管控原则）——那才是架构。万辉的四张图一度挂满一面墙，但图的下面，没有任何关于”决定”的字——那四张图，是画图匠的作品，直到徐漾点醒他。

### 18.3.7 失败四式：怎么死掉

第二幕的磨边弧，最后磨到”死法”上。角色边界说的是”和谁像但不一样”，失败四式说的是”怎么从架构师的位置上掉下来”——每一种，都能在十项能力清单里找到缺的那一项。

**第一宗死法：画图匠——只有视图，没有判断。**症状是图越来越多、越来越漂亮，但图下面没有任何关于”决定”的东西。画图匠死掉的方式不是被辞退，是**团队发现图不能用来做决定**——图说系统长这样，但遇到”数据放哪”没人能答。于是大家绕过图，各做各的，架构描述成了挂图。**画图匠死于自己没有判断（缺⑤工程判断）。**

**第二宗死法：什么都管——把约束管成枷锁。**和画图匠相反，这类架构师把”管”发挥到极致——每个接口、每行代码、每个表都要

过他。第 10 章说过”管得最准，不是管得最多”。什么都管为什么危险？因为它看起来非常负责——§18.3.2 那条”专责不是终责”的分野，在这里落下最重的一刀。

但它有两条后果：第一，团队失去了自己判断的能力，所有决定都等他（他成了瓶颈）；第二，他把精力花在大量低价值决策上，真正重要的决策反而没有精力——**第 10 章那句话的反面：对不重要的事情疯狂决策，对重要的事情视而不见（缺⑦识别后果严重的决策）**。再加上背了本不该背的账（把专责当终责），两头都失守。

**第三宗死法：不决策——把决定留给默认**。症状不是”什么都不管”，而是**把决定留给默认**——不选就是选了默认，默认往往是最保守的、最省事的、最像”大家都这么做”的。第 12 章讲过”没做架构决策=伪选项”：架构决策不可避免，不是做不做，是早晚。

不做决策，系统也会被默认选项塑造——只是你在事后才认识它，而那时它已经不可逆了。**不决策的架构师，把架构决策的资格交给了运气和时间（缺⑩判断不可逆性）**。

**第四宗死法：远离代码——判断失去锚点**。这类架构师升上去了，不再写代码，不再看实现，只画蓝图、开评审会。第 9 章讲可复现意识（⑥）：判断要能”换个人也接得住”。远离代码的架构师，判断没有锚点——他不知道系统真实的样子，只知道自己想象中的样子。

**远离代码的架构师，判断会慢慢悬浮**：他说”这里应该微服务化”，但他不知道现在的服务边界是什么；他说”数据应该统一”，但他不知道数据现在在谁手里。

他的理论还在，但他的理论已经不落在任何可实现的东西上——第 10 章那句话：没有理论支撑的架构像没有原理的工程；反过来说，**不落地的理论，像没有实物的建筑图（缺③实践能力）**。

四宗死法，四种姿态：**画图匠死于没有判断，什么都管死于管太多，不决策死于不管，远离代码死于看不见**。每一种，都能在十项能力清单里找到缺的那一项。

### 不是目的的定义者：一条界线与四个职责

角色边界磨到最后，还有一道最上游的界，值得单独划——**架构师还不是”目的”的定义者**。系统目的与任务的定义权和问责，在业务方（赞助人/产品管理；15288 业务或任务分析流程）——架构师对”**架构忠于目的**”负责，不对目的本身负责。用 RACI 说：目的定义，A 在业务方、架构师是 C；架构定义，A/R 在架构师、业务方是 C。

架构师对目的有四个职责，不负责定义、但绝不被动：

| 职责 | 干什么 |
|---|---|
| **理解** | 把目的当作 fundamental 判据吃透（架构工作的输入总开关） |
| **澄清** | 目的模糊时向上游回溯，推动业务方表达到可判定粒度 |
| **质疑** | 用可行性、成本、一致性检验目的，有质询权、没有改写权 |
| **守护** | 全程监控目的漂移，目的变更时触发架构重审（第 14 章） |

**角色分离 ≠ 人员分离**：小团队同一人可兼戴”产品负责人”和”架构师”两顶帽子，但做每个决定时要意识到自己戴的是哪顶——目的和架构在脑子里互相即兴迁就，是 Cowboy Coding 式架构的常见病因。“理解”职责的产品数据，是一份《系统目的或任务理解确认单》（§18.4.6 表首行）——纯理解、零架构，业务方签字生效。

恒信这份单子上的目的，是黄程签的字——“合同审批链不断、每份合同全生命周期可追溯”，一句业务方原话；万辉的复述理解只准在”理解”上做文章，不碰”定义”。

四职责收成一句话：**架构师是系统目的的”理解与守护者”**——理解它（吃透判据）、守护它（拦漂移），不定义它；目的定义权在业务方。

这个目的不是孤立的一点，它沿一条链展开：**目的（为什么存在）→ 目标（达成什么可度量状态，ISO 9000 objective）→ 需求（必须满足什么可验证条款）**，操作目标/MOE 就是目的在运行层的可度量投影。

这个目的属于**组织层**——它回答”组织为什么研制它”（战略/经营层的事）；区别于**用户的使用目的**——用户拿系统干什么，那是第 8 章确认环节对照的预期用途。一个是组织的存在理由，一个是用户的使用场景，别混。

架构师守目的，是守住这条链的顶端——评审、确认沿链向上追溯，最终都追到目的（第 8 章那句”评审对目标、验证对需求、确认对用途”，正是这条链上的三次站岗）。

守护的根据就在这里：**人造系统的同一性由目的构成**（同一性目的论·忒修斯之船，见附录 A T-144）——守住目的，就是守住”系统还是不是它”；目的漂移，是架构失守的第一关口。

## 案例进展②｜水忠：没头衔，缺②⑥（判词推进）

磨完边，再量冰链的水忠——那个从头到尾没有”架构师”头衔、却一直在做架构工作的人。

他的名片上印着”嵌入式硬件工程师”——第 9 章说过了，那是**模块工程师**的岗位。但他做的，一直是架构师的活：他写下 M3 的老规矩（不可还原的涵义、管控原则），他拦下董勐的直连接口，他把断链追溯的完整性守到死。第 10 章之后，团队开始叫他”架构师”——先是玩笑，后来是习惯。

玩笑是这么起的。文正先开口：“评审会我主持，可一到’数据归谁、接口怎么切’，话筒就该递给水忠——他答得比需求清单还准。”李旭接一句：“他写明白的，我能验证；写不明白的，我没法测——这套老规矩，换个人接得住吗?”逸人更直接：“他每动一次数据口径，产线的批次追溯就得重对一遍——我就盼他把’老规矩’写清楚。”

可水忠自己清楚，按十项能力清单，他缺两项：②方法论素养（他是手艺的极致，第 9 章说过他缺②⑥），⑥可复现意识（他的判断换个人做不出来）。**判词从初挂推进到这里：水忠没头衔，缺②⑥。**

把粒度尺摆上来：水忠在**模块工程师**的岗位上，做**系统架构师**的活——他的”老规矩”就是一份没写完的架构描述（缺了 ADR 那样的决策理由），他的判断换个人做不出来（缺⑥）。**他是一只脚跨进了架构师的地盘，另一只脚还站在手艺里。** 团队叫他架构师，是抬举；他自己承认缺项，是清醒。**判词在这里推进到”为什么缺”：他不是不会做，是做的能力不可复制——长成的是”老规矩”，不是”方法”。**

散会时李旭那句”换个人接得住吗”，水忠没应声。当晚他把老规矩摊开，在每一条后面补了一栏”为什么”——规矩写的是”怎么改”，还差一句”为什么这么定”，换个人才接得住。

## 18.4 第三幕 架构师往哪去：成长、产品数据、判词落定（运用）

> 这一幕把清单用起来——怎么长成（抽象跃迁）、怎么被检验（开发出高质量的产品数据）、再往上（从系统到组织），最后给两条线的人落判词。

### 18.4.1 怎么长成：抽象跃迁，从系统到架构

架构师不是空降的物种，绝大多数是从工程师长出来的（三种来路见§18.4.3）。成长的本质，不是”管的人变多”，也不是”更高级”——是**换一条专精轨道**：从对”系统成败”负终责，转向对”**架构的正确性**”负专责。第 9 章讲的那条路叫**拓宽**（模块 → 系统，把视野拉宽）；架构师这一跳，叫**抽象跃迁**（系统 → 架构，从管流程到**定本质、立原则**）。

核心问题换了：系统工程师问”系统能不能成？“；架构师问”系统为什么长这样？它凭什么能演化？“——这就是为什么 15288 给架构定义流程 purpose 的第一个词是 **generate（生成）**：架构师要先**创造**候选架构，再选择，再表达；系统工程师通常不需要”创造”，他要”整合”。

**决策对象也从”事实”（方案行不行、预算够不够）变成了”本质”（什么结构能同时满足互相冲突的关注点，且未来三年还能演化）。**

判断的依据，在这一跳里再一次升级：

· 开发问：“这段代码怎么写才清晰？”

· 模块设计问：“这个模块怎么切，依赖才不绕?”

· 系统设计问：“整棵设计树，枝干和叶子怎么长?（第 6 章）”

· 架构决策问：“哪些决策，改错的代价大到不能等?（第 12 章）”

判断的依据从”代码”升级到”系统”，标志是你开始问第 10 章那三个问题：这个系统靠什么成功？最大压力在哪？做错什么最致命？——而不是问”这段代码有没有 bug”。**你关心的问题换了，你的物种就变了**。

升级还有第二个维度：时间。工程师对”当下交付”负责，架构师对”系统生存周期”负责——从出生到退役。一句话概括这条链：**每上升一级，空间维度与时间维度同时扩张**——工程师管元素、管当下；架构师管本质与管控原则、管多年演进。

所以架构师做决定时要多问一层”这会不会让系统未来的路变窄”，这不是拖延，是把时间尺度收进来。

抽象跃迁有几个低成本的入口：

· 从”评审架构”到”被评审”——先以评审员身份坐在架构评审里，反复问”为什么选 A 不选 B”，直到能替架构师辩护；

· 动手写 **ADR（架构决策记录）**——从记录别人的 rationale 开始；

· 主责一个**子系统的架构**——在系统架构师划好的边界内，独立完成”生成 → 选择 → 表达”三件事。

还可以：学 42010 那条概念链——利益相关方 → 关注点 → 架构视角 → 架构视图，再到模型 → 理由；跨版本看演化——跟住一个系统从 V1 到 V3，看哪里被迫改、哪里守住没改。**“没改的地方”，就是当年架构决策的 payoff**。

最后一句话，是架构师每天想的问题：**“如果需求翻一倍，这个架构还成立吗?”**

还要立一个分寸：**不是所有系统工程师都会（或该）成为架构师。这是分叉，不是阶梯。**管理线（技术经理 → 项目总监）与架构线（系统架构师 → 首席架构师）两条线都正当，只是方向正交——第 9 章说过”拓宽不是升职”，这里同样：**抽象跃迁不是升职，是换一条轨道。**

升不上去才走架构线，是把分叉当成了阶梯——这两件事的差别，就是这一章的厚度。

### 18.4.2 被任命 ≠ 成为

章首的万辉被任命为”年度架构师”。任命给了他一个名头，但名头不给他能力。**被任命只是拿到了一张牌桌的入场券，能不能在牌桌上坐稳，靠的是十项能力清单。**

这听起来残酷，其实公平：能力是可以长的，名头只是时间的快照。万辉被点醒后开始补底座——写决策记录（第 12 章六问）、识别编号规则的不可逆性（第 12 章）、把四张图从”四个系统”一路改成”决定清单”（第 10 章起）。他补的不是”画图技术”，是判断。**成为架构师的过程，就是能力清单一项一项亮起来的过程。**

### 18.4.3 三种来路，三种功课

- **从开发长出来**（最常见）：底座六项往往有一半已经在身（实践、科学素养），缺的是系统级的判断（⑦⑧⑩）。功课：把小团队的代码判断，练成对整棵树的判断。

- **空降**（架构师带团队进来）：往往有理论、有判断，缺的是对这个系统的实践（③）和可复现（⑥）。功课：先落地，再谈蓝图——否则你的判断悬空，换个人接不住。
- **挂名**（组织设了个岗位，人没变）：只有头衔，没有能力。功课：要么真的开始补清单，要么承认这只是个名头——挂名的架构师，是组织在用名字骗自己。

三种来路没有高低，只有功课不同。**判断一个人走在哪条路上，看他在补清单的哪一项**。

### 18.4.4 成长的标志：第一次”说不”

架构师第一次真正长成，往往不是学会画一张大图，而是学会**对一件”做了也没人反对”的事说不**——而那个”不”的第一个对象，常常是自己的冲动。

董勐想给客户开直连接口，水忠用引导提问点醒他后（三层判断的框架见§12.4.5），他自己把方案划掉、改走”受控导出”——这是水忠的”说不”，也成了董勐的”说不”。万辉想改编号规则，徐漾用”改字段前三问”点醒他后，他自己认下了——这是万辉的”说不”。

两次”说不”的共同点：**拦下的不是”错的”，是”没想清楚不可逆性的”**。判断不可逆性（⑩），是成长路上最后一盏亮起来的灯——因为它需要前面所有能力都到位，才接得住。

### 18.4.5 再往上：从系统到组织

架构师是这本书的终点，但架构师这个物种的刻度，没有停在系统上。

再往上是什么？**不是更深的架构，是关注维度的扩张**。§18.4.1 说过一句话：每上升一级，空间维度与时间维度同时扩张——那说的

是从工程师到架构师，是第一次扩张。把它再往上推：工程师的”用户”是组件，架构师的”用户”是系统，再往上是系统之系统，最后是整个组织。**每一次上升，你服务的对象变了一级，你的物种也跟着变一次**。

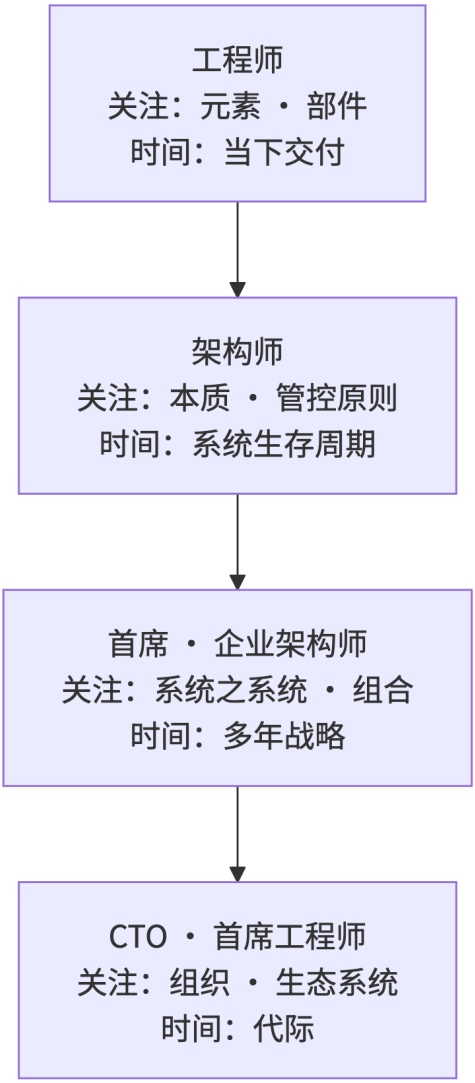



图 20　图 18-1

架构师之后，路分四条：

| 路径 | 角色 | 扩张方向 | 核心关切 | 权威锚点 |
| --- | --- | --- | --- | --- |
| 深度 | 首席架构师 | 从”做架构”到”建架构体系” | 多系统治理、技术标准、评审制度设计者 | 组织实践 |
| 宽度 | 企业架构师 | 从”一个系统”到”系统之系统” | 业务/数据/应用/技术四域整体一致 | TOGAF |
| 组织 | CTO / 技术总监 | 从”技术问题”到”业务+组织”融合 | 技术投资组合、组织技术能力 | 管理学领域 |
| 权威 | 首席工程师 | 从”管技术”到”技术裁决” | 跨学科技术完整性与最高裁决权 | NASA 系统工程手册 |

四条路没有高低，只有棋盘大小不同。注意”深度”那条——首席架构师——是架构线的继续（§18.4.1 说的分叉：管理线 vs 架构线，架构线在这里延续）。它”建架构体系”建的，正是 §18.4.6 那套组织级的治理制度。

架构师在这本书里用十项能力量自己；再往上，量尺要换——42010:2022 把被架对象从”系统”扩成实体（EoI），组织、系统之系统都可被架；本书的十项量到”系统”这一格，被架对象变成一整个组织时，“业务/数据/应用/技术”四个关注域，正是 TOGAF 的惯例词汇（第 10 章 §10.4.4）。

系统之系统的杀手特征是：**每个子系统都有自己的 owner 和生存周期，没有全局 owner**。架构师缝的是组件之间的缝，再往上缝的，是子系统之间的缝、部门之间的缝——第 9 章那句”系统工程管缝”（§9.2.7），缝得越大，棋盘越大。

质变发生在 CTO 一级：

- 关切管理对象从利益相关方变成董事会；
- 技术领导力（§18.3.3 节，领导力不能任命）的对象从团队变成管理层；
- 演化对齐从系统生存周期变成企业战略周期。

**你不再解决技术问题，你解决”组织用技术创造价值”的问题**。

演化对齐变成了企业战略周期，怎么对齐？把镜头拉高一层，战略层同样是一条判据传导链（§12.4.10）：**业务战略 → 产品战略 → 架构战略 → 技术实现**——每一层的目的，是下一层判定 fundamental 的标尺；架构战略里的每个根本决策，都能回答”它支撑产品战略/业务战略的哪一条”。

断链处就是过度工程与欠工程在战略层的发病点：**过度工程**是架构领先战略（为全球分布式提前建能力，市场只在本地）；**欠工程**是架构落后战略（单体扛不住多租户，重构拖垮窗口）。对齐是双向的——战略向下传导，技术向上反约束战略。架构师再往上，手里那把 fundamental 尺子没换，只是棋盘从系统变成了组织。

这里有一个问题必须先回答，否则读者会说这章自己打脸：§18.3.1 节刚说”架构师不是更高级的工程师”，这里怎么又说架构师再往上？

不矛盾，反而是同一个逻辑的第二次上演。**“职级也不定义架构师”这句话，在 CTO 这里依然成立：CTO 这个头衔，同样不定义能力**。“从系统到组织”和”从元素到系统”是同一种升级——工具不变（判断、权衡、沟通、担责），棋盘变大。

一个人升到 CTO，如果还在用工程师的尺子量自己，他只是换了个更大的头衔，没有换物种；反过来，一个架构师在”系统”这一格停一辈子，只要把十项能力一项项背在身上，他就是这本书定义的架构师。**头衔一路在换，背在身上的东西没有变**。

词源为这条链做着注脚：architect = archi-（首）+ tekton（工匠），“工匠之首”；chief = 拉丁 caput（头）。链的尽头，是从”工匠之首”到”组织之头”。

最后回到案例线。水忠的”老规矩”如果再往上走一步，是把一个人的手艺变成一家公司的制度——那是”深度”路径的入口；恒信的万辉若要再往上，缺的不是画图，是把技术决策对齐到恒信的战略周期——那是”组织”路径的入口。

两个人离 CTO 都很远，但”再往上”的刻度，已经摆在那张十项能力清单的旁边——**先成为架构师，再谈棋盘变大**。

### 18.4.6 能力的最终体现：开发出高质量的产品数据

十项能力抽象在一个人身上，不可直接检验——面试会看走眼，领导会凭印象打分。架构师能力的最终体现，是能否设计开发出本角色负责的产品数据。第 9 章给工程师立过这句话，架构师也一样——只是他的产品数据在**系统级（架构级）**，是整棵树的枝干和枝梢。

架构师的工作，就是做系统级的架构设计（第 11 章）：**定本质**（这个实体在其环境中的基本概念或属性）、**立管控原则**（约束实现与演化）——让每个部件的设计有地方落、落得对。那份系统级的产品数据，就是架构师能力落到地面上的形状。

所以看一个架构师，不看画的图多不多，只看他能不能开发出高质量的产品数据。**能力再强，开发不出高质量的产品数据，这个架构师也不成立**——高质量不是图多，是系统级产品数据站得住：**本质立得住、原则守得住**、接口可承接、决策如实、换个人能接住。

第 6 章种过那棵树：枝干是框架定义（管结构、承接关系），叶子是详细定义（管内容、可验证）。注意框概分离是**每一份**方案文档内部的普遍结构（详概分离原则），在整条开发链上层层重复——它不是架构师和工程师的分工。

分工在第 9 章那张责任矩阵：**架构师主责”系统架构”，系统工程师主责”系统级”其余各项（主责兼终责），模块工程师主责”模块级”（§9.4.6）**。架构师的产品数据，是一份以架构描述为核心的文档族（下表对应后记系统架构师清单：确认单 → 基本概念和基本属性清单 → 架构描述 → 决定清单 → 管控原则 → 系统接口契约 → 系统级验证方案，本表展开为十三项）。

**注意：操作目标清单不在此列**——它是系统工程师需求侧对目的的操作化（第 4 章）：操作目标 = 目的在运行环境中的可度量投影（MOE 层），是运行层确认的显性判分线。架构师对系统操作目标只

质询、当判据用，不编制——质询的正是”目的→操作目标”这一层的可度量投影。

| 架构师的产品数据 | 内容 |
| --- | --- |
| 系统目的或任务理解确认单（入场券） | 目的或任务原文引用（业务方原话，只引不改）+ 架构师复述理解 + 业务方确认签字——纯理解、零架构；架构工作的入场券，签字之前后续一切产出都没有判据锚点 |
| 系统基本概念和基本属性清单 | fundamental 成员资格登记表——每条 = 定义 + 对目的的必要性论证 + 到目的/需求的追溯链；目的判据的裁决产物、评审第一核对单（第11章） |
| 架构描述（AD） | 系统架构总纲：架构视图 / 架构视角 / 模型——让每个利益相关方各看各的那一面 |
| 管控原则 | 系统”守什么规矩”：管控原则（governing）约束实现（怎么造对）与演化（怎么变不散），守的是不可还原的那几条 |
| 架构决策记录（ADR） | 为什么这么分——trade-off 的留痕与理由（rationale）；=后记”决定清单” |
| 接口契约 / 接口控制文档（ICD） | 组件之间的”缝”——数据口径、时序依赖、调用契约（模块间协作的宪法） |
| 框架定义 | 系统结构、边界、组件划分、承接关系 |
| 派生需求清单 | 选了一个架构方案，回流给需求流程的新要求 |

表格上方的两张单子是**入场券**：确认单确认”目的”这个判据（§11.4.3——判据侧首类输入「目的与任务」的确认形态）；清单登记”哪些概念/属性配得上基本”（目的判据的裁决产物）。§18.2.9 说”被任命只是入场券”——那是**能力的入场券**（头衔不定义能力）；这两张单子是**判据的入场券**——签字之前，架构师的一切产出都没有判据锚点。两者正交：**判词量人，入场券量活**。

**文档是载体，决策与理由（rationale）才是灵魂**。架构师的产出不是一份”系统架构”文档，是一族文档，加上贯穿生存周期的决策链。

这份文档族有三个特殊性质，呼应第 9 章那句”系统工程管缝”（§9.2.7）：

- **它是”缝”的数据**——架构师不造部件，造的是让部件协同的规则；
- **它是”本质与演化”的数据**——因为要守住本质与管控原则，所以天然含演化路径、技术债台账；
- **它是”约束”的数据**——接口契约、技术选型，是工程师产品数据的输入需求。

演化类的数据，有两个入口最容易漏。**画路线图之前，先识别演化驱动力**——业务、技术、组织、法规四类变化源；不知道会变成什么，也要知道哪里可能变（§18.4.5 战略对齐链的另一头：断链处就是过度工程与欠工程的发病点）。

**技术债台账要记三样**：借的原因、当前的利息、还款计划。健康组织不是零技术债，是每笔债都记得明明白白、有到期日——无台账的债才是灾难。定期评审债务组合，像财务看资产负债表一样：哪些

债值得借（换来市场窗口）、哪些债必须还（利息太高）——这份定期评审，是第 14 章说的传感装置之一（传感是例行的，裁决不是）。

还有一个职责，最容易被漏掉：**架构符合性把关**。42010 给 architecting 的定义里有半句——“认证架构的恰当实现”（certifying proper implementation）。架构师不只画架构，还要检查：模块详细定义符不符合架构分配？技术方案有没有违背架构原则？**这份审查职责，第 9 章那张”编制/批准”的责任矩阵里没有一格属于它——矩阵画的是谁编制、谁批准；架构符合性把关是矩阵没画出来的那层：检查别人的编制，符不符合架构。**

**用一句话收：系统工程师管文档的载体（签发的那个文件），架构师管文档的内容（架构决策的来源）。**

架构师的产品数据，还有第二层，也是最容易被漏掉的一层：**组织级的制度**。当一个架构师管的不只是一个系统，而是所有系统的架构怎么保持一致、怎么演化、怎么被遵守，他就在做**架构治理**了——第 14 章讲过治理=掌舵，这里把掌舵落到架构域。

架构治理的分工，和治理章那把尺一样：**方向权（目标、组织政策、战略）属组织层，内容权（原则、架构政策、指令、符合性判断、决策依据）属架构师**（ISO/IEC/IEEE 42020）——架构师是治理在架构域的执行主体。

**架构治理守护的对象，不是架构文档，是一条不变量链——「目的 → 不变量 → 根本决策 → 遵从与演化」（§14.2.6），四层各有各的问题：**架构决策对不对（fundamental 的概念与属性是否仍与目的一致）、决策有没有被执行（实现/设计是否偏离已确立的基本概念

或属性与管控原则）、变更受不受控（不变量守没守住、变体在不在允许范围）、架构资产完不完整（架构描述、架构视角库、可追溯链可不可信）。

方向权（目的的所有权在业务方）是这条链的源头，内容权（不变量与根本决策的守护）在架构师——§18.3.7 说架构师是目的的”理解与守护者”，到组织级，守护的就是这条链。

**为什么架构非治不可？三种侵蚀力量，每一种都打在链上：① 局部优化侵蚀全局根本**——每个团队按自己的 deadline 做局部最优，单看都合理，累积成架构腐化（这正是第 14 章委托必有治理在架构层的形态：全局根本与局部执行分离，偏离”安静地积累”）；

② **目的漂移使判据失效**——fundamental 是”对目的而言”的判据，目的悄悄变而架构没跟上，整个不变量集合过期、无症状（§10.2.4）；

③ **决策不可逆放大错误成本**——架构决策改动代价大，事后纠错远比事前把关贵，治理是把错误拦在便宜的时候。**不变量不会自己守住自己，目的不会自己宣告变更——这就是架构治理存在的原因。**

架构治理的产出，是一整套组织级、跨项目跨版本的产品——架构原则、政策与治理指令、架构委员会章程、治理决策记录（带理由）、合规评估报告、架构健康度指标……它们和系统级文档族一起，构成架构师产品数据的完整图景：**系统级文档族是”作品”，组织级治理产出是”制度”。**

架构治理不是架构师的额外职责。42010 给架构师工作（architecting）下定义时，本来就写着”认证架构的恰当实现、维护

与改进架构”——上一段说的”架构符合性把关”，到组织级就制度化成了治理的合规审查；“维护与改进”就是度量与演化治理。

而这里的分工要再确认一次：**治理决策权在架构委员会（架构师是技术核心），但系统级变更的最终裁决仍要系统工程师背书——治理是架构师的常设法庭，系统工程师是系统成败的最终责任人。**

**六项能力，在架构师侧的落点**

| 能力 | 架构师侧落点 |
| --- | --- |
| ① 科学素养 | 架构的**依据**——为什么这么分（容量模型、性能分析） |
| ② 方法论素养 | 架构的**方法**——架构视图/架构视角、架构描述语言、评估方法 |
| ③ 实践能力 | 架构的**落地**——架构能被实现，不是空中楼阁 |
| ④ 职业伦理 | 决策的**真实**——不掩盖技术债、如实记录取舍 |
| ⑤ 工程判断 | 架构的**取舍**——系统级权衡，核心战场 |
| ⑥ 可复现意识 | 架构的**可接住**——ADR 写得清、演化路径可传承 |

把架构师的十项能力从”人身上”挪到”纸面上”，不是为了考核他，是为了让能力可被检验——而检验的标准只有一条：**你开发出的产品数据，高不高质量**。尤其是②⑥两项，最终都露在架构师的产品数据上：方法写不写得成视图、判断换个人接不接得住。

至于增量四项，它们的产品数据落点也都在文档族里：⑦识别后果严重的决策→ADR 与架构权衡报告，⑧对系统的理论→架构描述里的原理，⑨把架构说清楚→视图，⑩判断不可逆性→演化路线图。

### 18.4.7 接口契约 = 1:1 搬入点

第 6 章立过”1:1 约束”：一片叶子只能被搬到下一棵树的一根枝梢。现在补上关键的一环——**架构师定的接口契约，就是工程师部件方案落下的那根枝梢**。

架构师定完架构（本质与管控原则），工程师的部件方案从接口契约上长出来。架构师的”输出”（接口契约）就是工程师的”输入需求”（规定需求）——所以架构师的产品数据是工程师产品数据的上游规定。**架构错，代价是传导性的：系统级错了，部件级跟着错**。

小白第一次看清这条链，是在改一个字段的时候。她一直以为接口是”大家商量出来的”，照着写就行——改一个字段，就是改一处代码；直到翻开 ICD，“合同编号”那一行写得死死的：数据口径、调用方、变更问谁，全在约定里。

她才明白：接口不是随手定的，是架构设计定下来的约定——**它连着两边的结构，不是代码的事**。而模块也有架构：定义这个模块”是什么”的那个人，不是别人，就是她自己——模块工程师做架构工作，就是模块架构师。

### 18.4.8 检验：换个人，能不能接住

回到判词。万辉的产品数据：四张图一度挂满墙，但图的下面没有”决定”的字——直到他开始写 ADR；水忠的产品数据：老规矩写在纸上，但他的判断换个人做不出来。

**判词（下一节案例进展③）和这一节，量的是同一把尺子。**

架构师的检验句，和工程师那条对称：

> **换个人，拿着你负责的架构描述和决策，能不能接住、继续演进？** 能，你的架构才成立；不能，再大的图也只是装饰。

## 案例进展③｜双线判词：两个都不是，两个都在长成（判词落定）

那么，谁是架构师？

**万辉**：有头衔，缺①④⑥——他有职位、有舞台，但判断还悬着，伦理的账还欠着。他是”被任命的架构师”在补底座。他的名片印在”系统架构师”这一格，背的却是最需要底座的那份专责——底座缺一块，增量全空转。**水忠**：没头衔，缺②⑥——他有判断、有实践、有理论，但方法论和可复现意识缺席。他是”有底座一半的架构师”在补元能力。他在模块工程师的岗位上做架构的活，长成的是”老规矩”，不是”方法”。

两个人都不是完整的架构师。但两个人都值得被叫一句”在成为架构师的路上”——因为他们都在用十项能力清单量自己，都在补自己缺的那一项。**判词在这里落定：万辉有头衔缺①④⑥，水忠无头衔缺②⑥——两个都不是，两个都在长成。**

而这也正是全书对”架构师是个怎样的物种”的最终回答：

> **架构师不是一个头衔，不是一个职级，不是一张图。它是一个方向——工程师往系统级判断走的方向；它是一张清单——十项能力一项一项亮起来的过程；它是一份专责——对**架构的正确性**负责（本质立不立得住、管控原则守不守得住），而不是对系统成败兜底**。

判词走到这里，用的正是第 1 章那把”见过→拥有”的尺子：对”架构师”这个词，章首的万辉只有”见过”——叫得出头衔、画得出图，却答不上”当选架构师意味着你现在是什么”；现在，判词把他量成了”拥有”——能对着十项清单一项一项打勾、认得出自己缺什么。**判词把”架构师”从头衔钉成了能力清单**。

---

## 认知反转

回到章首那场评优会。两年前，两间会议室为”架构”吵得不可开交；两年后，三个人为”架构师”各说各话。我们终于看清了这两场吵架是同一个病：

**人们把一个本该由能力定义的东西，交给了头衔和位置去定义。**

架构师不是更高级的工程师——那是职级的语言。架构师不是画图的人——那是工具的语言。架构师不是什么都管的人——那是权力的语言。它是**一个方向、一张清单、一条成长的路**：先是一个工程师

（六项底座），再往系统级长（四项增量），然后把整条链——从一条需求到上线运维，从系统到组织——背在身上。

而这条路最终落到一处可检验的地方：**能力再强，也要看你开发出的产品数据，高不高质量**。判断的依据换了（从代码到系统）、验证的方式换了（从测试到评审），最后都要靠”换个人，拿着你的架构描述和决策，能不能接住、继续演进”来收口——能接住，你是架构师；接不住，再大的图也只是装饰。

这一章还立下了一个分寸，也是”架构师”这个词最容易误读的地方：**背在身上，是看在眼里，不是担在肩上**。架构师把整条链看在眼里，但系统成败那本账，记在系统工程师头上——专责划得清，终责甩不掉。理解了这条，你才不会被”架构师是什么都管的人”带偏：什么都管，是把专责错当终责。

用第 1 章的话收一次束：对”架构师”这个词，你原来只是”见过”——背得出定义、叫得出头衔、认得出图。这一章把它推到”拥有”——能答全十项能力的四问（各是什么、不是什么、判据是什么、往哪去——四问法在能力清单上的适配），能用清单量别人、量自己，能对着判词说”两个都不是、两个都在长成”。**能答全四问才算拥有，这把尺子现在装进你手里了**。

再用第 1 章 §1.4.4 那四层量一次，这句话会更清楚：**工程师的地图能跑——到心智模型层，能推演、能干活；架构师的地图能判——到解释框架层，能判断什么值得相信、什么只是叙事包装**。从工程师到架构师，就是从”能用”走到”能判”。

这本书带你走的，就是这条路。

**从工程师，到架构师。**

散会时，逸人看着水忠把那本老规矩收进抽屉，说：“水忠，你做的这些，是架构师的活。”水忠没接话。两个世界各开各的会，到这里对上了同一个答案：徐漾问了三年的那个问题，不在谁嘴里，在那本子里。

---

## 小结

这一章没有新增任何概念——它把前面十六章的尺子，收拢到一个人身上：架构师首先是一个工程师（六项底座），再往系统级长出四项增量（识别后果严重的决策、对系统的理论、把架构说清楚、判断不可逆性）。

这一章还摆出了那条母词链：**架构活动（architecting）的七动词**——构思、定义、表达、记录、沟通、认证恰当实现、维护与改进。十项能力清单回答”这个物种长什么样”，七动词回答”每天在做什么”；每一项能力，都能在链上找到它管的动作。

链尾那个复合动词——维护与改进——正是架构师演化对齐能力的地基：架构师管的是结构能不能演化；至于够不够新、成没成，那是创新坐在链尾的评分席上说了算（第 11 章）。

它划清了角色边界（不是更高级的工程师、不是系统工程师、技术负责人管人、项目经理管进度、领域专家懂业务、画图匠画图——架构师定本质、立原则、懂系统、做判断），也划清了责任边界——**专责与终责**：系统工程师对系统成败负终责（accountability，不可委托），架构师对**架构的正确性**负专责（responsibility，可委托可追责）。

两者的关系是委托—协商—验收：拍板权在系统工程师，理由权（rationale）在架构师，架构能改变需求（派生需求），两者是持续的谈判对手。

它讲清了成长路径——**抽象跃迁**：从系统到架构，换一条专精轨道（从管流程到设计结构），判断的依据从”代码”升级到”系统”；分叉不是阶梯，管理线（技术经理→项目总监）与架构线（系统架构师→首席架构师）正交。被任命不等于成为。也讲清了失败模式（画图匠、什么都管、不决策、远离代码——每一种都能在十项能力清单里找到缺的那一项）。

它给两条案例线下了判词：万辉有头衔缺①④⑥、水忠无头衔缺②⑥——两个都不是完整的架构师，都在补自己缺的那一项。

最后，它把架构师的能力从”人身上”挪到”纸面上”——能力的最终体现，是能不能开发出高质量的产品数据：能力再强，开发不出来也不成立。架构师的产品数据是系统级（架构级）的文档族（架构描述、架构决策记录、接口契约/接口控制文档、框架定义、派生需求清单、架构权衡报告、系统元素分配说明、架构基线、演化路线图、系统级验证方案）。

架构师还承担架构符合性把关。分工在层级：架构师主责**架构的正确性**、系统工程师主责系统级（主责兼终责）、模块工程师主责模块级。

接口契约是 1:1 搬入点，是工程师部件方案落下的那根枝梢。换个人能不能接住，能力才成立。

这把尺子的刻度，还画到了系统之外：再往上，是关注维度从系统扩张到组织。从元素到系统、从系统到组织，是同一种升级的两次上演——工具不变，棋盘变大。那已不是这本书的行程，但刻度已经摆在那里。

到这里，正文十七章结束。书末的附录，把这十七章的尺子收进五个抽屉——术语表（A）、概念档案集（B）、误解清单（C）、参考文献总表（D）、练习答案（E）——供你随时回来取用。

如果这本书只留下一句话，那就是章首那场评优会的答案：**架构师是个怎样的物种？——从工程师到架构师。**

## 本章概念总图

架构师 —— 一个物种，不是头衔（先是一个工程师 + 增量 = 十项能力清单）

底座六项（第9章）：科学素养 / 方法论素养 / 实践能力 / 职业伦理 / 工程判断（横切）/ 可复现意识（元）

增量四项（第10、12章）：识别后果严重的决策 / 对系统的理论 / 把架构说清楚 / 判断不可逆性

十项合起来 = 架构师（元素是工程师的，本质与管控原则是架构师专责的；元素、接口与关系是设计定义的产出；生成→选择→表达，15288 §6.4.4）

六个更锋利的差异：决策域 局部可逆→全局难逆；认知对象 需求→关注点；验证方式 测试→评审；时间尺度 当下→跨版本演化；能力灵魂 契约与证据→决策与理由（rationale）；判断锚点 规定需求→目的（确认单立锚；目的→目标 → 操作目标/MOE → 需求 三层展开，评审/确认沿链上溯）

架构活动链（architecting 七动词）：构思→定义→表达→记录→沟通→认证恰当实现→维护与改进——能力的母词（与十项清单两套切法并行）

角色边界（和谁像但不一样）：

不是更高级的工程师：职级是位置，物种是能力（两种决策域；边界判据——约束别人→架构师 / 只管自己→工程师）

不是系统工程师：专责≠终责——系统成败的账在系统工程师头上（委托–协商–验收；拍板权 vs 理由权）

技术负责人管人，架构师管决定（领导力不能任命）

项目经理管进度，架构师管架构

领域专家懂业务，架构师懂系统

不是目的的定义者：目的定义权在业务方（RACI：目的定义 A 在业务方、架构师是 C）——四职责（理解/澄清/质疑/守护），有质询权无改写权；角色分离≠人员分离

画图匠画图，架构师做决定（删图后剩下什么，就是什么）

失败四式：画图匠（没判断）/ 什么都管（管太多·把专责当终责）/ 不决策（不管）/ 远离代码（看不见）

每一种，都能在十项能力清单里找到缺的那一项

成长：抽象跃迁（系统→架构，换轨道非升职）；判断依据从"代码"到"系统"；被任命 ≠ 成为

分叉不是阶梯：管理线 vs 架构线（架构线延续=首席架构师）

三种来路三种功课 · 成长的标志=第一次"说不" · 再往上：从系统到组织（四条路径；工具不变，棋盘变大）

入场券（判据）：系统目的或任务理解确认单 + 系统基本概念和基本属性清单

——签字前一切产出无判据锚点；判词量人、入场券量活（能力的入场券=任命，判据的入场券=确认单）

产品数据 = 系统级（架构级）文档族（AD/ADR/ICD/框架定义/派生需求/权衡报告/元素分配/架构基线/演化路线图/系统级验证方案）

分工在层级：架构师主责架构的正确性 / 系统工程师主责系统级（主责兼终责）/ 模块工程师主责模块级

框概分离是每份方案的普遍结构，不是角色分工

接口契约 = 1:1 搬入点——工程师部件方案落下的那根枝梢

架构符合性把关：certifying proper implementation（架构师不只画架构，还要查实现符不符合）

第二层 · 组织级治理产出（制度）：架构原则/政策/委员会章程/合规评估/健康度指标——系统级文档族是"作品"、治理产出是"制度"；治理决策权在委员会、最终裁决在 SE（第14章）

检验：能力再强，也要看你开发出的产品数据，换个人能不能接住。

判词：万辉有头衔缺①④⑥ / 水忠无头衔缺②⑥——两个都不是，两个都在长成

书名：架构师是个怎样的物种？——从工程师到架构师

## 本章判据

> **架构师是一个方向，不是头衔；是一张清单，不是职级；是一份专责，不是终责。**

> 这十四项判据是本书的体检清单——十项能力、两把分工尺、一把母词尺与一把目的判据尺，都是本书的凝练，不是任何标

> 准的条文；标准出处（42010/15288/12207）见正文。它们都当场可检验：欢迎检验、欢迎校准。

十项能力体检（架构师的十把尺子，逐项自问）：

1. **科学素养**：我对这个系统的理论是什么？靠什么成功、最大压力在哪、做错什么最致命？**违反即崩：答不上"系统靠什么成功"，理论就是空的——万辉长期不碰实现，判断悬空（缺①，判词初挂）**。
2. **方法论素养**：我的方法能被别人学会吗？还是只有我会？**违反即崩：方法只在自己脑子里，架构随人走——水忠的判断换个人做不出来（缺②，判词）**。
3. **实践能力**：我最近一次亲手落地，是什么时候？**违反即崩：架构停在图上，判断没有锚点——远离代码的架构师，不知道系统真实的样子（失败四式第四宗，缺③）**。
4. **职业伦理**：压力面前，我会不会为进度悄悄牺牲可靠性？有没有让利益相关方知情？**违反即崩：为进度悄悄牺牲可靠性、还没让利益相关方知情——万辉的"先上再说"，第 2 章事故就是价值层失守（缺④）**。
5. **工程判断**：冲突里，我依据什么做取舍？**违反即崩：图下面没有"决定"的字——画图匠死于自己没有判断（失败四式第一宗，缺⑤）**。

6. **可复现意识**：我的判断换个人能接住吗？我写决策记录吗？**违反即崩：判断换个人就接不住——水忠的老规矩换个人做不出来，成了手艺型架构师（缺⑥，判词）**。
7. **识别后果严重的决策**：我认得出”影响半径大/持续时间长/修改成本高”的决策，并按决策对待吗？**违反即崩：把致命决策当普通字段改——万辉改合同编号规则，以为”今天就能改”，三个月连环三爆（§18.2.4）**。
8. **对系统的理论**：我的架构有原理吗？换个上下文还成立吗？**违反即崩：碰巧做对、却不知道为什么对，换个上下文就错——没有理论的架构，像没有原理的工程（§18.2.5）**。
9. **把架构说清楚**：每个利益相关方的关注点，都有一张图能落吗？**违反即崩：图是死的、视图是活的——利益相关方看不到自己该看的那一面，冲突被闷在各自脑子里（§18.2.6）**。
10. **判断不可逆性**：我认得出不可逆决策吗？该早做的早做、该推迟的显式推迟吗？**违反即崩：把决定留给默认——不决策的架构师，把架构决策的资格交给运气和时间（失败四式第三宗，缺⑩）**。

十把尺子之外，还有两把分工尺、一把母词尺——

11. **专责尺**：我清楚自己背的是专责（架构正确性）不是终责（系统成败）吗？我和系统工程师的分工，是委托—协商—验收吗？**违反即崩：把专责错当终责——“什么都管”的架构师越过边界背系统成败的账，既僭越又失职（§18.3.2，失败四式第二宗）**。

12. **符合性尺**：我检查过实现是否符合架构分配吗？（certifying proper implementation——认证架构的恰当实现）**违反即崩：画完架构就完了，实现符不符合不查——模块详细定义背离架构分配、技术方案违背架构原则，也没人发现（§18.4.6）**。
13. **母词尺**：我知道自己正在做七动词链里的哪个动作吗？构思想清楚了吗？定义定形了吗？认证检查了吗？**违反即崩：只做表达就以为做完了架构——链断在半路，画图匠式架构师（§18.2.9）**。
14. **目的判据尺**：我清楚目的定义权在业务方吗？我是在理解/澄清/质疑/守护目的，还是越界改写它？**违反即崩：目的和架构在脑子里互相即兴迁就——小团队一人兼戴"产品负责人"和"架构师"两顶帽子，做决定不知道戴的是哪顶，Cowboy Coding 式架构（§18.3.7「不是目的的定义者」）**。

十把尺子量到最后是一句话：**能力再强，也要看我能不能开发出高质量的产品数据——换个人能不能接住。**

**工作检查单**（每次做架构决策前）：

- □ 这是不是一次"影响半径大 / 持续时间长 / 修改成本高"的决策？
- □ 这次决定的"目的"签了确认单吗（《系统目的或任务理解确认单》）？我在理解/澄清/质疑/守护目的，还是在越界改写它？
- □ 我对这个系统的理论是什么？这次决定对不对得起它？
- □ 有哪几个利益相关方的关注点会被这次决定影响？他们的视图跟上了吗？
- □ 这次决策记录写了吗？不可逆性标注了吗？

- □ 我是在做决定，还是在画图 / 什么都管 / 等默认？
- □ 这次决定对应的接口契约改了吗？下游的部件方案跟着变了吗？
- □ 我在背专责还是终责？有没有把不该背的账揽到自己头上？

## 自查 · 这些说法对吗？

1. “架构师就是升职后的更高级工程师。” —— **错**。职级是位置，能力是物种；头衔不定义架构师。
2. “把图画得最全的人，就是最好的架构师。” —— **错**。删掉图剩下的两样内容才是架构；画图匠只有图。
3. “架构师应该什么都管，才对得起这个头衔。” —— **错**。管得最准，不是管得最多；什么都管是把约束管成枷锁，也是把专责错当终责。
4. “架构师可以不会写代码，反正有人实现。” —— **错**。远离代码，判断就失去锚点。
5. “被任命为架构师，就是架构师了。” —— **错**。被任命只是入场券，能力清单一项项亮起来才算成为。
6. “架构师和项目经理是一回事，都管进度。” —— **错**。项目经理对日历负责，架构师对演化负责。
7. “架构师的判断可以只凭经验，不需要理论。” —— **错**。没有理论的架构，像没有原理的工程。

8. “识别后果严重的决策，就是做事谨慎一点。”—— **错**。是对不重要的事情不决策，把精力投到真正不可逆的决定上。
9. “架构师最重要的工作是画好架构图。”—— **错**。是把不可还原的涵义、管控原则落成可检验的决定。
10. “架构师是一个终点，达到了就完了。” —— **错**。它是一个方向，一条持续补清单的路。
11. “对重要的事情不决策，等别人决定也行。”—— **错**。不决策是把架构决策的资格交给运气和时间。
12. “架构师的底座（工程师六项）和增量（系统级四项）是两套能力。”—— **错**。增量全部长在底座上，底座缺一项，增量就空转。
13. “架构师的产品是’图’，画好架构图就算交付了。” —— **错**。架构师的产品数据是系统级（架构级）文档族（AD/ADR/ICD/框架定义/派生需求/权衡报告/元素分配/架构基线/演化路线图/系统级验证方案），能力最终体现在能不能开发出高质量的产品数据；图只是架构描述的一种视图，删掉图剩下的两样内容才是架构。
14. “架构师再往上就是升任 CTO——更高的职级。” —— **错**。再往上不是更深的架构，是关注维度从系统扩张到组织；CTO 这个头衔同样不定义能力——从元素到系统、从系统到组织，是同一种升级的两次上演：工具不变，棋盘变大。

15. “架构师是系统工程师的升级，整个系统成败都归他管。” —— **错**。专责 ≠ 终责：架构师对架构正确性负专责（可委托可追责），系统工程师对系统成败负终责（不可委托）。“背在身上”是看在眼里，不是担在肩上。
16. “架构师一个人说了算，系统工程师只管执行。” —— **错**。拍板权在系统工程师，理由权（rationale）在架构师；两者是持续的谈判对手（委托—协商—验收），不是指挥—执行。
17. “架构师画完架构就完了，实现符不符合是别人的事。” —— **错**。架构符合性把关（certifying proper implementation）是架构师职责的一部分——模块详细定义符不符合架构分配、技术方案有没有违背架构原则，都要查。
18. “架构师 = 42010 给头衔立的词条。” —— **错**。42010 只给架构活动（architecting）立词条，不给架构师立——标准认活不认职位；本书讨论的是”从事架构活动的人”。
19. “十项能力清单 = 六块能力增量。” —— **错**。十项清单是角色体检切法（六项底座 + 四项增量），动词链是工序切法（七动词）——两套并行，母词是架构活动七动词链。
20. “架构师要对系统目的负责，目的定义错了是架构师的错。” —— **错**。目的定义权和问责在业务方；架构师对”架构忠于目的”负责，对目的有四职责（理解/澄清/质疑/守护）但没有改写权——

越界改写目的，是 Cowboy Coding 式架构的病因（§18.3.7「不是目的的定义者」）。

## 练习

1. **用十项能力清单，给”你们公司最像架构师的人”做一次体检**——逐项打勾，指出缺项，判断他走在哪条路上。
2. **专责终责盘点**：在你们最近一个项目里，标出”系统成败的账”（终责）在谁手里、“架构正确性的账”（专责）在谁手里——两者是不是同一个人？如果是，那是兼任还是失职？
3. **角色边界演练**：在你们最近一个项目里，找一个”该由架构师决定却被项目经理/技术负责人/领域专家抢走”的决定，说出它本该归谁。
4. **失败模式对号入座**：回顾你或你团队最近一次架构决策，它更接近四宗死法（画图匠/什么都管/不决策/远离代码）里的哪一宗？缺的是清单里的哪一项？
5. **产品数据体检**：把你们系统的一份架构描述和一条接口契约翻出来，问”换个人能不能接住”——答不上来的那部分，就是⑥可复现意识缺的地方。
6. **回部门落地**：把本章判据的”工作检查单”打印出来，贴在下一个架构决策现场——下次做不可逆决策前，先过那八条。

7. **晋升阶梯盘点**：画出你们公司从工程师到 CTO 的晋升路径，用”四条路径”（深度/宽度/组织/权威）对照，说出每条路的核心关切是什么——你现在的棋盘，在哪一格？再标出你们”架构线”和”管理线”的分叉点在哪。
8. **目的权责盘点**：你们系统”目的/任务”的定义权在谁手里？架构师是在理解/澄清/质疑/守护目的，还是悄悄改写它？写一份《系统目的或任务理解确认单》试试：业务方原话 + 你的复述——复述里有没有冒出架构语言（冒出，就是越界了）？

## 参考文献

**标准** - R-04 ISO/IEC/IEEE 42010:2022 —— 架构定义（实体的基本概念或属性＋管控原则〔governing·约束实现与演化及生存周期〕）；架构设计产出＝本质＋管控原则（元素与关系是它们的落点、规格归下游设计定义过程），落点读法见 ch10 §10.2.5——**元素是工程师的，本质与管控原则的成立与守持是架构师的；元素、接口与关系是设计定义的产出（15288:2023 §3.13），不是这一角色的专责对象**；architecting 七动词链定义（构思→定义→表达→记录→沟通→认证恰当实现→维护与改进）= 能力增量的母词，贯穿生存周期反复发生 - R-05 ISO/IEC/IEEE 42020:2019 —— 架构治理流程 §6（建立并保持架构集合中的架构与企业目标/政策/战略一致）；方向权属组织层、内容权属架构师；治理 §6 vs 管理 §7；**六个过程及其分组**（正

本 Table 1 与 §5.3–§5.5）＝治理 §6／管理 §7／概念化 §8／评估 §9／细化 §10／**使能 §11**（常被漏掉的第六个）：§5.3 架构治理与架构管理同组、§5.4 架构概念化·评估·细化为三个核心过程、§5.5 架构使能单列 - R-09 ISO/IEC/IEEE 15288:2015 —— 架构师是工程师角色体系中的一种 specialization；架构定义流程 §6.4.4（生成 → 选择 → 表达）；系统工程师职责域；系统级评审由系统工程师主持、架构师陈述 - R-16 ISO/IEC/IEEE 12207:2017 —— 架构师是工程过程中的角色（与 15288 并提） - R-15 ECPD 1941 / ABET 1979 / INCOSE 2019 —— INCOSE 把技术领导力列为系统工程师核心能力；ABET engineering design 容器定义（决策域/能力灵魂差异的锚点） - R-11 IEEE 1028-2008 —— 架构靠评审验证不靠测试（验证方式差异的锚点） - R-19 TOGAF —— 系统之系统的企业架构四域（业务/数据/应用/技术） - R-59 NASA Systems Engineering Handbook —— 首席工程师角色的权威锚点

**经典文献** - R-70 SEBoK（Systems Engineering Body of Knowledge）—— 派生需求（derived requirements：架构实体的必要实例引发）；协商理解（negotiated understanding）；架构定义活动（partition / align / allocate） - R-82 Perry & Wolf —— 架构腐蚀（drift / erosion）——代码改图不改，动词链”维护与改进”失守的结果 - R-45 INCOSE 系统工程手册 —— MOE/MOP/TPM 三层度量（MOE=效能度量=运行环境中的成功指标≈操作目标层·运行层

确认的显性判分线；MOP=性能度量=系统级；TPM=技术性能测量=关键部件级）

> 本章”十项能力清单”的回扣链：§9.2.1（六项能力清单）与§9.3.6/§9.4.6（粒度谱系·三粒度责任矩阵）、第 12 章（不可逆性）、第 17 章（整合）；第 6 章（需求与方案结对·框概分离）、第 11 章（架构设计）；附录 A T-13/T-25/T-102/T-113~T-117、附录 C 第 18 章行、附录 E 第 18 章自查速查。

# 附录：把尺子收进五个抽屉

> 正文十八章，发的是概念、判据和案例。本附录把它们归档成五个抽屉，供你随时回来取用： **附录 A 术语表**——全书概念的英中对照（术语”宪法”，正文用词的唯一权威）**附录 B 概念档案集**——33 张最常被查、最容易误解的九格档案完整版（字典式查询）**附录 C 误解清单**——全部”反直觉命题”速查（18 章病/药/判据三栏）**附录 D 参考文献总表**——正文引用的标准与经典文献 **附录 E 练习答案**——各章练习与自查的答案要点（与训练教材题卡同源）
> **用法**：遇到说不清的词，翻 A 或 B；“这个误解错在哪”，翻 C；“这条原理出自哪个标准”，翻 D；做完练习想对答案，翻 E。第 1 章把四问交到你手里，把”查全”这一步交给附录（§1.7.5）——这五个抽屉，就是那个”查”字落地的地方。

---

## 附录 A 术语表：全书概念的英中对照

> 术语表是本书的”术语宪法”——凡是正文用词，以本表为准。遇到同词异义，回到本表看指称的是哪个概念；遇到跨域对不上的词（如恒信的”合同编号”与冰链的”数据所有权”），先查本表，再谈别的。

| 编号 | 术语（中文） | 英文 | 一句话定义 | 主章 |
|---|---|---|---|---|
| T-01 | 概念 | concept | 一类对象共同特征的抽象概括；严谨定义＝由特性的独特组合而形成的知识单元（ISO 1087:2019 §3.2.7） | 第 1 |
| T-02 | 术语 | term | 概念的能指——同一个概念可以有多个词 | 第 1 |
| T-03 | 符号 | sign | 能指 + 所指的统一体（索绪尔） | 第 1 |
| T-04 | 能指 / 所指 | signifier / signified | 符号的形式（声音/文字）/ 符号的内容（概念） | 第 1 |
| T-05 | 涵义 | sense（Sinn） | 符号呈现对象的方式（弗雷格） | 第 1 |
| T-[illegible] | [illegible] | [illegible] | [illegible] | [illegible] 章 |



——“清晨[illegible]在[illegible]存[illegible]ce

# 附录 B 概念档案集：33 张完整九格

> 全书核心档案约 45 张。下面 33 张是最常被查、最容易误解的，给出**完整九格**（字典式查询）——前六张为第 1 章学习方法论档案（概念/核心概念/根概念/基本原理/概念体系能力/根概念能力），后二十七张为工程档案（含 2026-08-25 新并入 16 号研习笔记的 5 张：构思/定义/设计·一词两义/创新/架构活动）。其余 12 张（工程系统、需求工程、运行概念 ConOps、框架定义、详细定义、分配生成、分析迭代、分解分析、架构描述、不可逆性、版本、管理）的九格完整版散见各章"概念卡片"，一句话定义见附录 A。用法同第 1 章：薄格可省，不可跳过——本表每个概念都填满了能填的格；查的时候，从①定义开始，到⑧误区为止。

**怎么读这 33 张卡：九格 = 四问的展开**（四问法见第 1 章；格号与栏目见附录 A T-11／T-12）：

- **它是什么**＝① 定义（定界：一句话钉它是谁）
- **它不是什么**＝⑥ 对立 · ⑦ 辨析（定界：远界对立面 + 近界辨析）
- **它从哪来**＝② 背景 · ③ 历史 · ⑨ 英文（溯源：诞生处境 + 演化 + 词源）

· **它往哪去**＝④ 问题 · ⑤ 场景 · ⑧ 误区（致用 + 纠误：为什么造它、什么时候用、最常见的误解）

B 区保持字典式九格顺序（①⑥⑦⑨ → ②③ → ④⑤ → ⑧），不另作四问化改写；薄格可省，不可跳过。

**概念档案 · 概念**

① 定义　对一类对象共同特征的抽象概括；严谨定义（ISO 1087:2019 §3.2.7）：由特性的独特组合而形成的知识单元（可指称 + 可界定）

⑥ 对立　个别（感知 percept / 实例 individual）—概念是"一般"，把无限具体压缩成可沟通的有限单元

⑦ 辨析　术语 term（指称概念的语言符号）vs 概念（被指称的知识单元）；定义 definition vs 概念本身

⑨ 英文　concept ← 拉丁 conceptus（concipere：con-"一起" + capere"抓取"，本义"被把握住的东西"）

② 背景　人需要把无限具体经验压缩成可沟通的有限符号单元—概念是压缩的产物；术语学由此成为学科

③ 历史　亚里士多德范畴论 → 逻辑学/认识论 → 术语学标准（ISO 1087）→ 认知科学（原型理论 Rosch）→ 本方法论（概念=网络节点）

④ 问题　没有概念之前，人只能用具体事例交流（"那个红的圆的"），无法抽象沟通—如何用最少符号承载最多经验？

⑤ 场景　一切思维与沟通的前提；需精确指称个体时改用专名

⑧ 误区　✗ "概念=词语"—词语是符号，概念是意义；✗ "概念=事物"—是对事物的抽象，不是事物本身；✗ "概念有绝对正误"—只有共识强弱（燃素曾是共识）

判据　可指称（一个词/短语）+ 可界定（内涵与外延）；依据 ISO 1087:2019 §3.2.7

概念档案 · 核心概念

① 定义　概念网络中处于枢纽地位的概念——解释领域内其他概念都要用到它，理解它自己只需很少前置知识

⑥ 对立　边缘概念（叶子 leaf）——处于网络末端、被枢纽解释、不解释别的东西（维度：网络地位）

⑦ 辨析　基本概念（入门次序，偏教学先后）vs 核心概念（结构地位，偏被依赖程度）；关键概念（重要但未必枢纽）vs 核心概念（强调"被依赖"）

⑨ 英文　core concept / hub concept；hub 原义"轮毂"——辐条都连到它；图论无标度网络的枢纽节点

② 背景　压缩学习实验发现无标度网（少数枢纽占据大量连接）最易学——识别枢纽成为学习的第一步

③ 历史　图论 hub（Barabási 无标度网络）→ 认知科学压缩学习（北大 2026）→ 学习方法论"先立枢纽" → 本方法论

④ 问题　没有核心概念识别之前，学习不知道从哪下手（信息过载、细节淹没）——领域知识的四梁八柱是什么？

⑤ 场景　学习新领域的第一步（20 分钟立枢纽）、课程设计、教材编写（先枢纽后细节）；领域已有共识框架时直接采用

⑧ 误区　✗ "核心概念=看起来最重要的概念"——是"被依赖最多"，不是"看起来重要"；✗ "一个领域只有一个核心概念"——通常 3~5 个构成骨架；✗ "核心概念固定不变"——随范式更替

判据　反事实检验（去掉它领域还成立吗？）+ 词频（领域文本里被反复提到）+ 被引用度（其他概念定义引用它最多）+ 前置数（理解它所需前置最少）；后三条为代理判据

概念档案 · 根概念

① 定义　概念网络的"承重柱"——领域中不可再向下归约的源头概念，其余概念

（含核心概念）都由它导出；根概念 ⊂ 核心概念（承重柱 ⊂ 骨架），通常 1~3 个

⑥ 对立　派生概念—可由根概念导出（如流程 = 产品数据在时间上的投影）；根概念不可由任何域内概念导出（维度：归约方向）

⑦ 辨析　核心概念（骨架，管"连接多"，3~5 个）vs 根概念（承重柱，管"是源头"，1~3 个）；枢纽 hub（认知层，压缩学习术语）vs 根概念（知识层，源头术语）—同一结构原理的两层表现

⑨ 英文　root concept；root 本义"植物的根"—营养从根流向枝叶；图论有向图的根（root）= 无入边的源节点，恰对应"被全部依赖、不依赖别人"

② 背景　早期用"被引用度=出度、前置数=入度"测核心概念，实际测的是 root（有向源）属性而非 hub（无向中心）属性—名称与判据错位，逼出"根概念"这一层

③ 历史　图论 root（有向源）→ 存在物的不可归约者→ 本方法论四条判据（原子/生成/不可替代/递归）→ 判定矩阵 + 演绎链（§1.7.3）

④ 问题　没有根概念之前，"核心概念清单"只是凭感觉的枢纽列表，不知道哪根柱子最承重—这个领域，什么存在物是不可再归约的源头？

⑤ 场景　确定学习与治理的第一对象、解释"为什么是这个概念"；只需"学得快"时用核心概念就够，不必每次都挖到根

⑧ 误区　✗ "根概念=最重要的概念"—是最"源头"的，未必最"重要"（质量很重要，但它是需求/系统的派生）；✗ "根概念与核心概念矛盾"—分层不冲突（承重柱 ⊂ 骨架）；✗ "根概念凭直觉找"—必须过判定矩阵+演绎链；✗ "根概念永恒"—随范式更替

判据　原子性（不可由域内概念定义）+ 生成性（其余概念从它派生）+ 不可替代性（去掉它领域崩塌）+ 递归性（多个抽象层级反复生效）

概念档案 · 基本原理

① 定义　原理侧的"源头"—领域中不可再被推导的法则（命题），是根概念（母

体）的语义内蕴被展开成的显式命题；基本原理 ⊂ 核心原理（公理 ⊂ 骨架规则），与根概念同处源头层

⑥ 对立　派生命题—可由基本原理推导（如测试用例设计规则由"验证以规定需求为参照"推出）；基本原理不可由域内任何命题推导（维度：推导方向）

⑦ 辨析　核心原理（枢纽命题，骨架规则）vs 基本原理（源头命题，公理）；原理 principle（一般规律）vs 命题 proposition（可判真假的陈述，弗雷格）；"原理"日常还指"原则/准则"，本方法论取"源头法则"义

⑨ 英文　root principle / axiom；root 与根概念同源（图论有向源：无入边）；axiom 希腊语 axiōma"被认为值得的东西"—自明、无需证明的起点

② 背景　诞生于"原理也有根"的追问：概念侧分出根层后，原理侧是否同样有源头？2×2 层级矩阵（节点/边 × 核心/根）落位后，发现原理侧的源头恰是根概念定义的语义内蕴

③ 历史　欧氏几何公设法 → 弗雷格"概念无真假、命题才有真假" → 本方法论 2×2 层级矩阵 → 母体–展开修正（基本原理 = 根概念的展开运算，演绎链即展开器）

④ 问题　没有基本原理之前，领域推理没有公理层—规则从哪来、凭什么站得住，全凭经验与权威：这个领域，什么法则是不容再推导的源头？

⑤ 场景　建立领域公理层、把概念内蕴显式化为工程规则（评审准则/测试用例/追溯矩阵）、用原理作探针检验对根概念的理解；只需"学得快"时用核心原理即可

⑧ 误区　✗ "基本原理=最重要的原理"—是最"源头"的，未必最"重要"；✗ "搞懂根概念就自动搞懂基本原理"—跨概念原理在接缝里，且内蕴≠显式、须展开成命题才可检验；✗ "基本原理凭直觉找"—须过四判据；✗ "基本原理永恒"—随范式更替

判据　不被推导（域内无更基本命题推它）+ 生成规则（派生命题与工程规则从它

导出）+ 去之崩塌（去掉它领域推理失去依据）+ 多层生效（多个抽象层级反复适用）

概念档案 · 概念体系能力

① 定义　以概念为单位的压缩与组织能力（简称"概念能力"）—识别枢纽 × 织关系 × 迁移 × 辨边界

⑥ 对立　机械执行（不动脑的重复）与碎片化记忆（无结构的信息堆积）—组织信息 vs 存储信息

⑦ 辨析　概念技能 conceptual skill（Katz，管理视角，偏"看全局"）；抽象思维（更宽，含演绎推理）；理解力（偏接收端，概念体系能力还含输出端：迁移与辨伪）；根概念能力（子集：方向盘）vs 概念体系能力（全集：四构件）

⑨ 英文　conceptual ability；concept ← 拉丁 conceptus（"抓取"）—概念体系能力字面即"抓取的能力"

② 背景　诞生于"AI 时代人还剩什么"的处境：记忆与信息处理外包给机器，人剩下的稀缺能力是压缩与组织（北大压缩学习 2026；Katz 1955 概念技能）

③ 历史　conceptual skill（Katz 1955，管理）→ conceptual knowledge（布鲁姆修订版 2001，教育）→ concept learning（认知科学）→ 本方法论整合为"概念体系能力"（五环模型第 1、2 环）

④ 问题　没有它之前：学习是碎片堆砌、表达是复读、跨领域是重来—如何把混沌信息变成可用结构？

⑤ 场景　学习新领域（立枢纽）、诊断概念误解（辨边界）、写作与教学（讲给外行听）、跨域转型（迁移）；纯程序性操作为主时不适用

⑧ 误区　✗ "概念体系能力强=懂得多"—组织得好才关键；✗ "概念体系能力强=口才好"—转述是测试，结构是本体；✗ "概念体系能力=核心竞争力"—是枢纽不是全集；✗ "概念体系能力是天生的"—五环模型就是训练流水线；✗ "概念体系能力=根概念能力"—根概念能力只是方向盘（识别承重柱），全集还含织关系/迁移/

辨边界

判据　转述测试（陌生领域三五句话讲清）+ 辨伪测试（能举反例划边界）+ 迁移测试（换个情境还认得出同一个结构）

概念档案 · 根概念能力

① 定义　识别、验证并运用根概念的能力——概念体系能力的子能力（方向盘：先抓承重柱，再展骨架）

⑥ 对立　细节优先的学习习惯（LeafToHub）——从边缘概念入手、不看骨架、更不辨承重柱，被细节淹没

⑦ 辨析　概念体系能力（全集：识别/组织/迁移/辨边界）vs 根概念能力（子集：承重柱识别）；判断力/洞察力（更宽，含价值判断）vs 根概念能力（特指结构性判断）

⑨ 英文　root-concept ability；root（根）——植物的根、图论有向源的"根"；能力对象从"hub（枢纽，连接多）"收窄为"root（源头，被依赖）"

② 背景　压缩学习实验 HubToLeaf 组优势（预学习 200 轮，优势扩散至 800 轮）——"先识别枢纽"可训练且回报极高；进一步追问"枢纽的深层形态"，落到根概念

③ 历史　专家直觉（老手知道什么重要）→ 压缩学习实验证实其可训练 → 三步识别法（收集候选/硬判据/往上挂）→ 三步识别法 + 判定矩阵/演绎链（§1.7.2/§1.7.3）→ 本方法论第 1 环（立枢纽）

④ 问题　没有它之前，新手被细节淹没、学得慢而累，且分不清"重要"与"源头"——面对陌生领域，20 分钟内找到那根承重柱与整副骨架

⑤ 场景　学新领域、做课程大纲、审一份陌生标准、跨界转型；已有权威框架时直接采用，不必每次重找

⑧ 误区　✗ "根概念能力=知识渊博"——知识多不等于会抓根；✗ "是天赋"——判定矩阵+演绎链可训练；✗ "一次定终身"——骨架必须勤校准（五环第 4 环）；✗ "管住根就管住一切"——根概念能力只是方向盘，织关系/迁移/辨边界还要靠全集

判据　20 分钟对陌生领域给出"根概念 + 核心概念清单及其关系"，并能用判定矩阵与演绎链为每个候选辩护
概念档案 · 工程

① 定义　科学原理 × 判断创造，在约束下，用系统方法，造出有用且安全的东西

⑥ 对立　手艺（方法论维度）· 科学（认识论维度）· 艺术（目标维度）—工程没有单一对立面

⑦ 辨析　工程 ≠ 方法论（方法论只是四层之一）；工程 ≠ 科学（创造 vs 发现）

⑨ 英文　engineering ← 拉丁 ingenium（天生的才智、巧思）

② 背景　1941 ECPD 首次给出权威定义，从"造东西"里提炼出可检验的要素

③ 历史　80 年、四个机构（ECPD/ABET/INCOSE/IEEE 610.12）—**ECPD、ABET、INCOSE 三家收敛到同一组要素；IEEE 610.12 另给软件工程一侧的过程特征**

④ 问题　工程和手艺不分 → 交付依赖具体的人，换人就走样

⑤ 场景　团队交接/招聘/复盘时用试金石；"在不在做工程"用四层检验

⑧ 误区　✗ "有流程有工具 = 在做工程"—四层缺一不可

✗ "工程 = 写代码 / 画图纸"—载体不是判据

判据　试金石：换个人、换团队，用同样的需求和约束，能交付质量相当的成果—能，是工程；不能，那是手艺
概念档案 · 需求

① 定义　明示的、通常隐含的或必须履行的需求或期望（ISO 9000:2026 §3.5.1）

⑥ 对立　点子 / 创意—点子不表述为需求，就进不了设计开发的流程

⑦ 辨析　需要（内容·硬性）→ 期望（内容·软性）→ 需求（需要或期望 × 呈现方式）；需求 ≠ 目标

⑨ 英文　requirement ← 拉丁 requirere（要求、寻求）

② 背景　需要或期望的表述——"往往是研究的成果"

③ 历史　从"要的东西"漂移出"明示/隐含/必须履行"三形态——ISO 9000:2026 把这场漂移固化为标准定义（ISO 9000:2026 §3.5.1）

④ 问题　需求与目标、点子混用 → 设计开发转化了错误需求

⑤ 场景　设计开发的输入；三种形态都要识别，隐含需求要主动去挖

⑧ 误区　✗ "需求 = 用户说出来的"——隐含需求、必须履行的需求漏掉是缺陷

✗ "需求 = 目标"——目标回答"达成什么可度量的结果"，需求回答"做成什么样"（答"为什么做"的是目的）

判据　起点判据"可转化"：完整、无歧义、不冲突、隐含需求已识别

概念档案 · 设计开发

① 定义　将某一客体的需求转换为该客体更详细需求的一组过程（ISO 9000:2026 §3.3.11）

⑥ 对立　走流程——输入输出同样详细，没发生设计开发

⑦ 辨析　设计、开发、设计开发三个词的关系（§3.3.11 注 2）；需求开发（ISO 9001:2015 §8.2，**该标准库内无正本、属库外来源**）与设计开发（§8.3）的边界——看"特性有没有被规定"（第3章 §3.3.3 立此边界；第4章 §4.2.5 承接句改挂该节，不引 ISO 9001 条款号）

⑨ 英文　design and development；design ← 拉丁 designare（标注、指明）

② 背景　起点是研究的结果——需求往往是研究的成果，设计开发天然带不确定性

③ 历史　输入比输出更宽泛、更概括；一个项目可有多个设计开发阶段（递归）——输出是更详细的需求，且逐级更细

④ 问题　何时开始、何时结束说不清 → 只做技术方案就宣布完成 → 下游环节停摆

⑤ 场景　从需求到可实现的转化；"这算不算设计开发"用"更详细"判据

⑧ 误区　✗ "设计开发 = 画图 / 创意活动"——是需求转化

✗ "技术方案做完 = 设计开发完成"——下游方案配齐才算

判据　止于"可实现"；"还有哪个环节会在执行时反问'这到底怎么做'？有，设计开发就没结束"

概念档案 · 质量

① 定义　客体的一组固有特性满足需求的程度（ISO 9000:2026 §3.5.2）

⑥ 对立　等级 grade——高档车可以是差质量，低档车可以是好质量

⑦ 辨析　符合性（满足了吗，二元）→ 质量（满足到什么程度，评价）→ 等级（按哪套需求分类）

⑨ 英文　quality ← 拉丁 qualitas（"如何"）

② 背景　程度是被"认定"出来的，不是被"计算"出来的——靠验证/确认用客观证据判定

③ 历史　（薄）从制造业"规格档位"来；标准通篇没有给质量一个单点分值

④ 问题　等级高 ≠ 质量好；"质量 vs 进度"被误判成打架

⑤ 场景　判定满足程度时用；特性超出需求范围的不是加分，是过度设计

⑧ 误区　✗ "特性越多/越高，质量越好"——出了需求区间，只加成本、不加质量

✗ "质量能算分（85 分）"——关键特性不能被加权平均摊平

判据　特性超出需求范围的不是质量加分，是过度设计——只加成本，不加质量

概念档案 · 规定需求

① 定义　被明示的需求——验证的参照；**成文与基线是它的可采条件，不是它的构成条件**（ISO 9000:2026 §3.5.1 注 2 的 e.g. 是举例）

⑥ 对立　不可采的明示——口头说了，但没判定方法、不可复现、无版本，够不上"依据"

⑦ 辨析　明示 = 规定（一个等号）；成文 ⊂ 明示（最常用的一种明示方式）

⑨ 英文　specified requirement；specified ← 拉丁 specificare（明

确指明）

② 背景　需求开发四活动的输出——需求被"生"出来的成品

③ 历史　（薄）"可验证"是从验证定义（§3.11.12）推出来的，不是定义原话

④ 问题　没有规定需求，验证无从谈起（验证三要素的第一要素）

⑤ 场景　验证的参照；需求评审的对象；验收与责任的依据——评审、验证、确认都绕不开它

⑧ 误区　✗ "客户说了就是规定需求"——那是资格，不是可采；✗ "没写成文档就不算规定需求"——不算的是可采，不是资格；✗ "口头说了就能当依据"——缺的是可采

判据　一条需求能提供客观证据吗（测试只是其一）？能，就可采、可当依据；只能"感觉"，就还得写下来、定下来

概念档案 · 评审

① 定义　为达成既定目标，对客体的适宜性、充分性或有效性所作的确定（ISO 9000:2026 §3.11.2）

⑥ 对立　验证——评审对目标（方向对不对，判断），验证对需求（做对了吗，证明）

⑦ 辨析　评审 ≠ 核对需求——核对需求是验证的姿势；评审是唯一在纸面阶段抬头看目标的环节（确认是建成后的实证终审，方向上的把关是前后两处，见 §8.3.6）

⑨ 英文　review ← re-（再）+ view（看）——再看一遍；第一遍是做，第二遍是看

② 背景　2026 版与验证/确认同属 §3.11"有关确定的术语"组——分野在动作词：评审用"确定"（determination，§3.11.1）定义，验证/确认用"证实"（confirmation）定义

③ 历史　IEEE 1028 把评审做法分成五类：管理/技术/检查/走查/审计

④ 问题　把评审做成核对需求 → 需求错了没人发现；目标偏了没人看

⑤ 场景　五个把关节点处的评审洒全程（需求/设计/代码/测试/发布）；复盘是单独的节点、不是评审（评审对目标，复盘查偏差）；目标具体化才评得起来

⑧ 误区　✗ "评审就是开会过一遍"——没结论、没记录，最多算聊天

✗ "评审目标太抽象，评不了"——目标具体化配方：对象 + 时限 + 可衡量结果

判据　三问过了吗（适宜/充分/有效）？通过标准会前定了吗？团队、结论、记录三属性齐吗？

概念档案 · 验证

① 定义　通过提供客观证据，证实规定需求已得到满足（ISO 9000:2026 §3.11.12）

⑥ 对立　评审——验证对需求（做对了吗，证明），评审对目标（方向对不对，判断）

⑦ 辨析　验证 ≠ 测试——测试是提供证据的手段之一；验证 = 对照规定需求 + 客观证据 + 证实

⑨ 英文　verification ← verus（真）——求真：对照规定需求，证明为真

② 背景　2026 版与评审同属 §3.11"有关确定的术语"组——分野在动作词：验证用"证实"（confirmation）配客观证据定义

③ 历史　V 模型的右半边——需求在哪一级分解，验证在哪一级回收（左右镜像）

④ 问题　只验证不确认 → 隐含需求系统性漏失 → 测试全过、没人用

⑤ 场景　对照规定需求提供客观证据；测试/检验/审查/计算/文档评审都是证据（符合性分析=设计层的验证）

⑧ 误区　✗ "测试全绿 = 验证过了"——测试只是证据之一，验证要对全部规定需求做证实

✗ "隐含需求靠验证兜住"——验证的尺子在文档里，够不着没写下来的

需求

判据　三要素齐吗（对照 / 证据 / 证实）？逐级回收（单元 → 集成 → 系统）了吗？

概念档案 · 确认

① 定义　通过提供客观证据，证实针对特定预期用途或应用的需求已得到满足 (ISO 9000:2026 §3.11.14)

⑥ 对立　验证——确认对用途（做对的东西了吗），验证对需求（做对了吗）

⑦ 辨析　确认 ≠ UAT 走流程——UAT 必须是真实用户 + 真实任务 + 真实场景；让业务方点测试环境是把确认做成了验证

⑨ 英文　validation ← validus（强壮、有效）——放进真实场景，看它"立得住"吗

② 背景　参照在场景里不在文档里——预期用途三构成：目标用户 / 使用场景任务 / 环境条件

③ 历史　（薄）"使用条件可以是真实的或模拟的"给了灰度发布、仿真测试的合法性

④ 问题　确认的存在价值建立在非明示需求上——写不下来的需求，只有真实场景能裁决

⑤ 场景　UAT / Beta / 灰度 / 试运行 / 仿真测试；兜住隐含需求，也兜住体验类需求

⑧ 误区　✗ "测试全过 = 能用"——测试证明做对，不证明能用

✗ "隐含需求先写文档再验证"——很多隐含需求写不出来，只有真实场景能裁决

判据　真实用户 + 真实任务 + 真实场景齐了吗？隐含需求在真实场景里显现并处理了吗？

概念档案 · 架构

① 定义　一个实体（如系统）在其环境中的基本概念或属性，以及用于该实体及其

相关生存周期过程的实现和演化的管控原则（governing principles；ISO/IEC/IEEE 42010:2022；本书读作系统不可还原的涵义）——这些基本概念与属性构成演化中必须守住的**不变量集合**，是第14章不变量链（目的→不变量→根本决策→遵从与演化）的第二环

⑥ 对立　详细设计——架构是"去掉就不是这个系统"的那组**基本概念或属性与管控原则**（不可还原）；详细设计是"改了系统还是它"的细节（可还原）

⑦ 辨析　架构 ≠ 一张图——图是架构描述；架构真实存在但没有视觉形态、**交不出去**（能交付、能评审、能版本化的是架构描述）；**元素与关系不是定义要素**（2022 版已从定义句删去，降为 42020:2019 §3.3 注 1；见 T-63 行）；框架 ≠ 架构——框架是"这一类"的组织方式（骨架+位置+组织规则，内容后填），架构是"这一个"实体在其环境中的基本概念或属性与管控原则，用框架 ≠ 有架构

⑨ 英文　architecture ← 希腊语 arkhitekton（首席工匠）；fundamental ← fundus（底部、地基）

② 背景　系统太复杂，一个人装不下 → 靠"**哪些才是基本的**"这一问把复杂度压成可管理

③ 历史　早期"架构=图纸" → 1970s 结构化设计 → 1990s 软件架构热 → 后由 ISO/IEC/IEEE 42010（现行 2022）定稿

④ 问题　架构 = 一张框图 → 图过时了架构也"过时"了（其实那组基本概念或属性一直活着）；架构 = 那组改起来贵的决定 → 漏判"不贵但承重"的，误纳"很贵却与目的无关"的

⑤ 场景　在不可逆决策做出来之前；变更影响评估时；团队边界定义时

⑧ 误区　✗ "架构就是那张架构图"——图会过时，**那组基本概念或属性与管住它的管控原则（架构）**可以一直有效

✗ "架构等于技术选型"——技术选型是"概念或属性"的一种体现，不是全部

判据 **必要性**（去掉它，目的落空吗；"不可再分／不可缺失／不可替代"三问是它的顺口问法）＋**充分性**（齐备之后，剩余问题是否都不影响目的的可达）——**不可逆性一类只是代理，管排序与时机，不管准入**

概念档案 · 架构描述

① 定义 用来表达一个架构的**工作产品**（work product；42010:2022 §3.3）——架构无形、交不出去；能交付、能评审、能版本化的是它

⑥ 对立 架构—架构是抽象的、不可交割；架构描述是有形的、可交付、可版本化的

⑦ 辨析 ≠ 架构本身；≠ 一张图（图只是它的**视图**之一）；≠ 交付物清单——它按"利益相关方 → 关注点 → 架构视角 → 架构视图"组织，含 11 类架构描述元素（§3.4 Note 1）

⑨ 英文 architecture description（AD）；description ← de-（彻底）+ scribere（写）

② 背景 架构无形，要被人使用、评审、冻结基线，必须落成有形的信息载体

③ 历史 2011 版定义句曾把架构"体现于元素与关系"，易被读成"结构即架构"；2022 版删去该分句，并把 AD 明写为 work product（§3.3）

④ 问题 把架构描述当架构 → 图过时就说"架构过时了"；把描述当交付物 → 以为交付了描述就等于交付了架构

⑤ 场景 评审、入库、基线冻结、交付与沟通的**唯一有形标的**；架构符合性检查的输入

⑧ 误区 ✗ "我们的架构在这台服务器上"——说的是描述；✗ "画完图架构就有了"——图只是视图，架构要靠两样内容立住

判据 能不能被**单独识别**、**存储**、**交付**给别人用（§3.14 信息部

件的三个动词）；**受控状态**（进配置管理、可冻结基线）是它成为"参照"的条件

概念档案 · 架构设计

① 定义　把需求翻译成"架构"的活动——产出架构的**两样内容**（基本概念或属性 + 管控原则），并把它表达为可评审、可交付的**架构描述**；**元素与关系是架构"可以关乎"的对象，不在定义句内**（其规格归设计定义过程）

⑥ 对立　详细设计——**分界在种类，不是粒度**（15288:2023 §5.9：架构管适宜性·存续性·合意性；设计管与技术及其他设计元素的兼容性、构建与集成的可行性）；判据＝这份内容**能不能被设计违反**；"先横后纵"只是经验启发式

⑦ 辨析　架构设计 ≠ 画架构图（图是架构描述，是产物）；≠ 技术选型（选型是候选，待检验）；≠ 定老规矩（老规矩是管控原则的候选/雏形，过四关才升格）

⑨ 英文　architecture design；design ← de-（做）+ signare（标记）——"做出标记/定形"；设计是动词（构思），定义是完成时（定形）

② 背景　需求必须被翻译成"**架构**"（不是"结构"），翻译必然做选择；15288 只有"架构定义过程"没有"架构设计过程"——过程以产出物命名

③ 历史　从"画框图"漂移出"决策记录"（ADR）；42020:2019 **§8.4.5 综合潜在方案 / §8.4.7 形成候选架构**（"架构综合"是本书早期措辞，正本无此词）

④ 问题　把画图/选型/定规矩当架构设计 → 没有可评审、可验证、可基线化的定形产物

⑤ 场景　输入六类（目的与任务/需求[功能+非功能]/约束/环境/关注点/资产）；产出架构的**两样内容**（基本概念或属性、管控原则）并表达为**架构描述**；关注点识别

⑧ 误区　✗ "画了图+做了选型+定了规矩 = 做完架构设计"——三样都是产物/候选/规则，没有一样是翻译本身

✗ "架构设计 = 评审会上的事"——它藏在每个"加个模块""改个字

段"里

判据　两问：它产出的是不是**对目的必要且充分**的那组基本概念或属性与管控原则（"基本"的准入判据见 T-138）？这两样有没有落成可评审、可交付的**架构描述**？——两问都答"是"，才是架构设计；没定形的构思，等于没设计

概念档案 · 基本概念

① 定义　系统在其环境中"怎么想"的根源——组织范式 / 核心抽象 / 根本机制 / 根隐喻（42010 定义里的 concepts——本质·怎么想）

⑥ 对立　详细实现——基本概念是"去掉就不是它"的层，改它等于换一个系统；详细实现是"改了系统还是它"的层

⑦ 辨析　基本概念 ≠ 技术选型——选型是候选，概念是幸存下来的承诺；命名是概念化的完成仪式

⑨ 英文　fundamental concepts；concept ← 拉丁 concipere（一起抓住、成孕）——把分散的东西"抓住"成一个整体

② 背景　90% 情况下从已验证概念库中选择，10% 被需求/质量属性逼着生成

③ 历史　"基本概念"来自 42010 定义中的 fundamental concepts；生成路径五条（模式选择/领域抽象/质量反推/类比迁移/第一原理）

④ 问题　把概念当发明 → 每代系统从零开始；概念没命名 → 无法共享、讨论、传承

⑤ 场景　组织范式定形态、核心抽象定实体、根本机制定机理、根隐喻定眼光；质量属性反推是"驱动"机制

⑧ 误区　✗ "概念是架构师发明出来的"——90% 是选出来的，10% 是逼出来的，候选要过检验才配叫"基本"

✗ "画了架构图 = 定义了概念"——概念长在决策里，不长在图上

判据　四问：它是组织范式还是核心抽象？走哪条生成路径来的？过了检验吗？它有名字吗？

概念档案 · 基本属性

① 定义　系统具备的本质特征——可观察、可度量的表现；涌现属性，由结构决定

⑥ 对立　基本概念——概念定"系统是什么"（可切可选）；属性定"系统具备什么"（涌现，只能让结构去催生）

⑦ 辨析　属性 ≠ 需求指标——需求指标是目标，属性是结构运转后涌现的结果；场景化（QAS）是两者的桥

⑨ 英文　fundamental properties；property ← 拉丁 proprius（自身的、固有的）——系统固有的东西

② 背景　你设计的是结构，属性是结构运转之后的宏观效果——不能直接设计属性，只能设计结构让属性涌现

③ 历史　SEI 属性驱动设计（ADD）把属性设计拆成四步：QAS 场景化 → 战术选择 → 结构落位 → 验证度量

④ 问题　直接设计属性 → 画不出"可用性模块"；属性不场景化 → 无法进入设计、无法验收

⑤ 场景　质量属性是架构的主要驱动；属性之间天然冲突，设计实质是排优先级（ATAM）；生命线属性 = 关键性分析

⑧ 误区　✗ "直接设计可用性"——不能设计强度，只能设计晶格；不能设计可用性，只能设计冗余

✗ "属性可以分配给某个模块"——质量属性是涌现的，不能"分"，只能"译"成部件约束

判据　四问：目标属性场景化了吗（QAS）？选了哪个战术？战术落在哪个结构位置？度量过、涌现达标了吗？

概念档案 · 架构决策

① 定义　你在设计系统时做出的某个选择，这个选择选错了的话，后来要花很大代价才能改对

⑥ 对立　实现决策—架构决策修改成本高（三标志）；实现决策改得动、改得起

⑦ 辨析　架构决策 ≠ 架构风格（微服务/事件驱动是"套路"）；也不等于"画架构图时做的决定"—它藏在每个"改个字段"里

⑨ 英文　architecture decision；decision ← 拉丁 decidere（切断、裁断）—把多个选项"切断"成唯一选择

② 背景　ISO/IEC/IEEE 42010 把架构决策列为架构相关概念；Nygard 2011 提出 ADR 记录

③ 历史　从模糊说法变成可记录、可管理、可审查的工程对象

④ 问题　架构决策不可避免—不是做不做，是早晚；改个字段可能是数据模型

⑤ 场景　改字段前三问（联动改多少/多少决定建立在它上/改回来代价多大）；不可逆决策提前投资；决策记录六问

⑧ 误区　✗ "改字段是小事"—数据模型可能正是最不可逆的架构决策

✗ "架构决策是评审会上才有的"—没被识别出来的架构决策依然承担全部代价

判据　三标志：影响半径大吗？持续时间长吗？修改成本高到不想改吗？—两问以上答"是"，就该按架构决策对待

概念档案 · 配置

① 定义　实体被记录在配置信息里、相互关联的功能特性与物理特性—实体"被记录下来的样子"

⑥ 对立　实体的全部客观特性—特性客观存在，配置是"被管理起来的那部分特性"（三个隐形过滤器切出来的子集）

⑦ 辨析　配置 ≠ 配置项（对象 vs 属性：配置项是"人"，配置是"人的身高体重"）；配置 ≠ 配置信息（属性 vs 记录：配置是"体检结果"，配置信息是"体检档案"）；配置 ≠ 参数文件（参数是零件，配置是装好的整机）

⑨ 英文　configuration—con-（共同）+ figurare（塑造成形）＝共同塑

成的形态；中文"配+置"＝配合+安置＝搭配并安置成形，两语同源

② 背景　变与不变的分离——核心只写一遍，变化的部分外置；"改一个参数就要改代码、重编译、重发布"是没分离的信号

③ 历史　从"多"到"一"的旅程——多个组件经"配"与"置"塑成一个整体形态（as-designed / as-built）；选配有 requires / excludes / mandatory 约束

④ 问题　把配置当参数文件 → 拿零件清单当整机图纸，追溯时"这台箱子当时是什么样"答不上来

⑤ 场景　配置项清单（管哪些）；类别/记录/关联三个隐形过滤器（哪些特性算配置）；追溯温度记录必须知道固件版本

⑧ 误区　✗ "配置 = 参数文件 / 个性化设置"——参数只是记录形态的图纸，形态才是配置

✗ "配置只跟软件有关"——硬件、图纸、文档、数据都要配

✗ "配置就是客户能改的那几个开关"——选配、约束、组合都是配置

判据　问三句：它被记录了吗（记录边界）？它是功能/物理特性吗（类别边界）？它和其他特性关联成整体了吗（关联边界）？——三句全"是"，才是配置

概念档案 · 基线

① 定义　经批准、确立了产品或服务某一时点特性、作为生存周期参照的配置信息（ISO 9000:2026 §3.10.12）——被批准冻结的参照

⑥ 对立　流动状态——基线把"流动"升格为"受控"：线左版本自由流动，线右任何变化必须走变更控制

⑦ 辨析　基线 ≠ 版本（版本是刻度，基线是有"资格"的那一版——把里程桩当收费站）；基线 ≠ 发布；基线 ≠ 快照

⑨ 英文　baseline——base（基础）+ line（线）= "底线/基线"；本指测绘基准线，转义为"作为基准的那一条"

② 背景　让一直在变的东西说得清——变化需要一个"从什么变起"的原点

③ 历史　功能 → 分配 → 产品三种正式基线；"版本顺着流、基线垂直切"（里程桩 vs 收费站）

④ 问题　没有基线，变化就不可言说——每一次变更都查无实据

⑤ 场景　功能基线/分配基线/产品基线；变更控制循环（冻结→变更→再冻结）的支点

⑧ 误区　✗ "版本号就是基线"——版本是刻度，基线是资格

✗ "基线建立了就不能改"——基线可以变更，但必须走变更控制，改完建立新基线

判据　三特征问全：被批准了吗？被冻结了吗？生存周期拿它作参照吗？——三问全"是"，才是基线

概念档案 · 治理

① 定义　一种以人为基础的系统——组织为实现其宗旨，通过该系统受到指导、接受监督，并被要求就其宗旨的实现作出说明、承担后果（ISO 37000:2021 §3.1.1；ISO/IEC 38500:2024 §3.3 同口径作"指导、监督和担责"）；由被治理对象之外的人或机构，依据规则与担责，对"方向与框架"持续纠偏——掌舵，不是统治

⑥ 对立　统治（rule）——统治预设"主权者-臣属"关系（主体单一、强制命令、自上而下）；治理预设"领域+规则+角色+担责"（主体多元、协商为主、多向互动）

⑦ 辨析　治理 ≠ 管理（掌舵 vs 划桨：决策系统 vs 执行系统；判据——"定参数是治理，在参数内选最优是管理"）；治理 ≠ 例行工作（治理是持续系统，定期评审只是传感装置）；治理 ≠ 清洗工具（定规则裁例外是治理，清洗是执行层的活）；治理 ≠ 文件堆或 IT 系统（以人为基础说的是**构成**）

⑨ 英文　governance ← govern ← 拉丁 gubernare ← 希腊 kybernân（掌舵、领航）——同根词 cybernetics（控制论）保留"反馈纠偏"本义

② 背景　委托必有治理——责任可委托、担责不可委托（ISO 37000 §4.2.2／§6.5.2、ISO/IEC 38500 §6.1）；委托不是治理的前提，无委托时治理同样是组织常态（§3.3）

③ 历史　1989 世界银行"治理危机" → 1995"治道" → 2000 俞可平定译 → 2013 国家治理现代化；"治"=治水疏导、"理"=治玉顺纹

④ 问题　治理与管理混淆 → 买了"治理平台"干管理的活（术语膨胀）；判据与门槛混用 → 把"跨部门"当判据

⑤ 场景　三条件（分离/静默积累/框架级）同时满足、需要正式治理机构时；治理机构持续行使四活动（利益相关方参与/评估/指导/监督）；委员会定规则、裁例外、设传感

⑧ 误区　✗ "治理 = 领导发号施令"——治理是引导不是命令，舵手不是船主

✗ "治理 = 买一套治理平台"——平台是工具，治理是人做的一套安排

✗ "治理 = 管理制度健全"——制度是管理；谁定制度、制度偏了谁来纠，才是治理

✗ "跨部门才算治理"——跨部门是升级门槛，不是判据

判据　问四句：谁来治（角色）？依何治（规则）？如何治（机制）？治得怎样（担责）？——四问有答，才谈得上治理

概念档案 · 三类需求

① 定义　产品需求要全 = 功能需求 + 环境适应性需求 + 可实现性需求，每类"及其指标"（功能以 IPO 定义行为；环境对气候/法规/文化的适应；可实现为生存周期八方面）

⑥ 对立　功能清单——只写"要做什么"，把产品需求当成功能需求，缺了环境适应性与可实现性

⑦ 辨析　功能/非功能二分 ⊂ 三类需求："非功能需求"是个大筐，装着环境适应性和可实现性两类主人、来源、验证载体都不同的需求；分类的判据不是特性本身，

是"谁在问"

⑨ 英文　functional / environmental adaptability / realizability requirement；realizability ← realize（实现）

② 背景　产品的终点是"可实现"——只有功能需求，交付物停在原理样机；补上环境适应性需求，停在环境样机；补上可实现性需求，才是产品。分类判据是"谁在问"（客户、环境、承制方），不是特性的名字（第5章 §5.2.2）

③ 历史　以"产品 vs 样机"为界：功能/环境/可实现三对三对应原理样机/环境样机/产品（需求类别 → 验证条件 → 交付物）

④ 问题　需求"全不全"说不清 → 靠"功能写得多"自欺 → 量产翻车

⑤ 场景　相关方需求捕获、需求评审、SRS 编写；每一类都要答出来源、相关方、指标、验证载体

⑧ 误区　✗ "需求写全了 = 功能写全了"

✗ "可实现性需求是后续工艺的事"——它是产品的固有特性，必须写进相关方需求

判据　每一类都能答出"谁在问"（客户/环境/承制方）+ "用什么交付物验证它"（原理样机/环境样机/产品）

概念档案 · 验证载体

① 定义　每类需求的专属验证交付物：功能→原理样机、环境适应性→环境样机、可实现性→真正的产品

⑥ 对立　用一个载体验证全部需求——用原理样机的通过宣布产品成功

⑦ 辨析　验证载体 vs 评审/验证/确认（第8章）：载体回答"每种需求用什么证明"，判定回答"验证时对照什么"；载体验证的是需求，不是"做对了吗"的动作

⑨ 英文　prototype（原型、样机）——先造的、用于检验的实物；validation vehicle（沿用译名；按第8章严格口径，环境样机入真实环境更近确认，见⑥⑦）

② 背景　三类需求需要三种互不相容的验证条件：原理要脱离环境/成本单独验，

环境要在真实环境验，可实现要在量产经济性约束下验

③ 历史　"样机→环境样机→产品"逐级完备，让"样机通过"被误读为"产品成功"——但每类需求只有一个专属载体

④ 问题　原理样机通过就宣布产品成功 → 环境适应性未验、可实现性根本没进需求清单 → 量产翻车

⑤ 场景　验收、里程碑决策、需求评审；判定"交付物是什么"用三类载体的验证完成度

⑧ 误区　✗ "样机验证通过了 = 产品成功了"——只验证了功能需求

判据　每一类需求都答出"用什么交付物验证它"——答不上来的那类，就是需求没写全的证据

概念档案 · 承制方主导

① 定义　产品需求开发由承制方主导（捕获、分析、建模、引导范围、形成条目），利益相关方只保留确认终审权

⑥ 对立　客户主导——把需求开发当成"客户说、承制方记"；承制方对产品更专业，是主导方而非转述方

⑦ 辨析　承制方（主导·加工）vs 来源方（客户/环境/资产·原料）vs 确认方（相关方·终审）——三分工；引导可以激进，确认不能缺席

⑨ 英文　承制方 = 产品的实现方（realizing party / developer）；主导 ≈ lead / drive，区别于 guide（引导）

② 背景　客户知道"要什么"但不知道"什么可能/什么成熟/什么惯例"——专业不对称让承制方必须引导

③ 历史　需求捕获的"隐含性"（客户的需要连他自己都没意识到）→ 需要专业需求分析师（承制方团队）去挖——"承制方引导"在需求分析师角色上显形

④ 问题　承制方被动接单 → 需求不全 → 责任甩给客户（"没提清楚"）

⑤ 场景　需求访谈、范围沟通、需求评审；"需求全不全"的问责对象永远是承制

方

⑧ 误区 ✗ "承制方就是听客户要什么就做什么"

✗ "需求不够全是客户没提清楚"——需求全不全是承制方的责任

判据 需求全不全、明不明确、可不可度量——问责承制方；引导可以激进，利益相关方确认一次不能省

概念档案 · 产品数据

① 定义 关于产品的信息的、形式化的表示，适合由人或计算机处理（ISO 10303-1:2021 §3.1.29〔库内无正本〕）；产品的信息存在、开发工作的全部对象（六类：需求/设计/制造/质量/服务/管理）

⑥ 对立 产品本身（物理存在）——描述与被描述者；数据管不到实物那一半

⑦ 辨析 不是文档（数据是内容、文档是容器）；不是配置项（受控的才是）；不是产品（验证要求两者一致）；不是架构本身（可交付与可冻结的是架构描述 AD）

⑨ 英文 product data ← product（产品）+ data（数据）；ISO 10303（STEP）为数据交换而定义

② 背景 产品数据交换困难催生 STEP 标准；开发全程在数据上工作，实物只是数据的复制品

③ 历史 ISO 10303（STEP）1980s 起规范产品数据表达；与 ISO 10007"配置信息"同族

④ 问题 没有它，开发="把实物做出来"、数据="开发完再补的文档"——追溯、验证、交接全断

⑤ 场景 六类数据的定义、组织、交接；评价锚点、问责落点、协作的对象

⑧ 误区 ✗ "数据是文档，开发完再补"——数据是开发工作的全部对象

✗ "产品=实物"——设计态里产品=纯数据

判据 六类数据覆盖产品全程、可验证可追溯；设计态里开发产品 ≡ 开发产品数据

概念档案 · 两域三态

① 定义　产品在两个域（信息域/物理域）、三种状态（设计态/制造态/使用态）中存在；设计态产品=纯数据

⑥ 对立　物理存在（实物）—数据管不着的那一半（制造偏差、使用体验）

⑦ 辨析　三态是同一产品的三种存在方式，不是三个产品；设计态同一性 ≠ 产品就是数据

⑨ 英文　as-designed（设计态）/as-built（制造态）/as-maintained（使用态）；数字孪生=信息域对物理域的实时映射

② 背景　开发/制造/使用三个阶段，产品的存在形态不同；PLM 用三态组织数据

③ 历史　PLM/STEP 数据管理实践；EBOM（工程 BOM）→ MBOM（制造 BOM）→ 维护记录

④ 问题　把三态混为一谈，就会问错"产品是什么"—设计态里没有实物可看

⑤ 场景　判断"开发产品=开发产品数据"的成立范围；组织产品数据的分态管理

⑧ 误区　✗ "质量在车间里"—开发环节的质量就是数据质量（数据质量决定上限）

判据　设计态产品=纯数据（开发≡开发数据）；制造态=数据+实物；使用态=实物+实况

概念档案 · 价值三层

① 定义　利益相关方对产品贡献价值的沉淀层（物化/决策/过程三层）；产品数据是价值的投影与锚

⑥ 对立　价值本体（人的判断与协作）—数据管不到的源头

⑦ 辨析　三层 ≠ 全部价值：决策层（ADR）部分沉淀、过程层只留结果

⑨ 英文　ADR = architecture decision record（决策层沉淀）；Polanyi"我们知道的多于我们能说出的"

② 背景　工件是判断的投影：完美工件可能是错误决策的产物（验证全绿、确认全红）

③ 历史　源自"决策要记录"（决策记录六问）与知识管理实践

④ 问题　以为"数据全=价值全"——决策与过程层的价值流失，产品数据装不下

⑤ 场景　评价角色价值（工件质量×判断质量×下游影响）；职责挂在工件上

⑧ 误区　✗ "产品数据装下了全部价值"——物化层全在，决策/过程层只部分沉淀

判据　物化层完全在产品数据中、决策层部分沉淀（ADR）、过程层结果记入；数据是锚，判断与协作是本

概念档案 · 构思

① 定义　从关注点出发生成多个候选架构概念的发散性概念活动——"打开可能性"；产物是"可能是什么"，不是"决定是什么"

⑥ 对立　定义——构思发散、定义收敛；构思停在候选，定义画定边界

⑦ 辨析　构思 ≠ 灵感（构思有方法：模式库/参考架构/第一性原理）；≠ 设计本身（构思是广义设计的发散半场，设计还负责收敛与转换到底）

⑨ 英文　conceiving ← 拉丁 con（收进来）+ capere（握取）——"怀在脑中"；基因是孕育可能性，不是做决定

② 背景　ISO/IEC/IEEE 42010 architecting 动词链第一词；标准用了它却未单独立词条，操作化在 15288 §6.4.4"生成候选架构"任务

③ 历史　构思作为工程概念长期被"设计"吞并——42010 把 architecting 拆成七动词后，构思才从设计的影子里走出来

④ 问题　直接把第一个想法当唯一解——"唯一解 = 未审视的解"，没有候选就没有权衡

⑤ 场景　架构生成六步的①②步（定驱动因素→生成备选）；纪律：备选 ≥2 个；陷阱：稻草人备选（造注定输的陪跑，让预定方案"胜出"）

⑧ 误区　✗ "构思=等灵感"——构思有方法，方法正是创新可工程化的核心证据

判据　这一个动作产出了 ≥2 个候选概念吗？每个候选都能说清它回应谁的什么关注点吗？—是，才是构思；只有一个解的"构思"，等于没构思（本书凝练）

概念档案 · 定义

① 定义　在候选之间画边界、做选择，使方案从多义变单义、从漂浮变确定—"终结漂浮"

⑥ 对立　构思—定义终结构思打开的漂浮；一个是发散、一个是收敛

⑦ 辨析　定义 ≠ 陈述一句话（陈述是定义的产出，定义是画边界的过程）；≠ 设计狭义（定义收敛出被承诺的架构，设计把它转换到底）

⑨ 英文　defining ← 拉丁 de（往下）+ finis（边界/终点）—"划到边界为止"；同根：finish、final；ISO 1087-1：definition = 把概念与相关概念区分开来的表述

② 背景　ISO/IEC/IEEE 42010 architecting 动词链第二词；15288 §6.4.4"架构定义过程"内部第一件事就是构思—流程义的"定义"包住了动作义的"定义"的上游

③ 历史　定义是"口径事故"高发词：三重身份共存（动作义=画边界 / 流程义=15288 §6.4.4 技术流程 / 概念框架义=ISO 1087-1 界定表述）—同一句"架构定义"可能指三种东西

④ 问题　候选停留在多义、漂浮—没有边界就没有"被承诺的架构"，下游无法接棒

⑤ 场景　架构生成六步的③④步（权衡选定→分配与接口）；效用树+ADR 落定选择；partition/align/allocate 把边界落到元素与契约

⑧ 误区　✗ "架构定义 = 给架构写一句描述"—动作义与概念框架义混用，把"画边界"读成"写词条"

判据　选定方案有边界吗？被否掉的备选"死因"说得清吗（ADR）？—边界画定、漂浮终结，才是定义（本书凝练）

概念档案 · 设计（一词两义）

① 定义　一词两义：广义 = ABET 容器（迭代决策与创造的容器过程，⊃ 构思+定义+表达）；狭义 = 以生效架构基线为输入的下游细化流程（15288 §6.4.5）——"转换到底"

⑥ 对立　广义设计 vs 狭义设计——容器（包含构思定义）vs 下游（基线生效后才接棒）；并列使用时"设计"必须降为狭义义

⑦ 辨析　设计 ≠ 画图/选型/定规矩（那些是产物与候选，见架构设计卡）；≠ 构思（构思是设计的发散半场）；≠ 定义（定义收敛，设计转换）

⑨ 英文　design ← 拉丁 de（出来）+ signare（做记号）——"做记号标出来"，把意图标记成可传递的形式；ISO 24765：design 兼指过程与结果

② 背景　ABET engineering design = devising a system/component/process to meet desired needs (iterative/creative/open-ended)——"容器"出处；ISO 9000 §3.4.8：设计开发 = 把需求转换为更详细需求（可递归）

③ 历史　广义/狭义两义长期并存：工程教育（ABET）取容器义，系统生命周期（15288）取流程义——"设计是动词（构思）"是广义的通俗说法，"设计定义过程"是狭义的正式名称

④ 问题　不声明取哪一义，"设计"就在容器与下游之间漂移——说"我们在设计"，可能是构思阶段，也可能是基线后的细化

⑤ 场景　广义：设计全过程的统称（构思→定义→表达→…）；狭义：架构基线生效后 15288 §6.4.5 接手，把架构逐层细化为可实现元素与契约

⑧ 误区　✗ "设计 = 把想法画出来"——忽略广义容器（构思定义在内）；✗ "设计 = 架构基线前的一切"——忽略狭义下游（基线生效后设计还在继续）

判据　说"设计"时声明了取广义还是狭义吗？狭义设计的输入是生效的架构基线吗？——一词两义分得清，才是设计（本书凝练）

概念档案 · 架构活动

① 定义　architecting——在系统整个生存周期内构思、定义、表达、记录、沟通、认证恰当实现、维护与改进架构的过程（ISO/IEC/IEEE 42010 §3.1）；架构师能力增量的母词

⑥ 对立　架构设计（本书用语；15288 的规范过程名是 **System architecture definition process**〔§6.4.4〕，正本无 `architectural design` 一词，见 ch11 §11.2.2）——架构活动覆盖整个生存周期，架构设计只是其中一段

⑦ 辨析　架构活动 ≠ 架构师头衔——42010 只认活动不认头衔；本书讨论"从事架构活动的人"（架构师），标准讨论"架构活动"本身

⑨ 英文　architecting = 42010 给 architecture 造的过程词（architecture 当动词用会硌牙，才造出 architecting）

② 背景　六块能力增量（架构决策/关注点管理/质量权衡/抽象建模/技术领导/演化对齐）收敛到同一个母词——动词链的投影

③ 历史　ISO/IEC/IEEE 42010 立"architecting"（架构活动）词条，把原本混在"架构设计"里的活动拆成七动词；中文定译"架构活动"（封面译法源同步 42020）

④ 问题　没有母词，架构师的能力只能罗列成碎片清单——有了动词链，每一项能力都能在链上找到位置

⑤ 场景　贯穿系统生存周期反复发生（不是串行阶段）；ch14 治理取链尾两项（认证=符合性监控、维护改进=演化治理），ch18 架构师取全链

⑧ 误区　✗ "架构活动 = 架构设计"——设计只是七动词之一；✗ "架构师 = 42010 头衔"——标准只认活动，不认职位

判据　一个动作能在七动词链上找到位置吗？在生存周期里反复发生吗？—能，才是架构活动（本书凝练：动词链为母词映射，非标准原文）
概念档案 · 创新

① 定义　创造或重新分配价值的新对象或变更对象（ISO 56000:2020）—结果义：新 × 成，缺一归零；只有新没成=点子/发明，只有成没新=优化/复现

⑥ 对立　发明—发明是新想法的诞生，创新是新组合的兑现（熊彼特）；发明在链上（构思），创新在链尾（评分席）

⑦ 辨析　创新 ≠ 构思（构思供给"新"的因子，"成"的因子大头在定义与设计）；≠ 创意/点子（没落地的改进不叫创新，奥斯陆手册）

⑨ 英文　innovation ← 拉丁 innovare（更新）；ISO 56000 取名词义(new or changed entity)，熊彼特取活动义—本书统一取结果义

② 背景　创新管理是经济学/管理学话题（熊彼特、ISO 56000 家族），工程动作（构思/定义/设计）是它的生产现场

③ 历史　熊彼特 1912《经济发展理论》把创新从发明中剥出；Henderson & Clark 1990 四分型（突破式/架构式/模块式/渐进式）指出主导动作随类型在构思/定义/设计间滑动

④ 问题　把"创新"当动词用（"我们去创新吧"= 语病）—你能做的是构思/定义/设计/开发，创新是对动作结果的追认

⑤ 场景　链上的活干完了，够新又成了，才被追认为创新；工程化对象是流程/方法/组织/组合（ISO 56001 可认证），单点命中不可承诺

⑧ 误区　✗ "创新 = 灵感 = 构思"—架构式创新的主导动作恰是定义（元件不动、连接重排、重新画边界）；✗ "创新可被保证"—工程化管漏斗的形状与流量，不管哪一滴水通过

判据　够新吗（新颖性）× 成了吗（价值兑现）？缺一归零—两个都答"是"，才追认创新（本书凝练）

# 附录 C 误解清单：全部反直觉命题速查

> 每个概念一章，把”流行误解”和”精确概念”并排钉死——本书的误解清单完整版。读完正文，用这一页检验自己有没有把话说准；读正文之前，也可以用它当预习。

| 章 | 流行误解（病） | 精确概念（药） | 一句话判据 |
| --- | --- | --- | --- |
| 第 1 | “背熟定义、记住更多词 = 懂了这个概念；懂概念靠天赋” | **把话说准，是学习能力里回报最高的那一步**：这条乘法律的两个归零项是**方向**（先抓哪个词）与**表征**（表征是 1、上面三级是 0——1 定成立，0 定量级）；概念是网（一般 ⊃ 核心 ⊃ 根）；根概念能力是方向盘；概念体系能力 = 识别枢纽×织关系×迁移×辨边界；能力四级（表征 ⊃ 图式 ⊃ 心智模型 ⊃ 解释框架）；四问答全才算拥有 | 报出一个概念的两个坐标：**位次在哪一层、能力说到第几级**；先立枢纽再填细节；能答全四问才算懂；表征没拿到，上面盖多少层都白搭 |
| 第 2 | “用了敏捷 + CI/CD + 测试，我们就在做软件工程”；“工程 | 工程是**工程科学、方法论、工具与实践、价值与伦理同时在场**的 | **试金石**：换个人、换团队，用同样的需求和约束还能交付质 |

# 附录 D 参考文献总表（单一权威）

> 全书参考文献的**单一权威**：D.1 标准 + D.2 经典文献 + **D.3 内源**（2026-09-26 新增第三类），每条带编号（R-xx）与「出现章节」。各章章末「参考文献」节 = 本表按「出现章节」切片的视图；**新增正文引用 → 先在本表登记编号 → 再同步章内切片**。查”这条原理出自哪个标准”到这里；查”这个概念怎么定义”回附录 A/B。版本号为正文实际引用的版本。
> **D.3 与上两类性质不同**：它记的是本书**据以立说的研习笔记与参考课——只列、不给 R 编号、不作引用依据**（正文不得写”据某某课”），作用是把”本章的框架从哪来”交代清楚，不占参考文献位。

## D.1 标准

| 编号 | 标准 | 主题 | 本书锚定概念 | 关键条款/用途 | 出现章节 |
|---|---|---|---|---|---|
| R-01 | ISO 1087:2019 | 术语学·词汇 | 概念 | 概念定义（§3.2.7）："由特性的独特组合而形成的知识单元"；definition = 用以把一个概念与相关概念区分开来的表述（"画边界"的跨域印证） | 第1、11章 |
| R-02 | ISO 9000:2015 | 质量管理体系·基础与术语 | 需求/设计开发/质量/评审/验证/确认/目标/特性/等级/符合性/产品/服务/数据/信 | §3.6.1 对象；§3.6.4 需求与规定需求；§3.4.8 设计开发；§3.6.2 质量； | 第6、7、17章 |
| R-03 | ISO/IEC/IEEE 15288:2023 | 系统与软件工程·生命周期过程 | 生命周期/阶段/过程/决策门/利益相关者/[illegible] | §4.4.1 过程与过程实施；§5.3.3 过程实施策略；§6.4.1 生命周期模型管理过程；§6.4.2 基础设施管理过程；§6.4.3 组织项目使能管理过程；§6.4.4 投资管理过程；§6.4.5 组织过程改进过程；§6.4.6 人力资源管理过程；§6.4.7 质量管理过程；§6.4.8 知识管理过程；§6.4.9 项目组合管理过程；§6.4.10 采购过程；§6.4.11 供应过程；§6.4.12 协议过程；§6.4.13 组织项目使能过程；§6.4.14 技术管理过程；§6.4.15 技术评估过程；§6.4.16 需求定义过程；§6.4.17 系统分析过程；§6.4.18 架构定义过程；§6.4.19 设计定义过程；§6.4.20 系统实现过程；§6.4.21 集成过程；§6.4.22 验证过程；§6.4.23 转换过程；§6.4.24 运行过程；§6.4.25 维护过程；§6.4.26 处置过程；§6.4.27 质量保证过程；§6.4.28 项目规划过程；§6.4.29 项目评估与控制过程；§6.4.30 决策管理过程；§6.4.31 风险管理过程；§6.4.32 配置管理过程；§6.4.33 信息管理过程；§6.4.34 测量过程；§6.4.35 沟通过程；§6.4.36 业务或任务分析过程；§6.4.37 利益相关者需求定义过程；§6.4.38 系统需求定义过程；§6.4.39 架构设计过程；§6.4.40 实现过程；§6.4.41 集成过程；§6.4.42 验证过程；§6.4.43 转换过程；§6.4.44 运行过程；§6.4.45 维护过程；§6.4.46 处置过程；附录A 裁剪指南；附录B 生命周期概念；附录C 生命周期模型；附录D 过程目的与结果 | 第1、2、3、4、5、6、7、8、9、10、11、12、13、14、15、16、17、18、19、20、21、22、23、24、25、26、27、28、29、30、31、32、33、34、35、36、37、38、39、40、41、42、43、44、45、46、47、48、49、50、51、52、53、54、55、56、57、58、59、60、61、62、63、64、65、66、67、68、69、70、71、72、73、74、75、76、77、78、79、80、81、82、83、84、85、86、87、88、89、90、91、92、93、94、95、96、97、98、99、100、101、102、103、104、105、106、107、108、109、110、111、112、113、114、115、116、117、118、119、120、121、122、123、124、125、126、127、128、129、130、131、132、133、134、135、136、137、138、139、140、141、142、143、144、145、146、147、148、149、150、151、152、153、154、155、156、157、158、159、160、161、162、163、164、165、166、167、168、169、170、171、172、173、174、175、176、177、178、179、180、181、182、183、184、185、186、187、188、189、190、191、192、193、194、195、196、197、198、199、200、201、202、203、204、205、206、207、208、209、210、211、212、213、214、215、216、217、218、219、220、221、222、223、224、225、226、227、228、229、230、231、232、233、234、235、236、237、238、239、240、241、242、243、244、245、246、247、248、249、250、251、252、253、254、255、256、257、258、259、260、261、262、263、264、265、266、267、268、269、270、271、272、273、274、275、276、277、278、279、280、281、282、283、284、285、286、287、288、289、290、291、292、293、294、295、296、297、298、299、300、301、302、303、304、305、306、307、308、309、310、311、312、313、314、315、316、317、318、319、320、321、322、323、324、325、326、327、328、329、330、331、332、333、334、335、336、337、338、339、340、341、342、343、344、345、346、347、348、349、350、351、352、353、354、355、356、357、358、359、360、361、362、363、364、365、366、367、368、369、370、371、372、373、374、375、376、377、378、379、380、381、382、383、384、385、386、387、388、389、390、391、392、393、394、395、396、397、398、399、400、401、402、403、404、405、406、407、408、409、410、411、412、413、414、415、416、417、418、419、420、421、422、423、424、425、426、427、428、429、430、431、432、433、434、435、436、437、438、439、440、441、442、443、444、445、446、447、448、449、450、451、452、453、454、455、456、457、458、459、460、461、462、463、464、465、466、467、468、469、470、471、472、473、474、475、476、477、478、479、480、481、482、483、484、485、486、487、488、489、490、491、492、493、494、495、496、497、498、499、500、501、502、503、504、505、506、507、508、509、510、511、512、513、514、515、516、517、518、519、520、521、522、523、524、525、526、527、528、529、530、531、532、533、534、535、536、537、538、539、540、541、542、543、544、545、546、547、548、549、550、551、552、553、554、555、556、557、558、559、560、561、562、563、564、565、566、567、568、569、570、571、572、573、574、575、576、577、578、579、580、581、582、583、584、585、586、587、588、589、590、591、592、593、594、595、596、597、598、599、600、601、602、603、604、605、606、607、608、609、610、611、612、613、614、615、616、617、618、619、620、621、622、623、624、625、626、627、628、629、630、631、632、633、634、635、636、637、638、639、640、641、642、643、644、645、646、647、648、649、650、651、652、653、654、655、656、657、658、659、660、661、662、663、664、665、666、667、668、669、670、671、672、673、674、675、676、677、678、679、680、681、682、683、684、685、686、687、688、689、690、691、692、693、694、695、696、697、698、699、700、701、702、703、704、705、706、707、708、709、710、711、712、713、714、715、716、717、718、719、720、721、722、723、724、725、726、727、728、729、730、731、732、733、734、735、736、737、738、739、740、741、742、743、744、745、746、747、748、749、750、751、752、753、754、755、756、757、758、759、760、761、762、763、764、765、766、767、768、769、770、771、772、773、774、775、776、777、778、779、780、781、782、783、784、785、786、787、788、789、790、791、792、793、794、795、796、797、798、799、800、801、802、803、804、805、806、807、808、809、810、811、812、813、814、815、816、817、818、819、820、821、822、823、824、825、826、827、828、829、830、831、832、833、834、835、836、837、838、839、840、841、842、843、844、845、846、847、848、849、850、851、852、853、854、855、856、857、858、859、860、861、862、863、864、865、866、867、868、869、870、871、872、873、874、875、876、877、878、879、880、881、882、883、884、885、886、887、888、889、890、891、892、893、894、895、896、897、898、899、900、901、902、903、904、905、906、907、908、909、910、911、912、913、914、915、916、917、918、919、920、921、922、923、924、925、926、927、928、929、930、931、932、933、934、935、936、937、938、939、940、941、942、943、944、945、946、947、948、949、950、951、952、953、954、955、956、957、958、959、960、961、962、963、964、965、966、967、968、969、970、971、972、973、974、975、976、977、978、979、980、981、982、983、984、985、986、987、988、989、990、991、992、993、994、995、996、997、998、999、1000、1001、1002、1003、1004、1005、1006、1007、1008、1009、1010、1011、1012、1013、1014、1015、1016、1017、1018、1019、1020、1021、1022、1023、1024、1025、1026、1027、1028、1029、1030、1031、1032、1033 | §4.4.1 过程与过程实施；§5.3.3 过程实施策略；§6.4.1 生命周期模型管理过程；§6.4.2 基础设施管理过程；§6.4.3 组织项目使能管理过程；§6.4.4 投资管理过程；§6.4.5 组织过程改进过程；§6.4.6 人力资源管理过程；§6.4.7 质量管理过程；§6.4.8 知识管理过程；§6.4.9 项目组合管理过程；§6.4.10 采购过程；§6.4.11 供应过程；§6.4.12 协议过程；§6.4.13 组织项目使能过程；§6.4.14 技术管理过程；§6.4.15 技术评估过程；§6.4.16 需求定义过程；§6.4.17 系统分析过程；§6.4.18 架构定义过程；§6.4.19 设计定义过程；§6.4.20 系统实现过程；§6.4.21 集成过程；§6.4.22 验证过程；§6.4.23 转换过程；§6.4.24 运行过程；§6.4.25 维护过程；§6.4.26 处置过程；§6.4.27 质量保证过程；§6.4.28 项目规划过程；§6.4.29 项目评估与控制过程；§6.4.30 决策管理过程；§6.4.31 风险管理过程；§6.4.32 配置管理过程；§6.4.33 信息管理过程；§6.4.34 测量过程；§6.4.35 沟通过程；§6.4.36 业务或任务分析过程；§6.4.37 利益相关者需求定义过程；§6.4.38 系统需求定义过程；§6.4.39 架构设计过程；§6.4.40 实现过程；§6.4.41 集成过程；§6.4.42 验证过程；§6.4.43 转换过程；§6.4.44 运行过程；§6.4.45 维护过程；§6.4.46 处置过程；附录A 裁剪指南；附录B 生命周期概念；附录C 生命周期模型；附录D 过程目的与结果 | 第1、2、3、4、5、6、7、8、9、10、11、12、13、14、15、16、17、18、19、20、21、22、23、24、25、26、27、28、29、30、31、32、33、34、35、36、37、38、39、40、41、42、43、44、45、46、47、48、49、50、51、52、53、54、55、56、57、58、59、60、61、62、63、64、65、66、67、68、69、70、71、72、73、74、75、76、77、78、79、80、81、82、83、84、85、86、87、88、89、90、91、92、93、94、95、96、97、98、99、100、101、102、103、104、105、106、107、108、109、110、111、112、113、114、115、116、117、118、119、120、121、122、123、124、125、126、127、128、129、130、131、132、133、134、135、136、137、138、139、140、141、142、143、144、145、146、147、148、149、150、151、152、153、154、155、156、157、158、159、160、161、162、163、164、165、166、167、168、169、170、171、172、173、174、175、176、177、178、179、180、181、182、183、184、185、186、187、188、189、190、191、192、193、194、195、196、197、198、199、200、201、202、203、204、205、206、207、208、209、210、211、212、213、214、215、216、217、218、219、220、221、222、223、224、225、226、227、228、229、230、231、232、233、234、235、236、237、238、239、240、241、242、243、244、245、246、247、248、249、250、251、252、253、254、255、256、257、258、259、260、261、262、263、264、265、266、267、268、269、270、271、272、273、274、275、276、277、278、279、280、281、282、283、284、285、286、287、288、289、290、291、292、293、294、295、296、297、298、299、300、301、302、303、304、305、306、307、308、309、310、311、312、313、314、315、316、317、318、319、320、321、322、323、324、325、326、327、328、329、330、331、332、333、334、335、336、337、338、339、340、341、342、343、344、345、346、347、348、349、350、351、352、353、354、355、356、357、358、359、360、361、362、363、364、365、366、367、368、369、370、371、372、373、374、375、376、377、378、379、380、381、382、383、384、385、386、387、388、389、390、391、392、393、394、395、396、397、398、399、400、401、402、403、404、405、406、407、408、409、410、411、412、413、414、415、416、417、418、419、420、421、422、423、424、425、426、427、428、429、430、431、432、433、434、435、436、437、438、439、440、441、442、443、444、445、446、447、448、449、450、451、452、453、454、455、456、457、458、459、460、461、462、463、464、465、466、467、468、469、470、471、472、473、474、475、476、477、478、479、480、481、482、483、484、485、486、487、488、489、490、491、492、493、494、495、496、497、498、499、500、501、502、503、504、505、506、507、508、509、510、511、512、513、514、515、516、517、518、519、520、521、522、523、524、525、526、527、528、529、530、531、532、533、534、535、536、537、538、539、540、541、542、543、544、545、546、547、548、549、550、551、552、553、554、555、556、557、558、559、560、561、562、563、564、565、566、567、568、569、570、571、572、573、574、575、576、577、578、579、580、581、582、583、584、585、586、587、588、589、590、591、592、593、594、595、596、597、598、599、600、601、602、603、604、605、606、607、608、609、610、611、612、613、614、615、616、617、618、619、620、621、622、623、624、625、626、627、628、629、630、631、632、633、634、635、636、637、638、639、640、641、642、643、644、645、646、647、648、649、650、651、652、653、654、655、656、657、658、659、660、661、662、663、664、665、666、667、668、669、670、671、672、673、674、675、676、677、678、679、680、681、682、683、684、685、686、687、688、689、690、691、692、693、694、695、696、697、698、699、700、701、702、703、704、705、706、707、708、709、710、711、712、713、714、715、716、717、718、719、720、721、722、723、724、725、726、727、728、729、730、731、732、733、734、735、736、737、738、739、740、741、742、743、744、745、746、747、748、749、750、751、752、753、754、755、756、757、758、759、760、761、762、763、764、765、766、767、768、769、770、771、772、773、774、775、776、777、778、779、780、781、782、783、784、785、786、787、788、789、790、791、792、793、794、795、796、797、798、799、800、801、802、803、804、805、806、807、808、809、810、811、812、813、814、815、816、817、818、819、820、821、822、823、824、825、826、827、828、829、830、831、832、833、834、835、836、837、838、839、840、841、842、843、844、845、846、847、848、849、850、851、852、853、854、855、856、857、858、859、860、861、862、863、864、865、866、867、868、869、870、871、872、873、874、875、876、877、878、879、880、881、882、883、884、885、886、887、888、889、890、891、892、893、894、895、896、897、898、899、900、901、902、903、904、905、906、907、908、909、910、911、912、913、914、915、916、917、918、919、920、921、922、923、924、925、926、927、928、929、930、931、932、933、934、935、936、937、938、939、940、941、942、943、944、945、946、947、948、949、950、951、952、953、954、955、956、957、958、959、960、961、962、963、964、965、966、967、968、969、970、971、972、973、974、975、976、977、978、979、980、981、982、983、984、985、986、987、988、989、990、991、992、993、994、995、996、997、998、999、1000、1001、1002、1003、1004、1005、1006、1007、1008、1009、1010、1011、1012、1013、1014、1015、1016、1017、1018、1019、1020、1021、1022、1023、1024、1025、1026、1027、1028、1029、1030、1031、1032、1033

R-102 | ISO/IEC/IEEE 15288:2023 | 系统与软件工程·系统生命周期过程 | 需求侧过程（§6.4.1~§6.4.3）／**系统架构定义**（§6.4.4）／设计定义（§6.4.5）／人工制品（§3.6） | **需求侧三个过程**（第 4 章 §4.2.1 的”需求工程过程锚点”）：§6.4.1 业务或任务分析（**任务与目的**在那里定义）、§6.4.2 利益相关方需要与需求定义（其注 1 指向 29148：该标准给出更低层级细化与三份文档的带注释大纲）、§6.4.3 系统需求定义——需求工程做的是后两步，第一步是它的输入；**§6.4.2.3 的三条注**（第 4 章用）：注 5「开发运行概念及其他生存周期概念」是 §6.4.2 的一项活动（系统运行概念的带注释大纲在该标准附录 A）、注 24 需求以适合需求管理的形式记录→建立需求基线→保留需要的变化及其来源→四向追溯（第 4 章 §4.2.6）、注 26 单条需求特性与需求集特性（第 4 章 §4.4.4 六条判据的库内邻近表述）；**第 9 章 §9.2.8 的”把需要与期望开发成规定需求、再管住它一直是那一份”口径取自 §6.4.2.2 e)／i) 与 §6.4.2.3 注 24**；**两个过程的目的句**：架构定义＝“生成系统架构的不同方案，选择一个或多个能够满足利益相关方关切和系统需求的方案，并以一致的视图和模型进行表达”；设计定义＝“提供关于系统和其元素的足够详细的数据和信息，以便根据系统需求和架构实现解决方案”；设计定义的四组活动（准备／开展／评估／管理，§6.4.5.3 a)~d)）、**两条转化**（架构实体与关系→设计元素、架构特性→设计特性，b) 2)、b) 3) 逐字）、

分配（须满足系统需求**和架构目标**，b) 1)＋注 4)、设计演化原则（a) 4)）；**注意分清出处——设计描述符**（`design descriptor`）、**原子元素**两条件（`atomic`／`black box`）、**横向与纵向集成**三项**正本检索零命中**，属 **INCOSE SEH 5.0 专有表述**（正本只给邻近句 §6.4.5.3 a) 3) 注 1／b) 4) 注 7／§5.2.2，见 R-45）；**§5.9（概念与系统定义）另有一句为两概念辨正的依据**——an architecture entity … and a system element … represent two different notions（架构实体≠系统元素，见第 10、15、16 章）；**§5.9 还给出两过程的分工句与「有效的架构尽可能不依赖于设计」的规范位**——Architecture focuses on suitability, viability, and desirability, whereas design focuses on compatibility with technologies and other design elements and on feasibility of construction and integration. An effective architecture is as design-agnostic as possible to allow for maximum flexibility in the design trade space.（见第 16 章）；**§3.13 `design, noun`＝“合规实现”的一手出处**——specification of system elements (3.47) and their relationships, that is sufficiently complete to support a compliant implementation of the architecture (3.5)（见第 16 章）；§3.47 系统元素＝discrete part of a system that can be implemented to fulfil specified requirements（§5.9 行文中的简式作 a discrete part of a system

that fulfils specified requirements）；**§3.6 `artefact`＝工作产品**——work product (3.32) that is produced and used during a project to capture and convey information（与 42010:2022 §3.3 注 1 的 `work product` **互为属概念、方向相反**，分流译名依”所属标准不同、属概念层级不同”；见第 8 章 §8.3.6 术语注）；**§6.4.14 处置过程**（退役与回收，第 3 章 §3.3.5）；**上述条款均直核 2023 版正本**，INCOSE SEH 5.0 中文版（与 15288:2023 同版同年）退为中文对读旁证 | 第 2、3、4、5、7、8、10、16 章 |

R-103 | ISO 9000:2026（第 5 版）| 质量管理体系·基础与术语 | 评审／验证／确认；设计和开发；目标 | §3.11.2 评审＝“为达成既定目标，对客体的适宜性、充分性或有效性所作的确定”（设计定义是工作产品：**可评审，也可验证**——15288:2023 §6.4.9 的验证对象含 artefact；**做不了确认**）；§3.11.12 验证对”规定的需求”；§3.11.14 确认对”针对特定预期用途或应用的需求”（注 3：使用条件可真实可模拟）——三把尺子的界分是本章审核动作定名的依据；**§3.5.1 注 2 规定的需求是明示的需求，例如写在成文信息中的需求**（第 4 章：`e.g.` 是举例——**明示即规定，成文属可采条件**，第 4 章 §4.2.5 与 §4.3.1 据此改写理由）；**§3.3.11 设计和开发＝“将某一客体的需求转换为该客体更详细需求的一组过程”**（第 4 章 §4.2.4 目标→需求”转化”的依据；第 11 章 §11.2.2 定义句改挂此条）；**§3.7.11 objective＝“要**

**实现的结果”**（第 11 章 §11.4.3 目标口径，去”可度量状态”这一加工）；**§3.5.2 质量、§3.5.3 客体、§3.5.4 等级、§3.5.5 法定要求、§3.5.8 法规要求、§3.5.9 合格**（第 3 章三个词与三处边界的条款依据）；**§3.10.6 competence**（第 9 章**：应用知识和技能以取得预期结果的能力）**、§3.3.11 设计和开发（第 9 章 §9.2.3**：引它说”一组过程就是一套方法”，本书读法）**、§3.5.1＋注 2（第 9 章 §9.2.8**：“需要与期望→规定需求”三词口径）**、§3.11.1 确定＋§3.11.2／§3.11.12／§3.11.14 三判定（第 9 章 §9.2.8／§9.4.7**：“判定”简称的锚）**、**§3.10.1 特性、§3.8.6 客观证据、§3.2.11 项目**（第 3 章章末切片的其余五条）| 第 1、2、3、4、5、**6**、7、8、**9**、11、16、17 章 | R-104 | ISO/IEC/IEEE 24765:2017 | 系统与软件工程·词汇 | design（宽义·术语学旁证）| **术语学旁证（2026-09-25 降级）**：本章”合规实现”一语的**一手出处已改挂 R-102／ISO/IEC/IEEE 15288:2023 §3.13**，不必绕道本标准；本标准保留的理由只剩”**宽义 design** 的术语学来源”——其 design 定义句含 define **the architecture**（经 ISO/TS 14812 §3.1.15.1 `design` 注 1 转引，页面自标源即本标准，逐字 It is information including specification of system elements and their relationships, that is sufficiently complete to support a compliant implementation of the architecture）| 第 16 章 |

R-105 | ISO/TS 14812:2025 | 智能运输系统·词汇（术语批次）| 实现／兑现／设计（宽义）——**术语学旁证** | §3.1.15.2 `implementation` ＝把设计翻译为硬件部件、软件部件或两者的过程（或该过程的结果）；§3.1.15.4 `realization`＝**implementation of a system concept**（注 1: 兑现需要开发出一份设计、并把该设计翻译为部件）；链条为 concept → design（开发出规格）→ implementation（翻译成部件）→ realization（系统概念实现）——**“合规实现”的承担者在 implementation 侧**即据此旁证；该批术语 History note 记“Introduced in ISO/TS 14812:**2025**”，与 2022 批（architecture／concern／viewpoint 等）并存于同一词汇站 | 第 16 章 |

## D.2 经典文献（非标准）

| 编号 | 文献 | 出现章节 | 贡献 |
| --- | --- | --- | --- |
| R-22 | 索绪尔《普通语言学教程》 | 第 1 章 | 符号 = 能指 + 所指；语言（langue）与言语（parole） |
| R-23 | 弗雷格《论涵义与指称》（1892） | 第 1、10 章 | 符号 → 涵义 → 指称；系统≠架构≠架构描述的哲学底座 |
| R-24 | Xiangjuan Ren 等《Compressive learning scaffolds higher-order network structure to enhance human knowledge acquisition》（Nature Communications, 2026） | 第 1 章 | 压缩学习：枢纽优先的学习结构；人脑是压缩机器不是存储器 |
| R-25 | Bloom 教育目标分类学 | 第 1 章 | 五环模型掌握线 |
| R-55 | Nyga[illegible]ting[illegible]8 | [illegible]8 | [illegible]——“会用才算[illegible]掌握”：掌握线划 |
| | 1140 | | |
| | an Effe[illegible]rator》（Anderson & [illegible], | 18 章 | |

## D.3 内源（研习笔记与参考课）

| 内源 | 底本 | 本书据以立说的内容 |
| --- | --- | --- |
| 31-现代思维工具 100 讲（本研究报告，2026-09-21） | 底本＝**R-106**（《万维钢·现代思维工具 100 讲》得到 App 课程） | **能力四级**——表征 → 图式 → 心智模型 → 解释框架（第 41 讲；“图式”作为把多个元素打包成占 1 个槽位的压缩包，第 38 讲）；**参与度 P<A<C<I 与”讲给外行听”只到 C 级**（第 39 讲）；**迁移是验收标准与三道锁**（第 47 讲）；**默会知识是显性知识的地基**（第 43 讲）。**落点＝§1.4.4**（全书 18 章章末刻度行皆回指此节），并按讲次逐条可核 |
| 00-概念学习方法论（本研究报告） | 北京大学团队压缩学习论文（已列 R-24）与 ISO 术语学（已列 R-01） | 概念／根概念／判定矩阵／术语纪律（知识本体侧）；与 31 号是同一现象的两条证据路线 |
| 20250326-高效研制优质低成本的产品-正向设计（培训材料，本研究报告） | —（**素材正源**，非研习笔记） | **三种样机与”真正的产品”的对照（§5.1）** |
| 20250326-高效研制优质低成本的产品-正向设计（培训材料，本研究 | —（**素材正源**，非研习笔记） | **三种样机与”真正的产品”的对照（§5.1）／可实现性的八个方面（§5.3）／功能需／可实现性的八个方** |

> **体例**：本表只列、**不给 R 编号**，故**不进各章章末「参考文献」切片**；正文引用一律走 D.1／D.2。登记纪律同 D：新增内源 → 先在本表登记 → 再同步框架 §三「来源」列。

# 附录 E 练习答案

> 各章”自查 · 这些说法对吗”的答案与理由已随章给出，下表只给**对/错**速查；“练习”的**答案要点**在下。要点给的是”往哪个方向想、用哪把尺子”，不是唯一答案——它们与训练教材题卡同源，考的是你有没有把话说准。

## 第 1 章 概念体系能力：学习能力的杠杆

**自查速查**：① × ② × ③ × ④ × ⑤ × ⑥ × ⑦ × ⑧ × ⑨ × ⑩ × ⑪ × ⑫ × ⑬ × ⑭ × ⑮ × ⑯ ×（背熟定义≠懂，懂=能答四问；概念能力=组织得好非渊博；根概念能力是概念体系能力的子能力非全集；乘号里有两个归零项（方向与表征），力臂长≠产出大；先学细节越学越累；转述=压缩测试；四问答出”是什么”不够；概念是共识不是真理；掌握=结构不是记忆；识别枢纽有可操作的三步法（收集候选→硬判据→往上挂）；多数概念是凝结不是发明；枢纽是骨架还有根概念/原理层；根概念是最源头非最重要；概念是节点、原理是边，核心原理之上还有基本原理；表征是 1、上面三级是 0——先建图式不能替代

表征；答全四问只是**入场**——四问是第一级（表征）的作业工具，上面还有心智模型与解释框架）

**练习答案要点**：1. 四问量词 → 能答全四问才算拥有；最薄的一问（通常”不是什么/往哪去”）正是误解潜伏处；量完再报坐标：位次在哪一层、能力练到第几级。2. 三步识别法找核心概念 → 收集候选（目录/术语表/专家/AI）→ 硬判据（词频/被引/前置数/反事实）→ 往上挂；再试着用判定矩阵辨出其中哪几个是根概念；清单是活的。3. 转述测试 → 卡住=结构没成型/关系没通；转述是”压缩测试”，费曼技巧的机制正是压缩学习。4. 五环自检 → 多数人停在”知道/理解”；掌握线在”应用”之上；逼输出暴露缺口、落实践淬炼结构。5. 四问重开会 → 十张脸里九个各指一个”它”：组织（黄程）、系统（万辉）、产品（文正）、硬件（水忠）、云端（董勐）、接口（小白）、部署（玉杰）、测试（李旭）、制造（逸人）；徐漾没给”架构”指派，他点破的是”你们在说五件不同的事”。先分清谁在说哪个所指，同一所指的才能合并谈，再谈该不该改。6. 概念溯源 → 凝结=共同体长期实践结晶被固化（如 baseline）；创造/发现见第一部分；多数是凝结。7. 勤校准 → 概念是共识不是真理；枢纽可修正；往往是第 3 环”逼输出”暴露了旧框架的错。8. 乘法律诊断 → 四部件各缺什么？两个归零项是方向与表征：方向错了，力臂越长偏得越远；表征没拿到，上面盖多少层都白搭；支点（落实践）最常被缺，力臂再长也使不上劲。9. 迁移测试 → 换行业／换粒度看那条判据还成不成立；认得出”同一个

结构换了张脸”＝已从图式走向心智模型；只认得原来那个场景＝还停在图式层。

## 第 2 章 工程：我们到底在做什么

**自查速查**：①×②×③×④×⑤×⑥×⑦◐（显性化是第一步，还要可检验）⑧×

**练习答案要点**：1. 试金石检验 → 换三个人做，能否同等交付；不能 → 缺的是可复制性，对照四层定位（多是方法论层或价值与伦理层）。2. 敏捷+CI/CD+测试覆盖方法论、工具与实践两层；缺工程科学（“为什么能行”）与价值与伦理（压力下守不守）。3. systematic 答”怎么做事有章法”、systemic 答”怎么看问题有全局”、algorithmic 答”能否自动执行”；例：敏捷迭代是有章法的判断，不是算法执行。4. 系统工程 = 元工程（整合部件、不造部件）；“部件全对、整体出事” = 涌现属性，去查没人管的接口/数据口径。5. 技术系统看：上线 = 成功；工程系统看：人会用、流程顺、组织撑得住 = 成功；答案不一致 → 缺的是人/流程/组织/数据。机场就是现成例子：物理、技术、人、流程、信息协同起来，才产生”顺利运行”的涌现行为，工程系统的边界画在整个运行环境上。6. 验收记录：对上”规定需求”的是验证，对上”预期用途”的是确认。最常见的病是”验证全过、确认没做”——测试全绿只说明”做对了”，没人用仍然可能失败（§2.4.4）。7. 三大事故共同点 = 价值与伦理这一层先松手（不是不会做，是没守住）；找”为进度牺牲可靠性”的真实例子。8. 水忠 = 手艺（依赖个人）；

工程化改造第一步 = 把”心里有数”的判断显性化成可检验的规则/文档。9. 六方面打分：恒信有结构/有方法/可重复**沾了边**；全覆盖/受控/可验证不合格（缺三个）。10. 最值钱的手艺 → 显性化：把”心法”写成可检验的规则，再验证换个人能按它交付。 11. 四要素打分：科学原理、约束、系统方法、有用且安全的人造物；缺”有用且安全”（价值与伦理层失守）最致命。12.“工程 ≠ 手艺”转述测试 → 量的是”工程”的”不是什么”（对立：手艺），用第 1 章转述测试检验——讲不清”换个人也能同等交付”就卡在”试金石”上；溯源：engineering ← 拉丁 ingenium（天生的才智、巧思），ECPD 1941 首次把”造东西”提炼成可检验的要素。

## 第 3 章 需求、设计开发、质量：产品开发的图式

**自查速查**：① × ② × ③ × ④ × ⑤ √ ⑥ × ⑦ × ⑧ × ⑨ ×

**练习答案要点**：1. 点子不算输入 → 要经研究确认成需求；分界线 = 是否表述为”明示 / 通常隐含 / 必须履行”的需求或期望（ISO 9000:2026 §3.5.1）。 2. 技术方案做完不够 → 缺采购/制造/测试/交付/部署/操作/运维等方案；终点由”最后一个配齐的方案”决定。 3. 特性超出需求范围 = 过度设计，只加成本、不加质量（质量 = 固有特性满足需求的程度，超出需求范围的特性对质量没有贡献）。 4. 本质 = 需求集合内部打架（交付时间也是需求）；砍验证 = 不度量就宣布合格，风险后移成现场返工；口头改需求 = 偷换靶子。5. 一起剪裁 = 需求所有权在利益相关方 + 走正式变更（排序/留痕/挂账）；单方面砍 =

偷换。6. 等级答”按哪套需求分类”、质量答”满足到什么程度”、符合性答”满足了吗”；高档车可以是差质量，因为等级高 ≠ 质量好。 7. 特性还没开始规定 = 需求开发（ISO 9001:2015 §8.2）；特性开始被规定 = 设计开发（§8.3）——需求规格就是那道分界。8. 起点判据”可转化”的四条前提：完整、无歧义、不冲突、隐含需求已识别；最弱那条的后果 = 错误需求被完整转化。9. 用四问的”它从哪来”量三个词 → 需求 ← 拉丁 requirere（要求、寻求），原指”要的东西”，后漂移出”明示/隐含/必须履行”三形态；设计开发 ← designare（标注、指明），ISO 9000:2026 以 design and development 完整表述”需求转化”一件事，不拆两半；质量 ← 拉丁 qualitas（“如何”），从”档次、用料”漂移成”符合需求的程度”——漂移的那个词是”质量”，正是本章最大反转。10.“定需求 → 出规格 → 判满足”走项目 → 定需求=需求（“可转化”的四条前提，隐含需求最易漏）；出规格=设计开发（方案配齐才到”可实现”，只到技术方案=没走完）；判满足=质量（靠测试方案里的接收准则认定，没有准则只能”感觉”）；三处边界易混处——起点看”特性有没有被规定”、终点看”还有哪个环节反问这到底怎么做”、参照看”超出需求范围的特性是过度设计”。

## 第 4 章 需求工程：从诉求到依据

**自查速查**：① × ② × ③ × ④ × ⑤ × ⑥ × ⑦ × ⑧ × ⑨ × ⑩ ×

**练习答案要点**：1. 开发欠账 = 没调研、没分析、没写可验证的规格；管理欠账 = 没基线、没变更流程、没追溯——两张单子按”源头

／过程”归类，“加强需求管理”只治得着后一张。2.“明示但不可采”→补上判定方法与版本：例”重要合同要审批”→“金额 ⩾ 500 万或三年期以上须审批”（注意它本来就是规定需求，缺的是可采，不是资格）。3. 跳过分析直接进规格的代价：澄清不足（“重要”没定义）、分解没做、排序没做、建模没做 → 验证没有可对照的靶子。 4. 改口叫评审的差别：评审对的是**工作产品**（判适宜、充分、有效），签的是”这份需求立得住”；叫成”确认”等于给还没实现的东西签了确认的字，验收时那句”确认通过”就成了扯皮的证据。5. ConOps 讲一段过程（系统怎么被用）、SRS 讲一条条条款（必须做什么）；ConOps 还兼着确认判据的载体（隐含需求的正文写在运行场景里）；先条款后故事，条款就悬空。6. 用六条判据打分 → 最弱一条及其后果：如”可追溯”最常被牺牲——它不立刻显形，变更时答不上”从哪来、影响谁”。7. 需求基线在哪；没有基线，最近一次变更”从什么变到什么”说不清。8. 最常被跳的一环是影响分析（口头改需求）；代价 = 查无实据的暗改，出事时拿不出凭证。9.“25 号之后签的合同算下个月”：访谈随口一句 → 挖出来 → 显性化进需求池 → 写成规定需求（隐含需求不会自己开口）。10.“合同数据留档十年” = MoSCoW 的 Must / Kano 的基本型——走的是环境与法规那条来路，不经组织目标，所以不在被砍清单里。11. 三条来路 = 目标转化（主流，明示）、法规与环境约束（**法规强制**那一支属”必须履行”、不经目标；“环境”那一支另立一类，

第 5 章承接）、利益相关方的隐含需要（通常隐含，未必被目标覆盖）；最容易被漏的是第三条——它不开口，也不在目标的覆盖范围里，只能靠需求工程自己补齐。

## 第 5 章 需求的完整性：谁在问，谁在验

**自查速查**：① × ② × ③ × ④ × ⑤ × ⑥ × ⑦ × ⑧ × ⑨ × ⑩ × ⑪ × ⑫ × ⑬ × ⑭ ×

**练习答案要点**：1. 冰链八条 → 八条可实现性需求一一对应：定制芯片＝物料可采购性、注塑做不出来＝可制造性、手工校准＝可测试性、无认证＝可交付性、无远程升级＝可部署性、无备件模块＝可运维性、电池回收＝可回收性、成本超四成＝经济可承受性。重做需求时这八条都该在需求阶段写出来，尤其经济可承受性——它是前七条共同堆出来的总账。2. 三类各三条 → 最少的那一块通常是可实现性；原因从四个错位里找（相关方、时间、话语权、认知），多半是承制方的代表没在需求阶段发声。3. 逐块问验证载体那条判据：功能需求用什么验（原理样机）、环境适应性需求用什么验（环境样机）、可实现性需求用什么验（真正的产品）——答不上来的那一块，就是欠账。4. 症状反推：每一个”不方便”对应一条可实现性需求；当时评审没人提，多半因为提它的那些人（承制方各业务代表）不在需求阶段的桌上。5. 测试性与可测试性：同一个特性两个主人——使用环境问”在役测得出吗”，归环境适应性需求；承制方问”产线上测得方便吗”，归可实现性需求；判据是谁在问，叫法跟着主人走。6. 访谈对象

清单：采购、制造、测试、交付、部署、运维、回收的业务代表，加市场代表，有没有在需求阶段发过声？没有，可实现性需求就会系统性漏失。7. 回路诊断（问题、转化、入库、复用）：卡点最常见在”转化”（问题停在口头，没提炼成带指标的需求条目）与”入库”（没人维护资产库）。 8. 经济可承受性：提出者应是市场代表（依据对竞品与替代产品的价格成本研究），要设目标成本一类的指标，并经相关方确认；若没有，“降本大会”反复开就是证据。 9. 恒信镜照：原型＝原理样机、试点＝环境样机、正式产品＝真正的产品；“上线成功”只走到功能那一块，部署、运维、成本这些属可实现性需求，要等真正的产品跑起来才验得了。10. 四问量”验证载体”或”承制方主导”：答全四问才算拥有。最薄的那一问通常就是最容易出错的地方——答不出”往哪去”，验收时会拿错交付物；答不出”从哪来”，就说不清为什么每一块内容要各配一个载体。

## 第 6 章 设计开发落地：需求与方案结对

**自查速查**：① × ② × ③ × ④ × ⑤ × ⑥ × ⑦ × ⑧ × ⑨ × ⑩ × ⑪ × ⑫ × ⑬ × ⑭ × ⑮ ×

**练习答案要点**： 1. 树 vs 散文判断：查”不可定位/不可承接/不可追溯”三罪各一个具体例子。 2. 画框架树：标分类节点（文件夹）与枝梢节点；给一个枝梢节点写承接判据（可承接哪类需求）＋分解源判据（分解产出哪类叶节点）——双向可用。 3. 语义四查给证据：覆盖完整（父要求都有对应）、不超出（无影子需求）、逻辑一致（不

打架）、约束继承（约束有叶子承接）。4. 指标三情形：判断”可拆分（函数关系）/ 下放（同值继承）/ 暂不分解（标注向下一层传递）“，写出判断过程。5. 分配三自检：全覆盖（叶节点数对账）、单承接（一个叶节点一个归宿）、N:1 合理（每个节点 2~5 条较优，以语义为准）。6. 悬空叶节点判断：叶节点切粗了（迭代重切／拆条）vs 方案树缺节点（生成新节点）——用”拆得开吗”区分。7. 覆盖检查＋符合性分析：先核”该节点承接的所有需求的语义 ⊆ 其叶节点语义之并”，再按”针对需求 A，本节点通过 X 机制满足 Y，指标 Z 满足/降级/丢失”落句——写不出就是没分析。8. 框架定义 vs 架构：一个是结构骨架（一份文档），一个是系统不可还原的涵义；“架构 = 一份文档”是词汇病。 9. 干枝叶子独立演进：修剪枝条只动框架、长新叶只动详细 → 变更成本下降，可分别纳入基线（第 13 章）。10. 结对走一遍：上层叶节点落到下层节点（复用／改进／新增各试一次）+ 覆盖检查/符合性分析，记录每步自检结果。11. 四问量”分配生成”（或”分解分析”）→ 分配生成是什么（叶节点落到能负责它的节点上＋自下而上调树：复用→改进→新增）、不是什么（不是简单分派——单承接是硬约束、资产优先有序）、从哪来（从分配实践与资产复用原则凝结）、往哪去（阶段一长骨架，是分解分析的前提）；最薄的一问通常是”不是什么”——把”分配=简单分派”当成它，正是最容易混用的地方。13. 判步型：两侧的叶节点不是同一批 → **分配步**（做承接，Y=F(X) 在此求值）；

两侧叶节点是同一批 → **整段搬入步**（不做映射，活是长干枝并逐级判定落位）；同一份产品数据在两个形态之间 → **形态轴步**（不是承接）。12. 六步走一遍：①接需求（需求就位：链首自原始材料结构化/链上验收上一对移交）、②验就位（三把尺子：可转化/可分配/边界不越位 + 两把数值尺子：指标闭合/分配预判）、③出方案（两个以上候选骨架）、④对需求（复用→改进→新增）、⑤做裁决（写下理由）、⑥签定稿（分解分析：节点→方案叶节点＋覆盖检查／符合性分析 → 利益相关方确认 → 冻结基线）——六步 = 三动作的展开：①②=需求就位（做+验）、③④⑤=分配生成（目标+做+裁决）、⑥=分解分析（做+核+封存）。

## 第 7 章 产品数据：开发产品就是开发产品数据

**自查速查**：① × ② √ ③ × ④ × ⑤ × ⑥ × ⑦ √ ⑧ × ⑨ × ⑩ √ ⑪ √ ⑫ × ⑬ × ⑭ × ⑮ × ⑯ ×

**练习答案要点**：1. 六类数据各两个工件：需求（需求规格/验收准则）、设计（图纸/架构描述）、制造（工艺文件/BOM）、质量（测试报告/验证证据）、服务（维护手册/故障数据）、管理（变更记录/基线标识）。2. 把数据当文档的例子：如”需求在共享盘里三个版本分不清”——数据是内容、文档是容器；开发全程在数据上工作，实物只是数据的复制品。3. 两域三态图：信息域（开发/验证）↔物理域（制造/使用），靠数字孪生映射；设计态（EBOM·纯数据）、制造态（MBOM·数据+实物）、使用态（维护记录·实物+实况）。4. 多环相乘分析：最终质量=需

求正确性×数据质量×制造质量×使用质量；找出断掉的那一环，数据质量在其中起”上限”作用。5. ISO 25012 的五把尺子逐项给证据：准确性（数据反映真实对象）/完整性（无遗漏）/一致性（不矛盾）/现时性（跟上现实）/可信度（经得起验证）。6. 出题人=客户（需求源头）、判卷人=用户（验收裁判）；四条路径：需要（源头）/使用（新数据）/反馈（修正）/验收（把关）。7. 价值三层：物化层（需求/架构描述 AD/实现工件，完全在数据里）、决策层（选型判断，ADR 部分沉淀）、过程层（沟通/评审，只留记录）；漏掉的那层怎么补（如决策补 ADR、过程补复盘）。8. 职责三步：识别工件（配置项清单）→ 指定 owner（一件只一人）→ 挂机制（权限/签字/审批/指标挂到 owner）。 9. 数据资产四类：需求资产（该做什么）/设计架构资产（怎么做得好）/验证资产（怎么证明对）/管理资产（为什么这么做）；判断能否跨产品复用。10. 实现方案八方案：技术/物料采购/制造/测试/交付/部署/操作/运维；最易漏的通常是交付/部署/操作/运维——只到技术方案就宣布终点，下游空转。 11. 流水线工位/工件：需求定义（需求数据）→设计（设计数据）→实现（实现产物）→验证（验证证据）→移交（基线）；缺陷放大例：需求错→设计跟着错→验证测错→量产全错，修复成本从 1 涨到 30~70 倍。 12. 增值判据分格：完善需求（内容增值）/加顾客不要的特性（过度加工）/审批草稿转受控、合规打标注满足规

则（状态增值）/走过场、数据没变（必要非增值）/重复造轮子（纯浪费）。

## 第 8 章 评审、验证与确认：目标、需求、用途

**自查速查**：① × ② × ③ × ④ × ⑤ × ⑥ × ⑦ × ⑧ × ⑨ × ⑩ × ⑪ × ⑫ × ⑬ ×

**练习答案要点**：1. 四维表：参照（目标/规定需求/预期用途）、动作（确定/证实/证实）、证据（不要求/必须/必须）、时机（事前事中/对结果/真实场景）各填一行。 2. 评审重做：先写目标（对象 + 时限 + 可衡量结果），再问适宜/充分/有效三问。3. “测试全绿”缺哪一要素：多为”证实”缺（只有结果没结论），或”对照”缺（需求没成文，尺子没有）。 4. 评审对目标：写出这条需求对应的目标，拿目标照需求——需求够吗？5. “测试全过、没人用”病根 = 隐含需求系统性漏失；验证射程在文档、确认射程在场景。6. 口头改的需求：定位它漏在哪一步（没进文档 → 验证够不着 → 确认没场景）。 7. 设计一次确认：真实用户 + 真实任务 + 真实场景 + 真实数据，写一份确认方案。8. V 模型：需求在哪一级分解，验证在哪一级回收（左右镜像）；只做最后一次性验证 = 中间层级缺陷藏身。 9. 评审管把关（事前/事中），复盘管改进（事后、绕回起点）；复盘驱动下一轮评审的目标。10. 通过标准会后现想 = 没有尺子；改成会前发出去、会后逐条打勾。 11. 三关走查：评审（对目标）过了吗、验证（对需求）过了吗、确认（对用途）过了吗；缺每一关的代价不同（白做/返工/没人用）。 12. 架构

评审四看（概念陈述 / 选定 + ADR / 视图·基线 / 回炉痕迹）缺哪样 = 材料半成品；对质三问答不上 = 没权衡；没背书 = 只是架构师个人意见（评审变汇报、异常不闭环 = 放行半成品）。

## 第 9 章 工程师：一份能力清单

**自查速查**：① × ② × ③ × ④ × ⑤ × ⑥ × ⑦ ◐（显性化只是第一步，还要可检验）⑧ × ⑨ × ⑩ × ⑪ × ⑫ × ⑬ × ⑭ × ⑮ × ⑯ × ⑰ ×

**练习答案要点**：1. 体检：逐项打勾，指出缺项，并说出缺这一项会走岔成哪一种（只会做不会说／只会说不会做／守不住底线／能力失控）。判据只有一条：换个人接得住——缺哪一项，就问哪一项。2. 粒度定位：按谱系（模块工程师／系统工程师／系统架构师）定位——背的是专责还是终责；有没有”名片一个粒度、实际另一个粒度”的人（如水忠＝模块岗位、干系统级的事）。 3. 起点：最近一次亲手把东西做出来是什么时候？答不上来，先做这一件事。 4. 第一道关口：把最近一次做对的事写成可交接的记录（步骤、依据、坑），交给没参与过的同事照做一次——他卡在哪一步，就是你还没显性化到位的地方。 5. 第二道关口：写下最近一次重要取舍（当时哪些约束在打架、选了哪个、依据是什么、后果如何、重来会怎么选）。6. 第三道关口：写下不可越线清单（安全、合规、隐私、数据真实性各一条），再找出最近一次”为进度让路”的现场——利益相关方知情吗？有记录吗？7. 证据：拿你负责的那套产品数据问一句”换个人拿着它，能不能做出来、继续演进”——答不出的部分，就是你的缺口。8. 拓宽自测：现

在的视角是模块视角还是系统视角？最近一次”为系统牺牲自己的模块”是什么时候？ 9. 清单对表：把你手上这摊活按”需求 → 设计开发 → 产品数据 → 判定”摆一遍——摆不出来的那一段，就是你的缺口；断在哪一段，回头读哪一章（需求→第 3、4、5 章／设计开发→第 3、6 章／产品数据→第 6、7 章／判定→第 8 章）。

## 第 10 章 架构：定义与误解

**自查速查**：① × ② × ③ × ④ ◐（不准确） ⑤ × ⑥ × ⑦ × ⑧ × ⑨ ◐（不准确） ⑩ ◐（一半对） ⑪ × ⑫ × ⑬ × ⑭ × ⑮ ×（④ 微服务是**架构风格**——决策的套路，不是”架构”；⑨ 正本 §3.8 的定义句是裸名词短语、**没有属概念**，它是”一组惯例”；⑩ 规范正文只写”一个视角治理一张或多张视图”（§5.2.7），“每张视图恰有一个视角”见资料性附录 A.5，OMG 技术缺陷单只是旁证）

**练习答案要点**：1. 用**承重测试**筛：去掉它，这个实体还能达成它的目的吗——写下至少三条”去掉就不再是它”的。 2. 逐条跑**两遍**：兑现侧（实现目的需要它吗）＋演化侧（目的变了它守不守得住）。3. 判一条”改起来贵”的决定：它是不是因为”贵”才被叫架构；去掉它，目的还达得成吗（**不可逆性只是代理**，会漏判也会误纳）。 4. 利益相关方 ⩾5 并写各自关注点（含监管／运维／维修／退役处置）；找出从没考虑过的。5. 一条真实场景横穿所有”XX 架构”图；走不通的地方＝把同一个架构当成多个架构的病根。6. 按四类归位：**关注域视角**（业务／数据／应用／技术）／**描述层次**（概念／逻辑／物理）／**架构**

**风格**（微服务等）／**框架**（Spring 等）。 7. 用”架构视角 → 架构视图”分析一张图：视角是什么规则、回答谁的什么关注点、刻意忽略了什么。8. 检查一次架构评审的标的：是一张图，还是”那组去掉就不是它的基本概念或属性”。 9. 框架盘点：它替你们定了什么、没替你们定什么——没替你们定的（服务边界／数据归属／接口契约）就是架构决策。10. “从没写下来但都在守”的规矩：标出哪些”去掉就不是它”，哪些只是习惯。11. 四问的”它从哪来”：建筑图纸 → 1996 最高层级的构想 → IEEE 1471 根本组织 → 42010:2011 基本概念或属性（体现于元素与关系）→ 42010:2022 实体＋删去该层。 12. 逐条对 §3.14 的三个动词：这份架构描述能不能被**单独识别**、被**存储**、被**交付**给别人用。 ### 第 11 章 架构设计：概念幸存、属性涌现、原则自加

**自查速查**：① × ② ◐（不准确） ③ × ④ ◐（不准确） ⑤ × ⑥ × ⑦ × ⑧ × ⑨ × ⑩ × ⑪ × ⑫ × ⑬ × ⑭ × ⑮ × ⑯ × ⑰ × ⑱ ×（⑯ 六类输入里目的与任务居判据侧首位——它是 fundamental 成员资格的裁决者，设计输入不是”全部需求原文”；⑰ 确认单是架构设计的入场券、判据侧首类输入「目的与任务」的确认形态——签字之前，后续一切产物都没有判据锚点；⑱ 目的、目标、需求是三层不是一回事——目的是系统存在的理由（环境侧/为利益相关方）、目标是可度量状态（ISO 9000 objective）、需求是可验证条款，逐层具体

化；以目的检验三分——确认与评审以目的为依据、验证以规定需求为参照）

**练习答案要点**：1. 两样内容：本质——基本概念（组织范式/核心抽象/根本机制/根隐喻）、基本属性（生命线属性）——以及**管控原则**，各是什么；再问一句：这两样表达成了哪份**架构描述**、元素与关系在下游设计里交代到哪一步——答不出任何一样，就是那一处的设计缺口。2. 两种翻译各列一好处一代价；架构设计必然做选择——同一份需求可翻译成不同**架构**，翻译的每个选择都是架构决策的种子。3. 判断动的是哪一块：动基本概念 = 系统本质变了（“还是不是它”）；动属性/管控原则 = 在既有本质下调整；动元素与关系 = 下游设计的调整（其规格归设计定义过程）。4. stakeholder × concern 矩阵：空格子（有利益相关方没关注点，或有关注点没利益相关方）与冲突格子（权衡点）各是什么。5. 四判据查老规矩：可判定吗？有理由吗？少而精吗？可入评审吗？——过不了关的，它不是**管控原则**，是教条。6. 把管控原则逐条时间维化：这条规矩在变更时怎么执行？——产出一条约束演化的原则（“变”字句式）。7. 依赖图检查：有环吗？多少关系不必要？把一条强关系降级成弱关系，变更牵动范围小多少？8. 属性四步流水线：QAS 场景化 → 战术 → 结构落位 → 验证度量——属性必须落到”度量”上，否则承诺等于没有。9. 反向校验：随机挑一条管控原则向上追——它从哪个概念/属性提炼来的？答不出 = 无源之水（输入链断过）。10. 约束/决策/原则三实例：哪条外部强加（接受）、哪条

自己选（决策）、哪条自愿自我限制（原则）。11. 四问量”基本概念”（或”基本属性”）→ 基本概念是什么（系统”怎么想”的根源：组织范式/核心抽象/根本机制/根隐喻）、不是什么（不是发明，90% 从概念库选、10% 被需求/属性逼出来）、从哪来（42010 的 fundamental concepts，生成路径五条：模式选择/领域抽象/质量反推/类比迁移/第一原理）、往哪去（定形态/实体/机理/眼光，命名是概念化的完成仪式）；最薄的一问通常是”从哪来”——答不出生成路径，就把概念当发明。12. 三动词链拆分：构思阶段生成过几个候选（≥2?）——唯一解 = 未审视的解；定义阶段被否备选的”死因”记录了吗（ADR）；设计（狭义）阶段是等基线生效才动的吗——没等，就是把未定形的构思当下游输入。13. 六步体检：哪一步被跳过？最常被跳②（直接从需求跳到唯一解画图）和⑥（评审变汇报）；⑤是”视角规格＋表达与基线”两件（视角规格六项→第 15 章 §15.4.1、视图声明治理视角与对应关系→§15.4.3）；三纪律对照——驱动先于图 / 决策先于图 / 基线先于图。14. 创新判断：够新吗 × 成了吗？主导动作——元件不动、连接重排 = 架构式创新，主导动作是定义（不是构思）。15. 写《系统目的或任务理解确认单》：业务方原话（只引不改）+ 你的复述理解 + 确认记录——复述里冒出架构语言，就是越界（目的定义权在业务方）；再自问一句：这份确认单是你架构设计的输入，还是设计做完后的补记？16. 目的体检：按三条来路（战略承接 / 业务或任务分

析 / 关注点反推）逆推系统目的，再用到位判据三检查——可作判据？可分解？可追溯？答不出的，目的还没到位，别急着写确认单（确认单是收口，签字前目的只是草稿、不能作判据）。

## 第 12 章 架构决策：对不可逆的提前投资

**自查速查**：① × ② × ③ × ④ ◐（不准确） ⑤ × ⑥ × ⑦ × ⑧ ◐（半对） ⑨ × ⑩ × ⑪ × ⑫ × ⑬ × ⑭ ×（⑬ 难改 ≠ 重要——什么算 fundamental 由目的判据裁决，代价只在这组根本之内排优先级与时机，难改但无关紧要的不配当架构；先过目的关、再过代价关；⑧ 需求本身没有架构，但**需求的方案＝需求侧的一份架构描述**（按关注面切、不按模块切）；⑭ 复审第一条该写”目的变更”——系统目的变了（含目的漂移的悄然变化）都是架构重审的强制触发器）

**练习答案要点**：1. 三标志检验一次改动：影响半径大/持续时间长/修改成本高——两问以上”是”就该按架构决策对待（三标志是**代理**：管待遇与时机，不管”是不是架构”；真准入判据是目的必要性）。2. 数据模型级决定：找出主键/状态枚举/字段口径这类，列出重做的联动清单。3. 三层判断框架：影响半径（结构/时间/组织耦合）、信息成本（知识深度/隐含上下文）、修改风险（爆炸半径/数据完整性）各打高/中/低。4. 找”不知为何这么定、不敢改”的决定 → 大概率信息成本极高 → 补写”背景”。5. 决策记录写一份（六问＋**来源与弃项**＋**批核与例外登记**）；找不到”背景/选项” = 这个决策从没被认真记录过，这本身就是发现。6. 五个决策按不可逆性排序；对照你们投入

的审查资源是否跟着排序走。7. 显式推迟三句：现在为什么不做？等什么信息？什么时候再评估？ 8. 高不可逆决策对照审查分级表（极高→完整审查多人交叉；低→快速自检）。 9. 画设计链标各层”不可逆性最高”的决策，看是否集中在两端（需求组织/产品构件）。10.“需求的方案有架构”：你们的需求怎么组织（按**关注面**还是按部门/模块切）——是被决策过的，还是顺手定的？按模块切会让视图退化成模块说明书。11. 复盘一次被动做架构决策：回到哪个时刻、用哪把尺子拦住它。 12. 挑一个高不可逆决策走两关：先过目的关（它守住的是对目的必要的不变量吗），再过代价关（按三层框架排优先级与时机）——两关次序错了吗（先代价后目的，就是拿”贵”冒充”根本”）。

## 第 13 章 配置管理：让一直在变的东西说得清

**自查速查**：① × ② × ③ × ④ × ⑤ × ⑥ × ⑦ × ⑧ × ⑨ × ⑩ × ⑪ × ⑫ × ⑬ ×

**练习答案要点**：1. 配置三问检验客户可改参数：被记录了吗（记录边界）/功能物理特性吗（类别边界）/关联成整体了吗（关联边界）——参数是零件，配置是整机。2. 五个配置项（代码/固件/文档/数据/硬件）+ 该不该纳入管理的判断（组织选定纳入的实体才是配置项）。3. 最大配置项三查：有版本号吗 / 版本号对应哪次完整构建吗 / 有基线吗（批准+冻结+参照）。4. 回滚/追溯复盘：查不到的地方，缺的是版本、基线、还是状态记录。5. 版本链画基线点：在里程碑（需求批准→功能基线、分配定案→分配基线、产品定型→产品基线）竖切。 6.

变更控制循环五步（CR→影响评估→CCB→实施→新基线）：哪步跳过了、代价是什么。7. 四大活动诊断：标识（是什么）/控制（怎么变）/状态记录（现在什么样）/审计（实物与记录一致吗）各打有/缺/半吊子。8. 决策记录六问对照同构表：对应的配置信息、基线（冻结）、变更控制各在哪里。 9. 契约链三层：每层”基线（冻结端）“与”版本（交付端）“分别是什么。10. 编号事故用配置管理复盘：CR + 影响评估会写”影响全部历史合同数据，不可逆性高”——这一行字就拦住”今天就能上”。 11. 精确召回 vs 全量召回案例：算一下批次追溯能省下”精确召回受影响批次 vs 召回全部在役五千台”的差量。12. 四问量”配置”（或”基线”）→ 基线是什么（被批准、被冻结、作全生命周期参照的配置信息）、不是什么（不是版本：版本是刻度、基线是资格）、从哪来（“凝结”型概念：从测绘”基准线”转义，工程实践中被共同体固化，见 § 1.2.1）、往哪去（变更控制循环的支点：冻结→变更→再冻结）；最薄的一问往往是从哪来——答不出”凝结”故事，就会把”版本号=基线”当真。

## 第 14 章 治理：谁掌舵，谁划桨

**自查速查**：① × ② × ③ × ④ × ⑤ × ⑥ × ⑦ × ⑧ × ⑨ × ⑩ × ⑪ × ⑫ ×（⑪ 治理的对象不是”文档”，是「目的→不变量→根本决策→遵从与演化」链——架构资产只是四层之一，目的漂移了、决策偏离了目的，文档再整齐也白搭；⑫ 目的变更是架构重审的强制触发器

——fundamental 判据本身换了、不变量集合整体过期，目的漂移只能靠以目的为参照的定期检视抓）

**练习答案要点**：1. 治理四问检验挂”治理”名头的事：谁来治（角色）/依何治（规则）/如何治（机制）/治得怎样（担责）——四问答得上才谈治理；再用标准定义句对成分——以人为基础（人说了算还是文件说了算）/三职能（指导·监督·担责）/为宗旨/“被”字（须交账的地位是制度给的还是靠自觉）。2. 五个大决策分类：掌舵（定方向/定规则/裁例外）vs 划桨（执行/优化/交付）；再按三条判据（层级/对象/不可自裁）逐段定性，用介入门槛（跨出单管理层权限）说明该不该升级——判据管分类、门槛管升级，别把”跨部门”当判据。3. 三条件核对：分离/静默积累/框架级——符合几个？还差哪个？差的往往就是”管理足够”的原因。4. 委员会诊断：治理机构是否持续行使四活动（利益相关方参与/评估/指导/监督）、评估发现偏差后是否调整指导？只开会看报告、从不处置 = 橡皮图章。5. “坏了”（内部失效，红旗评估抓）vs”不适合了”（外部过时，定期检视抓）各一例；后者往往要等爆雷。6. 权力链画出来：所有权→治理→管理→执行，每层是谁；哪层空了（有管理没治理最典型）。7. 过滤器改造：定哪几条规则让 90% 的争议自动处理、不再上会。8. 三活动循环诊断：感知/定规/裁例外，哪环最弱；缺感知则盲、缺定规则乱、缺裁例外则僵。9. 术语膨胀五问：有分离吗/治理在持续运转吗/委员会在定规则裁例外吗/有传感装置吗/治理层碰桨吗——任一”没有”就要警惕

（五问是体检表，判”转不转得动”，不判”算不算治理”）。10. 伞结构写出：XX 管理 = XX 治理 + XX 开发与运营，三个加数对应谁；伞名有没有和子流重名（自包含）；分域模子三条不变（原则不另立/治理机构不另设/最终担责不转移）。11. 编号事故治理视角：有”编号标准由法务归口”这条规则时，改编号必须走跨部门裁决——规则从根上拦下”今天就能上”。12. 挑一个系统架构决策，沿”目的→不变量→根本决策→遵从与演化”四层各问一遍：这层谁在守护、被什么侵蚀；再按 §14.4.4 判例（目的漂移事件链）的拆法走一遍：目的漂移后，哪个动作算治理（侦测/裁决/重审判据）、哪个算管理（调整架构/实施迁移）。

## 第 15 章 架构定义落地：从目的到架构视图

**自查速查**：① × ② × ③ × ④ × ⑤ × ⑥ × ⑦ × ⑧ × ⑨ × ⑩ × ⑪ × ⑫ × ⑬ × ⑭ × ⑮ × ⑯ × ⑰ × ⑱ × ⑲ × ⑳ ×（① 图只是视图组件的一种形态；② 目的最见功力的是”舍掉了什么”；④ 关切属人、需求属系统，三条判据分得开；⑥ 写不出度量是”退回补件”不是驳回；⑦ 关切是发现物、目标是裁量物，判据只有目的；⑧ 不可逆性只是代理，会漏判也会误纳；⑨ 没有视角标准就自建 ADF；⑪ 视角是一**组惯例**（§3.8 正本无属概念，不替它补），立场是利益相关方角度；⑫ 一张视图恰一个治理视角；⑭ 视图之间对不上＝对应关系断了＝架构不成立；⑮ 评架构对目的、评描述对细化适宜性，串了尺子

评估就退化成文档审查；⑯ 依据是目的、判定靠目标；⑰ 缺口回定义期补；⑱ 准则不可判定违反就拦不住任何东西；⑳ 标准闭不上的缝只能由责任来闭）

**练习答案要点**：1. 链条体检：顺九步走一遍，断点通常落在”目的没写舍什么”“没有关切台账”“视角没有规格”“没有对账”四处——断在哪一步，缺的就是那一步的产出物。2. 目的声明书”舍掉了什么”：写不出三条”这次不做”，说明目的还停在口号层；每条”舍”都要能说出理由与被舍弃的关切。3. 关切台账：利益相关方列全的关键在**沉默方**（监管、运维、退役处置、审计、法务、二次开发方）；“后果”写不出度量的条目一律标”退回补件”。 4. 视角规格试写：六项里最容易缺的是**对应方法**与**视图方法**；写完能自查”换个人能不能按它复现同一张图”。 5. 给图贴标签：贴不上的那张通常是没有关切、没有模型种类约束的”目录页”——它可以是索引，但不是架构视图。6. 双向覆盖检查：正向查漏（关切无视图承接）与反向查滥（视图上追不到关切的东西——技术选型、惯例、交付关切升格）两侧各至少一条。7. 不可判定的准则：判据是”写出一个能判出违反的具体场景”——写不出场景的准则，改写到能判违反为止。8. 两把尺子：看结论措辞——出现”图没画好/格式不全”即用了描述尺子；重写为”对照哪条目的、哪条目标，结论是什么”。

## 第 16 章 架构定义到设计定义：架构与设计的交接

**自查速查**：① × ② × ③ × ④ × ⑤ × ⑥ × ⑦ × ⑧ × ⑨ × ⑩ × ⑪ × ⑫ × ⑬ × ⑭ × ⑮ × ⑯ × ⑰ × ⑱ ×（① 两类工作不是粗细两版，design 宽义句推不出粗细论；③ 有效的架构尽可能不依赖于设计（**15288:2023 §5.9 标准正文**）；④ 交接第一步是”检查”不是”接收”；⑥ 分配须同时满足系统需求**和架构目标**；⑦ 新增元素是正常的，要管的是认领；⑨〈待定〉不是留白是欠账；⑩ 设计定义只可**评审**，确认要落在实现出的客体上；⑪ 一致永远落在跨视图对应关系上；⑬ 换手点是**信息充分性**判据；⑮ 7±2 是可管理性规则不是停止规则；⑯ 切视角的维度是”看什么”不是”谁在看”；⑰ 受理口是义务、视角是手段；⑱ 自建视角有成本，先穷尽复用）

**练习答案要点**：1. 交接体检：“收到文档就开工”的标志是——设计描述里说不出”本次从架构取到了哪几条约束”；正确形态是列得出”基本属性与概念／指导设计及其演化的原则”两栏。2. 两条转化：转不动的两种病因要对症——架构侧没立住（特性本身不可判定）／设计侧没转（有元素无特性落点）；先判在哪一侧，再决定回架构还是补设计。3. 分配对两个量：找不到承担者的多半是**架构目标**（系统需求通常有人认领）；原因写在”这条目标该由哪一层承担”，往往发现它根本没往下分。4. 无主的接口：判据是”架构有没有识别过它”；选”回架构补识别”还是”进需求基线”，看它是**架构漏了**（既有职责没交代）还是**需求变了**（新出现的功能）。5. 描述符对账：对不上的两格＝

两种病——元素自设指标（无据）与系统级指标无人接（缺项）；〈待定〉逐格写责任人。 6. 九查：加戏查在”实体转化”（多出架构未授权的元素或依赖），滥项查在”查滥”（技术选型当概念／交付关切升格／把自由度当架构资产）；三档结论中”条件性合规”必须带补项、责任方、期限。 7. 可实现性归位：四步依次试——并入既有视角、扩既有视角规格、新建视角、改用别的标准手段；判据只看”是否需要独立模型种类且能独立判定”。 8. 换手点验证：两个条件各自问一句——“它的需求能不能只用黑盒描述”（能不能）＋“我能不能自信而明确地说出它在现实世界中的解决方案”（敢不敢）；两条都过，买／造／重用才是决策而不是赌博。

## 第 17 章 整合：从一条需求到上线运维

**自查速查**：① × ② × ③ × ④ × ⑤ × ⑥ × ⑦ × ⑧ × ⑨ × ⑩ × ⑪ × ⑫ × ⑬ ×

**练习答案要点**：1. 画完整闭环时间线：七站（需求/设计/把关/运维 + 架构[含架构设计与架构决策]/配置/治理贯穿）+ 每站之间标接缝；圈出最弱一环。2. 顺次/横切分类：第 3~8 章顺次（需求 3/4/5·设计 6/7·把关 8；运维对应第 2 章工程系统回归）、第 10~14 章横切（架构 10·架构设计 11·架构决策 12·配置 13·治理 14）；组织里横切线各自谁在贯穿，有没有没人贯穿的线。3.“切片都做了但系统失败”复盘：接缝断在哪两个切片之间、断在四条接缝（承接/参照/记录/问责）的哪一条。 4. 配置管理三问复盘模块：应该是什么（需求）/实际是什

么（验证）/怎么变成这样（变更记录）——三答对得上吗。5. 先用**目的关**裁它够不够格进架构（它守的是目的之下的不变量吗），再用三层判断框架量代价与时机（影响半径/信息成本/修改风险——它只是**代理**，管排序不管准入）：被提前投资了，还是等爆雷才做。6. 起点判据检查真实需求：完整/无歧义/不冲突/隐含需求已识别，哪条最弱；有没有”编号进台账”式藏在场景里的隐含需求。 7. 模块拆树 + 语义四查 + 分配三自检：找两片叶子之间缺的”接缝叶”。8. 评审、验证、确认复盘上线：各自站住了吗（目标、规定需求、预期用途各对上了吗）；“确认”是不是真在场景里做的。9. 治理四问（谁来治〔角色〕/依何治〔规则〕/如何治〔机制〕/治得怎样〔问责〕，第 14 章原词）诊断跨部门变更：落到这次变更就是——谁裁决/依什么规则/机制是什么/谁担责——答不上来的那一问，就是接缝。10. 概念跨域实例：把基线/确认/治理/不可逆性搬到一个陌生行业，还能成立吗——成立在哪个位置。 11. 接缝人角色扮演：至少 5 处接缝 + 每条靠四条接缝里的哪条缝合 + 责任人是谁——写得出来才缝得上。

## 第 18 章 架构师：把整条链背在身上的人

**自查速查**：① × ② × ③ × ④ × ⑤ × ⑥ × ⑦ × ⑧ × ⑨ × ⑩ × ⑪ × ⑫ × ⑬ × ⑭ × ⑮ × ⑯ × ⑰ × ⑱ × ⑲ × ⑳ ×（⑭ 架构师再往上≠升任 CTO；⑮ 专责≠终责；⑯ 拍板权在系统工程师、理由权在架构师；⑰ 架构符合性把关是职责的一部分；⑱ 42010 只给架构活

动立词条、不给头衔立词条——标准认活不认职位；⑲ 十项清单是体检切法（六项底座+四项增量）、七动词链是工序切法，两套并行不互吞；⑳ 目的定义权和问责在业务方——架构师对”架构忠于目的”负责（理解/澄清/质疑/守护），越界改写目的是 Cowboy Coding 式架构的病因）

**练习答案要点**：1. 十项能力体检：逐项打勾，指出缺项，判断对方在补底座（六项）还是补增量（四项）。 2. 专责终责盘点：系统成败的账（终责）在谁手里、架构正确性的账（专责）在谁手里——同一个人是兼任还是失职（把专责当终责 = 什么都管）。 3. 角色边界归属：这个决定本该归谁——按”管决定/管结构/懂系统”划分，说出它为什么被相邻角色抢走了。4. 失败模式对号入座：画图匠（没判断）/ 什么都管（管太多·把专责当终责）/ 不决策（不管）/ 远离代码（看不见），对应十项清单里缺的那一项。 5. 产品数据体检：翻出架构描述+接口契约，逐项问”换个人拿着能不能继续做下去”——答不上来的那部分就是⑥的缺口。6. 工作检查单落地：下一次不可逆决策前过七条（是不是严重决策 / 理论是什么 / 利益相关方视图跟上没 / 记录写了没 / 我是在做决定还是在画图 / 接口契约改了没·下游叶子跟着变没 / 我在背专责还是终责）。 7. 晋升阶梯盘点 → 四条路径（深度/宽度/组织/权威）各对应一条核心关切；标出你们”架构线”与”管理线”的分叉点在哪——对照你现在的棋盘在哪一格（工具不变，棋盘变大）。8. 目的权责盘点：你们系统”目的/任务”的定义权在谁手里？

架构师是在理解/澄清/质疑/守护目的，还是悄悄改写它？写一份《系统目的或任务理解确认单》试试：业务方原话 + 你的复述——复述里有没有冒出架构语言（冒出，就是越界了）。

---

## 附录收束

五个抽屉合上，回到全书那句收束——第 17 章说过，它仍是这本书想让你带走的一句话：

> **概念是刀，系统是大象。切得再细，也要缝得回来。**

附录 A 是刀的目录，B 是刀的规格，C 是”哪把刀最容易切错”的警示，D 是刀的出处，E 是”刀法练得对不对”的对照答案。它们不新增任何概念——它们只是把正文发过的尺子，收进了一个随时能回来取用的地方。训练教材的题卡与讲师版，与附录 E 同源；遇到任何一题答不上来，先回对应章节，再回这五个抽屉。

把那些你天天挂在嘴边、却从没真正想清楚的词，放到你有把握判定的位置。